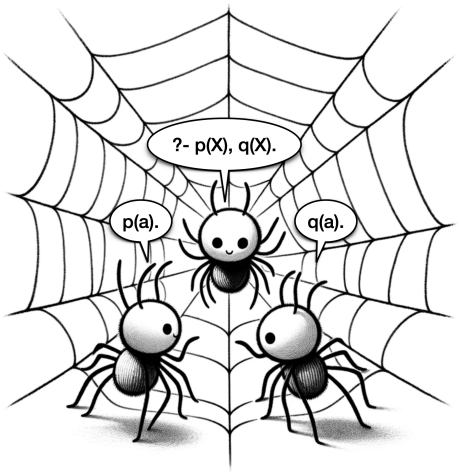
Torbjörn Lager

The Prolog Trinity ecosystem

Let's webize Prolog, prologize the Web, and populate the Web with clever Prolog agents!

(Draft printed 2026-10-17 at 09:01:43)

Preface



This book explores how Prolog might become a language for programming the Web: for sharing and querying knowledge, for running distributed computations, and for building agents that communicate and act over time. It proposes *Web Prolog*, a profile of Prolog extended with actor-based concurrency and distribution; describes the *Prolog agents* that use it; and develops the *Prolog Web*, the networked environment in which they interact. Together, these form what we call the *Prolog Trinity ecosystem*– a niche within the vast ecosystem of the World Wide Web.

The proposal brings together ideas from Prolog, Erlang, the Web, and statecharts. Prolog supplies relations, unification, and nondeterministic computation. Erlang supplies much of the inspiration for actors and message passing. The Web supplies an open environment in which independently developed systems must communicate across boundaries of implementation, ownership, and trust. Statecharts supply a way to organise behaviour that unfolds through events and interaction. Much of the book is concerned with what happens when these ideas meet: where they fit naturally, where they constrain one another, and where familiar mechanisms need to be reconsidered.

The book has grown through a close exchange between writing and implementation. Explaining a design exposes questions that a prototype must answer; implementing it exposes assumptions that the text must make explicit. What began as a vision has consequently become a fairly detailed account of a language, its interaction protocols, and the responsibilities of the systems that implement them. The accompanying research demonstrator, built on SWI-Prolog, supports both native and browser-resident actors and gives many of the proposals an executable form. The appendices provide language and protocol references, discuss implementation, and report measurements of several actor substrates.

This is still work in progress. A running demonstrator gives us something to inspect, experiment with, and criticise, but considerable work is needed before it could become a production platform. The GNU Prolog and Trealla prototypes provide initial interoperability evidence across runtimes, while complete profile conformance and independent validation remain to be established. The longer-term possibilities include both a useful platform centred on SWI-Prolog and a federated Prolog Web involving several systems. The book examines what each would require, without assuming that either outcome is inevitable.

The book is written first for Prolog programmers and implementers. Some knowledge of Prolog is assumed,[1] but expertise is not: an unfamiliar view of a familiar language may sometimes be easier to appreciate with a beginner's mind. Familiarity with web technology is not assumed. Where the argument depends on HTTP, WebSocket, JavaScript, or browser machinery, the necessary background is introduced along the way.

Erlang developers and others who work with actors may also find the book interesting. They will recognise many of its mechanisms, though Prolog's backtracking and relational style, together with the conditions of the open Web, lead to some different choices. Their experience should be valuable in assessing both the programming model and its implementation.

For logic programming researchers, the contribution lies chiefly in the architecture around Prolog: how computations are exposed, how answers are obtained across a network, and how logical reasoning can coexist with communication, state, and control. The underlying logic programming remains familiar Prolog. One ambition is to make logic programs more readily available to other programs and communities; another is to understand how far their logical reading can be preserved when computation is distributed.

Semantic Web researchers and practitioners will find an attempt to connect executable logic programming with shared vocabularies and web knowledge. Readers interested in AI will find an account of how symbolic reasoning, actor communication, and statechart control can contribute to agents that use learned services, including large language models. A running service-request example explores how an LLM's proposals can be checked against explicit rules, when a user or human decision-maker must be involved, and where such rules cease to be sufficient.

These concerns give the book its particular character. It combines programming examples and technical reference material with architectural argument and a discussion of what would be needed for wider adoption. We hope readers will use it both to try the ideas and to question them. Whatever becomes of the larger ecosystem, the language, protocols, and demonstrator should provide a concrete basis for that exchange.

It may all be a pipe dream, but it is not a castle in the air![2]

---

[1] For an introduction, see Sterling and Shapiro's *The Art of Prolog*; for a more advanced treatment, see O'Keefe's *The Craft of Prolog* (Sterling and Shapiro, 1994; O'Keefe, 1990).

[2] https://blogs.bmj.com/bmj/2017/01/27/neville-goodmans-metaphor-watch-dream-a-dream-of-ivory-castles-in-the-air/

Use of artificial intelligence and computational tools

AI systems were used extensively during the later stages of preparing this manuscript and developing its accompanying software prototypes.

The origins of Web Prolog, however, substantially predate the recent development and widespread adoption of such systems. The language, its underlying architectural ideas, and an earlier proof-of-concept implementation had already been developed and were described in the author's paper *Intro to Web Prolog for Erlangers*, published in 2019. Substantial parts of the present manuscript likewise originated in work undertaken before large language models became practical tools for research and software engineering; extracts documenting this work have been made publicly available at `https://torbjornlager.github.io/`. Later, AI systems helped refine, implement, investigate and present these ideas.

In preparing the present manuscript, these systems were used as interactive tools for technical discussion and criticism, linguistic revision, investigation of technical questions, and comparison of design alternatives. They also assisted in identifying potentially relevant specifications, documentation, and literature. The author independently consulted and evaluated those sources before relying on them in the text.

In later implementation work on the Web Prolog prototype and its associated demonstrator, AI systems assisted with programming, debugging, architectural analysis, user-interface design, and system integration. This assistance concerned, among other subjects, Prolog, JavaScript, HTML, CSS, SVG, and browser technologies. Some of the later architectural and design decisions discussed in this book emerged from iterative exchanges between the author and these systems. All proposed designs, explanations, and implementation materials were reviewed by the author and were adopted, revised, or rejected at the author's discretion. The author directed the development process and was responsible for integrating, testing, and validating the resulting software.

AI systems were treated as computational tools, not as authors or authoritative sources. Their outputs were subject to critical review, and no responsibility for the work is attributed to them. The author assumes full responsibility for the manuscript's arguments and statements, the technical design and implementation of the software, and the reported experimental results.

The book therefore spans two periods: one before the author used these tools and one in which they were used as described above.

Acknowledgements

This book has grown out of a long personal and intellectual trajectory, and I owe thanks to many people and institutions that have shaped it, directly or indirectly. I am especially grateful to the University of Gothenburg, and to the departments and programmes through which I first encountered philosophy, linguistics, computer science, computational linguistics, dialogue systems, language technology, and the

Web. The former Department of Linguistics, the Department of Philosophy, and later the Department of Philosophy, Linguistics and Theory of Science have provided the academic environment in which most of the ideas in this book gradually took shape. I am also grateful to Chalmers University of Technology, the Centre for Language Technology, the Faculty of Arts, the IT Faculty, and the Swedish National Graduate School of Language Technology for forming parts of the institutional landscape in which much of this work became possible.

Several teachers and colleagues were important in my early intellectual formation. Björn Haglund and Staffan Carlshamre gave lectures in philosophy that left a lasting impression on me, and helped turn my interest toward cognition, logic, language, and the philosophy of science. Bosse Dahlbom, by helping me gain access to a DEC VAX computer at Chalmers, gave me my first real taste of networked computing, `comp.lang.prolog`, FTP archives, and the strange feeling of being intellectually present in machines located far away. That experience may have mattered more than I realised at the time.

I am also indebted to the computational linguistics and dialogue-systems community in Gothenburg and elsewhere. Joakim Nivre was both a collaborator and an important intellectual companion; without his talent for formalisation and for structuring arguments, some of the work I am most fond of would have been much weaker. Björn Beskow joined Joakim and me in the small adventure of Linguistic Instruments. Robin Cooper and Staffan Larsson helped shape the dialogue-systems environment in Gothenburg. The PLUS, TREE, TRINDI, and related dialogue and language-technology projects gave me problems to think about, systems to build, and reasons to keep returning to Prolog.

A central debt is owed to Jan Wielemaker. My work on Pengines, SWISH, and eventually Web Prolog would not have developed as it did without his help, generosity, technical insight, and long-standing work on SWI-Prolog. He kindly helped with the implementation of an earlier demonstrator, and he also came up with the original idea on which the stateless HTTP API described in this book is based. SWI-Prolog showed me that Prolog could be a practical platform for the Web, not merely a beautiful language for small examples and elegant programs. I am grateful not only to Jan, but also to the wider SWI-Prolog community and to the Prolog and logic programming communities more generally. This book is, among other things, an attempt to give something back to them.

The design ideas in this book also owe much to Erlang, Oz, and SCXML. I am grateful to the late Joe Armstrong for his enthusiastic support, for his writings and talks, and for his personal comments on early versions of these ideas. He encouraged me to think more seriously about actors, message passing, fault tolerance, and the possibility of making Prolog look outward toward a distributed world. Richard O'Keefe and Markus Triska provided encouraging and constructive comments and suggestions, and Richard O'Keefe's remarks on Erlang-style concurrency in Prolog were an important source of confidence. Peter van Roy and Seif Haridi, through their work on Oz and *Concepts, Techniques, and Models of Computer Programming*, helped me understand concurrency, dataflow, message passing, and multi-paradigm programming in a deeper way. Finally, my involvement in the W3C SCXML Working

Group gave me valuable experience of standardisation work, and helped convince me that languages and protocols intended for the Web need not only good ideas, but also specifications, implementations, tests, communities, and patience.

I also thank the anonymous reviewers who, at different stages, offered useful suggestions and criticism. Their comments helped sharpen the presentation and reminded me that even a book driven by vision must eventually submit to the discipline of explanation.

Skärhamn, October 2026
Torbjörn Lager

Contents

Part I Foundations

1 Introduction 3
- 1.1 The Prolog Trinity ecosystem 4
- 1.2 Walking in the footsteps of inventors 5
- 1.3 The Prolog Trinity ecosystem concretised 6
  - 1.3.1 Prolog nodes and Prolog actors 7
  - 1.3.2 Relations, actors, and interaction across nodes 8
- 1.4 Motivations and goals 11
  - 1.4.1 Challenges in the Prolog ecosystem 11
  - 1.4.2 New opportunities on the Web and in artificial intelligence . 12
  - 1.4.3 The five principal goals of the Prolog Trinity 13
  - 1.4.4 Webizing Prolog and prologizing the Web 14
- 1.5 Thirty years of Prolog on the Web 15
- 1.6 The Trinity demonstrator 17
- 1.7 The structure of the book 19

2 Web Prolog 23
- 2.1 The essence of Prolog 23
- 2.2 Web Prolog in a nutshell 25
  - 2.2.1 Web Prolog is inspired by Erlang 28
  - 2.2.2 Web Prolog is a multi-paradigm programming language 31
  - 2.2.3 Web Prolog as a scripting language for the Web 33
- 2.3 Erlang-style message-passing concurrency in Web Prolog 37
  - 2.3.1 Programmer talking to actor, actor talking to itself 37
  - 2.3.2 Two utility tools for programming in the shell 39
  - 2.3.3 Actors talking to other actors 40
  - 2.3.4 A closer look at receive/1-2 49
- 2.4 Erlang-style programming examples in Web Prolog 58
  - 2.4.1 A count server 58
  - 2.4.2 A bigger, tastier example 59

2.4.3 Hiding the details of protocols 61
2.4.4 Using delayed sending 62
2.4.5 Prolog actors playing ping-pong 62
2.4.6 Event-driven state machines 64
2.4.7 Getting answers through backtracking 65
2.4.8 The toplevel behaviour: a preliminary sketch 66
2.5 Summary 68

3 Prolog agents 71
3.1 Prolog agents in a nutshell 71
3.2 More about Prolog actor agents 73
3.2.1 An actor agent is equipped with a private Prolog database . . 73
3.2.2 The lifecycle of a typical Prolog actor agent 74
3.2.3 The dynamic (and still private) Prolog database 76
3.2.4 Private beliefs and shared knowledge – shadowing 77
3.3 Prolog shells and other toplevel actors 80
3.3.1 A Prolog toplevel is an actor with a built-in protocol 80
3.3.2 The Prolog Toplevel Communication Protocol 81
3.3.3 Shell talking to a Prolog toplevel 83
3.3.4 Programming with toplevel actors 90
3.3.5 Coordinating concurrent goals 93
3.3.6 Server and supervisor actors 95
3.4 Prolog nodes are also agents 96
3.4.1 The node controller 97
3.4.2 The shared Prolog database 97
3.4.3 Deploying a node 98
3.4.4 Settings 99
3.4.5 Two transports – three web APIs 101
3.5 Node profiles 103
3.5.1 Browser talking to a RELATION node 104
3.5.2 Browser talking to an ISOBASE node 107
3.5.3 Browser talking to an ISOTOPE node 108
3.5.4 Browser talking to an ACTOR node 111
3.5.5 A terminal talking to a browser node 113
3.5.6 Social robot talking to an ACTOR node 114
3.5.7 Could there be other profiles? 115
3.6 Summary 116

4 The Prolog Web 117
4.1 The concurrent Prolog Web 118
4.1.1 Actor talking to remote actor 118
4.1.2 Actors playing ping-pong while on different nodes 119
4.1.3 Terminal authority across node boundaries 119
4.1.4 Acknowledged I/O and output ordering 122
4.1.5 The node option works for toplevels too! 123

4.1.6 Uniform messaging across node boundaries . . . 123
4.1.7 Remote links and monitors . . . 124
4.1.8 Containment: lifetime discipline . . . 125
4.1.9 Node-resident actor processes . . . 127
4.2 The sequential Prolog Web . . . 134
4.2.1 Nondeterministic remote procedure calls . . . 134
4.2.2 Two operational realisations . . . 136
4.2.3 Browser talking to a node talking to a node . . . 136
4.2.4 Backtracking as paging . . . 138
4.2.5 Programming with `rpc/2-3` . . . 141
4.2.6 Promise and yield over stateless HTTP . . . 144
4.3 The programmable Prolog Web . . . 147
4.3.1 Injecting code with the `src_*` options . . . 148
4.3.2 Code shipping and data locality . . . 149
4.3.3 A small design space . . . 149
4.4 Timing and failure boundaries on the Prolog Web . . . 150
4.4.1 Four blocking surfaces . . . 150
4.4.2 Late replies . . . 151
4.4.3 Design principle . . . 152
4.5 Security on the Prolog Web . . . 152
4.5.1 Inexpressibility: the sandbox . . . 152
4.5.2 Invisibility: name discipline . . . 153
4.5.3 Containment . . . 155
4.5.4 Deployment concerns . . . 156
4.6 Summary . . . 156

Part II Extensions and contexts

5 Implementing Web Prolog generics . . . 161
5.1 The server behaviour . . . 162
5.1.1 The client interface . . . 164
5.1.2 A failing callback . . . 166
5.1.3 Hot code replacement . . . 166
5.1.4 Graceful shutdown . . . 168
5.1.5 From a bespoke actor to a reusable behaviour . . . 168
5.2 The supervisor behaviour . . . 169
5.2.1 A restart we can observe . . . 170
5.2.2 The generic mechanism . . . 171
5.2.3 From restart to escalation . . . 173
5.2.4 Strategies, dynamic children, and trees . . . 173
5.3 The toplevel behaviour . . . 174
5.3.1 The three inner states . . . 175
5.3.2 Why `fail/0` resumes the search . . . 179
5.3.3 Production lifecycle management . . . 180
5.3.4 The enclosing state: abort and halt . . . 181

5.4 The `rpc/2-3` meta-predicate . . . 183
5.4.1 From protocol messages to Prolog nondeterminism . . . 183
5.4.2 An actor-based reconstruction . . . 184
5.4.3 The stateless HTTP implementation . . . 185
5.4.4 Comparing the two realisations . . . 188
5.5 The parallel/1 meta-predicate . . . 189
5.5.1 From two goals to a list . . . 189
5.5.2 Detecting failure and errors . . . 190
5.5.3 Race-free cleanup . . . 191
5.5.4 Behaviour and performance . . . 193
5.5.5 Relation to `concurrent/3` . . . 194
5.6 The first_solution/2-3 meta-predicate . . . 194
5.6.1 Racing alternative solvers . . . 195
5.6.2 Failure and error policies . . . 196
5.6.3 An example . . . 197
5.7 Summary . . . 197

6 The statechart behaviour . . . 199
6.1 STATECHARTS . . . 199
6.2 Statechart actors . . . 200
6.2.1 A first statechart . . . 200
6.2.2 Running a statechart actor . . . 202
6.3 Hierarchy and history . . . 203
6.4 Parallelism and broadcast communication . . . 204
6.4.1 An NPC controller . . . 204
6.4.2 An unbounded internal cascade . . . 206
6.4.3 What "broadcast" means here . . . 207
6.4.4 Why not model each region as a separate actor? . . . 207
6.5 The Web Prolog data model . . . 207
6.5.1 Guarded transitions can derive information . . . 208
6.5.2 Eventless transitions as forward chaining . . . 209
6.5.3 Timed transitions . . . 210
6.5.4 Deferred events . . . 210
6.6 Process invocation and communication . . . 211
6.6.1 What does a confirmation confirm? . . . 213
6.7 The execution model in summary . . . 213
6.8 A flat subset of SXML . . . 214
6.9 From XML source to execution . . . 215
6.10 Discussion . . . 215
6.10.1 Previous work . . . 215
6.10.2 Comparison with an ordinary message loop . . . 216
6.10.3 Could PTCP itself be an SXML statechart? . . . 218
6.10.4 Relationship to SCXML . . . 218
6.10.5 Toward an actor-oriented successor to SCXML . . . 219
6.11 Summary . . . 219

7 The Prolog Web ≈ the Semantic Web? . . . 221
7.1 The Semantic Web tower . . . 222
7.2 RDF as `rdf/3` . . . 223
7.3 SPARQL-style querying in Prolog . . . 225
7.4 OWL as an external reasoning service . . . 227
7.5 Positioning the Prolog Web alongside the Semantic Web . . . 228
7.6 Cooperation patterns . . . 229
7.6.1 A: Web Prolog as a SPARQL client and post-processor . . . 229
7.6.2 B: RDF as `rdf/3`, Prolog rules as the API . . . 230
7.6.3 C: OWL reasoning as a service, Web Prolog for orchestration 231
7.6.4 Summary . . . 232
7.7 Proof and trust . . . 232
7.8 User interfaces and applications . . . 232
7.9 Related work: towers of logic programming . . . 234
7.10 Outlook: Prolog Trinity towers . . . 235
7.11 Summary and discussion . . . 237

8 Prolog AI agents on the Agentic Web . . . 239
8.1 Why Prolog agents now? . . . 240
8.2 Symbolic competence in one engine . . . 241
8.2.1 Knowledge representation and inference . . . 242
8.2.2 Planning as search in a state-transition model . . . 242
8.2.3 Meta-interpreters as a control and explanation lever . . . 243
8.2.4 An expert system with a query-the-user facility . . . 243
8.2.5 Parsing as deduction: DCGs . . . 244
8.2.6 Summary . . . 245
8.3 Prolog AI agents on the Agentic Prolog Web . . . 245
8.3.1 An expert-system agent and a simulation . . . 245
8.3.2 Explainable AI across nodes: distributed proof trees . . . 247
8.3.3 Meta-interpreters as generic behaviours . . . 249
8.3.4 Parsing as remote deduction . . . 249
8.3.5 Implementing agent frameworks in Web Prolog . . . 250
8.3.6 Scalability and interoperability on the Web . . . 252
8.4 Neuro-symbolic AI agents . . . 252
8.4.1 Why hybridise? . . . 253
8.4.2 Web Prolog as symbolic backend and agent layer . . . 254
8.4.3 Architectural patterns for hybrid agents . . . 254
8.4.4 Typed probabilistic decisions as semantic observations . . . 257
8.4.5 A statechart-wrapped service-request agent . . . 260
8.4.6 Open engineering and research questions . . . 264
8.5 User-interface agents . . . 265
8.5.1 UI agents versus MAS agents . . . 266
8.5.2 Turn-taking and repair as control structure . . . 266
8.5.3 Dialogue as proof: the missing-axioms view . . . 267
8.5.4 Statecharts as dialogue managers . . . 267

8.5.5 Dialogue as a governed interface 269
8.6 Summary 269

Part III Synthesis and outlook

9 The Trinity as a web-native symbolic paradigm 273
9.1 The two partitions as a foundational principle 274
9.1.1 Two levels of abstraction 274
9.1.2 Unification as the universal data contract 275
9.1.3 Backtracking as paging 276
9.2 From compatibility to interaction portability 277
9.2.1 Node profiles as interaction contracts 278
9.3 Agents as the primary abstraction 280
9.3.1 Interaction as inference 281
9.3.2 Mutual constraint 281
9.3.3 Identity, correlation, and authority 282
9.3.4 Five complementary views 283
9.4 The logical foundation: purity, belief, and scoped inference 283
9.4.1 Pure Web Prolog and the pure Prolog Web 283
9.4.2 Epistemic reading and scoped inference 284
9.4.3 The less pure Prolog Web 285
9.5 The distribution-first model 285
9.5.1 The browser as a Web Prolog runtime 286
9.6 Control as a first-class dimension 287
9.6.1 Governance of interaction 287
9.7 Executable logic on the Web 288
9.8 Comparison to other approaches 289
9.8.1 Qu-Prolog and distributed Prolog agents 289
9.8.2 Linda and tuple-space coordination 289
9.8.3 Concurrent Prolog and the committed-choice languages 289
9.8.4 Erlang/OTP and actor behaviours 290
9.8.5 Oz/Mozart and the multiparadigm kernel language 290
9.9 Why the Trinity fits Prolog 291
9.10 The dual process 291
9.11 Summary 292

10 Communities, interoperability, and adoption 293
10.1 From fragmentation to federation 294
10.1.1 Incremental convergence through PIPs 294
10.1.2 One Prolog to rule them all 294
10.1.3 The middle way: interoperability as a federating force 295
10.2 Community infrastructure and adoption 297
10.2.1 Shared objects for collaboration 297
10.2.2 Playgrounds and learning 298
10.2.3 Conditions for wider adoption 299

10.3 Opening the network to neighbouring communities 300
10.3.1 For the broader logic programming community? 301
10.3.2 For the Semantic Web community? 304
10.3.3 For the AI community? 304
10.3.4 For web programmers? 305
10.3.5 For the Erlang community? 307
10.4 Risk analysis 308
10.5 Summary 310

11 From vision to execution 311
11.1 A research and engineering agenda 312
11.2 Two routes from prototype to platform 316
11.2.1 A production-oriented SWI-Prolog platform 316
11.2.2 A federated and standardised Prolog Web 316
11.2.3 A decision that need not be made all at once 317
11.3 From incubation to standardisation 317
11.3.1 What should be standardised, and when? 317
11.3.2 Incubation and institutional division of labour 318
11.3.3 The SXML notation and SCXML 319
11.4 The first specification targets 320
11.4.1 The actor substrate as the distinctive target 320
11.4.2 Actor identity, messaging, and lifecycle 321
11.4.3 The semantics of `spawn/3` 321
11.4.4 The resumable toplevel actor 321
11.4.5 Solution cursors and the ISO liaison 322
11.4.6 Layering the first standards 322
11.5 Implementation and interoperability 322
11.5.1 Why implementation comes early 323
11.5.2 Independent implementations and test suites 323
11.6 AI-assisted implementation and standardisation 323
11.7 Shared ownership 324
11.8 Closure 325

A Glossary 327

B Manual 339
B.1 Predicates for programming with core actors 339
B.2 Predicates for programming with toplevel actors 350
B.3 Predicates for programming with server actors 355
B.4 Predicates for programming with statechart actors 357
B.5 Predicates for programming with supervisor actors 358
B.6 Prolog-style generics 361
B.7 Predicates for remote Prolog-style calls 362
B.8 Node introspection 366
B.9 Common error contract 367

B.10 ISO predicates and related constructs 367
B.11 Standard library and utility predicates 369

C SXML notation reference 371
C.1 A complete minimal document 371
C.2 Vocabulary at a glance 372
C.3 XML and embedded Prolog 373
C.3.1 XML conventions 373
C.3.2 Where Prolog occurs 373
C.4 Document root and states 374
C.4.1 The <statechart> element 374
C.4.2 The <state> element 374
C.4.3 The <parallel> element 375
C.4.4 The <final> element and completion 375
C.4.5 The <history> element 376
C.5 Transitions: the <go> element 376
C.5.1 Selection and execution order 378
C.5.2 Timed transitions 378
C.6 Entry, exit, and the Prolog data model 378
C.6.1 The <onentry> and <onexit> elements 378
C.6.2 The <datamodel> element 379
C.6.3 Statechart built-ins 379
C.7 Deferred events: the <defer> element 380
C.8 Actor invocation: the <spawn> element 380
C.8.1 Common syntax and value rules 381
C.8.2 Option matrix 381
C.8.3 Actor, toplevel, server, and supervisor examples 382
C.8.4 Nested statecharts 383
C.9 Run-to-completion and event queues 383
C.10 Validation, tracing, and diagnosis 384
C.11 Relationship to SCXML 1.0 385
C.12 Authoring checklist 386

D Node protocol reference 389
D.1 Profiles and protocol surfaces 389
D.2 Common conventions 390
D.2.1 Versioning 390
D.2.2 Terms, pids, and formats 390
D.2.3 Compression, binary formats, and transport protection 391
D.2.4 Common answer and event forms 393
D.3 The toplevel communication protocol 394
D.4 The stateless HTTP binding 395
D.4.1 HTTP method semantics and caching 396
D.5 The semi-stateful HTTP binding 397
D.6 The stateful WebSocket binding 398

D.6.1 Version negotiation 399
D.6.2 Toplevel session commands 400
D.6.3 Bare actor commands 401
D.6.4 Browser actor addresses and output acknowledgements 401
D.7 Lifecycle and ownership 402
D.8 Starting a native node 403

E How to implement a Prolog node 405
E.1 From a profile to a running node 405
E.1.1 A useful decomposition 406
E.2 Notes on node implementation 407
E.2.1 Statelessness as a protocol property 407
E.2.2 Semi-stateful HTTP sessions and response retrieval 407
E.2.3 Stateful WebSockets and push-driven interaction 409
E.3 A native node on top of SWI-Prolog 409
E.3.1 The stateless HTTP endpoint 409
E.3.2 Avoiding spurious recomputation 412
E.3.3 Implementing the actor primitives 416
E.3.4 Actor I/O 425
E.3.5 Private databases and source installation 427
E.3.6 Distribution and toplevel actors 428
E.4 Sandboxing, isolation, and authority control 429
E.4.1 Whitelist and blacklist strategies 429
E.4.2 Language sandboxing and host isolation 430
E.4.3 WASM and WASI 430
E.4.4 Policy permissions inside the node 430
E.4.5 Resource governance 431
E.4.6 A layered deployment strategy 432
E.5 A browser-resident node 432
E.5.1 Implementing a browser node 432
E.6 Alternative implementation approaches 435
E.6.1 Other Prolog systems and host languages 435
E.6.2 Complementary GNU Prolog and Trealla prototypes 436
E.6.3 Prospects for a dedicated Web Prolog VM 438
E.6.4 Narrower profiles require less work 438
E.7 Summary 439

F Benchmarking three actor substrates 441
F.1 What is being compared 441
F.2 Method 442
F.3 Actor lifecycle 443
F.3.1 Chained spawning 445
F.4 Steady-state message passing 446
F.4.1 A browser measurement caveat 447
F.5 RPC, paging, and query placement 448

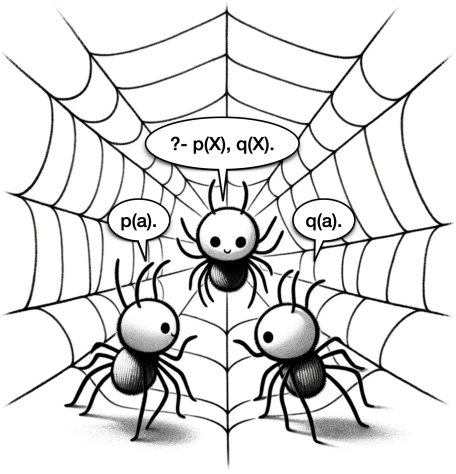
F.6 Interpretation and limits 448

References 451

Index 459

Part I
Foundations

Chapter 1
Introduction



This book proposes a new way to use Prolog on the Web. At its centre is *Web Prolog*, a *profile* of Prolog: a subset of the language, extended for concurrent and distributed programming. Web Prolog is meant for writing logic programs and agents that live on the Web.

Programs written in Web Prolog can act as *Prolog agents*: processes that reason, communicate and coordinate with each other across the network. We use the word *agent* in a modest, architectural sense. An agent is anything we can address and interact with by exchanging Prolog terms. The word implies no intelligence, autonomy, private state or minimum lifetime; those can be built on top.

The agents live in what we call the *Prolog Web*, an extension of the ordinary Web in which programs and agents can share knowledge and query each other.

Taken together, these three components form the *Prolog Trinity ecosystem*. We think of it as a niche within the World Wide Web. Behind it lies a conviction: symbolic reasoning still matters, and the Web is where it should be put to work.

1.1 The Prolog Trinity ecosystem

As suggested by the simple diagram in Figure 1.1, we think of Web Prolog, Prolog agents and the Prolog Web as three different kinds of entities, playing three distinct roles in the comprehensive Prolog-based ecosystem that we think of as the Trinity.

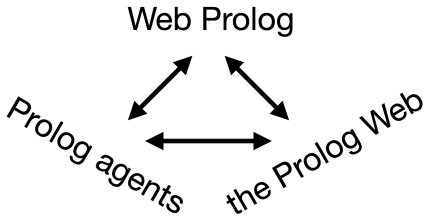


Fig. 1.1 The Prolog Trinity ecosystem: Web Prolog (a shared language), Prolog agents (active participants at node and actor scale), and the Prolog Web (their networked environment), together with their mutual dependencies.

The diagram illustrates the main idea on the highest level of abstraction we shall care about. The arrows are meant to indicate the many interdependencies and symbiotic relationships that exist between these entities, and to remind ourselves that we are looking at one and the same ecosystem from three different angles.

Put simply, on this level of abstraction the Prolog Web is a network of Prolog agents being programmable in Web Prolog and talking Web Prolog to each other. Expressed more elaborately, the Prolog Web is populated, and indeed also propped up, by Prolog agents, and provides, in the form of distributed Web Prolog databases, some of the knowledge that these agents need to do their thing. Prolog agents in the Prolog Web environment interact with other Prolog agents, using Web Prolog as their language of communication. They are also able to talk with (other kinds of) agents outside this environment such as humans, maybe through a conventional graphical user interface (a GUI), or perhaps using spoken natural language in a voice user interface (a VUI), and maybe even in a dialogue with an *embodied conversational agent* (an ECA).

It is important to understand that the Prolog Trinity ecosystem is not an island. The Prolog Web does not exist in complete isolation, cut off from the rest of the Web. Rather, it is meant to serve as an *extension* of the traditional Web, along with other extensions such as the Semantic Web and the Web of Things. Just like these, the Prolog Trinity can be regarded as a *sub-ecosystem* of the Web ecosystem in its entirety. It can also be thought of as a sub-ecosystem of the larger ecosystem surrounding Prolog – the already existing Prolog systems and the communities of logic programming researchers, system implementers, and other users of Prolog.

1.2 Walking in the footsteps of inventors

Before we continue, let us first pay tribute to five people who invented technologies that form the basis for the work presented in the book and laid the groundwork for the Prolog Trinity ecosystem. We have put them side by side in Figure 1.2.

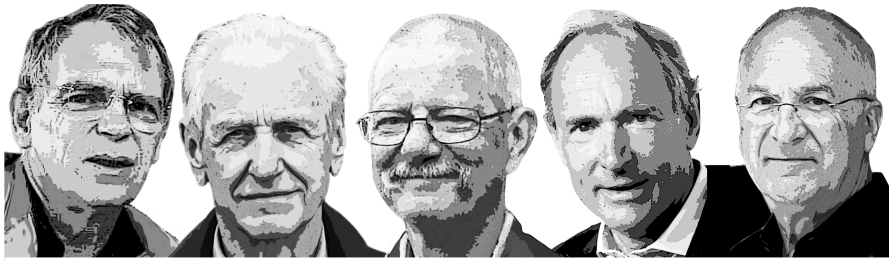


Fig. 1.2 Five inventors whose ideas underpin the Trinity: Colmerauer, Kowalski, Armstrong, Berners-Lee, and Harel (photo montage).

On the left we find Alain Colmerauer. He invented Prolog in 1972 and built the first implementation. Sadly, he passed away in 2017. To the right of him is Robert Kowalski – who came up with a lot of the theory behind Prolog which inspired Colmerauer. He is still around, and is still doing a lot of logic programming research and development. Third from the left, we find Joe Armstrong, who invented another programming language called Erlang. He liked Prolog, and the first version of Erlang was in fact implemented in Prolog. Regretfully, Joe Armstrong died in 2019. To the right of him, we find Tim Berners-Lee. He invented the Web, the first web browser, and the protocols and algorithms allowing the Web to scale. He is still very much alive and active trying to build a better Web. Finally, on the right, we find David Harel, the inventor of Statecharts, a visual state-machine language. Web Prolog uses statecharts to describe how an agent reacts to events (Chapter 6).

These five inventors are (or were, for those that are no longer with us) scientists and engineers who have made significant contributions to the field of computer science. We do not mean to suggest that they were alone in inventing these technologies.[1] The work presented in this book is indebted also to the many, many scholars who evolved the original ideas into the shape they have today. This book tries to build on their achievements, combining chunks of their work into what we hope will emerge as a coherent whole.

[1] For example, the Association for Logic Programming has given the official title "Founders of Logic Programming" to fifteen pioneering researchers who laid the groundwork for the field (see `https://en.wikipedia.org/wiki/Association_for_Logic_Programming`), and Robert Virding and Mike Williams were heavily involved in the design and development of Erlang.

1.3 The Prolog Trinity ecosystem concretised

A deeper understanding of what the Prolog Trinity is all about can hopefully be gained by considering the tiny fragment of the Prolog Web suggested in Figure 1.3.

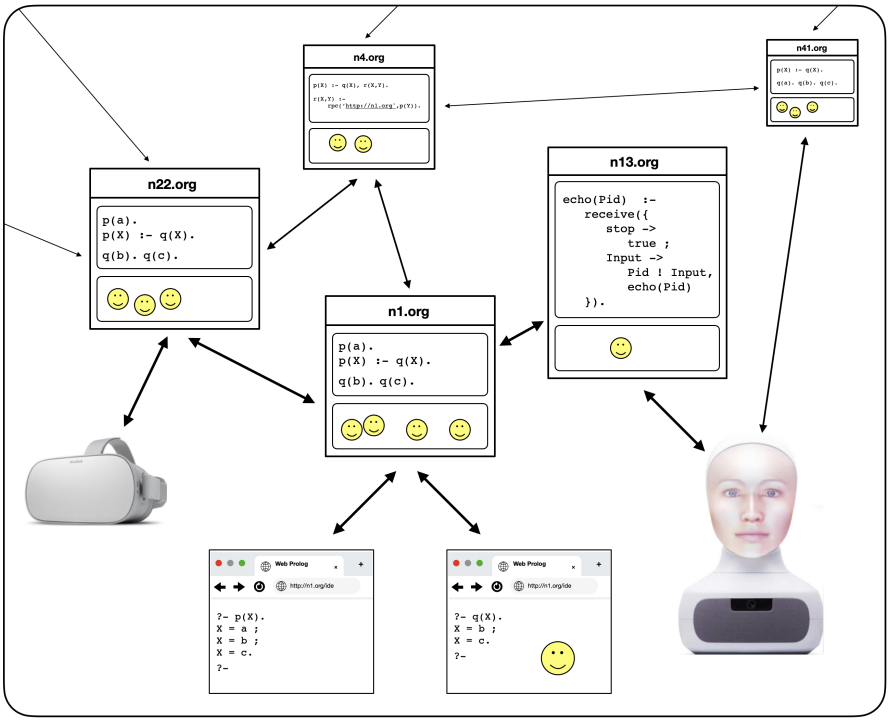


Fig. 1.3 A small fragment of the Prolog Web. Nodes (boxes) store Web Prolog programs and data (upper container) and host actor processes (lower container, shown as smileys).

On this fairly low level of abstraction, the Prolog Trinity ecosystem can be understood as a concrete network of concrete *Prolog nodes*. In Figure 1.3, nodes are shown as boxes labelled by a *domain name* at the top. The arrows symbolise connections between different nodes, and between end-user clients and nodes. As suggested by the diagram, each node is divided into two "containers," one container dedicated to the management of actor processes, the other to the storage of Web Prolog programs and data. The boxes and the actors they contain depict the two principal scales of Prolog agent: the node as a network-facing agent and the actor as its elementary unit of concurrent computation.

1.3.1 Prolog nodes and Prolog actors

Inspired by Erlang's notion of an actor, the *Prolog actor* is the fundamental unit of computation – and the simplest type of agent – in the Prolog Trinity ecosystem. The smiley emojis in Figure 1.3 represent Prolog actors of various kinds that live and work at nodes. When a node is computing something for a client, it is often one or more of these actors that are responsible, even if this is not evident from the outside.

Hiding an actor behind an emoji makes it look simpler than it actually is, and an actor has in fact an interesting internal structure. The diagram in Figure 1.4 shows three actors, one of which has just been spawned and one that has been opened up so that we can look inside (where, at this level of abstraction, they all look the same).

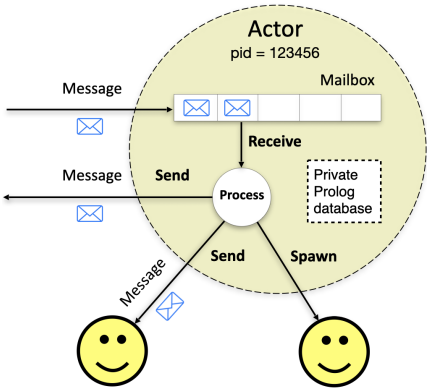


Fig. 1.4 The anatomy of a Prolog actor.

A Prolog actor is organised around a *process* – a thread of control capable of communicating with the world around it through messaging. An actor has a unique address – a *process identifier*, or *pid* for short. If we have the pid of an actor, we can message it, and since pids can themselves be part of a message, it allows other actor processes to communicate back. Furthermore, an actor has a *mailbox* that stores incoming messages. There is a *receive* operation that allows the actor to select and process messages from the mailbox, and a *send* operation that can be used to send messages to any other actor. Note that, as the figure shows, incoming messages are delivered to the mailbox, not to the process itself: sending is asynchronous, and the actor retrieves messages using the receive operation at a time of its own choosing. There is a *spawn* operation that allows an actor to create other actors – child actors as it were. Finally, every actor is equipped with a private Prolog clause database, a dedicated space where programs and data exclusive to that actor are stored.

Authorised end-user clients can connect and interact with nodes. These clients may range from web browsers running simple JavaScript-based Prolog terminals or intelligent web applications, to more sophisticated devices like VR headsets or social

robots. (Note that not all clients are end-user clients because nodes can be clients too.) Human users interacting with end-user clients typically trigger activity in the Prolog Web, where agents as well as connections between agents are more or less frequently being created, interacted with, and then destroyed. Some agents and some connections live for a long time, while others may only last for fractions of a second. This is obviously a form of dynamics that is not easy to capture in a static diagram such as Figure 1.3.

A node can be seen as a special-purpose web server serving data in the form of Prolog terms (possibly encoded in JSON) to clients as well as to other nodes. Just like a web server, a node has a domain name, makes use of HTTP and/or WebSockets, and is usually capable of serving more than one client at the same time.

We use the term *node* rather than *server*, partly because this is what Erlang uses, but a more important reason is that a node is much *more* than just a web server. A node is also a *runtime system* for Web Prolog capable of executing Web Prolog programs stored by the node and/or passed to it by the client over the web APIs. The Prolog Web is a network of such code, ready to be executed by actors, ready to be queried by clients.

1.3.2 Relations, actors, and interaction across nodes

Data and programs stored at a node come in two flavours, characterised by two kinds of Web Prolog clauses. One flavour might have clauses looking like this:

```
mortal(Who) :- human(Who).
```

Clauses of this flavour have logical interpretations, and this particular clause translates into the formula $\forall x[human(x) \rightarrow mortal(x)]$ in first-order predicate logic.

As suggested in Figure 1.3, a node can be queried from a terminal running in a web browser. Given our definition of `mortal/1` and a set of Prolog *facts* such as

```
human(socrates).
human(plato).
human(aristotle).
```

we can ask for the solutions to a query `?-mortal(Who)`. When we run the query from a terminal we enter into a kind of interaction with a Prolog process that most readers are probably familiar with – in which the system only offers one solution at a time and the user may request further solutions by typing a semicolon:

```
?- mortal(Who).
Who = socrates ;
Who = plato ;
Who = aristotle.
?-
```

Such queries can also be given logical interpretations, and the existential closure of this particular query is the formula $\exists x[mortal(x)]$. The interaction above not only establishes that there exists an x such that x is mortal, but also computes *witnesses* for x – terms that make the existential statement true. The point here is to remind ourselves that Prolog is a *logic programming language* eminently suitable for *knowledge representation*, or for other uses that require logic. The interaction also demonstrates that Prolog is a *nondeterministic* programming language, meaning that a query may have several solutions, which can be lazily iterated over using Prolog's backtracking mechanism.

In comparison with the clause defining the predicate `mortal/1`, the following clause has a very different flavour and looks a lot more like a program, and indeed something from the Erlang world – a clause in the *style* of Erlang:

```
echo_actor :-
    receive({
        echo(From, Msg) ->
            From ! echo(Msg),
            echo_actor
    }).
```

As its name suggests, it implements the core of an echo server: an actor capable of receiving messages and returning them to the sender as an "echo." In contrast to the definition of `mortal/1`, this clause cannot as a whole be given the same logical reading; both `receive/1` and `!/2` are built-in predicates with committed side effects, so the clause must also be understood procedurally – as a procedure that, when executed by an actor process, determines that actor's *behaviour*.

These examples illustrate two characteristic uses of Prolog clauses. A clause may describe a logical relation, an actor's sequence of actions, or a mixture of the two.

The predicate `spawn/2` creates a concurrent process running in parallel with the caller, binding the variable in its second argument to a process identifier:

```
?- spawn(echo_actor, Pid).
Pid = 10783471.
?-
```

An interaction exercising the echo actor might look like this:

```
?- self(Self).
Self = 81902375.
?- 10783471 ! echo(81902375, hello).
true.
?- receive({Message -> true}).
Message = echo(hello).
?-
```

Readers familiar with Erlang will recognise the pattern. Readers less familiar with Erlang only need to know, at this point, that the predicate `self/1` retrieves the pid

of the calling process – in this case the *shell*, which is itself an actor – that `!/2` sends a message to another process, and that `receive/1` waits for an incoming message. Even this tiny example already conveys the essential properties of an actor. It has its own mailbox, reacts asynchronously to messages, can run indefinitely, and maintains identity through its process identifier.

Beyond local concurrency, Web Prolog also supports two complementary modes of distributed interaction across node boundaries. We distinguish them by their *interaction contracts*: the operations available to a caller and the outcomes and events the caller can observe.

A *sequential interaction contract* presents the evaluation of a goal: the caller observes answers, failure, or errors, and may request further answers from the same computation. A session may also retain state, perform effects, or ask for input. These interactions can be understood as a sequence of steps in goal evaluation without exposing the interleaving of independently running processes. The simplest example is a remote procedure call that lets one node query another as naturally as it would query its own database.

The built-in predicate `rpc/2-3` evaluates a goal at a remote node addressed by a Uniform Resource Identifier (URI). From the caller's perspective, it behaves like an ordinary Prolog query, returning solutions one at a time through backtracking:

```
?- rpc('http://n2.org', human(Who)).
Who = socrates ;
Who = plato ;
Who = aristotle.
?-
```

A *concurrent interaction contract* makes independently progressing actors part of the caller's programming model. Programs address actors, send messages asynchronously, select messages from mailboxes, and observe lifecycle events such as a monitored actor's termination. The order in which actors progress and messages become available can affect the interaction. This builds on the actor model introduced above. The `spawn/3` predicate accepts a `node` option that takes a URI, allowing an actor to be created on a remote node:

```
?- spawn(echo_actor, Pid, [
       node('http://n4.org')
   ]).
Pid = 41983256@'http://n4.org'.
?-
```

Here we assume that `echo_actor/0` is already installed on `http://n4.org`. Remote spawning does not automatically transfer its definition; Chapter 4 introduces the source-loading options for that purpose. The returned pid can be used for messaging exactly as a local pid would be:

```
?- self(Self), 41983256@'http://n4.org' ! echo(Self, hello).
Self = 81902375.
?-
```

The shell abbreviates a local pid when displaying it; the value returned by `self/1` retains its node component for the reply route. Typing the displayed integer into the remote message would lose that route.

These two classes of contract define what we call the *sequential and concurrent partitions* of the Prolog Web. The distinction concerns the interface visible to a program: actors may implement a sequential query interface internally, and one application may use both kinds of interaction.

1.4 Motivations and goals

The Trinity responds to old problems in the Prolog ecosystem and to new opportunities on the Web and in AI. We describe both, then state the *goals* by which the proposal should be judged.

1.4.1 Challenges in the Prolog ecosystem

Created in 1972, Prolog is an old programming language, but age should not be mistaken for stagnation. Many implementations remain under active development, new ones continue to appear, and established systems have acquired major facilities such as tabling and constraint logic programming libraries. Prolog also continues to support research, practical applications, books, courses, and an active – if small – community.

The survey *Fifty Years of Prolog and Beyond* (Körner et al., 2022) provides a useful assessment of this situation. Published during the 2022 Year of Prolog,[2] it examines the evolution and current state of Prolog systems and considers how the language and its community might develop. Its account reveals both the continuing vitality of Prolog and the difficulties facing its ecosystem.

The diversity of Prolog systems is both a strength and a source of difficulty. Different implementations have developed distinctive features in response to different research and application needs. Most share the ISO Prolog standard's core, but they often differ in extensions, interfaces, libraries, and tools. These differences limit the portability of non-trivial programs. The survey stresses, however, that many of them are not fundamental and could be bridged through common interfaces and shared implementations.

Fragmentation also has a social and organisational dimension. Users tend to gather around particular systems, so knowledge, discussion, and support are often organised by implementation rather than shared across the Prolog community. Implementers likewise have few shared facilities for discussing interfaces, exchanging tests and

[2] `https://prologyear.logicprogramming.org`

libraries, or coordinating future work. Compatibility is widely valued, but the time and stewardship needed to maintain it are scarce.

The tooling picture is similarly mixed. Prolog has produced sophisticated debuggers, analysers, specialisers, and development environments, but their availability varies between systems. There is no standard package manager, support in mainstream development environments is uneven, and important work is duplicated across a small community.

Finally, Prolog faces what the survey calls the "post-desktop world of JavaScript web applications." Browsers, mobile devices, and networked services have become central computing environments, while other languages are generally easier to deploy in them. Individual Prolog systems provide web facilities, but the ecosystem lacks a portable web-facing layer through which different systems can expose services and interact.

These concerns (portability, fragmentation, shared tooling and the Web) motivate the Trinity. The Trinity does not replace existing Prolog systems. It defines a common language, agent model and network through which they can cooperate while keeping their differences.

1.4.2 New opportunities on the Web and in artificial intelligence

The post-desktop world that challenges Prolog also creates opportunities. HTTP, WebSocket, JSON, web workers, WebAssembly, and containers provide much of the infrastructure needed to expose a language runtime as a network service.[3] They leave questions of interoperability, security, and resource control unresolved, but allow Prolog systems to participate in the Web using established transport and deployment mechanisms.

Artificial intelligence presents a related opportunity. As *Fifty Years of Prolog and Beyond* (Körner et al., 2022) observes, data-driven methods are powerful at learning patterns, but their conclusions can be difficult to explain and their behaviour difficult to constrain. Symbolic methods offer complementary strengths: explicit rules, background knowledge, and inference that can be inspected. Logic programming can check proposed results and constrain actions, while learned methods can guide search and help acquire symbolic knowledge. Neuro-symbolic AI seeks to combine these strengths.

Prolog is well suited to this combination. Its terms represent programs, data, rules, and queries uniformly; its meta-programming facilities support interpreters,

[3] For readers less familiar with web technology: HTTP is the protocol by which a client, such as a browser, sends a request to a server and receives a response. WebSocket keeps a two-way connection open, so that either side can send a message at any time. JSON is a simple text format for structured data. Web workers let a browser run code in the background, and WebAssembly (WASM) lets programs written in languages other than JavaScript, including whole Prolog systems, run inside the browser. Containers package a program together with everything it needs to run on a server. Later chapters explain these technologies further where they matter.

analysers, and verifiers. Together with unification and constraints, these facilities provide a basis for planning, policy enforcement, and explanation.

Large language models make this opportunity more immediate. A model can generate text, code, and proposed actions, but a persistent, accountable agent also needs an *agent harness*: a system that maintains state, mediates tool use, checks outputs, enforces permissions and limits, and handles failure. Logic programming is a natural candidate for the inspectable and testable parts of such a harness.

The emerging Agentic Web also requires agents to be addressable, interact over time, and coordinate with other agents and services. Web Prolog can contribute both logical facilities for rules, constraints, and plans, and an actor model providing identity, private state, messaging, and lifecycle management. They do not make an agent safe, but they give us a place to put the checks.

A Prolog engine can thus offer planning, verification, or policy enforcement through a stable web interface. Conversely, a Prolog actor can use learned models and other external services. The Web enables this composition across runtimes; a common interaction layer extends it across Prolog implementations.

1.4.3 The five principal goals of the Prolog Trinity

Against this background, the Prolog Trinity pursues five principal goals.

1. **A simpler model for concurrent programming in Prolog.** Provide a portable actor model based on isolated processes, shared and private databases, mailboxes, asynchronous messaging, monitoring, linking, and supervision. The model should be easier to use than direct manipulation of operating-system threads and general enough to support a wide range of concurrent applications.
2. **A coherent model for distributed Prolog programming.** Extend querying and actor communication across node boundaries while preserving the same basic programming concepts. Distribution cannot be made indistinguishable from local computation: latency, partial failure, authority, and resource limits must remain visible where they affect program behaviour.
3. **A lingua franca for Prolog systems.** Define a common term language, operational core, and set of interaction interfaces through which different Prolog implementations can exchange goals, answers, programs, data, and messages. The aim is not to eliminate differences between systems, but to establish a shared boundary across them.
4. **A web-native language for intelligent agents.** Support addressable, stateful agents that can reason, react to events, coordinate with other agents, and interact with services and human clients. The actor model supplies the execution framework, while reusable behaviours – including statechart-based behaviours expressed in a combination of Web Prolog and XML – provide structure for more complex agents. These are building blocks rather than a prescribed cognitive architecture.

5. **Deployable Prolog nodes.** Make it straightforward to run a Prolog system as a networked service that can be discovered, secured, tested, and connected to other nodes and Web applications. A node should combine a Prolog runtime, shared programs and data, actor hosting, documented interfaces, and basic operational controls. The demonstrator developed in this book is intended to show that such a design is practical, not to serve as a production platform.

These goals rest on three commitments. First, the Trinity is federated: participating systems retain their own implementations and internal extensions. Second, participation is graded: cumulative profiles allow a node to implement only the parts of the architecture that it needs. Third, public behaviour is defined by precise, testable contracts rather than by assumptions about internal implementation. The same interfaces should also be usable by non-Prolog clients where appropriate.

The three parts of the Trinity divide the responsibility between them. *Web Prolog* provides the common language and interaction interfaces. *Prolog agents* provide the concurrent and distributed execution model. The *Prolog Web* is the open network environment formed by nodes, agents, services, and clients. The three parts are designed together: the shared language must support the agents, the agent model must work across the Web, and the Web interfaces must preserve the relevant Prolog semantics.

1.4.4 Webizing Prolog and prologizing the Web

> The essential process in webizing is to take a system which is designed as a closed world, and then ask what happens when it is considered as part of an open world. Practically, this effect on a computer language is to replace the names/tokens/identifiers for URIs. Thus, where before reference could only be made to something in the same document/program/module one can with equal ease make reference to something in a different one somewhere in that abstract space which is the Web.
>
> *Tim Berners-Lee, "Webizing existing systems," 1998* [4]

The relationship between Prolog and the Web runs in both directions. Webizing Prolog means making systems designed largely as closed worlds participants in an open one. Prologizing the Web means adding the distinctive capabilities of logic programming to the Web. These are complementary views of the goals just stated.

Webizing Prolog begins with identity. URIs identify nodes, knowledge bases, query solutions, and other network resources. Actor pids identify actors and include the identity of their hosting node when they cross node boundaries. Meaningful names are thereby no longer confined to a single program or runtime.[5]

---

[4] `https://www.w3.org/DesignIssues/Webize`.

[5] The Semantic Web applies the same discipline to entities, relations, and concepts. Web Prolog terms can likewise contain globally identified resources and concepts, allowing services and agents to reuse established Semantic Web vocabularies. Section 7.2 develops the relationship through the representation of RDF triples as `rdf/3` facts.

The sequential and concurrent forms of interaction described in Section 1.3.2 use ordinary web infrastructure, including HTTP, WebSocket, and representations such as JSON. A node can therefore be used by non-Prolog clients as well as by other Prolog systems. Participation in an open network also brings obligations that a stand-alone interpreter can often ignore: authentication, authorisation, sandboxing, containment, and resource limits are part of webization.

Prologizing the Web means making relations and agents available as network resources. A client can submit a goal to a node and receive bindings for its variables. Because requests and answers share the same term structure, unification provides a declarative interface between client and node. A relation can support several patterns of use without being recast as a collection of unrelated endpoints.

Multiple solutions can be returned incrementally, one slice at a time. The concurrent interface also lets a client address a persistent agent, send it messages, and take part in an ongoing interaction. Prolog thereby contributes unification, search, stateful agents, and executable logic to the wider Web. The result is an executable logic layer *within* the Web rather than a closed network available only to Prolog clients.

1.5 Thirty years of Prolog on the Web

Prolog on the Web is not new. In the mid-1990s the arrival of the Web sparked much interest in the logic programming community, reflected in two workshops devoted to the subject and in the development of libraries that mostly relied on creating so-called CGI programs in Prolog. Here is a snippet of text lifted from the call for papers for the second workshop:[6]

> This workshop is the second in a series intended to explore the elective affinities between Logic Programming and Internet technologies with emphasis on enhancing the World Wide Web with knowledge, deductive abilities and superior forms of interactive behavior. With the paradigm shift to highly inter-connected computers and programming tools, logic programming languages have a unique opportunity to contribute to practical Internet application development. Simplicity, remote executability, robustness, automatic memory management, are among the features some LP languages share with emerging tools like Java. Superior meta-programming and high-level distributed programming facilities, built-in grammars and dynamic databases, declarative semantics are among their competitive advantages.

Since then the Web has developed at an amazingly fast pace, and web applications have become increasingly speedier and more interactive. The Web of today is *very* different from what it was in the mid-1990s – faster, more reliable, more mobile, and with a fantastic selection of browser APIs that let a web developer add features such as spoken interaction, video or 3D graphics to an application. There is a reason we nowadays talk about, not only the Semantic Web, but also the Conversational Web, the Mobile Web, the Web of Things, and (more recently) the Agentic Web.[7] Fur-

[6] `http://www.cliplab.org/lpnet/proceedings97/preface.html`

[7] See (Lager and Myrendal, 2012) for a description of the many facets of the current Web.

thermore, many (potentially) disruptive technologies such as automation, artificial intelligence, blockchain technology, wearable technology, augmented reality, virtual reality, robotics and quantum computing are likely to more or less depend on and be integrated into the Web. In this book we focus on symbolic artificial intelligence and its integration into the Web.

With the Web (technically) so much better and so different from what it used to be, there is, in our opinion, every reason to make a new attempt at "enhancing the World Wide Web with knowledge, deductive abilities and superior forms of interactive behavior." In particular, "superior forms of interactive behavior" is likely to depend on the WebSocket protocol and/or the implementation of Web Prolog in browsers or (mobile) apps. The WebSocket standard did not exist in the mid-1990s, which meant that every approach to web logic programming had to work solely with HTTP. While HTTP remains the most important protocol (two of a node's three web APIs use it), important aspects of some of the inventions described in this book rely on the WebSocket protocol, a so called server-push technology which opens a whole slew of opportunities for developers of web applications based on Prolog.

Much has happened since. From the 1990s onward, Prolog has repeatedly adapted to the web's changing landscape. Early work such as the CLIP group's PiLLoW library showed how HTML and HTTP could be handled declaratively by treating web structures as Prolog terms. With the rise of the Semantic Web in the 2000s, Prolog gained renewed relevance: SWI-Prolog's RDF store, web libraries, and the ClioPatria platform demonstrated that a Prolog system could serve simultaneously as a high-performance knowledge base and a full HTTP server (Wielemaker et al., 2008, 2016).

In the 2010s, more interactive and distributed styles of web programming led to further innovations. Pengines (Lager and Wielemaker, 2014) made server-side Prolog engines accessible from JavaScript through a simple RPC model, enabling incremental solution streaming and smooth integration with modern web applications. At the same time, Tau Prolog brought ISO-style Prolog directly into the browser, allowing entirely client-side logic applications implemented in JavaScript (Riaza, 2024).

More recently, WebAssembly has made it possible to run full Prolog systems inside the browser. SWI-Prolog can now be compiled to WASM via Emscripten, and systems such as Ciao, Scryer and Trealla have been ported as well. These provide fast, sandboxed Prolog engines that interoperate smoothly with JavaScript, opening the way for high-performance logic programming in modern web applications.

Among Prolog systems, SWI-Prolog in particular has all the right tools for building server-side support for web applications, in the form of mature libraries for protocols such as HTTP and WebSocket, and excellent support for formats such as HTML, XML and JSON (Wielemaker et al., 2012, 2008). Therefore, it might be argued that since Prolog can and has been used for building server-side support for web applications, it should already be counted as a web programming language. But since this is true also for Python, or Ruby, or Java, or just about any other general-purpose programming language in existence, it would make the notion of a web programming language more or less empty.

What makes Web Prolog a web programming language in our sense is something else. Web Prolog defines a shared, public interaction surface for Prolog computation on the Web. Independently operated nodes are addressed by URIs, expose language-neutral HTTP and WebSocket interfaces, and implement explicit profiles whose observable behaviour can be tested across different Prolog systems. These interfaces specify how goals, answers, alternatives, messages, failures, and actor lifecycles cross a network boundary. They are also designed for interaction between parties that need not share either an implementation or a common trust domain.

In this respect, Web Prolog is more than a collection of web libraries for Prolog. It is a portable web-facing profile of Prolog: a way for logic programs and Prolog actors to become addressable, interoperable components of the Web while remaining recognisably Prolog. Prolog programs outside that profile remain entirely within the scope of conventional Prolog programming.

On that basis, we believe that Prolog should claim a more prominent position in this space. Web Prolog should be used not only for building server-side support for web applications, but should be considered as *part* of the Web, in much the same way as HTML, CSS, JavaScript, RDF and OWL are parts of the Web. In other words, as we think it deserves it, we would like it to be recognised as *a web technology in its own right*.

1.6 The Trinity demonstrator

The concepts described in the chapters ahead are accompanied by a running implementation: the *Trinity demonstrator*, a browser-based demonstrator that puts the Prolog Trinity ecosystem in working form.

It lets readers see and try out the ideas, not only read about them. The demonstrator is accessible from the Trinity homepage at

`https://trinity.elfenbenstornet.se`

and its companion source code is available in the Git repository linked from that page. Readers who prefer to read the book without stopping to experiment may do so; the text is self-contained. But readers who wish to try things out – to inspect which demonstrator nodes are currently available, to run a query against a live node, to spawn an actor, to load and exercise a statechart, or simply to watch the system respond in real time – will find the demonstrator a useful companion.

The demonstrator presents a workspace on the left and a terminal pane on the right. The workspace has three modes – a *Prolog editor*, a *Statechart editor*, and an embedded *Tutorial* – while the *Terminal* executes queries against the currently selected node using whichever transport mode the node allows. An *Examples* drawer provides one-click loading of the actor and statechart examples resident on each node. A *Logger* panel records all transport-level activity, and its trace filter exposes the step-by-step execution of a statechart as an inspectable audit trail. The implementation is

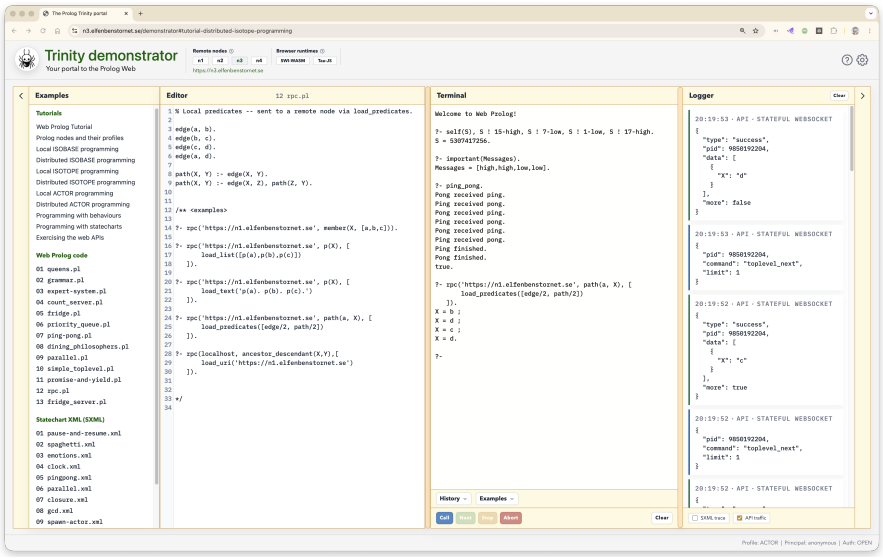


Fig. 1.5 The Trinity demonstrator.

written in SWI-Prolog and JavaScript, and the work has been done in cooperation with several AIs.

A second browser-facing component of the demonstrator is the *Discovery Hub*, itself implemented as the node `n0` and accessible at

`https://n0.elfenbenstornet.se/discovery-hub`.

The hub maintains a live directory of the demonstrator nodes, periodically probing them and presenting their advertised profiles, authorisation modes, database portability, and provided predicates and services. Its browser interface is backed by ordinary relations exposed through `n0`'s stateless API: in this sense, the demonstrator uses the Prolog Web to describe the Prolog Web.

The demonstrator is a proof of concept, not a production system. Browser-side execution is implemented using SWI-Prolog compiled to WebAssembly (see Appendix E). Further security hardening and more robust browser and multi-node operation remain open work. The demonstrator is hosted with best-effort availability, and readers should not be surprised if it is occasionally unreachable.

The SWI-Prolog demonstrator remains the principal proof-of-concept implementation and the basis for the examples and implementation account in this book. Complementary prototypes are being developed for an ISOBASE node in GNU Prolog and an ACTOR node in Trealla Prolog. They provide initial cross-system interoperability evidence at selected query and actor boundaries, rather than implementations of the entire Trinity ecosystem. Chapter 11 discusses their significance; Appendix E describes their implementation strategies and current limitations.

1.7 The structure of the book

Figure 1.6 maps the three principal aspects of the Prolog Trinity ecosystem and the relationships developed in the chapters that follow.

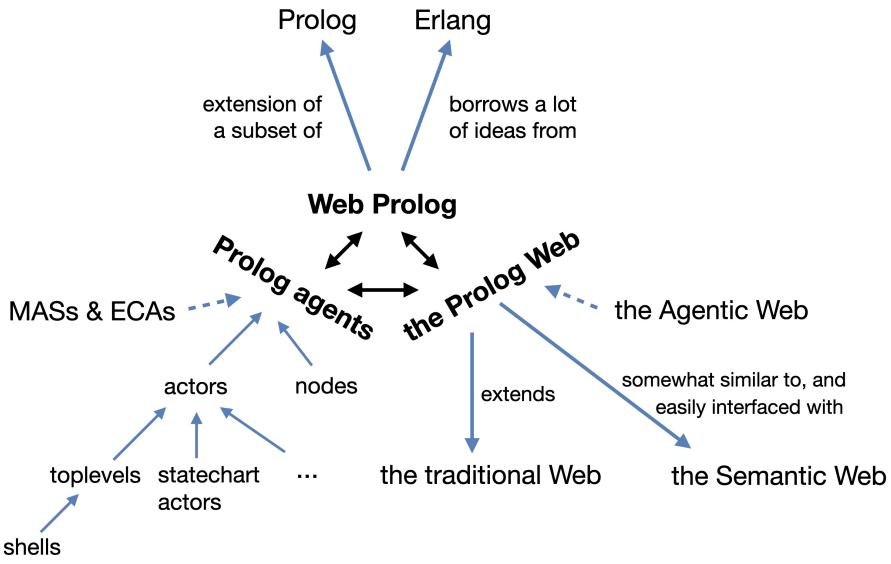


Fig. 1.6 Mind map of the Prolog Trinity ecosystem and its main conceptual threads. MASs means multi-agent systems and ECAs means embodied conversational agents. Chapter 11 redraws the map at the journey's end, distinguishing established relationships from proposed standardisation and adoption paths.

The book has three parts. The first defines the foundations of the Trinity. The second develops reusable extensions and places the proposal in its technological contexts. The third examines the combined paradigm, its possible audiences, and the path from proposal to implementation.

Part I: Foundations

This introductory chapter states the book's proposal, identifies the three components of the Prolog Trinity ecosystem, and explains how they fit together.

Chapter 2 introduces Web Prolog as a multi-paradigm language for concurrent and distributed programming. It explains what is inherited from standard Prolog, what is added to support actor-style concurrency, and why Erlang is a particularly useful

reference point. A range of Erlang programs are translated into Web Prolog to clarify the intended operational behaviour.

Chapter 3 develops the agent strand. It focuses on the two agent forms that matter most for the rest of the book – the Prolog toplevel actor and the Prolog node – and describes the built-in interaction protocol, node architecture, and client–node interaction patterns through a series of scenarios.

Chapter 4 presents the Prolog Web: how multiple nodes compose into an open, decentralised network. It introduces `rpc/2-3` for nondeterministic remote procedure calls, delineates the sequential and concurrent partitions, and addresses timing, failure, programmability, and security as first-class architectural concerns.

Part II: Extensions and contexts

Chapter 5 shifts from individual actors to reusable behavioural patterns – servers, supervisors, and toplevels – showing how common coordination concerns can be packaged as generic, configurable building blocks. It also introduces higher-level constructs such as `parallel/1` and `first_solution/2`.

Chapter 6 introduces the statechart actor: an event-driven state machine whose control structure is expressed in SXML, a specialised declarative notation within Web Prolog. The chapter explores hierarchy, history, orthogonality, and process invocation, and situates SXML relative to W3C's SCXML and Erlang's `gen_statem`.

Chapter 7 compares the Prolog Web with the Semantic Web and argues that the relationship is cooperative rather than competitive. It moves from conceptual comparison to concrete cooperation patterns involving RDF access, SPARQL-style querying, and OWL-based reasoning.

Chapter 8 turns to AI applications, exploring how statechart-style control and symbolic scripting can be combined for building interactive agents. It revisits classical symbolic AI strengths, examines neuro-symbolic combinations involving large language models, and discusses the governance of user-interface agents.

Part III: Synthesis and outlook

Chapter 9 asks what conceptual pattern emerges when the Trinity's components are considered together, and argues that they constitute a candidate paradigm for web-native symbolic computation – characterised by the orthogonality of the two partitions, interaction portability, the agent as the primary unit of structure, a logical foundation that survives distribution, a distribution-first programming model, and the treatment of interaction governance as a first-class dimension.

Chapter 10 asks what the Trinity offers both to the Prolog community and to neighbouring communities. Beginning from the current state of Prolog, it compares responses to systems and community fragmentation and develops the case for federation. It then surveys the wider logic-programming landscape and opens the same network to logic programming, the Semantic Web, AI, web development, and Erlang. The chapter closes by considering the risks and conditions of adoption.

Chapter 11 moves from vision to execution: standardisation, implementation paths, shared ownership, and the practical work required to turn the proposal into a testable reality.

Appendices

Six appendices provide supporting reference and implementation material. The first defines the book's principal terms; the second documents the Web Prolog language and predicate APIs; the third provides the SXML statechart reference; and the fourth specifies the public HTTP and WebSocket protocols of a node. The remaining two describe how a Prolog node can be implemented and compare the performance of the browser, native Prolog, and Erlang actor substrates.

Chapter 2
Web Prolog

> Imagine a dialect of Prolog with actors and mailboxes and send and receive – all the means necessary for powerful concurrent and distributed programming. Alternatively, think of it as a dialect of Erlang with logic variables, backtracking search and a built-in database of facts and rules – the means for logic programming, knowledge representation and reasoning. Also, think of it as a web programming language, and as a possible lingua franca for logic-based programming systems on the Web. This is what **Web Prolog** is all about.
>
> *Web Prolog – the elevator pitch*

2.1 The essence of Prolog

Based on formal logic, a subject dating all the way back to antiquity and tried and tested by generations of logicians and philosophers, logic programming forms a paradigm of its own, very different from the imperative or functional programming paradigms. Prolog is generally regarded as the first logic programming language, and is arguably the most important one. Consider the following program:

```
husband(Wife, Husband) :- wife(Husband, Wife).

wife(socrates, xantippa).
wife(aristotle, pythias).
```

We have here a *rule* that translates into the following formula in predicate logic:

$$\forall x \forall y [wife(x, y) \rightarrow husband(y, x)]$$

The rule in combination with the two *facts* of the predicate `wife/2` (that need no translation) allow us to query the predicate `husband/2`. We may for example ask if it is true that Aristotle is the husband of Pythias, ask who is the husband of Pythias, ask who is the wife of Socrates, or enumerate the married couples one by one:

```
?- husband(pythias, aristotle).
true.
?- husband(pythias, Husband).
Husband = aristotle.
?- husband(Wife, socrates).
Wife = xantippa.
?- husband(Wife, Husband).
Wife = xantippa, Husband = socrates ;
Wife = pythias, Husband = aristotle.
?-
```

This simple example serves as a reminder that Prolog is not only a logic language, but also a *relational* language that can be used to check if a sentence is true, to look up the value of one argument given the value of another, or to enumerate all pairs of values. When querying a binary relation, these are the *modes* that exist.

Because Prolog can pattern-match on structured terms in clause heads, `append/3` defines list concatenation with a base case and a recursive case that peels off the head `H` of the first list and rebuilds it on the result:

```
append([], L, L).
append([H|L1], L2, [H|L]) :- append(L1, L2, L).
```

All three arguments are relational – any of them may be given or left as variables – so `?-append(Xs,Ys,Zs)` holds precisely when `Zs` is `Xs` followed by `Ys`.

Complex terms in the arguments of clauses ensure that Prolog is a *Turing complete* programming language. This is what makes it different from a language such as Datalog, which is similar in many ways, but is not Turing complete.

In order to be not only Turing complete but also a practical and efficient programming language, serious Prolog systems need to offer a lot more, and they always do. In Section 2.1 of *Fifty Years of Prolog and Beyond*, the authors provide a conceptual and minimalist definition of the important features of Prolog, and are thus able to draw a line between what can be considered a Prolog implementation and what can not. Actually, they draw *two* lines, since they distinguish the *essential* features of Prolog from *important* (yet non-essential) features. Below, we reproduce their list of features, where 1-6 are considered essential features and 7-12 less essential. As it turns out, all the essential features except for 2) and 6) are involved when querying the binary relation between the married couples, or the ternary relation between the lists, in the examples given above.

1. Horn clauses with variables in the terms and arbitrarily nested function symbols as the basic knowledge representation means for both programs (a.k.a. knowledge bases) and queries;
2. the ability to manipulate predicates and clauses as terms, so that meta-predicates can be written as ordinary predicates;
3. SLD-resolution (Kowalski, 1974) based on Robinson's principle (1965) and Kowalski's procedural semantics (Kowalski, 1974) as the basic execution mechanism;

4. unification of arbitrary terms which may contain logic variables at any position, both during SLD-resolution steps and as an explicit mechanism (for example, via the built-in =/2);
5. the automatic depth-first exploration of the proof tree for each logic query;
6. some control mechanism aimed at letting programmers manage the aforementioned exploration;
7. negation as failure (Clark, 1978), and other logic aspects such as disjunction or implication;
8. the possibility to alter the execution context during resolution, via ad-hoc primitives;
9. an efficient way of indexing clauses in the knowledge base, for both the read-only and read-write use cases;
10. the possibility to express definite clause grammars (DCG) and parse strings using them;
11. constraint logic programming (Jaffar and Lassez, 1987) via ad-hoc predicates or specialised rules (Frühwirth, 2009);
12. the possibility to define custom infix, prefix, or postfix operators, with arbitrary priority and associativity.

As far as we know, no other language community has had to decide what counts as a *real* implementation of its language. Many systems can rightly claim to implement Prolog while remaining incompatible in important ways. This diversity helped motivate a shared standard. Since 1995, the core features of Prolog have been standardised by ISO, documented in a report known as ISO/IEC 13211-1:1995.[1]

Prolog belongs to a much broader family of logic-programming languages and formalisms, including abductive, inductive, constraint, probabilistic, Datalog, and answer-set approaches. Web Prolog is specifically grounded in Prolog and its ISO core; Section 10.3.1 surveys the wider landscape and considers how systems with different semantics might expose selected capabilities through shared interaction surfaces.

2.2 Web Prolog in a nutshell

Web Prolog is designed as a superset of a subset of the ISO Prolog core standard. The dialect defined by ISO is well understood, and is reasonably close to most of the dialects used in Prolog textbooks. The ISO Prolog syntax specification as well as a large subset of its built-in predicates form an excellent point of departure for our attempt to create a design and a standard for Web Prolog.

The conventional Prolog source syntax of Web Prolog is exactly as in ISO Prolog, except that three infix operators (`!/2`, `if/2` and `@/2`) that are not in the standard are defined. They can easily be added by means of `op/3`, the ISO Prolog predicate allowing a programmer to define prefix, postfix or infix operators and specify their

[1] `https://www.iso.org/obp/ui/#iso:std:iso-iec:13211:-1:ed-1:v1:en`

precedences. This means that syntactically well-formed Web Prolog programs can always be *read* (but not necessarily *run*) by a conforming implementation of ISO/IEC 13211-1:1995. It is easy to imagine circumstances where this might come in handy.

The expressive power of a programming language is to a large extent determined by the built-in constructs it provides. Rather than to make a long list, Figure 2.1 shows a "cloud" of built-in control constructs and predicates specified by the ISO standard.

Fig. 2.1 A cloud of ISO Prolog built-in predicates.

The current Web Prolog definition includes most of the predicates in this cloud; the precise portable predicate requirements are listed in Appendix B. Those predicates are necessary but not sufficient. The current definition also makes the following choices:

- In addition to the built-in predicates offered by the ISO Prolog core, nodes that accept general Prolog goals must supply a set of *standard library predicates*. These predicates are available without explicit loading. The *Prologue for Prolog* lists `member/2`, `append/3`, `length/2`, `between/3`, `select/3`, `succ/2`, `maplist/2-8`, `nth0/3`, `nth1/3`, `nth0/4`, `nth1/4`, `call_nth/2`, `foldl/4-6`

and `countall/2`.[2] See also the public-domain set of libraries developed by the Prolog Commons Working Group.[3]

- Item number 10 in the list of essential and important features of Prolog points to the possibility to express Definite Clause Grammars (DCGs) and parse strings using them. DCG notation and the associated parsing primitives are specified separately as an extension, in ISO/IEC 13211-3 (Part 3: Definite clause grammar rules).
- Web-facing access is provided through web APIs and remote-call predicates described later in the book. A host system may additionally expose facilities such as `http_open/3`, but such host-library predicates are not silently part of the portable Web Prolog surface. JSON representation at the protocol boundary is specified separately in Appendix D.
- At its fullest, Web Prolog extends its ISO-based sequential core with carefully crafted concurrency and distribution primitives heavily inspired by Erlang. It uses the same kind of actor and, where appropriate, the same names for corresponding operations, while making the deliberate deviations described later in this chapter.
- Web Prolog also provides specialised declarative notations for particular programming tasks. Most notably, SXML provides the statechart notation described in Chapter 6. This integrated notation shares Web Prolog's actor runtime, term and message model, logical environment, and executable content. The present chapter concentrates on conventional Prolog source notation.

The first three choices largely consolidate existing Prolog and Web practice. The fourth is a more substantial language-design commitment, with Erlang providing a clear precedent for its process, messaging, and failure mechanisms. The fifth builds on this actor model to support declarative descriptions of agent behaviour.

The portable definition omits some features, built-in predicates, and libraries. Some would cross the security boundary of a public node, some duplicate facilities supplied by the node itself, and others are not yet ready for standardisation:

- Only a subset of ISO Prolog is exposed. Predicates that grant ambient access to the operating system are outside the portable surface; a node owner may provide controlled capabilities separately.
- Predicates for constructing web servers, such as those in SWI-Prolog's HTTP server library, are not portable Web Prolog facilities. A Prolog node already supplies the required server boundary.
- Constraint programming is not currently standardised beyond facilities inherited from ISO. Individual nodes may offer additional constraint predicates; a shared definition can be adopted later if implementation experience supports one.
- The portable definition does not currently standardise a module system. Nodes and, where present, actors already provide important namespace and encapsulation boundaries, while host-specific modules remain an implementation facility.

[2] `https://www.complang.tuwien.ac.at/ulrich/iso-prolog/prologue`

[3] `http://prolog-commons.org`

Web Prolog is therefore a profile of Prolog for programming on the Web. Section 3.5 divides it into nested sub-profiles with progressively broader language and interaction surfaces. Narrower sub-profiles may omit database updates, terminal I/O, or `op/3`.

The word *profile* is used at two related levels. Web Prolog is a language profile: the portable language surface defined by this book. The node sub-profiles introduced in Section 3.5 are conformance profiles: cumulative, testable promises about the language and interaction surfaces a particular node exposes. This chapter describes the full Web Prolog language unless a profile-specific restriction is stated. At the narrowest relational boundary, the terms accepted as goals may be restricted to query patterns over an exposed schema; conformance does not require a general Prolog engine behind that surface.

2.2.1 Web Prolog is inspired by Erlang

> And if you think I would prefer multi-threading in Prolog to look as much as possible like Erlang, why yes, I would. I would like that very much.
>
> *Richard O'Keefe* [4]

Erlang is a general-purpose concurrent functional programming language developed by Joe Armstrong, Robert Virding and Mike Williams in 1986. It began as a proprietary language within Ericsson, but was released as open source in 1998. Originally designed for telephony applications, Erlang later proved highly effective for backend web systems and for a wide range of soft real-time control problems. Regarded by many as the first actor programming language to achieve broad practical success, Erlang is especially notable for its built-in support for concurrency, distribution and fault tolerance.[5]

A good overview of the language, its design intent and its underlying philosophy is given in Joe Armstrong's Ph.D. thesis (Armstrong, 2003b). There, the role of concurrency is described as threefold:

1. In Erlang, the fundamental role of concurrency is architectural. Systems are structured as sets of communicating processes that interact exclusively by message passing, which yields a natural decomposition into well-defined components that can be independently implemented and tested.
2. A system designed in this way can also be executed efficiently on multiprocessor machines or distributed across a network. But concurrency was not introduced in Erlang merely as a mechanism for improving performance. Efficiency is

[4] `https://groups.google.com/g/erlang-programming/c/1jdsnqZ4XfQ/m/ve9WfFl2YBwJ`

[5] The actor model itself originated with Carl Hewitt and colleagues in the early 1970s (Hewitt et al., 1973); Web Prolog's concrete concurrency design, however, draws most directly on Erlang. Book-length introductions to Erlang include *Concurrent Programming in Erlang* and Armstrong's *Programming Erlang* (Virding et al., 1996; Armstrong, 2007b)

only a potential benefit, and depends on whether the application can in fact be partitioned into genuinely independent tasks.

3. The main purpose of this architectural discipline is to limit the consequences of software errors. As Armstrong argues, large reliable systems are best built by partitioning them into independent concurrent processes that share no data and communicate only by message passing, combined with mechanisms for monitoring and restarting processes when they fail (Armstrong, 2003b, pp. 18–19).

Armstrong refers to this paradigm as *concurrency-oriented programming* (COP). This idea is central to the design of Web Prolog, which is itself conceived as a language for COP. In this book, actors are the main units of organisation in a distributed system on the Web, which is why we also call them *agents* (Chapter 3).

Inspired by the list of essential features of Prolog given in the previous section, we may also ask what it would mean for a language to count as Erlang-style. At a minimum, such a language would seem to require the following:

1. the ability to execute a large number of actor processes concurrently;
2. the ability to enforce process isolation, so that actors share no mutable state and communicate exclusively by asynchronous message passing;
3. the ability to support network-transparent concurrent programming, allowing actors to create other actors on remote computers and communicate with them seamlessly.

Added to the essential features of ISO Prolog, these facilities provide crucial parts of the foundation for Web Prolog, and indeed for the wider Prolog Trinity ecosystem. For this synthesis to work, however, the two sets of features must be compatible. Our current view is that they are, and part of the purpose of this book is to demonstrate that claim.

Why did we choose Erlang as a source of inspiration, rather than any alternative? After all, there are other actor programming languages – Akka, Pony and E, to name a few. The main reason for the choice of Erlang is that it is arguably the most *mature* concurrency-oriented language in existence, with a larger community than the alternatives, and a community that includes a fair number of industrial users. Many millions of lines of source code have been written in Erlang, many books teaching the language have been authored, and university courses teaching concurrent and distributed programming often use Erlang. In our opinion, since we are aiming for a standard it makes a lot of sense to borrow the required concurrency-oriented features from a language as mature and battle-tested as Erlang.

Another reason for our choice is that Erlang and Prolog are in many ways remarkably similar, both when it comes to syntax, and the reliance on dynamic typing, assign-once variables, pattern matching and recursion. The similarities can be explained by the fact that historically, Erlang evolved from Prolog and the first version of Erlang was actually implemented as an interpreter in Prolog (Armstrong, 2007a). The following listings of Erlang code (on the left) and Web Prolog code (on the right) show some of the similarities as well as some of the differences between the two languages:

```
append([], L) -> L ;
append([H|L1], L2) ->
    [H|append(L1, L2)].
```

```
append([], L, L).
append([H|L1], L2, [H|L]) :-
    append(L1, L2, L).
```

Note the use of capital letters for variables, the familiar notation for lists, and the reliance on pattern matching and recursion. A major difference is that Erlang is a functional programming language, whereas Web Prolog is relational. Since a function is just a special case of a relation this difference must not be exaggerated, but it does show in the way function calls may be nested, something that cannot be done in Prolog (or Web Prolog) since arguments are used to represent outputs as well as inputs. What *can* be done in Prolog, however, but not in Erlang, is to call `append/3` in more than one *mode* – not only to append one list to another, but also, for example, to nondeterministically split a list up into two parts. Furthermore, thanks to logic variables, `append/3` is tail-recursive in Prolog, but not in Erlang's `append/2` (as it is written here).

Despite such differences, as long as we do deterministic computation only, and no search is involved, logic programming and functional programming are fairly similar in the way they work, and methods used to achieve success with one often transpose to the other. Again, features such as pattern matching, assign-once variables and recursion play important roles in both languages.

During its evolution, Erlang gradually shed several features that Prolog had – and still has – while at the same time developing exceptionally strong support for concurrent and distributed programming, an area where Prolog historically lacked, and still lacks, comparable built-in support. Key here is the concept of an actor, and the ways actors can be scripted. Below, the echo actor written in Web Prolog is placed side-by-side with an implementation in Erlang – a function in Erlang and a predicate in Web Prolog – both making a recursive call that implements a loop:

```
echo_actor() ->
    receive
        {echo, From, Msg} ->
            From ! {echo,Msg},
            echo_actor()
    end.
```

```
echo_actor  :-
    receive({
        echo(From, Msg) ->
            From ! echo(Msg),
            echo_actor
    }).
```

The Web Prolog code on the right demonstrates the use of two constructs foreign to traditional Prolog, implementing the sending and receiving of messages in the style of Erlang. The programs have a similar look, and this is intentional. We have *strived* to make Web Prolog look as similar as possible to Erlang within the constraints imposed by the syntax and semantics of ISO Prolog.

A function call such as in `Pid = spawn(fun() -> echo_actor() end)` (or, but only for zero-argument functions, `spawn(fun echo_actor/0)`) can be made in order to spawn our echo actor in Erlang, while doing it in Web Prolog would use

something like `spawn(echo_actor, Pid)`. These calls look somewhat similar too, but while Erlang is a higher-order language in which the spawn function takes an anonymous function as its argument, Prolog (or Web Prolog) is not a higher-order language in this sense. In Web Prolog, `spawn/2` is a *meta predicate* which expects a callable term to be passed in the first argument, a term that when called will execute a procedure that determines how the actor will behave.

Both forms create a process with its own mailbox and identity. Erlang also supports a module–function–arguments form of spawning. Web Prolog collects monitoring, linking, and source choices in the option list of `spawn/3`; Erlang offers specialised spawning functions as well as the option-bearing `spawn_opt` interface. An ordinary actor can be named using `register/2`; the generic behaviour constructors also support a `name/1` option.

Web Prolog borrows isolated processes, asynchronous messaging, selective receive and lifecycle observation from Erlang, but changes several details. Comparison boxes along the way identify the Erlang precedent and the points where the designs differ.

2.2.2 Web Prolog is a multi-paradigm programming language

A common way to categorise programming languages is in terms of the programming paradigm(s) they support. Peter Van Roy's taxonomy of programming paradigms (Van Roy, 2009), depicted in Figure 2.2,[6] shows where Web Prolog fits. We have enclosed the relevant square boxes in boxes with rounded corners.

This suggests that we can regard Web Prolog as a language that supports three programming models: *relational logic programming* (as in Prolog), *imperative search programming* (also as in Prolog), and (as in Erlang) *message-passing concurrent programming*.

With access to predicates for asserting and retracting clauses in the dynamic database, traditional Prolog has always, albeit somewhat reluctantly, been able to support imperative programming (and later additions such as `nb_setval/2` have of course amplified this). Also, the built-in backtracking *search* that is so useful in Prolog has little to do with logic. Thus Prolog has never really been a single-paradigm programming language, and this is reflected in Van Roy's taxonomy. In (Van Roy, 2009) he suggests that this is not a bad thing:

> A good language for large programs must support several paradigms. One approach that works surprisingly well is the *dual-paradigm language*: a language that supports one paradigm for programming in the small and another for programming in the large.

With the addition of support for the actor programming model, Web Prolog becomes a much clearer as well as (in our view) a more interesting case of a multi-paradigm programming language. In particular, it can be argued that the support for programming in the large has been greatly improved.

[6] The diagram is available at https://www.info.ucl.ac.be/~pvr/paradigmsDIAGRAMeng108.pdf

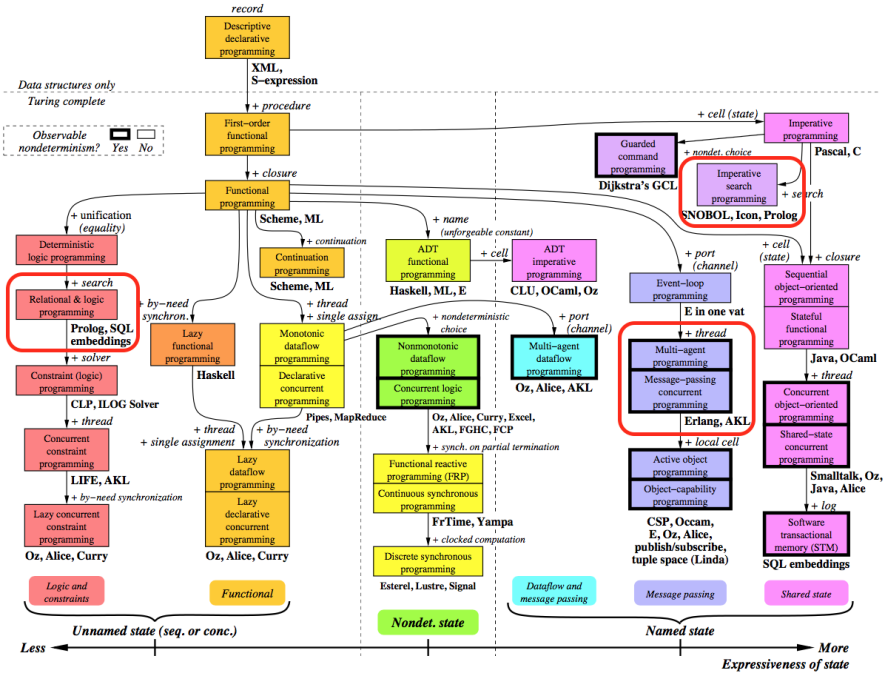


Fig. 2.2 A taxonomy of programming paradigms due to Peter Van Roy.

Any unexpected interactions between paradigms?

It is probably wise to entertain a suspicion of unexpected interactions between language features and possible impedance mismatches between the two paradigms – between Prolog's relational, nondeterministic programming model based on logic and Erlang's functional and message passing model. How well do the Erlang-style constructs mix with Prolog – with backtracking for example, or with the features for imperative programming? What do we get if we combine them? A kludge, or something quite beautiful?

In theory, we should be on the safe side. Sequential Erlang is basically Erlang with its data types, one-way pattern matching, functions and control structures, but without spawn, send, receive, and other constructs used for concurrent programming. The idea behind Web Prolog can thus be described as an attempt to "plug out" the sequential part from Erlang and "plug in" sequential Prolog instead. There seems to be no principled reason why we would not be able to replace the sequential functional language with a sequential relational language such as Prolog. Indeed, Armstrong (2003a) writes:

> Erlang is a concurrent programming language with a functional core. By this we mean that the most important property of the language is that it is concurrent and that secondly, the sequential part of the language is a functional programming language.

> The sequential subset of the language expresses what happens from the point it time [sic] where a process receives a message to the point in time when it emits a message. From the point of view of an external observer two systems are indistinguishable if they obey the principle of observational equivalence. *From this point of view, it does not matter what family of programming language is used to perform sequential computation.* (Our emphasis.)

Alternatively – and this has been our choice – the approach can be described as an attempt to *extend* Prolog with constructs such as spawn, send and receive. The idea is to keep everything that core Prolog has to offer, and extend it with a number of those primitives that make Erlang such a great language for programming message-passing concurrency. The choice between extending Prolog with Erlang-style constructs and extending Erlang with Prolog-style constructs is easy to make, and a lot has to do with syntax. Provided we can accept using a syntax which is relational rather than functional, precluding the nesting of function calls, it may be clear already at this point that the surface syntax of Prolog can easily be adapted to express the needed Erlang-style primitives. It is arguably a lot harder to express Prolog rules and other constructs using the syntax of Erlang.

2.2.3 Web Prolog as a scripting language for the Web

> Scripting languages are a lot like obscenity. I can't define it, but I'll know it when I see it.
>
> *Larry Wall*

Over the years, the Web has become a key delivery platform for sophisticated interactive applications in almost every domain, and JavaScript, more or less the only game in town for front-end programming, has become one of the most widely used programming languages on Earth: *the* web programming language of our times. Originally a small, highly domain-specific scripting language confined to the browser and used to modify the page being shown, it has since evolved into a widely portable general-purpose programming language.

Modern JavaScript is a high-level, dynamically typed, interpreted multi-paradigm programming language with several essential features that make it a versatile tool for web development. JavaScript's syntax supports various programming paradigms, including imperative, functional, and object-oriented programming styles. With features such as callbacks, promises, and async/await, JavaScript supports asynchronous programming, enabling non-blocking operations, especially useful in web applications. JavaScript's native format for data interchange, JSON, is lightweight and widely used for data transmission. JavaScript runs on almost all modern web browsers and platforms, making it universally applicable for web development. It easily interfaces with numerous web APIs for tasks like accessing the Document Object Model (DOM), making HTTP requests, and handling events. This makes it an excellent tool for connecting disparate systems in web applications.

With Node.js,[7] JavaScript also runs on servers. A whole web application, the part in the browser as well as the part on the server, can then be written in one language.

It is clear that as a relational logic programming language with the essential and important features listed above, Prolog is very different from JavaScript in terms of paradigms supported, and Erlang-style concurrency does not narrow the gap. What matters here is the *role* that JavaScript plays on the Web. We want Web Prolog to play the same role in the Prolog Trinity ecosystem.

The connection between scripting languages and the Web has been noted before. For example, in his article "Scripting: higher level programming for the 21st Century," John Ousterhout (1998) writes (about the Internet rather than the Web, but that makes little difference):

> The growth of the Internet has also popularized scripting languages. The Internet is nothing more than a gluing tool. It does not create any new computations or data; it simply makes a huge number of existing things easily accessible. The ideal language for most Internet programming tasks is one that makes it possible for all the connected components to work together; that is, a scripting language.

So what about the suitability of Web Prolog for programming the Web? Can Prolog challenge JavaScript in this space, at least for some applications, such as web-based AI? For this to be possible, we must ensure that Web Prolog is

- viable as a scripting language,
- suitable for programming the Web,
- possible to implement in browsers, and
- secure.

Web Prolog as a scripting language

As regards the viability of Web Prolog as a scripting language we note that while hardly a definition, Ousterhout (1998) characterises scripting languages as follows:

1. They are suitable for "programming in the large,"
2. they are designed for "glueing" applications together,
3. they tend to be typeless, thus making it easier to connect components,
4. they are usually interpreted, thus providing rapid turnaround during development by eliminating compile times,
5. they are higher level than a system programming language in the sense that a single statement does more work on average, and
6. they allow rapid prototyping.

JavaScript fits this characterisation, but given the development described above, it should now probably be described as a general-purpose language which also happens to be suitable for scripting. By contrast, languages such as C++ and Java are not well suited to that purpose.

---

[7] `https://en.wikipedia.org/wiki/Node.js`

What about Prolog? Does it satisfy Ousterhout's characterisation? The points 3-6 in his list are probably true of the Prolog programming language in general. (With the possible exception of 4 – some systems compile, but do it very quickly.) Whether points 1 and 2 apply varies between systems. For example, the people behind the commercial SICStus Prolog appear to regard Prolog more as a language for writing problem-solving modules to be embedded into other code, written in Java, C++, Python or what have you.[8] In contrast, acting as a glue language and a language for programming in the large is the ambition of SWI-Prolog, witness the following quote from the online manual:[9]

> SWI-Prolog positions itself primarily as a Prolog environment for 'programming in the large' and use cases where it plays a central role in an application, i.e., where it acts as 'glue' between components.

The people behind Erlang also think of it as a glue language:

> We use Erlang as the glue to handle all orchestration, and then we use Python, C, Julia, ... It is actually a language *intended* to act as a hub towards other languages. The interfaces could be protocols, could be RESTful APIs, or other programming languages. It's ideal for that.[10]

Erlang-style, network-transparent message-passing concurrency seems ideal for programming in the large on the Web, and for orchestrating both local and remote processes.

Erlang glue is likely a different kind of glue, with different properties, than Prolog glue. There is hope that combining Prolog and Erlang might yield an even stronger and more flexible glue of the required kind – a *superglue*, if you will.

Despite the good fit with Ousterhout's characterisation, neither Prolog nor Erlang are usually *advertised* as scripting languages, and they were hardly *designed* for the purpose of glueing applications together (or at least Prolog was not). They are both full-blown very flexible general-purpose programming languages with "batteries included." Like JavaScript, they should probably be regarded as general-purpose languages which also *happen* to be suitable for scripting.

Web Prolog is a web-facing Prolog profile rather than a special-purpose language. It can serve as a scripting and glue language, but also supports applications for which "scripting" is not really the right word. Its design focuses on web *logic* programming and web *agent* programming, in client-side browsers as well as on servers. Using Web Prolog, the owner of a node can, for example, build knowledge bases consisting of many millions of facts and rules, and web agents that make use of those knowledge bases.

Is Web Prolog suitable for programming the Web?

To demonstrate that Web Prolog is suitable for programming the Web is what this book is about. But we are going further than that, as we are also aiming for a novel

[8] Mats Carlsson, main developer of SICStus Prolog, personal communication

[9] `http://www.swi-prolog.org/pldoc/man?section=swiprolog`

[10] See `https://www.youtube.com/watch?v=K8nxTSPHZhs`, 8:58 into the discussion.

extension of the Web – the Prolog Web and even an Agentic Web – and a complete ecosystem, for which Web Prolog will serve as the default language.

Web Prolog running in browsers

Regarding our third point, for Web Prolog to become a viable web programming complement to JavaScript it is vital that it can be implemented in browsers too, since this comes with a number of advantages:

1. When network connection is slow, it is best to perform the majority of computations in the browser.
2. This is where, in most scenarios, the *state* of an interaction between a user and an application should preferably be represented.
3. Client-side computation reduces the demands placed on nodes.
4. This is also where we find browser APIs that let a web developer manipulate the DOM, store data locally, and add features such as spoken interaction, video or graphics to an application.

For Web Prolog in the browser to play the same role as JavaScript in the browser currently does, it must allow Web Prolog source code to be loaded from any server on the Web by means of the HTML `<script>` element, inline or with a link.

We do not, however, go all the way in our attempt to replicate JavaScript's abilities as a scripting language. One thing that we do not at this time aim for is to make Web Prolog into a language with primitives for scripting the Document Object Model (DOM), the browser's internal representation of the page being shown. With time it may well be interesting to develop such facilities, but we believe it is simply too early to try to standardise them. In fact, we suspect that JavaScript will reign supreme in that role, and that if a browser is running Web Prolog locally, it will be as a *web worker*,[11] a program running in the background of the browser, with which the main JavaScript program talks.

Web Prolog and security

As for security, similar to JavaScript, Web Prolog is a *sandboxed language*, open to the execution of untested or untrusted source code, possibly from unverified or untrusted clients without risking harm to the host machine or operating system. Therefore, Web Prolog does not include predicates for file I/O, socket programming or persistent storage, but must rely on the host environment in which it is embedded for such features.

Web Prolog shares some of the properties that made JavaScript succeed on the Web. A visit to a web application server starts a JavaScript process on the client, running code that has been downloaded from the application's host to the user's

[11] `https://en.wikipedia.org/wiki/Web_worker`

client. Such a process must be allowed to run there (that is, the owner of the client must allow it), and when it is (and most users allow it by default), it must execute in a way that does not harm the client. When a Web Prolog process is run on a node it must be allowed by its owner to do so, and it must run without harming the node. So one thing they share, Web Prolog and JavaScript, is the ability to run untested and untrusted source code, authored by unverified and untrusted programmers, in a sandbox on someone else's computer.

2.3 Erlang-style message-passing concurrency in Web Prolog

> In a nutshell, if you were an actor in Erlang's world, you would be a lonely person, sitting in a dark room with no window, waiting by your mailbox to get a message. Once you get a message, you react to it in a specific way: you pay the bills when receiving them, you respond to Birthday cards with a "Thank you" letter and you ignore the letters you can't understand.
>
> *Fred Hébert*

Corresponding to the send, receive, and spawn operations are built-in Web Prolog predicates `!/2`, `receive/1-2`, and `spawn/1-3`, among others. Their use is demonstrated through a series of simple examples, many drawn from Erlang tutorials and introductory texts. The examples are pitched at two kinds of reader. Prolog programmers taking their first steps into concurrent programming may wish to work through them carefully, perhaps alongside the Trinity demonstrator. Experienced Prolog teachers, by contrast, are invited to read as evaluators: does this material constitute a sound basis for teaching actor-style concurrency to Prolog students, and does Erlang-style concurrent programming sit naturally within a Prolog setting? We think it does, and that one web-based system can be used to teach both.

2.3.1 Programmer talking to actor, actor talking to itself

If we need to hold a Prolog-style conversation with an actor, we may want to talk to a Prolog *shell*. By using a terminal attached to a shell, we can interact with it the way we normally interact with Prolog, by querying it, updating its dynamic database, and so on. For example, here is how we instruct the shell to split a list up into a prefix and a postfix using `append/3`:

```
?- append(Xs, Ys, [a,b,c]).
Xs = [], Ys = [a, b, c] ;
Xs = [a], Ys = [b, c] ;
Xs = [a, b], Ys = [c] ;
Xs = [a, b, c], Ys = [] ;
false.
?-
```

This book, as we have warned, is not a beginner's textbook, so how Prolog is able to come up with solutions to such queries is not something we will cover. Our focus is rather on the concurrency-oriented, message-passing features of Web Prolog, so we begin instead by introducing a couple of simple messaging primitives.

The examples in this subsection and the next form one continuing shell session. First, using `self/1`, we need to determine the identity of the shell we will be talking to:

```
?- self(Self).
Self = 85234512.
?-
```

What we got back is a *pid*, an address for an actor process. Every pid has the form `Id@Node`, where `Id` is assigned by the node and `Node` is its URI. The shell may omit the node component when displaying a local pid, so `85234512` abbreviates a term such as `85234512@'http://n7.org'`. A node-assigned identifier is unique for the node's lifetime and is never reused. The demonstrator uses ten-digit integers to keep interaction compact; this book uses eight-digit examples for readability. Production identifiers should be substantially larger, for example with 128 random bits. Their entropy is defence in depth, not authorisation. A pid names an actor and is not a capability. Fresh request and monitor references provide correlation instead: they are single-use and should be hard to guess, but a party that legitimately holds a reference can construct a matching reply. The demonstrator also keeps public references at ten digits; that compact space does not provide production guessing resistance. Production references should use a larger space, such as 128 random bits.

The full term is retained internally and when sent in a message, allowing the recipient to use it as a return address.

Like every actor, the shell has a mailbox. A call `Actor ! Message` sends `Message` to the actor identified by a pid or, as we shall see, a registered name. Using the shell's own pid, we can make it send itself a message:

```
?- 85234512 ! hello.
true.
?-
```

Now we can use `receive/1` to make sure that the message we sent was received by the shell and is present in its mailbox. A single receive clause with a variable in the head and `true` in the body does the job:

```
?- receive({Message -> true}).
Message = hello.
?-
```

This is one of the simplest uses of the receive operation we can think of, but `receive/1` really is more complex than this, so expect more to come ahead in this chapter.

There is an even simpler call to `receive/1` that one can make, namely this:

```
?- receive({}).
```

This call does not wait for any specific message, which just means that it will block forever, or until the process that runs it is killed. It is never used in real programs, but it is handy for demonstrating linking.

2.3.2 Two utility tools for programming in the shell

Copying and pasting pids is not very convenient. Instead, we can use a small shell feature: if a variable name is prefixed with a dollar sign, the shell substitutes it with the variable's most recent binding before running the query. For example:

```
?- $Self ! hello.
true.
?-
```

Here, the shell kept track of the most recent binding of `Self` to a value (`85234512` in this case) and substituted the term `$Self` with that value before the query was run.[12]

During interactive development, it is also important to inspect the shell's mailbox. Using `receive/1` is often awkward here: if the mailbox happens to be empty, `receive/1` will block, effectively freezing the session until the call is interrupted (for example, with Control-C). Moreover, if we want to examine *all* pending messages, we would need to call `receive/1` repeatedly, without necessarily knowing how many messages there are – which again risks blocking, or that messages are left behind.

A more convenient tool is the utility predicate `flush/0`. It prints (and removes) all messages currently in the shell's mailbox and, crucially, it never blocks.

Below, we demonstrate both of these utilities by first sending yet another message to our shell and then use `flush/0` to inspect and empty the contents of its mailbox:

```
?- $Self ! goodbye.
true.
?- flush.
Shell got hello
Shell got goodbye
true.
?-
```

The $Var substitution mechanism (or "dollar notation") in combination with `flush/0` comes in very handy during interactive programming in the shell and we shall rely on them extensively when running examples through all of the book.

[12] Dollar notation is a convenience provided by the demonstrator's shell, not part of ISO Prolog or the portable Web Prolog language. Its notation and the lifetime of remembered bindings are properties of the client interface; Web Prolog programs should not depend on them.

2.3.3 Actors talking to other actors

An actor talking to itself is not very interesting. Fortunately, as already mentioned, an actor is capable of spawning other actors and then start talking to them.

The built-in predicate `spawn/1-3` is used to create new actor processes. In a call such as `spawn(Goal,Pid)` it expects a callable goal to be passed in the first argument, and will bind the variable in the second argument to the pid of the actor created. Here is how this might look:

```
?- spawn(echo_actor, Pid).
Pid = 72347585.
?-
```

The goal calls a predicate which determines the behaviour of the actor. The actor process runs in parallel with the caller, the shell in this case. Here is an example script that implements a simple echo server:

```
echo_actor :-
    receive({
        echo(From, Msg) ->
            From ! echo(Msg),
            echo_actor
    }).
```

We have seen this piece of code before and it is now time to explain how it works. The predicate `echo_actor/0` implements a simple receive loop: each call to `receive/1` selects a message of the form `echo(From,Msg)` from the mailbox, sends `echo(Msg)` back to the actor identified by `From`, and then recurses.

To test the server interactively, we send it a message with `From` bound to the shell's own pid, obtained via `self/1`:

```
?- self(Self), $Pid ! echo(Self, hi).
Self = 85234512.
?- flush.
Shell got echo(hi)
true.
?-
```

Sending is asynchronous: `!/2` does not block waiting for a response but continues immediately. An accepted send does not reveal whether the pid resolves to a live actor: a message addressed to a non-existing pid is discarded. Admission can nevertheless fail because of node policy, a full mailbox, or a term-depth limit. This means that short of having the targeted actor return a confirmation message we will not always know if the message reached it. However, the above case did not leave us in the dark, as the actual echo served as a confirmation message.

For local actors, repeated messages from one sender to the same pid are delivered in send order. Messages from different senders have no guaranteed order. Chapter 4 extends this account to names and network routes.

Monitoring

In a call such as `spawn(Goal, Pid, Options)` the optional third argument is a list of options used for configuring the actor. One such option is `monitor`, which if set to `true` requests that the parent process be notified when the spawned actor terminates.

In the following example, `monitor(true)` instructs the runtime to deliver a `down` message to the parent when the child terminates:

```
?- spawn(some_goal, Pid, [
       monitor(true)
   ]).
Pid = 60367387.
?- flush.
Shell got down(60367387,60367387,true)
true.
?-
```

A `down` message has the form `down(Pid, Ref, Reason)`, where `Pid` identifies the monitored actor and `Ref` identifies the *monitor instance* that produced the notification. For an observed lifecycle outcome, `Reason` describes why the actor terminated. For a remote observation failure it is instead `connection_closed`, with the distinct meaning given below. When monitoring is installed via `monitor(true)`, the monitor reference is chosen to be identical to the child's pid, which is why the first two arguments coincide in the example.

In this example, `true` reports successful completion of the actor's outermost goal. If that goal fails, the reason is `false`; if an exception `E` escapes uncaught, it is `exception(E)`. Some exceptions are errors.

ISO Prolog represents errors by terms of the form `error(Formal,Context)`. The formal part classifies the error; the context has an implementation-defined structure. Code inside an actor may handle an exception with `catch/3`, arrange cleanup with predicates such as `setup_call_cleanup/3`, translate the exception, or rethrow it.

In addition to the `monitor` option, the predicate `monitor(Pid, Ref)` is provided, which installs a monitor and returns a fresh reference in `Ref`. The reference matters whenever there may be more than one monitor, because each monitor produces its own `down` message and we must be able to tell them apart:

```
?- spawn(sleep(3), Pid), monitor(Pid, Ref1), monitor(Pid, Ref2).
Pid = 41544343,
Ref1 = 10924110,
Ref2 = 92041933.
?- flush.
Shell got down(41544343, 10924110,true)
Shell got down(41544343, 92041933,true)
true.
?-
```

Here the same parent process installs two monitors on the same child. When the child terminates after three seconds, two `down` messages are delivered, one per monitor. The distinct references (`Ref1` and `Ref2`) identify which monitor instance each message corresponds to.

Using `monitor(true)` in `spawn/3` installs monitoring as part of process creation, before the child can terminate. This ensures that the monitor observes the child's actual exit reason, even if the child completes immediately. In contrast, calling `monitor/2` after `spawn/2-3` leaves a gap in which the child may terminate. If that happens, the later monitor still produces `down(Pid,Ref,no_process)`, but cannot recover the original exit reason. Spawn-time monitoring avoids this loss of information.

Monitoring is allowed for any target the caller may send to. Only the actor that installed a monitor may remove it, by passing its ref to `demonitor/1-2`; another actor's ref has no effect. When monitoring is installed using the `monitor` option, the pid of the monitored actor can also be used since it is identical to the ref.

Box 2.1 — Erlang comparison: Monitors and process outcomes

Erlang monitors establish a one-way observation relationship. When a monitored process terminates, its observer receives a `'DOWN'` message containing a monitor reference, pid, and reason. Web Prolog follows the same pattern: `monitor/2`, or the atomic `monitor(true)` spawn option, causes the observer to receive `down(Pid, Ref, Reason)`.

When an Erlang process runs out of code, its exit reason is `normal`. Web Prolog instead distinguishes outermost goal success, `true`, from logical failure, `false`, and an uncaught exception, `exception(E)`. Erlang has no counterpart to ordinary Prolog failure as a separate outcome.

These conventional outcomes do not exhaust the possible reasons. A Web Prolog actor may terminate explicitly with `exit(Reason)`, or another actor may send it an exit signal with `exit(Pid,Reason)`. The reason may be any nonvar term, for example `shutdown` or `cancelled(Job)`; it is then reported by the monitor. Such a reason describes the explicit exit, rather than proving that the actor's goal succeeded or failed. Erlang also permits application-chosen exit reasons, with `normal` and `kill` having special signal-handling meanings. Web Prolog's containment termination remains untrappable (Section 2.3.3).

Registering

Calling `register(Name,Pid)` associates the atom `Name` with `Pid`. The name can be used instead of the pid when calling `!/2`. For example, we can register our shell under the name `shell`:

```
?- self(Self), register(shell, Self).
```

```
Self = 85234512.
?- shell ! hello.
true.
?- spawn(shell ! goodbye).
true.
?- flush.
Shell got hello
Shell got goodbye
true.
?-
```

Spawned actors inherit the caller's execution namespace, which is why the child above can resolve `shell`. Registering gives actors in that namespace a stable name for a process. After a crash, a replacement must be registered under the name again. Public service names across nodes use the separate `register_service/2` interface introduced in Chapter 4.

Exiting

Web Prolog supports two predicates, `exit/1` and `exit/2`, that can be used to terminate an actor process.

If a process calls `exit(Reason)`, it terminates immediately with the supplied reason. The reason need not denote an error: it is an arbitrary nonvar Prolog term chosen by the process. If the process is monitored, the runtime delivers a `down` message containing that term. Here is a simple example where a process is spawned and monitored by the shell. Since the process exits immediately with the reason `my_reason`, the shell receives a `down` message with that atom in its third argument:

```
?- spawn(exit(my_reason), Pid, [
       monitor(true)
   ]).
Pid = 91325643.
?- flush.
Shell got down(91325643,91325643,my_reason)
true.
?-
```

It sometimes happens that we need to terminate an actor process by force. Our echo server, for example, can only be terminated in this way. The predicate `exit/2` can be used to terminate an actor in the caller's ownership tree. A node owner may terminate any actor; other callers receive a permission error when they target another tree. Host thread numbers and aliases, including `main`, are not actor pids and cannot be terminated through this operation. Calling `exit(Pid,Reason)` sends an exit *signal* to the actor with that pid, killing it. Signals are runtime-control operations rather than mailbox messages.

If we do not know the pid but do know a registered name, we can first resolve the name to its pid. To see how this works, let us spawn a new monitored echo server and register it:

```
?- spawn(echo_actor, Pid, [
       monitor(true)
   ]),
   register(echo_actor, Pid).
Pid = 21562390.
?-
```

Now, let us say we change our minds and want to get rid of it again:

```
?- whereis(echo_actor, Pid),
   exit(Pid, 'We changed our minds!').
Pid = 21562390.
?- flush.
Shell got down(21562390,21562390,'We changed our minds!')
true.
?-
```

By calling `exit/2` with the pid in the first argument and a term detailing the reason for exiting in the second, a signal was sent to the process that terminated it. Note that a call to `exit/2` will only accept a pid in its first argument, so if all we have is a name, the built-in predicate `whereis/2` must be used to locate it.

Note that the reason passed to `exit/1-2` can be any nonvar term, not just an atom. This means that it can be used to transfer an arbitrarily large chunk of information back to the parent. However, in most cases an atom is all that is needed, and it is worth noticing that using `true` as a reason can be used to suggest to the parent that the termination was *normal*, even though it was caused by a call to `exit/1-2`.

What might seem a bit odd is that even though `21562390` is now provably dead, trying to send it a message using a pid or calling `exit/2` still succeeds without an error, although no `down` message is being sent:

```
?- self(Self), $Pid ! echo(Self, bye).
Self = 85234512.
?- exit($Pid, 'We changed our minds!').
true.
?- flush.
true.
?-
```

Treating `!/2` and `exit/2` as no-ops when the pid points to a non-existent process is consistent with how it works in Erlang. However, trying to send a message using the *name* of a non-existent process generates an error:

```
?- echo_actor ! echo($Self, bye).
Error: The name 'echo_actor' is not associated with a pid.
?-
```

This too is consistent with how Erlang works.

Linking

A further spawn option, `link`, governs the lifetime of the spawned actor. If the option is set to `true`, the life of the new actor is tied to the life of its parent: should the parent terminate – for whatever reason – the child is forced to terminate too. In our proposal, the default value of this option is `true`.

In the following example, the outer call to `spawn/2` creates an actor whose goal spawns a second, non-terminating actor and then completes. When its goal completes, the outer actor terminates – and because the inner actor was created with the default `link(true)`, it is automatically killed along with it:

```
?- spawn(spawn(receive({})), Pid).
Pid = 10782012.
?-
```

Calling the built-in predicate `actors/1` confirms that only one actor remains alive – the shell itself:

```
?- actors(Alive).
Alive = [85234512].
?-
```

The dependence does not run the other way. A parent whose child terminates – even abnormally – is unaffected:

```
?- spawn((spawn(exit(die)), receive({})), Pid).
Pid = 40276924.
?-
```

Even though the child killed itself immediately, calling `actors/1` again shows that its parent `40276924` is still alive:

```
?- actors(Alive).
Alive = [85234512, 40276924].
?-
```

The link, then, makes the life of a child dependent on the life of its parent, and never the reverse.

Since the option is on by default, declining it must be done explicitly, by passing `link(false)` when the child is spawned. Note carefully where the option goes: it belongs to the child's own spawn, and describes that child's relationship to its parent. In the following example, it is the *inner* actor that is spawned with `link(false)`, and it consequently survives the termination of the outer actor that created it. This example runs in a node-owner context:

```
?- spawn(spawn(receive({}), _, [link(false)]), Pid).
Pid = 10023812.
?- actors(Alive).
Alive = [85234512, 24917621].
?-
```

The outer actor (`10023812`) is gone, its goal having completed, but the inner actor (`24917621`) lives on – now without a living parent. Actors of this kind, detached from the hierarchy that created them, are useful for long-lived services, but they are also actors that no parent's termination will ever clean up. On a public node, such actors are therefore reserved for the node owner. A non-owner's language-level attempt raises `permission_error(spawn,detached_actor,link(false))`. Detaching rehomes the actor under the node: its output goes to the node log, and its linked descendants depend on that service rather than on the creator's connection. The node owner now manages its lifetime.

A hierarchy of actors

The lifetime dependence introduced earlier is *transitive*: because each child depends on its parent, terminating an actor tears down not just its immediate children but its entire descendant subtree. The following example makes this cascade visible.

The predicate `root/0` spawns a child and then blocks indefinitely. The child spawns a grandchild and does the same. Neither names anything or monitors anything; the only structure is the spawn hierarchy itself:

```
root :-
    spawn(child),
    receive({}).

child :-
    spawn(grandchild),
    receive({}).

grandchild :-
    receive({}).
```

We spawn the root from the shell and then confirm, using `actors/1`, that all three processes (plus the shell itself) are alive:

```
?- spawn(root, Pid).
Pid = 50318842.
?- actors(Alive).
Alive = [85234512, 50318842, 71200934, 33910576].
?-
```

The shell is `85234512`; the other three are the root, the child, and the grandchild. Now we terminate the root, and *only* the root, by force:

```
?- exit($Pid, die).
true.
?- actors(Alive).
Alive = [85234512].
?-
```

The root is gone, as expected – but so are the child and the grandchild, even though we sent an exit signal to neither. This is the cascade: terminating the root made its child terminate, which in turn made the grandchild terminate. Lifetime dependence propagates down the whole hierarchy.

The reverse does not hold: terminating a child does not terminate its parent. Because lifetime dependence propagates downward, a client's linked workers are reclaimed once termination of their parent or loss of their owning connection is detected. This is a conditional containment guarantee: contact loss does not prove that remote work has already stopped, and host operations can delay the safe point at which termination takes effect. A child cannot trap this downward termination. Remote detection and leases are discussed in Section 4.1.7.

Box 2.2 — Erlang comparison: Links, monitors, and lifetime dependence

Erlang links are symmetric failure relationships: an abnormal exit can propagate an exit signal to the linked process. Web Prolog uses a different parent–child relation. With `link(true)`, the child depends on the parent: termination of the parent terminates the child, but termination of the child does not automatically terminate the parent. A monitor reports the child outcome separately.

A spawn tree expresses ownership and lifetime containment; a supervision tree additionally specifies how failures are observed, whether a child is restarted, and when recovery escalates. Ordinary Erlang `spawn` creates no link, whereas Web Prolog makes this parent–child tie the default. The default suits the open Web: temporary work started for a client normally disappears when its owner disappears, without making the owner depend for survival on every worker.

The separation of lifetime dependence from observation follows Svensson et al. (2010, Section 2.2). Web Prolog additionally makes the directional relation part of spawning and enables it by default. Chapter 4 explains why this suits the open Web.

Signals

Messages are not the only way to affect an actor. Runtime control uses a separate *signal channel*. Signals bypass the mailbox and selective receive, so they can affect an actor while it is evaluating a goal or waiting for a particular message (Figure 2.3).

The signals we have met so far are *termination signals*: `exit/2` sends one, and when a parent terminates, the runtime sends one to each linked child. A signal takes

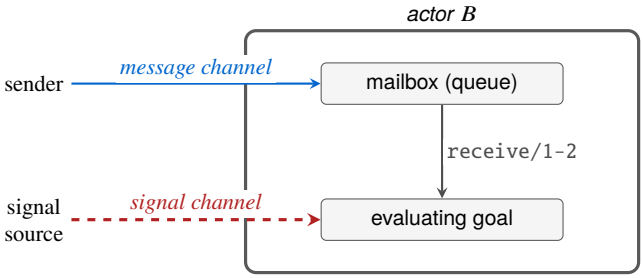


Fig. 2.3 Messages enter the mailbox and are selected by `receive/1-2`; runtime signals bypass the mailbox.

effect at the earliest safe point provided by the host runtime; an atomic built-in need not be interrupted part-way through. Supported cleanup handlers run as the actor unwinds.

Termination cannot be refused by application code. Web Prolog therefore has no counterpart of Erlang's `trap_exit`: allowing a child to turn its parent's termination into an ordinary message would let it escape the lifetime dependence described above. Inside the actor, a termination signal arrives as a *control exception*, and so do the interruption signal of Section 3.3.3 and the running-time limit that a node enforces. Other resource failures, including inference exhaustion, are ordinary catchable exceptions. Control exceptions propagate through receive guards, through the recovery goals of `catch/3` and through cleanup handlers. Application code can observe that one is passing, but it cannot stop it. Only the toplevel's own protocol layer consumes an interruption.

Termination commits when the runtime first accepts a termination signal for the actor. From then on the actor is *terminating*: its exit reason is fixed, and later termination or interruption signals have no further effect. Monitors report the completed lifecycle event as an ordinary `down/3` message.

"Cannot be refused" does not mean "takes effect at once". A signal waits for a safe point, and some operations delay it: a call into foreign code that does not check for signals, a blocking network operation inside a region that the host protects from interrupts, or a cleanup handler that itself blocks or loops. Such operations should carry their own timeouts. The runtime does not promise a bound on how long an actor takes to unwind.

The message and signal channels have no shared ordering guarantee. Sending a "prepare to stop" message before a termination signal therefore does not ensure that the actor processes the message first. A protocol that requires preparation must wait for an acknowledgement before terminating the actor.

2.3.4 A closer look at receive/1-2

As shown already, but only for a trivial case, an actor process uses the receive primitive to extract messages from its mailbox. Since the syntax and semantics of `receive/1-2` is fairly complex, a closer look and more examples are needed. Below we give several simple examples illustrating different ways to use the `receive/1-2` predicate in Web Prolog, demonstrating how to handle different *types* of messages, use *timeouts*, apply *guards*, and more. Other examples illustrate how messages are deferred and handled in subsequent `receive/1-2` calls, demonstrating the flexibility of passing and processing messages in Web Prolog.

Basic receive

In Web Prolog, just like in Erlang, the receive operation specifies an ordered sequence of *receive clauses* delimited by semicolons. A receive clause always has a *head* (a term) and a *body* of Prolog goals. Schematically, the basic form of a receive call looks like this:

```
receive({
    Head1 -> Body1 ;
    Head2 -> Body2 ;
    ...
    HeadN -> BodyN
})
```

Often, the head is just a single term serving as a *pattern*. Any term will do, ground or non-ground, and even a bare variable is fine.

As in Erlang, `receive/1` scans the mailbox looking for the oldest message that matches a pattern in any of the receive clauses, blocking if no such message is found. If a matching clause is found, the message is removed from the mailbox and the body of the clause is called. In Web Prolog, just like in Erlang, values of any variables bound by the matching of the pattern with a message are available in the body of the clause.

Often, a `receive/1` call waits for a specific message and executes the corresponding code in the body of the receive clause if a message that matches the pattern shows up. For example, the following call waits for a message in the form of `hello(Name)` and prints a greeting when it appears:

```
receive({
    hello(Name) ->
        format("Hello, ~s!~n", [Name])
})
```

We can specify multiple patterns in a single `receive/1` call. For example, this call handles messages of either the form `hello(Name)` or the form `goodbye(Name)`, but only one of them:

```
receive({
    hello(Name) ->
        format("Hello, ~s!~n", [Name]) ;
    goodbye(Name) ->
        format("Goodbye, ~s!~n", [Name])
})
```

We can use the _ pattern to catch any message. In the following case, any message that does not match `hello(Name)` will be caught by the _ pattern:

```
receive({
    hello(Name) ->
        format("Hello, ~s!~n", [Name]) ;
    _ ->
        format("Unknown message received.~n")
})
```

Matching is *one-sided*. A pattern may bind its own variables to parts of the message, but it never binds variables that occur in the message. The pattern `offer(X)` matches the message `offer(coffee)`, binding `X`, whereas the pattern `offer(coffee)` does not match a message `offer(Y)` whose argument is a variable. Messages may contain variables, since they are copied like any other term, but a variable in a message never becomes equal to some value merely because a receiver expected that value. Erlang's pattern matching follows the same rule. It matters most for correlation: a reply expected under a fresh reference `Ref` is selected only if the message actually contains `Ref`.
We can nest `receive/1-2` calls. Here, after having received the `start(Name)` message, the call waits for a `continue(Msg)` message:

```
receive({
    start(Name) ->
        format("Starting with ~s.~n", [Name]),
        receive({
            continue(Msg) ->
                format("Continuing with ~s~n", [Msg])
        })
})
```

Messages deferred

If no pattern matches a message in the mailbox, the message is *deferred*, which means that the message does not match any of the patterns in the current `receive/1-2` call and remains in the process's mailbox, possibly to be handled later in the control flow of the process. The receive is still running, waiting for more messages to arrive, and for one that will match.

In the following example, the `goodbye("Bob")` message – which is the oldest and therefore first in line – does not match the clause in the first receive call and is deferred. The `hello("Alice")` message matches that clause, and then the clause in the second receive call handles the `goodbye("Bob")` message:

```
?- self(Self),
   Self ! goodbye("Bob"),
   Self ! hello("Alice"),
   receive({
       hello(Name1) ->
           format("Hello, ~s!~n", [Name1])
   }),
   receive({
       goodbye(Name2) ->
           format("Goodbye, ~s!~n", [Name2])
   }).
Hello, Alice!
Goodbye, Bob!
Name1 = "Alice",
Name2 = "Bob",
Self = 63487504.
?-
```

This behaviour is particularly useful if we expect two messages but are not sure which one will arrive first.

Guards

The head of a receive clause can, in addition to the pattern, optionally specify a *guard* in the form of a Prolog goal. A head of the form `Pattern if Goal` makes pattern matching more expressive: the clause is selected only if the incoming message matches `Pattern` *and* `Goal` succeeds. Here is an example that distinguishes positive from non-positive numbers:

```
receive({
    number(N) if N > 0 ->
        format("Positive number: ~p~n", [N]) ;
    number(N) if N =< 0 ->
        format("Non-positive number: ~p~n", [N])
})
```

Here the guards test values supplied by the pattern. A guard can also derive new bindings. It is an ordinary Prolog goal, evaluated within the actor's *logical environment* – the logical resources available to the actor – and can *bind variables that the pattern never mentions*:

```
?- self(Self),
   Self ! hello(xantippa),
   receive({
       hello(W) if husband(W, H) ->
           format("Hello, ~w, say hello to ~w!~n", [W,H])
   }).
Hello, xantippa, say hello to socrates!
H = socrates,
Self = 63487504,
W = xantippa.
?-
```

The message supplied `W = xantippa`. The variable `H` appears nowhere in the pattern `hello(W)`; the guard `husband(W, H)` introduces it and binds it to `socrates`, and the body then uses that binding.

A successful Web Prolog guard has the conceptual form

$$\textit{partial substitution} \times \textit{logical environment} \longrightarrow \textit{extended substitution}.$$

The pattern first contributes bindings obtained from the message. The guard then extends those bindings by proving a goal within the actor's logical environment. In the example above, matching supplies `W = xantippa`, while proving `husband(W,H)` supplies `H = socrates`. The body receives the combined substitution.

The logical environment is not confined to the actor's private database, or even to a single database. It may include the actor's private clauses, the node's shared database, imported predicates, and built-in predicates – and, across node boundaries, remote logical services as well. A guard can therefore combine resources from the logical environment in one goal.

This matters because a Prolog predicate is relational rather than tied to one direction of computation. A conjunction such as

```
authorized(User), owns(User, Printer), online(Printer)
```

states constraints joining the message with three relations available in the actor's logical environment. Variables are shared freely across the pattern, the guard and the body; conjunction composes further constraints; and the same predicates can be queried in other modes elsewhere in the program.

For clause selection, however, a guard is evaluated only for its first solution, as if by `once/1`. Pattern matching and guard evaluation use a tentative substitution: if either fails, all bindings made for that candidate are discarded before another clause or mailbox message is tried, and the mailbox remains unchanged. Bindings produced by a successful guard remain available in the clause body, but backtracking through the body does not ask the guard for another solution and does not reconsider the selected message or receive clause. The one-sided rule applies to the pattern, not to the guard: a guard is an ordinary goal and can bind variables in the message. A guard that checks a correlation reference or a pid should therefore test it rather than

bind it, writing for example `ground(Pid),memberchk(Pid,Pids)` rather than `memberchk(Pid,Pids)` alone.

For portable code, a guard must be observationally free of side effects. The runtime need not enforce that restriction, and effects performed by a non-conforming guard are not undone on failure. An ordinary exception raised by a guard is treated as a non-match: its tentative bindings and the exception are discarded, and selection continues. Control exceptions are different. Abort, termination and the enforced running-time limit always propagate out of `receive`, and the message being examined stays in the mailbox (Section 2.3.3). A guard that calls a fallible external service should translate failures into explicit results if the actor must distinguish them from an ordinary non-match. Unrestricted goal evaluation also carries operational risk: an expensive or nonterminating guard can delay message selection.

This discipline has precedents in the committed-choice logic-programming languages. Concurrent Prolog, Parlog, and Guarded Horn Clauses use guards to determine which process clause may commit, rather than as ordinary goals through which execution may later backtrack (Shapiro, 1987; Clark and Gregory, 1986; Ueda, 1986). Web Prolog does not adopt their execution model wholesale: ordinary nondeterministic Prolog remains available within an actor, including in the body selected by a receive clause. The narrower lesson adopted here is that selection itself is committed and that a rejected guard must leave no logical bindings behind. Requiring guards to be observationally free of side effects makes that discipline coherent without pretending that arbitrary external effects can be undone.

We return to this feature in Section 2.4.6, where guards that consult the actor's logical environment become the basis for event-driven state machines whose transitions depend on what the process can establish.

Box 2.3 — Erlang comparison: Selective receive and relational guards

Both languages scan a mailbox selectively: an older non-matching message remains available while a later matching message may be selected. Both refine patterns with guards, but Erlang's `when` guards are restricted tests on values already bound. In Web Prolog, `Pattern if Goal` evaluates a relational query in the actor's logical environment. The query can extend the pattern's bindings and pass them to the selected body.

The distinction concerns the guard's contract. Even if Erlang guards allowed arbitrary pure function calls, a Boolean result would still only accept or reject the message. Passing a derived value, such as a printer satisfying several conditions, to the body would require a convention for returning and binding that result. Web Prolog uses ordinary logical variables and conjunction to express both the conditions and their witnesses.

Web Prolog writes `if` to emphasise this role as a logical antecedent: proving the guard contributes bindings to the clause's computation. Selection nevertheless commits to the guard's first solution. Failure or an ordinary exception rejects the candidate and discards its tentative bindings, following Erlang's error-as-false discipline. Control exceptions propagate instead: in Web Prolog, abort, termination and time limits arrive as exceptions, whereas an Erlang guard never receives an exit signal in that form. Portable Web Prolog guards must also be observationally free of side effects.

Timeouts

The optional second argument of `receive/1-2` expects a list of options. The `timeout` option takes an integer or float specifying the number of seconds before a timeout occurs if no match has been found. The `on_timeout` option takes a goal that is called on timeout. If this option is omitted, its goal defaults to `true`, so the receive succeeds. An explicitly supplied timeout goal is ordinary Prolog code: its success, failure, exception, and any choice points determine the outcome of the receive on the timeout path. Here is an example of its use:

```
receive({
    hello(Name) ->
        format("Hello, ~s!~n", [Name])
},[ timeout(5),
    on_timeout(format("No match received in 5 seconds.~n"))
])
```

If no message of the form `hello(Name)` is received within 5 seconds, the code in the `on_timeout` option is executed.

An implementation of sleep/1

Here is how a predicate `sleep/1` that suspends execution `Time` seconds can be defined:

```
sleep(Time) :-
    receive({},[
        timeout(Time)
    ]).
```

The call to `receive/2` does not wait for messages, but when `Time` seconds have passed, it still succeeds.

An implementation of flush/0

As we noted earlier, being able to inspect the contents of the shell's mailbox during interactive programming is important, and `flush/0` is a nice tool for doing just that. Its definition also serves as yet another example of the use of the `timeout` option:

```
flush :-
    receive({
        Message ->
            format("Shell got ~q~n",[Message]),
            flush
    },[
        timeout(0)
    ]).
```

The value `0` of the `timeout` option ensures that the loop terminates immediately if no messages remain in the mailbox. This is how the hanging of the `receive/1` call is avoided.

A simple priority queue

To demonstrate the use of the `if` operator and the `timeout` and `on_timeout` options of `receive/2` – the latter of which causes a goal to run on timeout – we show a priority queue example borrowed from Fred Hébert's textbook on Erlang (Hébert, 2013). The purpose is to build a list of messages with those with a priority above 10 coming first:

```
important(Messages) :-
    receive({
        Priority-Message if Priority > 10 ->
            Messages = [Message|MoreMessages],
            important(MoreMessages)
```

```
    },[ timeout(0),
        on_timeout(normal(Messages))
    ]).

normal(Messages) :-
    receive({
        _-Message ->
            Messages = [Message|MoreMessages],
            normal(MoreMessages)
    },[ timeout(0),
        on_timeout(Messages=[])
    ]).
```

The timeout set to `0` means that it will occur immediately, but the system tries all messages currently in the mailbox first.

Below, we test this program by first sending four messages to the shell process, and then calling `important/1`:

```
?- self(S),
   S ! 15-high, S ! 7-low, S ! 1-low, S ! 17-high.
S = 34871244.
?- important(Messages).
Messages = [high,high,low,low].
?-
```

Note that the implementation of the priority-queue example relies on the deferring behaviour of `receive/1-2` and would not work without it.

One message, possibly several solutions

A call to `receive/1-2` is *single-message*, but not necessarily single-solution. During one invocation it selects and consumes at most one message: the oldest message accepted by any clause, using the first clause that accepts it. This selection is committed. Backtracking through the invocation never selects another message or another receive clause.

The number of messages consumed and the number of Prolog solutions are different matters. After committing to the message and clause, the receive calls the selected body as ordinary Prolog code. The body may fail, succeed once, throw an exception, or yield several solutions. Choice points left by the body enumerate further solutions of that same body; they do not inspect the mailbox again. On timeout no message is consumed, and the same principle applies to the `on_timeout` goal. The determinism declaration `nondet` therefore concerns the possible number of logical solutions, not the number of reception events performed by one invocation.

To see how it pans out in a simple corner case, consider the following two calls:

```
receive({foo(X) -> true})        receive({foo(X) -> fail})
```

The call on the left will succeed if a message matching the pattern `foo(X)` appears in the mailbox of the actor process executing the call, a term such as `foo(314)` for example. The call on the right will fail (and possibly cause backtracking) once `foo(314)` appears. Only by the first call will the variable `X` be bound (to `314`). In both cases the message is removed, and in both cases a non-matching message is deferred and remains in the mailbox.

A nondeterministic body makes the distinction explicit. Assuming the mailbox initially contains one `foo(314)` message, the following receive has two solutions:

```
?- receive({foo(X) -> member(Y, [a,b])}).
X = 314,
Y = a ;
X = 314,
Y = b.
?-
```

Both solutions come from the body selected for the same consumed message. The message is consumed only once. Producing the second solution does not inspect the mailbox again, and asking for still more solutions does not make the receive try another message or clause. In short, `receive/1-2` is single-message, not single-solution.

Send and receive are effects on a mailbox. Like `assert/1` and `retract/1`, they are not undone on backtracking. A message sent by `!/2` remains in its target mailbox even if the sending goal is later backtracked into. A message matched by a receive is removed whether the clause body then succeeds or fails, and is not restored by backtracking past the call. Consequently, a goal re-entered through backtracking does not see the mailbox as it was on first entry – it sees the current mailbox, reflecting every send and consumption (including those of other actors) that has occurred in the meantime. This is the same discipline Prolog programmers already accept for the dynamic database, now extended to the mailbox.

To implement a looping behaviour, Prolog programmers occasionally use a *failure-driven* loop that relies on backtracking rather than recursion. Using this technique, our echo server can be rewritten like so:

```
echo_actor  :-
    repeat,
    receive({
        echo(From, Msg) ->
            From ! echo(Msg),
            fail
    }).
```

For an Erlang programmer, this use of `receive/1` may come as a surprise and is not a technique that can be used in Erlang. In this particular case, a Web Prolog programmer would be advised to stick to the recursive version. However, there are cases when a failure-driven loop is the only way forward and we shall look at an important such case towards the end of this chapter.

2.4 Erlang-style programming examples in Web Prolog

> Reading the code was fun – I had to do a double take – was I reading Erlang or Prolog – they often look pretty much the same.
>
> *Joe Armstrong* (p.c. June 18, 2018)

Not only do Web Prolog programs *look* a lot like Erlang, they *behave* a lot like Erlang too. A good way to demonstrate this is to translate a fair number of different Erlang programming examples borrowed from tutorials and text books into Web Prolog and show that they run just like in Erlang. In this section we shall look at a count server, a fridge simulation, a technique for hiding message protocols behind predicate APIs, delayed sending, actors playing ping-pong, event-driven state machines whose transitions consult the knowledge base, and – most importantly – a demonstration that Prolog's backtracking search can be placed under external, message-driven control. They are all concurrent programs, but only locally so. Distributed programming is dealt with in Chapter 4.

2.4.1 A count server

We have already looked at an example of a *stateless* server, namely the echo server presented earlier in this chapter. Let us now turn to *stateful* servers and show how state may be programmed. In the following example, we demonstrate how to script an actor that can keep a *count*:

```
count_actor(Count0) :-
    receive({
        count(From) ->
            Count is Count0 + 1,
            From ! count(Count),
            count_actor(Count) ;
        stop ->
            true
    }).
```

The predicate `count_actor/1` defines a loop where a call to the built-in predicate `receive/1` tries to select a message of the form `count(From)` from the mailbox, increment the counter, send the current count back to the actor referenced by the pid to which the variable `From` is bound, and continue looping by making a recursive call. Note how the state of the counter – the current count, that is – is kept in the argument of the predicate. Note also that we added a second receive clause that will allow us to terminate the server without using `exit/2`. We just have to send it a message of the form `stop`.

Here is how an actor following this script can be made to perform when spawning it with the current count initialised to `0`:

```
?- spawn(count_actor(0), Pid, [
       monitor(true)
   ]).
Pid = 62841905.
?-
```

In the next step the send operator `!/2` is used for sending a message to the spawned actor, instructing it to increment the count and to return the result in the form of a message:

```
?- self(Self), $Pid ! count(Self).
Self = 41167597.
?-
```

A call to `receive/1` can be made in order to collect the message arriving from the actor:

```
?- receive({Count -> true}).
Count = count(1).
?-
```

Here, `!/2` is used to send two messages to the actor that will end up in its mailbox, in the order they were sent:

```
?- $Pid ! count($Self), $Pid ! stop.
true.
?-
```

Finally, `flush/0` is used to inspect the contents of the mailbox belonging to the toplevel process:

```
?- flush.
Shell got count(2)
Shell got down(62841905,62841905,true)
true.
?-
```

Because the actor was monitored, a `down` message was found in addition to the current count.

2.4.2 A bigger, tastier example

As a tastier example of how a process can be made to hold an updatable state during a conversation we have adapted a fridge simulation example from Fred Hébert's introduction to Erlang (Hébert, 2013):[13]

[13] `http://learnyousomeerlang.com/more-on-multiprocessing#state-your-state`

```
fridge(FoodList0) :-
    receive({
        store(From, Food) ->
            self(Self),
            From ! Self-ok,
            fridge([Food|FoodList0]) ;
        take(From, Food) ->
            self(Self),
            (   select(Food, FoodList0, FoodList)
            ->  From ! Self-ok(Food),
                fridge(FoodList)
            ;   From ! Self-not_found,
                fridge(FoodList0)
            ) ;
        terminate ->
            true
    }).
```

The program creates a process allowing three operations: storing food in the fridge, taking food from the fridge, and terminating the fridge. It is only possible to take food that has been stored beforehand. Again, with the help of recursion the state of a process can be held entirely in the argument of the predicate. In this case we choose to store all the food as a list, and then look in that list when someone needs something.

Assuming the above program is already available, the following session creates the server process. We can then call `self/1`, `!/2` and `flush/0` from the shell in order to simulate the actions of a client:

```
?- spawn(fridge([]), Pid, [
       monitor(true)
   ]).
Pid = 77346122.
?- self(Me), $Pid ! store(Me, meat), $Pid ! store(Me, cheese).
Me = 97216744.
?- flush.
Shell got 77346122-ok
Shell got 77346122-ok
true.
?- self(Me), $Pid ! take(Me, cheese).
Me = 97216744.
?- flush.
Shell got 77346122-ok(cheese)
true.
?- $Pid ! terminate.
true.
?- flush.
```

```
Shell got down(77346122,77346122,true)
true.
?-
```

2.4.3 Hiding the details of protocols

In the previous example, we expected programmers to know the details of the protocol that must be followed when interacting with our fridge simulation, which forced them to make raw calls using the send operator in combination with the `flush/0` utility predicate. As stated by Fred Hébert in his book, that is often a useless burden, and a good way around it is to abstract messages away with the help of predicates (or in Hébert's case, functions) dealing with receiving and sending them:

```
store(Pid, Food, Response) :-
    self(Self),
    Pid ! store(Self, Food),
    receive({
        Pid-Reply -> Response = Reply
    }).

take(Pid, Food, Response) :-
    self(Self),
    Pid ! take(Self, Food),
    receive({
        Pid-Reply -> Response = Reply
    }).
```

Calling `receive/1` immediately after having sent the message guarantees that the communication between client and server stays synchronous. Using `store/3` and `take/3`, the interaction with the fridge becomes somewhat easier:

```
?- spawn(fridge([]), Pid, [
       monitor(true)
   ]).
Pid = 55289322.
?- store($Pid, cheese, Response).
Response = ok.
?- take($Pid, cheese, Response).
Response = ok(cheese).
?-
```

When dealing with actors with more complex protocols, such abstractions turn out to be of considerable value.

2.4.4 Using delayed sending

Appendix B documents a built-in predicate `send/2-3` that allows a process to delay the sending of a message, but also to cancel the sending if needed, by calling `cancel/1`. This can be used to implement an alarm that can be cancelled, like so:

```
alarm :-
    receive({
        ring ->
            writeln('Alarm ringing!'),
            alarm;
        stop ->
            true
    }).
```

Here we schedule the alarm to ring in 5 seconds, with an `id` set to `alarm1`:

```
?- spawn(alarm, Pid),
   send(Pid, ring, [
       delay(5),
       id(alarm1)
   ]).
Pid = 60034123.
?-
```

Then we cancel the alarm after a couple of seconds:

```
?- cancel(alarm1).
true.
?-
```

Cancellation is scoped to the calling actor: `cancel(alarm1)` tries to withdraw all delayed sends with that identifier that were scheduled by this actor, without affecting another actor that happened to choose the same identifier. If cancellation succeeds, the message is withdrawn and the alarm never rings. If `cancel/1` is called too late, the alarm message has already been delivered and "Alarm ringing!" will be printed.

2.4.5 Prolog actors playing ping-pong

Since the servers in the previous sections are running in parallel to the shell and are talking to it using asynchronous messaging, we have already demonstrated the use of concurrency. Below, in a probably more convincing example inspired by a user's guide to Erlang,[14] two processes are first created and then start sending messages to each other a specified number of times:

[14] See `http://erlang.org/doc/getting_started/conc_prog.html`

```
ping(0, Pong_Pid) :-
    Pong_Pid ! finished,
    format('Ping finished.~n').
ping(N, Pong_Pid) :-
    self(Self),
    Pong_Pid ! ping(Self),
    receive({
        pong ->
            format('Ping received pong.~n')
    }),
    N1 is N - 1,
    ping(N1, Pong_Pid).

pong :-
    receive({
        finished ->
            format('Pong finished.~n');
        ping(Ping_Pid) ->
            format('Pong received ping.~n'),
            Ping_Pid ! pong,
            pong
    }).

ping_pong :-
    spawn(pong, Pong_Pid),
    spawn(ping(3, Pong_Pid)).
```

When `ping_pong/0` is called the behaviour of this program exactly mirrors the behaviour of the original version in Erlang:

```
?- ping_pong.
true.
Pong received ping.
Ping received pong.
Pong received ping.
Ping received pong.
Pong received ping.
Ping received pong.
Ping finished.
Pong finished.
?-
```

The received-message lines alternate because pong prints before replying, and ping prints before sending the next ping. Figure 2.4 illustrates this exchange. The actors execute concurrently, but each round waits for the preceding reply. The two finishing lines may appear in either order, and the shell may report `true` before the conversation ends.

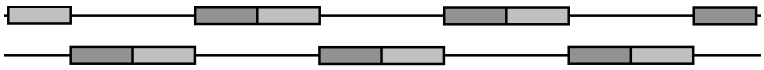


Fig. 2.4 A timeline where the upper line represents the "pinger" and the lower line the "ponger," and where each grey bar represents a task consisting of the sending of a message (in light grey) and the reception of the response (in darker grey).

2.4.6 Event-driven state machines

Guards make it easy to write event-driven state machines whose transitions consult the actor's logical environment. Clause selection can depend on any relational condition, and the bindings it produces are available to the transition's action.

Web Prolog's `receive` primitive can implement such machines directly. Message patterns represent events, guards represent transition conditions, and clause bodies implement actions and state updates. Figure 2.5 depicts a machine with two states and four transitions.

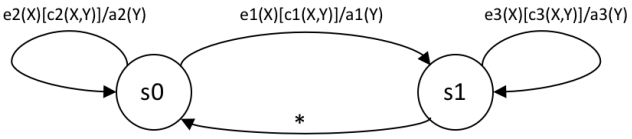


Fig. 2.5 A simple event-driven state machine. `s0` is the initial state.

Using predicate names to encode states, one possible implementation is:

```
s0 :-
    receive({
       e1(X) if c1(X,Y) ->
           a1(Y),
           s1 ;
       e2(X) if c2(X,Y) ->
           a2(Y),
           s0
    }).
```

```
s1 :-
    receive({
       e3(X) if c3(X,Y) ->
           a3(Y),
           s1 ;
       _AnyEvent  ->
           s0
    }).
```

In state `s0` the machine waits for an event message of the form `e1(X)` or `e2(X)` in the current process's mailbox. If `e1(X)` matches and `c1(X,Y)` succeeds, the action `a1(Y)` is executed and the machine transitions to `s1`. If `e2(X)` matches and `c2(X,Y)` succeeds, the action `a2(Y)` is executed and the machine remains in `s0`. The other state is analogous: `e3(X)` triggers a guarded self-loop in `s1`, while any other event falls back to `s0`.[15]

[15] Note that `s0` defers unknown events while `s1` discards them; the designer must choose deliberately.

The variables `Y` illustrate the guard semantics described in Section 2.3.4: event matching binds `X`, while the transition condition may derive `Y` and make it available to the action. What this example adds is a control pattern: receive clauses can encode states, events, transition conditions and actions directly. Chapter 6 develops SXML, a specialised statechart notation within Web Prolog, for expressing the same style of reactive programming more structurally.[16]

2.4.7 Getting answers through backtracking

In Section 2.2.2 we asked how Prolog's backtracking would fit with Erlang-style messaging. The following example brings the two together: a goal can produce many solutions, while `receive` consumes one message at a time. Can incoming messages control the search, asking for another solution or stopping it?

Consider a spawned goal that enumerates the solutions to `human(Who)`, but instead of collecting them all at once, sends the *first* solution back to its parent and then waits. It will produce the next solution only when told to, by a `next` message, and will stop on `stop`:

```
?- self(Self),
   spawn(( human(Who),
           Self ! Who,
           receive({
               next -> fail ;
               stop -> true
           })
         ), Pid).
Pid = 76123351,
Self = 90054377.
?- flush.
Shell got socrates
true.
?- $Pid ! next.
true.
?- flush.
Shell got plato
true.
?- $Pid ! stop.
true.
?-
```

[16] Event-driven state machines have historically been less prominent in the Prolog world, most likely because message-passing concurrency arrived relatively late and unevenly across Prolog systems, gaining wider traction only with facilities such as those in the ISO Prolog Threads draft. By contrast, Erlang/OTP treats this programming model as foundational, as exemplified by `gen_statem`.

The spawned goal solves `human(Who)`, binding `Who = socrates`, sends that solution to the parent's mailbox with `Self ! Who`, and then blocks in `receive`. So far this is ordinary sequential Prolog with one message send. The `next` clause has body `fail`, so receiving `next` makes the `receive` *fail* – and a failure here propagates backwards into `human(Who)`, forcing it to backtrack and produce its next solution, `plato`. Execution then runs forward again through `Self ! Who`, sending `plato`, and blocks once more. Each `next` thus resumes the search for another solution; `stop`, whose body is `true`, lets the goal succeed and the actor terminate.

This works because message selection is committed and message operations are not undone on backtracking (Section 2.3). When the goal backtracks from `socrates`'s `receive` back into `human(Who)`, the send that already delivered `socrates` is *not* retracted, and `plato` is delivered as an *additional* message.

Messages now place Prolog's search under external control: each `next` requests another solution, much as pressing semicolon does at an interactive toplevel. This small actor is the starting point for the *toplevel behaviour* developed below.

The goal is still hard-coded, however, and the actor neither reports failure nor handles exceptions. The next section extends the sketch so that a client can submit goals and receive explicit information about their outcomes.

2.4.8 The toplevel behaviour: a preliminary sketch

In essence, a Prolog toplevel is a failure-driven loop running inside another loop – recursive or failure driven. The failure-driven inner loop allows a client to ask for one solution at a time to a goal, while the outer loop allows it to call several goals in a row and run each of them to completion.

Below, we show how we can build a simple Prolog toplevel by using a meta-predicate (such as `call_cleanup/2`) and by specifying a small set of custom messages carrying answers and/or the state of the process that needs to be returned to the calling process. The predicate `call_cleanup/2` is here used not only to call a goal, but also to check if any choice points remain after the goal has been called or backtracked into.[17]

```
simple_toplevel(Pid) :-
    simple_toplevel(Pid, []).

simple_toplevel(Pid, Options) :-
    self(Self),
    spawn(session(Pid, Self), Pid, Options).

session(Pid, Parent) :-
    receive({
        '$call'(Template, Goal) ->
```

[17] `http://www.swi-prolog.org/pldoc/man?predicate=call_cleanup/2`

```
            (   call_cleanup(Goal, Det=true),
                (   var(Det)
                ->  Parent ! success(Pid, Template, true),
                    receive({
                        '$next' -> fail ;
                        '$stop' -> true
                    }),
                    !
                ;   Parent ! success(Pid, Template, false)
                )
            ;   Parent ! failure(Pid)
            )
    }),
    session(Pid, Parent).
```

In `spawn(session(Pid, Self), Pid, Options)`, `spawn/3` binds the child's pid before copying the goal, so `session/2` receives that pid as a value.

A suitable predicate API hiding the details of the protocol from the programmer can be written like so:

```
simple_toplevel_call(Pid, Template, Goal) :-
    Pid ! '$call'(Template, Goal).

simple_toplevel_next(Pid) :-
    Pid ! '$next'.

simple_toplevel_stop(Pid) :-
    Pid ! '$stop'.
```

Here is a session that shows that these predicates work as intended:

```
?- simple_toplevel(Pid, [
       monitor(true)
   ]).
Pid = 23891373.
?- simple_toplevel_call($Pid, Who, human(Who)).
true.
?- flush.
Shell got success(23891373,socrates,true)
true.
?- simple_toplevel_next($Pid).
true.
?- flush.
Shell got success(23891373,plato,true)
true.
?- simple_toplevel_next($Pid).
true.
```

```
?- flush.
Shell got success(23891373,aristotle,false)
true.
?-
```

Note that because of the outer recursive loop, our simple toplevel actor is again poised to run another query:

```
?- simple_toplevel_call($Pid, X, q(X)).
true.
?- flush.
Shell got down(23891373,23891373,
               exception(error(
                   existence_error(procedure,q/1),_)))
true.
?-
```

Here the actor terminated because the code for `simple_toplevel/1-2` does not say what should happen when the called goal throws an exception, as an undefined predicate does. Since the spawned process is monitored, a termination notification containing the exception classification nevertheless reached the spawning process in the form of a `down` message. The call to `flush/0` is displaying the boundary-sanitised Prolog term found in the shell's mailbox. Note that the actor runtime elides the implementation-supplied context when it forms the monitor notification. The underscore in this transcript denotes that elided, unbound argument.

A similar problem is evident should we tell the actor to execute a query that does not terminate. Using `exit/2` works, but will once again kill the entire actor process.

Chapter 3 introduces built-in toplevel predicates that address these limitations through a standardised protocol. These provide the reusable Erlang-style behaviour anticipated by this sketch.

2.5 Summary

This chapter introduced Web Prolog as a hybrid language that extends a substantial subset of ISO Prolog with Erlang-style constructs for programming concurrency: actors with private state, asynchronous message passing (`!/2` and `receive/1-2`), process creation (`spawn/1-3`), and fault-handling primitives such as links, monitors, and exit signals. A program that never mentions these constructs behaves like a conventional Prolog program, while programs that do use them gain access to an Erlang-like world of concurrent processes.

We also argued that Erlang provides an unusually suitable point of departure for such an extension. Its actor model is mature and close enough in syntax and spirit to Prolog that both languages can share patterns, idioms, and even didactic material. Many of the examples in this chapter are thin Prolog translations of familiar

Erlang patterns. The combination of Prolog's backtracking search with Erlang-style concurrency is made workable by committing message and receive-clause selection without suppressing the ordinary Prolog control behaviour of the selected body. Backtracking may explore that body, but it neither selects another message nor undoes mailbox effects. This makes it possible to treat a Prolog toplevel as just another actor and to encapsulate interactive sessions as behaviours implemented on top of a common concurrency substrate.

Taken together, these choices define Web Prolog as a multi-paradigm language. Its logical expressivity remains that of ISO Prolog, but its operational repertoire is enlarged by an actor-based concurrency model that is compatible with the rest of the Prolog Trinity ecosystem. There are other possible ways of adding concurrency to Prolog, but the thesis of this chapter is simply that Erlang-style actors provide a coherent and practically manageable foundation on which the subsequent chapters can build.

Box 2.4 — Erlang comparison: What Web Prolog borrows and changes

The table collects the differences from Erlang introduced in this chapter.

Erlang	Web Prolog
Normal return reports `normal`	Outermost success and failure report `true` and `false`; uncaught exceptions are distinct (Box 2.1).
Symmetric links; ordinary spawn is unlinked	Directional parent–child dependence, enabled by default (Box 2.2).
Restricted `when` guards	Relational `if` guards can extend the pattern's bindings (Box 2.3).
Per-pair signal ordering	Message FIFO is limited to a stable route; mailbox messages and runtime signals have no cross-channel ordering (Section 2.3.3).
Exit signals can be trapped	Containment termination cannot be refused; monitors provide observation (Section 2.3.3).

The main reasons for these changes are Prolog's logical execution model and ownership across an open Web.

Chapter 3
Prolog agents

> Imagine a world of **Prolog agents**, some useful, others playful, bringing joy to games and virtual worlds; some short-lived, others long-lived, some simple, others complex – but maybe built from simpler ones. Written in Web Prolog, talking Web Prolog with other agents, using Web Prolog knowledge bases to guide their actions and conversations, making sure important abilities of clever conversational agents, such as natural language understanding, knowledge representation, reasoning and real-time interaction, are accounted for.
>
> *Prolog agents – the elevator pitch*

Chapter 1 introduced Prolog agents in outline, while Chapter 2 developed the Prolog actor as Web Prolog's elementary unit of concurrent computation. This chapter completes the first-level picture by treating actors and nodes as the two principal kinds of Prolog agent and by distinguishing the main actor behaviours used in the book.

3.1 Prolog agents in a nutshell

We treat actors and nodes as two kinds of *Prolog agent*. A one-line echo actor that lives for a fraction of a second and a node that serves thousands of clients for months differ enormously, but we meet both in the same way: we send them goals or messages and receive answers, messages or effects. As in Chapter 1, "agent" names this role and nothing more. A node need not even be written in Prolog, as long as it can talk it.[1]

Figure 3.1 shows the taxonomy. It expands the actor branch; nodes are organised into profiles in Section 3.5.

Actors and nodes. These are the two principal architectural kinds. Actors are independently scheduled loci of concurrent computation, equipped with pids and mailboxes. Nodes operate at a larger scale: they host computation and knowledge, expose web APIs, route messages, and enforce owner policy.

[1] In treating "agent" as an analytical lens rather than an absolute classification, we follow Russell and Norvig (2016).

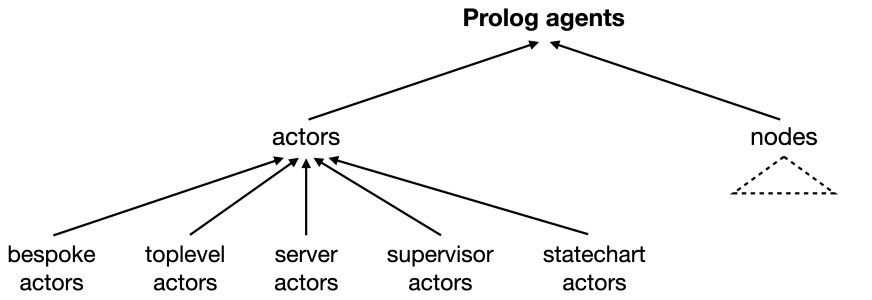


Fig. 3.1 A first-level taxonomy of Prolog agents. Arrows point towards the more general kind. The actor branch distinguishes bespoke actors from actors governed by four principal reusable behaviours used in this book. The dashed node branch announces a further division into four cumulative sub-profiles, whose names and requirements are deferred to Section 3.5.

Bespoke actors. A bespoke actor is programmed directly for a particular task. Its initial goal and, where it is interactive, its receive loop encode the application's own messages, state changes, and stopping conditions. The echo actors, the actors that play ping–pong, and the hand-written count actor are simple examples. "Bespoke" does not imply short-lived or disposable: the same style can be used for a durable, named, node-resident service. It means that the application author supplies the control loop rather than instantiating one of the reusable behaviours shown alongside it in the figure.

Generic actors and behaviours. The remaining four actor branches name generic forms, more precisely actors governed by reusable *behaviours*. A behaviour factors recurring protocol and lifecycle machinery out of the application:

- A *toplevel actor* implements the Prolog Toplevel Communication Protocol (PTCP), retains a query's suspended search, and delivers solutions incrementally.
- A *server actor* provides correlated request–response handling and threads private state from one request to the next; an application supplies the callback that interprets requests.
- A *supervisor actor* starts, monitors, terminates, and, according to policy, restarts children; an application supplies the child specifications and restart strategy.
- A *statechart actor* provides event selection and run-to-completion control; an application supplies the SXML statechart, including its states, transitions, guards, and actions.

Thus a generic actor is still a concrete running actor with a pid and a mailbox. What is generic is the control machinery that governs it.

This list is not exhaustive. Other reusable actor behaviours could be defined – for example, an *event-manager* behaviour analogous to Erlang/OTP's `gen_event`, dispatching events to dynamically installed handlers. Nor is the design space limited to patterns inherited from OTP. The toplevel behaviour already demonstrates how

a reusable actor behaviour can package capabilities distinctive to Prolog; further Prolog-specific forms of search and interaction may be worth exploring.

Application-specific instances of generic behaviours. Specialising a behaviour produces an application-specific actor. A shell is a user-facing arrangement built around a toplevel actor, with terminal-oriented facilities supplied by the node controller. An expert-system agent can be realised as a toplevel actor, made distinctive by the inference engine and domain knowledge loaded into it. A particular callback turns the server behaviour into a particular service, and particular child specifications turn the supervisor behaviour into a concrete supervision tree. Statechart actors can likewise be specialised as agent harnesses, service-request agents, or dialogue managers. The figure therefore classifies actors primarily by where their control machinery comes from, not by every application role they may eventually play.

Nodes are classified into profiles in Section 3.5.

3.2 More about Prolog actor agents

Actor agents are the workhorses of the concurrent strand of the Trinity: they execute Prolog code concurrently, interact by exchanging Prolog terms, and may maintain actor-local state across repeated interactions. This section makes that operational picture more concrete. We first introduce the actor's private Prolog database and the `src_*` options used to initialise it. We then sketch the lifecycle of a typical actor and explain how receive loops realise explicit communication protocols. Finally, we discuss the dynamic (still private) database as a non-logical but sometimes convenient mechanism for actor-local updates.

3.2.1 An actor agent is equipped with a private Prolog database

When running the examples in Chapter 2 we were somewhat vague about the origin of the code being executed. The only clue provided was that it might reside in the node's shared database, which defines read-only data and programs accessible to any actor living on that node. Here we turn to the other storage area – the one located *within* the actor itself and where programs and data exclusive to that actor are stored. While all actors have a private database, it starts out empty by default.

During the spawn operation, the (soon-to-be) parent process can initialise the private database of the (soon-to-be) child actor with clauses defining one or more predicates. Consider the following example where the `src_text` option is passed to `spawn/3`, specifying that the source code for our count server should be loaded into the actor's private database before calling the goal in the first argument:

```
?- spawn(count_actor(0), Pid, [
       src_text("
```

```
            count_actor(Count0) :-
                receive({
                    count(From) ->
                        Count is Count0 + 1,
                        From ! count(Count),
                        count_actor(Count) ;
                    stop ->
                        true
                }).
        "),
        monitor(true)
    ]).
Pid = 45092311.
?-
```

In addition to the `src_text` option, three other options can be used to populate the private database of an actor. The `src_list` option loads a list of clauses or directives, the `src_uri` option loads the content specified by a URI, and `src_predicates` takes a list of predicate indicators and loads the clauses for the indicated predicates that are accessible by the caller.

The clauses in an actor's private database represent its unique beliefs and skills, distinguishing them from the shared beliefs and skills accessible to other actor agents on the same node. In this sense, the private database can be seen as the core of the actor's individuality, encapsulating the knowledge and skills that are exclusive to it.

The `src_*` options make a kind of sharing of beliefs and skills between a parent process and a child process possible. The child is "born" with them, they are "innate." It is up to the parent actor to determine exactly what should be passed on to the child actor. We can think of this as "learning by injection."

3.2.2 The lifecycle of a typical Prolog actor agent

> We have the black boxes that do things, and we have the messages in between. What goes on inside the black boxes does not really matter; what takes place there is abstracted over.
>
> *Joe Armstrong*

The diagram in Figure 3.2 illustrates the lifecycle of a typical Prolog actor.

When an actor is spawned, it first passes through an initialisation phase where it is configured in a way that will suit its intended purpose. This includes specifying values for options such as `monitor`, `link` as well as the `src_*` options.

For example, when the count actor in the previous example was spawned, the source code specified by the `src_text` option was loaded into the actor's private Prolog database and the `monitor` option instructed the runtime to deliver a `down` message to the shell when the actor terminated. The call to `spawn/3` succeeded, binding the variable in the second argument to a pid. By this, the initialisation phase

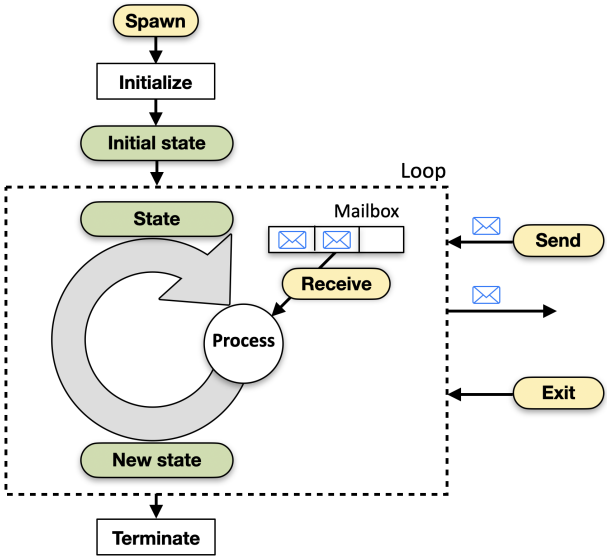


Fig. 3.2 Lifecycle of a Prolog actor: spawn, interact, terminate, and clean up

is completed, the actor is now in its *initial state*, and the loop that implements the actor's *communication protocol* is entered. Typically, processing stops at a call to `receive/1-2` where it blocks, waiting for a `count` message or a `stop` message to appear in its mailbox.

Communication protocols are central to Erlang-style concurrent programming. The actor's receive loop implements a protocol that clients must follow: it determines which messages the actor accepts and how it responds. The protocol governs three aspects of their interaction:

- **Syntax**: The structure and format of messages, including the organisation of their data elements.
- **Semantics**: What each message means, which actions it requests, and how errors or unexpected messages are handled.
- **Timing**: When messages may be sent, in what order, and how long either process waits for a response. These rules let clients and actors coordinate despite delays, reordering, and failures.

Web Prolog's `receive/1-2` supports these protocols through selective receive and timeouts. Messages that do not match the current receive clauses remain in the mailbox for possible later processing. A timeout lets the actor take an alternative action if no matching message arrives within the specified interval.

In a typical receive loop, the actor processes one selected message at a time. Its response may depend on its current state, and processing the message may produce a new state for the next iteration.

Some actors, such as the echo actor, are stateless. Others carry state in the arguments of their receive loop: an integer in the count actor, for example, or a list of atoms in the fridge simulation. State can also be represented by the explicit states and transitions of a state machine, or implicitly through the choice points of a failure-driven loop. Web Prolog additionally allows an actor to maintain state by updating its private dynamic Prolog database, although such updates are non-logical and are not undone on backtracking.

3.2.3 The dynamic (and still private) Prolog database

Each actor owns a private Prolog database that exists for the lifetime of the actor. While the actor remains alive, it may update this database using predicates such as `assert/1`, `retract/1`, and `retractall/1`. In accordance with the ISO standard, predicates modified in this way must be declared using the `dynamic/1` directive.

Such updates are strictly actor-local. A clause asserted by one actor is added only to that actor's private database and is not visible to the shell, to other actors on the same node, or to actors elsewhere on the Prolog Web. Dynamically asserted clauses also share the lifetime of the actor: when the actor terminates, its private database and any changes made to it disappear.

For example, assuming that `foo/1` has been declared dynamic, the following goal is permitted:

```
?- spawn(assert(foo(a)), Pid).
Pid = 32861299.
?-
```

The spawned actor asserts the clause `foo(a)` in its own database and then terminates. The clause is never added to the database of the shell and disappears together with the actor that asserted it. Although this particular use of `spawn/2` has little practical value, it illustrates both the ownership and the lifetime of dynamically asserted clauses.

The dynamic database gives an actor a direct way to represent evolving state without threading an explicit state term through the arguments of its predicates. An actor may, for example, retract clauses describing its current state and assert clauses describing a new state before processing the next message. This can occasionally be convenient, but it should be used judiciously.

Private mutation is not shared mutation

The familiar objections to the Prolog dynamic database still apply. Database updates are not undone on backtracking, and the meaning of a dynamic predicate may therefore depend on the operational history of the actor rather than solely on the

arguments of a particular call. Dynamic predicates consequently introduce hidden mutable state and may make programs more difficult to understand, test, and debug.

Their actor-local scope nevertheless changes the concurrency picture considerably. Two actors cannot race to update the same private dynamic predicate, because each actor owns a separate database. For the same reason, `assert/1` and `retract/1` cannot be used as an implicit mechanism for inter-process communication. Private updates are still non-logical, but they cannot cause races between actors.

Box 3.1 — Erlang comparison: The process dictionary and the private Prolog database

Every Erlang process has a private process dictionary, a key–value store inaccessible directly to other processes. Web Prolog's private dynamic database has the same decisive concurrency property: mutable state belongs to one actor, so private mutation cannot become an implicit shared communication channel.

The analogy should not be pushed too far. A process dictionary maps keys to values. A Prolog database stores clauses and facts, may shadow shared definitions, and can be queried relationally. `assert/1` and `retract/1` consequently update part of an actor's executable logical environment. Both facilities should be used sparingly: actor-locality prevents shared-state races, but it does not make hidden mutable state logical or easy to understand.

The restriction to actor-local databases does not rule out explicit forms of shared storage. Web Prolog could, for example, provide access to a transactional RDF store or to a Linda-style blackboard. Such facilities would, however, constitute separate abstractions with their own policies for access, lifetime, consistency, and concurrency; they would not turn the private databases of individual actors into shared state.

For the time being, Web Prolog restricts the ordinary dynamic database to the private scope of individual actors.

3.2.4 Private beliefs and shared knowledge – shadowing

A predicate defined in an actor's private database takes precedence over a predicate with the same name and arity in the shared database. Thus, for an unqualified call made from the actor's private database, Prolog first looks for a private definition and otherwise continues to the shared database. We say that the private predicate *shadows* the shared predicate.

Suppose the shared database contains the following price list and default pricing rule:

```
list_price(widget, 100).
```

```
list_price(gadget, 250).
list_price(gizmo,  400).

price(Item, Price) :-
    list_price(Item, Price).
```

An actor without a private definition of `price/2` uses the shared rule:

```
?- price(widget, Price).
Price = 100.
?-
```

We can give a newly spawned actor a private pricing policy by passing it with the `src_text` option. The actor uses the private `price/2`, but its call to `list_price/2` still resolves to the shared facts:

```
?- self(Self),
   spawn(
       ( price(widget, Price),
         Self ! discounted_price(Price)
       ),
       _,
       [ src_text("
             price(Item, Price) :-
                 list_price(Item, ListPrice),
                 Price is ListPrice * 0.9.
         ")
       ]
   ),
   receive({discounted_price(Price) -> true}).
Self = 34001234,
Price = 90.0.
?-
```

The private pricing rule belongs only to the spawned actor. It neither modifies the shared rule nor becomes part of the parent's private database. Consequently, another actor spawned by the parent without that `src_text` option uses the shared definition:

```
?- self(Self),
   spawn(
       ( price(widget, Price),
         Self ! ordinary_price(Price)
       ),
       _
   ),
   receive({ordinary_price(Price) -> true}).
Self = 34001234,
```

```
Price = 100.
?-
```

Shadowing applies to a complete predicate, not to individual clauses. The following actor receives a private `list_price/2` containing only one fact:

```
?- self(Self),
   spawn(
       ( findall(Item-Price,
                 list_price(Item, Price),
                 Prices),
         Self ! private_prices(Prices)
       ),
       _,
       [src_text("list_price(widget, 80).")]
   ),
   receive({private_prices(Prices) -> true}).
Self = 34001234,
Prices = [widget-80].
?-
```

The private fact does not combine with the shared facts. Once the actor has defined its own `list_price/2`, all shared clauses of `list_price/2` are hidden from unqualified calls made from that actor's private database. In particular, the actor does not obtain prices for `gadget` or `gizmo`.

The shared database itself remains unchanged. The parent still sees the complete shared price list:

```
?- findall(Item-Price,
           list_price(Item, Price),
           Prices).
Prices = [widget-100, gadget-250, gizmo-400].
?-
```

Shadowing is therefore local to an actor: it changes how that actor resolves a predicate without modifying the node-provided knowledge seen by other actors.

Shadowing is also local to the *caller*. An unqualified call made from inside an owner-defined predicate – one in the shared database, or in anything the shared database loads – resolves in the shared database, even when the calling actor has defined a predicate of the same name. A client's definition shadows a name only for calls the client's own code makes, so it cannot change what an owner predicate computes. The one way a client reaches back into an owner predicate is a goal the client passes as an argument, such as the goal given to `findall/3`; that goal is the client's code and runs in the client's context, as it would anywhere else.

3.3 Prolog shells and other toplevel actors

When a programmer starts a traditional Prolog system, they are usually presented with an interactive query prompt, commonly called the *toplevel*. In this book, we use *shell* for the actor that provides this terminal interaction, and *toplevel* for the underlying query service, which can also be controlled by another actor or an external client. In this chapter we use each behaviour through its interface. Chapter 5 shows how they are built from the actor primitives of Chapter 2.

As we saw already in Chapter 2, calling `self/1` in a terminal binds its argument to the pid of the currently running shell process:

```
?- self(Pid).
Pid = 72097632.
?-
```

Shells handle all the things that toplevels handle, but also add a number of useful things for when we are talking to Prolog over a terminal. This includes I/O (read and write) and utilities such as `flush/0` and the dollar notation. Such features are supplied by the *node controller* rather than a toplevel alone.

Here is how we instruct the shell to spawn a new toplevel:

```
?- toplevel_spawn(Pid).
Pid = 16226587.
?-
```

Note that `16226587` is a toplevel actor but not a shell. As with every other Prolog actor, it comes with a mailbox, a private Prolog database, the ability to send messages to other actors, as well as the ability to create other actors.

3.3.1 A Prolog toplevel is an actor with a built-in protocol

A Web Prolog toplevel actor encapsulates an independent Prolog session. A program can create one, submit queries to it, request further solutions, and end the interaction. Its clients may be other Web Prolog actors, JavaScript programs, or any other clients capable of speaking its protocol.

Solutions are retrieved incrementally. The client requests a solution or a slice of solutions and, if more remain, may request another. This gives programs control over the progression of a query, much as a programmer controls backtracking at the shell.

The toplevel's behaviour is defined by the Prolog Toplevel Communication Protocol (PTCP). It specifies the requests a client may make, the answers the toplevel may return, and the permitted order of these exchanges. The interaction also includes output, requests for input, failure and error reports, and control over the current query and the session.

The protocol follows the familiar interaction at a Prolog prompt: submit a query, inspect a solution, request another by typing a semicolon, or finish with the current

query. Each move changes what may happen next. The next section makes these rules explicit using a state machine.

3.3.2 The Prolog Toplevel Communication Protocol

The diagram in Figure 3.3 is a statechart specifying the Prolog Toplevel Communication Protocol (PTCP) – a protocol for the communication between a client and a toplevel actor. The client can be any process (including another actor or (say) a JavaScript process) capable of sending the messages and signals in bold to the toplevel. The toplevel actor is responsible for returning the messages with a leading / back to the client. The use of a statechart allows us to show that no matter the current state of the protocol, **abort** will always take it to the state from which a new goal can be called and **halt** will always terminate the toplevel process. At the predicate level, the **halt** move is issued through `toplevel_halt/1-2` and implemented using the general actor exit mechanism.

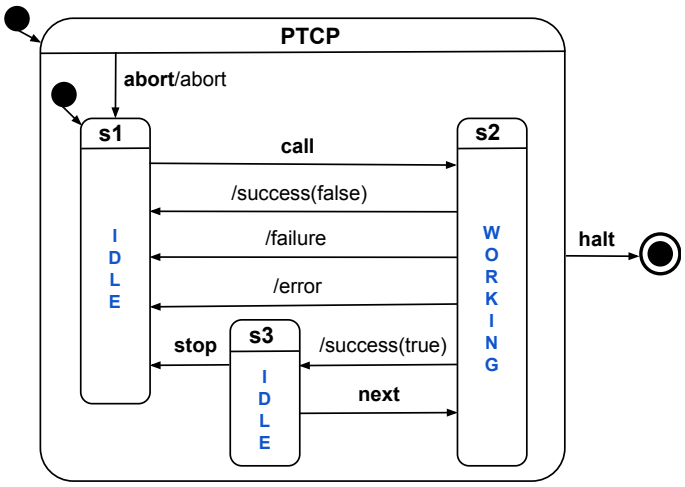


Fig. 3.3 Statechart specifying the Prolog Toplevel Communication Protocol (PTCP). Bold transitions denote messages sent from the client to the toplevel, whereas transitions prefixed by '/' denote responses sent by the toplevel back to the client.

A process is able to *spawn* a toplevel. After having done so, the process becomes the parent of the toplevel and can start communicating with it according to the protocol. Initially, the protocol is in state **s1**, where the toplevel rests idle, waiting for a **call** message containing a goal. When such a message arrives, the protocol transitions to state **s2**, where the toplevel is doing actual work. The protocol will remain in state **s2** until some work is done and the toplevel sends a message indicating either `/success`, `/failure`, or `/error`. If the actor terminates while monitored, the

runtime separately delivers `/down` to the client. On the event of the toplevel sending `/failure` or `/error`, the protocol will transition back to state **s1**. This will be the case also for a `/success` message that indicates no more solutions to the query can be found (marked with false in the chart). However, if a `/success` message indicates more solutions may exist (marked with true in the chart), the protocol transitions to state **s3**, where it will wait for a message **next**, **stop**, **abort** or **halt** to arrive from the client. If the message is **stop** or **abort** the protocol will transition back to state **s1**, if it is **next** it will transition to state **s2** and trigger the search for more solutions, and finally, if it is **halt**, the toplevel terminates and the runtime delivers `/down` for each installed monitor. Any other messages will be deferred to the code being executed by the toplevel.

Clients should send **next** or **stop** only after receiving `success(Pid,Data,true)` for the current query. After sending **next**, await its answer before sending another control message of either kind. The **abort** and **halt** signals remain available in every PTCP state, as Section 3.3.3 explains. The current `toplevel_abort/1` predicate addresses local toplevels only; HTTP and WebSocket clients request abort at the hosting node.

Web Prolog comes with six built-in predicates which allow a client to spawn a toplevel actor and interact with it during its lifecycle:

```
toplevel_spawn/1-2    toplevel_call/2-3    toplevel_next/1-2
toplevel_stop/1       toplevel_abort/1     toplevel_halt/1-2
```

As witnessed by some of the examples in Chapter 2, such predicates can be defined in terms of the more primitive `spawn/1-3`, `!/2` and `receive/1-2` predicates.

The sections that follow introduce the `toplevel_*` predicates and their most important options through examples. Complete signatures, option descriptions, and default values are given in Appendix B.

Toplevel actors are stateful

A toplevel is stateful even if its program keeps no explicit state and does not use the dynamic database. It remembers where the search left off: the suspended computation, with its remaining choice points. This implicit computational state is what enables the familiar lazy interaction model of Prolog, where successive solutions are generated only on demand.

State **s3** of the PTCP exists because the toplevel retains this suspended computation while waiting for the client to decide whether to continue the search. If the client sends a **next** message, execution resumes where it previously left off by backtracking to the next remaining choice point. If it sends **stop**, the suspended computation and its remaining choice points are simply discarded.

3.3.3 Shell talking to a Prolog toplevel

Below, we show an example of how to create and interact with a toplevel process from a shell. We start by spawning a new toplevel, using `toplevel_spawn/2` with the `monitor` option to ask the runtime to notify us with a `down` message when the process eventually terminates. The `src_list` option is used to populate the private Prolog database belonging to the toplevel actor with two simple facts `p(a)` and `p(b)`:

```
?- toplevel_spawn(Pid, [
       session(true),
       monitor(true),
       src_list([p(a),p(b)])
   ]).
Pid = 74122981.
?-
```

The `monitor` and `src_list` options are inherited from `spawn/1-3`. The additional option `session(true)` keeps the toplevel available for further queries after the first has completed.

The toplevel is now in PTCP state **s1**, ready to accept a query through `toplevel_call/2-3`.

Making the toplevel call a goal

Let us see what happens if we call `toplevel_call/2` with the default values for options:

```
?- toplevel_call($Pid, p(X)).
true.
?- flush.
Shell got success(74122981,[p(a),p(b)],false)
true.
?-
```

The answer is returned to the mailbox of the calling process in the form of an *answer term* with three arguments. The functor of the term represents its *type*. In this case, it shows that the goal succeeded. (In other cases it might indicate a failure or an error.) The first argument of the success term is the pid, and the solution slice in the second argument contains the two solutions that were computed. The value `false` in the third argument indicates that the toplevel has no further solution slice to return.

Since the `session` option was set to true, the PTCP is now, after a brief visit to state **s2** for the execution of the goal, back in state **s1** and is prepared to accept another use of `toplevel_call/2-3`.

A toplevel catches exceptions raised by submitted goals at its protocol boundary and reports them as `error(Pid,E)` answers. In a multi-goal session, it remains

available for another query. An error answer therefore does not imply that the toplevel actor has terminated.[2]

Using the template option

Below, `toplevel_call/3` is called with the goal `p(X)` and with the `template` option set to the variable `X`:

```
?- toplevel_call($Pid, p(X), [
       template(X)
   ]).
true.
?- flush.
Shell got success(74122981,[a,b],false)
true.
?-
```

Note how the value of the `template` option determined the form of the solution slice in the second argument of the success term. The relation to the Prolog built-in standard predicate `findall/3` is evident.

Using the `offset` and `limit` options

When querying a relational database using SQL or an RDF dataset using SPARQL, the use of the `OFFSET` and `LIMIT` keywords serves to control the subset of results that are returned by a query. In Web Prolog they can be used as options for the configuration of calls to `toplevel_call/3`. The `limit` option specifies the maximum number of solutions to return, while the `offset` option specifies the number of solutions to skip before starting to return solutions. Here is what happens if `limit` is set to 1:

```
?- toplevel_call($Pid, p(X), [
       template(X),
       limit(1)
   ]).
true.
?- flush.
Shell got success(74122981,[a],true)
true.
?-
```

[2] The demonstrator uses SWI-Prolog's message machinery to display diagnostics concisely by default. Its Terminal settings can instead display a Prolog exception term with the implementation-supplied context elided. For an exception caught by the toplevel, this changes only its presentation; the caught exception remains unchanged. For an uncaught actor exception, the runtime has already sanitised the monitor reason, and `flush/0` prints that term from the shell's mailbox.

The value `true` in the `success` term means that the state machine is now in state **s3**, waiting for a message **next** or **stop** to appear. Calling `toplevel_next/1` produces the next solution:

```
?- toplevel_next($Pid).
true.
?- flush.
Shell got success(74122981,[b],false)
true.
?-
```

`toplevel_call/3` supports the `offset` option. Together with the `limit` option it allows the call to pick out an arbitrary solution slice for a goal, even when the number of solutions is infinite:[3]

```
?- toplevel_call($Pid, between(1,infinite,I), [
       offset(100),
       template(I),
       limit(3)
   ]).
true.
?- flush.
Shell got success(74122981,[101,102,103],true)
true.
?-
```

Calling `toplevel_next/1` produces three more solutions:

```
?- toplevel_next($Pid).
true.
?- flush.
Shell got success(74122981,[104,105,106],true)
true.
?-
```

Note that the value of the `limit` option set in the call to `toplevel_call/3` was still in effect. However, `toplevel_next/2` accepts the `limit` option too, which allows us to increase or decrease the limit on the number of solutions returned:

```
?- toplevel_next($Pid, [
       limit(5)
   ]).
true.
?- flush.
Shell got success(74122981,[107,108,109,110,111],true)
```

[3] It may seem odd and not all that useful for `toplevel_call/2-3` to support the `offset` option, but it turns out to be important when implementing the stateless HTTP protocol.

```
true.
?-
```

Calling `toplevel_stop/1` will stop the execution of the goal:

```
?- toplevel_stop($Pid).
true.
?-
```

A toplevel serves one controlling client at a time, and each call starts afresh. When the toplevel accepts a new goal, it discards any `toplevel_next/1-2` or `toplevel_stop/1` request, and any input, left over from an earlier query. A `toplevel_next/1` sent by mistake after a stop therefore cannot leak into the next query.

The `offset` and `limit` options are particularly useful in several scenarios: paging of results in web applications, where it is impractical to retrieve and display all records at once; implementing infinite scrolling interfaces that load data dynamically as the user scrolls; handling goals with very many or infinitely many solutions without overwhelming the client; and reducing the number of network roundtrips on the Prolog Web by fetching solutions in batches. A special case is when `limit` is set to 1, which makes it straightforward to implement a Prolog-style REPL in JavaScript, where each press of `;` triggers a new request for the next solution.

Using the `once` option

The `once` option only makes sense in combination with the `limit` option; its default is `false`. Here it is set to `true`:

```
?- toplevel_call($Pid, between(1,infinite,I), [
       offset(100),
       template(I),
       limit(3),
       once(true)
   ]).
true.
?- flush.
Shell got success(74122981,[101,102,103],false)
true.
?-
```

The `once` option requests *committed* evaluation: the node returns the first slice (of up to `limit` solutions) and then discards any remaining choice points rather than retaining them for a possible `toplevel_next`. This is reflected in the answer term, whose third argument is `false` even when further solutions could in principle have been found – the call reports completion because the node has undertaken to produce nothing more. It differs from simply declining to call `toplevel_next/1`: that would

leave the session suspended in state **s3**, holding resume state until the next request or an idle timeout, whereas `once(true)` releases that state at once. The option is therefore a resource directive as much as a control one, useful when a client knows it wants at most `limit` solutions and wants the node to be free of any obligation to retain the computation.

A toplevel can do a kind of I/O

Whenever a toplevel is in state **s2** and doing some real work, it is able to send messages to its parent (or elsewhere) using the ordinary send operator. However, a better idea can often be to use `output/1`:

```
?- toplevel_call($Pid, output(hello)).
true.
?- flush.
Shell got output(74122981,hello)
Shell got success(74122981,[output(hello)],false)
true.
?-
```

Input can be collected by calling `input/2`, which sends a `prompt` message to the client, which in turn can respond by calling `respond/2`:

```
?- toplevel_call($Pid, input('Input', X)),
   receive({Answer -> true}).
Answer = prompt(74122981,'Input').
?- respond($Pid, hello),
   receive({Answer -> true}).
Answer = success(74122981,[input('Input',hello)],false).
?-
```

Aborting a nonterminating goal

Our toplevel actor is still not dead so let us see what happens when we ask the toplevel to first update its private Prolog database with a silly recursive clause for a predicate `p/0` and then call it:

```
?- toplevel_call($Pid, assert((p :- p))),
   receive({Answer -> true}).
Answer = success(74122981,[assert((p:-p))],false).
?- toplevel_call($Pid, p).
true.
?-
```

Although nothing is shown in the terminal, it is clear that the toplevel is now just wasting CPU cycles to no avail. Fortunately, a nonterminating goal can be aborted by calling `toplevel_abort/1`:

```
?- toplevel_abort($Pid).
true.
?- flush.
Shell got abort(74122981)
true.
?-
```

With this, the PTCP is back in state **s1**. The message `abort(74122981)` is the toplevel's acknowledgement.

Abort uses a second kind of signal. Chapter 2 introduced *termination signals*, which end an actor. An *interruption signal* instead abandons a delimited computation without ending the process. Like a termination signal, it bypasses the mailbox, so it can reach a toplevel that is busy computing in **s2**.

The abort passes through any exception handlers in the submitted goal and is consumed by the privileged PTCP boundary, which resumes the persistent toplevel in state **s1**. A catch-all `catch/3` in the goal therefore cannot defeat an abort or enter its recovery branch, although cleanup still runs. Halting, described next, instead sends a termination signal and destroys the actor. Figure 3.4 contrasts the two.

Abort applies to the goal that is running. Sent to a persistent toplevel that is idle in **s1**, it leaves it idle, although it is still acknowledged. Sent while the toplevel waits between solution slices in **s3**, it acts like `toplevel_stop/1`. A toplevel spawned with `session(false)` exists for a single goal, so aborting it ends the actor. Once a termination signal has been accepted, a later abort is ignored and cannot bring the toplevel back to **s1**. Every abort is acknowledged with `abort(Pid)`, which the toplevel sends to its client once the abort has taken effect, even when there was nothing to abort. Signals and messages are not ordered with respect to each other, but the acknowledgement is an ordinary message from the toplevel, so any answers the aborted query had already produced arrive before it. A client that waits for the acknowledgement before submitting its next goal therefore knows that everything received before it belongs to the aborted query.

Halting the toplevel

When we are done talking to the toplevel we can halt it:

```
?- toplevel_halt($Pid, Reply).
Reply = true.
?- flush.
Shell got down(74122981,74122981,true)
true.
?-
```

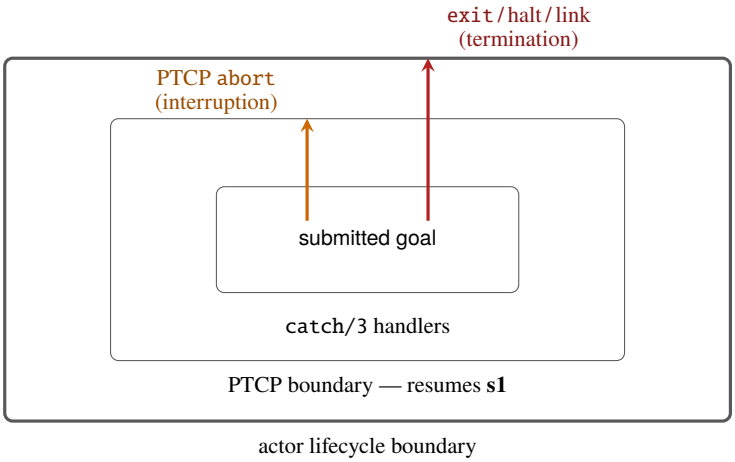


Fig. 3.4 Abort crosses client exception handlers but stops at the PTCP boundary; termination crosses the actor lifecycle boundary. Cleanup runs during either unwind.

The acknowledgement returned by `toplevel_halt/2` and the original monitor notification are distinct: halting waits on a private monitor, while the monitor installed at spawn time still delivers its own `down/3` message.

Redirecting answers

The `target` option can be used to redirect the answer terms produced by the toplevel to any actor of choice. By default, it is set to the pid of the parent but allows us to instruct a toplevel actor to send answer terms to a different destination. To demonstrate how this works, we first create a simple actor that can serve as a target:

```
?- spawn(( repeat,
           receive({
               Msg ->
                   format("Received ~p~n", [Msg]),
                   fail
           })
         ), Pid0).
Pid0 = 98380209.
?-
```

The pid of that actor can now be used as the value of the target option:

```
?- toplevel_spawn(Pid, [
       target($Pid0)
   ]).
```

```
Pid = 97919106.
?-
```

When `toplevel_call/3` is called, the `success` answer term is printed by the designated target actor:

```
?- toplevel_call($Pid, between(1,1000,N), [
       template(N),
       limit(5)
   ]).
true.
Received success(97919106,[1,2,3,4,5],true)
?-
```

So, `98380209` rather than the shell received the message from `97919106`, and calling `flush/0` shows that the mailbox belonging to the shell is empty:

```
?- flush.
true.
?-
```

The `target` option is supported by `toplevel_spawn/2`, `toplevel_call/3` and `toplevel_next/2`. It is not supported by the other `toplevel_*` predicates as they do not produce answer terms. It can also be used in calls to `output/2` and `input/3`. See Appendix B for more details.

3.3.4 Programming with toplevel actors

The following exercises build familiarity with toplevel actors.

A synchronous predicate API to toplevels

The communication between a client and a toplevel illustrated above is asynchronous, but if we want, it is quite easy to define a synchronous alternative using a technique we have already seen. A predicate `wait_for_answer/3` can be defined like so:

```
wait_for_answer(Pid, Answer, Options) :-
    receive({
        Msg if (compound(Msg), arg(1, Msg, P), P == Pid) ->
            Answer = Msg
    }, Options).
```

The guard tests the sender pid without binding variables in an unrelated message. Unifying the selected message with `Answer` in the body also lets a pre-instantiated answer fail normally instead of leaving the response unmatched.

Then `toplevel_call_synch/3-4` can be implemented as follows:

```
toplevel_call_synch(Pid, Goal, Answer) :-
    toplevel_call_synch(Pid, Goal, Answer, []).

toplevel_call_synch(Pid, Goal, Answer, Options) :-
    toplevel_call(Pid, Goal, Options),
    wait_for_answer(Pid, Answer, Options).
```

and `toplevel_next_synch/2-3` like so:

```
toplevel_next_synch(Pid, Answer) :-
    toplevel_next_synch(Pid, Answer, []).

toplevel_next_synch(Pid, Answer, Options) :-
    toplevel_next(Pid, Options),
    wait_for_answer(Pid, Answer, Options).
```

These synchronous wrappers wait in the calling actor's mailbox, so the toplevel's answer target must be that actor. This applies both to the target selected at creation and to any `target/1` override. Omitting the option is insufficient if the toplevel was created with another target. Directing answers elsewhere can leave the wrapper waiting indefinitely, or until its receive timeout expires.

The native demonstrator's corresponding wrappers enforce this routing: each request explicitly selects the caller's pid, and a conflicting `target/1` is rejected before sending. They also separate query options from local receive options. The definitions above keep that production option processing out of the example; the native contract is documented in Section B.2.

These wrappers illustrate a simplification used elsewhere in the book's example code: option lists are sometimes passed on without filtering them for each receiving predicate. This keeps option-processing machinery from obscuring the interaction being explained. Where this relies on SWI-Prolog's usual tolerance of unknown options, it assumes ordinary non-ISO mode; that tolerance is not a portable API guarantee, and remote protocol endpoints may reject unknown options. Separating and validating options for each destination is part of the engineering agenda in Section 11.1.

Note that since `toplevel_stop/1` does not produce a response message, no `toplevel_stop_synch/2-3` is defined, and `toplevel_stop/1` can be used as it is.

Here is a simple test that shows how it works:

```
?- toplevel_spawn(Pid).
Pid = 67590967.
?- toplevel_call_synch($Pid, mortal(Who), Answer, [
       template(Who),
       limit(1)
   ]).
Answer = success(67590967, [socrates], true).
?- toplevel_next_synch($Pid, Answer).
```

```
Answer = success(67590967, [plato], true).
?- toplevel_next_synch($Pid, Answer).
Answer = success(67590967, [aristotle], false).
?-
```

Reconstructing findall/3 and findnsols/4

In order to specify the semantics of a new Prolog predicate API it can be a good idea to implement built-in Prolog predicates with a well-known semantics using the API. For example, if we did not already have a built-in predicate `findall/3`, using a toplevel actor we could have implemented it like so:

```
findall(Template, Goal, List) :-
    toplevel_spawn(Pid, [
        session(false)
    ]),
    toplevel_call(Pid, Goal, [
        template(Template)
    ]),
    receive({
        success(Pid, List0, false) ->
            List = List0 ;
        failure(Pid) ->
            List = [] ;
        error(Pid, Error) ->
            throw(Error)
    }).
```

Of course, implementing each of these predicates in a separate process makes little practical sense. These contrived examples merely exercise some of the features of the `toplevel_*` API. Nevertheless, they can illuminate the relationship between `findall/3` and the `toplevel_*` predicates.[4]

We can take this one step further by also implementing `findnsols/4`, which works like `findall/3`, but generates at most `N` solutions to the specified goal. This predicate is especially useful if the goal may have an infinite number of solutions.

```
findnsols(N, Template, Goal, List) :-
    toplevel_spawn(Pid, [
        session(false)
    ]),
    toplevel_call(Pid, Goal, [
        template(Template),
        limit(N)
    ]),
```

[4] Here, we were inspired by (Tarau, 2000).

```
    wait_solutions(Pid, List).

wait_solutions(Pid, List) :-
    receive({
        success(Pid, List0, false) ->
            List = List0 ;
        success(Pid, List0, true) ->
            (   List = List0
            ;   toplevel_next(Pid),
                wait_solutions(Pid, List)
            ) ;
        failure(Pid) ->
            List = [] ;
        error(Pid, Error) ->
            throw(Error)
    }).
```

Ciao Prolog provides this predicate, but only in a deterministic version. The version provided by SWI-Prolog is nondeterministic, and so is our reconstruction. We borrow an example of its use from the SWI-Prolog manual:[5]

```
?- findnsols(5, I, between(1, 12, I), L).
L = [1, 2, 3, 4, 5] ;
L = [6, 7, 8, 9, 10] ;
L = [11, 12].
?-
```

The version provided by SWI-Prolog serves as a building block in our proof-of-concept implementation of the `toplevel_*` predicates.

These reconstructions focus on solution collection. A complete wrapper must also monitor the toplevel and arrange cleanup if the caller cuts or throws while further slices remain; otherwise the actor can be left waiting for a request that will never arrive. We return to that lifecycle discipline in the actor-based RPC reconstruction in Section 5.4.2.

3.3.5 Coordinating concurrent goals

The preceding examples used a predicate interface to hide exchanges with a toplevel actor. Two built-in meta-predicates, `parallel/1` and `first_solution/2-3`, apply the same idea to coordinating several computations. They run goals concurrently, but present a synchronous predicate interface: the caller waits for a result and observes

[5] See `https://www.swi-prolog.org/pldoc/doc_for?object=findnsols/4`. This predicate was implemented in SWI-Prolog by Jan Wielemaker in order to support `library(pengines)`, the library on top of which SWISH is built.

success, failure, variable bindings, or an exception. It does not manage the workers' mailboxes or lifetimes.

Solving independent goals concurrently

As Armstrong notes (Section 2.2.1), speed is only one of the reasons for concurrency, but it is the one `parallel/1` serves. It runs a list of independent goals concurrently and waits for all of them to succeed, potentially reducing their combined execution time. The following example uses delays to simulate three computations of different durations:

```
?- _Goals = [(X=a,sleep(1)),(Y=b,sleep(3)),(Z=c,sleep(2))],
   time(parallel(_Goals)).
% 189 inferences, 0.000 CPU in 3.006 seconds
X = a,
Y = b,
Z = c.
?-
```

The three delays overlap: the call completes in approximately three seconds, rather than the six seconds required by sequential execution.[6] The caller receives the bindings only after all three goals have succeeded.

The goals must not depend on bindings or side effects produced by sibling goals. The call commits to the first solution of each goal; it does not enumerate further combinations on backtracking.

If a goal fails, `parallel/1` terminates the remaining workers and fails. If a goal throws an exception, it terminates the remaining workers and rethrows the exception. When several workers fail or throw, the first abnormal outcome observed determines the result; there is no left-to-right precedence as in a sequential conjunction. Worker cleanup completes before the predicate returns, fails, or throws.

Racing alternative solvers

Sometimes the goals are alternative ways to find an answer, and only one needs to succeed. The predicate `first_solution/2` runs these alternatives concurrently and returns the first successful result it observes. We can again use delays to simulate computations of different durations:

```
?- time(first_solution(X, [(sleep(2),X=a),(sleep(1),X=b)])).
```

[6] The timing figures in this and the following example are illustrative and vary between runs and implementations. The delays simulate time-consuming operations and can overlap even on a single core. Independent CPU-bound goals can also execute in parallel when the runtime permits it, but their speedup depends on the number of processor cores available to the runtime, the workload, and the overhead of creating workers and returning results.

```
% 164 inferences, 0.000 CPU in 1.006 seconds
X = b.
?-
```

In this run, the second solver supplies the result after approximately one second. The slower worker is terminated before the call returns. Each solver has its own copy of `X`; the winning result binds the caller's variable through the solution template in the first argument. Backtracking does not produce another result.

A solver may fail without ending the race: the remaining solvers continue, and the call fails only if all of them fail. An exception, however, terminates the remaining workers and is rethrown to the caller. These default policies can be changed through the options of `first_solution/3`; see Appendix B.

Both predicates coordinate worker actors locally and belong to the ACTOR profile, introduced below. A worker may itself perform a remote call.

3.3.6 Server and supervisor actors

Two more generic behaviours appear in Figure 3.1. A *server* keeps private state and answers requests from any number of clients. A *supervisor* starts other actors and restarts them when they fail. As with the toplevel, we use them here through their interfaces.

The fridge of Chapter 2 makes a good server. The application supplies only a callback describing one transition. Its four arguments are the request, the old state, the response and the new state:

```
fridge(store(Food), Foods, ok, [Food|Foods]).
fridge(take(Food), Foods, ok(Food), Rest) :-
    select(Food, Foods, Rest), !.
fridge(take(_Food), Foods, not_found, Foods).
```

The server behaviour supplies everything else: the receive loop, the threading of state from one request to the next, and the matching of each reply to its request. We assume that `fridge/4` has been installed in the node's shared database, so that a spawned server can call it. A client starts a server with `server_spawn/3` and talks to it with `server_request/3`:

```
?- server_spawn(fridge, [], Pid).
Pid = 99491660.
?- server_request($Pid, store(milk), Response).
Response = ok.
?- server_request($Pid, take(milk), Response).
Response = ok(milk).
?- server_request($Pid, take(milk), Response).
Response = not_found.
?-
```

A supervisor is given a list of *child specifications*. Each says how to start a child and whether to restart it. Here the fridge runs as a supervised child, registered under a name that survives restarts:

```
?- supervisor_spawn([
       child(fridge_service, [
           start(server(fridge,[initial_state([])])),
           restart(permanent)
       ])
   ], Sup, [strategy(one_for_one)]).
Sup = 58310427.
?- supervisor_which_children($Sup, Children).
Children = [info(fridge_service, 91467230, worker, permanent)].
?- exit(91467230, simulated_failure).
true.
?- supervisor_which_children($Sup, Children).
Children = [info(fridge_service, 26083951, worker, permanent)].
?-
```

The child was killed and the supervisor started a replacement with a new pid. Once the replacement is running, clients can address it through the same registered name, `fridge_service`, without learning its new pid. Requests made during recovery may still fail. The replacement starts from the initial state, so the contents of the old fridge are lost. A supervisor restores availability, not data.

Supervisors matter most for services that should keep running on a node for a long time. Chapter 4 uses one to keep a publish–subscribe service alive.

3.4 Prolog nodes are also agents

As shown in the agent taxonomy in Figure 3.1, we regard a Prolog node as a kind of Prolog agent. This classification rests on its stable, term-based interface. Functionally, a node may also expose some of the query–answer facilities associated with a toplevel actor: clients can submit goals and receive solutions lazily, either one at a time or in bounded slices, in the order in which Prolog finds them. This functional overlap does not imply that a node is implemented as, or derived from, a toplevel actor.

The diagram in Figure 3.5 illustrates the anatomy of an executing Prolog node, which is accessed by a user-client that knows its URI and is authorised to use its services.

As suggested by the diagram, a node is comprised of components such as a node controller, web APIs, a shared database containing predicates defined by the owner, a container serving as the “home” of actors such as echo actors, supervisors and toplevels, and various settings for the owner to configure the node. As we shall see, not all nodes include all these components.

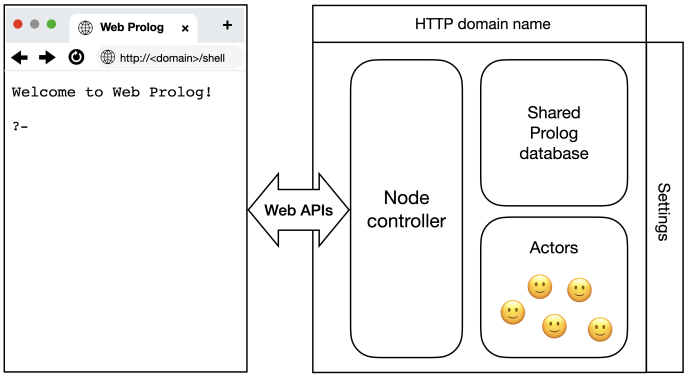


Fig. 3.5 The anatomy of a Prolog node. A terminal running in a web browser is attached to it.

3.4.1 The node controller

The node controller manages the node's operation. Depending on the supported profile, its responsibilities include managing web API connections, authenticating and authorising clients, spawning actors, enforcing resource limits, and maintaining registries, monitors, and links. It also formats replies to clients and applies the node's cleanup policy when connections close.

For interactive shells, the controller provides facilities such as the `$Var` substitution mechanism and the routing of Prolog input and output to a terminal. Actor programs use these services through the language primitives and APIs, without having to manage the underlying connections themselves.

The controller's role is compatible with treating the node itself as an agent. Clients address the node to submit goals and obtain results; the controller manages the actors and communication needed to provide that service.

3.4.2 The shared Prolog database

A node may host a shared Prolog database accessible over HTTP. It can be:

- *self-contained* – every predicate the exported relations need is available on any node: defined locally, a Web Prolog built-in, or a standard library predicate. Such a dataset is directly consumable by other nodes or clients.
- *dependent* – it relies on a predicate that is not guaranteed to be present wherever the database is consumed: one imported from a custom (non-standard) module, a remote predicate, or one with no provider at all. Portability is then conditional on that dependency being resolvable at the destination.

The predicates – which will be referred to as the *node-resident* predicates – are typically maintained by the owner of the node. External users are not able to make any changes to them, only the owner is allowed to do so. Any client connected with the node has access to these predicates in addition to the Web Prolog built-in predicates, and can pose queries over the web APIs formulated in terms of them. Suppose, for example, that the owner stores the following source in `mortal.pl`.

The contents of the file must be valid Web Prolog source code:

```
mortal(Who) :- human(Who).

human(socrates).
human(plato).
human(aristotle).
```

This is also what a user is presented with when steering an ordinary browser to the URI `http://n7.org`. This is an example of a shared Prolog database that is *self-contained*. In Chapter 4 we will demonstrate how this property makes it possible to achieve what we think of as *instant portability*.

However, a file might also look like this:

```
:- use_module(philosophers, [philosopher/1]).

mortal(Who) :- human(Who).

human(socrates).
human(Who) :- philosopher(Who).
```

This database is *dependent*: it draws `philosopher/1` from a separate `philosophers` module rather than defining it here. It functions at this node only if that module is present, and it is portable to another node only if `philosophers` is deployed there as well. (Had the file simply called `philosopher/1` without providing it anywhere, it would be dependent in a stronger sense – on a predicate with no provider at all.)

3.4.3 Deploying a node

To make this as concrete as possible, let us assume that the owner of a node wants to make the data in the file `mortal.pl` available to clients. From a demonstrator source checkout, the owner can start a local development node with the following command:

```
$ WP_AUTH=dev WP_PORT=3060 WP_SHARED_DB_FILE=mortal.pl \
    ./Deployment/start_node.pl
```

This command starts the Prolog service on port 3060, loads `mortal.pl` into its shared database, and grants execution access to clients connecting directly from the

local machine. Configuration and public deployment are described in Section D.8 of Appendix D. Making that service available on the public Web requires further deployment work: depending on the hosting environment, this may include a container or service manager, a reverse proxy (a front-end web server that forwards requests to the node), DNS configuration (so that the node's domain name leads to the right machine), and TLS certificates (so that clients can connect securely over `https`). Once deployed, the node offers the interfaces listed in Table 3.1:

API endpoint	**Resource or functionality**
`http://n7.org`	The shared Prolog database
`http://n7.org/demonstrator`	A simple shell environment
`http://n7.org/call`	The stateless HTTP API
`http://n7.org/toplevel_spawn`	The semi-stateful HTTP API
`ws://n7.org/ws`	The stateful WebSocket API

Table 3.1 Interfaces of the deployed demonstrator node. Note that the semi-stateful API has additional endpoints.

By steering an ordinary browser to the URI `http://n7.org` an authorised user may want to just browse the program. Or perhaps query it by making a visit to `http://n7.org/demonstrator`. Or perhaps the plan is to build an end-user application talking to the node over any of its web APIs.

Note also that because a Prolog node is also a web server, it may offer other HTTP endpoints as well, some of which may not be relevant to its core functionality.

This returns us to the goal of *deployable Prolog nodes* formulated in Chapter 1. A turn-key node should package the runtime, shared programs and data, actor hosting, documented interfaces, and basic operational controls into a system that an owner can configure and deploy. Reusable deployment tools and instructions should make it straightforward to discover, secure, test, and connect that system to other nodes and Web applications. The aim is to spare each developer from assembling the same infrastructure afresh. The demonstrator explores how such a package could work; it is not a production platform.

3.4.4 Settings

As suggested by Figure 3.5, a Web Prolog node is governed by a set of *owner-controlled settings*. Only the node owner may change these settings; some of them may be *readable* by clients over HTTP, so that clients can discover the node's advertised profile, public features, and operational limits. Collectively, the settings determine which features the node offers, who may access them, and the resource envelope within which actors and sessions are allowed to run.

It is useful to group settings into five categories:

1. **Identity and advertised profile.** The node advertises a *profile* to clients. This is primarily a discovery and compatibility mechanism: it tells clients what kind of service they can expect.
2. **Resource governance.** The node may enforce limits on injected code size, actor-local database size, cache bounds, and the maximum number of concurrent actors that client code is allowed to spawn. At the transport layer it may also enforce connection-related limits (for example, the maximum number of connections, and the maximum number of agents sharing a connection). These limits are part of the node's resource policy and are set by the node owner.
3. **Admission control.** The node may restrict which clients may connect and which goals may be executed, for instance via allow/deny lists and related authorisation rules.
4. **Session lifecycle and PTCP timeouts.** Two lifecycle limits are particularly important for PTCP sessions (cf. the PTCP state diagram in Figure 3.3). A *running time limit*, configured by the `time_limit` option, bounds each continuous period in which the session executes a goal in the working state **s2**. If the limit is exceeded, the node aborts the current goal and returns an error such as `error(resource_error(time),_)` to the caller. The session actor remains alive and returns to **s1**, ready to accept another call. Time spent waiting for a continuation in **s3** does not count towards the running time limit. An *idle limit*, configured by `idle_limit`, bounds how long the session may wait inactive in **s1** or **s3**. If this limit is exceeded, the node terminates the session actor normally. A client monitoring the session subsequently observes `down(Pid,Ref,true)`. These lifecycle limits are distinct from the `timeout` on a semi-stateful HTTP request: that timeout only bounds the wait for the next session event and does not abort the goal or terminate the session. A public client may request tighter lifecycle limits, but cannot weaken the ceilings set by the node owner.
5. **Bundle loading policy.** The node may require, or merely prefer, that loaded bundles are self-contained, meaning that they do not rely on unresolved external dependencies.

Note that readable over HTTP does not mean modifiable: the intention is that the node may expose a read-only subset of the settings that are relevant to interoperability and capacity planning, while keeping sensitive policy (notably admission rules) private.

Several of these settings are best understood as *policy gates* rather than mere tuning parameters. In particular, a node distinguishes between what the owner may do and what external clients may do. Clients are intended to program *session-local* concurrent computations, while the owner configures the long-lived surface of the node and installs durable services. This distinction will later be reflected in which concurrency-oriented predicates and options are admitted for client code, especially for lifetime control (for example whether a spawned actor may outlive its parent) and, in the distributed case, which combinations of remote placement and lifetime options are permitted.

3.4.5 Two transports – three web APIs

The primary responsibility of the node controller is to facilitate web API interactions. Nodes exposing the full interaction surface are designed to support two different transports – HTTP and WebSocket – and three distinct API types: a stateless HTTP API, a semi-stateful HTTP API, and a stateful WebSocket API. Different applications need different APIs.[7]

The HTTP transport

The *Hypertext Transfer Protocol* (HTTP) is the protocol a web browser uses to fetch pages. The client sends a request and the server sends back a response. A GET request asks for a resource; a POST request sends data to the server. HTTP is *stateless*: the server need not remember anything about a client between requests.

A client interacting with a node by making requests over an HTTP connection is illustrated in Figure 3.6.

Fig. 3.6 Interaction between a client and a node over an HTTP connection.

HTTP provides the request–response semantics independently of the representation carried in a request or response. Media types identify that representation, while HTTP content coding can compress it without changing its underlying format. HTTPS applies TLS to the HTTP connection, providing confidentiality and integrity in transit and normally authenticating the server; client authentication depends on the deployment. TLS neither chooses between JSON and Prolog text nor replaces application-level authorisation. The protocol appendix states the precise representation and transport contracts used by Web Prolog.

The WebSocket transport

With HTTP, only the client can start an exchange: it asks, and the server answers. WebSocket instead keeps a connection open, so that either side can send a message whenever it likes, with little delay. A node that lets clients control toplevels and

[7] Strictly speaking, HTTP and WebSockets both live in the application layer, not the transport layer. But in practice, even if TCP is the real transport protocol under the hood, it makes sense to think of them as alternative ways of transporting data between agents.

other actors therefore offers WebSocket, with a small protocol of its own on top of it, called a *sub-protocol*. The three phases in the lifecycle of a WebSocket connection are illustrated in Figure 3.7.

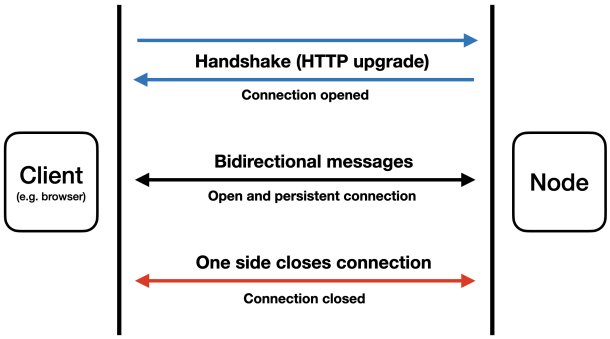


Fig. 3.7 The lifecycle of an interaction between a client and a node over a WebSocket connection.

In the first phase (in blue in the diagram) a WebSocket connection is initiated by the client via an HTTP connection which is then switched over to the WebSocket protocol. In the second phase (in black), the client and the node start talking to each other. This is where the sub-protocol comes in.

The Web Prolog WebSocket binding uses JSON objects as its protocol envelopes: client frames carry a `command` field and server frames a `type` field. Prolog goals and messages carried by fields defined as term text remain Prolog text inside those envelopes; JSON parsing alone does not turn them into Prolog terms. The HTTP bindings additionally allow a client to request either a JSON response envelope or a Prolog answer term. Compression, when enabled, belongs to the HTTP content-coding or WebSocket extension layer and does not change these representations. Exact rules and negotiation requirements are given in Appendix D.

WebSocket client libraries are available for the most common general programming languages. However, the most common use of WebSockets is in communication between a web browser and a server. In all the major web browsers, the WebSocket object provides a JavaScript API for creating and managing a WebSocket connection to a server, as well as for sending and receiving data on the connection.[8] The role of JavaScript here is to create a new WebSocket, to define the necessary handlers for `onopen`, `onmessage`, `onerror` and `onclose` messages, and to call the methods `send` or `close` for sending messages or closing the connection.

In the next section, and with the help of a number of scenarios, the actual APIs will be demonstrated. For details, see Appendix B.

[8] See `https://www.w3.org/TR/websockets/`

3.5 Node profiles

Prolog nodes need not offer every feature described in this chapter. One node might only answer queries over published relations; another might let clients run Prolog programs, maintain sessions, or create communicating actors. A *node profile* tells clients which facilities they can expect.

The four profiles form a cumulative hierarchy, shown in Figure 3.8. Each includes the requirements of the preceding profiles.

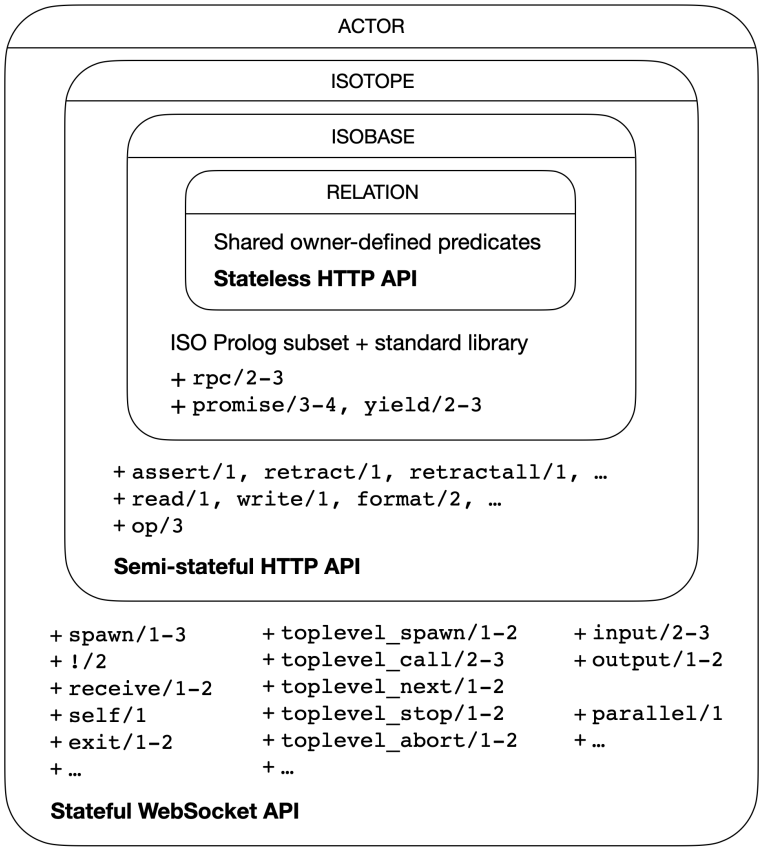


Fig. 3.8 The node-profile hierarchy. Each profile includes the requirements of those inside it.

RELATION Clients query relations advertised by the node owner and retrieve their solutions through the stateless HTTP API. The node need not contain a Prolog engine: it may obtain the answers from another kind of data source.

ISOBASE Clients can submit general Prolog goals within the supported Web Prolog language subset, including calls to owner-defined predicates. They may supply source code with a request, but its definitions belong only to that call.

ISOTOPE Clients can create persistent Prolog toplevel sessions through the semi-stateful HTTP API. Definitions and database updates can be retained between queries, and goals can perform interactive I/O.

ACTOR Clients can create actors, exchange messages, and monitor their termination through the stateful WebSocket API. The node also provides owner-only operations for registering services under stable names.

A broader profile does not change the meaning of the APIs inherited from the simpler profiles. On an ACTOR node, for example, a request to `/call` still runs a stateless ISOBASE goal. To maintain a toplevel session, the client uses the semi-stateful HTTP API; to create and communicate with general actors, it uses the WebSocket API. A client that only needs stateless queries can therefore use an ACTOR node without having to handle sessions or actor messages.

A persistent session does not by itself expose actor-style concurrency. ISOTOPE lets a client continue a Prolog conversation across requests. ACTOR additionally lets the client coordinate multiple processes through messages and termination notifications. This is the distinction between the sequential and concurrent interaction contracts introduced in Chapter 1.

Advertising a profile commits a node to providing all its required facilities. An ACTOR node must therefore offer both HTTP APIs as well as the WebSocket API. An experimental node that offers only actor communication over WebSocket may be useful, but does not meet the ACTOR profile.

A node may offer more than its advertised profile requires. The profile specifies a minimum guarantee: clients can rely on its required facilities, while additional capabilities must be advertised separately if clients are to discover and use them. For example, a node advertising ISOBASE may also offer persistent sessions without providing everything required by ISOTOPE.

Profiles describe client-visible facilities, not implementation choices. A RELATION node may use actors internally without exposing actor operations to clients.

The following subsections illustrate the four profiles through concrete client interactions.

3.5.1 Browser talking to a RELATION node

As indicated by the profile hierarchy in Figure 3.8, the stateless HTTP API is the only web API that *must* be supported by a RELATION node. This does not distinguish it from nodes having a broader profile, as this is required by them too. What makes it stand out is that it is not required to offer access to any Web Prolog built-in predicates or control constructs *at all*. Even control constructs such as `,/2` and `;/2`, may not be available.

In the scenario depicted in Figure 3.9, a user equipped with an ordinary web browser is talking to the RELATION node at `http://n1.org`. The web-based terminal displays the remnants of a session during which the user executed two queries. The two question marks signify that neither the owner-defined resources nor the way answers are produced is known by the user.

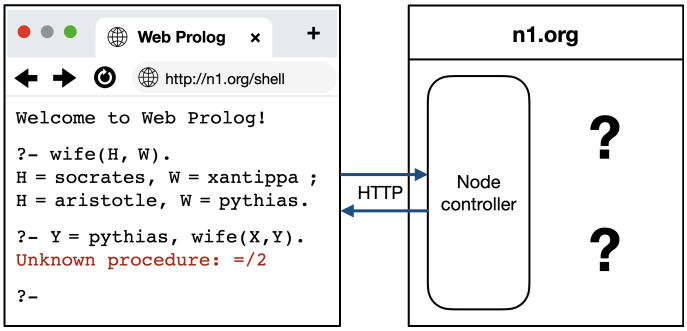


Fig. 3.9 A scenario involving a web browser in conversation over HTTP with a remote RELATION node. The two question marks signify that neither the owner-defined resources nor the way answers are produced is known by the user.

The session began when the user loaded the Prolog shell at `http://n1.org/shell`. After displaying a welcome banner and a `?-` prompt, the shell used a JavaScript loop to read each query, translate it into an HTTP request, and display the resulting answer in the terminal.

Let us look at how the first solution to `?-wife(H,W)` was retrieved by using the stateless HTTP API. The URI pointing to this solution is:

```
http://n1.org/call?goal=wife(H,W)&limit=1&offset=0
```

Note the use of `limit` and `offset` parameters. Although here they are parameters rather than options, their defaults and the effect that they have on answer terms is the same as for the options accepted by `toplevel_call/3`.

URIs such as these describe a query and a solution slice, and can be bookmarked. Responses may also be cached by intermediaries when HTTP caching rules permit it. Stateless interaction makes each request self-contained; it does not by itself guarantee repeatable answers or shared-cache suitability. A stateless request cannot be interrupted once it has started, so nodes must enforce timeouts.

When the user entered the query `?-wife(H,W)`, (and assuming that a relative path was used instead of an absolute one) the following GET request was made to the node:

```
GET /call?goal=wife(H,W)&limit=1&offset=0 HTTP/1.1
Accept: application/json
```

The `Accept` header requested responses in JSON. (In this case the header could have been left out since JSON is the default.)[9]

The node responded with the following JSON structure, indicating that the query succeeded:

```
{ "type": "success",
  "data": [{"H":"socrates","W":"xantippa"}],
  "more": true }
```

The `data` field contains the variable bindings and the value `true` of the `more` field signals that further solutions may be available. This triggered the printing of the variable bindings and served as a prompt for how to continue the exploration.

The user wanted to know if there were more solutions and typed a semicolon which led to the following request being made, looking exactly like the previous request, except that the `offset` parameter was incremented from `0` to `1`:

```
GET /call?goal=wife(H,W)&limit=1&offset=1 HTTP/1.1
```

The node responded like so:

```
{ "type": "success",
  "data": [{"H":"aristotle","W":"pythias"}],
  "more": false }
```

The `more` field was now `false`, indicating that no further solutions existed, and after having printed the next set of bindings and a full stop, the shell again produced the `?-` prompt, inviting the user to enter another query.

At this stage of the interaction, it is reasonable for the user to expect that `wife/2` has the mode `wife(?,?)` and that queries such as `?-wife(socrates,W)` or `?-wife(H,pythias)` should also succeed. A query such as `?-wife(plato,W)` should fail, however, and the response would be the following:

```
{ "type": "failure" }
```

Finally, an erroneous query, such as the attempt in the scenario to use the built-in operation `=/2`, which this RELATION surface does not expose, should raise an error, and show up as a response like so:

```
{ "type": "error",
  "data": "Unknown procedure: =/2" }
```

A RELATION node must answer as if it were Prolog, letting the client backtrack lazily over solutions, whether or not it actually backtracks.

The lack of built-in predicates naturally makes a RELATION node very easy to implement. There are two quite different ways a RELATION node can come into existence. First, a node that already implements a richer profile can be turned into a

[9] A `format` parameter is also supported as this allows us to easily make a request from the address field of a web browser.

RELATION node by installing a filter that inspects the goal parameter in each `/call` request and only allows it through if it matches a declared relation. Such a filter can be as simple as matching the submitted goal against a whitelist of terms – for instance, accepting only `employee(_,_,_)` and `name(_,_)`, and rejecting everything else with an error. A node that is supposed to serve a particular application rather than act as a general compute server will often operate this way: it implements ISOBASE or more internally, but presents a RELATION interface to its clients.

Second, a RELATION node can be implemented from scratch in any programming language. (Recall that we do not require that a Prolog node is written in Prolog, only that it can talk Prolog.) At a minimum, an implementation must parse Prolog terms and produce variable bindings for queries against its data. A Python web service backed by a relational database, or a microservice wrapping a key–value store, can present itself as a conformant RELATION node as long as it implements the stateless HTTP API and responds with the expected JSON structure. This is the profile that makes the Prolog Web interoperable with the rest of the Web: any structured data endpoint can, in principle, participate in the ecosystem.

3.5.2 Browser talking to an ISOBASE node

For a node to adhere to the requirements of the ISOBASE profile, it *must* support the same stateless HTTP API as required by the RELATION profile. In our next scenario, illustrated in Figure 3.10, the user is running the same query as in the previous scenario. It means that we do not need to describe the details of requests and responses again. Instead, we focus on the language surface of the node and the contents of the shared Prolog database offered by it, and visible in the figure.

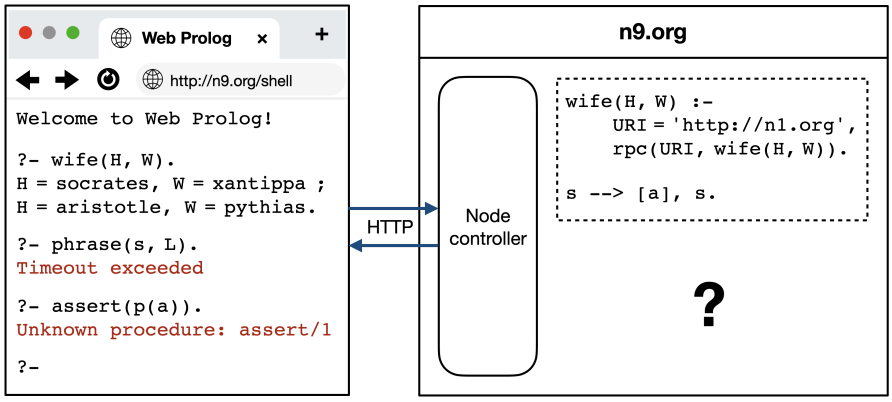


Fig. 3.10 Stateless interaction in a scenario involving a web browser and a remote node.

The main difference compared to the previous scenario is that an ISOBASE node *must* make a fairly large number of Web Prolog built-in predicates available, including a large subset of ISO Prolog predicates (recall the cloud of ISO Prolog predicates in Chapter 2), standard library predicates, definite clause grammars, as well as predicates for working with the Web.

An ISOBASE node must also support `rpc/2-3`, an easy-to-use predicate for making *nondeterministic remote procedure calls* from one node to another over stateless HTTP, communicating in a way that is essentially *synchronous*. In the scenario in Figure 3.10, the URI `http://n1.org` in the first argument of the call to `rpc/2` shows that it accesses the relation `wife/2` made available by the node `n1.org` in the previous scenario.

In addition to the synchronous `rpc/2-3`, an ISOBASE node must also support *asynchronous* RPC using two built-in predicates: `promise/3-4` and `yield/2-3`. All three predicates can be implemented on top of the stateless HTTP API.

Stateless interaction has one significant constraint: once a query has been submitted, the client has no way to abort it through the protocol. Because there is no control channel back to the running computation, ISOBASE nodes must enforce a timeout. This is what happened with the user's second query in the scenario shown in Figure 3.10: the node responded with the message `Timeout exceeded`. Unlike the previous scenario, this one allowed the user to inspect the source code defining the predicates made available by the node owner. This revealed that the node owner had apparently forgotten to add the DCG rule `s --> []`, which would have allowed the call to terminate.

Another limitation is that a truly stateless request-response protocol simply cannot support updates to the dynamic private database of an actor, nor I/O. It means that ISO predicates for modifying the dynamic database (for example, `assert/1` and `retract/1`), or I/O (for example, `read/1` and `write/1`), are *not* available. This is what happened with the user's third query, which was met with an `Unknown procedure` error message when trying to assert the clause `p(a)`. The predicate `assert/1` is simply not available to a client talking to an ISOBASE node.

3.5.3 Browser talking to an ISOTOPE node

An ISOTOPE node offers clients a shell backed by a toplevel actor. (Importantly, this does not mean that the internal `toplevel_*` predicates are directly exposed to programmers.) By using the process identifier of this actor as a session key in otherwise stateless HTTP requests, the node provides a semi-stateful API: each request is stateless in itself, but thanks to the pid they address the same long-lived toplevel actor.

The client's initial request can populate the actor's private database, which can then be updated dynamically throughout the session. Furthermore, although the client must initiate all interactions, the toplevel actor can not only respond to queries but

can also prompt the client for further input or produce output at arbitrary moments. As long as the client keeps supplying the session identifier, this will work just fine.

In the scenario in Figure 3.11, the client established communication with a toplevel actor on an ISOTOPE node, which is a node that must satisfy all the requirements of ISOBASE, and then some.

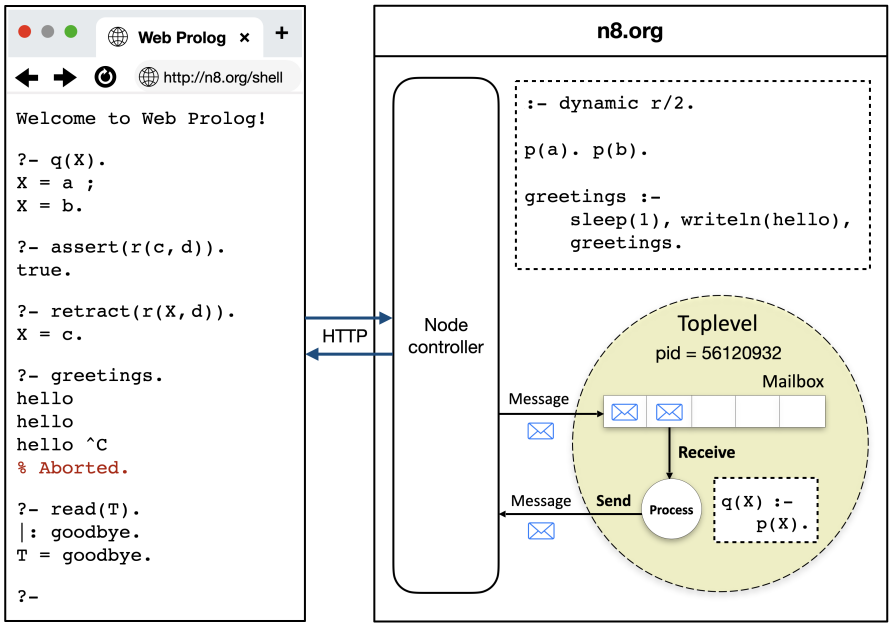


Fig. 3.11 A scenario involving a web browser in conversation over semi-stateful HTTP with a remote ISOTOPE node running a toplevel actor.

The request below instructed the node controller to spawn a toplevel actor running as a session, and to populate its private database with the value of the `src_text` option. (Let us assume that the clause `q(X):-p(X)` was picked up from an editor window that is not shown in the diagram.)

```
POST /toplevel_spawn HTTP/1.1
Content-Type: application/json

{ "options": "[session(true), src_text('q(X):-p(X).')]" }
```

The node responded with a JSON structure containing the pid for the spawned toplevel actor:

```
{ "type": "spawned",
  "pid" : 56120932 }
```

This produced the prompt `?-` in the terminal, inviting the user to enter a query. The query `?-q(X)` caused the JavaScript terminal to make the following request to the node:

```
POST /toplevel_call HTTP/1.1
Content-Type: application/x-www-form-urlencoded

pid=56120932&goal=q(X)&limit=1
```

(Note that the pid must be sent along when querying the toplevel.) The request told the node controller to make a call to `toplevel_call/3` and to convert the resulting answer term into a JSON structure that encodes the first solution:

```
{ "type": "success",
  "pid" : 56120932,
  "data": [{"X":"a"}],
  "more": true }
```

Except for the pid, the JSON structure representing this solution is identical to the response when making a stateless request.

The value `true` of the `more` property signifies that more solutions may be available. When the user asked for another solution, the following request was made:

```
POST /toplevel_next HTTP/1.1
Content-Type: application/x-www-form-urlencoded

pid=56120932
```

This results in a similar message, except that the value of the `data` property becomes `[{"X":"b"}]` and the value of `more` becomes `false`.

Note that it took the *combination* of the clause that was loaded into the toplevel's private Prolog database, and data in the shared database to answer the query `?-q(X)`.

Because the toplevel persists across requests, it makes sense to allow the client to update its private database using `assert/1` and `retract/1` during the session.

```
POST /toplevel_call HTTP/1.1
Content-Type: application/x-www-form-urlencoded

pid=56120932&goal=assert(r(c,d))
```

With this, the clause `r(c,d)` ended up in the private database of `56120932`, but was removed again when the user decided to call `retract(r(X,d))`.

The user then called `greetings/0`, which calls `sleep(1)` and `writeln(hello)` in a tight loop. The runtime routes `writeln/1` through the session's terminal I/O channel. This uses internal actor communication; the actor-level `output/1` predicate is not exposed to ISOTOPE clients. This resulted in a series of output responses of the following form, returned to the client approximately once per second:

```
{ "type": "output",
  "pid" : 56120932,
  "data": "hello" }
```

HTTP lets only the client start an exchange, so the node cannot push output to the client on its own initiative. The client therefore uses a technique similar to what web developers call *long-polling*: every time a JSON structure of type `output` arrives, the client immediately sends a new request, and the node answers it as soon as it has more output or some other response. See Section E.2.2 for more details.

The call to `greetings/0` will run forever unless the user does something about it. When the user hits Control-C, the nonterminating query is aborted, and the node responds with the following message:

```
{ "type": "abort",
  "pid" : 56120932 }
```

If the client does not exit the remote process, the node may terminate it after a set time with no activity. The pid will no longer be valid and any changes made to the workspace of the process will be lost.

3.5.4 Browser talking to an ACTOR node

Unlike an ISOTOPE node, an ACTOR node gives clients access to the full set of Erlang-style concurrency predicates and, in the bindings specified here, exposes them over a bidirectional WebSocket connection. This combination enables push-driven interaction, mixed-initiative communication, and concurrent actor programming – interaction patterns that the current HTTP bindings do not expose. The contract, rather than WebSocket as a transport, is what makes the surface concurrent. Figure 3.12 illustrates a scenario in which a user engages with an ACTOR node over such a connection.

After having loaded the shell and established a WebSocket connection with the node, the JavaScript process sent the following command to the node's controller, instructing it to spawn a toplevel actor running a session with the clause `q(X) :- p(X)` loaded into the actor's private database:

```
{ "command": "toplevel_spawn",
  "options": "[session(true), src_text('q(X):-p(X).')]" }
```

As requested by the client, the pid of the toplevel actor was returned to the client, not in the form of an HTTP response, but embedded in a JSON formatted message sent over the WebSocket connection:

```
{ "type": "spawned",
  "pid" : 23981144 }
```

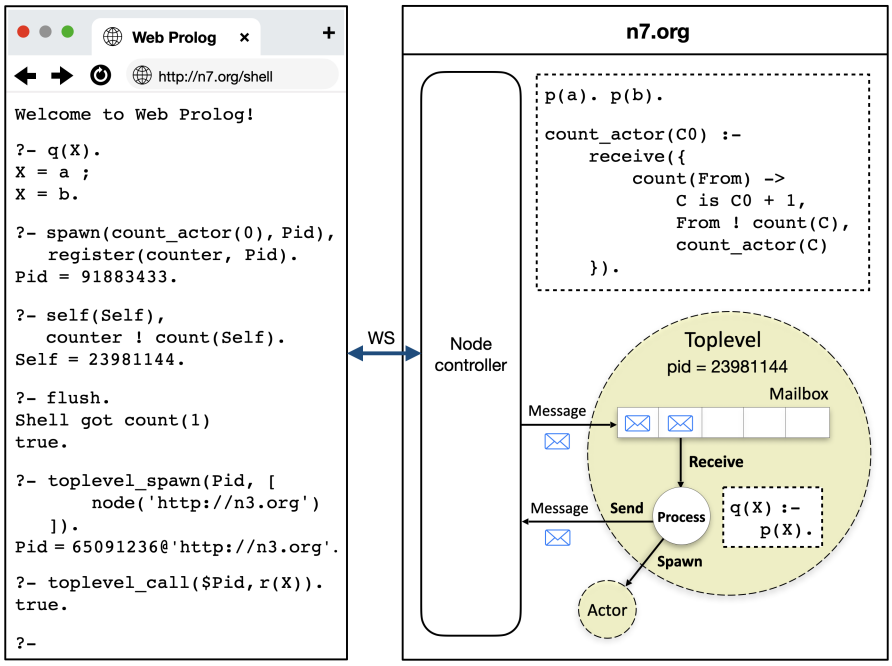


Fig. 3.12 A scenario involving a web browser in conversation with a remote node running a toplevel actor over a stateful WebSocket connection.

From here on, the solving of the query `?-q(X)` proceeds in a way very similar to how it worked in the ISOTOPE scenario, except that instead of the JavaScript shell making an HTTP request, it sent the following command message to the node over the WebSocket connection:

```
{ "command": "toplevel_call",
  "pid": 23981144,
  "goal": "q(X)",
  "options": "[limit(1)]" }
```

Knowing that the node's shared database offers a definition of `count_actor/1`, the user then decided to spawn a count server of the kind we saw in Chapter 2. Here is the message sent over the WebSocket connection:

```
{ "command": "toplevel_call",
  "pid": 23981144,
  "goal": "spawn(count_actor(0),Pid),register(counter,Pid)",
  "options": "[limit(1)]" }
```

This resulted in:

```
{ "type": "success",
```

```
"pid" : 23981144,
"data": [{"Pid":91883433}],
"more": false }
```

While the toplevel actor continued to serve shell queries, the spawned count actor ran concurrently and received `count` messages independently of the shell interaction, illustrating the concurrent, actor based nature of the ACTOR profile. With access to the pid of the spawned count server, the user could send it a `count` message in order to try it out, and call `flush/0` to inspect the response.

Finally, the user called `toplevel_spawn/2` with the `node` option. This is yet another way to do network-transparent concurrency.

In this implementation, if the user leaves the terminal at `http://n7.org/demonstrator`, closing the WebSocket causes the controller to terminate the session toplevel (via `exit/2`); before exiting, the toplevel terminates the spawned count actor. This yields a supervision-like hierarchy.

3.5.5 A terminal talking to a browser node

The previous scenario placed the ACTOR node on a remote machine. The same session can run against a node inside the user's own browser tab. The terminal shows no difference: the same predicates, the same answers, the same messages. We therefore do not repeat the traffic.

The demonstrator builds this node on SWI-Prolog compiled to WebAssembly (WASM), a format that lets programs written in languages other than JavaScript run inside the browser. The WASM build of SWI-Prolog is single-threaded, so it cannot by itself run several actors independently. The actor runtime is therefore provided by an extra layer around it, described in Section E.5.1 of Appendix E.

Pids have the same form, `Id@Node`, with the reserved node name `localhost` for actors in the tab. Programs may construct and take apart this term, but constructing a pid does not create an actor or grant access to one. Sending to a valid pid works the same wherever the actor lives.

A browser node can also act as a client. The last query in the figure spawns a toplevel on `n3.org` with the `node` option, and the tab opens a WebSocket connection to do so. A browser node cannot, however, accept incoming connections.

The real difference is lifetime. A remote node outlives any single session, which is why a remote session must clean up the actors it spawned (Chapter 4). A browser node and its actors end when the page closes, so there is nothing to clean up locally. Actors it spawned on remote nodes are governed by those nodes' policies.

3.5.6 Social robot talking to an ACTOR node

In the previous scenarios, the shell and its toplevel actor served as the root of control: it was the first long-lived process from which other actors were spawned, monitored, and addressed. In general, however, the root of an application need not be a toplevel at all. Any actor can play that role, and a client may interact with such an actor directly, without routing interaction through a shell-style PTCP conversation.

Figure 3.13 sketches a minimal toy scenario in which a social robot interacts with a remote ACTOR node over the stateful WebSocket API.

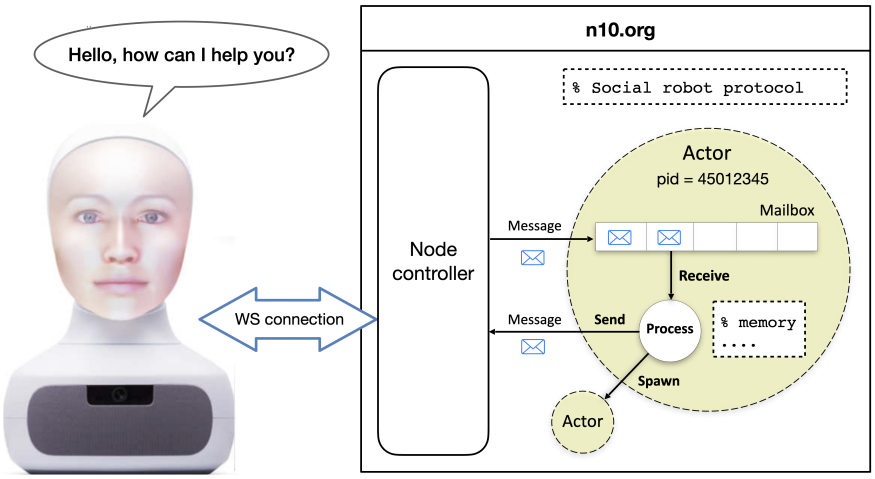


Fig. 3.13 Stateful interaction in a scenario involving a social robot and a remote ACTOR node.

The robot itself provides speech recognition and speech synthesis: it turns the user's spoken input into text, sends that text to the node, receives a reply as text, and speaks it back to the user. In this toy setup the remote side does nothing clever – it simply echoes the input – so the interaction is a "parrot" loop. The purpose is not to propose a useful robot skill, but to isolate the transport-level point: unlike the shell scenarios, the client is not a terminal driving a toplevel, and yet it can still engage in persistent, bidirectional interaction with a remote node.

At the protocol level, the scenario begins with the robot requesting that the node spawn an echo actor. The request is sent as a WebSocket message to the node controller:

```
{ "command": "spawn",
  "goal": "echo_actor",
  "options": "[monitor(true)]" }
```

The node responds by returning the pid of the spawned actor, which becomes the address the robot uses in subsequent messages. Subject to the node's policy, possession of this valid address gives the robot the capability to send to the actor.

```
{ "type": "spawned",
  "pid": 47110023 }
```

The WebSocket transport matters here because the interaction is push-driven and continuous. The robot establishes one connection and then exchanges messages in both directions as events occur: it sends new utterances as soon as they are recognised, and the node can return responses as soon as they are produced, without requiring a new HTTP request per response. This is the sense in which the interaction is stateful at the connection level: the channel remains open, latency is low, and messages can be delivered whenever either party chooses to speak.

In realistic conversational systems, of course, echoing recognised text is not enough. One typically needs additional components such as natural language understanding, a dialogue manager, grounding in a knowledge base, and response generation, as well as timing-sensitive coordination of speech, gaze, and facial expressions. We return to such agent-level concerns in Chapter 8; the present scenario only serves to show that, with the ACTOR profile, the transport and interaction model already supports the kind of persistent, low-latency, bidirectional exchange that embodied clients require.

A statechart actor offers a natural next step for the robot's controller. States can represent listening, waiting for a reply, and speaking; transitions can react to recognised utterances, completed actions, or timeouts. The controller can invoke other actors for tasks such as interpretation or knowledge lookup and coordinate their replies with the dialogue.

Unlike a toplevel actor, a statechart actor does not impose a built-in application conversation such as PTCP. The chart's author chooses the incoming event terms and outgoing messages, thereby defining the protocol spoken with the robot and its supporting actors. The runtime still supplies the ordinary actor lifecycle and statechart control facilities. Chapter 6 develops this behaviour; Chapter 8 returns to its use in agent and dialogue architectures.

3.5.7 Could there be other profiles?

The hierarchy RELATION $\subset$ ISOBASE $\subset$ ISOTOPE $\subset$ ACTOR is useful because many applications need less than a full ACTOR node. Simpler profiles are easier to implement, govern, and deploy, while still supporting portable client behaviour.

At the same time, the hierarchy is not meant to be a closed taxonomy. The existing profiles mostly specify *what* features are exposed at the node interface. One can also imagine profiles that express *how* a node chooses to do it: restrictions, guarantees, or disciplines that cut across the profile hierarchy.

A PURE profile is one example: clients would be restricted to a declarative subset (excluding cut, non-logical arithmetic, and related extra-logical constructs), while still potentially allowing features such as source injection. Another example is a statechart discipline exposed as an optional ACTOR sub-profile for reactive conversational control. At the language level, Chapter 6 also identifies a *flat* subset of SXML (Section 6.8) for non-hierarchical machines. Unlike a node profile, this is a descriptive subset rather than a separately declared and validated offer.

3.6 Summary

This chapter developed Prolog agents at two scales. Actor agents are independently scheduled processes with private state and message-based interfaces; node agents host programs, knowledge, and actors while presenting a stable, owner-governed boundary to clients. Toplevel actors connect the two scales through PTCP, turning the familiar query–answer conversation into an explicit protocol for incremental solutions, I/O, aborts, and termination.

The RELATION, ISOBASE, ISOTOPE, and ACTOR profiles make progressively broader language and interaction commitments observable. They let clients discover what a node promises without constraining how it is implemented.

The three web APIs establish how these agents can be reached through stateless requests, stateful sessions over HTTP, or bidirectional WebSocket connections. So far every agent has lived on a single node; the next chapter lets them cross the node boundary.

Chapter 4
The Prolog Web

> Imagine the Web wrapped in Prolog, running on top of a distributed architecture – a network of nodes exposing HTTP and WebSocket APIs, speaking web formats such as JSON. Think of it as a high-level Web, with Prolog agents capable of serving answers to queries – answers that follow from what the Web knows. Moreover, imagine it being programmable, allowing Web Prolog source code to flow in either direction: from client to node, or from node to client. This is what **the Prolog Web** is all about.
>
> *The Prolog Web – the elevator pitch*

The previous chapters introduced Web Prolog as a language and Prolog agents as programmable entities. This chapter turns to the third element of the Prolog Trinity: the *Prolog Web* itself – a network of nodes hosting actors that communicate across node boundaries. In short, we extend Web Prolog into a language for distributed programming.

The model draws on Erlang's *network-transparent concurrency*: if we know a node's identity and we are authorised, we can spawn an actor there; if we know an actor's pid or its registered name, we can send it a message, even across node boundaries. Unlike an Erlang cluster, however, the Prolog Web is as open as the Web itself. Here, *openness* means more than mere reachability. It means heterogeneous ownership, weak trust assumptions, communication over ordinary Web infrastructure, and an environment in which latency, disconnects, and partial failure are part of normal operation. The intended transparency is therefore selective: it unifies the programming surface for local and remote actors without pretending that their operational conditions are identical.[1] This follows the argument of Waldo et al. (1994) that distributed interaction must expose the consequences of latency, concurrency, and partial failure rather than paper them over. Openness brings risk, so the chapter pays close attention to safety: how lifetime discipline and authority separation reclaim abandoned work after loss is detected.

[1] The categories "local" and "distributed" do not exhaust the possibilities. Two Prolog systems may run on the same physical computer while occupying separate processes and address spaces. Communication between them still crosses an address-space boundary, and may use the same Web Prolog protocols as communication between different computers, although it does not incur all the costs or failure modes of a physical network.

Openness also forces a basic separation of authority. On the Prolog Web there are two roles: *clients*, who temporarily borrow execution resources in order to run queries and coordinate short-lived concurrent work, and *node owners*, who are responsible for the node as an ongoing public resource. For clients, the default discipline is *containment-by-default*: spawned processes remain tied to the client session and are reclaimed once its termination or loss of its owning connection is detected. For owners, the corresponding power is *provisioning*: the ability to install node-resident predicates and start long-lived actor services meant to be shared and kept available. In practice such services are often supervised (Section 3.3.6). This role split will recur in concrete form when we discuss remote placement and the various code-shipping mechanisms via the `src_*` options.

Sections 4.1 and 4.2 cover the concurrent and sequential partitions; Section 4.3 shows how code moves between nodes; Sections 4.4 and 4.5 deal with timing, failure and security.

4.1 The concurrent Prolog Web

As introduced above, all the mechanisms described in the preceding chapters – spawning, sending, linking, monitoring, registering – work transparently across node boundaries. This section makes that claim concrete by introducing remote placement via the `node` option and showing how actor-style interaction spans nodes. The focus is on *where* an actor runs and how it exchanges messages across the network, but also on the safety and authority questions that network-transparent concurrency raises on an open Web.

4.1.1 Actor talking to remote actor

For specifying the node on which we wish to create an actor process, `spawn/3` can be passed the `node` option:

```
?- spawn(echo_actor, Pid, [
       node('http://n8.org'),
       monitor(true)
   ]),
   register(echo_actor, Pid).
Pid = 99231321@'http://n8.org'.
?-
```

The name `echo_actor` is a local alias in the caller's ordinary namespace for the remote pid. The send below resolves that alias locally; registration does not publish a service at `n8.org`. The monitor reports termination or connection loss, but does not withdraw the alias: the client must call `unregister(echo_actor)` explicitly.

If `n8.org` is a node conforming to the ACTOR profile, and if the node's shared database defines the predicate `echo_actor/0`, then we can have a conversation with our remote echo server like so:

```
?- self(Self),
   echo_actor ! echo(Self, hello).
Self = 71773120.
?- flush.
Shell got echo(hello)
true.
?- whereis(echo_actor, Pid),
   exit(Pid, die).
Pid = 99231321@'http://n8.org'.
?- receive({A -> true}).
A = down(99231321@'http://n8.org',99231321@'http://n8.org',die).
?-
```

4.1.2 Actors playing ping-pong while on different nodes

The actors playing ping-pong back in Chapter 2 kept their game inside one node. Passing the option `node('http://n2.org')` to one of the calls to `spawn/3` allows the game to be played between two nodes:

```
ping_pong :-
    spawn(pong, Pong_Pid, [
        node('http://n2.org')
    ]),
    spawn(ping(3, Pong_Pid)).
```

Figure 4.1 illustrates this scenario.

4.1.3 Terminal authority across node boundaries

Figure 4.1 illustrates a detail that is not obvious from the program text. The ponger runs on `n2.org`, yet its output appears in the browser terminal on `n1.org`, interleaved with output from the local pinger. This is not a broadcast of everything written on the remote node: the runtime routes terminal I/O back to the terminal from which the actor's process lineage originated.

The WebSocket arrows are transport channels, not terminal authority. A node-to-node connection carries events for many lineages and clients, so merely possessing it cannot confer permission to write to their terminals. The mechanism must identify both where terminal events return and which lineage is authorised to send them.

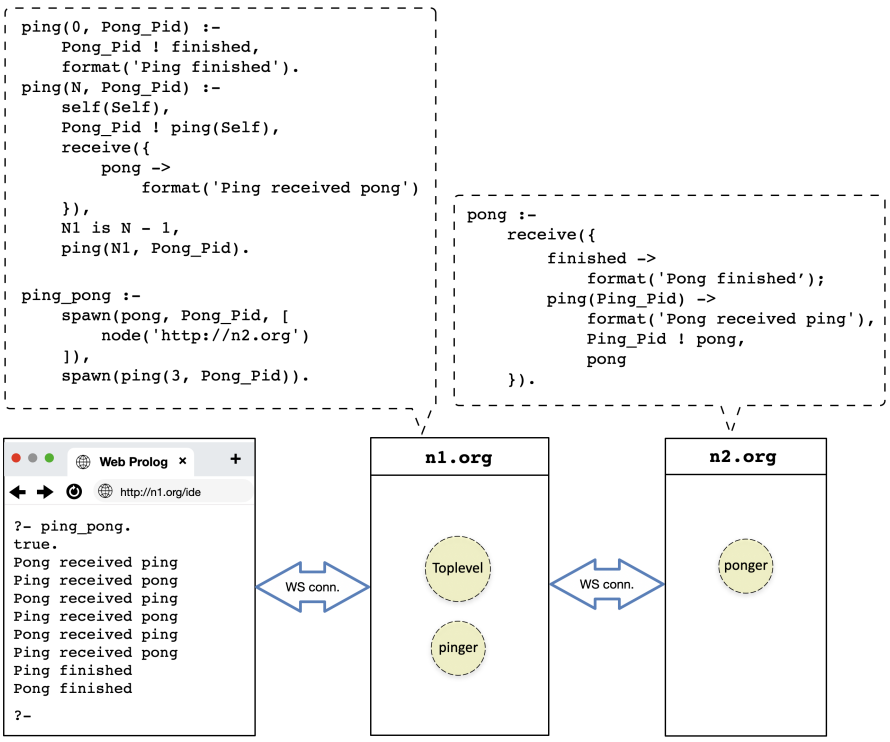


Fig. 4.1 Actors playing ping-pong while on different nodes. Output from both the local pinger and the remote ponger is returned to the terminal of the originating toplevel.

The more general rule is that terminal authority follows the process lineage, even when that lineage crosses a node boundary. A toplevel that is attached to a terminal has an I/O target. A locally spawned child inherits that target. If the child is spawned remotely, the spawning node instead supplies an opaque *terminal endpoint*, conceptually of the form

`'$io_endpoint'(Token)@HomeNode`.

The endpoint is an internal capability, not a public actor address. `HomeNode` says where terminal events must return, while the unguessable `Token` identifies the one terminal lineage to which they belong. The remote runtime installs the endpoint as the new actor's default I/O target. If that actor later spawns another actor – locally or on a third node – the same endpoint is inherited again. The rule is therefore transitive and does not depend on how many node boundaries the lineage crosses.

Box 4.1 — Erlang comparison: Group leaders and inherited I/O

Erlang provides the model for this arrangement. Every Erlang process has a *group leader*; I/O from the process group is channelled to that process, and a newly spawned process inherits the group leader of its spawner. [a] This inheritance is not merely a convention of the local implementation. In Erlang's distribution protocol, a `SPAWN_REQUEST` carries both the pid of the spawning process and the pid of the group leader to assign to the new process. [b]

Web Prolog borrows the semantic pattern, not Erlang's exact representation. An Erlang group leader is a pid inside a mutually trusting Erlang distribution domain. A Web Prolog terminal may instead be a browser connection on an open Web, so its inherited I/O destination is represented by a restricted capability that can be exercised only through authenticated node transport.

[a] See the Erlang Run-Time System documentation for `group_leader/0`: `https://www.erlang.org/doc/apps/erts/erlang.html`.

[b] See *Distribution Protocol*, under `SPAWN_REQUEST`: `https://www.erlang.org/doc/apps/erts/erl_dist_protocol.html`.

When the ponger calls `format/2`, its terminal event is sent to the inherited endpoint. The home node validates the endpoint and delivers the event to the browser connection that owns it. Input works in the opposite direction: a remote `input/2` sends a prompt through the same endpoint, and the home node grants that browser connection a one-use permission to return the answer to the prompting actor.

This I/O route is separate from ordinary actor communication. In particular, `output/1-2` sends an ordinary actor message to its explicit target or, by default, the parent; terminal writes such as `format/2` follow the inherited I/O target. Prompts from `input/2-3` also use that inherited target when one is installed, falling back to the parent otherwise. The remote actor's spawn reply, query answers, links, monitors, exit signals, and mailbox messages continue to use their normal targets. The terminal endpoint accepts only the small terminal protocol – output and prompts – and cannot be used as a general message-forwarding channel. Only remote descendants of the terminal's lineage get the authority to write to it; other remote events are not copied to the browser.

The capability also has a lifecycle. A public client cannot invent or attach one, and an internal spawn that arrives without inherited terminal authority receives a non-forwarding I/O target rather than gaining authority by default. When the browser connection closes, the home node revokes its endpoints and any unanswered prompts. Consequently, the rule can be stated without mentioning locality:

> An actor may perform terminal I/O when it possesses the terminal endpoint inherited from the toplevel lineage to which that terminal is attached.

This establishes which terminal an actor may use. The ordering of writes to that terminal requires one further part of the I/O protocol.

4.1.4 Acknowledged I/O and output ordering

Routing output to the correct terminal is only part of the problem. A remote write must also establish when the actor may continue. If a call to `format/2` returned as soon as its output had been sent over a socket, a subsequent actor message could reach its recipient before the output reached the terminal. Messages sent over different connections can take different amounts of time. Correct routing alone would therefore allow all three of the ponger's lines to appear before any of the pinger's lines.

In the ping-pong example, the resulting order is easy to follow:

1. Pong writes `Pong received ping` and waits for the write to be acknowledged before sending its `pong` reply.
2. Ping receives that reply and writes `Ping received pong`.
3. Only after that write is acknowledged does ping send the next `ping` message.

These dependencies make the received-message lines alternate, including when the actors run on different nodes. When the common terminal is hosted in SWI-WASM, the acknowledgement must come back from the browser terminal itself. Acceptance into separate queues on the remote nodes would not establish an order at that common destination.

The guarantee is limited to the order established by the program. Unrelated actors may still write in either order. In particular, ping sends `finished` before printing its own finishing line, so the two finishing lines may appear in either order. Also, `ping_pong/0` returns after spawning the actors; the shell may report `true` before their conversation ends. Ordinary actor sends remain asynchronous. Section E.3.4 describes how the runtime correlates acknowledgements and handles failures while a writer is waiting.

Box 4.2 — Erlang comparison: The I/O protocol and acknowledged writes

Erlang I/O appears synchronous to the calling process, but is implemented as a message protocol. A process sends an `io_request` and waits for the corresponding `io_reply`, with a value in both identifying the request being answered.[a]

Web Prolog adopts this request/reply principle for remote terminal writes. A call such as `format/2` waits until the common terminal service acknowledges acceptance. If the actor then sends a message, output causally triggered by that message follows the accepted write at that terminal. The acknowledgement concerns acceptance by the service, not the instant at which pixels appear. Unrelated actors may still write in either order. The surrounding ping-pong example shows exactly which ordering dependencies the program establishes.

[a] `https://www.erlang.org/doc/apps/stdlib/io_protocol.html`.

4.1.5 The node option works for toplevels too!

Toplevels are actors, so it should come as no surprise that the `node` option can also be used to create a toplevel process on a remote node:

```
?- toplevel_spawn(Pid, [
       node('http://n1.org')
   ]).
Pid = 83110912@'http://n1.org'.
?-
```

The query `?-human(Who)` executes on the remote node `n1.org`, with answers returned to the local shell:

```
?- toplevel_call($Pid, human(Who), [
       template(Who),
       limit(1)
   ]).
true.
?- receive({Msg -> true}).
Msg = success(83110912@'http://n1.org',[plato],true).
?- toplevel_next($Pid).
true.
?- receive({Msg -> true}).
Msg = success(83110912@'http://n1.org',[aristotle],false).
?-
```

The current `toplevel_abort/1` predicate does not route aborts to remote pids; such a call has no effect. HTTP and WebSocket clients can request abort at the hosting node through its protocol interface (Appendix D).

4.1.6 Uniform messaging across node boundaries

The preceding examples show that the same primitives – spawning, sending, linking, monitoring, registering – work whether the target process is local or remote. They share a programming interface and an abstract delivery model, although distribution remains visible where it affects latency, failure, authority, or ordering. This design follows the unified semantics proposed by Svensson et al. (2010): a node controller mediates side-effecting operations so that the core actor semantics do not depend on a particular transport.

Two-step messaging model

Following the *ether* abstraction of Svensson et al. (2010), messaging has two steps. A *send* creates an in-flight record in a virtual system queue; a later *delivery* appends the message to the receiver's mailbox. The ether is a semantic device, not a required implementation structure. A local runtime may deliver immediately, while a remote runtime may carry the record over a network transport (Figure 4.2).

Fig. 4.2 The two-step messaging model. Sending creates an in-flight record; delivery appends the message to the receiver's mailbox, where `receive/1-2` may consume it.

The in-flight step may be realised by a local queue, a direct WebSocket connection, an HTTP request, or a relay. In each case, ordering is defined over *delivery* into mailboxes. A *binding epoch* is a continuous period during which a name denotes the same actor, and a *routing epoch* is a continuous use of one ordered controller route. A local runtime remains in one routing epoch; a remote reconnect or reroute starts another. The message-ordering guarantees on which portable programs may rely are summarized below:

Situation	Guarantee
Same sender to the same pid within one routing epoch	Send order is preserved.
Same sender through the same name within one binding and routing epoch	Send order is preserved.
One send by pid and one by name, even if both resolve to the same actor	No ordering guarantee.
Sends separated by name rebinding	No guarantee across binding epochs.
Sends separated by reconnect or rerouting	No guarantee across routing epochs.
Sends from different senders	No ordering guarantee.

The table states the minimum ordering guarantees on which portable programs may rely. An implementation may provide stronger guarantees.

4.1.7 Remote links and monitors

Signals work as described in Chapter 2: they bypass the mailbox, and termination cannot be refused. Across nodes, links and monitors depend on what the runtime can detect. Links and monitors serve different purposes across node boundaries. A link ties a child's lifetime to its parent: when the runtime learns that the parent has

terminated, it terminates the linked child. The demonstrator also terminates actors owned by a remote client connection when that connection is detected as closed.

A remote monitor reports what the observing node can establish. If the connection to the target closes, it delivers `down(Pid,Ref,connection_closed)`. This means that observation has been lost; the target may still be running. The notification ends that monitor, and reconnecting does not restore it automatically.

These mechanisms depend on detecting termination or connection loss. The demonstrator uses a node-configured WebSocket ping/pong lease, with a 30-second silent-interval limit by default. An owner may change the limit or disable it with `ws_lease_seconds(infinite)`. With a finite lease, the transport times out when no frames arrive within that interval, provided that the runtime continues servicing the connection. This bounds detection of lost contact; it neither proves remote termination nor bounds the time needed to unwind a blocked host operation.

Confirmed termination is reported only when the hosting node reports the actor's outcome. A remote `spawn/3` that loses its transport raises the following error:

```
error(remote_spawn_failed(connection_closed), Context)
```

The remote actor may already have been created. For work with side effects, a client must treat lost contact as an unknown outcome. Retrying requires duplicate suppression, for example an application idempotency key, or an explicit protocol result recording that the outcome is unknown.

4.1.8 Containment: lifetime discipline

On an open network, a client may disconnect mid-session or disappear without warning. If server-side actors created on that client's behalf continue after the client has gone, they may keep consuming CPU and memory or sending messages into mailboxes that will never be read. Over time, such orphaned processes can accumulate into unbounded state and a potential security exposure.

The preventive measure is to make process lifetimes *containment-shaped*: every actor spawned by a client is lifetime-dependent on its parent. When parent termination or connection loss is detected, the child-hosting controller propagates termination through the linked subtree. For remote links it does so either from explicit knowledge of the parent's termination or when the connection owning the remote subtree reports close, error, or lease expiry, as described in the previous subsection. Reclamation does not depend on a final exit packet crossing a connection already known to have failed. In Web Prolog, `link(true)` is the default; creating an independent actor requires an explicit `link(false)`, and that option is reserved for node owners. A client cannot detach work from its session.

WebSocket-level root spawns have no actor parent on the receiving node; the owning connection contains them instead. This runtime operation does not grant client code permission to use `link(false)`. An owner's language-level detach

transfers ownership to the node and redirects output to its log, so a later connection sweep leaves the detached service and its linked descendants under node ownership.

Node-to-node traffic may share a connection while carrying work for many clients. Each delegated spawn therefore carries a lineage token. Ownership, visibility, and reclamation are scoped to the connection and lineage, rather than to the shared connection alone. A separate principal account token charges multiple lineages belonging to one principal to the same local resource budget. Trusted peer nodes propagate these tokens; public clients cannot choose them. Explicit parent cleanup remains necessary while the shared connection is healthy. When the last local actor in a lineage ends, the source also requests remote lineage cleanup, including work whose spawn acknowledgement has not yet arrived. For compatibility, a peer that omits the tokens receives the older per-connection scope; it cannot claim client isolation within that connection.

The demonstrator charges locally hosted actors and aggregate CPU to the principal's subtree, including children spawned by sessions. Its default limits are 32 actors per principal, 1,024 queued or deferred messages per actor, term depth 128, and 30 CPU seconds per 60-second accounting window. Actor admission is atomic and raises `resource_error(actors)` at the limit. CPU is sampled every 50 milliseconds; exhaustion terminates the principal's actors with `resource_error(cpu)` and rejects new spawns for the remaining window. Mailbox and term admission raise `resource_error(mailbox)` and `resource_error(term_depth)` respectively. These are configurable per-node budgets, not a global CPU or actor-count limit across nodes. They supplement the existing node, connection, session, and inference limits; they do not constitute a separate atom-allocation quota.

Why lifetime dependence is directional

Ownership supplies an argument for the direction of this dependence, as well as for its default. If a client terminates, its temporary workers should be reclaimed. If a worker fails, however, the client should remain able to retry, choose another provider, or report the failure. These requirements call for compulsory termination downward and optional monitoring upward. Automatically making the reverse direction fatal would impose a dependency that the application may not have. The client may decide that a worker's failure invalidates the whole computation, but that is a recovery decision.

Independent administration makes this distinction especially useful on the open Web. Suppose a client spawns workers at three different providers. If one worker's failure automatically terminates the client, downward cleanup can then destroy useful work at the other two providers. Directional links confine automatic termination to the dependent subtree, while a monitor leaves the surviving parent able to decide what to do. Applications that require fate sharing can establish it explicitly.

Remote ownership and detached work

Remote placement sharpens the point. A client may place a child on a remote node via `node(URI)`, but only under lifetime dependence. Allowing a remote detached child would turn `spawn/3` into a deployment primitive requiring a separate ownership and reclamation contract under disconnects and network partitions. For that reason, client-side remote spawn rejects `link(false)`.

A durable service should not depend on an arbitrary client's lifetime: such services belong under node-owner control, like the node-resident services described next.

4.1.9 Node-resident actor processes

In addition to predicates in the shared database, the owner of a node may install *node-resident actor processes* – long-lived actors that are intended to be durable, shared, and continuously available. They are usually of a kind that we do not want unauthorised external clients to be able to create or destroy; doing this is the privilege of the node owner. This is also where supervision trees become most useful: crash recovery for node-resident services is part of the node owner's responsibility.

A node-resident count service

We begin with a small example. Assume that the owner of the node `n8.org` starts a count actor, detaches it from the owner's own session, and publishes it under a stable name:

```
?- spawn(count_actor(0), Pid, [
       link(false)
   ]),
   register_service(counter, Pid).
Pid = 69220912.
?-
```

Two details distinguish this service from an ordinary client-created helper. The first is `link(false)`. Under the default lifetime discipline (that is, when `link` is set to `true`), the new actor would be tied to the life of the owner's shell and torn down when that session ends – exactly the right behaviour for client-created helpers, and exactly the wrong one for a service meant to outlive any particular session. Detaching an actor from the hierarchy that created it is the privilege that Section 4.1.8 reserves for node owners; here we see what the privilege is *for*.

The second detail is `register_service/2`, a publication operation distinct from ordinary `register/2`; the distinction is developed below. Publication does not itself alter an actor's lifetime: `register_service/2` neither detaches nor supervises the

actor. Lifetime management and service publication are separate owner responsibilities, and the example performs both explicitly.

A published name can be addressed from other nodes by qualifying it with the node's URI, just as pids are: where `69220912@'http://n8.org'` is the globally qualified pid of a process at `n8.org`, the term `counter@'http://n8.org'` denotes whatever process is currently published under the name `counter` there. A client on some other node can therefore interact with the service without ever learning its pid:

```
?- self(Self).
Self = 51230945.
?- counter@'http://n8.org' ! count($Self),
   receive({count(Count) -> true}).
Count = 1.
?- counter@'http://n8.org' ! count($Self),
   receive({count(Count) -> true}).
Count = 3.
?-
```

Unlike a client's helper actor, this one has a stable public name and its state is shared. The second reply is `3`, not `2`, because another client has incremented the counter in the meantime.

A node-resident publish-subscribe service

The same service perspective becomes even clearer in a publish-subscribe example. Consider the following actor:

```
pubsub_actor(Subscribers0) :-
    receive({
        publish(Msg) ->
            forall(member(Pid, Subscribers0), Pid ! msg(Msg)),
            pubsub_actor(Subscribers0);
        subscribe(Pid) ->
            pubsub_actor([Pid|Subscribers0]);
        unsubscribe(Pid) ->
            (   select(Pid, Subscribers0, Subscribers)
            ->  pubsub_actor(Subscribers)
            ;   pubsub_actor(Subscribers0)
            )
    }).
```

This actor maintains a list of subscribers. A `subscribe(Pid)` message adds a subscriber, an `unsubscribe(Pid)` message removes one, and a `publish(Msg)` message broadcasts `Msg` to all currently subscribed processes. On its own, this is just another actor definition. It becomes a service when the owner starts it as a durable, detached, node-resident process and publishes it:

```
?- spawn(pubsub_actor([]), Pid, [
       link(false)
   ]),
   register_service(pubsub_service, Pid).
```

A client can then subscribe to the service and wait for messages from it:

```
?- self(Self),
   pubsub_service@'http://n8.org' ! subscribe(Self),
   repeat,
   writeln('Waiting for a message ...'),
   receive({
       msg(Message) ->
           format('Received: ~p~n', [Message]),
           fail
   }).
Waiting for a message ...
Received: hello
Waiting for a message ...
```

Note that the subscriber list held by the actor at `n8.org` contains *remote* pids, and `Pid ! msg(Msg)` delivers across node boundaries with the same syntax as local delivery. The message `hello` was received because some client published it to the same named actor:

```
?- pubsub_service@'http://n8.org' ! publish(hello).
true.
?-
```

This example illustrates a useful contrast with the counter. The counter service exposes shared *state*: what one client observes depends on what other clients have already done. The publish-subscribe service instead exposes shared *coordination*: clients interact indirectly through a common mediating process. In both cases the essential pattern is the same – a node-resident actor with a stable public name behaves as a service.

The implementation shown here is intentionally minimal. In practice, one would monitor subscribers so that dead ones are pruned from the list, possibly persist subscriptions in a more durable store, and attach access control or topic structure. We can add supervision using the existing predicate API.

Supervising the publish-subscribe service

A supervisor can restart the service if its actor terminates. We use `supervisor_spawn/3` and `supervisor_which_children/2` as in Section 3.3.6; Section B.5 of Appendix B documents them in full.

Each replacement has a new pid, so publication must be part of the child's start goal. The owner adds this wrapper alongside `pubsub_actor/1` in the node's shared program:

```
published_pubsub :-
    self(Self),
    register_service(pubsub_service, Self),
    pubsub_actor([]).
```

The following is an alternative to the unsupervised startup above. Assume that no service is currently published as `pubsub_service`, and that the supervisor and its child run with node-owner authority:

```
?- supervisor_spawn([
       child(pubsub_service, [
           start(published_pubsub),
           restart(permanent)
       ])
   ], Sup, [
       link(false),
       strategy(one_for_one),
       intensity(3),
       period(10)
   ]).
Sup = 58310427.
?-
```

Here `link(false)` detaches the supervisor from the owner's shell. Its child remains linked to the supervisor, so stopping the supervisor also stops the service. The `permanent` policy requests a restart after either normal or abnormal child termination. The supervisor permits three restart attempts within ten seconds; if another is required within that window, it gives up and terminates.

The child identifier `pubsub_service` in the specification is used for supervisor bookkeeping. It does not publish an ordinary actor as a service; the wrapper's `register_service/2` call does that. Starting the supervisor does not itself acknowledge successful publication. Once the child has published, the owner can inspect it:

```
?- supervisor_which_children($Sup, Children).
Children = [info(pubsub_service, 9146723@, worker, permanent)].
?- whereis_service(pubsub_service, Pid).
Pid = 9146723@.
?- exit($Pid, simulated_failure).
true.
?-
```

After the supervisor has started a replacement and the wrapper has published it, the same queries show a new pid:

```
?- supervisor_which_children($Sup, Children).
Children = [info(pubsub_service, 26083951, worker, permanent)].
?- whereis_service(pubsub_service, Pid).
Pid = 26083951.
?-
```

The runtime removes the old service mapping when the child terminates. The replacement publishes itself under the same name, so clients can continue to use `pubsub_service@'http://n8.org'`. Restart and publication are asynchronous: during recovery, the child may not yet be running or the service name may not yet be published. The queries above show the state after recovery has completed.

The replacement starts with an empty subscriber list. Recovering subscriptions is the responsibility of the pubsub program, not the supervisor; that recovery mechanism is not shown here. Publications lost during the interruption are not replayed by this example.

Naming, discovery, and service publication

Clients can give their actors local names using `register/2` and look them up with `whereis/2`. These names belong to the caller's execution namespace, inherited by its spawned actors, so separate client namespaces can use the same name. To make an actor available as a public service, the node owner instead uses `register_service/2`. Only the owner can publish or withdraw a service name.

A published service has an address of the form `Name@Node`. Our counter is addressed as `counter@'http://n8.org'`. The receiving node looks up the name in its public service registry and routes the message to the actor without revealing its pid. This lookup does not include names registered with `register/2`.

A successful send does not confirm delivery. If the service is absent or the sender lacks permission to use it, the send succeeds without delivering the message. A client that needs confirmation must wait for a reply or acknowledgement.

Box 4.3 — Erlang comparison: Pids, registered names, and services

Erlang and Web Prolog both allow processes to be addressed by pid or registered name. A pid identifies one actor instance; a name can be registered again for a replacement actor.

Web Prolog's published service names extend this indirection across nodes. A supervisor can publish each replacement under the same `Name@Node`, so clients need not learn its new pid (van Steen and Tanenbaum, 2023). The address still contains the node URI, however. Moving the service to another node requires a directory, forwarding entry, stable front-end URI, or similar arrangement.

Publishing a service makes it addressable, but does not tell potential clients that it exists. A directory can advertise available nodes and their services. The following example shows one such directory.

The Discovery Hub

The proof-of-concept deployment includes a Discovery Hub at `https://n0.elfenbenstornet.se/discovery-hub`, itself implemented as the Prolog node `N0`. The hub maintains a directory of the registered demonstrator nodes and periodically probes each one. It publishes the resulting information – including current reachability, advertised profile, authorisation mode, database portability, and provided predicates and services – as owner-defined relations over the stateless `/call` API. The filters in the browser interface are translated into ordinary Prolog goals and evaluated against these relations.

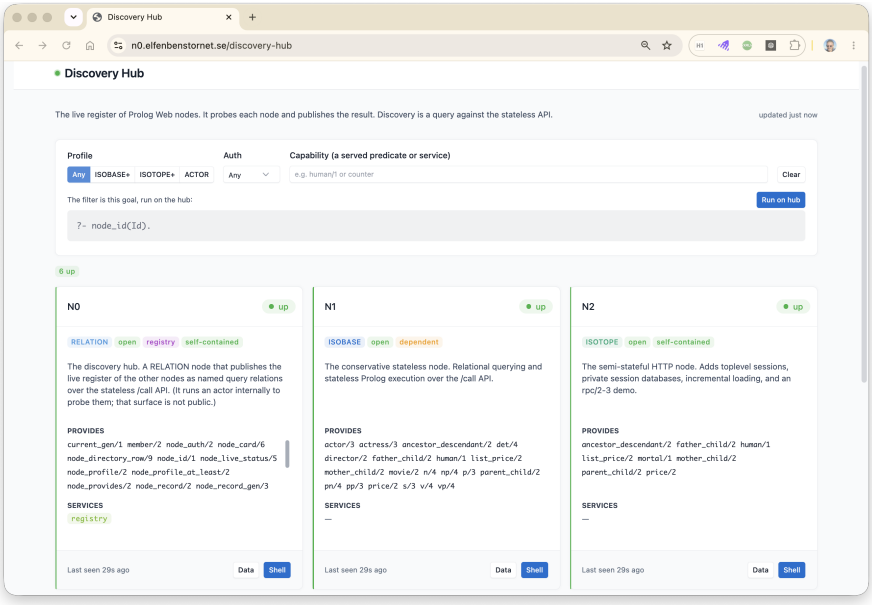


Fig. 4.3 The Discovery Hub at node `N0`. The browser interface queries a live directory of demonstrator nodes by profile, authorisation mode, and advertised predicate or service. Although `N0` uses an actor internally to probe the registered nodes, it exposes the directory through a stateless RELATION interface.

Node `N0` advertises the RELATION profile. Internally, however, it uses an actor to perform the periodic probes. That actor and the corresponding actor operations are not exposed to clients. The hub therefore provides a compact illustration of a profile distinction: a profile describes a node's observable contract, not everything used to

implement it. It also demonstrates that useful ecosystem infrastructure, including node discovery, can be constructed from ordinary relations and the stateless Web Prolog API.

The hub should not be understood as a mandatory central authority for the Prolog Web. It is one possible directory implementation: other hubs may coexist, participation may be voluntary, and applications may use owner-maintained peer information or other discovery mechanisms instead.

Service discovery is an owner-curated form of advertisement, distinct from publication itself. A call to `register_service/2` makes a name addressable in the public service namespace; it does not expose a client-queryable enumeration of that namespace. A node may instead advertise selected services through owner-maintained metadata, while leaving other published services unadvertised.

This separation is a design invariant. If clients could enumerate the service registry or all running processes, discovery would collapse into introspection and name discipline would be lost. Ordinary client naming operations must likewise neither expose nor remove published services. The security implications are developed in Section 4.5.2.

In the Trinity, such an advertisement can be expressed as an ordinary relation in the node's shared database. For example:

```
service(counter, meta(actor, protocol(count_v1))).
service(pubsub_service, meta(actor, protocol(pubsub_v1))).
```

The relation is maintained by the owner and may advertise all published services or only a selected subset. The owner is responsible for keeping its entries consistent with the services actually published. Richer metadata – versioning, authorisation requirements, or documentation URIs – can be represented by extending the second argument.

A client discovers advertised services with an ordinary goal-oriented query:

```
?- rpc('http://n8.org', service(Name, Meta)).
Name = counter,
Meta = meta(actor, protocol(count_v1)) ;
Name = pubsub_service,
Meta = meta(actor, protocol(pubsub_v1)).
?-
```

Discovery therefore remains declarative and compositional: clients can enumerate advertised services, filter by protocol, or combine discovery with other constraints. Because `service/2` is owner-controlled metadata rather than an enumeration of the service registry, the node reveals only what it has chosen to advertise and may apply ordinary authorisation policy to the query.

4.2 The sequential Prolog Web

Often a program needs another node to solve a goal and return its solutions. The sequential Prolog Web supports this through `rpc/2-3`, which presents remote execution as a synchronous, nondeterministic Prolog call. The predicates `promise/3-4` and `yield/2-3` separate starting a remote call from collecting its result, allowing the caller to do other work while it runs. Both interfaces let the programmer work with goals and answers without managing remote actors or mailbox protocols. We begin with `rpc/2-3`, then examine solution paging and asynchronous calls using promise and yield.

4.2.1 Nondeterministic remote procedure calls

The meta-predicate `rpc/2-3` calls a goal in the Prolog context of another node, using the programs and data available there. It is synchronous: the caller waits while remote execution supplies answers. It is also nondeterministic: the caller can obtain successive solutions through ordinary Prolog backtracking.

A Web Prolog client performs a remote call by providing a URI identifying the node and a goal to be solved in that node's context. The optional third argument is a list of options. Here is a simple example:

```
?- rpc('http://n1.org', human(Who)).
Who = plato ;
Who = aristotle.
?-
```

Each solution binds variables in the caller's goal. Backtracking requests another solution; when no further solutions exist, the call fails. An exception raised by the remote goal is propagated to the caller.

The caller sees solutions one at a time, but they may be retrieved from the remote node in batches. Without a `limit` option, all solutions are requested in one exchange.

Using rpc/3 with the limit option

The `limit` option of `rpc/3` controls the number of solutions retrieved per network roundtrip. Passing `limit(K)`, where K is a positive integer, requests slices of at most K solutions. Execution-time and result-size bounds still apply, as discussed in Section 4.2.4. Here is what happens if we request smaller slices with `limit(1)`:

```
?- rpc('http://n1.org', human(Who), [
       limit(1)
   ]).
Who = plato ;
```

```
Who = aristotle.
?-
```

From the caller's perspective, the result is identical. What changed is invisible: instead of retrieving both solutions in a single roundtrip, the call now required *two* – one per solution. In this case, the option adds cost without benefit. Section 4.2.3 shows when it helps.

Slicing also decides what the caller sees when the remote goal raises an exception after some solutions. With a `limit`, the solutions found before the exception are delivered first, and the exception is raised when backtracking asks for more, just as in a local call. Without a `limit`, the whole answer is computed in one exchange, and the exception is reported instead of any solutions.

Other options include `src_*` for supplying source, `timeout` for bounding remote execution, and `once` for requesting only the first slice. See Appendix B for the complete option reference.

The semantics of rpc/2-3

For intuition it is useful to note that a call to `rpc(URI,Goal)` is semantically equivalent to the following pattern, which makes explicit that solutions arrive as instantiations of a copy that is then unified back into the original goal:

```
copy_term(Goal, Copy),
call(Copy),           % executed on node at URI
Goal = Copy.
```

The key implication is that no variables are shared across nodes: the remote execution operates on a renamed-apart copy, and only the induced substitution is returned to the caller via unification with the original goal.

This three-line characterisation is intended as a *logical* specification of `rpc/2-3`: it defines what answers the caller may observe, not how those answers are obtained. Web Prolog restricts the answers of remote goals to plain terms. An answer whose variables still carry constraints, such as those left by `dif/2` or CLP(FD), is not delivered without them: the call raises `error(representation_error(constraints),_)` instead, so that a conditional answer is never presented as an unconditional one. The serialisation protocol also ensures that bindings returned to the caller are always meaningful in the caller's context, ruling out node-local references such as stream handles or database pointers. What the characterisation abstracts over is the operational machinery required to deliver those answers across a network: the creation of a remote process, the protocol that sustains nondeterminism across multiple solutions, and the handling of timeouts, exceptions, and partial failure.

A notable property of `rpc/2-3` is that it preserves the logical purity of the goal it calls: if the goal is pure, then the answers returned by `rpc/2-3` are exactly the logical consequences of the remote database – the network introduces no additional

bindings, side effects, or observable state changes.[2] This holds at the level of answer substitutions, assuming successful communication; in practice, timeouts, connection failures, and resource limits may prevent all answers from being delivered, but they do not corrupt the answers that are delivered.

4.2.2 Two operational realisations

The logical specification leaves transport and process management open. One way to realise it is to spawn a toplevel actor on the target node, submit the goal using PTCP, and collect answer terms. A `success/3` answer term contains a solution slice. The caller enumerates that slice locally and requests another only when backtracking exhausts it. In this way, a sequence of deterministic protocol exchanges is presented as ordinary Prolog nondeterminism.

The demonstrator instead implements `rpc/2-3` over the stateless HTTP API. The same slice-at-a-time structure applies, but each exchange is a self-contained request to the remote node. No client-addressed actor is required, and the remote node need not expose its programmable ACTOR layer. This makes the sequential abstraction available between ISOBASE nodes and allows it to pass through ordinary HTTP infrastructure.

A stateless node may cache suspended computations to avoid repeating work; Section 4.2.4 explains resumption, cache eviction, and their consequences for paging consistency.

The two realisations therefore expose the same answers while assigning transport, failure detection, and resource ownership differently. Chapter 5 implements both.

4.2.3 Browser talking to a node talking to a node

In the scenario depicted in Figure 4.4, a user of a Prolog terminal running in a browser interacts with a remote node on the Prolog Web, which in turn interacts with another node.

The solutions shown in the terminal follow from the *combination* of two programs: the code on the upper left in Figure 4.4 belongs to `n2.org`, and it contains a URI pointing to the code offered by `n1.org`. The browser-to-node interaction as well as the node-to-node interaction take place over HTTP and are stateless in the sense that each request can be understood and processed without relying on server-side session state.

To retrieve the first solution to `?-mortal(Who)`, the browser issues a request to `n2.org` asking for exactly one solution:

```
GET http://n2.org/call?goal=mortal(Who)&limit=1
```

[2] This is a property it shares with the `call/N` family of Prolog predicates.

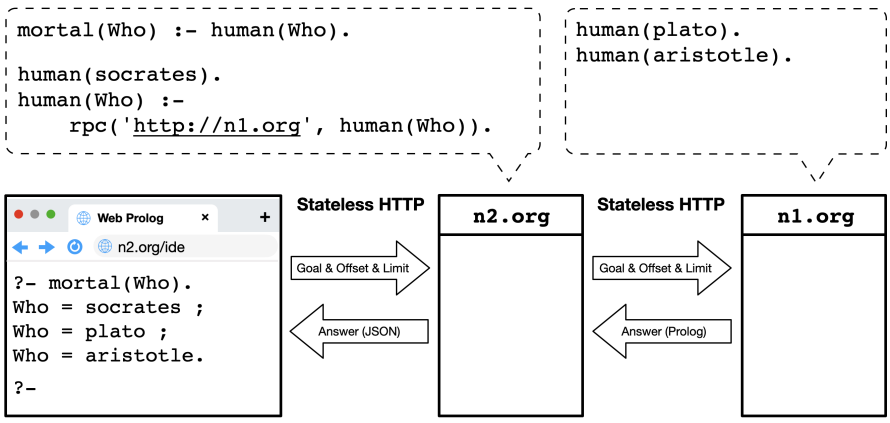


Fig. 4.4 Stateless interaction in a scenario involving a web browser and two remote nodes.

Since the user requests solutions one-by-one, the response contains a single binding together with a flag indicating that more solutions exist:

```
{ "type": "success",
  "data": [{"Who":"socrates"}],
  "more": true }
```

When the user types a semicolon, the browser requests the next solution explicitly:

```
GET http://n2.org/call?goal=mortal(Who)&offset=1&limit=1
```

In this example, producing that next solution requires `n2.org` to consult `n1.org`. Concretely, `n2.org` evaluates `mortal/1` and, as part of doing so, triggers an `rpc/2-3` call for `human(Who)` against `n1.org`. Operationally, this involves an HTTP request to `n1.org` such as:

```
GET http://n1.org/call?goal=human(Who)&format=prolog
```

Because of the default values of the corresponding slicing parameters, the response in this case contains a complete slice with all solutions to `human(Who)` currently returned in one roundtrip. Since the URI specified `format=prolog`, this was the result:

```
success([human(plato),human(aristotle)], false)
```

At this point the implementation of `rpc/2-3` on `n2.org` must convert the returned slice into the intended nondeterministic behaviour. The usual mechanism is to keep the slice locally and enumerate it using `member/2`. This yields solutions to `human(Who)` on backtracking without further network communication, until the slice is exhausted.

Since the browser client still wants only one solution at a time, `n2.org` returns only the first derived solution, `Who=plato`, to the browser:

```
{ "type": "success",
  "data": [{"Who":"plato"}],
  "more": true }
```

When the user again types a semicolon, the browser requests another solution to `mortal(Who)`. In this particular run, no additional request to `n1.org` is needed: `n2.org` can simply backtrack within the local `member/2` enumeration of the slice already obtained from `n1.org`. The browser request therefore looks like:

```
GET http://n2.org/call?goal=mortal(Who)&offset=2&limit=1
```

and the response contains the next binding, `Who=aristotle`, now with `more:false`:

```
{ "type": "success",
  "data": [{"Who":"aristotle"}],
  "more": false }
```

This completes the description of what happens "under the hood" in the scenario of Figure 4.4: the browser drives one-solution-at-a-time enumeration through stateless HTTP requests to `n2.org`, while `n2.org` uses `rpc/2-3` to consult `n1.org` and reconstructs nondeterminism locally from solution slices.

Large result sets and the `limit` option

If `n1.org` had millions of clauses of `human/1`, returning a huge slice in one response would be undesirable. In that case `n2.org` can pass a smaller `limit` to `rpc/2-3` to retrieve answers in manageable batches:

```
human(Who) :-
    rpc('http://n1.org', human(Who), [
        limit(1000)
    ]).
```

This avoids swamping the caller with data. A goal with very many solutions will be delivered incrementally over multiple roundtrips, but since `rpc/2-3` is nondeterministic, the call remains logically complete provided the caller keeps asking for more.

The same principles apply to many nodes: stateless requests, small slices, and no unnecessary recomputation.

4.2.4 Backtracking as paging

Section 3.3.3 introduced the `offset` and `limit` options for a single toplevel. Here we use them across repeated requests. Prolog backtracking and web paging share

an abstract idea: incremental enumeration under the caller's control. Backtracking requests another solution; paging requests another slice of solutions. By collecting solutions into slices, an RPC implementation lets the caller backtrack locally within each slice and makes another network request only when that slice is exhausted.

Three ways to retrieve successive slices

The stateless HTTP API identifies a slice using `offset` and `limit`. For example:

```
GET /call?goal=hotel(Name,paris,Rating)&offset=20&limit=10
```

This requests up to ten solutions, starting with solution 21. The offset identifies a position in the answer sequence; it does not prescribe how the node reaches that position. We can distinguish three retrieval strategies, the first two implementing the same stateless API:

1. **Re-execution.** The node starts the goal again, skips the first `offset` solutions, and returns the requested slice. It need not retain a computation between requests, but repeats the work needed to reach later slices.
2. **Cached continuation.** The node retains a suspended toplevel computation. A later request can resume it when the goal, offset, and other relevant request context match. The client still requests a slice by position and has no guarantee that the cached computation survives until the next request.
3. **Explicit session.** The client addresses a particular toplevel actor through the semi-stateful API. Successive `next` requests resume its current computation without specifying a new offset, although the initial call may use one. The actor may also retain private definitions and database updates across successive goals.

The last two strategies use the runtime's retained execution state, including choice points, to continue the search. An explicit session additionally gives the client an address for that actor. Its pid identifies the session, not a particular choice point: the same actor can finish one query and accept another. If the session is lost, that loss must be reported rather than silently replacing it with a fresh computation.

Consistency and the update view

Re-executing a goal for each page can produce duplicates, omissions, or changes in solution order if the underlying data changes between requests. Resuming a cached computation avoids replaying earlier search and retains the choice points of calls already in progress.

Under Prolog's *logical update view*, each invocation of a dynamic predicate sees the clauses that existed when it began (Lindholm and O'Keefe, 1987). That view is preserved when the invocation resumes across solution slices. It does not, however, give the whole query a database snapshot: a predicate first called after resumption may see intervening updates.

For example, suppose the node's shared database contains the following dynamic predicate, whose solutions are enumerated in clause order:

```
:- dynamic p/1.
p(a). p(b). p(c).
```

A call of `p(X)` with `limit=1` first returns `a`. The next stateless request uses `offset=1`; a session client instead requests `next`. If the owner retracts `p(a)` between pages, fresh evaluation enumerates `b,c`: offset 1 skips `b` and returns `c`. A retained computation continues the original invocation of `p/1`, whose clause view still includes all three facts, and returns `b`. Table 4.1 compares these outcomes, assuming each request reaches the node.

After the first answer	Re-execution	Retained computation	Explicit session
Code and data unchanged	`b`	`b`	`b`
Owner retracts `p(a)`	`c`	`b`	`b`

Table 4.1 The next answer under re-execution, retained computation, and an explicit session.

Cached computations may be evicted (Section E.2.1). A subsequent page request then restarts the goal and skips earlier solutions, losing the old computation's clause views and search state. In the table's second row, it would therefore return `c` instead of `b`.

Clients should rely on positional paging only when the goal's solution sequence remains stable across requests. Re-execution repeats work that resumption avoids and may produce different answers if the goal's dependencies have changed. A stronger snapshot guarantee requires an additional mechanism and must specify what happens if the cached computation is lost.

Many uses of the stateless API avoid these paging complications. For example, the owner may guarantee that the shared database remains unchanged. With fixed source code and no other changing dependencies, a goal that enumerates a stable solution sequence can be paged predictably.

There is no continuity between pages to preserve when the client obtains everything it needs in one request. Omitting `limit` requests all solutions, without a bound on their number. The node still enforces owner-configured bounds on execution time and result size. Exceeding either bound produces an error rather than a truncated success response. The client may then retry with a finite `limit` to retrieve solutions in smaller slices.

A smaller slice does not guarantee success: computing even the first solution may exceed the time bound, or one solution may exceed the size bound. Retrying starts the computation again, so work already performed may be repeated. Alternatively, `once(true)` ends enumeration after the first slice; combined with `limit(1)`, it requests at most one solution. This is useful when that slice is all the client needs, rather than when every solution must be retrieved.

Source loading and temporary state

A stateless call may supply source code for its temporary execution context. For example:

```
?- rpc('http://n8.org', solve(X), [
       src_uri('http://n5.org/planner'),
       limit(5)
   ]).
```

In the demonstrator, the calling runtime fetches the source from `n5.org` and sends its text to `n8.org`, where the goal runs. `promise/3-4` prepares source in the same way, before returning its reference. Remote `spawn/3` and `toplevel_spawn/2` instead send the URI to the destination node, which fetches the source before starting the actor. Relative source URIs are resolved in the fetching node's context, under its source-access policy; an absolute HTTP(S) URI makes the source location explicit.

Source supplied with a stateless request belongs to that request's execution context. Retaining the computation may avoid loading it again, but clients cannot rely on that retention. After eviction, a later page request may reload the source and restart the goal. The retained context thus includes loaded code and private data as well as the state of the search.

If the source or its dependencies have changed, the reconstructed computation may produce a different solution sequence. A client that needs to continue using a particular private context should use an explicit session and handle its possible expiry or termination.

Caching and the Web

An execution cache retains a suspended computation; an HTTP cache stores a response that may be reused under HTTP caching rules. A response may be cacheable whether its solutions were obtained by replaying a goal or resuming a retained computation.

Neither form of caching establishes a snapshot across successive pages. HTTP cacheability depends on the response and its caching policy, while continuity of execution depends on retaining the computation or explicitly maintaining a session. The HTTP method, representation, and cache-control requirements are discussed in Section D.4.1 of Appendix D.

4.2.5 Programming with `rpc/2-3`

Because `rpc/2-3` is a simple, synchronous, nondeterministic interface, it is easy to build small programming schemes on top of it. We sketch a few patterns here.

Parallel RPC calls

Two independent RPC calls can run sequentially:

```
?- rpc('http://n1.org', foo(X)),
   rpc('http://n2.org', bar(Y)).
```

On an ACTOR caller node, `parallel/1`, introduced in Section 3.3.5, lets their execution overlap:

```
?- parallel([
       rpc('http://n1.org', foo(X)),
       rpc('http://n2.org', bar(Y))
   ]).
```

The caller still waits for both goals to succeed. If the remote nodes proceed independently, the waiting time is approximately the longer of the two response times rather than their sum. The workers are local; the RPC calls they execute reach the remote nodes.

Unlike the sequential conjunction, this call commits to the first solution of each goal. To obtain all solutions as two lists, make collection part of each remote goal:

```
?- parallel([
       rpc('http://n1.org', findall(X, foo(X), Xs)),
       rpc('http://n2.org', findall(Y, bar(Y), Ys))
   ]),
   append(Xs, Ys, Solutions).
```

If the caller needs to do other work before collecting results, or handle each outcome separately, promise/yield provides that control (Section 4.2.6).

Remote modules

Below we sketch an implementation of *remote modules*.[3]

This implementation installs local wrapper clauses using `assert/1`, so it requires a calling context that permits dynamic database updates, such as an ISOTOPE session or an ACTOR process.[4] The remote node may still provide only ISOBASE: it receives ordinary RPC calls. An ISOBASE caller can instead use wrapper clauses supplied as source.

---

[3] What we here call *remote modules* is similar in spirit to Ciao Prolog's *active modules*. The operator `as` is local notation for this sketch, declared by `:- op(700, xfx, as).`; it is not part of Web Prolog's syntax.

[4] For ISO portability, the predicates receiving the asserted wrapper clauses also need appropriate `dynamic/1` declarations; this sketch omits declaration machinery.

```
:- op(700, xfx, as).

use_remote_module(URI, ImportList, Options) :-
    maplist(import(URI, Options), ImportList).

import(URI, Options, Functor1/Arity as Functor2) :- !,
    functor(Head1, Functor1, Arity),
    Head1 =.. [Functor1|Args],
    Head2 =.. [Functor2|Args],
    assert((Head2 :- rpc(URI, Head1, Options))).
import(URI, Options, Functor/Arity) :-
    functor(Head, Functor, Arity),
    assert((Head :- rpc(URI, Head, Options))).
```

Thanks to `src_*` options, an import directive can define local wrappers even for predicates not directly exported by the remote node, provided they can be expressed in terms of predicates that are exported.

Global goals

Our second example takes a step towards even more network transparency and is inspired by (Loke, 2006). The idea is to support *global goals* of the form `?- *Goal`, where a local peer database maps goal patterns to peers and options:

```
peer(foo(_), 'http://n1.org', [limit(10)]).
peer(bar(_), 'http://n2.org', [timeout(1),limit(100)]).
peer(bar(_), 'http://n3.org', [timeout(3),limit(100)]).
peer(baz(_), 'http://n3.org', [timeout(1),limit(50)]).
```

Given such a database,[5] `*/1` can be implemented like so:

```
:- op(100, fx, *).

* Goal :-
    peer(Goal, Peer, Options),
    catch(rpc(Peer, Goal, Options),
          error(resource_error(time), _), false).
```

A registry-based variation can be obtained by retrieving `peer/3` information remotely:

```
* Goal :-
    rpc('http://registry.pl', peer(Goal, Peer, Options)),
    catch(rpc(Peer, Goal, Options),
          error(resource_error(time), _), false).
```

[5] We make no assumptions on how the peer database is updated.

4.2.6 Promise and yield over stateless HTTP

The `rpc/2-3` predicate is synchronous: the caller waits until the remote node returns an answer. For many applications this is fine, but sometimes a client wants to initiate a remote computation, continue with other work, and collect the result later. The `promise/3-4` and `yield/2-3` predicates support this split-phase pattern. They do not use the semi-stateful HTTP API: `promise/3-4` initiates an asynchronous call through the stateless HTTP API, while `yield/2-3` waits on a local reference for the answer to that call.

A call to `promise/3-4` starts a query to a remote node and returns without waiting for its answer, handing back a reference that identifies the pending computation. In the demonstrator, source preparation, including any `src_uri/1` fetch by the caller, finishes before the reference is returned:

```
?- promise('http://n1.org', human(Who), Ref).
Ref = 19303011.
?-
```

The remote node begins evaluating `human(Who)`. Meanwhile, the caller is free to do other work. When it's ready for the answer, it calls `yield/2-3` with the reference:

```
?- yield($Ref, Answer).
Answer = success([human(plato), human(aristotle)], false).
?-
```

If the answer has already arrived, `yield/2-3` returns immediately. If not, it waits. The native SWI-Prolog demonstrator retains promises in a runtime-wide reference table and does not restrict collection to the issuing process; collection across separate stateless HTTP requests is also exercised by its tests. The portable ownership contract remains unresolved, so transfer between processes or requests is implementation-specific. A successful collection consumes the single answer. A wait timeout leaves the promise available for another attempt, provided its reference has not expired. These operations do not require the caller's actor mailbox. In the native demonstrator, request and response-processing errors are delivered as `error(Error)` answers. Both yield variants throw `error(existence_error(promise,Ref),_)` for an unknown, consumed, or expired reference. Appendix B records the current lifecycle limitations.

The familiar options carry over: `template/1` controls which bindings are returned, while `offset/1` and `limit/1` control slicing. By default, `promise/3-4` requests all solutions; with `limit(K)`, the answer contains at most K solutions, allowing large result sets to be fetched in batches. Again, it makes sense to think of these batches as *pages* of an answer stream: `limit(K)` sets a page size, and `offset` selects which page to retrieve next.

Overlapping remote waits

The split-phase nature of `promise/3-4` and `yield/2-3` also makes it possible to overlap the waiting time for several remote computations. The promises must be issued before yielding on any of them:

```
?- promise('http://n1.org', foo(X), R1),
   promise('http://n2.org', bar(Y), R2),
   yield(R1, A1),
   yield(R2, A2).
```

As with the parallel RPC calls in Section 4.2.5, initiating both computations before waiting lets their execution overlap. If the remote nodes proceed independently, the elapsed time is approximately the longer of the two response times rather than their sum.

The interfaces give the caller different control. `parallel/1` waits for all goals to succeed, returns their first-solution bindings, and fails or throws if a worker fails or throws. Promise/yield instead returns an answer term for each pending call, including its solution slice, failure, or error. The caller can do other work before collecting results, bound an individual wait, and decide how to combine the outcomes. It need not impose an all-or-nothing success condition. Promise/yield also belongs to ISOBASE, so it does not require the caller to offer the ACTOR profile.

This distinction also clarifies the issue of variable sharing. A parallel conjunction normally requires its component goals to be independent, in particular not to share variables whose bindings make the goals mutually dependent. The `promise/yield` pattern does not impose that restriction in the same way, because the promised computations are not parts of one live conjunction. Variables occurring in several promised goals should not be understood as shared variables spanning the remote computations. Each promise sends its own query, and each answer is later returned as data. Any consistency condition between the answers must therefore be imposed explicitly by the caller after collection.

For example, a caller that wants to combine two independently computed pages can do so locally:

```
?- promise('http://n1.org', foo(X), R1),
   promise('http://n2.org', bar(Y), R2),
   yield(R1, success(Page1, false)),
   yield(R2, success(Page2, false)),
   append(Page1, Page2, Solutions).
```

This is often the right design for web-facing computation: remote nodes compute independently, answers are returned as ordinary terms, and the caller decides how to compare, combine, or discard them.

Thus, `parallel/1` and `promise/yield` both allow waiting time to be reduced, but they express different ideas. The former is a parallelised conjunction with all-or-nothing success semantics. The latter is an asynchronous, split-phase interaction pattern with per-call answer handling.

Bounded waiting and timeouts

A client that cannot afford to block indefinitely can bound the wait performed by `yield/3`:

```
?- promise('http://n1.org', expensive_query(X), Ref),
   yield(Ref, Answer, [
       timeout(2.0),
       on_timeout(Answer = timeout)
   ]).
Answer = timeout,
Ref = 48468904.
?-
```

If no answer arrives within two seconds, the `on_timeout/1` goal is called once in the caller's context and `Answer` is bound to `timeout`. This is also the default result when no callback is supplied, or when it leaves `Answer` unbound. A failing callback raises an explicit error; a callback exception propagates. Timing out does not cancel or consume the promise: the caller may invoke `yield/3` again with the same reference and collect the answer if it has since arrived.

A polling pattern therefore emerges naturally. The following code initiates an asynchronous call and periodically checks for its answer while doing other work:

```
?- promise('http://n1.org', (sleep(1), X=a), Ref),
   repeat,
     yield(Ref, Answer, [
         timeout(0.3),
         on_timeout(Answer = timeout)
     ]),
     (   Answer == timeout
     ->  writeln('Still waiting...'),
         writeln('working...'),
         sleep(0.1),
         fail
     ;   !
     ).
Still waiting...
working...
Still waiting...
working...
Answer = success([(sleep(1),a=a)], false),
Ref = 13936890.
?-
```

Each iteration waits briefly for the answer. On timeout, the caller does some work and backtracks to try again. The binding `Answer = timeout` is undone by that backtracking, while the pending promise remains available. Once the answer arrives,

the cut terminates the polling loop. The precise number of polling iterations depends on scheduling and network delay.

This is an explicitly operational pattern, but it remains sequential from the caller's perspective: at each step the caller either receives the answer or decides what to do after a bounded wait.

4.3 The programmable Prolog Web

Programmability is a separate design dimension from the sequential and concurrent partitions. A client may supply source to a toplevel and query it through a sequential interface, or supply source to an actor and interact with it through asynchronous messages. In either case, code shipping changes the program available at the execution site; the interface determines the interaction contract. The term *programmable Prolog Web* therefore describes a capability shared across the two partitions, rather than a third partition.

The Prolog Web is programmable in a stronger sense than the familiar "Web of APIs". In its original form as a hypertext Web of documents, the Web was not itself a programmable execution substrate. This changed early with server-side scripts that generated documents dynamically and with client-side JavaScript. Later, remote procedure calling and open web APIs made it natural to treat the Web as a platform for composition, where independently developed services could be combined into "mashups", often using a client language as glue.

Web Prolog supports this conventional style of integration, but also generalises it. A node may offer clients not merely a fixed set of endpoints, but a Web Prolog runtime environment that can be programmed directly by queries of varying complexity and, when permitted, by the injection of source code into long-lived processes on the node. In addition, a node owner may install node-resident predicates that become part of what clients can call. In this sense, the Prolog Web is programmable both from the outside, by clients issuing queries and spawning processes, and from the inside, by node owners shaping the execution environment with which clients interact.

This dual perspective aligns naturally with the actor model. An actor may begin with a private program supplied through the `src_*` options and subsequently acquire behaviour through interaction. Its placement determines the host policy, resource limits, and trust assumptions under which that behaviour executes.

Once code and data can cross node boundaries in either direction, a central systems question arises. Should a computation be moved to where the data resides, or should data, including code represented as data, be imported to where the computation runs? This question has no single correct answer. The optimal direction depends on data volume, update frequency, reuse patterns, latency requirements, and trust. The Prolog Web therefore does not enforce a fixed architecture, but instead exposes a small set of composable mechanisms that let programmers choose, per interaction, how code and data should flow.

The owner's side of this exchange was illustrated in Section 4.1.9: installing predicates in the shared database and running node-resident actors that define what clients can call. The remainder of this section focuses on the client's side: how the `src_*` options let a client determine not only *where* to run, but *what* to run.

4.3.1 Injecting code with the `src_*` options

Looking back at the ping-pong scenario in Section 4.1.2, what if the definition of `pong/0` is available at `n1.org`, but not at `n2.org`? The `src_predicates` option ensures that the appropriate source code is shipped from the caller's database and injected into the private database of the actor that will run at `n2.org`:

```
ping_pong :-
    spawn(pong, Pong_Pid, [
        node('http://n2.org'),
        src_predicates([pong/0])
    ]),
    spawn(ping(3, Pong_Pid)).
```

The `src_predicates` option names predicates that already exist in the caller's context. When the clauses themselves should be supplied literally, the `src_list` option serves that purpose. The following example uses it to populate a remote toplevel actor's private database with a clause that participates in the query:

```
?- toplevel_spawn(Pid, [
       node('http://n1.org'),
       src_list([(mortal(Who) :- human(Who))])
   ]).
Pid = 68427109@'http://n1.org'.
?-
```

The query `?-mortal(Who)` now executes on `n1.org`, combining the injected rule with the node's own `human/1` facts:

```
?- toplevel_call($Pid, mortal(Who), [
       template(Who)
   ]).
?- receive({A -> true}).
A = success(68427109@'http://n1.org',[plato,aristotle],false).
?-
```

4.3.2 Code shipping and data locality

The most direct way to bring code to the data in Web Prolog is to inject a small, task-specific predicate into a process placed near the data, execute it there, and return only the derived result. This corresponds closely to the classical systems idea of *function shipping*: rather than transferring a large dataset across the network, one ships a small computation to where the data resides and retrieves only a compact summary.

Conversely, remote facts or rules can be imported into a local process and reused there. Since the default value of the `node` option for `spawn/1-3` and `toplevel_spawn/1-2` is the special-purpose URI `localhost`, the same `src_*` mechanisms used for remote execution also support local composition. Code can therefore flow from client to node or from node to client, with applications choosing the direction at runtime according to latency, data size, or load. This reflects Prolog's natural blurring of the distinction between code and data: facts and rules can be moved, cached, and reused much like datasets.

4.3.3 A small design space

Taken together, the mechanisms described in this chapter form a small but expressive design space structured around a handful of recurring choices:

- **Interaction model**: one-shot remote procedure call, interactive toplevel, or actor-style orchestration (`rpc`, `toplevel_spawn`, or `spawn`)?
- **Placement**: locally or remotely (`node`)?
- **Program source**: from the caller's database, supplied literally, or already resident on the target node (`src_*`)?
- **Synchrony**: should the caller block, or initiate work and collect the result later (`promise/yield`)?
- **Lifetime**: one-shot request–response or session-oriented exchange?

Code shipping is also the point where portability stops being an academic concern and becomes operational. If nodes can execute code originating elsewhere, then even small semantic differences or library mismatches become visible immediately. For this reason, the Prolog Web treats mobile code as profile-scoped: a piece of code should state, implicitly or explicitly, which profile it assumes, so that execution is either well-defined or rejected early.

When a program is self-contained – all predicates on which it depends are either included or guaranteed by the target profile – it can be downloaded for local execution or shipped to another node without adaptation. We call this property *instant portability*: profile-scoped code, symmetric code flow, and uniform local and remote execution reduce relocation to a deployment decision rather than a porting effort.

4.4 Timing and failure boundaries on the Prolog Web

Distributed execution makes the duration and outcome of waiting less predictable. A goal that would ordinarily terminate promptly under local execution may, when it depends on a network or a remote node, fail to produce a result within an acceptable time. A timeout does not establish that the goal itself has failed; it marks the point at which waiting ends and the absence of a result becomes visible to the programmer.

In the Prolog Web architecture, two transports are used – HTTP and WebSocket – but three Web APIs are exposed: a stateless HTTP API, a semi-stateful HTTP API, and a stateful WebSocket API. Together they expose several blocking surfaces. In addition, `promise/3-4` and `yield/2-3` split submission from collection at the language level even though the underlying remote call uses the stateless HTTP API. Timeouts attach to operations at these surfaces, and their meaning and appropriate values depend on the interaction model.

Box 4.4 — Context: Network transparency has limits

Web Prolog gives local and remote actors the same basic vocabulary: pid-or-name addressing, sends, selective receive, monitors, and lifetime operations. The programming vocabulary is transparent; timing, delivery, and failure remain network-dependent.

Remote operations still cross timing and failure boundaries. A remote monitor may report `connection_closed`, meaning observation was lost rather than that the target is known to have terminated. Remote guard calls can delay mailbox scanning; sends can succeed without confirmed delivery; and an acknowledgement protocol is needed when confirmation matters. Uniform syntax removes accidental distinctions, while these outcomes keep the network's real boundaries observable.

4.4.1 Four blocking surfaces

Stateless HTTP API – request–response blocking. Blocking occurs during an HTTP request. A client submits a goal and waits for a response; a stateless remote procedure call may retrieve results through one or more such requests. Two bounds must be distinguished. The `timeout/1` option limits remote goal evaluation, subject to the ceiling imposed by the node owner, whereas `http_timeout/1` bounds the client's HTTP operation. The former belongs to computation policy; the latter belongs to transport waiting. The `limit` and `offset` parameters bound the size of individual result slices and support incremental retrieval.

Promise/yield – deferred collection. A client initiates one asynchronous stateless call with `promise/3-4`, receives a local reference, and later invokes `yield/2-3`. A `timeout/1` option supplied to `yield/3` bounds only the current local wait

for the result. It does not consume the promise, so a later invocation may still collect the answer. Short polling intervals, retries, and fallback actions through `on_timeout/1` therefore become explicit application logic. Promise cleanup and the execution and transport bounds applied to the underlying stateless request remain separate from the timeout on an individual yield.

Semi-stateful HTTP – session exchanges. Each request addresses the same long-lived toplevel actor by pid. Calls to `/toplevel_call`, `/toplevel_next`, and `/toplevel_poll` wait for the next relevant PTCP event within an ordinary HTTP exchange. An HTTP timeout bounds that exchange; it does not by itself erase the session actor or its evolving private state. Subject to session lifetime and node policy, a later request may continue the conversation or retrieve an event that has since become available.

Stateful WebSocket API – mailbox waiting. Blocking occurs inside `receive/2`. Its `timeout/1` option bounds a wait for a matching message, while `on_timeout/1` specifies the action to perform if the wait expires. The timeout therefore bounds participation in an interaction protocol rather than completion of a single computation. A mailbox wait may represent a reply, a coordination step, or a conversational turn. Its value is determined by protocol design, not merely by transport latency. Longer waits are natural in sustained dialogues, provided that recovery behaviour is defined.

4.4.2 Late replies

A timeout ends a wait; it does not necessarily cancel the underlying computation or actor. A result or message may therefore become available after the client has stopped waiting. For `promise/3-4` and `yield/2-3`, a timed-out `yield/3` leaves the local promise available for a later collection attempt. In the semi-stateful HTTP API, timing out an individual request need not terminate the session; a later request may continue the PTCP conversation or retrieve a queued event. In the stateful API, a message that arrives after a timeout may remain available to a subsequent `receive/2`. The corresponding behaviour of an ordinary stateless HTTP operation depends on whether the remote evaluation is also terminated.

What happens to a late result – whether it is retained, queued, delivered while the interaction context remains alive, or discarded – is part of the *semantics of timing*. It determines what the client can observe and what guarantees the API provides. The Prolog Web exposes these semantics at the language level through options such as `timeout/1` and `on_timeout/1`, rather than hiding them entirely inside transport configuration.

The complementary question – what happens to the *resources* that produced the late reply, and who is responsible for reclaiming them – belongs to lifetime discipline (Section 4.1.8). Timing semantics determine what the client observes after a timeout; lifetime discipline determines when the associated processes are reclaimed.

4.4.3 Design principle

A simple rule emerges:

> Place a timeout at each operation that may block, and choose both its duration and its timeout action according to the interaction model – not merely according to transport defaults.

4.5 Security on the Prolog Web

A Web Prolog node may be configured to accept goals from any client that knows its URI. Openness of this kind raises an obvious question: what makes it safe to let strangers run code on your node?

The answer has two layers. Three mechanisms – *inexpressibility*, *invisibility*, and *containment* – define the structural security posture: what client code can express, what it can reach, and how long its effects may persist. Operational security adds authentication, resource governance, transport protection, and host isolation. The structural mechanisms arise from the Web Prolog design; the operational mechanisms are realised jointly by node policy and deployment infrastructure.

The relevance of the structural mechanisms varies with the node profile. For profiles such as RELATION and ISOBASE, where interaction is stateless or request–response over HTTP, inexpressibility and the controls familiar from conventional web services are the dominant concerns. These include input validation, resource governance, authentication, and transport protection. Invisibility and containment become particularly important for profiles that support long-lived sessions and bidirectional communication, such as ISOTOPE and ACTOR, where process identity and message passing cross node boundaries. The remainder of this section focuses on this richer setting, where the interplay of all three structural mechanisms becomes visible.

The relationship between the structural and operational mechanisms is illustrated in Figure 4.5.

4.5.1 Inexpressibility: the sandbox

Web Prolog follows the same broad strategy as browser-based JavaScript: untrusted code executes within a restricted environment. A publicly reachable node should state its restricted language as a *whitelist* of admitted predicates, and it is to that whitelist that inexpressibility refers: a host effect left off it cannot be named, however the client composes its goals. A blacklist, which instead names the predicates to exclude, permits a broader language but gives a weaker guarantee. The demonstrator defaults to blacklist mode, including on its public tutorial nodes; the inexpressibility claim does not hold for those nodes. Appendix E explains why only the whitelist makes the

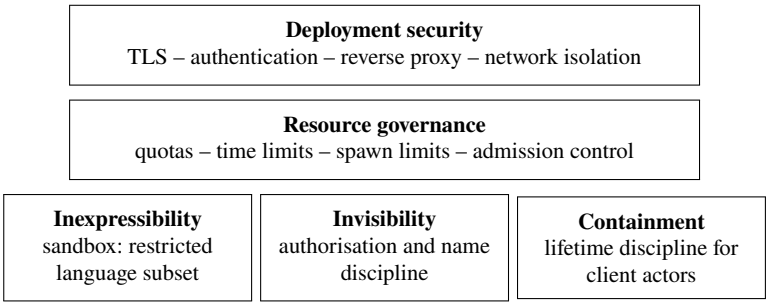


Fig. 4.5 Structural and operational security mechanisms of a Web Prolog node.

inexpressibility claim exact. The client language excludes file I/O, direct operating-system access, raw socket programming, and durable node storage. These facilities, if needed, are provided by the host platform and mediated through predicates and services that the node owner chooses to install – they are not directly expressible by client-injected code. Installing such a predicate does not lend the client the owner's authority: an owner-defined predicate runs in the calling actor, under the caller's profile and sandbox, and reaches a host effect only through the specific primitives the sandbox declares safe. The node owner therefore widens the client's reach only by the primitives it exposes, not by the ordinary predicates it happens to define.

This is the price of allowing strangers to run code on your machine: certain classes of host effect must be unavailable to client code. Submitted goals and source are therefore checked against the active profile and sandbox policy before execution. Where the language permits goals to be constructed dynamically, guarded execution paths apply the policy again to the resulting goal. The sandbox substantially narrows the client language, while host isolation provides a separate backstop should the language-level boundary fail.

Even when an operation exists in the underlying system, it is exposed to clients only through owner-controlled predicates and services. Access to those predicates and services may in turn be constrained by authentication and authorisation policy. The authorisation policy is therefore part of the node's security boundary: it defines which actions are available to which roles, and thereby what the node counts as its public surface.

4.5.2 Invisibility: name discipline

Joe Armstrong articulated a useful design principle for actor systems: if a process cannot be named, it cannot be addressed. Once process names become widely known, the system's communication surface expands. Web Prolog adopts this principle as a discipline for process identity.

A pid is an ordinary Prolog term, `Id@Node`, and anyone can construct one. The node assigns `Id`, does not reuse it, and makes it hard to guess. Constructing a pid does not create an actor or grant access to one.

The language permits any holder to send, subject to node policy. The demonstrator's public WebSocket binding restricts targets to connection-owned actors and authorised published services. Monitoring uses the same target policy as sending, whereas `exit/2` requires membership in the caller's ownership tree and `demonitor/1-2` removes only the caller's own monitors. A pid may thus be needed to address an actor without being enough to use it. Clients normally learn pids only when they are told them, and `actors/1` shows a public client only its own actors. Keeping pids undisclosed narrows what a client can reach, but it is no substitute for authentication.

Discoverability creates a natural tension with name discipline. If processes are protected in part because they cannot be addressed without a name, public services must still provide a controlled way for clients to learn which names are intended for use. The safe resolution is to treat discovery as an owner-curated *publication mechanism*: discovery should resolve *published* names, not reveal everything that happens to be running. One concrete realisation of this idea – a declarative service directory queryable via `rpc/2-3` – was given in Section 4.1.9.

Reply channels and correlation

The common request–reply idiom illustrates why the distinction matters. A client often includes its own pid in a request so that the provider can reply. Where node policy permits the provider to send arbitrary messages to that address, this enables messages unrelated to the request as well as the expected reply. Disclosure alone does not override a policy that restricts such access.

A safer protocol uses a fresh, request-local reply endpoint, such as a temporary proxy actor that accepts only the expected reply and then terminates. The proxy narrows the accepted protocol; possession of its pid alone is still not proof of authority. A fresh, single-use reference correlates the reply with the request and prevents accidental matches. A sufficiently large random space also resists guessing; the demonstrator's compact ten-digit references are a presentation compromise, and production references should be larger. A reference does not authenticate a sender that legitimately knows it: such a sender can forge a matching reply. Where that distinction matters, the protocol needs sender authentication or an enforced proxy policy. Communication authority should be as narrow and short-lived as the interaction that requires it. This discipline becomes particularly important when reply channels cross node boundaries and therefore cross trust domains.

4.5.3 Containment

Containment, the third mechanism, was introduced in Section 4.1.8: every actor a client spawns depends on its parent for its lifetime, and only node owners may create detached actors. When parent termination or loss of the owning connection is detected, the runtime reclaims the linked subtree. Here we place the design among its precedents and alternatives.

The one-way link itself is adopted from Svensson et al. (2010, Section 2.2), not introduced here. In their proposed Erlang semantics, a child links to its supervisor so that the supervisor's termination kills the child, while a monitor carries failure information in the other direction. Their proposal removes exit trapping, separating enforced termination from notification. It provides a precedent for the primitives, rather than a proof of their suitability for every open network. Web Prolog turns that asymmetric relation into an ownership discipline: it is established by spawning, made the default, and extended across node and session boundaries.

There are useful parallels elsewhere. Go's context design propagates cancellation from a parent operation to its sub-operations, explicitly denying those sub-operations the corresponding authority to cancel the parent.[6] Go cancellation is cooperative, whereas Web Prolog's downward termination is enforced; the shared principle is asymmetric lifetime control. Ergo likewise motivates unidirectional links with the example of workers that depend on a shared service whose lifetime should not depend on each worker. It allows mutual dependence through two explicit links.[7] These are parallels, not proofs.

Loss of contact gives another reason to preserve the parent's ability to recover. Erlang documents that a remote link can produce `noconnection` while the other process is still alive.[8] Web Prolog separates the parent's notification of lost observation from the remote host's reclamation of connection-owned work. Directionality does not solve failure detection: as Section 4.1.7 explains, a finite lease bounds lost-contact detection while the runtime services the connection, without proving that remote work has stopped. It determines where the consequences propagate once loss is detected.

Bidirectional links remain useful for components intended to fail together. Erlang can also implement asymmetric supervision by trapping exits, so the case for this design concerns explicit dependencies and defaults, not an inability to express the policy in Erlang. Mutual failure dependence must be an additional policy where the application requires it.

Should client-originated detached jobs be needed in the future, they should be introduced as an explicit abstraction – with quotas, lifetime parameters, and auditable handles – rather than as a general relaxation of the containment default.

---

[6] Sameer Ajmani, *Go Concurrency Patterns: Context* (2014), `https://go.dev/blog/context`.

[7] Ergo Framework documentation, *Links and Monitors*, `https://docs.ergo.services/basics/links-and-monitors`.

[8] Erlang reference manual, *Processes*, discussion of exit signals, `https://www.erlang.org/docs/26/reference_manual/processes.html`.

4.5.4 Deployment concerns

The three structural mechanisms above define the security architecture of the Prolog Web, but they do not by themselves constitute a complete operational security model.

First, the node must establish *who* the caller is. Without authentication, the distinction between client and owner has no firm operational basis. Second, the node must enforce *resource governance*. A client may remain entirely within the sandbox and yet still exhaust CPU time, memory, process slots, or network bandwidth. Third, communication between client and node must be protected in transit. Without TLS (the encryption behind `https` addresses) or equivalent transport protection, a third party may observe or tamper with messages, undermining confidentiality and integrity.

Fourth, the runtime process itself must be constrained by the host environment. Even a correctly specified language sandbox offers limited assurance if the underlying process retains unrestricted filesystem or network access. Host isolation – through operating-system-level confinement, WebAssembly runtimes, or WASI-based embeddings – removes classes of risk that the language boundary alone cannot prevent.

These concerns extend beyond language semantics. They are realised jointly by Web Prolog runtime policy and deployment infrastructure: the node enforces identities, permissions, and resource ceilings, while the host environment and network provide transport protection and process isolation. A production node therefore requires both structural security and secure operation.

What remains after these mechanisms have done their work is the node's public surface: the predicates it exports, the services it publishes, and the policies under which they may be accessed. A well-configured node should therefore present a small, explicit, and auditable interface: everything else should be either inexpressible, invisible, or contained.

4.6 Summary

This chapter developed the Prolog Web as a distributed execution environment for Web Prolog, with concurrent and sequential interaction contracts.

The concurrent Prolog Web extends actor programming across node boundaries. Actors can be spawned remotely and addressed through the same primitives used locally, while latency, delivery uncertainty, and partial failure remain observable. Directional links enforce lifetime dependence; monitors report outcomes separately. Node-resident actors provide long-lived, shared services that owners can publish under stable names.

The sequential Prolog Web exposes remote computation through predicate interfaces. The nondeterministic `rpc/2-3` lets a caller backtrack over answers obtained from another node, while `promise/3-4` and `yield/2-3` separate initiating a call

from collecting its result. These interfaces keep mailbox management and scheduling out of the caller's code.

Programmability cuts across both partitions. The `src_*` options let clients supply task-specific code to their execution contexts, while owners install the predicates and services a node offers.

Timing and failure belong to the interaction contract. Each blocking surface exposes a different kind of wait, and timeout policy must reflect what the application is waiting for. A timeout ends a wait; it does not necessarily mean that the computation failed, cancel the underlying work, or prevent a reply from arriving later.

Opening this environment to untrusted clients requires both structural restrictions and operational protection. Inexpressibility, invisibility, and containment limit what client code can do, inspect, and leave running. Authentication, authorisation, resource limits, transport protection, and host isolation complete the deployment picture. The resulting public surface consists of the predicates and services a node chooses to expose under its policies.

The following chapters develop generic behaviours and statecharts that structure applications built on these interaction and lifetime contracts.

Part II
Extensions and contexts

Chapter 5
Implementing Web Prolog generics

Generic constructs provide reusable actor patterns that can be specialised, configured, or extended after creation, and that encapsulate common concerns such as message protocols, supervision, and state management.

It is useful to distinguish two broad categories of generics considered in Web Prolog. The first category comprises *Erlang-style behaviours*, which are message-driven components exposing a stable protocol, typically realised as a fixed vocabulary of messages and replies. They do not expose the callee's proof search directly through caller backtracking. A client wrapper may bind an argument to an explicit reply term, but the interaction itself remains protocol-oriented. The `server_*` predicates provide a request–response service analogous to Erlang's `gen_server`; the `supervisor_*` predicates provide a behaviour similar to an Erlang `supervisor`; and statechart actors implement a state-machine model in the spirit of `gen_statem`. The `toplevel_*` predicates also belong to this category, although a Prolog toplevel actor has no ordinary counterpart in Erlang.

The second category comprises what may be termed *Prolog-style generics*. Some are meta-predicates that orchestrate actors internally while presenting a conventional predicate-level interface. Examples include `parallel/1` and `first_solution/2-3`, which spawn worker actors, coordinate them with selective receive, and expose success, failure, exceptions, and variable bindings directly to the caller. They are semideterministic and do not enumerate further solutions on backtracking. By contrast, `rpc/2-3`, specified in Chapter 4 and implemented below, is nondeterministic: as a distributed analogue of a predicate call, it participates directly in proof search and can return multiple solutions.

Both kinds of generic take communication, lifecycle and concurrency out of application code. Programmers can then choose between explicit actor protocols and ordinary-looking predicates.

This chapter uses code in three ways. *Usage examples* show calls to the public interfaces and their expected interaction; their helper predicates and application data must also be supplied. *Implementation sketches* explain how a behaviour can be built, with omissions stated beside the relevant listings. *Intermediate versions* isolate one

mechanism before later listings add failure handling or cleanup. A group of listings inherits the status stated in its introductory paragraph.

Appendix B defines the normative public interfaces adopted by the demonstrator. The sketches illustrate those interfaces without replacing their contracts or reproducing the demonstrator's source. An interface introduced here but absent from the manual is a design proposal. Readers can run the complete implementation using the Trinity demonstrator and companion source repository described in Section 1.6.

5.1 The server behaviour

Section 3.3.6 used the server and supervisor behaviours through their interfaces. Here we build them. The *server behaviour* implements a classic client–server pattern in Web Prolog: a central server process maintains private state and serves an arbitrary number of clients through message passing. The behaviour is specialised by supplying an application-specific callback predicate, while the generic part provides request–response communication, state threading, monitoring, graceful shutdown, and hot code replacement.

The fragments below form one example. We first define a refrigerator callback and the reusable server loop, then add a compact client side of the behaviour. Unless a fresh server is explicitly created, the shell transcripts continue to use the same refrigerator process and therefore the same state.

To keep the example focused on the behaviour itself, we assume that callback predicates named in `server_spawn/3-4` or `server_upgrade/2` are already available to the server process. The demonstrator's `server_upgrade/3` can instead transfer and install callback source; that fuller interface is documented in Appendix B, but its implementation is not needed to explain the server protocol.

The application-specific part is a callback describing one transition. Its four arguments are the request, the old state, the response, and the new state:

```
fridge(store(Food), Foods, ok, [Food|Foods]).
fridge(take(Food), Foods, ok(Food), Rest) :-
    select(Food, Foods, Rest), !.
fridge(take(_Food), Foods, not_found, Foods).
```

The callback contains only the application transition; process ids, correlation references, reception, and recursion belong to the generic behaviour. Its contract is semideterministic: a supported request succeeds at most once, while failure means that the request is not handled. The cut makes the second clause deterministic when `fridge/4` is called independently; the server loop also commits to the callback's first solution.

A server is started with a callback predicate `Pred/4` and an initial state. The following implementation sketch gives the spawn predicates and server loop; the client-side options and source-loading upgrade interface are outside its scope:

```
server_spawn(Pred, State, Pid) :-
    server_spawn(Pred, State, Pid, []).

server_spawn(Pred, State, Pid, Options) :-
    spawn(server_loop(Pred, State), Pid, Options).
```

For an application request, the loop calls the callback as

```
call(Pred, Request, OldState, Response, NewState)
```

and passes the resulting state to its next iteration:

```
server_loop(Pred, State0) :-
    receive({
        '$call'(From, Ref, Request) ->
            (   call(Pred, Request, State0, Response, State)
            ->  From ! Ref-Response,
                server_loop(Pred, State)
            ;   exit(false)
            ) ;
        '$control'(From, Ref, upgrade(Pred1)) ->
            upgrade_result(Pred1, Result),
            From ! Ref-Result,
            (   Result == ok
            ->  server_loop(Pred1, State0)
            ;   server_loop(Pred, State0)
            ) ;
        '$control'(From, Ref, stop) ->
            From ! Ref-true
    }).

upgrade_result(Pred, Result) :-
    (   current_predicate(Pred/4)
    ->  Result = ok
    ;   Result = error(
            error(existence_error(procedure, Pred/4), _))
    ).
```

A call message contains the client pid, a fresh correlation reference, and the application request. If the callback succeeds, the server sends the response tagged with the same reference and recurs with the new state. If no callback clause handles the request, the server exits with reason `false`. An uncaught callback exception `E` terminates it with reason `exception(E)`. After acknowledging `stop`, the loop has no recursive call and the server terminates normally, with reason `true`. An upgrade is acknowledged only after the replacement has been validated. If validation fails, the error is returned and the loop continues with its old callback and state.

5.1.1 The client interface

The client side uses the same wire protocol as the loop. The following implementation sketches cover ordinary synchronous calls, requests kept in flight, and fail-fast monitoring. They omit the option-bearing waits in Section B.3; `await_reply/3` is an internal helper. We begin with the two operations from which a call is built: a *promise* sends a request, and a *yield* collects its reply.

```
server_promise(To, Request, Ref) :-
    self(Self),
    make_ref(Ref),
    To ! '$call'(Self, Ref, Request).

server_promise(To, Request, Ref, MonRef) :-
    self(Self),
    monitor(To, MonRef),
    make_ref(Ref),
    catch(To ! '$call'(Self, Ref, Request), Error,
          (demonitor(MonRef, [flush]), throw(Error))).
```

The four-argument form installs a monitor before sending and returns both references. If sending throws an exception, the monitor is removed before the exception is rethrown. Monitoring before the send closes the race in which the server terminates just before the monitor is installed. The corresponding collection predicates are:

```
server_yield(Ref, Response) :-
    receive({
        Ref-Response -> true
    }).

await_reply(Ref, MonRef, Response) :-
    call_cleanup(
        receive({
            Ref-Response ->
                true ;
            down(_Pid, MonRef, Reason) ->
                throw(server_down(Reason))
        }),
        demonitor(MonRef, [flush])
    ).
```

The unmonitored yield waits only for a matching reply. `await_reply/3` additionally accepts the notification that the server has terminated. Its cleanup removes the monitor whether the receive succeeds or raises an exception. The `flush` option also removes a `down` notification that may already have been queued if the server replied and then terminated before the client cancelled its monitor. This helper is the

compact sketch's counterpart of the public `server_yield/4` operation documented in Appendix B.

Figure 5.1 makes the ordering explicit. The monitor is installed before the request is sent, so either the correlated reply or a termination notification can settle the wait.

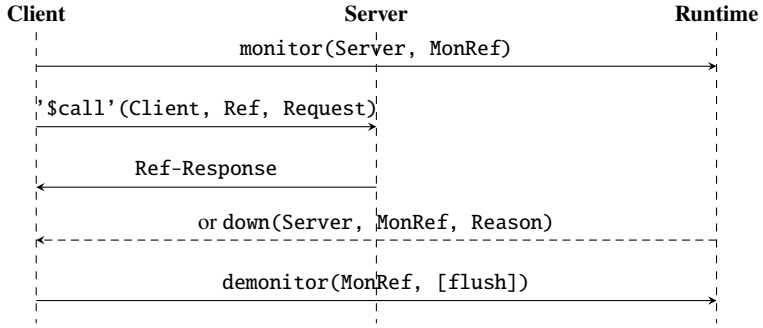


Fig. 5.1 Correlation and failure detection in a monitored server call.

An ordinary synchronous request uses the monitored forms by default:

```
server_request(To, Request, Response) :-
    server_promise(To, Request, Ref, MonRef),
    await_reply(Ref, MonRef, Response).
```

We now have a complete path through the code: `server_request/3` sends `'$call'(Self, Ref, Request)`, the server loop invokes `fridge/4`, and the client selects the resulting `Ref-Response`. For example:

```
?- server_spawn(fridge, [], Pid).
Pid = 99491660.
?- server_request($Pid, store(milk), Response).
Response = ok.
?- server_request($Pid, store(cheese), Response).
Response = ok.
?- server_request($Pid, take(milk), Response).
Response = ok(milk).
?-
```

The refrigerator's state is now `[cheese]`. To keep several requests in flight, a client instead uses the unmonitored promise and yield forms directly. Each request carries a fresh reference, so replies can be collected independently. Here the client deliberately collects the second reply first:

```
?- server_promise($Pid, store(meat), StoreRef),
   server_promise($Pid, take(meat), TakeRef).
StoreRef = 11025019,
```

```
TakeRef = 45018213.
?- server_yield($TakeRef, Taken).
Taken = ok(meat).
?- server_yield($StoreRef, Stored).
Stored = ok.
?-
```

Although the server processed the requests and sent their replies in order, selective receive found the reply tagged with `TakeRef` and left the earlier reply in the mailbox for the following yield. The state is again `[cheese]`. Code that needs both pipelining and fail-fast behaviour retains the monitor reference with the request reference and uses the same monitored collection pattern as `await_reply/3`. The fuller option-bearing interface is documented in Appendix B. Bounded waiting is omitted from this sketch because a sound timeout policy must say what happens to a reply that arrives after the wait has ended.

5.1.2 A failing callback

Monitoring an actor that has already terminated does not lose the failure notification: the caller receives a `down` message with reason `no_process`. Thus a synchronous request whose callback fails does not wait forever either. Using a fresh refrigerator so that the running example above can continue:

```
?- server_spawn(fridge, [], TypoPid),
   server_request(TypoPid, sore(milk), Response).
ERROR: Unhandled exception: server_down(false)
?-
```

The misspelling `sore(milk)` matches no clause of `fridge/4`. The callback fails, the fresh server exits with reason `false`, and `await_reply/3` reports that exit to the waiting client. The original server in `$Pid` is unaffected.

5.1.3 Hot code replacement

Upgrade and shutdown use the same monitored acknowledgement pattern as an application request. A small helper installs the monitor before sending the control message, so either an acknowledgement or the server's termination is observed:

```
server_control(To, Command, Reply) :-
    self(Self),
    monitor(To, MonRef),
    make_ref(Ref),
    catch(To ! '$control'(Self, Ref, Command), Error,
```

```
          (demonitor(MonRef, [flush]), throw(Error))),
    await_reply(Ref, MonRef, Reply).

server_upgrade(To, Pred) :-
    server_control(To, upgrade(Pred), Result),
    (   Result == ok
    ->  true
    ;   Result = error(Error), throw(Error)
    ).
```

The server validates the replacement before acknowledging it and recurs with the new callback while preserving `State0`; thus the client knows that the upgrade has been accepted, not merely placed in transit. A rejected upgrade raises the reported error at the client while the server continues with its previous callback and state. The demonstrator's three-argument form performs source loading before this validation and reports loading errors through the same reply.

To make the upgrade observable, the replacement callback adds an `inventory` request and delegates the existing requests to `fridge/4`:

```
fridge_with_inventory(inventory, Foods, inventory(Foods), Foods).
fridge_with_inventory(Request, Foods0, Response, Foods) :-
    fridge(Request, Foods0, Response, Foods).
```

Assuming that `fridge_with_inventory/4` is already available to the server process, the callback of our original refrigerator can now be replaced:

```
?- server_upgrade($Pid, fridge_with_inventory).
true.
?- server_request($Pid, inventory, Response).
Response = inventory([cheese]).
?-
```

The operation returns only after the server has accepted the new callback and, as the inventory response confirms, retains the current state. The replacement must therefore accept the existing state representation. If an upgrade changes that representation,

a separate state-migration protocol is required; simple callback replacement is not enough.

5.1.4 Graceful shutdown

A server is stopped through the same acknowledged control path:

```
server_halt(To, Reply) :-
    server_control(To, stop, Reply).
```

The helper's fresh reference makes the shutdown acknowledgement subject to the same selective-receive discipline as ordinary requests and upgrades. We can now stop the refrigerator used throughout the section:

```
?- server_halt($Pid, Reply).
Reply = true.
?-
```

5.1.5 From a bespoke actor to a reusable behaviour

The refrigerator in Chapter 2 was already a stateful concurrent service, but it was implemented as a bespoke actor. The predicate `fridge/1` served simultaneously as the application logic and the actor loop: it recognised incoming messages, addressed replies to clients, carried the food list in its recursive argument, and handled termination.

The server behaviour separates these responsibilities. The `fridge/4` callback shown above describes a single application transition, mapping a request and the current state to a response and a successor state. The generic server owns the state, receives and correlates messages, invokes the callback, and continues with the resulting state. Recursion has therefore moved out of the refrigerator predicate and into the reusable behaviour. On the client side, the server interface similarly hides the construction and matching of request and reply messages performed manually in the earlier example.

Box 5.1 — Erlang comparison: `gen_server` and the server behaviour

Web Prolog's server behaviour occupies much the same architectural position as Erlang/OTP's `gen_server`. In each case generic machinery owns the receive loop, correlation, and lifecycle protocol, while application code describes how one request transforms state.

An Erlang `gen_server` callback receives a request and state and returns a reply with a successor state. A Web Prolog server callback similarly relates a request and old state to a response and new state. The latter is a Prolog relation, used under a semideterministic callback contract, rather than an Erlang function. In both systems, the application programmer writes the service transition rather than repeatedly implementing the actor loop, reply routing, and failure observation that make it a service.

5.2 The supervisor behaviour

The *supervisor behaviour* starts a set of child actors, monitors them, and applies a declared policy when one terminates. Application code describes the children; the generic supervisor owns their lifecycle.

Box 5.2 — Erlang comparison: OTP supervision and recovery policy

Erlang/OTP supervision is not simply that processes can be linked. A supervisor owns child specifications, observes linked children's exits, and applies a declared recovery policy: whether an exit should restart a child, which related children must be restarted, and when repeated failures make the supervisor give up.

The Web Prolog supervisor behaviour adopts this separation of responsibility. Directional links provide containment during shutdown; monitors report child outcomes; child specifications, restart types, strategies such as `one_for_one` and `one_for_all`, and an intensity window determine recovery. Letting a worker crash is not a licence to ignore errors. It delegates the recovery decision to a separately specified supervisor, particularly appropriate for node-resident services whose availability is a node-owner responsibility.

A failed worker can discard its process state while the supervisor decides what to replace (Armstrong, 2003b). This division of responsibilities does not itself recover lost application data or undo external effects. Erlang's supervision principles give the broader account of the pattern.[a]

[a] `https://www.erlang.org/doc/system/sup_princ.html`.

5.2.1 A restart we can observe

We continue with the refrigerator from the previous section, adding one request that deliberately terminates the server:

```
supervised_fridge(crash, _Foods, _Reply, _NewFoods) :-
    exit(simulated_failure).
supervised_fridge(Request, Foods0, Reply, Foods) :-
    fridge(Request, Foods0, Reply, Foods).
```

Assume, as before, that the callback is available to the new actor. The child specification gives the child an identifier, a start instruction, an initial state, and a restart policy. The registered name remains stable even though the child pid changes:

```
?- supervisor_spawn([
       child(fridge_service, [
           start(server(supervised_fridge,[initial_state([])])),
           restart(permanent)
       ])
   ], Sup, [strategy(one_for_one), intensity(3),
            period(10), monitor(true)]).
Sup = 58310427.
?- supervisor_which_children($Sup, Children).
Children = [info(fridge_service, 91467230, worker, permanent)].
?-
```

Here `worker` is the child's type. Both server and ordinary actor starts are workers; a nested supervisor has type `supervisor`. The compact internal specification below omits this production field since the one-for-one termination path does not inspect it.

We put some state in the child and then make it fail:

```
?- server_request(fridge_service, store(milk), Reply).
Reply = ok.
?- catch(server_request(fridge_service, crash, _), Error, true).
Error = server_down(simulated_failure).
?- supervisor_which_children($Sup, Children).
Children = [info(fridge_service, 26083951, worker, permanent)].
?-
```

The supervisor itself still has pid `58310427`, but the child pid has changed from `91467230` to `26083951`. The child specification was reused to start a replacement and to register it again as `fridge_service`. Process state is not recovered automatically: the replacement starts from the declared initial state.

```
?- server_request(fridge_service, take(milk), Reply).
Reply = not_found.
?-
```

The `permanent` policy requests a restart after either a normal or an abnormal termination. A `transient` child is restarted after any termination reason other than `true`, while a `temporary` child is never restarted. These are child policies; the supervisor strategy, discussed below, determines which other children are affected by a required restart.

Without the supervisor behaviour, the application would have to retain every child's start information, monitor each pid, classify its termination reason, enforce the restart budget, and register any replacement under the stable service name. That hand-written loop would be almost identical for every application. The supervisor moves it into one reusable mechanism and leaves the application to declare only the child specification and restart policy.

5.2.2 The generic mechanism

The following implementation sketch covers one-for-one restarts. It assumes validated child specifications and leaves management requests, other restart strategies, and orderly shutdown to the fuller implementation described below.

Validation lowers the public `server(Pred, ServerOptions)` child form to a start instruction containing the extracted initial state. The loop then uses the small internal form `spec(Id, Start, Restart)`. That start instruction is either `server_start(Pred, State)` or `actor(Goal)`; both kinds of child install a monitor and a directional link:

```
start_child(Spec, running(Pid, Spec)) :-
    Spec = spec(Id, server_start(Pred, State), _),
    server_spawn(Pred, State, Pid,
                 [name(Id), monitor(true), link(true)]).
start_child(Spec, running(Pid, Spec)) :-
    Spec = spec(_, actor(Goal), _),
    spawn(Goal, Pid, [monitor(true), link(true)]).

should_restart(spec(_, _, permanent), _Reason).
should_restart(spec(_, _, transient), Reason) :-
    Reason \== true.
```

There is no `should_restart/2` clause for a temporary child. The monitor reports termination; the link has the other directional purpose of ensuring that children do not survive an unexpected termination of their supervisor. A server child uses `Id` as its stable registered name. For an ordinary actor, `Id` is supervisor bookkeeping unless `Goal` registers a name itself.

The loop also needs to find a child without losing its position in start order. `take_child/5` returns the children to its left and right:

```
take_child(Pid, [running(Pid, Spec)|Right], Spec, [], Right).
take_child(Pid, [Child|Children], Spec, [Child|Left], Right) :-
    take_child(Pid, Children, Spec, Left, Right).

retire_child(spec(_, _, temporary), Left, Right, Children) :-
    append(Left, Right, Children).
retire_child(Spec, Left, Right, Children) :-
    Spec = spec(_, _, Restart),
    Restart \== temporary,
    append(Left, [running(undefined, Spec)|Right], Children).
```

A temporary child's specification is removed. A non-temporary child that is not restarted remains available for an explicit later respawn, but has no current pid. The two `retire_child/4` clauses are exclusive, so backtracking cannot reinsert a temporary child. With these helpers the complete one-for-one termination cycle remains small:

```
supervisor_loop(Children0, Limit0) :-
    receive({
        down(Pid, _, Reason)
             if take_child(Pid, Children0, Spec, Left, Right) ->
            (   should_restart(Spec, Reason)
            ->  charge_restart(Limit0, Limit),
                start_child(Spec, Child),
                append(Left, [Child|Right], Children)
            ;   Limit = Limit0,
                retire_child(Spec, Left, Right, Children)
            ),
            supervisor_loop(Children, Limit) ;
        down(_, _, _) ->
            supervisor_loop(Children0, Limit0)
    }).
```

The guarded clause accepts a notification for a child in the current list; a stale or unrelated `down/3` notification is consumed by the fallback clause rather than accumulating in this dedicated supervisor's mailbox. As described in Section 2.3.4, the guard may bind `Spec`, `Left`, and `Right`; receive commits to its first solution. Child pids are unique, so there is at most one such split. The matching specification supplies both the restart policy and the start instruction, while the two list segments preserve start order.

This listing shows only the termination path. The full loop also has branches for `supervisor_which_children/2` and other management requests, using the monitored request pattern from Section 5.1.

5.2.3 From restart to escalation

Restarting without a limit can turn one fault into an endless crash loop. The options `intensity(3)` and `period(10)` in the example establish a rolling restart budget: three restart attempts may occur within ten seconds. The next required restart in the same window makes the supervisor terminate with reason `shutdown`. Because the example also uses `monitor(true)`, the shell would then receive `down(58310427, 58310427, shutdown)`. The loop charges that window as follows:

```
charge_restart(limit(Max, Period, Times0),
               limit(Max, Period, [Now|Recent])) :-
    get_time(Now),
    Cutoff is Now - Period,
    include(recent_restart(Cutoff), Times0, Recent),
    length(Recent, N),
    (   N < Max
    ->  true
    ;   exit(shutdown)
    ).

recent_restart(Cutoff, Time) :-
    Time > Cutoff.
```

The initial value for the example is `limit(3, 10, [])`. Old timestamps are discarded before the count is tested, so failures far apart do not consume a lifetime allowance. By contrast, several rapid failures are treated as one unresolved incident. The allowance is charged before `start_child/2`, so every requested replacement counts. If the supervisor gives up, its directional links terminate the remaining children, and a parent supervisor can apply a broader recovery policy.

5.2.4 Strategies, dynamic children, and trees

The loop above implements `one_for_one`: only the failed child is replaced. Under `one_for_all`, the supervisor shuts down the other children and restarts the affected group. Under `rest_for_one`, it does so only for the failed child and the children started after it. Start order is therefore part of the supervisor's state rather than an incidental property of a list. Order also matters during shutdown, when the supervisor stops its children one at a time in reverse start order.

Because termination is untrappable and the first committed exit reason wins, a supervisor cannot shut a child down "gently, then harder" with two termination signals: a later kill has no effect on a termination already in progress. A timed shutdown is therefore a message-level stop request followed by a termination signal. For a server child, the supervisor sends the server's ordinary stop request and waits up to the

specified time for the child to exit; only then does it send `exit(Pid, shutdown)`. A child without a stop protocol receives the termination signal at once, and its cleanup handlers run as it unwinds. This orderly sequence applies when the supervisor itself decides to shut down. A supervisor that is terminated from outside, by its parent or a link, is torn down by the runtime, and its linked children are terminated together, without order.

Dynamic child management changes the stored set of specifications: an API may add and start a child, terminate one while retaining its specification, delete a stopped specification, or respawn it later. These operations require no new actor primitives. They use the same spawning, monitoring, linking, termination, and selective receive shown in the smaller loop; synchronous management calls additionally use the monitored request pattern developed in the preceding section.

Supervisors can themselves be children. If a supervisor exceeds its restart intensity, its parent observes the termination and applies the parent-level policy. Repeating this arrangement yields a supervision tree in which a local failure is first handled locally and escalates only when that level exhausts its restart allowance. The demonstrator implements all three strategies, dynamic child operations, timed restart intensity, and orderly shutdown in the sense just described. The complete public interface and child-specification format are documented in Section B.5 of Appendix B. Erlang's OTP supervision principles provide a fuller account of the model.[1]

5.3 The toplevel behaviour

Chapter 3 introduced the toplevel actor through client interactions: submitting goals, requesting further solutions, and stopping a search. We now turn to the machinery that makes those interactions possible. The implementation must preserve a query's variable bindings and choice points while the actor waits for a message, then resume the same search when the client asks for more.

This gives the receive loop a different role from the server loop. A server callback completes a state transition before the next request is handled. A toplevel may instead wait with a computation still suspended. The implementation must distinguish an idle actor (**s1**), an actor computing a solution slice (**s2**), and an actor retaining the search while awaiting the client's decision (**s3**). Figure 5.2 shows these states for a persistent session.

The following sketch explains that suspension boundary, the collection of solutions into slices, and the handling of requests to resume or discard a query. It follows the core of the demonstrator's local toplevel while omitting source injection, module isolation, most validation, inference limits, transport hooks, and some options. The public `once(true)` option, for example, commits a call to its first returned slice and reports `More = false`; with `limit(1)`, that slice contains at most one solution. The

[1] `https://www.erlang.org/doc/system/sup_princ.html`

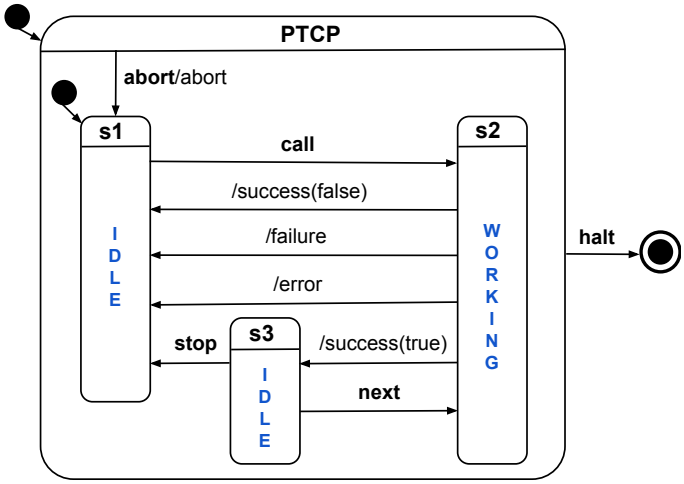


Fig. 5.2 The persistent form of the Prolog Toplevel Communication Protocol.

option is not implemented by the simplified loop below. Complete interfaces remain in Section B.2 of Appendix B.

The small wrapper below spawns an actor running that protocol:

```
toplevel_spawn(Pid) :-
    toplevel_spawn(Pid, []).

toplevel_spawn(Pid, Options) :-
    self(Self),
    option(target(Target), Options, Self),
    option(session(Session), Options, false),
    spawn(ptcp(Pid, Target, Session), Pid, Options).
```

The `target` option determines where answers are sent. The `session` option controls what happens after a query. Its default, `false`, terminates the actor once the query is exhausted or stopped; `true` returns it to idle so that it can accept another query.

5.3.1 The three inner states

The implementation mirrors the three states in Figure 5.2 with the predicates `state_1/3`, `state_2/6`, and `state_3/2`.

The statechart shows the persistent, `session(true)` form of the protocol. The simplified implementation also supports a one-shot `session(false)` form, which terminates normally after its query has been exhausted or stopped; this lifecycle variation is not shown in the statechart.

The idle state **s1** is implemented by `state_1/3`:

```
state_1(Pid, Target0, Session) :-
    wait_call(Goal, Options),
    option(template(Template), Options, Goal),
    option(offset(Offset), Options, 0),
    (   option(limit(N), Options)
    ->  must_be(positive_integer, N),
        Limit = count(N)
    ;   Limit = all
    ),
    option(target(Target1), Options, Target0),
    Target = target(Target1),
    state_2(Goal, Template, Offset, Limit, Pid, Answer),
    arg(1, Target, Out),
    (   Answer = pending_error(Pid, Slice, Error)
    ->  Out ! success(Pid, Slice, true),
        state_3_error(Pid, Error, Target)
    ;   Out ! Answer,
        (   arg(3, Answer, true)
        ->  state_3(Limit, Target), !
        ;   true
        )
    ),
    (   Session == false -> true
    ;   state_1(Pid, Target0, Session)
    ).

wait_call(Goal, Options) :-
    receive({
        '$call'(G, O) ->
            Goal = G, Options = O ;
        Message if stale_command(Message) ->
            wait_call(Goal, Options)
    }).

stale_command(Message) :-
    nonvar(Message),
    (   Message = '$next'(_)
    ;   Message = '$stop'
    ;   Message = '$input'(_, _)
    ).
```

The `nonvar/1` check prevents a bare variable application message from being mistaken for a stale command. The helper `wait_call/2` consumes obsolete continuation, stop, and input commands up to the next `'$call'`, which supplies the goal

and its options. Commands following that call are preserved, whether they are already deferred or still in the host mailbox. A toplevel has one controlling client at a time, and per-sender ordering establishes this boundary. A client may therefore send `toplevel_next/1` after its call without waiting for the first answer. If `limit(N)` is supplied, `N` must be a positive integer and the internal `count(N)` cell controls slicing. Otherwise the internal marker `all` selects complete collection; it is not a public option value. With a finite limit, the limit and target are stored in mutable one-argument compounds so that a later `'$next'` message can change either value. Halting is not a mailbox command and therefore requires neither a `'$halt'` branch nor a mutable control term. It is implemented by the actor runtime, as described below.

The test `arg(3, Answer, true)` succeeds only for `success(Pid, Slice, true)`. It simply fails for the lower-arity `failure(Pid)` and `error(Pid, Term)` answers, so those outcomes do not enter `state_3/2`.

The working state calls the `answer/5` helper defined below and adds the toplevel actor's pid to the outward-facing answer:

```
state_2(Goal, Template, Offset, Limit, Pid, Answer) :-
    answer(Goal, Template, Offset, Limit, Answer0),
    add_pid(Answer0, Pid, Answer).

add_pid(success(Slice, More), Pid, success(Pid, Slice, More)).
add_pid(failure, Pid, failure(Pid)).
add_pid(error(Term), Pid, error(Pid, Term)).
add_pid(pending_error(Slice, Term), Pid,
        pending_error(Pid, Slice, Term)).
```

When an answer reports that more solutions may remain, the actor enters `state_3/2`:

```
state_3(Limit, Target) :-
    receive({
        '$next'(Options) ->
            (   option(limit(NewLimit), Options)
            ->  must_be(positive_integer, NewLimit),
                nb_setarg(1, Limit, NewLimit)
            ;   true
            ),
            (   option(target(NewTarget), Options)
            ->  nb_setarg(1, Target, NewTarget)
            ;   true
            ),
            fail ;
        '$stop' -> true
    }).
```

After `'$next'`, the call to `fail/0` backtracks into `state_2/6` and resumes the suspended `findnsols/4` computation. The updated limit controls the next slice, while the updated target determines where its answer is sent.

If `'$stop'` is received instead, the current search is abandoned. A persistent session returns to **s1**; a non-session toplevel then terminates. The cut after `state_3/2` discards the abandoned query's choice points and runs its pending cleanup before either step. A `'$next'` makes `state_3/2` fail, so it bypasses the cut and resumes the search instead.

The protocol assumes that `'$next'` and `'$stop'` belong to the currently suspended query. With one controlling client and per-sender ordering, that holds for every message that arrives after the call; obsolete commands preceding the selected call are consumed by `wait_call/2`. No query reference is needed in the messages or the answers.

When a page ends with an exception rather than with the last solution, the solutions already found are not lost. The answer is first reported as an ordinary page with more to come, and the error answers the next request:

```
state_3_error(Pid, Error, Target) :-
    receive({
        '$next'(Options) ->
            (   option(target(NewTarget), Options)
            ->  nb_setarg(1, Target, NewTarget)
            ;   true
            ),
            arg(1, Target, Out),
            Out ! error(Pid, Error) ;
        '$stop' -> true
    }).
```

A client therefore observes the same sequence as a local call: some solutions, then the exception.

The client-side operations translate directly into protocol messages. For example:

```
toplevel_call(Pid, Goal) :-
    toplevel_call(Pid, Goal, []).

toplevel_call(Pid, Goal, Options) :-
    Pid ! '$call'(Goal, Options).
```

The predicates `toplevel_next/1-2` and `toplevel_stop/1` are defined similarly, by sending `$next` and `$stop` messages. These operations do not wait for an answer: replies are sent to the selected target, while selective receive in the toplevel determines when each request is admissible. Abort and halt instead use the runtime's signal path and are described with the toplevel state below.

5.3.2 Why `fail/0` resumes the search

SWI-Prolog's `findnsols/4` returns at most a requested number of solutions, but is itself nondeterministic: backtracking asks it for the next slice.[2] An omitted limit instead selects `findall/3`, which collects all solutions and leaves no continuation. Together with `offset/2`, these collectors give the operation used by `state_2/6`:[3]

```
slice(Goal, Template, Offset, Limit, Slice) :-
    (   Limit == all
    ->  findall(Template, offset(Offset, Goal), Slice)
    ;   findnsols(Limit, Template, offset(Offset, Goal), Slice)
    ).
```

The cleanup argument of `call_cleanup/2` runs when the call becomes deterministic. We use that fact to distinguish a final slice from one whose suspended computation may still produce another.

```
answer(Goal, Template, Offset, Limit, Answer) :-
    Pending = pending(none),
    catch(
        call_cleanup(
            slice(capture(Goal, Pending), Template,
                  Offset, Limit, Slice),
            Det = true
        ),
        Error,
        true
    ),
    arg(1, Pending, Cell),
    (   nonvar(Error), control_exception(Error)
    ->  throw(Error)
    ;   nonvar(Error)
    ->  Answer = error(Error)
    ;   \+ term_attvars(Slice, [])
    ->  Answer = error(error(representation_error(constraints), _))
    ;   Cell = caught(E), Slice \== [], Limit \== all
    ->  Answer = pending_error(Slice, E)
    ;   Cell = caught(E)
    ->  Answer = error(E)
    ;   Slice == []
    ->  Answer = failure
    ;   var(Det)
    ->  Answer = success(Slice, true)
```

[2] https://www.swi-prolog.org/pldoc/doc_for?object=findnsols/4

[3] https://www.swi-prolog.org/pldoc/doc_for?object=offset/2

```
    ;   Answer = success(Slice, false)
    ).

capture(Goal, Pending) :-
    catch(Goal, E, record_error(E, Pending)).

record_error(E, _) :-
    control_exception(E),
    !,
    throw(E).
record_error(E, Pending) :-
    nb_setarg(1, Pending, caught(E)),
    fail.
```

Thus the `fail/0` in `state_3/2` backtracks through `state_2/6` into the same `findnsols/4` call. Its remaining choice points are the toplevel's implicit state. The compound `count(N)` is accepted directly by SWI-Prolog's `findnsols/4` and is retained with that computation, so changing its argument with `nb_setarg/3` changes the size of the next slice without restarting the goal. An implementation on another Prolog system must provide equivalent resumable slicing; accepting this mutable count term is a host-specific mechanism. Both catchers rethrow the control exceptions – the abort, the running-time limit and the engine's own unwinding, as listed by `control_exception/1` in Appendix E – because they are protocol signals, not errors produced by the submitted goal; a time limit must never wait behind solutions for another request. An ordinary exception raised by the goal is recorded by `capture/2` instead of being allowed to discard the solutions already collected; it ends the collection, and `answer/5` reports it after them. The cell holds `caught(E)` rather than `E` itself, so an exception term that happens to equal the empty marker, as `throw(none)` does, is still recognized. A request for all solutions at once has no later page, so there the error is reported alone. Finally, a page whose solutions still carry constraints is never sent: it ends the query with a representation error, as Appendix B specifies.

5.3.3 Production lifecycle management

The sketch leaves resource governance to the surrounding runtime. A toplevel can otherwise remain indefinitely in **s1**, **s2**, or **s3**, and a client-side cut cannot tell it that a suspended search has been abandoned. Production systems therefore add running and idle timeouts, parent–child lifetime containment, and `once(true)` for calls that should discard their remaining choice points after the first slice. A client that abandons an ordinary resumable call should use `toplevel_stop/1`, `toplevel_abort/1`, or `toplevel_halt/1`, according to the actor's current state and the desired outcome.

5.3.4 The enclosing state: abort and halt

The three predicates above implement the inner states. Transitions attached to their enclosing state, **PTCP**, apply regardless of which inner state is active. The `abort` transition interrupts the current computation and, for a persistent toplevel, re-enters **s1** without terminating the actor:

```
ptcp(Pid, Target, Session) :-
    catch(
        state_1(Pid, Target, Session),
        '$abort_goal',
        after_abort(Pid, Target, Session)
    ).

after_abort(Pid, Target, Session) :-
    (   terminating
    ->  true
    ;   Target ! abort(Pid),
        (   Session == false
        ->  true
        ;   ptcp(Pid, Target, Session)
        )
    ).
```

Every abort is acknowledged with `abort(Pid)`. The acknowledgement is sent by the toplevel after any answers of the aborted query, so a client that waits for it knows where the old query ends. In this sketch it goes to the target given at spawn time; the demonstrator sends it to the client of the most recent call. After an abort, a persistent toplevel resumes in **s1**. A one-shot toplevel, spawned with `session(false)`, ends instead of waiting for a call that will not come. If termination has already been committed, `terminating/0` succeeds (its definition is in Appendix E), and the abort never resumes the actor. This is why `answer/5` must rethrow `'$abort_goal'` instead of packaging it as an error answer. The `catch/3` shown here is a privileged host-level catcher inside the behaviour implementation; it is not the filtered `catch/3` exposed to submitted Web Prolog code. This PTCP boundary is the only layer permitted to consume the reserved abort control. Every client-reachable catcher propagates it, and ordinary application exceptions remain catchable in the usual way.

The local backend delivers this control through the host thread that belongs to the actor. The public predicate hides that private pid-to-thread mapping:

```
toplevel_abort(Pid) :-
    (   pid_thread(Pid, Thread)
    ->  catch(
            thread_signal(Thread, abort_signal),
            error(existence_error(_, _), _),
            true
```

```
        )
    ;   true
    ).

abort_signal :-
    (   terminating
    ->  true
    ;   throw('$abort_goal')
    ).
```

Application programs still address the actor pid, not a host thread identifier. The portable meaning of abort is the protected unwind to the PTCP boundary above; `pid_thread/2` and `thread_signal/2` belong to the local runtime rather than the actor API.

The `halt` transition, by contrast, leaves the enclosing **PTCP** state and terminates the actor. It is implemented with the general actor exit mechanism rather than a mailbox command, so it can terminate the toplevel while it is waiting in **s1** or **s3**, or while a goal is executing in **s2**. This is the operational counterpart of attaching `halt` to the enclosing state rather than to any one inner state. In particular, no separate transition from **s1** to the final state is required.

The one-argument operation requests termination immediately. The two-argument form first installs a private monitor and then waits for its matching notification:

```
toplevel_halt(Pid) :-
    exit(Pid, true).

toplevel_halt(Pid, Reply) :-
    setup_call_cleanup(
        monitor(Pid, Ref),
        (   toplevel_halt(Pid),
            receive({
                down(Pid, Ref, connection_closed) ->
                    Reply = error(connection_closed);
                down(Pid, Ref, no_process) ->
                    Reply = true;
                down(Pid, Ref, _) ->
                    Reply = true
            })
        ),
        demonitor(Ref, [flush])
    ).
```

Installing the monitor before calling `exit/2` avoids losing a fast termination. The exit signal itself is asynchronous;[4] a matching notification with a termination reason

[4] The local implementation uses `thread_signal/2`, which queues its goal and returns immediately: `https://www.swi-prolog.org/pldoc/doc_for?object=thread_signal/2`.

confirms observed termination, even if the actor exited for a reason other than the requested `true`. The special reason `no_process` confirms that the target is already absent and also yields `Reply = true`; it does not establish when or why it terminated. For a remote target, `connection_closed` means that observation was lost, so the reply is `error(connection_closed)` and termination remains unconfirmed. Making `monitor/2` the setup action ensures that the private monitor is removed even if the wait is interrupted, while the `flush` option removes any late notification bearing its fresh reference and leaves notifications from other monitors untouched.

5.4 The rpc/2-3 meta-predicate

Chapter 4 introduced `rpc/2-3` as the central construct of the sequential Prolog Web. Its interface resembles an ordinary predicate call: variables are bound on success, failure is reported as Prolog failure, exceptions are rethrown, and backtracking requests further solutions. The fact that the goal is evaluated on another node is hidden behind this familiar interface.

In the terminology introduced at the beginning of this chapter, `rpc/2-3` is a Prolog-style generic. It does not expose a message protocol to its caller. Instead, it encapsulates remote execution, answer collection, slicing, and transport behind a nondeterministic predicate interface.

There are two useful ways of implementing this abstraction. The first reconstructs it on top of a remote toplevel actor and makes the underlying PTCP interaction explicit. The second uses the stateless HTTP API and is closer to the implementation provided by the demonstrator. They expose the same logical interface but make different operational and deployment trade-offs.

5.4.1 From protocol messages to Prolog nondeterminism

A toplevel actor communicates through deterministic messages. A call produces an answer term reporting one of three protocol outcomes: success with a solution slice, failure, or an error. If further solutions may be available, the client may explicitly request the next slice. Nothing in this protocol is itself nondeterministic.

The task of `rpc/2-3` is to turn that deterministic exchange into ordinary Prolog control. Each received slice is enumerated locally using `member/2`. When a slice is exhausted, the choice point associated with `rpc/2-3` requests another slice. Failure of the remote goal becomes local failure, while a remote error becomes a local exception. An `error(Pid,E)` answer causes `rpc/2-3` to throw `E` locally. Timeouts, explicit aborts, and transport failures remain separate outcomes.

Two distinct forms of continuation are therefore involved. Within a slice, the continuation is an ordinary local Prolog choice point. Between slices, continuation

requires another protocol exchange with the remote node. The caller does not see this distinction: pressing semicolon or backtracking into the call simply asks for the next solution.

This translation from answer messages to choice points is what makes `rpc/2-3` more than an HTTP client or a request–response wrapper. It reconstructs a remote stream of substitutions as a predicate participating directly in the caller's proof search.

5.4.2 An actor-based reconstruction

The actor-based reconstruction builds on three layers. Actor processes and asynchronous messaging provide the foundation. The toplevel behaviour adds goal evaluation and answer-stream management. Finally, `rpc/2-3` hides the PTCP exchanges behind a synchronous, nondeterministic predicate interface.

The reconstruction spawns a toplevel actor on the target node, submits the goal, and then waits for PTCP answer messages. The toplevel is created with `session(false)` because it is intended to serve this one computation rather than remain available for an interactive sequence of queries.

The following code is a pedagogical reconstruction. It exposes the essential control structure but is not the implementation used by the demonstrator:

```
rpc(URI, Goal) :-
    rpc(URI, Goal, []).

rpc(URI, Goal, Options) :-
    toplevel_spawn(Pid, [
         node(URI),
         session(false),
         monitor(false)
       | Options
    ]),
    toplevel_call(Pid, Goal, Options),
    wait_answer(Pid, Goal).
```

On the ordinary protocol outcomes, the answer loop terminates on failure or error, or after the final slice has been consumed:

```
wait_answer(Pid, Goal) :-
    receive({
        success(Pid, Slice, true) ->
            (   member(Goal, Slice)
            ;   toplevel_next(Pid),
                wait_answer(Pid, Goal)
            ) ;
```

```
        success(Pid, Slice, false) ->
            member(Goal, Slice) ;
        failure(Pid) ->
            fail ;
        error(Pid, Error) ->
            throw(Error)
    }).
```

The two clauses handling `success/3` correspond to the two possible kinds of slice. If the final argument is `false`, the slice contains the last solutions and can simply be enumerated. If it is `true`, exhausting the local enumeration falls through to `toplevel_next/1`, after which the same answer loop is entered again. The recursion thus occurs only at slice boundaries; no network communication is needed while solutions remain in the current slice.

This is a direct use of the `receive/1-2` contract in Appendix B: selecting a message is committed, but the selected clause body may yield several solutions. Here `member/2` leaves the choice points that make `rpc/2-3` nondeterministic, and the recursive call receives another answer message only after the current slice is exhausted.

Despite being implemented with actors, the resulting abstraction is synchronous. The caller and callee proceed in request–response lockstep, and the caller performs no independent work while waiting for the next answer. The actors supply the mechanism, but the predicate presents a sequential interface.

This reconstruction also exposes lifecycle problems that the predicate interface would otherwise conceal. It does not monitor the toplevel or bound the receive operation, so a node failure or network partition may leave the caller waiting indefinitely. More subtly, the caller may abandon the computation by cutting away the `rpc/2-3` choice point. If the remote toplevel has reported that more solutions are available, it may then remain suspended waiting for a request that will never arrive.

The option `session(false)` does not by itself solve this abandonment problem. It terminates the actor when the query has been exhausted or explicitly stopped; it cannot detect that a choice point has disappeared on another node. A production actor-based implementation would therefore need failure detection, bounded waiting, and cleanup that associates the lifetime of the remote toplevel with the lifetime of the nondeterministic call.

5.4.3 The stateless HTTP implementation

The request–response structure just described maps naturally onto HTTP. The caller submits a goal and receives a slice of solutions. It sends another request only after local backtracking has exhausted that slice. There are no unsolicited answers or server-initiated notifications, and the remote node does not need to expose its programmable actor interface.

Each exchange can therefore be represented as a self-contained request to the node's `/call` endpoint. The request carries the goal, an answer template, an offset, a limit, and the desired representation format. The response is a success, failure, or error term of the same general form used by the toplevel protocol.

The implementation sketch below shows the central mechanism. It omits some of the validation, option processing, authentication, and error handling required by the complete demonstrator:

```
rpc(URI, Goal) :-
    rpc(URI, Goal, []).

rpc(URI, Goal, Options) :-
    parse_url(URI, Parts),
    term_variables(Goal, Vars),
    Template =.. [v|Vars],
    format(atom(GoalAtom), "(~p)", [Goal]),
    format(atom(TemplateAtom), "(~p)", [Template]),
    (   option(limit(Limit), Options)
    ->  must_be(positive_integer, Limit)
    ;   Limit = none
    ),
    Request = request(
        Parts, GoalAtom, TemplateAtom, Limit, Options
    ),
    rpc_page(Template, 0, Request).

rpc_page(Template, Offset, Request) :-
    Request = request(
        Parts, GoalAtom, TemplateAtom, Limit, Options
    ),
    Search0 = [goal=GoalAtom, template=TemplateAtom,
               offset=Offset, format=prolog],
    (   Limit == none -> Search = Search0
    ;   Search = [limit=Limit|Search0]
    ),
    parse_url(URI, [
        path('/call'),
        search(Search)
      | Parts
    ]),
    http_read_answer(URI, Options, Answer),
    rpc_answer(Answer, Template, Offset, Request).

http_read_answer(URI, Options, Answer) :-
    Holder = stream(none),
    call_cleanup(
```

```
        (   http_open(URI, Stream, Options),
            nb_setarg(1, Holder, Stream),
            read(Stream, Answer)
        ),
        close_held_stream(Holder)
    ).

close_held_stream(stream(Stream)) :-
    (   Stream == none
    ->  true
    ;   close(Stream, [force(true)])
    ).

rpc_answer(success(Slice, true), Template, Offset, Request) :-
    !,
    (   member(Template, Slice)
    ;   length(Slice, Count),
        NextOffset is Offset + Count,
        rpc_page(Template, NextOffset, Request)
    ).
rpc_answer(success(Slice, false), Template, _, _) :-
    member(Template, Slice).
rpc_answer(failure, _, _, _) :-
    fail.
rpc_answer(error(Error), _, _, _) :-
    throw(Error).
```

The public predicate first constructs a template containing the variables whose bindings must be returned. Goals and templates are serialised before being added to the HTTP request. The answer dispatcher then translates the response back into Prolog control: `member/2` enumerates a received slice, failure becomes failure, and an error term is thrown as an exception. When a non-final slice is exhausted, the offset is advanced and another HTTP request is made. The request is opened in the body of `call_cleanup/2` rather than as the setup goal of `setup_call_cleanup/3`. SWI-Prolog protects setup goals from signals, so a slow or silent remote node would otherwise delay an abort or a termination signal until it replied (Section 2.3.3). The stream is closed in cleanup only if it was opened. A caller who needs a bound on the wait passes a timeout among the HTTP options.

With no `limit` option, the request omits that parameter and asks for all solutions. A finite limit enables paging. The offset-based reconstruction may re-execute the goal to reach later slices, repeating both earlier work and any side effects performed along the way. Section E.3.2 of Appendix E explains the demonstrator's bounded continuation cache and the circumstances in which recomputation is still required.

The HTTP interface is stateless in the sense that every request contains the information needed to identify the requested portion of the answer sequence. This does not require the server to discard all execution state after each response. The

demonstrator may cache a suspended toplevel and resume it when a later request asks for another slice. Such a cached computation is an internal optimisation rather than a client-addressed session: it may be reused, expired, or evicted without changing the public protocol.

This distinction is important when a caller abandons enumeration. No explicit stop request is required by the HTTP interface, because the caller does not own an addressable remote session. A cached computation may nevertheless remain unused after the final request received from that caller. The node must therefore bound its cache and terminate suspended computations on expiry, eviction, or resource pressure.

The control structure is simple, but a complete implementation must also serialise terms unambiguously, preserve variable relationships through templates, construct URIs correctly, validate options, enforce profiles and authorisation, and deal with TLS, redirects, authentication, timeouts, malformed responses, and connection failures.

5.4.4 Comparing the two realisations

The actor and HTTP realisations implement the same predicate-level contract. Given the same remote program and the same successful sequence of protocol exchanges, they return the same substitutions in the same order. Both enumerate answers locally within a slice and perform remote communication only when another slice is required.

Their operational boundaries are different. The actor reconstruction creates an addressable remote process and communicates with it using PTCP. This makes the relationship with the toplevel behaviour particularly clear, but it requires an ACTOR node and introduces an explicit remote lifecycle. The HTTP implementation requires only the sequential query interface. It avoids a client-addressed actor and can operate between ISOBASE nodes, but it must encode the computation in HTTP requests and rely on server-managed caching when efficient resumption is desired.

The actor version is consequently the better explanatory model: it shows how deterministic answer messages become nondeterministic Prolog execution. The HTTP version is the more practical implementation for the sequential Prolog Web: its interaction pattern fits ordinary web infrastructure and does not require callers to participate in actor lifecycle management.

Neither realisation changes the logical meaning of `rpc/2-3`. The observable abstraction remains a remote predicate call over renamed-apart terms. The differences concern transport, resource ownership, failure detection, and the management of suspended computation.

The next sections consider two further Prolog-style generics, `parallel/1` and `first_solution/2-3`. Like `rpc/2-3`, they hide actor coordination behind conventional predicate interfaces. Unlike `rpc/2-3`, however, they commit to selected solutions rather than exposing a remote computation nondeterministically on backtracking.

5.5 The parallel/1 meta-predicate

In this section we implement `parallel/1`, the meta-predicate introduced in Chapter 3 (Section 3.3.5). A call to `parallel(Goals)` should wait until its outcome is known and its workers have been cleaned up, succeed with the returned variable bindings if all goals succeed, fail promptly if any goal fails, and promptly rethrow an exception raised by any goal. These guarantees require the goals to be independent: they must not communicate through shared variables, global state, or other side channels.

The predicate is semideterministic. Each goal contributes its first success, if it has one. From the caller's perspective this is a conventional predicate-level interface; underneath, one monitored actor is used for each goal.

The listings below assume that `Goals` is a proper list and omit the public predicate's list and callability validation. The complete interface raises the errors specified in Appendix B.

5.5.1 From two goals to a list

We begin with an incomplete two-goal version:

```
parallel(Goal1, Goal2) :-
    self(Self),
    spawn((call(Goal1), Self ! Pid1-Goal1), Pid1),
    spawn((call(Goal2), Self ! Pid2-Goal2), Pid2),
    receive({Pid1-Reply1 -> Goal1 = Reply1}),
    receive({Pid2-Reply2 -> Goal2 = Reply2}).
```

Each worker calls its copied goal and, on success, sends its pid paired with the instantiated copy back to the parent. The receive pattern selects the copy into a fresh variable; its body then unifies that copy with the corresponding goal in the caller to return the bindings. This explicit unification is essential when goals share variables: after one reply binds a shared variable, a later incompatible reply must make the call fail rather than remain unmatched in the mailbox. The two replies may arrive in either order: selective receive defers the second worker's reply if it arrives before the first one being awaited.

The same pattern generalises naturally from two goals to a list:

```
parallel(Goals) :-
    maplist(par_solve, Goals, Pids),
    maplist(par_yield, Pids, Goals).

par_solve(Goal, Pid) :-
    self(Self),
    spawn((call(Goal), Self ! Pid-Goal), Pid).
```

```
par_yield(Pid, Goal) :-
    receive({Pid-Reply -> Goal = Reply}).
```

This version works when every goal succeeds. If a goal fails or throws an exception, however, its worker sends no result and the corresponding `receive/1` waits indefinitely. Monitors supply the missing outcome.

5.5.2 Detecting failure and errors

Each worker is now monitored, and `par_yield/3` accepts either the expected result or an abnormal termination notification from any worker belonging to this invocation:

```
parallel(Goals) :-
    maplist(par_solve, Goals, Pids),
    maplist(par_yield(Pids), Pids, Goals).

par_solve(Goal, Pid) :-
    self(Self),
    spawn((call(Goal), Self ! Pid-Goal), Pid, [
        monitor(true)
    ]).

par_yield(Pids, Pid, Goal) :-
    receive({
        Pid-Reply ->
            (   Goal = Reply
            ->  receive({down(Pid, _, _) -> true})
            ;   tidy_up_all(Pids), fail
            ) ;
        down(FailedPid, _, false)
                if known_worker(FailedPid, Pids) ->
            tidy_up_all(Pids),
            fail ;
        down(FailedPid, _, Reason)
                if (known_worker(FailedPid, Pids),
                    (Reason \== true ; FailedPid == Pid)) ->
            tidy_up_all(Pids),
            worker_error(FailedPid, Reason, E),
            throw(E)
    }).

known_worker(Pid, Pids) :-
    ground(Pid),
```

```
    memberchk(Pid, Pids).

worker_error(Pid, Reason, Error) :-
    (   nonvar(Reason), Reason = exception(E)
    ->  Error = E
    ;   Error = worker_exit(Pid, Reason)
    ).
```

On success, the worker sends its result and then terminates normally. Once its result has been accepted, the nested `receive` consumes the corresponding termination notification regardless of its reason: it now confirms that the worker has stopped, leaving neither message in the caller's mailbox. A failure or error from a worker still being awaited determines the outcome of the whole call, so the remaining workers are cleaned up immediately by `tidy_up_all/1`, defined in the next subsection. Matching only the current `Pid` in the abnormal clauses would unnecessarily delay detection until that particular worker was awaited.

The last clause preserves an exception's original term; any other exit without a result becomes `worker_exit(Pid,Reason)`. Its guard leaves a later worker's normal `true` notification deferred until that worker's result is collected. For the current worker, a `true` notification without a preceding result is an error too: `exit(true)` does not supply the goal's answer.

The guards admit notifications from every pid in `Pids` while leaving unrelated monitor messages in the caller's mailbox. They test the pid with `ground/1` before looking it up: a guard can bind variables in a message, so `memberchk/2` alone would accept a notification carrying an unbound pid as if it came from the first worker. This matters because `parallel/1` may be called by an actor that is monitoring other processes at the same time.

5.5.3 Race-free cleanup

Cleanup must do more than kill the remaining workers. It must also ensure that neither result messages nor monitor notifications from this invocation are left behind in the caller's mailbox:

```
tidy_up_all(Pids) :-
    maplist(tidy_up, Pids).

tidy_up(Pid) :-
    demonitor(Pid, [flush]),
    monitor(Pid, CleanupRef),
    exit(Pid, kill),
    receive({
        down(Pid, CleanupRef, _) -> true
    }),
```

```
    drain_mailbox(Pid).

drain_mailbox(Pid) :-
    receive({
        Msg if ( compound(Msg),
                 arg(1, Msg, P),
                 P == Pid ) ->
            drain_mailbox(Pid)
    }, [
        timeout(0)
    ]).
```

The creation-time monitor is first removed and any notification it has already produced is flushed. The spawn-time monitoring contract uses the worker pid as that monitor's reference, which is why the first argument to `demonitor/2` is `Pid`. A private cleanup monitor is then installed before the exit request is sent.

The cleanup procedure relies on two guarantees for workers and callers in the same runtime. First, every result enqueued by an immediate send directly to the caller's pid precedes the corresponding termination notification in that mailbox. This applies to existing monitors and to monitors installed during or after termination. If a new monitor reports `no_process`, any earlier result is already in the mailbox; that notification confirms absence, not the worker's termination reason. The guarantee concerns mailbox insertion, not the order in which the caller selects messages.

Second, `demonitor(Ref,[flush])` removes any queued notification from that monitor and prevents a later one from arriving. Once the caller receives the cleanup monitor's notification, it can therefore drain the worker's remaining messages with a zero-time receive: no worker result or notification from the old monitor can arrive afterwards. Results already consumed by the caller need no further handling. No ordering between the cleanup monitor installation and the worker's result send is needed.

Section E.3.3 explains how the demonstrator implements these guarantees.

This proof does not extend to remote workers, delayed sends, or forwarded results. In particular, `connection_closed` cannot justify a drain. A distributed version would need a delivery protocol that orders a final completion marker after all results on the same stream, with explicit handling of transport loss; merely adding a monitor would not establish that ordering.

The drain terminates because `receive/2` calls `true` by default when its zero timeout expires. Its guard accepts a pending message only if it is compound and its first argument is identical to `Pid`. The test uses == rather than unification so that a message whose first argument is an unbound variable is not mistaken for one from the worker. Unrelated messages, including atoms, stay deferred.

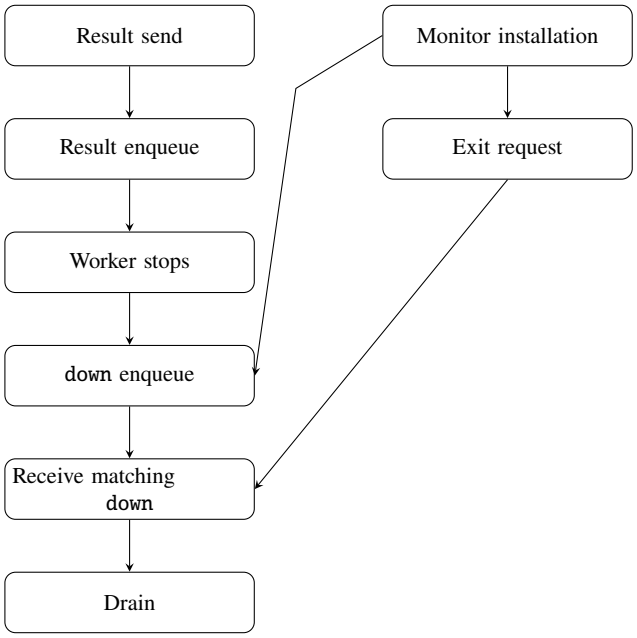


Fig. 5.3 Required local cleanup ordering when a result is sent. Arrows express required precedence; placement alone imposes none. Monitor installation and exit request are unordered with respect to result delivery and worker termination: the worker may already have exited. The path from result enqueue through worker termination to `down` enqueue is the local runtime guarantee, including a late monitor's `no_process`. If cancellation prevents the result, the first two events are absent.

5.5.4 Behaviour and performance

We can now check success, failure, and exception propagation. The following three sleeps overlap, so the call takes approximately as long as the slowest goal rather than the sum of all three delays:

```
?- _Goals = [(X=a,sleep(1)),(Y=b,sleep(3)),(Z=c,sleep(2))],
   time(parallel(_Goals)).
% 189 inferences, 0.000 CPU in 3.007 seconds
X = a,
Y = b,
Z = c.
?-
```

On this example the elapsed time is about three seconds, compared with about six seconds for sequential execution.

If one goal fails, the entire call fails without waiting for slower siblings:

```
?- _Goals = [(X=a,sleep(1)),(Y=b,fail),(Z=c,sleep(2))],
   time(parallel(_Goals)).
```

```
% 105 inferences, 0.000 CPU in 0.001 seconds
false.
?-
```

An exception is likewise propagated after the other workers have been cleaned up:

```
?- _Goals = [(X=a,sleep(1)),(Y=b,sleep(a)),(Z=c,sleep(2))],
   time(catch(parallel(_Goals),E,true)).
% 126 inferences, 0.000 CPU in 0.001 seconds
E = error(type_error(float, a), _).
?-
```

Actor creation, monitoring, message copying, and cleanup all have a cost. Goals must therefore perform enough work for their overlap to repay that cost. This implementation also creates one actor per goal; very large goal lists consequently have a correspondingly large resource footprint.

5.5.5 Relation to concurrent/3

SWI-Prolog's `library(thread)` provides the related predicate `concurrent/3`.[5] Its observable success, failure, exception, and variable-binding behaviour is close to that of `parallel/1`, but the interfaces serve somewhat different purposes. The first argument of `concurrent/3` bounds the number of worker threads, and its third argument supplies thread-creation options. By contrast, `parallel/1` presents a narrower interface and creates one actor for each goal.

Logtalk provides another direct precedent. Its `threaded/1` construct supports concurrent conjunctions and competing alternatives: with a disjunction, the first successful alternative terminates the remaining competitors (Moura et al., 2008, 2009). These operations address the same coordination patterns as `parallel/1` and `first_solution/2-3`, although their interfaces and execution contexts differ. The point of the reconstructions here is to show how Web Prolog's monitored actors compose into these patterns.

5.6 The first_solution/2-3 meta-predicate

Section 3.3.5 introduced `first_solution/2-3` as a way to coordinate alternative solvers. Here we implement its synchronous predicate interface using actors.

SWI-Prolog's `library(thread)` also provides `first_solution/2-3`: run several alternative solvers concurrently and return the result produced by the first one to succeed.[6] The Web Prolog version adopts this predicate-level idea but implements

[5] `https://www.swi-prolog.org/pldoc/man?section=thread`

[6] `https://www.swi-prolog.org/pldoc/man?section=thread`

the race with monitored actors. Once a winner has been selected, all remaining workers are terminated and their messages are removed from the caller's mailbox.

5.6.1 Racing alternative solvers

Section 3.3.5 illustrated the race with delayed goals. For a practical application, consider an example adapted from Wielemaker: when it is unclear whether a graph should be searched breadth-first or depth-first, we can start both strategies and accept whichever finds a path first:

```
search_graph(Graph, Path) :-
    first_solution(Path, [
        breadth_first(Graph, Path),
        depth_first(Graph, Path)
    ]).
```

Unlike `parallel/1`, which requires every goal to succeed, `first_solution/2-3` treats its goals as alternatives. Each worker receives a copy of the solution template, and the first result message selected by the caller binds the caller's template.

The following implementation sketch uses the local worker cleanup developed for `parallel/1`. It assumes valid goals and policy options; the public validation contract is given in Section B.6. The two-argument form supplies portable default policies: a failed solver is ignored while alternatives remain, whereas an error terminates the race and is thrown:

```
first_solution(Solution, Goals) :-
    first_solution(Solution, Goals, [
        on_fail(continue),
        on_error(stop)
    ]).

first_solution(Solution, Goals, Options) :-
    option(on_fail(OnFail), Options, continue),
    option(on_error(OnError), Options, stop),
    maplist(first_solve(Solution), Goals, Pids),
    wait_first(Pids, Solution, OnFail, OnError).

first_solve(Solution, Goal, Pid) :-
    self(Self),
    spawn((call(Goal), Self ! Pid-Solution), Pid, [
        monitor(true)
    ]).

wait_first([], _, _, _) :-
```

```
    !,
    fail.
wait_first(Pids, Solution, OnFail, OnError) :-
    receive({
        Winner-Solution
                if known_worker(Winner, Pids) ->
            tidy_up_all(Pids) ;
        down(FailedPid, _, false)
                if known_worker(FailedPid, Pids) ->
            (   OnFail == continue
            ->  select(FailedPid, Pids, Rest),
                wait_first(Rest, Solution, OnFail, OnError)
            ;   tidy_up_all(Pids),
                fail
            ) ;
        down(FailedPid, _, Reason)
                if known_worker(FailedPid, Pids) ->
            (   OnError == continue
            ->  select(FailedPid, Pids, Rest),
                wait_first(Rest, Solution, OnFail, OnError)
            ;   tidy_up_all(Pids),
                worker_error(FailedPid, Reason, Error),
                throw(Error)
            )
    }).
```

Every receive head is guarded by membership in `Pids`, tested with `known_worker/2` from the previous section. An unrelated pair in the caller's mailbox must not be mistaken for a winning result, and a `down/3` notification from an unrelated monitor must not affect the race. On success, `tidy_up_all/1` from the implementation of `parallel/1` supplies the same ordering barrier and race-free mailbox cleanup described in the previous section.

If no worker remains, the call fails. The predicate is therefore semideterministic: it returns the first solution selected by the mailbox but does not enumerate later solutions on backtracking.

5.6.2 Failure and error policies

The option `on_fail(continue)` removes a failed solver and keeps waiting; `on_fail(stop)` instead cleans up all workers and fails the entire call immediately. Similarly, `on_error(continue)` ignores an error from one solver while alternatives remain, whereas `on_error(stop)` cleans up and throws it. The accepted values are thus `continue` and `stop`; validation and the associated domain errors are omitted from this design sketch.

An exit without a result, other than ordinary failure or an exception, is reported as `worker_exit(Pid,Reason)` by the shared `worker_error/3` helper. It follows the same `on_error` policy: stop cleans up and throws that term; continue discards the solver and waits for another. This includes `exit(true)` without a result.

The defaults make failure local to an alternative but treat an error as significant. This is often useful when the solvers are different search strategies: one strategy may simply find no answer, while an error may indicate a defect or resource problem that should be reported. Applications can select different policies explicitly through `first_solution/3`.

If all alternatives fail under `on_fail(continue)`, or if all remaining alternatives are discarded under `on_error(continue)`, the worker list eventually becomes empty and the whole call fails.

5.6.3 An example

Using a simple simulation of two long-running goals, here is a trial run that shows that our predicate works as intended:

```
?- time(first_solution(X, [(sleep(2),X=a),(sleep(1),X=b)])).
% 164 inferences, 0.000 CPU in 1.05 seconds
X = b.
?-
```

The second solver wins after approximately one second. Cleanup then terminates the slower worker before returning `X = b` to the caller.

5.7 Summary

The examples in this chapter illustrate two complementary ways of packaging concurrent computation in Web Prolog. Protocol-oriented generics such as the server, supervisor, and toplevel actor expose stable message protocols and hide recurring operational machinery for state management, lifecycle control, monitoring, and query execution. They do not expose their internal proof search through caller backtracking. Predicate-oriented generics such as `rpc/2-3`, `parallel/1`, and `first_solution/2-3`, by contrast, hide protocol exchanges or actor coordination behind conventional predicate interfaces.

Several techniques recur across otherwise different abstractions. Fresh references correlate requests and replies. Selective receive lets a process react to whichever relevant outcome arrives first while deferring unrelated messages. Monitors turn silent process termination into explicit data. Acknowledgements distinguish an accepted operation from a message merely placed in transit. Cleanup monitors provide ordering barriers before mailbox draining. In the toplevel actor, controlled backtracking

and mutable cells support demand-driven solution slices; in `rpc/2-3`, local choice points turn those slices back into a nondeterministic predicate. These examples show ordinary Prolog mechanisms and actor mechanisms reinforcing one another.

Chapter 6
The statechart behaviour

The earlier chapters introduced several generic behaviours for actors in Web Prolog: the toplevel behaviour, the server behaviour, and the supervisor behaviour. Each separates recurring operational machinery from the application logic of an actor. This chapter introduces the *statechart behaviour*, in which the control logic of a single actor is represented explicitly using hierarchical states, parallel regions, and event-triggered transitions.

The behaviour is expressed in Statechart XML, or SXML, a compact XML notation whose tree structure mirrors the hierarchical structure of a statechart. An SXML document is directly executable as a Web Prolog actor: it can be launched with `statechart_spawn/2`, participate in a supervision tree, and exchange messages like any other actor. Events and data are Prolog terms, while guards and actions are ordinary Web Prolog code. The associated predicates form the `statechart` family of the ACTOR profile; a node owner may enable or disable that family as part of the node's advertised offer.

6.1 Statecharts

Invented by David Harel, Statecharts is a graphical formalism for specifying reactive systems (Harel, 1987). Building on the concept of a state machine, it adds hierarchy, parallel regions (orthogonality), and event visibility across the active states and regions. Like logic, it is a "tool for thinking" with a formal semantics (Harel and Naamad, 1996), but it is also explicitly visual – a way of "tapping the potential of high bandwidth spatial intelligence, as opposed to lexical intelligence used with textual information" (Samek, 2002).

Statecharts have been used in areas ranging from embedded control and user interfaces to conversational systems and communication protocols (Samek, 2002; Horrocks, 1999; Skantze and Moubayed, 2012; Cicirelli et al., 2009).

SXML is a lightweight textual adaptation of the statechart formalism, with Prolog as both its data model and executable content. Its surface syntax resembles SCXML, but it is intentionally not a conforming SCXML language binding. No familiarity with SCXML is assumed in this chapter. Appendix C.11 records which SCXML constructs SXML retains, adapts, or omits.

6.2 Statechart actors

A *statechart actor* is a Prolog actor whose control logic is given by a statechart document. Conceptually, it behaves like an ordinary actor with a mailbox, except that incoming messages are interpreted as *events* that drive a structured state machine rather than an ad hoc `receive/1-2` loop.

Events are just messages: any incoming actor message (a Prolog term) can serve as an external event. When an event is processed, it is *visible* to all currently active parts of the machine (including parallel regions), and it may trigger one or more transitions whose actions run as Web Prolog code.

Execution follows the standard statechart *run-to-completion* intuition: processing one external event may trigger a cascade of internal work, such as taking eventless transitions and handling internally raised events, before the actor consumes the next external message. A well-formed reaction is expected to terminate; a later example shows what happens when it does not. This makes each terminating reaction behave like an atomic control step at the statechart level, even though the actor itself remains an ordinary concurrent process.

In the examples below, a statechart document can therefore be read as a compact description of (i) which states are active, (ii) which incoming events matter in those states, and (iii) what effects and state changes occur when such an event is observed. SXML takes its transition-selection and run-to-completion model from SCXML, while adapting event matching, executable content, and process invocation to Web Prolog. Section 6.7 summarises the resulting execution model; Appendix C gives the element-level contract.

6.2.1 A first statechart

In its simplest form, a statechart is just a deterministic state machine, in which state transitions are triggered by events. The simple statechart in Figure 6.1 serves as our

first example. The labelling of one of the transitions with the symbol `play` suggests that the example is drawn from the field of game development.[1]

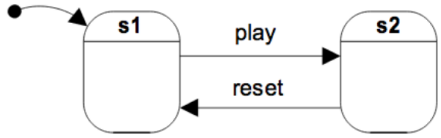


Fig. 6.1 A minimal statechart example: event-triggered transitions in a flat machine.

Here, `s1` is the initial state. If the next selected event is `play`, a transition to `s2` is taken. For any other event, the machine will remain in `s1` and the event will be discarded rather than deferred.

Statecharts within the SXML profile can be translated into documents written in an XML-based syntax. Here, a document that captures the statechart in Figure 6.1 is shown:

```
<statechart version="0.2" initial="s1">
    <state id="s1">
        <onentry>
            output('IDLE')
        </onentry>
        <go to="s2" on="play"/>
    </state>
    <state id="s2">
        <onentry>
            output('PLAYING')
        </onentry>
        <go to="s1" on="reset"/>
    </state>
</statechart>
```

The required attribute `version="0.2"` identifies the SXML notation accepted by the demonstrator. Note also that two <onentry> elements, the significance of which is not shown in Figure 6.1, have been added to the document. They contain *executable content* – instructions which allow the state machine to *do* things such as modifying its data model or interacting with external entities. Executable content must be specified using Web Prolog source code. In the case of the above example, the Web Prolog built-in predicate `output/1` sends `output(Pid,'IDLE')` or `output(Pid,'PLAYING')` to the spawning process each time `s1` or `s2` is entered, exactly as `output/1` does in other actors. Executable content may also appear in <onexit> and <go> elements.

[1] We are borrowing some of our examples from (Brusk and Lager, 2008).

6.2.2 Running a statechart actor

An SXML document is executed by an ordinary Prolog Web actor, either in a browser or on a node accessed from a browser. To create such an actor, we use `statechart_spawn/2`:

```
?- statechart_spawn(Pid, [
       src_uri('http://n6.org/game.xml'),
       monitor(true),
       trace(false)
   ]).
Pid = 78230912.
?- flush.
Shell got output(78230912,'IDLE')
true.
?-
```

When `statechart_spawn/2` is called, a new actor is spawned that interprets the statechart. As specified by the <onentry> element in the initial state, the statechart actor immediately sends `output(78230912,'IDLE')` to its parent. Equipped with the pid, we can use `!/2` to send an event to the actor, causing a transition from `s1` to `s2`. In general, an event may be any transportable Web Prolog term; here, the relevant `on` attribute tells us that the atom `play` will do:

```
?- $Pid ! play.
true.
?- receive({output($Pid, What) -> true}).
What = 'PLAYING'.
?-
```

We may now send the event `reset` to return to `s1`, and so on. For orderly termination, a client uses `statechart_halt/2`. This lets the interpreter run the <onexit> content of active states before replying. The form `statechart_halt/3` bounds the wait and forcibly terminates the actor if orderly shutdown does not complete within the specified time. The lower-level `exit/2` operation remains available when immediate actor termination is required.

Since the `monitor` option was set to `true`, the runtime also notifies the parent with a `down/3` message when the actor terminates, exactly as it does for an ordinary actor.

During development, the `trace/1` option can expose event selection, transitions, configurations, and invocation. Appendix C.10 describes the emitted trace terms and their diagnostic use.

Other options to `statechart_spawn/2` work as expected. For example, we can run the statechart on a remote node by passing a `node(URI)` option, and we can provide the source directly using `src_text` rather than fetching it from a URI.

The examples in the rest of this chapter are organised around a small set of extensions that make statecharts practically useful as an actor behaviour on the

Prolog Web. Each extension adds a distinct feature – some at the state-machine level, others at the level of interaction with the surrounding actor system. Concretely, the extensions we rely on are:

- Hierarchy and history states
- Parallelism (aka orthogonality)
- Broadcast communication
- Process invocation
- Inter-process communication
- Data model and scripting language

In the sections that follow, the left column highlights statechart-level structuring mechanisms, while the right column highlights how those mechanisms connect to actors and Prolog in the Web Prolog setting.

6.3 Hierarchy and history

Statecharts may be *hierarchical*: a state may contain another statechart down to an arbitrary depth. A complex state may contain a *history state*, serving as a memory of which substate S the complex state was in, the last time it was left for another state. Transition to the history state implies a transition to S.

The statechart in Figure 6.2 exemplifies both hierarchy and the history state by enhancing our simple game with a pause-and-resume functionality.

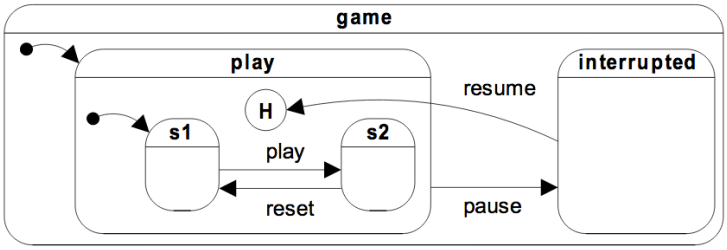


Fig. 6.2 Pause-and-resume via hierarchy and history state in the game controller.

Suppose that the current state is `s2`, and that the next selected event is `pause`. The transitions available in `s2` are considered from the inside out. The `reset` transition in `s2` does not match, but the `pause` transition inherited from the enclosing `play` state does, so a transition to `interrupted` takes place. If a `resume` event then appears, the system transitions to the history state `H`, which returns it to `s2`.

Documents are structured so that they mirror the topology of the statecharts they represent. A document that captures the statechart in Figure 6.2 is shown below:[2]

[2] A history state is shallow by default: it remembers the immediate child that was active. The optional attribute `type="deep"` instead remembers the complete active descendant configuration. The pause-and-resume example cannot distinguish the two because `s1` and `s2` are already atomic states.

```
<statechart version="0.2" initial="game">
    <state id="game" initial="play">
        <state id="play" initial="s1">
            <history id="H">
                <go to="s1" />
            </history>
            <state id="s1">
                <go to="s2" on="play" />
            </state>
            <state id="s2">
                <go to="s1" on="reset" />
            </state>
            <go to="interrupted" on="pause" />
        </state>
        <state id="interrupted">
            <go to="H" on="resume" />
        </state>
    </state>
</statechart>
```

Hierarchy supports two complementary design moves. In *refinement* (top-down), an abstract control state is elaborated into substates whose internal transitions make the behaviour explicit. In *clustering* (bottom-up), a family of closely related states is grouped under a superstate, so that shared transitions and common entry/exit behaviour can be expressed once. Both moves reduce state explosion and make large machines readable: the top level can remain a small "map" of the application, while local detail is pushed down into the regions where it belongs. Together, hierarchy and history give *interruption* and *resumption* a useful first-class representation.

6.4 Parallelism and broadcast communication

A hallmark of statecharts is *orthogonality*: a state may contain two or more regions that execute in parallel. Operationally, the machine is simultaneously in one active state in each region. The point is not CPU-level parallelism, but a way to factor one reactive controller into several facets that evolve together.

6.4.1 An NPC controller

To see why orthogonality matters, consider a toy controller for a non-player character with two independent emotion dimensions and one behaviour mode. Figure 6.3 sketches the idea: the character is simultaneously in one state in each emotion

dimension and in one behaviour state. When an event is processed, it may trigger coordinated changes across several regions.

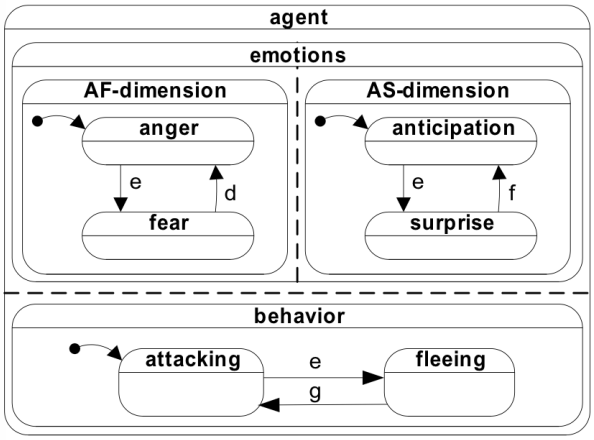


Fig. 6.3 Orthogonal regions: two emotion dimensions plus one behaviour mode in an NPC controller.

In the figure, an `e` event (think: an explosion) moves the character into `fear`, `surprise`, and `fleeing` "at once", because all three regions observe the same event while they are active. Without parallel regions, we would be forced to encode combinations such as `anger_anticipation_attacking`, `fear_surprise_fleeing`, and so on, and the number of states grows quickly as more dimensions are added.

Here is how to render this machine in XML:

```
<statechart version="0.2" initial="agent">
    <parallel id="agent">
        <parallel id="emotions">
            <state id="AF-dimension" initial="anger">
                <state id="anger">
                    <go to="fear" on="e"/>
                </state>
                <state id="fear">
                    <go to="anger" on="d"/>
                </state>
            </state>
            <state id="AS-dimension" initial="anticipation">
                <state id="anticipation">
                    <go to="surprise" on="e"/>
                </state>
                <state id="surprise">
                    <go to="anticipation" on="f"/>
```

```
                </state>
            </state>
        </parallel>
        <state id="behavior" initial="attacking">
            <state id="attacking">
                <go to="fleeing" on="e"/>
            </state>
            <state id="fleeing">
                <go to="attacking" on="g"/>
            </state>
        </state>
    </parallel>
</statechart>
```

6.4.2 An unbounded internal cascade

The following minimal example has two regions that raise events for each other. It shows how broadcast works, and what happens when it never stops.

```
<statechart version="0.2" initial="Start">
    <parallel id="Start">
        <state id="Pinger">
            <onentry>raise(pong)</onentry>
            <go on="ping">
                writeln('Ping'), raise(pong)
            </go>
        </state>
        <state id="Ponger">
            <go on="pong">
                writeln('Pong'), raise(ping)
            </go>
        </state>
    </parallel>
</statechart>
```

Note the use of targetless <go> elements (elements without a `to` attribute). These transitions consume an event (and may run executable content, such as `raise/1`) without changing the active configuration, so no <onentry> or <onexit> code is triggered.

On entry, `Pinger` raises `pong`; the `Ponger` region consumes it and raises `ping`; the `Pinger` region consumes that and raises `pong` again; and so on without end. Because run-to-completion drains the *internal* event queue before the next *external* message is consumed (as described in the next subsection), and because each reaction replenishes that queue, the queue never empties. The actor never completes its first

macrostep and never returns to consume an external event: it is livelocked. Run-to-completion executes whatever cascade the document describes, including one that never ends. A cascade is only well-behaved if it is guaranteed to terminate. A guard that counts rounds and stops raising after a given number would fix the problem.

An orderly request made with `statechart_halt/2` cannot be processed while the actor remains trapped in such a macrostep. `statechart_halt/3` addresses this failure boundary by escalating to forced termination after its timeout.

6.4.3 What "broadcast" means here

The key semantic idea is the *configuration*: the set of active states (and regions) that the machine is currently in. Whenever the interpreter selects an event for processing, that event is matched against transitions in *all* active states in the configuration. In this sense the event is "broadcast": it is visible everywhere in the current machine, even though only some region(s) react to it. Any internal events raised while handling the current input are processed before the next external message is consumed, so the effects within one reaction are deterministic given the current configuration, the data model, and the selected event.

6.4.4 Why not model each region as a separate actor?

A natural objection is that the Prolog Web already provides actor-level concurrency: why not represent each region as its own actor supervised by a parent, and coordinate by ordinary messaging? This is certainly possible, and it is often the right choice when true independence, failure isolation, distribution, or distinct lifetimes are required.

Orthogonality, however, is not primarily a claim about *parallel execution*; it is a way to specify a *single* reactive controller with an atomic, run-to-completion reaction. In a statechart, one selected event is visible to all active regions, transitions are chosen under a single set of priority and conflict-resolution rules, actions run, and the machine commits one coherent update of configuration and data model. An actor-only encoding can approximate the same effect, but typically only by introducing an explicit coordination protocol and a coordinator that reimplements the statechart scheduler in ad hoc code, while also making cross-region invariants harder to maintain under message interleavings.

6.5 The Web Prolog data model

Statecharts provide structure for reactive control, but applications such as conversational agents also need knowledge representation, planning, and reasoning. These require facilities beyond the statechart control model itself. In SXML, Web Prolog

supplies the data model and scripting language, allowing transitions to draw on logical inference and rule-based decisions.

6.5.1 Guarded transitions can derive information

SXML reuses the logical guard semantics of `receive/1-2`, but its event matching differs: an `on` pattern uses unification rather than one-sided matching. A transition first unifies its pattern with the selected event and then proves its `if` goal within the statechart actor's logical environment. A correlation guard must test the supplied value, for example with ==, rather than bind it to the expected value. The chart's private Prolog data model is only one component of this environment. The actor may also call predicates from the shared database of the node on which it is running. In the following example, the shared database contains the fact `online(printer2)`:

```
<statechart version="0.2" initial="ready">
    <datamodel>
        authorized(alice).
        owns(alice, printer2).
    </datamodel>
    <state id="ready">
        <go on="print(User,File)"
            if="authorized(User),
                owns(User, Printer),
                online(Printer)">
            format("Routing ~w to ~w~n", [File,Printer])
        </go>
    </state>
</statechart>
```

The guard can therefore combine private and shared knowledge while extending the bindings contributed by the event. If the event is `print(alice,'report.pdf')`, event matching binds `User` and `File`, while the guard derives `Printer = printer2`. That binding is then in scope in the transition's executable content. The guard both checks that printing is allowed and finds the printer to use.

As with receive guards (Section 2.3.4), a guard is evaluated for its first solution only. If matching or guard evaluation fails, the candidate's bindings are discarded before another transition is considered. Portable guards must be observationally free of side effects. The runtime need not enforce this restriction: discarding bindings does not undo effects performed by a guard, whether it fails or raises an exception.

6.5.2 Eventless transitions as forward chaining

So far, every transition has reacted to an event. Statecharts also support *eventless* transitions: transitions without an `on` attribute that may fire whenever their guard condition holds in the current configuration. Combined with a private and dynamic Prolog data model, this gives statecharts a form of forward-chaining inference: the machine repeatedly applies condition-action rules until no more apply, then waits for the next event. Here is a simple example, implementing Euclid's algorithm for calculating the greatest common divisor of a set of numbers:

```
<statechart version="0.2" initial="Run">
    <datamodel>
        :- dynamic int/1.
        int(25). int(10). int(15). int(30).
    </datamodel>
    <state id="Run">
        <go if="int(X), int(Y), X > Y">
            Z is X-Y,
            retract(int(X)),
            assert(int(Z))
        </go>
        <go to="Stop" if="int(X)">
            writeln(X)
        </go>
    </state>
    <final id="Stop"/>
</statechart>
```

The example illustrates another rule of statechart semantics: if the children of an active state contain more than one `<go>` element, they are tried in document order. In this case, if the order between the `<go>` elements was switched, the statechart would just write `25` (instead of the correct `5`) and then transition to the final state `Stop`, and terminate.

The entire calculation takes place within one macrostep as a cascade of eventless transitions – exactly the kind of cascade that the preceding section warned must terminate. Here termination is guaranteed because each subtraction strictly decreases the sum of the positive integers in the data model. Viewed another way, the example is a small forward-chaining production system: guarded rules repeatedly transform a Prolog knowledge base until no higher-priority rule applies.

These dynamic clauses are private to the statechart actor, with the properties discussed in Section 3.2.3.

6.5.3 Timed transitions

A state may also have a transition that becomes eligible after a specified delay. The `after` attribute gives the delay in seconds:

```
<state id="Color">
    <onentry>
        writeln('What color do you want?')
    </onentry>
    <go to="selectFormItem" on="says(color(Color))">
        assert(color(Color))
    </go>
    <go to="selectFormItem" after="5">
        writeln('I did not hear you.')
    </go>
</state>
```

The timer is armed when `Color` is entered. If a colour arrives first, the ordinary transition leaves the state and the timer is cancelled automatically. Otherwise, after five seconds the timed transition runs. The timeout does not interrupt executable content or a run-to-completion reaction already in progress; its firing is queued for subsequent processing. The assertion in this example has the same actor-local meaning as the updates in the preceding example: it changes only the statechart actor's private data model.

Timers are state-scoped. The precise activation, re-entry, and attribute rules are given in Appendix C.5.2.

6.5.4 Deferred events

Discarding an event that is irrelevant in the current state is usually the right default, but sometimes the event has arrived too early rather than being irrelevant. A device may receive a command while it is still initialising, for example. Such cases can be expressed with <defer>, a child of <state>:

```
<state id="initialising">
    <defer on="command(_)"/>
    <go to="ready" on="initialised"/>
</state>
<state id="ready">
    <go on="command(C)">
        perform(C)
    </go>
</state>
```

A `command` received in `initialising` enables no transition, but matches the `<defer>` declaration and is therefore postponed. When `initialised` changes the configuration, postponed commands are re-offered in their original order. They are processed before the actor receives a newer external message, so the reconsideration remains part of the run-to-completion machinery. Each pass re-offers a snapshot of the postponed queue; an event deferred again waits for a further configuration change. The trigger is a net change in the active-state configuration after internal processing settles. A targetless transition, or a self-transition that exits and re-enters the same states without a net configuration change, does not by itself cause re-offering.

An enabled transition always takes precedence over deferral. A declaration may also have an `if` guard, with variables from its `on` pattern in scope:

```
<defer on="command(C)" if="temporarily_unavailable(C)"/>
```

The postponed queue is separate from the actor mailbox and has no implicit size limit, so broad patterns in long-lived states require the same resource discipline as any other application-level buffer. Appendix C.7 states the precise search, re-offer, and discard rules.

6.6 Process invocation and communication

Statecharts become much more useful once they can spawn other processes and *communicate* with them. In the Web Prolog setting, the invoked processes are simply actors: toplevels, other statechart actors, or services running on remote nodes. For example, the statechart below spawns a remote toplevel actor, submits a query, prints each answer as it arrives, and terminates when the last answer has been received.

```
<statechart version="0.2" initial="spawn-ask-collect">
    <state id="spawn-ask-collect" initial="ask">
        <spawn type="toplevel"
               options="[node('https://n9.org'), session(false)]">
            p(a). p(b). p(c).
        </spawn>
        <state id="ask">
            <go to="collect" on="spawned(Pid)">
                toplevel_call(Pid, p(X), [
                    limit(1),
                    template(X)
                ])
            </go>
        </state>
        <state id="collect">
            <go on="success(Pid,Data,true)">
                writeln(Data),
```

```
            toplevel_next(Pid)
        </go>
        <go to="f" on="success(_,Data,false)">
            writeln(Data)
        </go>
    </state>
  </state>
  <final id="f"/>
</statechart>
```

How to read the example.

- Here the <spawn> element creates a toplevel actor on the node given by the option `node('https://n9.org')`. The five spawn types and their operands and option sets are specified in Appendix C.8.
- The text content of <spawn> is Prolog source transferred to the child as a `src_text/1` option. In this example it defines `p/1` privately in the spawned toplevel.
- After the child has been created, the SXML interpreter raises the internal event `spawned(Pid)`. The pid comes from the underlying spawn operation; the event is not a message sent by the child. The transition in state `ask` reacts by calling `toplevel_call/3`, which initiates query evaluation.
- Each answer arrives as an event of the form `success(Pid,Data,More)`. The option `template(X)` makes the successive slices `[a]`, `[b]`, and `[c]`. While `More` is `true`, the targetless transition requests the next slice with `toplevel_next/1` without exiting and re-entering `collect`. When `More` is `false`, the last answer has been delivered and the machine transitions to the final state.

A <spawn> also ties the child's lifetime to the invoking state: when the state that contains the invocation is exited, the invoked actor is terminated as part of leaving that state. This is the lifetime-containment discipline of Chapter 4 (Section 4.1.8) surfaced as a control construct: the invoked actor is lifetime-dependent on the invoking state in the same way that a client's spawned actors are lifetime-dependent on their session. This makes it easy to express resource ownership ("this control state owns this helper process") without relying on ad hoc cleanup code.

Executable content should not block while the statechart is stabilising. Interaction with other actors should be initiated asynchronously and completed by handling the resulting messages as events, as this example does.

The example crosses the boundary between SXML's own control events and ordinary actor traffic. The interpreter places `spawned(Pid)` on its internal event queue, whereas the PTCP `success/3` terms arrive through the actor mailbox. Once selected for processing, however, both are simply Prolog terms matched by unification. Query evaluation therefore remains asynchronous, and neither PTCP nor remote actor addressing requires a statechart-specific variant.

6.6.1 What does a confirmation confirm?

A service request illustrates why control state and application data must work together. Suppose Alice confirms a request and, while it awaits review, corrects its cost. Her earlier confirmation is still an event that occurred, but it no longer authorises the current request. The meaning of confirmation depends on the particular proposal that was presented, not merely on reaching a state called `confirmed`.

A chart can make this distinction explicit. While awaiting confirmation, it accepts an event only if the requester, revision, and exact field snapshot match the current proposal. A correction increments the revision, clears the old approval, and starts a new confirmation cycle. An old confirmation cannot advance that cycle even if it arrives while the chart is again in the same waiting state. State membership alone is therefore insufficient: the guard must also establish which request the event concerns. A trusted interface must supply the requester's identity; the event's name cannot establish it.

The statechart determines when confirmation can have an effect; the Prolog guard determines what is being confirmed and by whom. Section 8.4.5 develops this distinction into a complete service-request agent.

6.7 The execution model in summary

The examples above convey how the statechart behaviour works in practice. Five points deserve explicit statement here; the element-level contract on which documents may rely is given in Appendix C.

1. **The execution model is based on SCXML's.** SXML follows the transition-selection and run-to-completion model stated in Section 3.13 and formalised in Appendix D of the SCXML 1.0 Recommendation (Barnett et al., 2015), instantiated with Prolog terms and goals. One selected external event may initiate a *macrostep* consisting of one or more *microsteps*. States are exited from the inside out, transition content is executed in document order, and states are entered from the outside in. The element-by-element differences are tabulated in Appendix C.11.
2. **Events are terms and matching is unification.** At the actor level, the behaviour consumes messages in arrival order with an unconditional receive; selectivity lives in the transition rules rather than in the mailbox. A transition's `on` pattern matches an event by Prolog unification, and bindings produced by the match are in scope in the transition's `if` guard and executable content. An event that enables no transition is discarded unless it matches a `<defer>` declaration in the active configuration.
3. **Transition selection respects hierarchy and orthogonality.** For each active atomic state, candidate transitions are considered from that state outwards through its ancestors. Within one source state, document order determines which

enabled transition is selected first. If two selected transitions have overlapping exit sets, the transition whose source is the descendant pre-empts the transition from its ancestor; non-conflicting transitions, such as transitions in independent parallel regions, execute together in one microstep.

4. **Internal work has a fixed priority.** After the current event has been handled, the interpreter first tests for an eventless transition. If none is enabled, it takes the next internal event, after which eventless transitions again have priority. When both sources are quiescent and the configuration has changed, postponed events are re-offered. Pending invocations are performed only after this work has quiesced; the `spawned(Pid)` events they raise enter the internal queue in turn. Only when no such work remains does the actor consume the next external message.
5. **Executable content is committed.** The Prolog content of an <onentry>, <onexit>, or <go> element is executed once; alternatives are not exposed through backtracking. If the goal fails, the interpreter places the internal event `error(failure(Goal))` on the queue; if it throws an exception `E`, the internal event is `error(E)`, which transitions may handle. This is a policy for handling an ordinary Prolog exception within the statechart. Failure is thus an event inside the statechart rather than a control signal propagating out of it, a distinction taken up in Section 6.10.2.

The common forms of the <go> element, and their element-level contract, are given in Appendix C.5 (Table C.2).

6.8 A flat subset of SXML

Not every reactive controller needs Harel's structuring constructs. A protocol handler, a connection state machine, or a simple turn-taking loop is often a handful of states with no nesting and no orthogonal regions, and for such machines the full statechart vocabulary is more machinery than is needed. Rather than introduce a second behaviour for this case, we identify a flat subset of SXML: documents that omit the constructs a flat machine does not use.

We call this subset *Flat SXML*. It omits <parallel> (orthogonal regions), <history> pseudostates, and compound <state> elements (nested states), leaving flat states, the full range of <go> transition forms (Table C.2), state-scoped <defer> declarations, executable content, and the data model. Every document in the subset is also a valid full-SXML document; it simply uses none of the omitted constructs.

One thing the flat subset retains is the run-to-completion macrostep. RTC is not a structuring construct but part of the execution model: a flat machine still drains its internal event queue – including events raised by `raise/1` – before consuming the next external message. What the restriction removes is the *broadcast* aspect of RTC, since with no parallel regions there are no simultaneously active states for an event to be visible across. Eventless and guarded transitions, condition-action rules

over the data model, internally raised events, and explicit event deferral all remain available; only hierarchy, history, and orthogonality are given up.

The benefit of identifying a subset rather than defining a separate behaviour is economy. There is one language, one interpreter, and one document syntax. Flat SXML is a descriptive name for documents written within the restricted vocabulary; the current interpreter neither requires a separate declaration nor performs a distinct Flat SXML conformance check. A machine that outgrows the subset is not rewritten in a different formalism – it simply begins using constructs the full language already provides.

6.9 From XML source to execution

The interpreter does not operate directly on the XML tree. When a statechart actor is started, its SXML source is validated as SXML 0.2 and translated into an actor-local Prolog model of the constructs described in Appendix C; embedded patterns, guards, and executable content become Prolog terms. The runtime interprets this model while maintaining the active configuration and the statechart's runtime queues, history, and timers.

The present implementation uses the same general interpreter for all SXML documents. Flat SXML nevertheless admits a simpler implementation: with only one active leaf state, an engine need not maintain hierarchical configurations, resolve conflicts between parallel transitions, or record history. A flat document could therefore be compiled into a transition table or a specialised actor loop. This is an implementation opportunity rather than a semantic distinction; the current interpreter does not perform this specialisation.

6.10 Discussion

This chapter has presented statecharts as an executable actor behaviour in Web Prolog, using an SCXML-inspired document syntax and Prolog as both data model and scripting language. The discussion section briefly situates this design relative to prior Prolog–SCXML work, clarifies what the behaviour buys compared to a hand-written `receive/1-2` loop, and then positions the result relative to Erlang/OTP's `gen_statem` and the SCXML Recommendation.

6.10.1 Previous work

The idea of using Prolog as a data modelling and scripting language in a statechart formalism is not new. A proof-of-concept implementation of an SCXML system

with Prolog as data model and scripting language was developed by a team at TU Darmstadt (Radomski et al., 2013). Their focus is multimodal conversational systems, and the paper is well worth reading, since it argues – contrary to what one might initially expect – that SCXML and Prolog form a good combination for developers of such systems. However, we take a slightly different and, we believe, simpler and more lightweight approach to integrating statechart notation with Web Prolog. Our aim is to adapt an SCXML-inspired statechart profile as a specialised Web Prolog notation, harmonised with the architecture of the Prolog Web.

Box 6.1 — Erlang comparison: `gen_statem` and the statechart behaviour

Erlang's `gen_statem` and Web Prolog's statechart behaviour both organise reactive processes around state-dependent event handling. In `gen_statem`, callback code specifies state changes, data updates, timeouts, and further events. SXML instead makes the control structure explicit as a hierarchy that can be inspected and visualised. Hierarchical states, history, and parallel regions are built-in control constructs; comparable structure in `gen_statem` must be expressed in callback code.

Both behaviours process events within a single actor. SXML's parallel regions are coordinated within its run-to-completion macrostep under fixed priority rules; they are not independent Erlang processes. Across actors, both settings still use message passing. SXML also makes invocation a control construct: a child belongs to its invoking state and is stopped when that state is exited. In `gen_statem`, application code and OTP lifecycle mechanisms provide that arrangement.

The timing and deferral mechanisms have close counterparts. Erlang state timeouts correspond to SXML's `after` transitions, armed on entry and cancelled on exit. Erlang's `postpone` action retains the current event for retry after a state change, while SXML's `<defer>` declares the event pattern and state in which deferral applies. Flat SXML (Section 6.8) occupies much the same design space as `gen_statem`, with hierarchy, history, and orthogonality available when needed.

6.10.2 Comparison with an ordinary message loop

Why introduce a statechart behaviour instead of writing an ordinary message loop? Small controllers are easy to write directly, but as the control problem grows, Harel's extensions provide structure that is hard to recover from hand-written `receive/1-2` code:

- **Hierarchy and history** represent interruption and resumption directly, instead of through auxiliary flags.

- **Parallel regions** divide one reactive process into parts, and run-to-completion coordinates them deterministically: the same event in the same configuration fires the same transitions in the same order.
- **Invocation** ties spawned processes to control states, so that leaving a state terminates what it started.
- **The chart can be drawn.** The structure that guides execution can be visualised, which makes control logic easier to review and debug.

The statechart behaviour does not, however, depend semantically on `assert/1` and `retract/1`. Its defining machinery is the active configuration, history, internal and deferred event queues, timers, and the rules for selecting and executing transitions. Dynamic predicates are one Web Prolog representation of the optional application data that accompanies that control state. Because the database is private and reactions are sequential, deterministic database updates do not compromise actor isolation or run-to-completion determinism. This is an operational claim, not a claim of logical purity: a private dynamic database can hide part of the actor's state from the chart, and a failed action need not roll back updates already performed. The atomicity of a macrostep therefore means that another external event cannot interleave with it; it does not mean that the macrostep is a database transaction.

Beyond these structural benefits, the statechart behaviour also changes the *event-handling posture*. In a hand-written `receive/1-2` loop, selection is driven by the current mailbox contents and by pattern order, while non-matching messages typically remain in the mailbox (effectively deferred). The hand-written state machine in Chapter 2 illustrated why this must be a deliberate choice: one state deferred unknown events, while another discarded them. SXML makes the choice explicit: an unmatched selected event is discarded by default, while `<defer>` identifies the event patterns that should be reconsidered after a configuration change.

In SXML, event relevance is determined by the current configuration (the active states and regions): one selected event is matched against enabled transitions across the whole configuration, and a defer declaration in any active region may shelter it if none is enabled. Conversely, statecharts provide run-to-completion reactions: one external event may trigger a cascade of internal work (condition-only transitions, eventless transitions, internally raised events, and re-offered events) before the next external message is consumed. A reaction that never exhausts this work is livelocked and consumes no further external messages. With orthogonal regions, the selected event is visible across the entire active configuration, enabling deterministic broadcast-style coordination within one actor – something that actor-level messaging can emulate only by explicit fan-out and coordination protocols.

Another operational difference concerns what failure means. `receive/1-2` commits to one selected message and one receive clause, but the fired body may backtrack through its own choice points and, if it ultimately fails, propagate that failure into the surrounding computation. Failure remains an ordinary Prolog control signal. The statechart behaviour draws the line differently: it commits to one external event and executes a deterministic run-to-completion macrostep. Within that macrostep nothing backtracks into the machine; a failing action becomes the internal event

`error(failure(Goal))`. Failure-as-control versus failure-as-event is the main operational difference between the two behaviours.

6.10.3 Could PTCP itself be an SXML statechart?

The PTCP statechart of Chapter 3 invites a reflexive question: could the toplevel behaviour itself simply be supplied as an SXML document? Its outer control structure is an excellent statechart. In particular, attaching abort and halt to an enclosing state expresses exactly why those operations apply from every inner state.

The complete behaviour nevertheless requires capabilities below ordinary SXML. During state **s2**, the statechart interpreter is busy executing one run-to-completion macrostep and cannot itself receive an event that interrupts that computation; PTCP abort therefore requires the privileged signal mechanism of Section 3.3.3. During state **s3**, the toplevel retains a live nondeterministic Prolog computation, including its remaining choice points. Such a suspended search is not an ordinary Prolog term that can be stored in SXML's data model and later resumed by an action.

SXML can therefore describe PTCP's outer protocol, but cannot by itself supply the interruption and suspended-computation substrate that makes the protocol executable. This explains why the toplevel remains a built-in behaviour while SXML is an application-level notation: resumable Prolog computation is a distinctive runtime contract, not merely a statechart encoding technique.

6.10.4 Relationship to SCXML

SXML is intentionally SCXML-inspired, but it is not a full SCXML surface language. Its primary goal is to present statecharts as an executable actor behaviour in Web Prolog, with Prolog as both data model and executable content. For that reason we aim to implement the core of the SCXML 1.0 processing model, while adopting a smaller and more uniform set of constructs that harmonise with actor-style messaging and with Prolog scripting.

Several SCXML constructs are omitted because they are platform-specific, oriented toward browser integration rather than actor systems, or redundant once executable content is restricted to Web Prolog goals. SXML also renames a small number of constructs for readability in Prolog-centric examples and adds two of its own. Appendix C.11 gives the correspondence element by element, lists the SCXML constructs that SXML does not accept, and records the structural restrictions that follow from these choices.

Three further differences are semantic rather than notational. First, SCXML matches dotted event names by prefix against event descriptors, whereas SXML matches Prolog terms by unification and carries the resulting bindings into guards and actions. Second, SXML has no SCXML `_event` record: the structure and data of

an event are represented directly by the term and the bindings of the `on` pattern. Third, SCXML's `<donedata>` and automatic `done.invoke` protocol are omitted. SXML uses explicit actor messages, internal `done(Id)` events, and ordinary monitoring instead. What remains central is SCXML's transition-selection and run-to-completion model, instantiated by the rules of Section 6.7.

6.10.5 Toward an actor-oriented successor to SCXML

Seen as a language for programming reactive actors, SCXML's core ideas are compelling, but the Web Prolog setting suggests a slightly different "centre of gravity": a declarative data model, a single uniform scripting language and an explicit integration with actor messaging. The statechart behaviour in this chapter can therefore be read as a concrete sketch of what an actor-oriented successor might prioritise: the standard statechart execution model, combined with Prolog-level reasoning and with explicit, programmable interaction with the surrounding actor system. The idea is not specific to Prolog. The state-machine core is independent of the scripting language, so a successor could give the scripting layer more responsibility whether that language is ECMAScript (the standardised form of JavaScript) or Prolog. We develop this argument, and its implications for the SCXML lineage, in Chapter 11.

6.11 Summary

Statechart actors are ordinary actors whose control is a hierarchical state machine written in SXML, with Web Prolog for data and actions. They can be spawned, supervised and sent messages like any other actor.

SXML keeps SCXML's execution model but matches events by unification. It integrates timing, deferral and invocation with the actor model of earlier chapters.

Chapter 7
The Prolog Web ≈ the Semantic Web?

Since the Prolog Web adds logic programming and automated reasoning to the conventional Web, it is natural to ask whether we are approaching something like the *Semantic Web* – the machine-readable Web vision articulated by Berners-Lee et al. (2001). The gist of their proposal is captured by the following three statements:

> The Semantic Web is an extension of the current web in which information is given well-defined meaning, better enabling computers and people to work in cooperation.
>
> For the semantic web to function, computers must have access to structured collections of information and sets of inference rules that they can use to conduct automated reasoning.
>
> Adding logic to the Web – the means to use rules to make inferences, choose courses of action and answer questions – is the task before the Semantic Web community at the moment.

At first glance, words and phrases such as *logic*, *inference rules*, and *answer questions* could be read as an informal description of what Prolog already does. Likewise, the characterisation as an extension of the current Web fits the Prolog Web rather well. This invites a tempting but slightly misleading conclusion: perhaps the Prolog Web is simply a lightweight Semantic Web?

The resemblance is real, but it is a resemblance of ingredients rather than of engineering goals. The Semantic Web focuses on semantic interoperability by standardising shared meaning – data models, identifiers, vocabularies, and ontology semantics that many independent parties can agree on. The Prolog Web focuses on execution by standardising an executable substrate – a uniform rule language plus a node and process architecture in which predicates can be deployed as running services and agents.

The thesis of this chapter is therefore cooperative rather than competitive: the two are best viewed as complementary. In practice, Web Prolog can consume and produce RDF-style data, reuse established vocabularies and identifiers, and apply Prolog's execution model to "put the data to work" as queries, rules, and long-lived agent behaviour.

This comparison builds on a substantial Prolog tradition. Beginning with scalable RDF infrastructure, Jan Wielemaker and collaborators developed an integrated line

of Semantic Web tools in and around SWI-Prolog: query translation and optimisation, the Triple20 ontology-manipulation tool, Web libraries, and an application architecture in which Prolog serves as the foundation for Semantic Web systems (Wielemaker et al., 2003; Wielemaker, 2005; Wielemaker et al., 2005, 2007, 2008). ClioPatria and the Semantic Web integration of SWISH carried this work into full platforms for applications and interactive development (Wielemaker et al., 2016; Beek and Wielemaker, 2016). The fit between Prolog and RDF is therefore well established. Our question is what a distributed node and actor architecture adds to it.

The integration patterns described below are architectural patterns, not additional standard Web Prolog APIs. The Web Prolog profiles do not prescribe an RDF store, a SPARQL client, or an OWL reasoner, and the demonstrator does not provide standard predicates for them. A node owner may supply such facilities through trusted host libraries and expose them as ordinary predicates or services.

Although the Semantic Web serves as our principal reference point, the underlying distinction between interoperable representation and executable computation applies more broadly to knowledge graphs, property graphs, and typed data APIs.

7.1 The Semantic Web tower

The Semantic Web is often presented as a layered architecture, where each layer builds on the layers below it. The lower layers are shared with the conventional Web, while the upper layers introduce machine-processable representations of data, query mechanisms, and ontology-level semantics. A commonly used illustration is the so-called *Semantic Web tower*, one variant of which is shown in Figure 7.1.

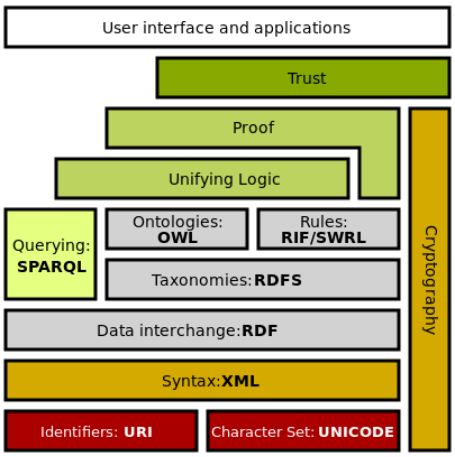


Fig. 7.1 One historical variant of the Semantic Web tower.

The diagram is a historical snapshot rather than a current inventory of standards. In particular, SWRL remained a 2004 W3C Member Submission,[1] whereas the RIF family later produced W3C Recommendations, including the Core, Basic Logic, and Production Rules dialects.[2]

The tower diagram is necessarily impressionistic, but it provides a useful scaffold for our comparison. At the bottom, URIs play the familiar role of locating resources (as URLs), but on the Semantic Web they also serve as global names for entities, relations and concepts (as IRIs). The Prolog Web uses URIs operationally to locate nodes and source; globally qualified pids then combine a process identifier with the URI of its node.

Unicode can likewise be viewed as a naming layer, ensuring that text and identifiers can be represented consistently across languages. Not all Prolog systems fully support Unicode, but a web-facing Prolog system effectively has to.

Many variants of the tower place XML at the syntactic level. In this chapter we treat XML mainly as a data interchange format among several. In the Prolog Web setting, Prolog terms provide a natural internal representation, and JSON is often the most convenient interchange format for client-facing APIs, while RDF serialisations such as RDF/XML and Turtle remain important for interoperability with existing Semantic Web tooling.

7.2 RDF as rdf/3

Right above the XML level of the historical Semantic Web tower we find the levels devoted to semantics proper, where RDF forms the basis. RDF represents information as subject–predicate–object triples. More complex structures are built by connecting resources and blank nodes through multiple triples. In RDF 1.1, reification has a narrower purpose: it allows an RDF graph to describe a particular triple token, but does not itself assert the triple being described.[3]

RDF 1.1 remains the latest W3C Recommendation. RDF 1.2 was a Candidate Recommendation Snapshot when this book was written; among other additions, it introduces triple terms and reifiers for making statements about statements, as well as directional language-tagged strings.[4] SPARQL 1.2 was still a Working Draft.[5] The examples in this chapter use the stable RDF 1.1 data model and SWI-Prolog's rdf11 library.

From a Prolog point of view, the triple pattern maps naturally to a three-argument relation. The SWI-Prolog Semantic Web libraries and ClioPatria therefore use rdf/3 as their elementary query predicate (Wielemaker et al., 2016). Current SWI-Prolog

[1] https://www.w3.org/Submission/SWRL/

[2] https://www.w3.org/TR/rif-overview/

[3] https://www.w3.org/TR/rdf11-mt/#reification

[4] https://www.w3.org/TR/rdf12-concepts/

[5] https://www.w3.org/TR/sparql12-query/

applications normally use `library(semweb/rdf11)`, whose literal notation follows RDF 1.1 more closely than the older `literal(type(...))` representation. The following self-contained fragment creates a small example store:[6]

```
:- use_module(library(semweb/rdf11)).

:- rdf_register_prefix(ex,     'http://example.org/').
:- rdf_register_prefix(foaf,   'http://xmlns.com/foaf/0.1/').
:- rdf_register_prefix(schema, 'http://schema.org/').

load_example :-
    rdf_assert(ex:bob, rdf:type, foaf:'Person'),
    rdf_assert(ex:bob, foaf:knows, ex:alice),
    rdf_assert(ex:bob, foaf:knows, ex:john),
    rdf_assert(ex:alice, schema:birthDate,
               date(1990,7,4)^^xsd:date),
    rdf_assert(ex:john, schema:birthDate,
               date(1992,9,6)^^xsd:date).
```

Once these triples have been loaded, ordinary Prolog clauses can define a task-specific interface to the store. For example, the following relation finds the birth dates of people somebody knows:

```
known_with_birthdate(Me, Person, Date) :-
    rdf(Me, foaf:knows, Person),
    rdf(Person, schema:birthDate, Date).
```

Prefixes registered with `rdf_register_prefix/2` allow IRIs to be written in the compact form `Prefix:LocalName`; the calls to `rdf/3` still operate on the corresponding full IRIs. This reflects Berners-Lee's broader account of webizing a database, in which URIs name concepts such as tables and columns and namespace prefixes keep those names manageable. A portable namespace mechanism analogous to `rdf_register_prefix/2` would also be valuable in Web Prolog. Prefixes would provide readable notation, while the full URI would remain the public identifier used across nodes.

This would not make RDF the native data model of Web Prolog. The layers must be kept distinct: `rdf/3` and the prefix declarations above are supplied by the SWI-Prolog host library, not by the standard Web Prolog predicate set. On a public node they would normally be installed by the owner and exposed through selected predicates rather than loaded by an untrusted client.

[6] The data is adapted from the RDF 1.1 Primer: `https://www.w3.org/TR/rdf11-primer/`.

7.3 SPARQL-style querying in Prolog

Instead of using SPARQL, querying Semantic Web data represented as above can be done in Prolog. The close correspondence between graph patterns and Prolog goals has also supported query-language translation and optimisation (Wielemaker, 2005). The interactive session below illustrates the use of `known_with_birthdate/3` to query the triple store:

```
?- known_with_birthdate(Me, Person, Date).
Me = 'http://example.org/bob',
Person = 'http://example.org/alice',
Date = date(1990, 7, 4)^^
       'http://www.w3.org/2001/XMLSchema#date' ;
Me = 'http://example.org/bob',
Person = 'http://example.org/john',
Date = date(1992, 9, 6)^^
       'http://www.w3.org/2001/XMLSchema#date'.
?-
```

The key point is modularity. Wielemaker et al. (2016) argue that the Prolog version using `known_with_birthdate/3` has several advantages over the SPARQL version. First, because the Prolog version has an explicit name, it is easy to reuse it in other queries. This allows developers to incrementally build more sophisticated functionality on top of existing, proven and tested building blocks. This can greatly simplify maintenance of complex queries.

Secondly, application logic written in Prolog is not confined to the filters and functions standardised by SPARQL. Any available predicate can serve as a guard, transformation, or derived relation at any point in the clause, and the same definition can combine RDF triples with other relations – for example, views backed by an RDBMS or calls into domain-specific code.

The query above is simple, but more complex relational queries are possible too. An ISOBASE node guarantees the usual Prolog control constructs, including conjunction, disjunction, and Negation As Failure, together with the ISO all-solutions predicates `bagof/3`, `setof/3`, and `findall/3`. SWI-based nodes may additionally expose `library(aggregate)`,[7] which provides operators such as `count`, `sum`, `min`, and `max`. These library predicates are useful implementation facilities, but are not guaranteed by the Web Prolog ISOBASE profile.

The same qualification applies to the SWI-Prolog library `solution_sequences`,[8] which was developed to support direct formulations of many database-style queries and provides `distinct/1-2`, `limit/2`, `order_by/2`, and related predicates. On a node whose owner has made these two libraries available, consider the following relation `r/2`:

---

[7] `https://www.swi-prolog.org/pldoc/man?section=aggregate`

[8] `https://www.swi-prolog.org/pldoc/man?section=solutionsequences`

```
:- use_module(library(aggregate)).
:- use_module(library(solution_sequences)).

r(a, foo).  r(a, bar).  r(a, bar).  r(c, bar).  r(c, bar).
r(b, bar).  r(a, foo).  r(d, baz).  r(a, foo).  r(c, baz).
```

Here is an example of how a SPARQL-like query can be formulated using such predicates, and what the solutions would look like:

```
?- limit(10, order_by([desc(C)], aggregate(count,X,r(X,Y),C))).
C = 3, Y = bar ;
C = 2, Y = baz ;
C = 1, Y = foo.
?-
```

The call `aggregate(count, X, r(X,Y), C)` groups on the free variable `Y`. Its discriminator `X` removes duplicate `X` values within each group, so the result corresponds to `COUNT(DISTINCT X)` rather than a count of all matching clauses. The call to `order_by/2` sorts the resulting `Y-C` pairs by descending `C`, and `limit/2` truncates the sequence – conceptually similar to a SPARQL query with `GROUP BY`, `ORDER BY`, `COUNT(DISTINCT ...)`, and `LIMIT`.

The resemblance should not be mistaken for semantic identity. SPARQL has defined multiset, unbound-variable, error, and negation semantics that do not coincide in every case with Prolog's solution sequences and Negation As Failure. The example illustrates how familiar query operations can be composed in Prolog; it is not a general semantics-preserving translation of arbitrary SPARQL.

If the owner has exposed `r/2`, `aggregate/4`, and `order_by/2` on a remote SWI-based node, the same goal can be submitted through `rpc/3`:

```
?- Goal = order_by([desc(C)], aggregate(count, X, r(X,Y), C)),
   rpc('http://n7.org', Goal, [limit(10)]).
```

The `limit(10)` option bounds the solutions returned through the RPC interface; it does not prevent `order_by/2` from collecting and sorting its complete input on the remote node. It is therefore a transport-level bound, not necessarily a computational optimisation.

An ISOBASE node is sufficient for this stateless call. ISOBASE also permits source to accompany a call through options such as `src_text/1` and `src_uri/1`. ISOTOPE becomes necessary when definitions or data must persist across a sequence of requests in a toplevel session. None of these profile guarantees includes the SWI-specific RDF or solution-sequence libraries; those remain owner-provided facilities.

The bottom line is that Prolog offers a particularly direct and idiomatic interface to RDF: triples map to relations, and rule layers can be expressed in the same language as the queries that consume them.

Wielemaker et al. (2016) summarise the position we adopt:

> We do not consider SPARQL adequate for creating rich semantic web applications. SPARQL often needs additional application logic that is located near the data to provide a task-specific API that drives the user interface. Locating this logic near the data is required to avoid protocol and latency overhead. RDF-based application logic is a perfect match for Prolog and the RDF data is much easier queried through the Prolog RDF libraries than through SPARQL.

This is the role the Prolog Web takes: Prolog predicates become the task-specific API layer that lives near the data, while remaining compositional, testable, and reusable as ordinary code.

7.4 OWL as an external reasoning service

Moving up the Semantic Web tower, we reach OWL, a family of ontology languages based on description logics. OWL's purpose is not merely to provide more rules, but to support a particular kind of interoperability: shared vocabularies with explicit semantic commitments – class hierarchies, property characteristics, domain and range axioms, cardinality restrictions, and so on – that can be processed by standard OWL *reasoners*. This matters whenever independent producers and consumers of data need to agree not only on identifiers, but on the intended interpretation of those identifiers.

The Semantic Web community makes a clear distinction between knowledge representation languages (such as RDF, RDFS) and query languages (such as SPARQL). As we have seen, Prolog (including Web Prolog) can, to some extent, fill both roles – but with different semantic commitments. Web Prolog is most naturally used as an *executable rule layer*: named predicates encode task-specific queries and derived relations, composable and testable as ordinary code. OWL, by contrast, targets standardisable ontology semantics under open-world assumptions and decidable reasoning fragments, enabling interoperability through shared class and property commitments rather than shared procedural conventions. Prolog is good at turning relational data into services; OWL is good at stating shared meaning that independent tools can agree on. Web Prolog does not replace OWL. Web Prolog is grounded in Horn clause logic and, in typical use, relies on closed-world idioms including Negation As Failure. The OWL 2 DL language and OWL profiles provide standard, decidable fragments under open-world assumptions, supporting entailment and consistency reasoning that plain Horn clauses cannot reproduce directly without changing the semantics or moving to richer logic programming extensions.

OWL also makes no unique-name assumption: two different IRIs may denote the same individual unless their distinctness is stated explicitly.[9] This differs from ordinary Prolog term semantics, in which distinct ground terms do not unify. The contrast therefore concerns both what may be inferred from missing information and how identity is treated.

---

[9] `https://www.w3.org/TR/owl2-primer/`

Ontology reasoning should also be distinguished from data validation. Under open-world semantics, a missing value does not normally constitute a violation. Applications that need to validate an RDF graph against structural conditions can use a shapes language such as SHACL,[10] again treating the corresponding validator as a specialised external service.

This does not make the Prolog Web irrelevant to ontology-heavy applications – it merely suggests a division of labour. When OWL is the right tool for defining a shared ontology, Web Prolog can still play at least three valuable roles: *consuming* OWL-derived results (such as classifications from an external reasoner) and combining them with application-specific rules; *mediating* between heterogeneous sources by translating Semantic Web representations into executable relations exposed through the Prolog Web APIs; and providing an *agent substrate* in which long-lived actors use ontology-backed knowledge as one input among many while the agent logic itself remains straightforward Prolog.

Here the Semantic Web contributes standardised ontology semantics and a mature ecosystem of reasoners and tools. The Prolog Web complements these facilities by turning ontology-backed data and reasoning results into executable relations, running services, and web-native agents.

7.5 Positioning the Prolog Web alongside the Semantic Web

The preceding sections have examined individual layers of the Semantic Web tower and shown how each relates to the Prolog Web. It is now worth stepping back to compare the two ecosystems as wholes. Figure 7.2 places the Prolog Trinity alongside what, by analogy, might be called a "Semantic Trinity": a comparable ecosystem assembled from Semantic Web technologies – RDF and SPARQL for data and querying, and a general-purpose language such as Java for application logic (and possibly OWL for ontology-level semantics, and an external reasoner for inference).

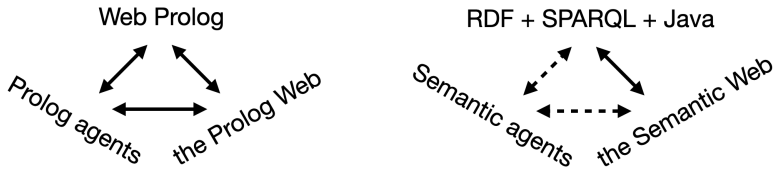


Fig. 7.2 Comparing the Prolog Trinity with a "Semantic Trinity." Solid arrows indicate tight integration; dashed arrows indicate looser coupling.

The comparison is schematic. Integrated Semantic Web platforms certainly exist; the dashed arrows describe boundaries between independently standardised layers,

[10] `https://www.w3.org/TR/shacl/`

not the quality of every implementation. Two differences nevertheless stand out. The first is integration. In the Prolog Trinity the arrows are solid, suggesting that the same language is used for knowledge representation, querying, rule-based computation, and agent scripting, and the execution substrate (nodes, actors, APIs) is part of the proposal rather than an afterthought. In the Semantic Trinity, the data layer (RDF), query layer (SPARQL), and application layer (Java or another host language) have distinct interfaces, and an application framework connects them.

The second difference is architectural. Semantic Web standards do not define a general process infrastructure with actor lifecycles, stateful services, and message-based coordination. These concerns are left to the chosen application framework. The Prolog Web, by contrast, includes an architecture as part of the design. Nodes serve answers over standard web protocols, calls compose local and remote resources, source can accompany calls and spawned processes, and actors coordinate over time.

These observations suggest a pragmatic positioning. Semantic Web standards provide interoperable representations and shared semantic commitments, but rich applications still require a host language to implement task-specific APIs, workflows, and integration with heterogeneous data sources. There is often an impedance mismatch between graph-shaped, relational data and imperative application code (Wielemaker et al., 2016). Prolog has an unusually good fit: RDF triples map directly to relations, and rule layers can be expressed in the same language as the queries that consume them. Application logic that would otherwise live as opaque client-side code or as ad hoc service glue can instead be expressed as named, reusable predicates close to the data.

From the Prolog Web perspective, then, Web Prolog can act as a programming and agent layer around Semantic Web assets: consuming RDF and SPARQL endpoints when appropriate, delegating ontology-level reasoning to OWL reasoners when needed, and exposing the resulting functionality as web-native services hosted on Prolog Web nodes. The next section makes this complementarity concrete by outlining three integration patterns that recur in practice.

7.6 Cooperation patterns

The complementary roles identified above lead to three recurring integration patterns, referred to as A, B, and C below.

7.6.1 A: Web Prolog as a SPARQL client and post-processor

In many applications the most pragmatic role for Web Prolog is to act as a consumer of Semantic Web data offered by existing endpoints. A Web Prolog program issues a SPARQL query to a remote endpoint, parses the result, and then applies ordinary Prolog predicates for filtering, aggregation, explanation, or task-specific inference.

The *logic that makes the results useful* is written as reusable predicates rather than being forced into a single monolithic SPARQL query.

Conceptually, the pattern looks like this:

1. Query a SPARQL endpoint and obtain a sequence or table of bindings.
2. Convert the bindings to a relational representation.
3. Run Prolog rules over that representation and expose the result as a Prolog Web service.

The first step is performed by an owner-installed library predicate or service that implements HTTP and one of the SPARQL result formats. The remaining steps are plain relational programming. If the application is interactive, the derived predicate can be offered through the Prolog Web APIs, allowing clients to consume its Prolog solutions incrementally. This does not imply that the underlying SPARQL endpoint is itself consumed lazily: an adapter may fetch or buffer its complete result before the Prolog relation begins producing answers.

A useful boundary case is to place a RELATION adapter directly in front of a SPARQL endpoint. The adapter advertises a fixed relation such as `rdf/3`, translates calls to that relation into SPARQL triple-pattern queries, and returns the resulting bindings through the stateless HTTP API. A client can then write:

```
?- rpc('http://sw-node.org', rdf(S, P, O)).
```

Here `rpc/2` is a client-side interface to the stateless call; it does not imply that the target accepts arbitrary ISOBASE goals. As stressed in Chapter 3, a RELATION node need not contain a Prolog engine at all. The adapter is conformant because it exposes only its advertised relation and behaves externally as a relational endpoint. In this way an existing SPARQL service can become a Prolog Web citizen without changing its storage or query engine.

7.6.2 B: RDF as `rdf/3`, Prolog rules as the API

A second pattern is to treat RDF as a storage and interchange format, but to treat Prolog as the *programming interface* to that data. In this pattern, RDF triples are ingested into a node-local indexed store queried through `rdf/3`, while the public surface consists of named Prolog predicates that encode the relevant domain.

Exposing named predicates rather than a SPARQL endpoint has a real advantage. Named predicates become stable, reusable building blocks, and they can be composed with arbitrary auxiliary relations, foreign data sources, and task-specific computations. The service remains readable, testable, and evolvable: if a predicate is refactored, callers can remain unchanged as long as the predicate interface is preserved.

In a Prolog Web setting, a fixed collection of such predicates can be advertised through a RELATION node. An ISOBASE or stronger node may additionally accept general calls through `rpc/2-3`, while an ISOTOPE or ACTOR node can support

longer conversations. The result is a service that is recognisably "Semantic Web" in the sense that it is grounded in RDF identifiers and vocabularies, but "Prolog Web" in the sense that its interface consists of executable relations.

A minimal bridge example

To make the idea more concrete, suppose a node maintains a local RDF store and provides the above derived predicate `known_with_birthdate/3`. A client can then query this service lazily using `rpc/3`, taking advantage of the Prolog Web's support for bounded solution slices:

```
?- Goal = known_with_birthdate(Me, Person, Date),
   rpc('http://n7.org', Goal, [limit(10)]).
```

The example shows the division of labour. The use of established RDF identifiers and vocabularies makes the underlying data interoperable with Semantic Web tooling, while the exposed predicate provides an application-friendly interface that can be maintained and composed like ordinary Prolog code. If more logic is needed, it can be layered on top of `known_with_birthdate/3` rather than being duplicated across multiple SPARQL queries.

7.6.3 C: OWL reasoning as a service, Web Prolog for orchestration

A third pattern is to treat OWL reasoning as an external service rather than as something to be reimplemented inside Web Prolog. This is often the most realistic approach when an application depends on ontology-level commitments such as class subsumption and property characteristics that are supported by mature OWL reasoners.

In this pattern, OWL plays the role of *semantic backbone*, while Web Prolog plays the role of *executable glue*:

1. An OWL reasoner (local or remote) is used to compute classifications, entailments, or consistency checks.
2. The computed results are imported into Web Prolog as ordinary relations (for example, `class/2`, `subclass/2`, `entailed/1`, or additional `rdf/3` facts).
3. Prolog rules and, if needed, long-lived actors use these relations as inputs to application-specific decisions, workflows, or interactive services.

This arrangement respects the strengths of both stacks. OWL reasoners provide well-understood semantics and tool support for ontology-level inference, while Web Prolog provides a convenient substrate for integrating that inference with other sources of information, additional rule layers, network calls, and agent-style control logic. Where RDF data must also be validated, a SHACL processor can be integrated

in the same way. The resulting system is typically easier to engineer than an attempt to force all application logic into SPARQL and OWL alone.

7.6.4 Summary

Pattern A consumes and post-processes existing endpoints. Pattern B places a named Prolog rule layer over an RDF store. Pattern C delegates specialised ontology reasoning or validation while Web Prolog orchestrates the application. In every case, Semantic Web assets retain their standard meanings and Web Prolog supplies the executable service and process layer.

7.7 Proof and trust

The upper layers of the Semantic Web tower are often labelled *Proof* and *Trust*. The labels point to real concerns, but the standards themselves do not prescribe a single, broadly adopted mechanism for either.

For the Prolog Web it is useful to read "trust" in a pragmatic engineering sense. A simple starting point is transparency: a node can let clients inspect the Web Prolog source code behind a published predicate or service, for example through a browsing facility, and optionally accompany it with version identifiers, digital signatures, or provenance metadata. Source visibility does not establish correctness, and a signature authenticates provenance rather than truth. These mechanisms nevertheless support auditability, reproducibility, and accountability – which is often what developers actually need.

This stance is also operationally natural in a Prolog setting. A node need not publish only answers; it can also publish the predicate definitions and provenance information required to reconstruct how those answers were obtained. Chapter 8 develops the idea further by showing how proof trees can be collected and inspected as a basis for explainable reasoning in Prolog agents.

7.8 User interfaces and applications

The cooperation patterns above are easiest to exploit in the familiar web application architecture: a JavaScript front end plus a back end that exposes task-specific predicates as services.

For example, a node can publish a UI-oriented relation over its RDF store:

```
display_card(Person, Name, Date) :-
    rdf(ex:bob, foaf:knows, Person),
    rdf(Person, foaf:name, Name),
```

```
rdf(Person, schema:birthDate, Date).
```

A browser requests a bounded solution slice:

```
/call?goal=display_card(Person,Name,Date)&limit=10
```

The node evaluates the rule against RDF data and returns ordinary JSON binding objects; JavaScript renders each object as a card and follows the `more` flag if another slice is required. The end-to-end boundary is now clear: RDF supplies interoperable data, the named predicate supplies task-specific logic, the Prolog Web API supplies lazy delivery, and the browser supplies presentation.

The developer-facing difficulty has been formulated sharply by Verborgh and Vander Sande (2020). As Web development shifted towards the front end (the part of an application that runs in the browser), Semantic Web technologies remained hard for application developers to approach. Meanwhile, apparently simpler technologies such as GraphQL, a query language for web APIs, began rediscovering problems long studied in SPARQL federation, the querying of several data sources at once. Verborgh calls bringing that expressivity within reach of front-end developers a research problem.

The Prolog Web perspective is pragmatic: conventional Web front ends remain free to use HTML, CSS, and JavaScript, while Web Prolog provides a relational and rule-based back end over HTTP or WebSocket. A JavaScript UI can talk to services that in turn query RDF stores, SPARQL endpoints, or OWL reasoners, using Web Prolog as the application-logic layer around Semantic Web assets. ClioPatria and SWISH have already demonstrated browser-facing Prolog environments for Semantic Web programming (Wielemaker et al., 2016; Beek and Wielemaker, 2016).

The demonstrator takes a further step by running Web Prolog in the browser through SWI-WASM. Its JavaScript bridge connects the browser-resident runtime to remote calls and actor messaging; a stable, application-facing JavaScript API over the same primitives would be a natural extension. This is a partial answer to the accessibility gap just described: front-end code can reach expressive, rule-based services through a small call-and-message interface rather than embedding all federation and inference logic in the UI. This lowers the entry cost where most application code is written.

This is also where agents enter the picture. Some agents are *UI agents*: long-lived processes on the user's side that keep track of a conversation or task, run queries in the background, coordinate several services, and present results bit by bit. In a conventional web application, such behaviour is written by hand in JavaScript.

On the Prolog Web, the same interaction model can be carried across the client–server boundary: UI agents and back-end services can be written against the same set of message-passing and call abstractions, with the agent logic expressed as stateful actors that communicate by messages, consume RDF/OWL-backed knowledge when relevant, and expose their functionality through web-native APIs.

7.9 Related work: towers of logic programming

As noted above, many variants of the Semantic Web tower have been proposed, and some align more closely with logic programming than others. The two-tower architecture of Horrocks et al. (2005), reproduced in Figure 7.3, contrasts two plausible instantiations of a layered Web stack.

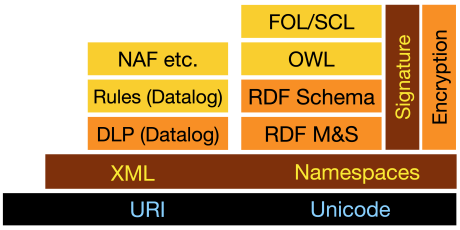


Fig. 7.3 The two-tower architecture, with the tower proposed in (Horrocks et al., 2005) to the left and a simplified version of the "original" tower to the right.

The right-hand tower captures essentially the same idea as Figure 7.1. The left-hand tower, by contrast, is more explicitly grounded in the logic programming paradigm. Here DLP denotes Description Logic Programs, a fragment designed to connect Datalog-style rule reasoning with part of the ontological expressiveness found in OWL. The later OWL 2 RL profile is especially relevant: it is a standardised, rule-oriented subset of OWL 2 designed for rule-based implementations and explicitly inspired by DLP and pD*.[11] Above DLP, one can formulate rules and enrich them with features such as Negation As Failure. The "etc" in the diagram is not mere hand-waving: it points to the broad family of logic programming formalisms surveyed later in Section 10.3.1, each with its own semantic foundations and practical strengths.

The key observation is that this left-hand tower is not a single monolithic stack but a family of possible stacks, each grounded in a different fragment or extension of logic programming. The Prolog Web's profile hierarchy (Chapter 3) provides a common interaction envelope for this diversity. A Datalog engine may expose fixed relations through the RELATION profile or a broader goal language through ISOBASE. A system using well-founded semantics or probabilistic reasoning can likewise remain addressable through standard Web Prolog protocols.

The profile name alone does not advertise the engine's entailment regime, termination properties, treatment of negation, or probabilistic semantics. Those logical commitments require separate service metadata. The profiles instead say how a client may interact with the node – as a relation endpoint, a stateless Prolog evaluator, a persistent toplevel, or an actor system – while the protocol layer provides the common ground.

In this reading, the left-hand tower of Horrocks et al. (2005) maps directly onto the architecture described in earlier chapters. The broader vision – a Logic Programming

[11] `https://www.w3.org/TR/owl2-profiles/`

Web rather than merely a Prolog Web – is taken up again in Chapter 10, where we consider what this architecture offers to the wider LP community.

7.10 Outlook: Prolog Trinity towers

Where does Web Prolog fit? One view places it as middleware between a conventional HTML, CSS, and JavaScript application layer and the Semantic Web stack beneath it, as suggested in Figure 7.4.

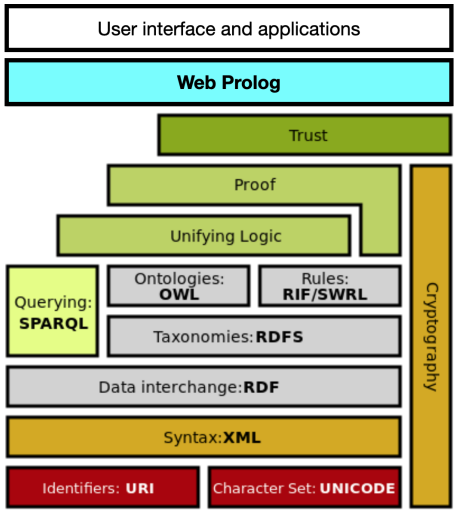


Fig. 7.4 The Semantic Web tower with an explicit Web Prolog middleware layer.

The same kind of arrangement can be built on top of the second tower, as suggested in Figure 7.5.

Or it might be regarded as a *third* tower, independent but interoperable with the other towers. Figure 7.6 suggests this option.

These pictures are complementary views rather than mutually exclusive implementation plans. Pattern B often makes Web Prolog look like middleware over an RDF store, whereas a network of independent Prolog Web nodes is better captured by the third-tower view. The latter is therefore the most faithful architectural picture of the proposal in this book: an independent but interoperable execution stack, able to sit beside Semantic Web stacks and to incorporate their assets when useful.

While the Semantic Web might be described as "layered," the Prolog Web might be described as "integrated" – not only because the same language is used for

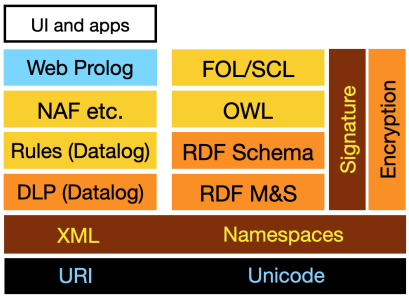


Fig. 7.5 Two-tower architecture (after (Horrocks et al., 2005)): logic-programming-oriented tower alongside the "original" tower.

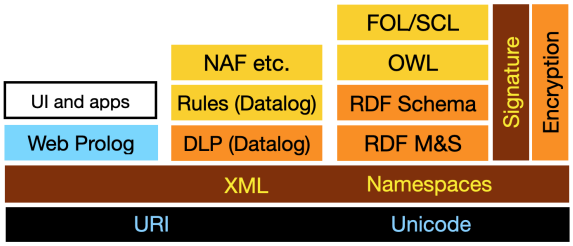


Fig. 7.6 Three-tower view: adding a Prolog Web tower alongside the two Semantic Web towers (after (Horrocks et al., 2005)).

knowledge representation and querying, but because Web Prolog is also a general-purpose programming language with a process architecture.

The case for allowing several interoperable towers was made explicitly by Kifer et al. (2005):

> While a single-stack architecture would hold aesthetic appeal and simplify interoperability, many workers in the field believe that such architecture is *un-realistic* and *unsustainable*. For one thing, it is presumptuous to assume that any technology will preserve its advantages forever and to require that any new development must be compatible with the old. If this were a law in the music industry (for example) then MP3 players would not be replacing compact disks, compact discs would have never displaced tapes, and we might still be using gramophones.

The authors have a point, though the analogy cuts both ways. If Web Prolog programs are the vinyl records of logic programming, with actors and nodes as the web-compatible gramophones on which to "play" them, vinyl has not gone away: it supports techniques such as *scratching* that newer media do not reproduce. Likewise, Prolog combines knowledge representation and querying with general-purpose programming, whereas the usual Semantic Web languages rely on a separate application language for that role. This integration has costs, but it is at the heart of Prolog. Without it, this would no longer be a Prolog Web.

7.11 Summary and discussion

The Prolog Web and the Semantic Web are similar in spirit: both treat the Web as a computational environment rather than merely a document store, and both rely on global naming (URIs) and on logic-based machinery to make data more useful than it would be in a purely syntactic web of documents. Both also aim at decentralisation and federation, in the sense that useful functionality should arise from composing resources offered by many independent parties rather than from a single central database.

At the same time, the two stacks emphasise different things. The Semantic Web places its centre of gravity on interoperable data models and shared semantic commitments, while the Prolog Web places its centre of gravity on executable relational services, process-level structure, and web-native agents. Table 7.1 summarises these differences.

One way to state the complementarity is that unification disciplines structure at the point of call, whereas shared vocabularies and ontologies make semantic commitments explicit. Neither substitutes for the other: structurally compatible terms can still be interpreted differently, and shared meaning does not by itself provide an execution model.

The Prolog Web	**The Semantic Web**
Uses the same language for querying and rule-based computation; queries can be packaged as reusable predicates	Typically separates data representation (RDF/OWL) from querying (SPARQL) and application logic
Provides a process and service architecture (nodes, APIs, long-lived actors) as part of the proposal	Provides standards for interoperable data and ontology semantics, but does not prescribe a single process architecture
Supports agent-style computation naturally through long-lived actors and message passing	Has strong support for shared ontologies and open-world reasoning through standardised languages and reasoners
Shared meaning is typically implicit in executable predicates and local conventions; shared ontologies are optional	Interoperability is centred on shared identifiers, vocabularies, and ontology axioms
Well suited as a programming and orchestration layer around heterogeneous services (including Semantic Web services)	Well suited as a lingua franca for knowledge interchange across independent parties

Table 7.1 Comparing the Prolog Web and the Semantic Web

Although the discussion in this chapter uses the Semantic Web as a concrete point of comparison, the underlying issues are not specific to RDF, OWL, or W3C standards. Property graphs and large-scale knowledge graphs raise many of the same questions about graph-structured data, pattern-based querying, distributed evaluation, and the relation between shared representation and executable computation. Web Prolog can use such representations without making any one of them its native data model.

The aim is not to replace the Semantic Web but to add what it leaves open: an execution architecture. RDF supplies data, OWL shared meaning, reasoners entailments; Web Prolog turns them into relations, services and agents.

Chapter 8
Prolog AI agents on the Agentic Web

Figure 8.1 illustrates a familiar way of viewing the history of artificial intelligence since the late 1950s: as a sequence of booms and busts, or "AI summers" and "AI winters." Prolog and Lisp played prominent roles during the second boom, when expert systems were widely seen as the flagship technology of applied AI. In the current boom, by contrast, the centre of gravity has shifted towards neural methods, especially deep learning and large language models (LLMs).

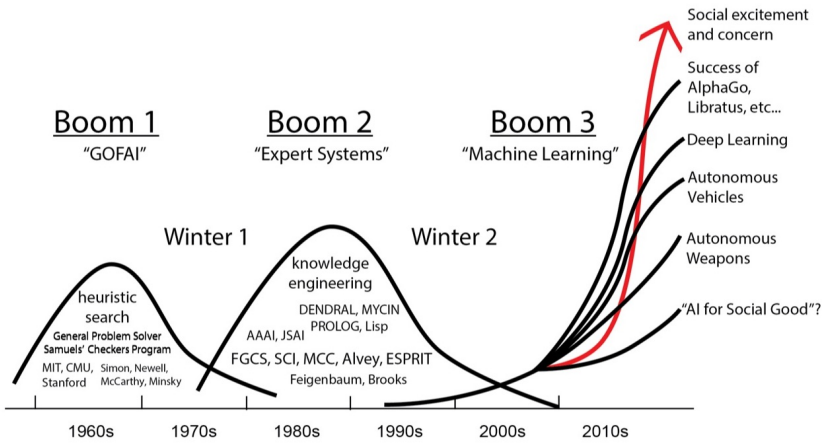


Fig. 8.1 The three booms of AI. Diagram by Colin Garvey, inspired by Yutaka Matsuo (Garvey, 2018).

This historical diagram was published in 2018 and should not be read as a chronology of the subsequent rise of generative AI. Its larger lesson remains useful: claims about AI recur in cycles, and architectural ideas can outlive the boom in which they first became prominent.

This chapter argues that the symbolic tradition remains relevant – not as a rival to contemporary AI, but as a complement to it. The reason is architectural. As the Web becomes populated by software processes that communicate, maintain state, use tools, and act over time, controllable behaviour, inspectable reasoning, and explicit constraint handling again become first-class engineering concerns. This development is often described as an *Agentic Web*. The term is still emerging rather than denoting one settled architecture; current W3C activity similarly treats agent protocols and trustworthy agent interaction as open areas of work.[1] In this chapter it means a Web in which autonomous, communicating processes operate alongside documents, services, and human users as first-class participants.

This development intersects naturally with logic programming. Earlier in the book, especially in Chapter 4, we argued that the Prolog Web already constitutes a multi-agent system in a minimal architectural sense. We now ask how Prolog can be used to structure agent behaviour, govern interaction, and combine reasoning, learning, and tool use. Because the vocabulary and products surrounding LLM-based agents change unusually quickly, specific descriptions in this chapter should be read as current in September 2026; the architectural claims are intended to outlast them.

8.1 Why Prolog agents now?

The classical Web is primarily a publishing and integration substrate: documents, hyperlinks, and request–response endpoints. The Agentic Web shifts the unit of composition towards interacting processes. By the *Prolog Web* in its agentic aspect, we mean the part of that environment in which agents are implemented as Prolog processes, exposed as web-native services, and capable of interacting through message passing.

The word *agent* itself carries three relevant senses. This book's default, established in Chapter 3, is architectural and modest: an agent is defined by a stable interface and behaviour, without any claim about intelligence. Classical AI uses a richer vocabulary of beliefs, desires, intentions, goals, and speech acts (Wooldridge and Jennings, 1995). Contemporary LLM engineering often uses *agent* for a model embedded in a tool-using control loop. The latter two are things that may be built on the first: a BDI agent may use an actor's private database for beliefs and its behaviour for deliberation; an LLM agent may delegate part of that deliberation to a model. Unless qualified as *BDI agent* or *LLM agent*, the word retains the book's architectural meaning.

Historically, the Web has standardised naming, transport, and representation far more successfully than it has standardised process structure. We have mature machinery for addressing resources and exchanging messages, but much less agreement about the lifecycle of long-lived services, the discipline of stateful interaction, the

[1] See the W3C AI Agent Protocol Community Group, `https://www.w3.org/community/agentprotocol/`.

handling of partial failure, or the way autonomous components should compose. Early discussions of web agents within the Semantic Web community made this gap explicit: representation and inference were advancing, but the process infrastructure in which agents would live and cooperate remained largely an application-level concern (Hendler, 2001).

The Prolog Web treats this gap as a design target. It preserves the Web's strengths – global naming, standard protocols, and interoperability – while adding an explicit execution model. Nodes host long-lived actors with mailboxes and supervised lifecycles, and predicates can be exposed as message-driven services. What remains is to decide how to specify, constrain, explain and govern what agents do.

Two current developments make Web Prolog a good candidate for filling this gap. The first is the need for enforceable semantics. Large language models have made fluent natural-language interaction widely available, but a fluent agent is not thereby a reliable one. Whenever an agent must satisfy non-negotiable constraints, follow explicit rules, or produce auditable justifications, part of its behaviour needs a meaning that can be inspected and tested. Web Prolog offers executable symbolic representations, explicit derivations, and programmable boundaries for failure and resource use. These properties apply to the fragment actually formalised in the program; they do not make arbitrary Prolog code terminating, consistent, or correct.

The second pressure is that the modern Web removes much of the old deployment bottleneck for symbolic AI. Symbolic systems have often remained laboratory artefacts because deployment required bespoke infrastructure, heavy integration work, and awkward maintenance. Standard protocols, cloud practices, and ubiquitous client platforms change this equation. A Prolog engine can now sit behind stable interfaces as a verifier, planner, policy engine, dialogue manager, or explanation service, and can be composed with other services without losing its semantic character.

We do not argue for Prolog everywhere, only where rules, constraints and explanations must be explicit, so that the Prolog component can supervise more fallible ones. Prolog has real costs: a steep learning curve, fragmented implementations, and a small ecosystem. These costs may be worth paying when the alternative is to reconstruct constraint handling, control, and auditability in ad hoc application code.

8.2 Symbolic competence in one engine

This section recalls five things a single Prolog engine already provides for AI. Each will reappear later as a service or an agent on the Prolog Web.

8.2.1 Knowledge representation and inference

A Prolog program can function simultaneously as a declarative knowledge base and as an executable inference procedure. A canonical example is a small relational theory of kinship, where recursion expresses transitive closure:

```
ancestor_descendant(X, Y) :- parent_child(X, Y).
ancestor_descendant(X, Z) :-
    parent_child(X, Y), ancestor_descendant(Y, Z).

parent_child(X, Y) :- mother_child(X, Y).
parent_child(X, Y) :- father_child(X, Y).

mother_child(trude, sally).

father_child(tom, sally).
father_child(tom, erica).
father_child(joe, tom).
```

A query such as `?-ancestor_descendant(joe,Who)` asks for logical consequences of the clauses and enumerates answers via backtracking. This combination – explicit relational structure plus a proof procedure – is the basic engine behind many symbolic systems, and in Prolog the proof procedure is built in.

8.2.2 Planning as search in a state-transition model

Many agent tasks are naturally expressed as planning: find a sequence of actions that transforms an initial state into a goal state. A small meta-program suffices when actions are represented as declarative transition rules:

```
plan(P, P, []).
plan(P0, Pf, [A|As]) :-
    step(P0, P1, A),
    plan(P1, Pf, As).

step(start, position(monkey, bananas, chair), start).
step(position(Y,Y,Y), final, climb).
step(position(X,Y,X), position(Y,Y,Y), carry(X,Y)).
step(position(X,Y,Z), position(Z,Y,Z), walk(X,Z)).
```

Here the entire world state is a first-order term, and each `step/3` clause specifies an action's preconditions and effects. Prolog's search then discovers action sequences by exploring transition alternatives under backtracking.

For example, the query `?-plan(start, final, Plan)` produces `Plan = [start,walk(monkey,chair),carry(chair,bananas),climb]`. The example

leaves search control visible: it has neither a visited set nor a depth bound, and may therefore diverge when applied to transition systems containing unproductive cycles. Practical planners add the resource and repetition controls appropriate to their state space.

8.2.3 Meta-interpreters as a control and explanation lever

A distinctive Prolog idiom is to treat programs as data and write interpreters for restricted logics, alternative control regimes, or explanation-producing variants. A minimal meta-interpreter mirrors Prolog's own proof search:

```
prove(true) :- !.
prove((A,B)) :- !, prove(A), prove(B).
prove(A) :- clause(A,B), prove(B).
```

Small changes to such an interpreter can add substantial functionality without changing the object-level knowledge base. For example, adding a second argument allows the interpreter to construct a proof object – a structured explanation that records which rules were used:

```
prove(true, true) :- !.
prove((A,B), (PA,PB)) :- !, prove(A, PA), prove(B, PB).
prove(A, A/Proof) :- clause(A,B), prove(B, Proof).
```

Proof terms of this kind are evidence-carrying data structures that can be inspected, summarised, or shown to a user. Later in this chapter the same construction lets a single explanation span a distributed computation.

8.2.4 An expert system with a query-the-user facility

Expert systems were a flagship application of symbolic AI during the 1980s. They did not become a lasting mainstream paradigm, but the core pattern remains instructive, especially in Prolog: it shows how deduction can be interleaved with interaction in a disciplined way.

An expert system typically consists of a knowledge base plus an inference engine. A practical system must also be able to acquire facts that are not already stored. In Prolog, a simple way to realise this is to extend the meta-interpreter from Section 8.2.3 with a single additional clause that queries the user whenever a goal is declared `askable`:

```
prove(A) :- askable(A, Q), writeln(Q), read(T), T == yes.
```

The accompanying knowledge base supplies both domain rules and a set of `askable/2` declarations that map predicates to human-readable prompts:

```
good_pet(X) :- bird(X), small(X).
good_pet(X) :- cuddly(X), yellow(X).

bird(X) :- has_feathers(X), tweets(X).

askable(tweets(_), 'Does it tweet?').
askable(small(_), 'Is it small?').
askable(cuddly(_), 'Is it cuddly?').
askable(has_feathers(_), 'Does it have feathers?').
askable(yellow(_), 'Is it yellow?').
```

When `prove/1` encounters an askable subgoal, it suspends deduction, poses the associated question, reads the reply, and then resumes the derivation. For example:

```
?- prove(good_pet(tweety)).
Does it have feathers?
|: yes.
Does it tweet?
|: yes.
Is it small?
|: no.
Is it cuddly?
|: yes.
Is it yellow?
|: yes.
true.
?-
```

The current proof attempt determines which information is requested next. The interaction is system-initiative rather than genuinely mixed-initiative: the engine advances the derivation whenever its knowledge suffices and consults the user when its proof strategy reaches an askable subgoal. The `askable` declarations therefore act both as an interaction interface and as a lightweight user model: they mark which facts are expected to come from dialogue rather than from stored knowledge.

This is, of course, a toy shell. In more realistic systems one would add typed answers, richer question forms, explanation facilities, and a persistent dialogue state. For a fuller treatment of Prolog-based expert systems, see (Merritt, 1989) and (Sterling and Shapiro, 1994).

8.2.5 Parsing as deduction: DCGs

Definite Clause Grammars (DCGs) turn parsing into a form of proof search: a sentence is grammatical exactly when it can be derived from the grammar rules. DCGs are also reversible, so the same grammar can be used for generation. Operationally,

naïve top-down execution makes left recursion problematic, but in tabled Prolog systems this restriction largely disappears: tabling turns parsing into dynamic programming while preserving a declarative rule notation. (Tabling is a system feature rather than part of the ISO core, so it should be read here as an optional acceleration of the same modelling stance.)

8.2.6 Summary

The examples above form a compact map of facilities. A single Prolog engine already supports (i) explicit relational knowledge representation, (ii) planning as systematic search in state-transition models, (iii) meta-level control and explanation via interpreters that construct proof objects, (iv) interactive acquisition of missing facts, and (v) grammar-based language processing as deduction.

The next section lifts these idioms onto the Agentic Web. Instead of treating predicates as internal procedures, we treat them as services: message-driven actors with interfaces, state, and failure modes.

8.3 Prolog AI agents on the Agentic Prolog Web

The idioms (i)–(v) need not remain local. Exposed as web-native actors, the same programs become distributed services that can be composed into interacting agents. No new formalism is needed. In this setting, what used to be a local `read/1` becomes a `prompt/2` message expecting a response; what used to be a local meta-interpreter becomes a mobile proof-producing component; and what used to be an internal predicate call may become a remote call whose provenance is part of the explanation.

The following examples cover interactive deduction, proof construction across nodes, remote parsing, the relationship to earlier agent frameworks, and the deployment properties of the resulting services.

8.3.1 An expert-system agent and a simulation

Classic expert systems faced two persistent bottlenecks: the difficulty of extracting knowledge from humans and the lack of a ubiquitous infrastructure for deployment. A web-native actor substrate does not remove the knowledge acquisition problem, but it lets elicitation be expressed as incremental interaction and lets a symbolic system be deployed as a service rather than as a local read/write loop.

To adapt the askable meta-interpreter pattern from Section 8.2.4 to an agent setting, we replace terminal I/O with an interaction predicate. Concretely, the clause that previously wrote a question and read an answer becomes:

```
prove(A) :- askable(A, Q), input(Q, T), T == yes.
```

The purpose of `input/2` is to decouple deduction from any particular user interface. In a local setting, `input/2` might still be implemented on top of `read/1`. As we saw in Chapter 4, on the Prolog Web it is implemented by sending a `prompt` to some conversational partner and waiting for a reply via `respond/2`.

We can now package the expert system as an expert-system agent by spawning a remote toplevel actor and preparing a call to `prove/1`:

```
expert_ask(Query, Pid, Options) :-
    toplevel_spawn(Pid, [
        session(false)
      | Options
    ]),
    toplevel_call(Pid, prove(Query), [
        limit(1),
        once(true)
    ]).
```

The caller's `Options` configure spawning and source loading; the query itself uses only `limit(1)` and `once(true)`. The simulation asks an existential question: it needs one proof that the conclusion follows, not an enumeration of every proof. The `limit(1)` option restricts the returned slice to one solution, while `once(true)` commits the remote computation after that slice and discards any remaining alternatives. The answer is therefore terminal even when another proof might exist. These options do not limit the number of prompts, which arise while that single proof is being sought.

To demonstrate the pattern, we construct a small simulation in which a second agent – in this case, the shell process – plays the role of the user. The simulation answers questions stochastically, illustrating that the expert system does not control the dialogue by a script; rather, the proof search itself determines which information must be acquired next.

```
simulation(Options) :-
    random_member(Name, [tweety,pingu]),
    format('A1: Is ~w a good pet?~n', [Name]),
    expert_ask(good_pet(Name), Pid, Options),
    interact(Pid, Name).

interact(Pid, Name) :-
    receive({
        failure(Pid) ->
            format('A2: No, ~w is not a good pet.~n', [Name]) ;
        success(Pid, _, false) ->
            format('A2: Yes, ~w is a good pet.~n', [Name]) ;
        error(Pid, Error) ->
            throw(Error);
```

```
        prompt(Pid, Question) ->
            format('A2: ~w~n', [Question]),
            random_member(Answer, [yes,no]),
            respond(Pid, Answer),
            format('A1: ~w~n', [Answer]),
            interact(Pid, Name)
    }).
```

For this run, assume that the shared database of `n9.org` contains the domain rules and declarations for the askable predicates. The inference engine is loaded into the new actor's private database, together with one task-specific fact:

```
?- simulation([
       node('http://n9.org'),
       src_predicates([prove/1]),
       src_list([yellow(tweety)])
   ]).
A1: Is tweety a good pet?
A2: Does it have feathers?
A1: yes
A2: Does it tweet?
A1: yes
A2: Is it small?
A1: no
A2: Is it cuddly?
A1: yes
A2: Yes, tweety is a good pet.
true.
?-
```

The architectural point is the separation between the node-resident knowledge, the privately loaded interpreter and fact, and the interaction process. The actor's questions are selected by its current proof attempt rather than by a separate dialogue script. The Web substrate supplies distribution and composability, while Prolog supplies the proof structure that drives the interaction.

The code is pedagogical. A deployed client should also monitor the actor and bound its receive operation, so that node failure or a lost connection does not leave the simulation waiting indefinitely.

8.3.2 Explainable AI across nodes: distributed proof trees

A defining advantage of logic-based, symbolic AI is the possibility of constructing explanations from explicit rules and facts. This transparency depends on the predicates involved: foreign code, side effects, cuts, and unexposed host predicates may

still be opaque. On the Agentic Prolog Web, a cooperating service can preserve the explanation discipline across a distribution boundary rather than reducing a remote subgoal to a black box.

A compact meta-interpreter can achieve this by treating remote calls as proof-producing sub-derivations. When the interpreter encounters a remote goal, it ships itself to the remote node and requests a proof there, then folds the result back into the local derivation:

```
prove(true, true) :- !.
prove(rpc(URI,A), Proof) :- !,
    prove(rpc(URI,A,[]), Proof).
prove(rpc(URI,A,Options), A@URI/Proof) :- !,
    rpc(URI, prove(A, Proof), [
        src_predicates([prove/2])
      | Options
    ]).
prove((A, B), (ProofA, ProofB)) :- !,
    prove(A, ProofA),
    prove(B, ProofB).
prove(A, A/Proof) :-
    clause(A, B),
    prove(B, Proof).
```

The `src_predicates([prove/2])` option is what makes the explanation discipline recursive: the meta-interpreter is transmitted to the remote node along with the goal, so the remote node runs `prove(A,Proof)` rather than the bare goal `A`. Proof construction therefore applies at every hop, not just the first.

The returned proof is not constrained to have the form `Query/Proof`: a remote conjunction returns a pair of proofs, and `true` returns `true`. The enclosing `A@URI/Proof` records the instantiated remote goal and its location, together with the complete proof obtained there.

When the remote node accepts the transmitted interpreter and exposes the clauses it may inspect, the same explanation discipline is applied at each node, and remote contributions are marked in the resulting proof term by the `@` operator.

Suppose, as in Section 4.2.3, that the local node states that every human is mortal by means of a rule `mortal(X) :- human(X)`, and defines `human(X)` by consulting a remote node. If the fact `human(plato)` is stored on `n1.org`, we obtain a proof object that explicitly records the distribution boundary:

```
?- prove(mortal(plato), Proof).
Proof = mortal(plato)/
          (human(plato)/
             (human(plato)@'http://n1.org'/
                (human(plato)/true))).
?-
```

The annotation `human(plato)@'http://n1.org'` documents not just what was concluded but where. The proof term is ordinary Prolog data: it can be inspected, stored, transmitted, or rendered into a human-readable explanation by a further layer of code.[2]

Proof-producing interpreters can of course generate very large derivations. On the Prolog Web this is best treated as a controllable engineering choice rather than an unavoidable pathology: one may bound proof depth, abstract away local details and retain only a node-level skeleton, cache recurring subproofs, or compress proof objects into summaries suitable for user-facing explanations.

8.3.3 Meta-interpreters as generic behaviours

The two preceding examples share a factoring familiar from Chapter 5. The meta-interpreter defines a reasoning policy – askable deduction or proof-term construction – while the actor substrate supplies a mailbox, lifecycle, and network address. The application supplies the knowledge base. Changing the knowledge base therefore produces a different expert under the same reasoning policy; changing the interpreter applies a different policy to the same knowledge.

Packaged as an actor, such an interpreter also acquires the deployment discipline of a generic behaviour: clients see a stable process interface, while supervision and addressing remain independent of the domain clauses. The analogy should not be pressed into a proposed fixed catalogue, but it suggests useful reusable components such as bounded, tabled, interactive, or provenance-producing interpreters.

8.3.4 Parsing as remote deduction

A client may offload parsing to a node that hosts a grammar, without embedding that grammar locally:

```
?- rpc('http://n6.org', phrase(s(PT), [joe,saw,a,telescope])).
PT = s(np(pn(joe)), vp(v(saw), np(det(a), n(telescope)))).
?-
```

This vignette illustrates the broader pattern: a symbolic competence can be hosted with the resources and execution strategy it needs, then accessed through a uniform remote call. A client need not embed the grammar or reproduce its implementation assumptions. The host may run an ordinary or tabled DCG, or it may front an external grammar system such as Grammatical Framework (Ranta, 2011); the client still sends a goal and receives a structured parse.

---

[2] One practical requirement is that predicates inspected through `clause/2` must actually be visible to the interpreter. Permission to inspect compiled predicates varies between Prolog systems.

When the grammar constructs the tree as a faithful record of the rules used, the returned PT also serves as a compact proof witness. Remote parsing is then a special case of the proof-carrying remote calls of Section 8.3.2: the desired syntactic structure is also evidence for the derivation.

8.3.5 Implementing agent frameworks in Web Prolog

Logic-based agent languages and frameworks have been implemented in Prolog for decades. Systems such as DALI (Costantini and Tocchio, 2004) and LPS (Kowalski and Sadri, 2015) show that one can retain a Horn-clause core while adding structures for events, actions, reactive rules, and persistent state. Their appeal lies in combining logic programming virtues – declarative clarity, inspectability, and verifiability – with practical mechanisms for autonomy and interaction.

This tradition is richer than an actor substrate alone. BDI architectures model deliberation in terms of beliefs, desires, and intentions, while agent communication languages attach speech-act semantics to messages. Web Prolog standardises neither mental attitudes nor an agent communication language. Its private databases, behaviours, and term messages provide places in which such notions can be represented, but their meaning belongs to the framework layered above the substrate. Web Prolog provides only actors with private databases and mailboxes. That is enough to implement such frameworks. A receive loop that pattern-matches on incoming messages, together with an explicit representation of pending goals, already suffices to encode reactive rules of the form "on event then action" or "on event and if condition, then action" as ordinary Prolog clauses over the mailbox and the private database. In statechart actors, this becomes even more structured, allowing rich reactive and mixed-initiative behaviours to be expressed as statechart specifications interpreted by Web Prolog rather than as ad hoc control code.

LPS-style engines, DALI-like event-driven agents, and more elaborate frameworks can be layered over Web Prolog, with actors as their concrete embodiment on the open Web. The extension is conservative in an architectural sense: neither Web Prolog's actor primitives nor the framework's own semantics need to change. Each framework remains responsible for what its agents mean, while Web Prolog supplies process identity, private state, message passing, and lifecycle. Whether a hosted engine behaves exactly as it would on its own is a separate claim, and one that can be tested.

LPS2, a reimplementation of LPS released as open source in October 2026,[3] makes such a test straightforward. Its core performs no I/O, starts no threads, and reads no clock. An execution is a single Prolog term, which a caller advances one cycle at a time, feeding in observed events and reading out the actions the engine has chosen. A Web Prolog actor supplies exactly what the core leaves out. In a small experiment, the session simply became the argument of an actor's loop:

[3] `https://github.com/LogicalContractsOrg/lps2`, under the Apache 2.0 licence.

```
lps_agent(S0, Pending0) :-
lps_session_status(S0, running), !,
collect_cycle(Pending0, Pending1),   % receive/2 until the cycle ends
admit(Pending1, Events, Pending),    % messages become LPS events
lps_session_observe(S0, Events, S1),
lps_session_step(S1, S2, cycle(_, _, _, _, Committed)),
dispatch(Committed),                 % mapped actions become messages
lps_agent(S2, Pending).
lps_agent(_, _).
```

The mailbox serves as the agent's inbox, and `receive/2` with a timeout sets the pace of its cycles. LPS2's own mode for long-running sessions builds the same things by hand: a thread, an inbox held in the dynamic database, and a mutex guarding every update.

Two LPS agents, a customer and a hotel, negotiated a room in this way. Random network delays changed the cycle in which each message arrived. Each agent logged the events it admitted and when. Written back into its program as ordinary `observe` declarations, that log reproduced the hosted run exactly on a standalone engine, in every one of 37 agent runs. This does not prove that hosting preserves the semantics of LPS in general, but it shows the shape of the argument. An LPS cycle is fully determined by the session and the events observed, so each hosted agent's run is an ordinary LPS run, and the explanations LPS2 gives of it remain local and complete.

What hosting does not settle is how the agents should behave towards one another, and the experiment made two such questions concrete. The first concerns actions and their success. In LPS, the effects of an action the agent chooses take hold in the next cycle. A request to another agent must therefore be modelled as establishing only that the request was made; the reservation itself arrives later, as an observed event, or never. Even then, a deadline counted in one agent's own cycles is not a protocol. A customer whose patience was too short gave up on every room and then cancelled each confirmation as it arrived late. Every agent behaved correctly, and the result was useless.

The second concerns admission. If a precondition forbids any of the events observed in a cycle, LPS refuses all of them together. With a single peer this is harmless. With several, the policy by which the hosting layer admits messages decides who suffers for whose misbehaviour. When the hotel barred one customer, a barred request and a legitimate one that happened to arrive in the same cycle were refused together. Admitting one event per cycle confined the refusal to the barred sender. Neither LPS nor Web Prolog settles this alone. It belongs to the adapter, the layer where a framework meets the Web.

This is the division of labour that Web Prolog proposes. Each framework's semantics stays inside the actor that hosts it. What happens between agents, including admission, deadlines, and agreement, needs explicit protocols between actors. Hosting several engines does not turn their local constraints into rules that hold across the whole system.

8.3.6 Scalability and interoperability on the Web

Two traits of the Web are important for symbolic agents: established deployment machinery and protocol-level interoperability.

Scalability

Web Prolog agents are processes that can be spawned, monitored, and terminated. Standard deployment practices can be used around nodes: containers can replicate stateless endpoints, and ordinary observability tools can monitor host processes. Stateful actors require more care. A deployment must preserve or discover actor location, route later messages to the correct process, and coordinate lifecycle and recovery; a conventional load balancer cannot provide these properties by itself. Inside a node, actor supervision supplies a finer-grained failure boundary. The architecture therefore supports distribution, while large-scale operation remains a deployment problem rather than an automatic consequence of using HTTP or WebSockets.

Interoperability

The Web rewards components that speak stable protocols. On the Agentic Prolog Web, terms travel as structured data, which makes it natural to expose symbolic services through documented message schemas and to interoperate with non-Prolog components through defined encodings. Conversely, Prolog agents can call ordinary web services, interact with browser clients, or orchestrate external systems while keeping their internal reasoning symbolic and inspectable. A single canonical Prolog–JSON mapping is not assumed; as Chapter 11 discusses, representation choices remain part of the interoperability contract.

Distribution does not make every component symbolic. Many agents incorporate neural services for language, perception, or retrieval while retaining explicit rules for the commitments that matter. The next section examines architectures in which Web Prolog supplies reasoning and control, while neural models remain replaceable components.

8.4 Neuro-symbolic AI agents

> To build a robust, knowledge-driven approach to AI we must have the machinery of symbol manipulation in our toolkit. Too much useful knowledge is abstract to proceed without tools that represent and manipulate abstraction, and to date, the only known machinery that can manipulate such abstract knowledge reliably is the apparatus of symbol manipulation.
>
> *Marcus and Davis (Marcus and Davis, 2019, p. 17)*

Large language models are powerful generators and interpreters of language, but sampled generation does not by itself establish that a constraint has been satisfied or that an action is permitted. Neuro-symbolic AI is the broad family of approaches that combines learned representations with explicit symbolic structure or reasoning (d'Avila Garcez and Lamb, 2023). The coupling ranges from modules that exchange structured data at run time to systems in which neural and symbolic representations participate in one learning procedure.

This section concentrates on the architectural end of that range. Models supply language, perception, retrieval, or heuristic proposals; Prolog supplies explicit rules, search, and checks; actors and statecharts control when the components may act and how their failures are contained. These roles are design choices.

8.4.1 Why hybridise?

The question is which component should hold authority over which commitment. Models are useful at turning heterogeneous human input into candidate structures and at turning structured results back into fluent language. Prolog is useful where rules, alternatives, and constraints must be named and executed. An actor substrate adds concurrency and explicit failure boundaries.

We do not present symbolic AI as the route to intelligence; the history of expert systems and the Fifth Generation project is reason enough for caution (Körner et al., 2022). Authority belongs with the component whose semantics match the obligation, and any guarantee extends only as far as the rules, trusted code and resource assumptions behind it.

Aspect	Prolog component	Model component
Representation	Explicit clauses, terms, constraints, and control.	Learned distributed representations and generated sequences.
Natural role	Checking, search, policy, planning, and structured state.	Interpretation, generation, retrieval, perception, and proposals.
Evidence	Can return derivations, failed constraints, and other inspectable artefacts.	Can explain in fluent language, but generated explanations are not proof artefacts.
Failure modes	Failure, exception, divergence, or incompleteness; bounds and monitoring must make waiting observable.	Invalid, unsupported, or unstable output; external validation is needed when correctness matters.
Change	Rules and facts can be inspected and edited directly.	Behaviour changes through context, tools, model versions, or training.

Table 8.1 Complementary engineering roles in a hybrid agent.

A useful shorthand for Table 8.1 is *the model proposes; the logic disposes*. The model may propose an interpretation, plan, tool call, or utterance. The symbolic layer decides which commitments follow under the rules it has actually been given.

8.4.2 Web Prolog as symbolic backend and agent layer

A hybrid agent needs more than machine-learning libraries. Web Prolog combines symbolic search with mailboxes, supervision, and remote communication. A model service can therefore be one actor among others, or a co-processor called by a wrapper actor, without making the model itself the owner of lifecycle and policy.

This also clarifies the role of multi-agent decomposition. Assigning subgoals to specialist actors is a concurrency and deployment pattern, not by itself a form of neuro-symbolic integration. It becomes neuro-symbolic when some of those actors exchange learned representations or model outputs with symbolic components. Actor boundaries then provide useful locations for monitoring, timeouts, provenance, and access control.

Normative reasoning as executable policy

Legal, ethical, and organisational constraints can be modelled as rule sets in Prolog, including defeasible rules, exceptions, and priority schemes. This makes it possible to treat compliance not as post-hoc documentation but as executable policy: before an action is taken, the agent can attempt to prove that it is permitted under the current facts, and if no proof exists it can either refuse, ask for missing information, or escalate to a human.

For computational-law treatments in a Prolog setting, see, for example, the discussion in *Prolog: The Next 50 Years* and related work on executable norms and obligations (Kowalski et al., 2023a; Kowalski and Datoo, 2022).

Bridging, not siloing

Hybrid architectures are not an either/or choice between Prolog and Python. Janus provides fast, bidirectional interoperation adopted by more than one Prolog implementation. Prolog can retain orchestration while delegating vectorised numerics, model inference, or tensor operations to Python libraries (Andersen and Swift, 2023; Wielemaker, 2023).

8.4.3 Architectural patterns for hybrid agents

The most useful patterns can be organised by the direction and closeness of the boundary between model and program.

A model calls Prolog as a tool

A model may emit a structured request for retrieval, arithmetic, symbolic query answering, or constraint checking. The surrounding orchestrator validates the request, invokes an authorised Prolog predicate, and returns a structured result. The model proposes the call; an ordinary program holds the authority and performs the effect. Allowed predicates, resource limits, and result schemas remain part of the service boundary.

The Model Context Protocol (MCP) is a protocol through which LLM-based applications call external tools. SWI-Prolog's experimental `mcp` pack connects Prolog to it.[4] A declaration `mcp_tool/3` exposes a Prolog predicate as an MCP tool, with a description and a JSON Schema (a machine-readable description of the data the tool accepts). MCP complements the Prolog Web protocols. It offers a bounded, schema-described tool call; the PTCP preserves a Prolog session with incremental nondeterministic answers, prompts, and control messages. A knowledge base may usefully offer both interfaces.

Prolog calls a model service

Conversely, Web Prolog can delegate an open-ended language task to a model and treat the call as an effectful predicate guarded by timeouts, budgets, and policy checks. The exact provider API will evolve, but the architectural point is stable: the Prolog side controls when the call is made, how its result is validated, and what happens when it fails.

The following provider-neutral sketch shows the shape of an HTTP call to a text-generation endpoint:

```
:- use_module(library(http/http_client)).
:- use_module(library(http/http_json)).
:- use_module(library(option)).

call_llm(SystemMsg, UserMsg, Options, Reply) :-
    option(endpoint(Endpoint), Options),
    option(authorization(Auth), Options),
    option(model(Model), Options),
    Payload = json([
        model=Model,
        input=[
            json([role=system, content=SystemMsg]),
            json([role=user,   content=UserMsg])
        ]
    ]),
```

[4] `https://github.com/JanWielemaker/mcp`. The repository describes the package as experimental; its version and supported protocol revision may change.

```
http_post(Endpoint, json(Payload), Reply,
          [request_header('Authorization'=Auth)]).
```

This is a provider-neutral sketch, not ACTOR-profile client code. A node owner would normally place the adapter behind an authorised predicate or service and align the payload with the provider's current specification.

Containing latency and failure

The HTTP predicate blocks while the provider responds. This is acceptable in a dedicated wrapper actor whose job is to wait for that service; it is not acceptable in SXML executable content, which should initiate long-running work asynchronously and consume the result as a later event. A statechart can send a correlated request and enter a waiting state whose transitions handle reply and failure events. A state-scoped `after` transition can handle the timeout: its timer is armed on entry, cancelled automatically when another transition leaves the state, and renewed with a fresh activation if the state is entered again. A wrapper actor may instead supply a correlated timeout event when the timeout belongs to the external operation rather than to the control state.

The wrapper is also the natural failure boundary. It may be monitored or run under the supervisor behaviour of Chapter 5; a network exception can then terminate or restart it without corrupting the governing statechart. Correlation references prevent a late reply from an earlier request from being accepted in a later waiting phase.

A statechart governs the interaction

The surrounding layer that decides when to call a model, validates its output, enforces limits, and routes control is often called an agent *harness*. In Web Prolog, this harness can be an executable statechart rather than an informal combination of prompts, conditionals, and retry logic. Its states, guards, transitions, and stopping conditions answer the question "what may happen next?" independently of the model's disposition. The model may propose actions or emit candidate events; only transitions admitted by the chart acquire behavioural effect.

Tighter neuro-symbolic integration

Some systems integrate learning and symbolic inference more closely. DeepProbLog permits neural predicates to supply probabilistic information inside ProbLog programs and supports learning from queries (Manhaeve et al., 2021). NeurASP combines neural-network outputs with answer-set programs (Yang et al., 2020). These systems use different semantics and training mechanisms; they should not be reduced to one generic "differentiable Prolog" architecture.

From the Trinity perspective, such a component could be exposed behind a predicate boundary like any other specialised reasoner. Keeping final policy in explicit rules, so that learned scores rank permitted alternatives rather than silently expanding the permitted action set, is a possible system design constraint. It is not a guarantee supplied automatically by DeepProbLog, NeurASP, or differentiability itself.

8.4.4 Typed probabilistic decisions as semantic observations

The preceding patterns leave a useful middle position between text generation and neural predicates embedded in probabilistic logic. A learned model can answer a bounded semantic question, while an ordinary program treats the typed, uncertain answer as an observation rather than a command.

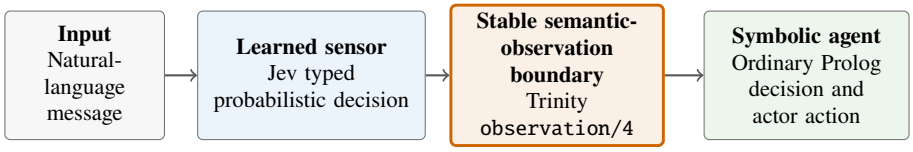
This section records the architectural result of a small Jev experiment. TypeSafe introduced Jev as its first public "System One Model" (Almeida, 2026). In September 2026 its early-access API accepted unstructured state and typed questions, then returned typed probabilistic decisions (TypeSafe AI, 2026a,b). Those product details will change. The online Trinity demonstrator therefore contains the current HTTP client, complete question wrappers, actor sources, SXML chart, transcripts, and tests.[5] The chapter keeps the boundary that those details were built to test.

The experiment has two stages:

```
Stage 1: Jev answers bounded semantic questions;
         Web Prolog observes, thinks, and acts.
Stage 2: the same observation enters SXML when explicit
         interaction state becomes useful.
```

Jev returns a typed decision, and the Trinity layer treats it as an *observation* (our term, not a Jev operation). Prolog clauses decide what follows from it. In Stage 2, a statechart also decides what may happen next; Stage 1 shows that a complete Observe–Think–Act loop does not need one.

The following pipeline makes the first-stage boundary explicit:



This is architectural neuro-symbolic integration (d'Avila Garcez and Lamb, 2023). The learned and symbolic components exchange an explicit data structure at run time; they do not share one training procedure or one probabilistic semantics. Each

[5] See `https://github.com/torbjornlager/trinity-demonstrator`; the sources are in `examples/jev/` and the web tutorial is `web/jev-sxml-tutorial.html`.

boundary can be inspected independently, and the symbolic portion remains ordinary Prolog.

A bounded semantic sensor

The metaphor of a *semantic sensor* helps distinguish this pattern from open-ended generation. A physical sensor maps some part of the world into a bounded reading with known units and an error profile. A semantic sensor maps heterogeneous input into a bounded vocabulary with uncertainty attached. In both cases the reading may be useful without being authoritative.

Jev supplied Choice, Score, and Noul questions. A Choice selected one item from a caller-defined vocabulary and returned the whole distribution plus a confidence value. A Score located the input on an ordered rubric. A Noul returned the probability of yes. These shapes are not interchangeable: a Choice winner is not certainty about the winner, and a Score is not a probability. The deployment example uses only Choice; the demonstrator keeps all three forms so their different uncertainty semantics remain visible.

Both stages consume one application-facing term:

```
observation(IntentChoice,
            UrgencyScore,
            QuestionNoul,
            provenance(RawResponse))
```

The typed fields become ordinary Prolog data; no returned string is parsed as a clause and no model-generated term is executed as a goal. The complete provider response is kept as provenance.

Stage 1: a complete actor without SXML

The node owner exposes one narrow predicate, `jev_deployment_observation/2`. Uploaded actor code cannot choose the endpoint, credential, model, or question vocabulary, and calls are bounded by size, timeout, admission rate, and concurrency. The provider may produce a semantic reading and the actor may request one; neither may perform a deployment. Every effect in the demonstrator is conversational or simulated.

The first actor is already a complete Observe–Think–Act agent. It obtains the shared observation, applies ordinary clauses, and sends a correlated reply:

```
jev_agent_loop :-
    receive({
        say(From, Ref, Text) ->
            jev_observation(Text, Observation),
            jev_stage1_decision(Observation, Action, Reply),
            Report = report(Text, Observation,
                            action(Action), Reply),
            From ! agent_reply(Ref, Report),
            jev_agent_loop;
        stop ->
            true
    }).
```

A confident `request_deploy` prepares a simulated preview; `status` reports status; and uncertain or unavailable readings cause no effect. Stage 1 needs no statechart.

Stage 2: explicit interaction state

The second stage introduces a confirmation protocol. The gateway still uses the same observation predicate and term. External inference completes before the gateway sends one `observed/4` event to the chart, so no guard or action waits for HTTP. Run-to-completion begins when the event arrives; it does not make the earlier network operation atomic.

The deployment assistant has two states, `idle` and `awaiting_confirmation`. The same confident `affirm` observation has two consequences:

Current state	Consequence
`idle`	Explain that nothing awaits confirmation; remain in `idle`.
`awaiting_confirmation`	Consume the stored proposal, record the simulated deployment action, and return to `idle`.

A `request_deploy` observation records the proposal as `pending_proposal(Ref, Text, Intent)`. The statechart then enters `awaiting_confirmation`. A `deny` observation consumes and cancels the stored proposal. SXML exposes the protocol and its permitted next moves (Harel, 1987). Ordinary Prolog could represent the state too, but the statechart makes it inspectable without changing the semantic-observation boundary.

Uncertainty without probabilistic control flow

The complete Choice distribution crosses the boundary, while a deterministic guard decides whether its winner may drive a transition. Below the example's `0.70` confidence threshold, the chart asks for clarification and stays in the same state. This is what the literature calls a reject option (Geifman and El-Yaniv, 2017). The chart carries no probability distribution over states, and Prolog does not backtrack over possible model readings.

Some limits apply. The value `0.70` has no general authority: thresholds must be evaluated against the particular model, question, error costs and users, and a cheap clarification and an irreversible action should not share one. A provider or transport failure, reported as `unavailable(Reason)`, is not a denial, and after a timeout a remote effect may still have happened. The provenance record is not a durable audit log. Finally, a typed answer is only as good as the question: the caller can still choose a poor vocabulary or omit an alternative, and type correctness says nothing about accuracy or calibration (Guo et al., 2017).

The experiment separates three questions:

1. What does the input appear to mean?
2. What follows from that meaning and the represented facts?
3. What is permitted to happen next in this interaction?

Jev addresses the first question, and Prolog addresses the second. In Stage 1, ordinary clauses also decide the immediate behaviour. In Stage 2, SXML makes the third question explicit once it depends on interaction state. Changing the semantic sensor need not change the control policy, and changing the dialogue policy need not retrain the sensor. Surface language also remains below the boundary: different utterances that produce the same observation enter the same Prolog clauses and SXML transitions. Model evaluation and program verification remain separate.

The next case study uses a more demanding service request. Its model proposes structured fields rather than a bounded intent, and its statechart must deal with authenticated confirmation, correction, policy conflict, asynchronous dispatch, and uncertain external outcomes.

8.4.5 A statechart-wrapped service-request agent

To make this pattern concrete, consider an internal service-request agent. A requester describes a service in ordinary language; a model proposes structured fields; the requester confirms a specific summary; Prolog rules select automatic fulfilment, rejection, or human review; and an asynchronous dispatch acknowledgement determines the final outcome. The example develops the distinction between confirmation and proposal discussed in Section 6.6.1. The model helps prepare a request; it does not authenticate the requester, grant approval, or establish that dispatch succeeded.

The runnable companion is in `companions/service-request/`. It contains the complete SXML chart, Prolog helpers, an offline demonstration, and executable tests. It runs on the demonstrator's local statechart interpreter. Its model and fulfilment adapters are asynchronous test doubles: no model account or real service desk is needed.

From the companion directory, the complete successful path is run with:

```
swipl -q -s service_request.pl -g service_request:demo -t halt
```

The accompanying README gives the runtime checkout layout and the test command. The demonstration proposes a laptop costing 700, confirms the summary, dispatches it, and reports `succeeded(ticket_42)`. The tests also exercise prohibition, policy conflict, missing information, reviewer decisions, malformed fields, stale events, and timeouts.

The trusted boundary and the request revision

The chart belongs to a trusted wrapper. That wrapper authenticates the user, authorises access to the request, and derives the principal from the session; it must not accept a claimed principal from model output or a request body. The companion fixes the requester as `alice` and the reviewer as `reviewer`. Its `submit/3` is a privileged local interface that simulates this boundary, not an implementation of authentication. The wrapper keeps the chart pid and a random envelope secret private. Only trusted adapters can attach that secret to model and dispatch replies. The model receives the request text and field schema, not these capabilities.

Each request has an identifier and an integer revision. Entering `collect` starts a linked adapter actor without waiting for the model. The adapter captures a fresh correlation identifier and the current revision, then wraps its eventual reply as `model_result(Key,C,R,Result)`. The offline adapter announces a `model_request` to the demo controller and waits for `deliver(Result)`; a provider adapter would perform the external call at that boundary. An error is also a result, not an exception allowed to bypass the chart.

The initiation itself is ordinary actor code:

```
begin_model :-
    context(Key, _, Observer, Request, _),
    revision(R, Text), uuid(C),
    retractall(pending(_, _)),
    assert(pending(model, C)), self(Self),
    spawn(service_request:model_adapter(
              Self, Key, Observer, Request, C, R, Text),
          _, [link(true)]).
```

The essential collection path is below. Module qualifications are omitted from these excerpts; the companion contains the full executable chart.

```
<state id="collect">
  <onentry>begin_model</onentry>
  <go to="confirm" on="model_result(Key,C,R,fields(Fs))"
      if="model_reply(Key,C,R), valid_fields(Fs,Canonical)">
    remember(Canonical)
  </go>
  <go to="failure" on="model_result(Key,C,R,_)"
      if="model_reply(Key,C,R)">
    finish(failed(invalid_model_reply))
  </go>
  <go to="failure" after="5">
    finish(failed(model_timeout))
  </go>
</state>
```

The full chart also distinguishes recognised model errors from malformed replies. The `model_reply/3` guard checks the envelope secret, correlation, and revision. A late reply cannot replace the current proposal. The SXML `after` transition supplies the timeout: the interpreter arms its timer on entry and cancels it on exit. No external component has to invent a `timeout` event. Confirmation, review, and dispatch have their own timed transitions. The companion makes the duration configurable; ignored events do not restart it.

`valid_fields/2` accepts only a ground list containing at most one `service/1` and one `cost/1`. Service comes from a fixed vocabulary; cost is a non-negative bounded integer in fixed monetary units. Unknown or duplicate keys and invalid values are rejected. A valid partial list is allowed, but cannot be dispatched. The requester identity comes from the wrapper, not a model-supplied field. The wrapper displays a canonical structured summary; this application does not display unvalidated model prose.

Confirmation, correction, and policy coverage

Confirmation binds an authenticated principal to both the revision and the exact field list shown to that principal. A term named `confirmed` would establish none of this. Here the accepting transition is:

```
<go to="assess" on="authenticated(Key,Who,confirm(R,Fs))"
    if="confirmed(Key,Who,R,Fs)">
  record_confirmation(Who,R,Fs)
</go>
```

The guard requires the wrapper's secret, the authorised requester, the current revision, and exact equality with the stored summary. The confirmation record retains who

confirmed which snapshot. A correction is accepted by the enclosing pre-dispatch state, including while collecting, awaiting confirmation, or awaiting review:

```
<go to="collect" on="authenticated(Key,Who,correction(R,Text))"
    if="correct(Key,Who,R,Text)">
  apply_correction(Text)
</go>
```

This increments the revision and clears the proposal, pending correlation, confirmation, assessment, and review decision before starting a new model request. Thus even a previously authentic confirmation is stale when it names an old revision. The requester can also cancel before dispatch. Once dispatch starts, that revision is frozen: a correction cannot retract an external effect that may already be under way.

Assessment is total over validated proposals. The example uses the following explicit precedence, rather than two guards that might both fail or both succeed:

```
classify(Fs, insufficient(missing_fields)) :-
    \+ complete(Fs), !.
classify(Fs, conflicting) :- permit(Fs), forbid(Fs), !.
classify(Fs, forbidden) :- forbid(Fs), !.
classify(Fs, permitted) :- permit(Fs), !.
classify(_, insufficient(policy_gap)).
```

Its small policy permits laptop and VPN requests costing at most 1,000 and forbids restricted services and VPN requests. The VPN overlap is deliberate: a request satisfying both rules goes to review, not automatic fulfilment. A pure prohibition is rejected. If neither rule succeeds, lack of policy coverage is recorded explicitly. Missing fields likewise produce an explicit information deficit. The chart cannot become stuck in `assess`. Its `assess/0` entry action retrieves the current fields, calls `classify/2`, and stores the result as `decision(Decision)` for the transition guards:

```
<state id="assess">
  <onentry>assess</onentry>
  <go to="route" if="decision(permitted)"/>
  <go to="failure" if="decision(forbidden)">
    finish(rejected(policy))
  </go>
  <go to="review"/>
</state>
```

Human review is an implemented state. It displays the revision, confirmed fields, and assessment, then awaits an authenticated `review(R,Fs,approve)` or `review(R,Fs,reject)`. Its guards check the reviewer role, current revision, and exact snapshot just as confirmation does. The example explicitly authorises a reviewer to resolve a conflict or a policy gap for a complete request. It does not authorise overriding a pure prohibition or approving missing fields. An incomplete

request must be corrected or rejected; silence ends in review timeout. A correction invalidates any outstanding review decision as well as the requester's confirmation.

Dispatch and terminal outcomes

Entering `route` starts a second asynchronous adapter, but only after checking a current confirmation and either permission or an authorised review approval. The adapter captures its own correlation and the frozen revision. It also receives the idempotency key `key(RequestId,Revision)` and the confirmed payload. The essential acknowledgement path is:

```
<go to="success" on="dispatch_result(Key,C,R,ok(Ticket))"
    if="dispatch_reply(Key,C,R), atom(Ticket)">
  finish(succeeded(Ticket))
</go>
<go to="failure" after="5">
  finish(failed(dispatch_timeout_unknown))
</go>
```

The full chart validates negative acknowledgements too: a known refusal or unavailability ends in `failed(dispatch(Code))`. An invalid acknowledgement has an explicit unknown-outcome result. Both final states publish the recorded outcome and terminate the chart; linked adapter workers are then stopped. Requester cancellation, reviewer rejection, invalid model output, and timeouts also have terminal results.

Dispatch timeout means that no usable acknowledgement arrived, not that the external action did not occur. The companion does not retry automatically. A production backend must durably deduplicate the key and reconcile status before an uncertain operation is retried. Likewise, stopping a local adapter cannot undo an accepted remote operation. These are obligations of the dispatch integration.

The model provides flexibility in interpreting a request; the statechart provides the interaction sequence; and Prolog rules decide which routes are permitted under the represented policy. Replacing the model need not change the confirmation boundary, stopping conditions, or escalation paths.

8.4.6 Open engineering and research questions

The service-request agent illustrates a relatively clean division of labour, but assigning roles to the model, logic program, and control layer does not by itself solve the problems at their boundaries. Evidence must remain traceable as it crosses components, structured representations must retain their meaning, generated explanations must not outrun their symbolic support, and policy checks must mediate every rel-

evant effect. The demonstrator does not solve the following problems; we list them as open questions.

None of this gives an agent "common sense" merely because its rules are written in Prolog. Commonsense reasoning is open-ended, contextual, often defeasible, and sensitive to time and incomplete knowledge. A Prolog program can implement a chosen fragment, but its conclusions inherit the coverage, negation policy, priorities, and consistency of that fragment. Learned components may broaden recall, yet their proposals still require validation when the missing knowledge concerns a safety or policy obligation.

- **Tracing and audit.** Symbolic execution can produce proof objects, constraint stores, and, where explicitly captured, search traces. Model interaction produces a different kind of record: prompts, tool calls, model and version identifiers, sampling settings, and returned outputs. Correlating these records across actors and nodes into a coherent audit trail remains a practical challenge and an opportunity for web-native agent engineering.
- **Proof artefacts as user-facing explanations.** Proof trees and derivation traces are precise but not always pleasant to read. An LLM could serve as a commentary layer that renders them as structured explanations. The generated explanation should remain linked to the proof fragments that support it and must not be treated as establishing more than the underlying derivation establishes. Minimal justifications and counterfactual analyses may also require additional symbolic computation rather than language generation alone.
- **Intermediate representations.** Stable, versioned schemas at the boundary between neural and symbolic modules reduce dependence on brittle prompt conventions. JSON provides a convenient encoding, but not by itself a semantic contract; validation rules, identifiers, and provenance conventions must also be specified. JSON-LD may help where linked identities and web-visible provenance matter.
- **Safety envelopes.** When an agent must enforce policy, the authoritative checks need to remain outside the model and execute before the corresponding effects. In Web Prolog they can reside in Prolog rules and in the supervising wrapper actor. The harder problem is ensuring complete mediation: indirect effects, delegated tool calls, retries, and actions performed by other agents must not bypass the same policy boundary.

8.5 User-interface agents

The Agentic Prolog Web is a substrate for multi-agent interaction in general, but an important class of agents mediates between a person and a computational environment. For these user-interface agents, the engineering problem is not only what can be inferred, but how an interaction is managed over time: when to ask, when to confirm, how to repair misunderstandings, how to handle silence and timeouts, and when to refuse or escalate.

The social-robot "parrot" of Section 3.5.6 can now be given a brain without changing its transport architecture. The robot node may continue to own speech recognition, synthesis, gaze, and embodiment, while a remote brain actor owns the dialogue governor, symbolic knowledge, and any model wrapper. Utterances, interpretations, and action requests cross the boundary as messages. Timing and effects stay with the robot; the dialogue policy can be inspected and replaced on its own.

This section therefore shifts emphasis from agent-to-agent coordination to dialogue as control. Conversational behaviour should be represented as a program with states, transitions, and failure handling rather than left as an emergent property of a prompt. Web Prolog can combine dialogue-relevant facts, executable parsing, and an actor-oriented statechart controller.

8.5.1 UI agents versus MAS agents

In classic multi-agent systems, interaction is often modelled as peer-to-peer exchange: negotiation, coordination, and distributed problem solving among relatively symmetric participants. User-interface agents are asymmetric. They exist to serve a person, and they are evaluated by human-facing criteria such as clarity, trust, timing, politeness, and error recovery. The success condition is not only that the agent eventually produces a correct plan, but that it maintains an intelligible and safe interaction while doing so.

On the Web, this role frequently manifests as a conversational web agent: an agent that accepts input in natural language (text or speech) and produces responses that guide the user through a task. In such systems, the boundary between reasoning and interaction is not clean. Questions, confirmations, clarifications, and refusals are themselves action choices that must be governed by policy.

8.5.2 Turn-taking and repair as control structure

Conversation, in the technical sense used in linguistics, is a jointly managed activity. Turn taking is locally coordinated (Sacks et al., 1974); questions and answers, offers and acceptances, and requests and responses create expectations about what may come next. When understanding is uncertain, participants use repair and grounding moves to establish enough common ground for the current purpose (Clark and Brennan, 1991).

For an engineered UI agent, the practical implication is that interaction cannot be reduced to a single request–response call. The agent needs persistent state and control over what may happen next. It must be able to suspend a task to ask a question, resume after an answer, back up after a misunderstanding, and time out or escalate when interaction fails. Statecharts are designed for exactly this.

A useful mental model separates *flow* from *content*. Flow governs when the system may advance, when it must confirm, what to do on silence, and how to recover after misunderstanding. Content covers parsing, data-model updates, and reasoning. Both can be expressed as programs, but a statechart makes the permitted interaction sequence particularly visible.

8.5.3 Dialogue as proof: the missing-axioms view

A useful way to connect dialogue management to symbolic AI is the Missing Axioms Theory (Smith et al., 1995). In this view, the agent's task can be phrased as an attempt to prove a goal. When the proof fails, the reason is often not that the rules are wrong, but that some required facts are missing. Dialogue then becomes a controlled process for acquiring the missing axioms.

This perspective is already latent in the askable expert-system pattern from Section 8.2.4 (and, in web-native form, Section 8.3.1). There, proof search determines which question must be asked next. The toy meta-interpreter has the right idea but cannot handle what real dialogue involves: interruptions, repairs, topic shifts and timeouts. To handle those robustly, we need an interaction controller with persistent state and well-defined failure handling.

8.5.4 Statecharts as dialogue managers

In Web Prolog, a statechart actor can serve as the authoritative dialogue manager, while Prolog code in its data model supplies parsing and reasoning. The following small statechart illustrates the arrangement. It collects two attributes and uses a history state to recover after a misunderstanding:

```
<statechart version="0.2" initial="Start">
  <datamodel>
    :- dynamic color/1, size/1.
    np(Attr) --> [a], adj(Attr), [apple].
    adj(color(red)) --> [red].
    adj(size(big))  --> [big].
  </datamodel>

  <state id="Start" initial="Color">
    <history id="H">
      <go to="Color"/>
    </history>
    <state id="Color">
      <onentry>
        writeln('What color do you want?')
```

```
      </onentry>
      <go to="Size"
          on="input(Input)"
          if="ground(Input), phrase(np(color(C)), Input)">
        assert(color(C))
      </go>
    </state>
    <state id="Size">
      <onentry>
        writeln('What size do you want?')
      </onentry>
      <go to="Final"
          on="input(Input)"
          if="ground(Input), phrase(np(size(S)), Input)">
        assert(size(S))
      </go>
    </state>
    <go to="H" on="input(_)">
      writeln('I did not understand.')
    </go>
  </state>
  <final id="Final">
    <onentry>
      size(S), color(C),
      format('You want a ~p ~p apple!~n', [S,C])
    </onentry>
  </final>
</statechart>
```

The `ground/1` guards require an actual utterance: an unbound input must not let the reversible grammar generate one and advance the dialogue. Three elements matter here.

Repair via history

The history state supports a simple repair policy for unrecognised input. When an `input(_)` event fails to match any more specific transition, the machine emits a complaint and transitions to the history pseudostate. This re-enters the most recently active substate of the dialogue, causing its `<onentry>` prompt to run again and thereby re-asking the current question. In contrast to restarting the whole interaction, the repair stays within the current plan, resembling how human repair sequences typically preserve the larger conversational trajectory.

Parsing as a guard

Transition guards call DCG parsing directly, so the controller advances only when the incoming `input(Input)` can be parsed into the intended structured form. This makes the interface between interpretation and control explicit: the grammar determines what counts as understood, while the statechart determines when understanding is sufficient to commit to a control move.

Data-model updates as explicit effects

Executable content makes data-model updates explicit and localised. When an input is successfully parsed, the extracted fact is asserted as a named predicate in the statechart's data model. These stored facts can later be queried, audited, and constrained by ordinary Prolog rules rather than remaining implicit in conversational context.

Together, history-based repair, guard-based interpretation, and explicit data-model updates yield a compact pattern: recognise → commit → store; otherwise complain → repeat.

8.5.5 Dialogue as a governed interface

The same statechart harness that governs tool use can govern what an agent may say and do with a person. A model may be invoked for paraphrasing, interpretation, or question generation, but its output re-enters the system through guards, schemas, and policies. The chart thereby preserves the observable interaction pattern – turn taking, repair, refusal, and escalation – even if the model is replaced.

8.6 Summary

This chapter framed Prolog's relevance to contemporary AI as an operational question: can symbolic constraints, search, and explanations be made web-addressable and composable, so that they can participate in agentic systems? We first recalled Prolog's symbolic competence on a single machine: knowledge as executable clauses, planning as search, proof objects, interactive acquisition of missing facts, and grammar-based language processing. We then lifted these idioms onto the Agentic Web, treating Prolog engines as networked actors and allowing proofs and explanations to span nodes.

On that foundation, we distinguished the book's architectural agents from classical cognitive agents and contemporary LLM agents. Hybrid systems must assign authority as well as work. The Jev experiment made the observation boundary concrete: a learned service supplied typed probabilistic decisions, the Trinity layer

treated them as semantic observations, Prolog retained their evidence, and SXML determined their state-dependent behavioural force. The service-request example extended the pattern to authenticated confirmation, policy, and uncertain external effects. Potentially blocking inference belongs in monitored wrapper actors; completed observations enter the statechart between macrosteps.

Finally, user-interface agents showed the same control structure from the human-facing side. Turn taking, repair, confirmation, timeouts, refusals, and escalation can be represented as executable interaction policy rather than left implicit in a prompt.

Web Prolog does not supply intelligence by itself. What it supplies is a place where state, permitted moves, failures and reasons are written down and executed.

Part III
Synthesis and outlook

Chapter 9
The Trinity as a web-native symbolic paradigm

The preceding chapters developed the three components of the Prolog Trinity: Web Prolog as a language profile, actors and nodes as computational agents, and the Prolog Web as the network over which they interact. They also introduced reusable behaviours, statechart actors, Semantic Web interoperability, and applications to AI.

Taken in isolation, none of these ideas is entirely new. Web Prolog extends established Prolog practice; its actor model draws on Erlang; remote querying has a long history; and statecharts extend conventional state machines. The interest lies in how these elements fit together.

Chapter 1 presented the two computational partitions, the affinities between Prolog and the Web, and the mutual dependence of the Trinity's components as motivations for the project. Having now developed them in detail, we can ask what programming model emerges from their integration. This chapter argues that the result is a coherent architectural stance – a candidate paradigm for executable symbolic systems on the Web.

Here, *paradigm* is used in the ordinary programming-language sense, not to claim a Kuhnian revolution. Chapter 2 used Van Roy's taxonomy (Van Roy, 2009) to locate Web Prolog among relational and logic programming, imperative search programming, and message-passing concurrency. The claim made here is that adding nodes, node-qualified process identities, profiles, and remote nondeterministic calls produces more than a collection of facilities: web distribution becomes part of the computation model, and the inherited and added concepts constrain one another strongly enough to deserve consideration as a further point in that design space.

Our answer consists of six commitments: a separation between sequential and concurrent computation; portability grounded in observable interaction; agents as the primary units of structure; a logical foundation that survives distribution; distribution as part of the programming model; and interaction governance as a first-class design dimension.

The commitments constrain one another (Section 9.3.2).

9.1 The two partitions as a foundational principle

Chapter 4 defined the sequential and concurrent partitions over observable interaction contracts. Sequential contracts present goal sessions whose answers, committed effects, prompts, and errors remain intelligible without exposing process interleavings. Concurrent contracts make independently progressing processes, asynchronous messages, mailbox selection, and lifecycle events part of what programs may observe. Constructs, programs, nodes, and profiles are classified only derivatively from the contracts they expose.

What matters is that the partition is independent of other design choices. Purity is separate: a sequential ISOTOPE session may update its private database or perform I/O, while a pure remote call can retain a logical reading. Transport and statefulness are separate: actors can implement a stateless RPC surface, and a stateful session can remain sequential. Implementation is separate for the same reason. Openness introduces yet another axis, since public execution surfaces create security and resource-governance obligations regardless of which transport carries them.

The profile hierarchy gives this independence an operational form. RELATION, ISOBASE, and ISOTOPE expose only sequential contracts, although ISOBASE adds the split-phase `promise/yield` interface and ISOTOPE adds session state. ACTOR is the first profile that also makes the concurrent partition client-expressible, while retaining the sequential surfaces inside it. The hierarchy therefore broadens the observable contract without making purity, state, or transport stand in for the partition boundary.

The boundary case `parallel/1` makes the same point from another direction. Its predicate surface is synchronous, but when several workers produce abnormal outcomes its first-observed result exposes scheduling. Thus synchrony alone does not settle the contract's partition; the observable multi-fault rule matters too.

Real systems will commonly mix the two: pure queries for declarative reasoning and actors for coordination, state, and lifecycle management. A useful discipline is incremental relaxation – begin with the purest model that fits the problem, and introduce procedural commitments where the application requires them.

This programming-model orthogonality is compatible with the architectural mutual constraint developed in Section 9.3.2. The former separates two classes of interaction contract; the latter says that the language profile, agent model, and network substrate must be designed together. Keeping the partitions separate depends on a disciplined interface between Prolog engine state and actor process state.

9.1.1 Two levels of abstraction

Because the sequential interface can be built from actors (Section 4.2), the Prolog Web can be read at two levels of abstraction. On the upper level lies a *level of logic programs* – interlinked Prolog clauses distributed across nodes, where URIs serve as references between knowledge bases and `rpc/2-3` calls serve as the operational

links. On the lower level lies a *level of communicating processes* – often realised as toplevel actors that are created, exchange messages, and eventually terminate.

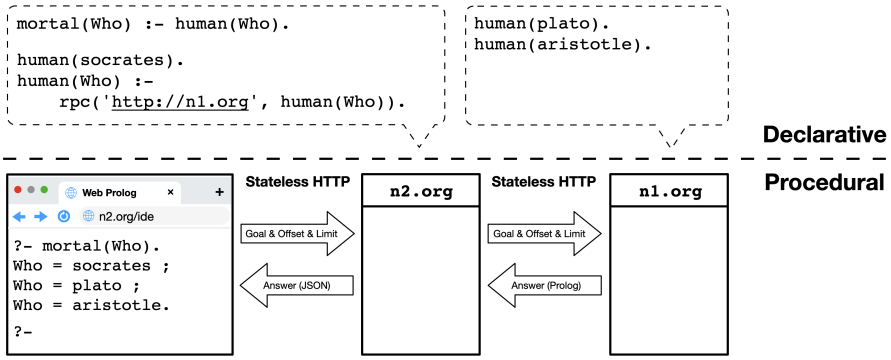


Fig. 9.1 Two levels of abstraction: interlinked logic programs above the line and communicating processes below it.

Figure 9.1 revisits the scenario of Figure 4.4, now drawing the distinction between the two levels explicitly. On the lower level, processes are created and destroyed; while alive, they send messages to each other. Nothing of this kind takes place on the level of logic programs. Entities such as processes, messages, and mailboxes do not exist there – they have been abstracted away. Drawing a line between these two levels is another way to express the distinction between declarative and procedural that so often appears in the story of Prolog. "What?" is specified at the level of logic and the corresponding "How?" is handled on the level of communicating toplevel processes.

When the sequential partition is pure, the Prolog Web can thus be read in two ways: *logically*, as inference over distributed knowledge bases, or *procedurally*, as message exchange among processes. When actors and side effects are essential, only the procedural reading remains. Otherwise the programmer can move between the two readings, even though the computation is distributed.

9.1.2 Unification as the universal data contract

The interface between client and node needs a structural contract. In many Web APIs, request schemas, response schemas, and the code that relates them are separate artefacts.

In Prolog, unification makes the coupling direct. A goal sent to a node is simultaneously a request for information, a specification of the expected structure of answers, and the compatibility test between the two. The query

```
?- employee(Name, Dept, salary(Min, Max)).
```

both asks for employees and states the expected answer structure. If a matching fact exists, bindings flow back. If `employee/3` is undefined, the Web Prolog contract reports an existence error at the point of invocation. The query, as Chapter 1 put it, *is* the contract.

The contract must distinguish three cases. A relation that has changed arity or disappeared should raise an error; a goal that fails to terminate may surface as a timeout; only the genuine absence of matching data should yield failure. This three-way distinction must be part of the node contract rather than left to a host Prolog's configurable treatment of undefined predicates; Section 9.2.1 below returns to this example.

Composition is native: constraints combine by conjunction, for example

```
?- employee(Name, engineering, salary(_, Max)), Max > 100000.
```

The combined query can be executed entirely on the node or split between remote and local evaluation. Because a Prolog predicate is relational, the instantiation pattern chosen by the client can provide different access modes through the same contract. Projection is expressed by choosing which variables the caller retains, rather than by defining a new response format. Evolution is also visible at the call boundary: if a relation changes name or arity, an old client receives a specific error instead of silently accepting a structurally unrelated response.

The contract is structural, not semantic. Unification can ensure that terms have compatible shapes and that shared variables satisfy the same equality constraints, but it cannot determine whether two parties attach the same meaning to a predicate or to its arguments. That agreement must still come from documentation, a shared vocabulary, or a shared ontology. Chapter 7 develops the complementary division of labour: unification disciplines structure at the point of call, while Semantic Web vocabularies and ontologies make intended meaning explicit.

9.1.3 Backtracking as paging

As Section 4.2.4 explained, Prolog backtracking and web paging both support incremental enumeration under the caller's control.

Within a slice, the caller enumerates solutions locally. Only when that slice is exhausted must it cross the network boundary to request another. The protocol can therefore expose nondeterminism incrementally without turning every individual solution into a separate remote call.

Chapter 4 developed the operational alternatives, the per-invocation stability supplied by the logical update view, and the additional mechanisms required for a point-in-time snapshot. Here the important alignment is that unification supplies the data contract, backtracking the enumeration model, and HTTP or WebSockets the transport. More generally, terms, clauses, goals, and substitutions give knowledge representation, communication, and computation a shared symbolic form.

9.2 From compatibility to interaction portability

The Prolog community has long understood portability as a matter of *compatibility*: the extent to which different systems happen to accept the same programs and produce similar results. Efforts such as the ISO Prolog standard have addressed this at the level of the core language. Yet, as surveyed in *Fifty Years of Prolog and Beyond* (Körner et al., 2022), portability issues remain widespread beyond the core – in libraries, constraint systems, concurrency models, and interfaces to the outside world.

Compatibility, even when it covers both syntax and behaviour, is an *empirical* property: it depends on the contingent alignment of implementations, and it is easily broken by innovation. It asks whether two systems happen to agree, not whether they are obliged to. The Prolog Web sharpens the problem further: in a network of nodes implemented on top of different Prolog systems, the question is no longer just whether two systems accept the same program and produce the same answers, but whether they preserve the same observable *interactions* – the same patterns of queries, answers, and message exchanges as seen by clients and peer agents.

In this work, we adopt a different perspective. Rather than treating portability as empirical compatibility between systems, we treat it as a designed property of programs, defined over their observable interactions.

> **Interaction portability.** A program is portable across a declared class of execution environments, under stated external dependencies and observations, when relocation preserves its permitted observable interaction behaviours – the relevant queries, answers, effects, messages, and lifecycle events.

Preservation here means observational equivalence, not identical wall-clock timing or reproduction of one fixed trace. A nondeterministic program may admit several answer orders or message interleavings; relocation is portable when it preserves the permitted set rather than selecting one particular trace.

The service-request agent in Section 8.4.5 makes the question concrete. If its request actor moves to another runtime, what must remain true? A confirmation must still concern the same revision and field snapshot, an old reply must not advance a corrected request, and dispatch must remain subject to the same policy. If an acknowledgement is lost, relocation must not turn an unknown external outcome into an assertion that nothing happened. The relevant observations therefore include which effects may be authorised and what the caller can learn about them, as well as returned terms.

This does not require the same pid or elapsed time on both runtimes. It does require a declared treatment of identity, timing, and dependencies: a different policy source or trust boundary is not merely a different placement. Running the local example in two environments supplies observations under its test conditions; it does not establish preservation across all admissible interactions. The example helps specify the obligation rather than discharge it.

This shift has three consequences. First, portability becomes relative to a well-defined class of environments: a design decision, not an accident of history. Second,

interaction rather than textual acceptance becomes central: two systems may accept the same program and yet differ in observable interaction behaviour, and under this notion of portability such differences determine whether portability holds. Third, portability must be *designed* rather than inferred: it requires explicit control over the aspects of execution that affect observable interactions.[1]

The Trinity provides the ingredients from which this stronger notion can be built. Web Prolog restricts the client-supplied fragment so that self-contained programs need not depend on uncontrolled host features; owner-defined predicates and optional libraries remain explicit dependencies. Agents and nodes structure execution around message passing and explicit interaction, making behaviour visible and comparable across environments. Node profiles then provide the first reference points relative to which interaction portability can be assessed.

The same principle applies to substitution as well as relocation. The statechart dialogue governors of Chapter 8, for example, constrain the observable sequence of turns, confirmations, repairs, and refusals while the language model behind them may be replaced or re-prompted. Node profiles support portability across execution environments; dialogue governors seek a related invariance across substitution of a fallible component. In both cases, the stable object is the declared interaction contract rather than the implementation that realises it.

9.2.1 Node profiles as interaction contracts

Returning to the primary axis – relocation across execution environments – the notion of interaction portability is meaningful only relative to a well-defined class of such environments, and those classes must therefore be made explicit. Chapter 3 introduced node profiles as conformance declarations – specifications of what language features and web APIs a node provides. Nodes bearing the same profile may still differ in their data, libraries, policies, and owner-defined predicates. The profile delimits the observable surface on which a stronger interaction contract can be stated and tested.

What varies across profiles is therefore both what a node can do and which aspects of its behaviour become relevant to portability. At each level the observable surface grows wider:

- **RELATION.** Observable interaction consists of answer substitutions delivered through the paged stateless API. Each node must provide an order stable enough for its own pages to be meaningful, as described in Section 3.3.3, but the profile specifies no canonical order shared by independent backends. Portability between two RELATION nodes therefore requires equivalent advertised relations and answers, including multiplicity and any ordering observed by the program; pages from different nodes cannot in general be spliced into one enumeration.

[1] The distinction drawn here has a parallel in the distinction between the meaning of expressions and meaning realised through their use in interaction.

- **ISOBASE.** Observable interaction extends to the evaluation of goals in the ISOBASE fragment, including Prolog answer order, control constructs, and synchronous and asynchronous RPC. Client-supplied source may accompany a request, with scope limited to its temporary execution context. Incremental delivery is already present at RELATION; what ISOBASE adds is a language-level basis for asking whether answer order and continuation behaviour agree. Such agreement is required for a particular program to be interaction-portable, but it does not follow from the profile alone when owner code, libraries, or policies differ.
- **ISOTOPE.** Observable interaction extends to side effects within a persistent toplevel session: supplied definitions and dynamic database updates can survive across requests, and the session supports interactive I/O. The extension is persistent session scope; supplied source is already available at ISOBASE with temporary scope. Portability must therefore cover not merely answers and their order, but also the effects produced along the way and the state retained between requests. This is the most demanding level within the sequential partition.
- **ACTOR.** Observable interaction extends to message exchanges, mailbox semantics, and process lifecycle. Portability here depends on the delivery guarantees, the selection semantics of `receive/1-2` (Section 2.3.4), the failure-notification vocabulary, and the lifetime discipline for spawned processes. These are the parts of the actor contract that must be pinned down independently of any one implementation.

The profiles thus form a hierarchy of increasingly rich observable surfaces, and interaction portability becomes more demanding as one climbs it. What may be sufficient for a RELATION program – agreement on answer streams and paging behaviour – is not sufficient for an ACTOR program, which also requires agreement on the dynamics of interaction itself. Portability then becomes a design principle: programs are written against a profile, not an implementation, and systems may change internally as long as they keep the contract. The current profiles are a first step; turning these requirements into testable guarantees is future work (Chapter 11). As shown in Section 4.3.3, when a program is self-contained and the target environment implements the required profile, relocation may require no adaptation. Programs that rely on owner resources, optional libraries, or particular service semantics must carry those dependencies explicitly.

Profiles must fix the latitude

Interface agreement must reach further than a common list of predicate names. ISO Prolog deliberately leaves some behaviour implementation-defined, and systems also differ in areas outside the standard. Two conforming systems may therefore disagree about matters that become observable at a network boundary: what happens when an undefined predicate is called, what a double-quoted literal denotes, which errors are raised, or how numeric limits are handled.

A Web Prolog profile must settle every such choice that affects its interaction contract. In this sense, a profile is partly a decision over the latitude left by the host language: it turns implementation-defined behaviour into profile-defined behaviour for the fragment it covers. The choice should follow from the contract, not merely from an implementation's default. For example, the ISO `unknown` flag permits different treatments of an undefined predicate. The Trinity's distinction between an error for a missing relation, failure for the absence of a solution, and timeout for a computation that does not answer requires the profile-equivalent of `unknown = error`, rather than silent failure. Similarly, portable source loading requires the meaning of double-quoted text to be fixed rather than assumed. The current SWI-Prolog demonstrator uses `double_quotes = string` in goals, answers and loaded source, so literals denote SWI-Prolog's native string values. This is an implementation convention; a common portable text contract remains unresolved. The GNU Prolog and Trealla prototypes provide evidence for selected interactions, without establishing agreement on string-sensitive operations. The opt-in `portable-text-0.1` draft instead proposes character-list semantics for double-quoted literals. It remains experimental and is not implied by advertising one of the four cumulative node profiles. Section 11.1 sets out the work required before adopting a common rule.

The design requirement is firm even where the final setting still belongs to standardisation. The present demonstrator's profile inventory primarily specifies routes, predicates, source-loading facilities, and actor behaviour; a standards-quality profile must additionally enumerate these language-level choices and test them. Conformance is consequently not a declaration of ancestry or intent but a claim about observable behaviour, checked by a suite shared among implementations. Such a suite compares the terms a client decodes and the categories of the errors it receives, not printed text: a canonical way of writing terms is how an implementation meets the contract, not the contract itself. This is how a narrow RELATION surface and a much richer ACTOR surface can both be precise without requiring the engines behind them to be alike.

9.3 Agents as the primary abstraction

At the centre stands the agent. As in Chapter 3, we use the word architecturally. Actors, toplevels, statechart actors and nodes are all agents; a node, for example, owns resources, hosts knowledge and services, advertises a profile and enforces policy. External services, browsers and AI components are agents too when they interact through defined boundaries.

The Trinity consequently adopts a process-and-message discipline in which identity, mailbox semantics, and lifecycle policy provide the main structure. Logical reasoning occurs *within* agents, distribution occurs *between* agents, behaviours constrain classes of agents, and statecharts govern reactive control.

The choice has far-reaching implications. It aligns naturally with distribution-first semantics: because the agent is already a process with a mailbox, the step from

local to distributed execution does not require a change of abstraction. It supports fault containment and explicit failure observation through links and monitors. It also allows sequential reasoning and reactive control to coexist inside one application without conflating their interfaces.

Once nodes host durable processes that interact over published protocols, the Prolog Web is a multi-agent system in this architectural sense: global behaviour emerges from locally scoped programs coordinating under failure and timing constraints.

We saw in Section 2.2.1 that Armstrong characterised the Erlang approach as structuring applications from large numbers of communicating parallel processes – an architectural infrastructure of well-defined, independently implementable subcomponents. The same principle applies to the Prolog Web.

9.3.1 Interaction as inference

The agent-centric and logical commitments of the Trinity meet when an agent decides how to react. An incoming message contributes a partial substitution by one-sided matching with a receive pattern or by unification with a statechart transition pattern. A logical guard may then extend that substitution by proving a goal within the agent's *logical environment*. Selection therefore produces not only a decision, but also information that can flow into the ensuing action. In this sense, a mailbox message is a transient premise rather than the complete input to a dispatch function.

The construction also connects the two partitions of Web Prolog. Receiving a message is an effect in the concurrent partition: a message is selected and consumed under an ordering discipline. Determining whether the message enables a receive clause or statechart transition invokes the sequential partition: a Prolog goal is proved within the agent's logical environment. The outcome of that proof then determines the next concurrent action.

Logical guards are consequently a small but concrete instance of the Trinity's central claim: symbolic reasoning need not sit beside interaction as a separate service, but can participate directly in deciding what an interaction means. The operational semantics are given in Section 2.3.4; Section 6.5.1 shows their reuse in statecharts.

9.3.2 Mutual constraint

The three components of the Trinity are mutually constraining – a property more characteristic of a paradigm than a toolkit. The language profile constrains the agent model: restricting remotely supplied code narrows what agents must defend against and which operations they may expose. The agent model constrains the network substrate: identities, mailboxes, sessions, and lifecycles require naming, message transport, and failure notification. The network substrate constrains the language

profile in return: goals, answers, messages, and code fragments must have portable representations and well-defined behaviour at transport and failure boundaries.

No single component can be redesigned independently without affecting the others. This is why the Trinity is not well described as "Prolog plus actors plus HTTP": the components shape each other.

For example, suppose a node admitted an unrestricted file-opening predicate into remotely supplied goals. The language profile would no longer provide inexpressibility as a sandbox boundary; actor containment alone could not prevent a client from reading node-owned files; and the HTTP and WebSocket bindings would need a substantially stronger authorisation and audit model. Inexpressibility is exact when the profile is stated as a whitelist of admitted predicates, which should be the default for a public node (Section 4.5.1). Conversely, retaining a portable profile without file I/O lets every transport and actor context rely on the same structural restriction. A change in one corner therefore changes the obligations of the other two.

9.3.3 Identity, correlation, and authority

One consequence of this mutual shaping is the security model. Because the language profile, the agent model, and the network substrate constrain each other, part of the security posture can be structural rather than imposed only by external checks.

The structural mechanisms that make the Prolog Web safe to open – inexpressibility, invisibility, and containment – are developed in Section 4.5. They support three distinct contracts. Identity names an actor: a pid is publicly constructible, unique for its node's lifetime, and never reused; its entropy is defence in depth, not authorisation. Correlation uses fresh, single-use references to match replies and monitor events; it does not protect against a party that legitimately holds the reference. Authority comes from enforced policy. The language permits sending subject to node policy, monitoring has the same target policy, termination is restricted to the caller's ownership tree, and monitor removal belongs to its installer. Public node URIs and published service names are discoverable addresses governed by access policy. Pids are addresses, not capabilities. Inexpressibility is enforced at the language boundary: dangerous operations cannot be named in the client fragment. Nondisclosure of a pid narrows the set of actors a client can address, but does not turn the pid into authentication. Containment is enforced by lifetime dependency: its linked actor subtree is reclaimed once parent termination or loss of the owning connection is detected. A transport lease bounds detection of lost contact while the runtime services the connection; it does not establish that remote work has stopped. These mechanisms reduce what scattered runtime checks must accomplish, but they complement rather than replace authentication, authorisation, resource governance, auditing, and transport security.

This structural character aligns the Prolog Web with a family of contemporary execution environments – browser-based JavaScript, WebAssembly, WASI – that share the same posture: code executes inside a bounded environment and acquires authority only through explicitly provided references. Traditional Prolog systems are

designed around maximal expressiveness; Web Prolog inverts the default, treating expressive power as owner-controlled authority that can be selectively granted rather than silently assumed.

9.3.4 Five complementary views

The agent-centric design refracts into several complementary perspectives, each emphasising different linked entities and activities:

The Prolog Web of source code. URIs link programs to programs. Nodes may expose source code over HTTP so that users and tools can follow those links.

The Prolog Web of query solutions. URIs link callers to answer streams. Interaction is sequential and pull-based: a client asks a node to solve a goal and enumerates its solutions incrementally.

The Prolog Web of actors. Node-qualified pids link live processes with mailboxes. Interaction is asynchronous, while links and monitors expose lifecycle dependencies and failure.

The Prolog Web of behaviours. Behaviours specify message syntax, semantics, and timing; actors are their running instances.

The Prolog Web of nodes. Nodes are the places of the system. They store and serve code, host actors, enforce policy, advertise profiles and services, and provide points of execution and accountability.

9.4 The logical foundation: purity, belief, and scoped inference

Logic programming is essential to the Trinity, and the notion of a *pure Prolog Web* shows why.

9.4.1 Pure Web Prolog and the pure Prolog Web

As noted in Chapter 4, `rpc/2-3` retains the logical purity of the predicates it calls. This allows us to form the notion of *pure Web Prolog*:

$$\textit{Pure Web Prolog} = \textit{Pure Prolog} + \textit{rpc/2-3}$$

This equation assumes that the called predicates and their logical context are pure and stable for the duration of the call. Timeouts and connection failures may truncate the answer stream, but do not change the logical status of answers already returned. The union of such fragments forms what we call the *pure Prolog Web*: an aspirational Web of Logic whose distributed answers remain consequences of pure programs.

Fig. 9.2 A pure Prolog Web spanning the globe remains aspirational, but the architecture does not preclude it.

9.4.2 Epistemic reading and scoped inference

If agents are the primary abstraction and the pure Prolog Web gives them a logical character, then a call such as `rpc(URI,Q)` admits an optional epistemic reading: it asks whether the logical environment at `URI` supports `Q`. A clause such as

```
human(X) :- rpc('http://n1.org', human(X)).
```

can, using a belief operator – equivalently, a predicate over reified formulas – be formulated as

$$\forall x[believes(n_1, human(x)) \rightarrow human(x)]$$

It makes perfect sense to say that one agent believes that `human(socrates)` is true while another does not:

```
?- rpc('http://n2.org', human(socrates)).
true.
?- rpc('http://n1.org', human(socrates)).
false.
?-
```

And the following query asks whether two agents *agree* that `human(X)`:

```
?- rpc('http://n1.org', human(X)),
   rpc('http://n2.org', human(X)).
X = plato ;
X = aristotle.
?-
```

The epistemic reading is not the only one available. More conservatively, the URI can be understood as specifying the *scope* of inference – the knowledge base with respect to which a query is to be evaluated. As argued in (Kifer et al., 2005), scoped inference is mandatory for realising Negation As Failure on the Web, because applying any form of the closed world assumption requires knowing the entire knowledge base first – something that is not possible for the Web since its boundaries cannot be clearly delineated. A third and more practical view is simply that `rpc/2-3` invokes a goal in a remote logical context whose predicates remain distinct from those of the caller. For pure relations on suitably aligned nodes, the three views – epistemic operator, scoped inference, remote context – can coincide operationally, although they remain conceptually distinct.

What about `rpc/3`? Code injection via `src_*` options does not jeopardise purity provided the final composition of the program executed is itself pure. The `limit` option is a *pragma*: it specifies the granularity of the conversation between client and node but has no effect on the meaning of the query.

9.4.3 The less pure Prolog Web

Not all of the Prolog Web is pure. ISOBASE nodes may use control constructs such as cut, and ISOTOPE nodes go further with side-effecting predicates, I/O, and dynamic database updates. The stateless HTTP API cannot preserve such updates between independent requests; stateful sessions and actors can.

More fundamentally, the concurrent Prolog Web – the world of the ACTOR profile – is inherently procedural. Actor-based programming involves process creation and destruction, message ordering, selective receive, and failure propagation – none of which have declarative readings. A realistic application will typically mix profiles freely, using pure queries for declarative reasoning and actors for coordination and state.

The Trinity does not force a choice between declarative and procedural, a tension as old as Prolog. It makes the boundary visible, so that the programmer knows which parts of a system have a logical reading.

9.5 The distribution-first model

Within the concurrent partition, Web Prolog adopts a distribution-first view. The same actor primitives apply locally and remotely, with local placement as the simpler case. Remote placement additionally introduces serialisation, latency, authentication boundaries, disconnects, and partial failure. The sequential partition remains distinct: it offers query evaluation rather than actor interaction, even when an actor-based protocol implements it underneath.

One useful starting point for formalising this stance is Svensson et al. (2010), who proposed a unified semantics for Erlang-like systems. Web Prolog draws two related design moves from that work. First, local and remote messages travel through a single conceptual mechanism, while a node controller handles side-effecting operations uniformly. Second, links are unidirectional rather than symmetric; their supervisor example makes the child's lifetime depend on the supervisor's, which is the orientation adopted by Web Prolog's spawn hierarchy. In their abstract account, placement can be treated as a configuration detail rather than as a split between two languages, even though it remains operationally observable. The separation of node controllers from process semantics offers Web Prolog a relevant model, although a Web Prolog semantics must additionally account for Prolog engine state, backtracking, the boundary between the two partitions, and the open-Web containment policy layered on the one-way link relation.

A useful way to read the distribution-first stance is through the classic goal of *distribution transparency*: making the distribution of processes and resources invisible to applications. In the Prolog Web, the *same* operations – such as spawning, sending and monitoring – apply locally and remotely. Latency, serialisation, security boundaries, and failure modes change with distance, but the programming model does not bifurcate into a "local" and a "remote" language. The profile hierarchy identifies the language features and protocol surfaces available across the boundary; precise conformance rules must say which observable guarantees travel with them. Distribution-first means uniform primitives with visible operational differences.

The combination of Prolog's execution model with this actor semantics suggests a modular route to a formal account. Conceptually, each process has both an actor component – including its mailbox and lifecycle state – and a *Prolog engine state* consisting of a goal stack and a current substitution. Most rules respect that separation, but the components meet at the actor primitives. A send constructs a message from the current substitution; a receive removes a selected message and extends the substitution through one-sided pattern matching and any successful guard; spawning, monitoring, and termination likewise connect goal execution with process lifecycle. The crucial discipline is that these crossings are committed: backtracking never unsends a message or returns a selected message to the mailbox. Backtracking therefore remains local to an actor's engine even though the two state components interact. A formal semantics, with a proof of composability, remains future work (Chapter 11).

9.5.1 The browser as a Web Prolog runtime

The distribution-first model extends naturally to the client side. A browser equipped with a Web Prolog runtime hosts what Chapter 3 called a *browser node*: a node in architectural terms, but not a public endpoint. It accepts no incoming network connections and therefore participates as a client and as a host of local actors. It can nevertheless use the same interaction primitives available to server-side code. The

programmer should be able to use synchronous calls when blocking is acceptable, non-blocking request–response calls when responsiveness matters, and actor-style interaction for cases that genuinely require long-lived conversational state – all without being exposed to transport-level details.

The longer-term ambition suggested by this design is Prolog on both sides of the Web. Not all clients need to use Prolog, however. Many developers will continue to prefer writing front-ends in JavaScript and communicating with Prolog back-ends through explicit web APIs. The Prolog Web should make such APIs visible, well-specified, and robust. A Prolog-everywhere experience may make them largely invisible in the common case, but engineering robustness requires that they remain accessible.

9.6 Control as a first-class dimension

Consider the famous equation coined by Robert Kowalski (Kowalski, 1979):

Algorithm = Logic + Control

The kind of control we have in mind here is different from search control. Conversational and reactive systems are hard not primarily because they require sophisticated algorithms, but because they require fine-grained *interaction control* – the kind that governs turn-taking, orchestration, and long-lived dialogue flow. In the Trinity, this motivates the combination of three programming models: logic programming for knowledge and inference; actor-based programming for concurrency, distribution, and interaction; and statecharts for explicit, inspectable control structure. The three are complementary: each addresses a dimension of system design that the others leave open, while – as we show below – remaining tightly coupled to them.

9.6.1 Governance of interaction

If the Trinity were only a story about distributed queries and actor concurrency, it would leave an important practical question unanswered: how does one *govern* complex interaction so that it remains inspectable, auditable, and robust? Protocol sequencing, timeouts, retries, cancellation, interruption, recovery, and exceptional cases can all be encoded by actors, but their structure is then dispersed through the actor loop.

Statechart actors are the Trinity's response. They do not replace actor programs; they offer a structure for reactive behaviour in which states, transitions, events, timers, guards and recovery paths are explicit and can be reviewed.

Control participates in the same mutual constraint as the rest of the architecture. Statechart guards, actions, and data-model code are Web Prolog goals, so there is

no second scripting language or security boundary. The interpreter implements run-to-completion and the internal event queue inside an actor, while invoked-process lifetimes can be tied to the states that invoked them. The statechart layer therefore reuses actor messaging and lifecycle mechanisms while making their control structure explicit.

Chapter 6 committed to one realisation: SXML, an SCXML-inspired statechart notation with Web Prolog as its guard, action, and data-model language. The wider design space still admits alternatives, including SCXML itself with Web Prolog as a scripting layer. Either way, the relationship follows a familiar web pattern: a declarative control layer whose behaviour is scripted in a general-purpose language.

This combination has a broader consequence for the Agentic Web discussed in Chapter 8. Conversational and tool-using agents need more than components capable of producing answers: they need explicit protocol state, bounded waiting, recovery from partial failure, and coordination over long-lived interactions. Actors provide the durable interaction substrate, statecharts make its control inspectable, and logic supplies knowledge and constraints.

This also supports a pragmatic neuro-symbolic division of labour. Statistical components can interpret uncertain input or propose actions, while symbolic programs retain responsibility for protocol, policy, and lifecycle. Not every agent needs all three mechanisms, but the platform offers them.

9.7 Executable logic on the Web

A common use of logic in Web architecture is representational: publishing structured meaning, expressing constraints, and drawing conclusions from shared data. This leaves an operational question: where does executable reasoning live, and how can computations hosted by independent sites be combined?

The Trinity treats logic on the Web as executable as well as representational. A node can publish facts, but it can also host relational services, rules, and protocols. Goals are shipped and enumerated in the sequential partition; long-lived agents exchange symbolic messages and apply constraints in the concurrent partition. Logic thereby becomes a callable, composable, and governable component of live Web systems. The sequential and concurrent partitions provide different operational forms, but both use the same terms, goals, and relations as their symbolic substrate.

This positions the Prolog Web beside, rather than against, the Semantic Web. The Semantic Web asks how independently published data can be understood and combined; the Prolog Web asks how independently hosted logic can be executed and composed. The former emphasises shared identifiers, data models, and entailment regimes; the latter adds relational services and long-lived agents. RDF or other graph data can therefore provide interoperable knowledge while the Prolog Web provides an execution stack over or alongside it. Chapter 7, especially Table 7.1, develops this relationship in detail.

9.8 Comparison to other approaches

The claim that the Trinity constitutes a candidate paradigm invites a natural sceptical response: has this not been attempted before? Several precedents are close enough to warrant explicit comparison.

9.8.1 Qu-Prolog and distributed Prolog agents

Qu-Prolog is a close precedent for the actor layer: it combines multi-threading with symbolic message addresses and high-level Internet communication for distributed intelligent applications (Clark et al., 2001). Its `message_choice` construct also supports selective message handling with tests on candidate messages.[2] Distributed Prolog messaging is therefore an established line of work. The Trinity's particular emphasis is the combination of HTTP/WebSocket interaction contracts, cumulative node profiles, resumable PTCP toplevels, and explicit authority and lifetime rules across node-owner/client boundaries. These choices position its actor layer within a broader architecture for the open Web.

9.8.2 Linda and tuple-space coordination

Linda (Gelernter, 1985) introduced coordination through a shared associative memory, with pattern matching over tuples as the retrieval mechanism. The Trinity shares the emphasis on pattern matching as coordination, but the Prolog Web is a network of active agents with explicit identities and lifecycles, not a passive shared store. Coordination is directional: a client submits a goal to a named node and receives answers through a well-defined protocol. This directedness makes reasoning about authority, lifetime, and failure tractable.

9.8.3 Concurrent Prolog and the committed-choice languages

Concurrent Prolog (Shapiro, 1987), Parlog (Clark and Gregory, 1986), and GHC (Ueda, 1986) showed that SLD-resolution and concurrency are not incompatible. Their commitment mechanisms deliberately change the usual Prolog relationship between search and backtracking, and backtracking is absent from the concurrent partition. The Trinity makes a different trade-off. The sequential partition retains ordinary nondeterministic Prolog execution; the concurrent partition adopts Erlang-

[2] See the Qu-Prolog reference manual, under `message_choice`: `https://staff.itee.uq.edu.au/pjr/HomePages/QPFiles/SEC3.html`.

style message passing, making the non-declarative character of concurrency explicit. The two are orthogonal: backtracking does not roll back mailboxes, and actor communication does not interfere with proof search. The guards of `receive/1-2` (Section 2.3.4) resemble committed-choice guards, but a guard here is an ordinary Prolog goal; once it succeeds, the selected clause commits.

9.8.4 Erlang/OTP and actor behaviours

Erlang is the direct precedent for the Trinity's process-and-message discipline and its treatment of failure (Armstrong, 2003b, 2007a). OTP further shows how generic behaviours and supervision can turn actor primitives into an application architecture. The Trinity adopts this lesson but gives each actor a Prolog engine and preserves ordinary nondeterministic querying in a separate sequential partition. It also makes HTTP, WebSocket, and URI addressing part of the public distribution surface rather than assuming an Erlang-only network. The specific departures collected in Box 2.4 are not miscellaneous inventions: they arise primarily from Prolog's success, failure, and relational semantics, and from the authority and lifetime pressures of an open Web.

9.8.5 Oz/Mozart and the multiparadigm kernel language

Oz (Smolka, 1995), implemented in the Mozart system (Van Roy et al., 2003), is the closest and most instructive precedent. Like the Trinity, it combines logic programming, concurrency, and distribution in a single coherent design. Distributed Oz (Van Roy et al., 1997) adds network-transparent mobility and fault tolerance, and the textbook (Van Roy and Haridi, 2004) provides an unusually clean formal account through a hierarchy of kernel languages. The comparison therefore cannot rest on missing features; it must rest on design stance.

The decisive difference is that Oz uses a fully compositional, higher-order kernel language that accommodates its paradigms uniformly, rather than preserving Prolog's characteristic combination of Horn-clause syntax, unification-driven clause selection, backtracking, and the program-as-database identity. Oz therefore belongs to a different language tradition, not to a compatibility profile of Prolog. The Trinity, by contrast, is defined as a *profile* of Prolog. It extends ISO Prolog with concurrency primitives and distribution infrastructure while remaining syntactically and semantically recognisable to anyone who already knows the language. This is a deliberately narrower ambition – but it is the right ambition for a proposal that aims to unify an existing fragmented community rather than replace it with a new one.

A second difference concerns the Web. Oz/Mozart supplies its own distribution subsystem, organised around sites and network-transparent language entities. The Trinity instead makes HTTP, WebSocket, and URI addressing part of its public

interface. URIs identify nodes, while actors have pids qualified by node identity. The result is less general than Oz in the space of distribution models but more directly embedded in existing Web infrastructure.

9.9 Why the Trinity fits Prolog

The Trinity relies on a unity already present in Prolog. Terms serve as data structures, messages, and query templates; goals are both executable requests and descriptions of the answers sought; clauses are program text but can also be transmitted and composed as symbolic data. Unification and backtracking then connect these forms without a separate schema language, serialisation model, or enumeration calculus.

This unity also matters inside actors. Structural term matching selects incoming messages, with one-sided matching for receive patterns and unification for statechart event patterns. A logical guard can extend the substitution produced by that match. Each actor is therefore not only a mailbox-driven process but also a locally scoped reasoning engine. Other languages can reproduce the Trinity's architecture, but they will typically require distinct mechanisms for data representation, message matching, rule evaluation, and remote invocation. Prolog expresses the architecture unusually compactly.

9.10 The dual process

Chapter 1 described the project as two simultaneous movements: *webizing Prolog* and *prologizing the Web*. They acted as a forcing function. Webizing required URI-addressed nodes, open transport surfaces, and explicit security boundaries. Prologizing contributed unification, nondeterministic querying, symbolic code, actor-style concurrency, and execution in the browser. Pursued together, these requirements produced the mutual constraints described in this chapter rather than two independent collections of features.

On the webizing side, node URIs make placement and inference scope part of ordinary programs, while node-qualified pids address live actors without confusing them with public Web resources. HTTP supplies stateless and semi-stateful query surfaces; WebSocket supports longer-lived actor interaction. Profiles, explicit dependencies, and the security mechanisms of Chapter 4 turn differences between deployments into declared boundaries rather than hidden assumptions. These choices also keep the Prolog Web accessible to non-Prolog clients and aligned with the Web's culture of open interfaces.

On the prologizing side, a goal becomes a Web request whose answers are matched by unification and consumed through backtracking and paging. Symbolic programs can be shipped to an execution context rather than reduced to passive data. Erlang-style actors add persistent interaction without replacing the sequential Prolog core,

and the browser runtime makes the same model available on the client side. The six commitments of this chapter are thus consequences of pursuing both directions together, not features selected independently.

The dual process leaves questions of community and execution beyond the paradigm itself. Chapter 10 considers federation as a response to fragmentation and as a way of opening the network to neighbouring communities. Chapter 11 then distinguishes the engineering paths that can follow from the demonstrated architecture.

The paradigm claim therefore does not depend on predicting an institutional destination. Its immediate test is whether the language profile, agent model, and network protocols continue to form a useful and coherent programming model in practice.

9.11 Summary

This chapter has argued for a candidate paradigm for web-native symbolic computation. Its sequential and concurrent partitions separate nondeterministic querying from actor interaction without preventing their composition. Interaction portability asks whether relocation preserves the permitted observable behaviours declared by a profile and a program's dependencies. Agents provide the main units of structure, while unification, backtracking, and scoped remote inference preserve a recognisably logical foundation.

The sequential partition can be read at two levels: as inference among interlinked logic programs and as communication among the processes that realise it. The agent abstraction spans several granularities – actors, toplevels, statechart processes, and nodes. In this architectural usage, an agent need not be an intentional agent in the AI sense. This provides the setting in which interaction can itself involve inference: messages unify with patterns, guards prove goals, and the resulting substitutions inform actions.

Within the concurrent partition, common primitives apply locally and remotely while latency, failure, and security boundaries remain visible. Statecharts add inspectable governance of long-lived interaction, and the executable-logic layer complements Semantic Web representation with hosted rules and relational services.

Whether this constellation becomes a paradigm in the stronger sense – a named point in the design space, implemented independently, taught, and built upon – is for a community to decide. What we claim is that language, agents and network fit together as one programming model, and that this model adds executable logic, actors and governed interaction to the Web.

Chapter 10
Communities, interoperability, and adoption

The preceding chapters developed the Prolog Trinity as a technical proposal. This chapter asks what that proposal might contribute to the communities that would have to implement, use, and sustain it. It begins with Prolog implementers, tool builders, teachers, and users, then widens the discussion to neighbouring communities in logic programming, the Semantic Web, AI, web development, and Erlang.

The Prolog community already encompasses a diverse collection of systems. Figure 10.1 offers an impressionistic reminder of that plurality. The challenge considered in this chapter is not how to remove these differences, but how to make cooperation across them possible.

Fig. 10.1 Logos of thirteen Prolog systems: B-Prolog, Ciao, ECLiPSe, GNU Prolog, JIProlog, Scryer Prolog, SICStus Prolog, SWI-Prolog, Tau Prolog, Trealla Prolog, tuProlog, XSB, and YAP. The collage illustrates plurality; it is not a ranking or a measure of current project activity. The logos remain those of their respective projects.

10.1 From fragmentation to federation

Most current systems implement the ISO Prolog core, but important differences remain in extensions, libraries, tooling, and choices left to the implementation. This makes it harder to move programs, and also to share teaching material, libraries and tests, or to standardise further.

Three responses deserve comparison. Incremental proposals can enlarge the portable common ground. A unified Prolog could bring major facilities into one coherent implementation. Federation instead asks distinct systems to agree on the surfaces through which they interact at runtime. The first two seek greater internal agreement; the third seeks useful cooperation across differences that remain.

10.1.1 Incremental convergence through PIPs

One response is the *Prolog Improvements Forum*.[1] Its *Prolog Improvement Proposals* (PIPs) document language and library features supported, or intended to be supported, by more than one system. PIPs are not ISO standards, but they can record implementation experience, coordinate common APIs, and prepare material for later standardisation.

This is a realistic approach to increasing the portable part of Prolog: bottom-up, incremental, and grounded in implementation experience. Agreement on such matters as library interfaces can narrow the portability gap without requiring systems to converge wholesale, and proposals tested in several implementations can provide useful evidence for later standardisation.

PIPs reduce the cost of moving programs between systems, but they do not by themselves let systems cooperate at runtime. The Trinity therefore treats PIPs and federation as complementary: one narrows avoidable dialect differences while the other specifies the boundaries across which differences may remain.

10.1.2 One Prolog to rule them all

Gupta et al. propose a more ambitious response: integrate tabling, constraints, coroutining, parallelism, and goal-directed ASP efficiently and seamlessly in one system (Gupta et al., 2023). This continues a long-standing hope that the strongest facilities associated with different Prologs might be brought into one coherent environment.

The integrative impulse is already visible in existing systems. SWI-Prolog, for example, combines broad libraries and interfaces with constraint programming,

[1] `https://prolog-lang.org/ImplementersForum/PIPs`

tabling, Just-In-Time indexing, and an implementation of s(CASP).[2] The unified-Prolog vision goes further: features that compose efficiently, not just features in the same system.

That is an attractive research programme, but it answers a different question from the Trinity. A unified system assembles facilities inside one implementation; federation lets existing and future implementations cooperate while remaining distinct. Progress in either direction could benefit the other.

10.1.3 The middle way: interoperability as a federating force

Language standards usually define a common language or API so that programs can be moved or reimplemented. Federation makes a narrower agreement: participants standardise behaviour at an interaction boundary while retaining internal differences. The two approaches are compatible, but they solve different problems.

The Trinity defines such boundaries as profiles, protocols, and APIs. A node need not run a particular Prolog system; it must conform to the profile it advertises, including the predicates, transports, and behavioural contracts that profile specifies. Different nodes may realise the same surface with different optimisations, libraries, and architectures. What is shared is the *interface*, not the *engine*.

This is federation in a precise sense: independent parties agree on how to communicate without agreeing on how to think. The agreement is operational – terms have specified meanings, interactions follow stated contracts, and capabilities are advertised – while implementation choices remain local. This resembles the organisation of the Web itself: boundaries are specified closely enough to permit cooperation, while the systems behind them remain independently owned and free to evolve.

Figure 10.2 illustrates the concept in concrete terms. Rather than collapsing the distinctive strengths of SWI-Prolog, ECLiPSe, XSB, and YAP into a single uniform system, we envision an ecosystem where each system is wrapped in a node implementation that allows it to converse with others. The diversity of approaches is preserved, yet they can all contribute to a common fabric of cooperation. Participation remains voluntary; a node interface of this kind is required only by a system that chooses to join the shared Prolog Web.

The SWI-Prolog demonstrator supports communication among nodes owned by different parties and running in different locations. Complementary GNU Prolog and Trealla prototypes now provide initial cross-runtime evidence at selected query and actor boundaries, including bidirectional interactions with SWI-Prolog (Section E.6.2). These tests do not establish complete profile conformance or independent validation of the design. Extending their coverage and involving other implementers are the next tests of the federation proposal.

Portability and runtime interoperability should not be confused. Portability moves code between systems; interoperability lets systems cooperate during execution.

[2] `https://github.com/SWI-Prolog/sCASP`

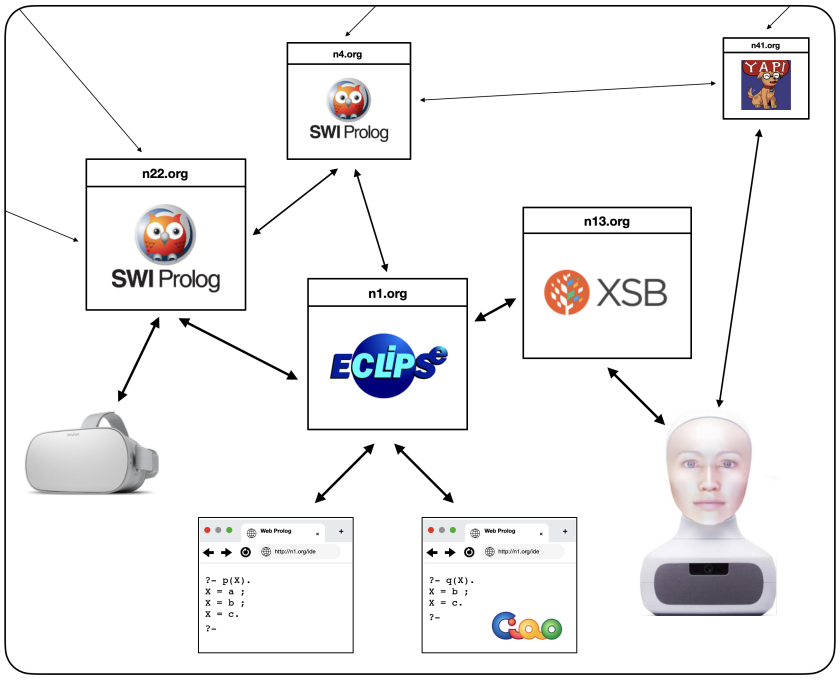


Fig. 10.2 The network introduced in Figure 1.3, now imagined with heterogeneous Prolog implementations behind its nodes. Clients and nodes share profile-defined interaction surfaces while implementations retain their own internal strengths. Later in this chapter the topology remains fixed while the picture opens to neighbouring ecosystems.

They are complementary, and both require implementer cooperation, but runtime disagreements are especially costly because they become visible inside a live distributed computation. The advertised contract must therefore be precise.

A profile must also fix the latitude that ISO Prolog leaves to implementations (Section 9.2.1), so that conformance is a claim about observable behaviour rather than about ancestry.

Diversification through shared surfaces

Federation allows Prolog systems to retain the extensions and optimisations that distinguish them. A node may expose those facilities through owner-defined relations or services without adding them to the common profile; indeed, a specialised facility may be exactly why a client chooses that node.

Imagine a future where an application orchestrated through Web Prolog can dispatch a deductive query to a remote XSB engine, feed the results into an ECLiPSe solver, and manage the user interface via SWI-Prolog. The names are illustrative

rather than commitments about a particular integration. The point is that profile conformance could allow specialised services to be composed without requiring their implementations to merge.

The profile hierarchy makes this approach inclusive. A small or single-threaded system can implement RELATION without taking on the actor runtime required by ACTOR. The work is still substantial: profile contracts, serialisation, security, implementations, and conformance tests must agree. Work in the Prolog Improvements Forum that catalogues divergences and clarifies semantics can inform those profiles; executable profiles can in turn reveal disagreements that portability work may otherwise miss.

If the Prolog Web succeeds, it will not be because every Prolog system has been replaced by a single implementation, but because many implementations have adopted compatible interaction surfaces. Success is measured by interoperation and reuse, not by replacement. The same principle extends to neighbouring communities: the Trinity does not ask the Erlang community to adopt Prolog, or the Semantic Web community to abandon RDF. It asks whether there is value in a shared execution substrate for logic-based agents on the Web.

10.2 Community infrastructure and adoption

Fifty Years of Prolog and Beyond (FYPB) (Körner et al., 2022) also identifies fragmentation of the Prolog *community* and a comparatively small user base as concerns. Grouping around implementations and research traditions is natural and often productive, but it can make knowledge, tools, and teaching material harder to share. A technical architecture cannot solve that social problem. It can provide common objects around which collaboration occurs.

10.2.1 Shared objects for collaboration

The "All Things Prolog" website, sponsored by the Association for Logic Programming, already provides a valuable system-independent home for information, education, discussion, and improvement proposals.[3] A Trinity portal should complement that work rather than duplicate it. Its distinctive role would be operational: discovering public nodes and services, running examples against them, displaying conformance information, and linking the resulting artefacts to documentation and discussion.

The Prolog Web also broadens what can be shared. In addition to source code, a developer could publish a bounded service or a live demonstration actor that others

[3] `https://prolog-lang.org/`

can use immediately. This does not replace repositories or package managers; it adds running computational resources to the things a community can exchange.

The same infrastructure can support work on the infrastructure itself. Profiles provide specifications to debate, conformance suites provide common tests, and nodes provide shared targets for clients, examples, and documentation. FYPB observes that portability problems extend beyond the language core to libraries and tools. A profile-governed node cannot make arbitrary host-system tools portable, but it can give cross-system tools a stable surface on which to operate.

10.2.2 Playgrounds and learning

The interactive and distributed character of Web Prolog suggests a useful role for browser-based playgrounds. Figure 10.3 shows three established environments: SWISH, based on SWI-Prolog; the Ciao Prolog playground; and the Tau Prolog sandbox. They let visitors program without first installing a local system and are useful for teaching, experiments, and shareable examples.

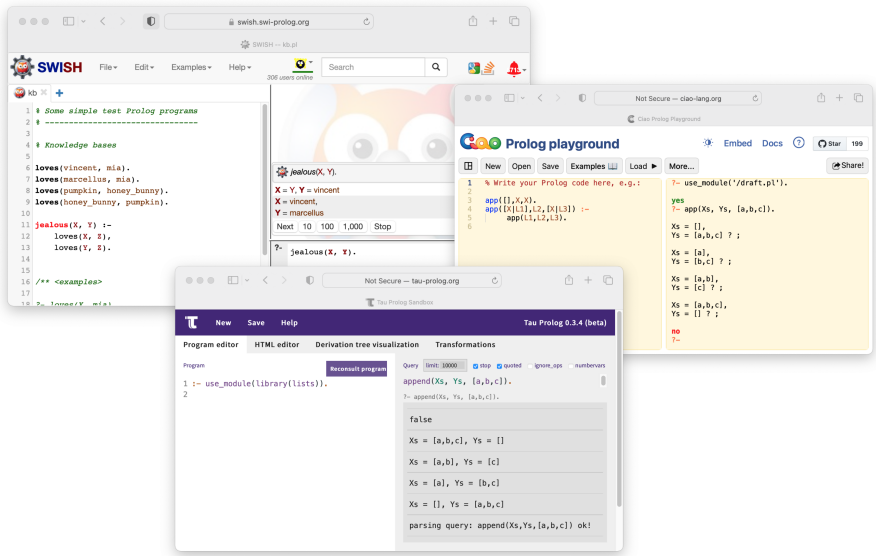


Fig. 10.3 A snapshot of three browser-based environments: SWISH for SWI-Prolog, the Ciao Prolog playground, and the Tau Prolog sandbox. Their different interfaces and dialects reflect their different purposes; a shared Web Prolog profile could add a portable teaching surface between them.

These playgrounds serve different systems and audiences, and their differences are often deliberate. They also reveal a portability boundary: a program written for

one environment may depend on its dialect, libraries, or interface conventions and therefore require changes in another. A shared layer could help: a common Web Prolog profile could make a useful core of exercises and queries portable while allowing each playground to retain the facilities that distinguish it.

A Web Prolog playground served by an ISOTOPE node could provide one such shared environment for general instruction. When backed by an ACTOR node, the same interface could extend from ordinary queries to actor programming and distributed interaction. It should be viewed as a complement to SWISH, the Ciao playground, and the Tau Prolog sandbox: a place to teach the common profile and its network model, not a universal replacement for system-specific environments.

Web delivery removes installation friction, but it does not remove Prolog's conceptual learning curve. Existing Prolog programmers can begin with the sequential profiles and encounter actor concepts only when they need them. Readers familiar with Erlang will recognise much of the concurrent vocabulary, while those new to actor programming can draw on the substantial Erlang literature. The profile ladder thus provides a pedagogical sequence as well as an implementation sequence.

Interactive, browser-based examples can present Prolog as a current language for queries, services, and distributed symbolic applications. Whether this attracts new users is an empirical question, but it lowers the cost of trying the language and of sharing a runnable example.

Staying close to traditional Prolog makes it possible to reuse the many good textbooks and tutorials already available. Erlang literature can likewise explain several principles behind Web Prolog's actor layer. Syntax, failure, logical variables, and the toplevel protocol remain specifically Prolog concerns, however, so dedicated Web Prolog tutorials are still needed. The demonstrator manual and browser tutorial are first steps rather than substitutes for such material.

10.2.3 Conditions for wider adoption

Prolog has a much smaller educational, industrial, and public presence than mainstream languages. FYPB treats this as a threat because a small user base can reinforce a small ecosystem: fewer users mean fewer libraries, contributors, examples, and deployment reports, which in turn give new users fewer reasons to begin. Technical quality alone does not break that cycle.

The Trinity can affect only some conditions for adoption. Browser access lowers the cost of trying Prolog, and runnable links lower the cost of sharing an example. A common specification and conformance tests allow clients, documentation, and services to accumulate around more than one implementation. A visible application domain can provide motivation to learn the language. None of these guarantees growth, but each removes a practical obstacle.

Usefulness must supply the pull. Developers will tolerate an unfamiliar language when it solves a problem they care about unusually well. The credible case for Web Prolog therefore lies in applications where relations, constraints, symbolic

knowledge, or actor-based coordination are central. Other web applications may be better served by mainstream languages and frameworks. Documentation, examples, responsive maintenance, and a welcoming community matter at least as much as the language design once a visitor arrives.

Familiarity remains a weakness. Prolog's syntax and execution model are foreign to many programmers trained on imperative languages. Erlang programmers will recognise much of the actor vocabulary, but unification, logical variables, failure, and backtracking still require explanation. The RELATION, ISOTOPE, and ACTOR progression can make that learning incremental: ordinary relations first, remote execution next, and explicit concurrency only when it becomes useful.

There is also a danger in gaining visibility through one domain. Prolog must not be reduced to a web language or detached from systems serving other applications. The structural answer is to define Web Prolog as a profile of general-purpose Prolog, not a replacement for it. Section 10.4 returns to that risk. The Web offers an easy place to encounter Prolog; shared contracts can lead newcomers on to the many implementations.

10.3 Opening the network to neighbouring communities

The case for federation does not stop at the boundary between Prolog implementations. Rule engines, triple stores, ASP solvers, Erlang services, model-serving stacks, and web applications need not adopt Prolog or surrender their own semantics in order to participate. They may instead expose selected facilities through a specified surface, or remain behind an owner-side adapter. The adapter work and semantic mapping required at such a boundary are real engineering costs. As Section 9.2.1 emphasised, profiles fix observable contracts; they do not prescribe the internals behind them.

Figure 10.4 is the third view of the network first shown in Chapter 1 and revisited earlier in this chapter. The topology and clients have not changed. What has changed is what some nodes expose and what external components may stand behind them. At `n4.org`, the general SWI-Prolog logo has given way to *cplint*, indicating probabilistic logic programming offered through that implementation. Database cylinders suggest RDF stores behind logical interfaces, while the Python and model-provider logos suggest an owner-side adapter to a neural service.

The following examples ask what such boundaries might offer to logic programming, the Semantic Web, AI, web development, and Erlang. Some of the illustrated integrations are supported by the demonstrator; the others describe integration paths or research possibilities.

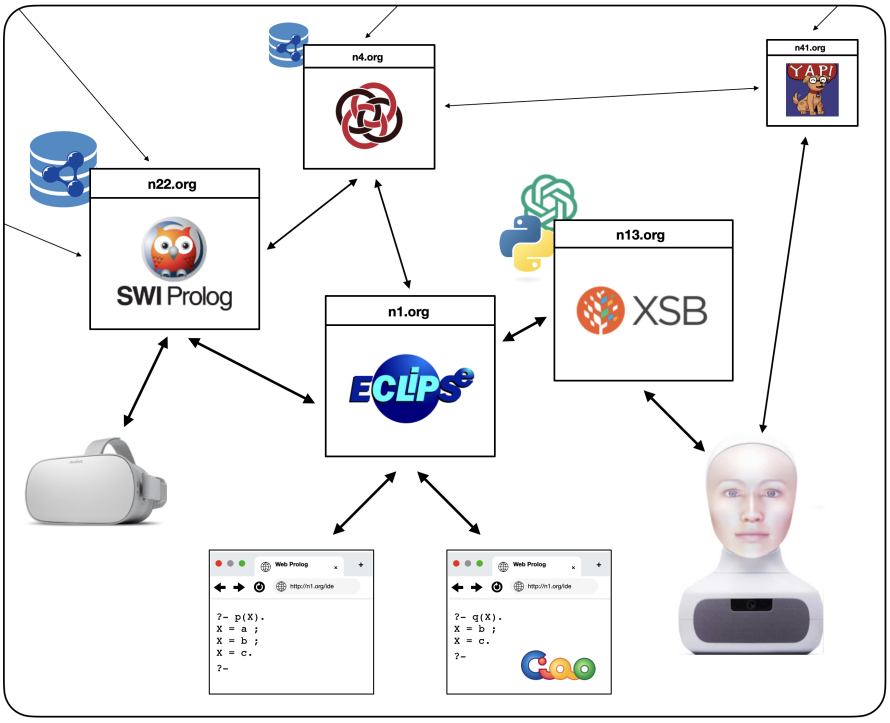


Fig. 10.4 The network of the opening chapter, now opened to neighbouring ecosystems. Node `n4.org` exposes probabilistic reasoning through *cplint*; the database, Python, and model-provider logos indicate possible backing services. The network and its clients remain unchanged.

10.3.1 For the broader logic programming community?

The logic programming landscape

Prolog is the most widely known logic programming language, but it is far from the only one. Over the past five decades, researchers have extended, restricted, or reinterpreted the Horn-clause core in many directions. The following survey is necessarily selective, but it covers formalisms and systems relevant to the possibility of a broader Logic Programming Web.

Prolog's native mode of reasoning is deduction – deriving consequences from given rules and facts – but logic programming as a field has long explored other modes of inference as well. Abductive logic programming (ALP) reverses the direction: given an observation, it searches for hypotheses that, together with background knowledge, would explain it (Kakas et al., 1992). Inductive logic programming (ILP) goes further still, learning new rules from examples and background theory (Muggleton and De Raedt, 1994; Cropper et al., 2022). Both traditions extend the reach

of logic-based reasoning well beyond what a standard Prolog system provides out of the box.

Nor is deductive logic programming itself a monolith. Datalog restricts Horn clause logic to function-free programs, sacrificing Turing completeness in exchange for guaranteed termination and well-understood complexity bounds (Ceri et al., 1989; Green et al., 2013). Well-founded semantics (WFS) addresses a different limitation, providing a three-valued account of negation in which every normal logic program has a unique minimal model assigning each ground atom the value *true*, *false*, or *undefined* (Van Gelder et al., 1991); systems such as XSB and SWI-Prolog implement WFS via tabling, giving them stronger semantic guarantees than standard Negation As Failure. Answer Set Programming (ASP) departs more radically, searching for the stable models of a logic program rather than computing answers by resolution; a recent variant, s(CASP) (Arias et al., 2018), combines ASP with constraint logic programming in a top-down, query-driven execution model with support for explanation generation. Probabilistic logic programming (PLP) equips logic programs with probabilistic reasoning: ProbLog (De Raedt et al., 2007) is a Python-based toolbox that defines distributions over possible worlds and supports inference, sampling, and parameter learning, while cplint (Riguzzi, 2018) is a complementary SWI-Prolog library for LPADs under the distribution semantics, accessible both locally and through "cplint on SWISH." Constraint logic programming (CLP) extends Prolog's unification with constraint solvers over domains such as finite integers, reals, and sets, and most major Prolog systems now ship with CLP libraries as standard. Finally, Logic Production Systems (LPS) (Kowalski and Sadri, 2015; Kowalski et al., 2023b) combines logic programming with reactive, event-driven computation.

The common thread is that logic programming is not a single language but a family of formalisms. That does not make every member of the family a Prolog implementation or give it the interaction semantics required by a Web Prolog profile. It does create opportunities for narrower, explicitly mapped interfaces through which different kinds of logical service can participate.

Different semantics at a shared boundary

ASP systems, Datalog variants, constraint systems, probabilistic languages, and multi-paradigm systems are not necessarily Prolog implementations in the sense required by ISOBASE. Such a system should not advertise a profile whose language contract it does not implement. It can instead expose a small logical API through RELATION, or place a profile-conforming controller in front of its native query interface. The wrapper must translate both data and meaning: what constitutes one answer, how alternatives are ordered, how errors are represented, and which calls are authorised.

Probabilistic and specialised services

The cplint system, described above, provides a useful intermediate case. It is a SWI-Prolog library and is available through cplint on SWISH. Because SWISH builds on Pengines, the following existing Pengine interaction can call it directly:

```
?- pengine_rpc('https://cplint.ml.unife.it', prob(blue,Prob), [
     src_text("
       :- use_module(library(pita)).
       :- pita.
       :- begin_lpad.
          blue : 2/3; red : 1/3 :- urn1.
          blue : 25/55; red : 30/55 :- urn2.
          urn1 : 0.5 ; urn2 : 0.5.
       :- end_lpad.
     ")
   ]).
Prob = 0.5606060606060606.
?-
```

This demonstrates remote access through the protocol lineage from which Web Prolog grew; it does not by itself establish conformance to a current Web Prolog profile. A node owner could expose an equivalent, bounded relation such as `urn_probability/2` through RELATION without exposing cplint's entire programming environment. LPS, also available through SWISH, presents a similar opportunity, though its visual results require an explicit agreement about how an execution trace is represented.

ProbLog, implemented primarily as a Python toolbox, is a different case. Its declarative character does not make it an ISOBASE implementation: ISOBASE fixes a Prolog language and toplevel contract, not merely the absence of side effects. A more realistic first integration would therefore publish selected ProbLog computations as RELATION predicates. The same pattern applies to Picat's constraint solving, planning, and optimisation facilities. A richer profile could be justified later if multiple implementations converge on a larger common contract.

One stable model as one solution

Answer Set Programming makes the semantic mapping especially visible. For the program

```
cross :- not wait.
wait  :- not cross.
```

a straightforward adapter convention is to expose each stable model (Gelfond and Lifschitz, 1988) as one solution of an owner-defined RELATION predicate:

```
?- model(M).
M = [cross] ;
M = [wait].
?-
```

The backend retains ASP semantics; the wrapper defines only how its results cross the boundary. It must choose a canonical representation and an ordering stable enough for the paging contract. Brave and cautious consequences could be exposed as separate relations. This illustrative adapter exposes ASP results through a relational interface while retaining ASP semantics; `model/1` remains an owner-defined predicate rather than part of the RELATION profile.

The larger opportunity is federation without forced unification. A probabilistic service, an ASP solver, and a constraint solver could be composed through small declared surfaces while remaining different systems. Achieving that requires more than a thin transport shim: profiles, schemas, error vocabularies, authorisation, resource bounds, and conformance tests all matter. The Trinity offers a place to specify those agreements; adoption must show whether the cost is justified.

10.3.2 For the Semantic Web community?

Chapter 7 compared the Prolog Web with the Semantic Web in detail. The modest claim here is complementarity. RDF, OWL, and SPARQL provide standard ways to identify, describe, and query data. A Web Prolog service can sit above such resources when an application also needs rules, stateful behaviour, or coordination with other services. It may front a SPARQL endpoint, translate selected RDF structures into terms, apply domain rules, and publish the result through a narrow logical API.

That boundary should not be mistaken for a universal representation. RDF graphs and Prolog terms have different semantics; an adapter must state how identifiers, literals, blank nodes, entailment regimes, and failures are mapped. Nor is every triple store thereby a Prolog node. The node is the component that implements the advertised Web Prolog contract; the store may be one of its private backing resources.

The practical offer is thus a programmable agent and orchestration layer that can reuse Semantic Web infrastructure without replacing it.

10.3.3 For the AI community?

Chapter 8 developed the case for symbolic, neural, and hybrid agents, including concrete interaction patterns between Prolog and large language models. It also described an experimental Model Context Protocol bridge. The point need not be repeated here. From the outside, the Trinity offers a control layer in which a model call is one effectful service among others: it can be authorised, timed out, checked

against symbolic constraints, combined with stored knowledge, and incorporated into a longer-lived actor.

This does not turn every model endpoint into an actor. A node owner must still write an adapter, choose a stable request and result vocabulary, protect credentials, impose budgets, and decide what model output is permitted to affect. Conversely, a Prolog relation exposed to a model-oriented orchestrator may use MCP when a bounded, schema-described tool call is appropriate, while PTCP remains useful when the interaction requires a Prolog session, nondeterministic answers, or actor messaging. One knowledge base may therefore offer both interfaces without pretending that they express the same protocol.

Agents and the Web

Hendler's early discussion of agents and the Semantic Web (Hendler, 2001), followed by his later question "Where Are All the Intelligent Agents?" (Hendler, 2007), identified an enduring problem: capable components do not by themselves supply an open architecture for long-lived, accountable interaction. Today's model-based agents make that problem more visible, not obsolete. Some maintain state and use tools; others are short-lived wrappers around a model call. The useful contrast is therefore not "stateless neural agent" versus "stateful symbolic agent," but whether state, policy, communication, and failure are explicit parts of the architecture.

Web Prolog actors are stationary rather than mobile. They can nevertheless reach remote knowledge by fetching a permitted representation, querying a resident service, or spawning a child actor on the node that owns the relevant resource. Mobility could be built above that substrate, but is not required merely to delegate work.

An active W3C Community Group on Autonomous Agents on the Web studies Web-aligned, transparent, and accountable multi-agent systems. [4] Its work is a relevant neighbour, not evidence that the Trinity already implements a Hypermedia MAS architecture. A useful research programme would compare the group's hypermedia interaction model with ACTOR messaging, identify the missing agent abstractions, and test interoperation rather than assume equivalence.

10.3.4 For web programmers?

For web programmers, the proposal is not a replacement for JavaScript or a general-purpose web stack. It is an additional tool for applications dominated by rules, constraints, symbolic data integration, or long-lived reactive behaviour. In those domains, relations can make decision logic explicit, while actors can maintain conversations and respond to events without reducing every interaction to an isolated request.

[4] `https://www.w3.org/groups/cg/webagents/`

There is a learning cost. Unification, logical variables, and backtracking do not behave like assignment, mutable variables, and loops. The case for Web Prolog must therefore be demonstrated in working tools and examples rather than asserted in general terms. The browser ACTOR node described in Section 3.5.5, with implementation details in Section E.5.1, is one such piece of evidence.

Constructing user interfaces with statecharts

User interfaces must govern asynchronous events, legal operations, and transitions between modes. Statecharts make this control structure explicit. Chapter 6 combined that model with Web Prolog: guards are goals, actions are predicates, and the data model can be queried relationally. This is particularly attractive when an interface must check policies, maintain a structured domain model, or coordinate remote services. Simpler interfaces may need no explicit statechart.

The relationship with JavaScript statechart tools is complementary. XState, for example, uses JavaScript functions for guards and actions; a Trinity statechart actor uses Web Prolog for executable content. Both make control state visible. The difference is the scripting and data model available inside the machine, and that difference matters most when logic and constraints are part of the application.[5]

Conversational and embodied agents

The demonstrator's portal provides a concrete starting point for a browser environment in which statechart-driven and receive-loop actors control avatars. A fuller IDE could combine SXML and Web Prolog editors with execution, logging, and state visualisation. This continues earlier work on the *Dialog Web Lab*, a browser environment for authoring and executing SCXML dialogues with a simulated embodied agent (Junger et al., 2012).

Such an IDE would let a developer inspect the state machine while the underlying artefact runs as a genuine actor on an ACTOR node. The same message-level control could also be connected, through suitable adapters, to avatars in an immersive environment or to a social robot.

Controlling a social robot.

Furhat, a tabletop social robot head developed by Furhat Robotics, provides a physical example. Its realtime API uses WebSockets and permits an external client to send commands and receive events. An owner-side adapter could translate between that JSON protocol and the terms used by a Web Prolog actor or statechart actor.[6] The

[5] `https://stately.ai/docs/xstate`

[6] Furhat Realtime API documentation: `https://docs.furhat.io/realtime-api/intro`. The author is not affiliated with Furhat Robotics, but knows members of the team personally.

control logic for an on-screen avatar may then be reusable, but the embodiment mapping is not free: gaze, speech, timing, errors, and physical functions need an explicit adapter.

10.3.5 For the Erlang community?

Erlang's influence is present throughout Web Prolog's actor layer: isolated processes, asynchronous messages, selective receive, monitors, and generic behaviours all acknowledge that lineage (Lager, 2019). Web Prolog is not, however, Erlang with Prolog syntax. It retains logical variables, relational guards, failure, and backtracking, while changing several concurrency choices to suit an open Web in which node owners and remote clients are distinct. Its links express a default one-way parent–child lifetime dependence rather than Erlang's bidirectional relation; a receive commits to a selected message but then executes an ordinary Prolog goal; monitoring distinguishes logical failure from successful completion and exceptions; and profiles, authorisation, sandboxing, and containment mediate an open trust boundary. These semantics were developed where the relevant operations were introduced, rather than repeated here as a comparison table.

The differences still leave room for hybrid systems. An Erlang service can call a bounded Web Prolog relation over HTTP, and a Web Prolog owner can place an adapter in front of an Erlang service. A genuinely profile-conforming ACTOR node on the BEAM would be a larger project: it would have to implement PTCP, the term and error contracts, authorisation, and the specified Web Prolog semantics rather than merely exchange messages with an existing node.

Robert Virding's Erlog already provides a Prolog interpreter implemented in Erlang and integrated with its runtime (Virding, n.d.). It is a relevant precedent for hosting Prolog execution on the BEAM, although its suitability as a base for a Web Prolog node remains to be evaluated. An ACTOR implementation would additionally have to establish the specified protocols, process semantics, and database isolation guarantees.

There is also a research question here. A runtime combining BEAM-like process scheduling and fault isolation with unification, backtracking, and meta-level execution would occupy a different point in the language-design space from both Erlang and today's demonstrator.

In every example, what is offered is a boundary. A system may expose one RELATION predicate, provide a richer profile-conforming controller, or remain outside the network and be called through an owner-side adapter. In every case, the important questions are observable: what is named, what one answer means, how alternatives and failures are represented, who may call the service, and which resources bound the call. Whether communities will invest in those boundaries is the adoption question to which we now turn.

10.4 Risk analysis

The proposal carries two related risks. The first is rhetorical: calling Prolog a web language may narrow the perception of a general-purpose language and repeat the damage caused when Prolog was identified too closely with earlier AI hype. Paulo Moura put the objection sharply: "The last thing we need is for Prolog to be downplayed as a Web thing."[7] That criticism is justified. An earlier description of Web Prolog as a "special-purpose language" was too narrow.

The second risk is technical. Jan Wielemaker welcomed the possibility that Web Prolog might succeed but warned that it could become detached from Prolog systems aimed at other applications.[8] A web-facing dialect could develop its own syntax, libraries, and assumptions until the shared name concealed two communities rather than connecting them.

The profile model is intended to answer both objections. Web Prolog is not a replacement for general-purpose Prolog; it is a bounded profile that fixes the observable choices required at an open network boundary. "Web-facing" need not mean "web-only." Implementations may expose the profile alongside their existing languages and facilities, and owner-defined services can draw on those facilities without adding them to the untrusted profile.

This answer is structural, but not automatic. Detachment would still occur if the profiles were specified without implementer participation, if teaching material presented them as a separate language, or if the web interfaces could not be maintained alongside host systems. Shared syntax where possible, explicit mappings where it is not, conformance suites run by several implementations, and continued use of ordinary Prolog libraries are therefore more important than branding.

The Web remains a plausible application surface because many of its problems have a logical shape: querying and transforming structured data, evaluating policies, selecting under constraints, reasoning over graphs, and coordinating stateful services. Adoption must be earned by precise interfaces and useful applications.

Figure 10.5 presents one possible long-term way to keep the web-facing and general-purpose layers connected. It extends the profile diagram from Chapter 3 with a hypothetical outer ring provisionally called *ISO Prolog 2.0*. The name is a design hypothesis in this book, not an existing standard or a claim about what a future ISO project must contain.

The outer ring restores the kind of host access that Chapter 2 excludes from an open, untrusted Web Prolog execution environment: ambient file I/O and unrestricted access to sockets, streams, threads, and the host system. The exclusions reflect trust and deployment context, not a smaller Prolog. A future general-purpose standard might also consider tabling, constraint logic programming, and other mature developments, but this book does not attempt to settle that inventory.

The nesting expresses the strongest version of the proposal: a future system conforming to the outer profile would also expose the Web Prolog profiles. That

[7] Paulo Moura, SWI-Prolog Discourse, "Encouraging industry about Prolog".

[8] Jan Wielemaker, in the same discussion.

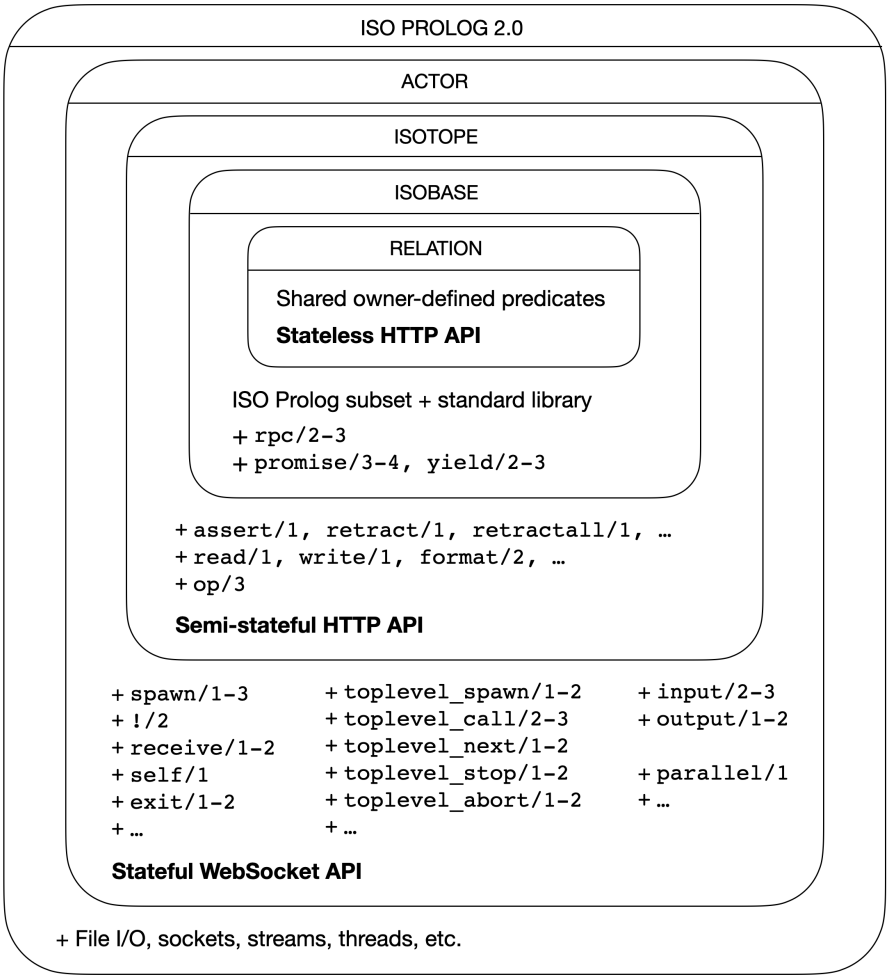


Fig. 10.5 The profile hierarchy completed by a hypothetical outer ring for ISO Prolog 2.0. Read from the inside out, the figure moves from open-web interaction surfaces towards a trusted, general-purpose environment that restores facilities such as file I/O, sockets, streams, and threads. The predicate lists are representative rather than exhaustive; the manuals define the current profile inventories.

is a possible direction, not a present conformance rule. A separately standardised Web Prolog layer implemented by adapters is also possible. The important claim is narrower: web-facing profiles and future general-purpose Prolog work should be designed to compose rather than drift apart.

10.5 Summary

For portability, the Trinity proposes a profile-defined fragment and conformance tests, not the portability of arbitrary host-system programs. For systems fragmentation, it proposes runtime interoperability without asking implementations to converge internally. For tooling, it offers common targets for clients, documentation, discovery, tutorials and playgrounds. For the post-desktop Web, it supplies browser-accessible nodes, services and actors. Beyond Prolog, what can be shared is a carefully described boundary.

None of this will happen without other implementations, specifications, tests and users; the demonstrator shows only that the design can be built. Turning that invitation into shared infrastructure is the subject of the final chapter.

Chapter 11
From vision to execution

> Vision without execution is just hallucination.
>
> *Thomas Alva Edison*

At the beginning of this book, the Prolog Trinity ecosystem was introduced by means of a mind map. Its purpose was primarily orienting: to sketch the conceptual territory, name its principal components, and hint at the relationships between them. That diagram accompanied the reader through the early stages of the argument, serving as a visual shorthand for a still-emerging design.

In Figure 11.1 we return to the same mind map – structurally unchanged, but now adorned with three banners of the kind commonly used by the W3C.

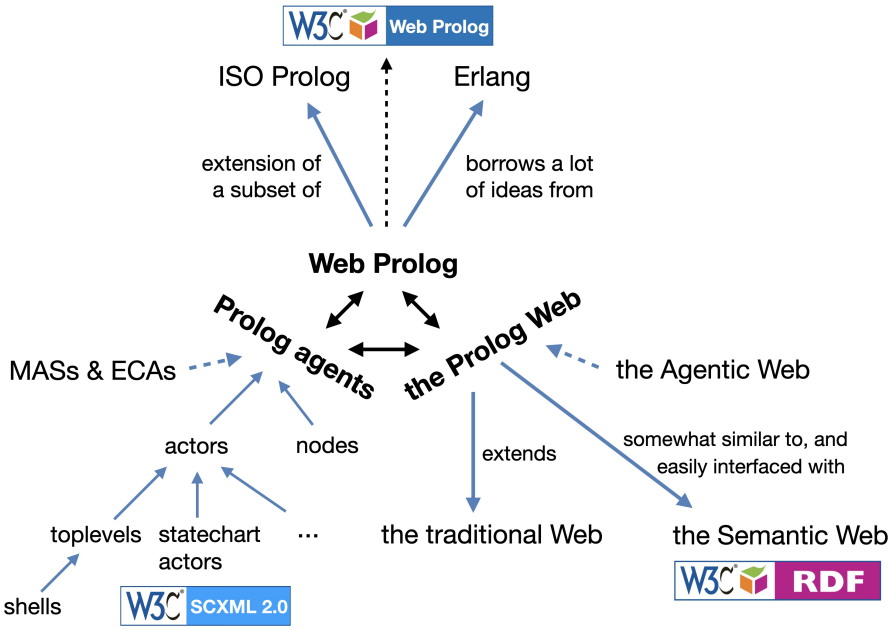


Fig. 11.1 The Prolog Trinity mind map revisited, now framed as shared infrastructure and adorned with three W3C-style technology banners. Of the three, only RDF is an actual, existing W3C standard; the other two are provisional logos of intent.

Banners like these are signs of collective agreement: indications that a technology has passed from an individual proposal to shared infrastructure, and that its definition, maintenance, and evolution are now matters of public process rather than private intent. A banner can compress years of discussion, specification work, experimentation, revision, and implementation experience into a single mark.

Of the three standards implicitly invoked by the Trinity vision, only RDF has completed that journey. The Web Prolog core and its SXML statechart notation have not. The difference shows where the proposal stands. RDF has been specified, standardised, implemented, deployed, criticised, and revised. Web Prolog and SXML remain forward-looking designs – argued for and partly implemented, but not socially ratified.

Nor is there only one possible destination. The demonstrator may develop into a serious platform centred on SWI-Prolog, with production quality valued above cross-system portability. The profile and protocol work may instead, or in parallel, become the basis of a federated programme involving several Prolog systems and perhaps standards bodies. The statechart work has yet another lineage through SCXML. These paths share much engineering, but they are not the same project and need not advance at the same pace.

Execution is therefore a set of choices rather than a predetermined roadmap. This chapter gathers the open research and engineering work, separates two plausible development routes, and identifies a layered set of specification targets if federation is pursued. It then considers the implementation, testing, and shared ownership needed to turn private design into public infrastructure. The two provisional banners in Figure 11.1 mark work that would have to happen outside this book; they claim no authority.

11.1 A research and engineering agenda

The demonstrator has converted several architectural claims into running experiments, including public services, browser-hosted actors, statechart actors, and cross-node PTCP interactions. Complementary GNU Prolog ISOBASE and Trealla ACTOR prototypes now exercise selected contracts across runtimes; Section E.6.2 describes their scope and limitations. These experiments have also made the remaining gaps more precise. The following workstreams mix product engineering, normative specification, and open research; each calls for a different kind of evidence.

Production hardening and browser operation.

The demonstrator is not yet a production platform. A serious deployment needs repeatable installation and upgrades, configuration management, persistent state and recovery policies, observability, resource accounting, overload behaviour, performance measurement, compatibility guarantees, and operational documentation. SWI-WASM additionally needs testing across browser engines and a precise account of the isolation, resource, and lifecycle guarantees offered by

its worker-per-actor design. These requirements matter even if no second Prolog implementation ever joins the network.

Option filtering and validation.

The example code sometimes forwards an option list to several predicates without filtering it, to keep the examples readable. Production wrappers should separate spawn, query, receive, and transport options and pass each predicate only the options it accepts. This work includes validating option names and values at the public entry point, reporting unsupported options rather than silently discarding mistakes, and testing calls against remote endpoints as well as local predicates. The resulting behaviour must follow the documented contracts rather than depend on SWI-Prolog ignoring unknown options. The native demonstrator now validates `rpc/3`, `promise/4`, and `yield/3` at entry and restricts their HTTP transport options to a documented subset. Extending this treatment to other option-bearing APIs remains future work. Native `spawn/3` now validates options before local setup or remote dispatch, with isolation-owned options declared through a validation hook. The remote bare-actor endpoint validates client options before source preparation and separates them from server-supplied framework metadata. Source ordering, ordinary option precedence, and existing authority checks are retained. Native toplevel, server, supervisor, and statechart creation wrappers also validate their own settings and forwarded spawn options before starting work; remote toplevel entry points validate before source preparation. These checks preserve framework metadata and existing defaults. Native supervisors also validate complete initial child lists, recognised server and nested-supervisor start forms, and dynamic additions before startup or contact with the supervisor. Defaults, option precedence, and start-goal variable sharing are retained. Local named creation now reserves names and publishes them through the startup handshake; remote naming and broader lifecycle changes remain separate work. Native synchronous toplevel wrappers now separate query and receive options, explicitly select the calling actor as the answer target, and reject conflicting targets before sending. This covers both creation-time defaults and later overrides without tracking remote target state. Request correlation and stale-answer handling remain separate work.

Layered operational semantics.

Chapter 9 separated a Prolog engine's goal-and-substitution state from an actor's mailbox-and-lifecycle state and identified the crossings between them as committed effects: backtracking never unsends a message or returns a selected message to a mailbox. Extending the unified actor semantics of Svensson et al. (2010) with Prolog engine states, followed by a proof of suitable composability properties, is a research problem rather than a completed result.

Profiles and language latitude.

The profile inventories must be completed by the language-level decisions identified in Section 9.2.1: undefined-predicate behaviour, double-quoted terms, numeric limits, error terms, and every other host choice visible at the network boundary. Each decision needs a normative statement and, if interoperability is claimed, a conformance test.

Portable text semantics and migration.

The current SWI-Prolog demonstrator uses native strings for double-quoted literals. The opt-in `portable-text-0.1` draft in `trinity-conformance` proposes lists of one-character atoms instead: `"abc"` would have the same meaning as `[a,b,c]`. This remains an experimental candidate, not a requirement of the cumulative node profiles. Before adoption, a separate SWI test configuration must apply the proposed semantics consistently to goal, option and input readers, supplied source, generated and private modules, and term serialization. Migration also requires an audit of string-dependent examples and predicates, with explicit conversions at native library boundaries where needed. The portable-text cases, including Unicode characters, empty text and round trips, must be run alongside existing actor and interoperability suites before changing a deployed node. The draft and the implementation evidence are described in Section E.6.2.

Ordinary registry lifecycle guarantees.

The namespace and alias semantics of `register/2`, `whereis/2` and `unregister/1` are documented in Appendix B. Native local named creation now reserves a name in the caller's namespace before launch, publishes it before the application goal runs, and releases pending reservations on failure or cancellation. Publication and retirement share the registry mutex, so a fast child cannot be named after its cleanup. Ordinary native `register/2` now checks local actor existence under the same mutex as insertion and retirement, rejecting stale local pids without changing remote alias registration. Neither operation guarantees continued liveness after returning. Native remote-alias cleanup now withdraws bindings across local namespaces on observed termination of the exact pid, or for all pids on a node on observed connection loss. It creates no implicit monitors and retains no retirement records: an unobserved termination can leave an alias, and later registration can recreate one. Connection loss withdraws local names without proving remote death. Explicit unregistration remains necessary where no lifecycle event is observed. Reclamation when a client namespace ends remains separate work. If registration gains automatic remote monitoring, ownership and cleanup of those monitors must also be specified. Regression tests cover namespace isolation, collisions, stale local pids, registration/termination races, observed remote retirement and connection loss. Further conformance tests should cover namespace teardown.

Promise ownership and lifecycle.

The portable ownership boundary for `promise/3-4` and `yield/2-3` remains unresolved, including isolation between clients and collection across separate stateless requests. A common contract must settle portable timeout and invalid-reference outcomes, building on the native deterministic behaviour documented in Appendix B, and make collection and expiry atomic. Queue and background-worker reclamation must cover successful collection, expiry and abandonment; local disposal must be distinguished from remote cancellation. Native request failures now reach the collector as error answers; coordinated expiry must also wake a collector already waiting when its reference is removed. Conformance tests should cover ownership, repeated collection, timeout and retry, expiry races,

abandonment, failure delivery, and failing or nondeterministic timeout callbacks. Appendix B distinguishes these tasks from the native demonstrator's present behaviour.

Security and resource governance.

Chapter 4 established the structural principles of inexpressibility, invisibility, and containment. A deployable public-node profile still needs baseline requirements for authentication, authorisation, transport security, quotas, logging, audit, credential handling, denial-of-service resistance, and adversarial testing. Where suitable web and Internet standards already provide identity, delegated authority, discovery, secure transport, or audit, Web Prolog should bind to them rather than invent parallel mechanisms.

Terms, answers, and continuations at the web boundary.

The transformation between Prolog terms and JSON needs a stable or explicitly negotiated contract covering atoms and strings, variables, compounds, dictionaries, character and numeric rules, cyclic or otherwise unrepresentable terms, and errors. Answer representations should admit structured metadata without changing the core success, failure, and error protocol. The version-1 baseline consists of JSON envelopes and Prolog term text. A future binary term representation must be introduced through an explicitly negotiated media type or protocol capability, together with a complete mapping of Prolog values; it must not be inferred merely from a binary transport frame. Transport compression is orthogonal to that mapping. Offset/limit paging additionally needs a consistency contract: mutation may invalidate a continuation, a point-in-time snapshot may be offered, or an opaque resume token may carry the required state. Generation identifiers and resume tokens are plausible mechanisms rather than settled API. Portable dictionary and URI facilities would support this boundary but belong to the language and library track rather than being silently assumed by the protocol.

Further work on behaviours.

The server, supervisor, toplevel, and statechart examples demonstrate the value of reusable interaction abstractions, but the status of a behaviour remains underdeveloped. More work is needed on callback contracts, syntax, semantics, versioning, introspection, failure behaviour, and the relationship between a behaviour and the protocol it implements. For SXML specifically, Sections 6.7 and 6.10.4 provide a worked starting point. Timed transitions and explicit event deferral now have implementations and stated semantics; the remaining task is to turn those and the rest of the processing model into a portable, testable contract.

Independent implementations and conformance evidence.

The SWI-Prolog demonstrator is one implementation lineage. If the goal is a federated Prolog Web, at least one independently engineered implementation of each proposed interoperability layer is needed to reveal hidden assumptions. A shared test suite and implementation report should record feature-by-feature agreement rather than reduce conformance to a single badge. This item is a condition of a federation claim, not a prerequisite for improving the SWI-Prolog platform itself.

The items have different owners, standards of evidence and time horizons, and should be judged separately.

11.2 Two routes from prototype to platform

The distinction between a production platform and a federation is important enough to make explicit. Both routes begin with the same demonstrator and can share protocols, tests, documentation, and much implementation work. They diverge in what counts as success.

11.2.1 A production-oriented SWI-Prolog platform

On the first route, the immediate goal is a dependable SWI-Prolog-specific platform. SWI-Prolog's threads, engines, module system, sandbox, HTTP stack, WebSocket support, browser port, libraries, and development tools are assets to be used fully rather than hidden merely because another Prolog system may not offer the same facilities. Success is measured by security, predictable semantics, performance, operability, useful services, and the experience of people building real applications.

This route does not make profiles or protocols irrelevant. A stable boundary between clients and nodes remains valuable even when every node uses the same engine. Profiles separate trusted from untrusted execution, make browser and server behaviour intelligible, support versioning, and give tests a public contract. What changes is the freedom to state explicitly that a facility is SWI-Prolog-specific and to optimise the platform around that fact.

A successful platform could later supply the implementation experience from which a portable core is extracted. It need not postpone useful engineering until consensus exists elsewhere.

11.2.2 A federated and standardised Prolog Web

On the second route, the defining goal is cross-system interoperability. Implementation latitude must then be constrained wherever it becomes visible at a node boundary. Profiles must be independent of SWI-Prolog defaults, term and error encodings must be portable, protocol state must be specified, and a second implementation becomes indispensable evidence.

Federation also changes the social problem. The work can no longer be owned by one implementation community alone. Other Prolog implementers must be able to alter the contract, reject assumptions, and demonstrate different internal strategies. If formal standardisation is eventually pursued, public drafts, issue tracking, confor-

mance tests, and implementation reports become part of the engineering rather than administrative decoration.

This route is more ambitious institutionally and may be better developed in a separate specification project. It nevertheless remains technically connected to the production route: a well-run SWI-Prolog platform can be its first source of experience, while a carefully designed portable boundary can prevent that platform from becoming needlessly isolated.

11.2.3 A decision that need not be made all at once

The routes are neither mutually exclusive nor automatically compatible. A feature may begin as an SWI-Prolog extension, acquire a documented remote contract, and only later prove suitable for a shared profile. Another may rely so deeply on one runtime that standardising it would destroy its advantage. The discipline required now is therefore classification: mark what is platform-specific, what is experimentally portable, and what is claimed as a stable interoperability contract.

11.3 From incubation to standardisation

This section is conditional. It describes a plausible path *if* Web Prolog is developed as shared, cross-system infrastructure. A production SWI-Prolog platform can and should proceed without waiting for that path to be chosen.

11.3.1 What should be standardised, and when?

It is too early to standardise the whole Trinity. It is not too early to publish small observable contracts against which independent implementations can experiment: profile boundaries, term and error encodings, node discovery, request and answer protocols, actor lifecycle rules, and the tests that reveal disagreement. If these surfaces remain implicit, they may still become de facto standards, but through accidental dependence on the first implementation.

The caution is close to home. ISO/IEC 13211–2:2000 standardised Prolog modules, yet widely used systems continued to expose materially different module facilities.[1] The lesson is not that module standardisation was pointless, but that a paper contract alone does not create convergence. Proposed surfaces should be incubated beside implementations, tests, and migration experience; speculative features should not be frozen in the hope that implementations will follow.

[1] ISO/IEC 13211–2:2000, `https://www.iso.org/standard/20775.html`.

11.3.2 Incubation and institutional division of labour

The first venue need not be a formal standards body. Public drafts, issue tracking, executable tests, and implementation reports can begin in a project open to all affected implementers. For language and library facilities shared by more than one Prolog, the Prolog Improvements Forum and its PIP process provide a natural incubation path.[2] A solution cursor or dictionary interface should accumulate implementation experience there before being presented as a settled language abstraction.

The existing Prolog standards belong to ISO/IEC JTC 1/SC 22, whose Prolog working group is WG 17. That work is active: ISO/IEC TS 13211–3:2025 standardises definite-clause grammar rules, and a fourth part concerning Unicode characters in quoted Prolog text is under development.[3] The name *ISO Prolog 2.0* used in this book remains a design hypothesis, not the title of an approved ISO project.

W3C supplies machinery for web architecture, formats, browser-facing APIs, and Recommendation-track standards. A W3C Community Group could provide an open place to incubate specifications, tests, and use cases, but Community Group reports are not W3C standards.[4] A later standards track would require broader support, a chartered Working Group, implementation evidence, and public review. There is presently no W3C Web Prolog project.

IETF supplies a third coordinate. Its remit includes much of the Internet substrate on which inter-agent communication depends: transport, authentication and authorisation, discovery, and security mechanisms. These determine how a service is located, how delegated authority crosses organisational boundaries, and what evidence remains after a consequential action. Web Prolog should reuse suitable Internet mechanisms rather than rebuild them.

No single standards body naturally owns the whole problem. ISO has the language remit and Prolog expertise; W3C has web-application and Recommendation-track machinery; IETF has the Internet protocol and security substrate. If formal standardisation becomes appropriate, a division of labour is more credible than assigning the Trinity wholesale to one organisation.

The division is conceptual before it is bureaucratic. ISO is the natural forum for questions such as a future common Prolog base, solution cursors, dictionary terms, and profile-defined language choices. W3C is a possible forum for browser APIs, web-application integration, and the statechart lineage. IETF work is relevant to transport bindings, delegated authority, discovery, and cross-domain security. Term encodings, the actor layer, and the resumable-toplevel protocol cross these boundaries and require coordination rather than a simple hand-off.

---

[2] `https://prolog-lang.org/ImplementersForum/PIPs`

[3] ISO/IEC JTC 1/SC 22 work programme: `https://committee.iso.org/committee/45202/x/catalogue/p/0/u/1/w/0/d/0`.

[4] `https://www.w3.org/community/`

Fig. 11.2 Turning the Trinity design into shared infrastructure may require a liaison between language and web communities. The image shows one part of a wider institutional picture and is not an indication of endorsement.

Coordination between W3C and ISO/IEC is not unprecedented. W3C's WCAG 2.2 was processed through JTC 1 as a Publicly Available Specification and published as ISO/IEC 40500:2025.[5] So the path exists.

In terms of banners, the possible division can be pictured as two related projects:

Fig. 11.3 Provisional banners for a future general-purpose Prolog project and a web-facing Web Prolog project. Neither denotes an existing standard or an approved standards activity.

11.3.3 The SXML notation and SCXML

Although SXML is treated in this book as a specialised notation within Web Prolog, its statechart semantics have a distinct standards ancestry in SCXML. It may therefore require a separate specification track without constituting a separate implementation language.

The statechart strand has a clearer ancestor than Web Prolog itself. SCXML 1.0 became a W3C Recommendation in 2015 (Barnett et al., 2015). Chapter 6 implemented its transition-selection and run-to-completion model while making the scripting language, event matching, failure semantics, and actor invocation explicit. SXML is not a conforming SCXML binding, but its execution model and notation are specified here: Section 6.7 summarises the execution model, Section 6.10.4 explains the rationale for the adaptation, and Appendix C specifies the notation element by element, with its relationship to SCXML tabulated in Section C.11.

[5] `https://www.w3.org/press-releases/2025/wcag22-iso-pas/`

The deeper proposal is that a successor should retain a declarative, inspectable state-machine core while allowing a capable host language to carry more of the executable content and communication. Timed transitions and state-scoped event deferral are SXML additions whose value and portability would have to be assessed alongside that core. The current author served as an invited expert and editor in the W3C Working Group that produced SCXML 1.0. That experience motivates the criticism, but confers no present W3C status on SXML.

The important research question is whether the smaller division of responsibility demonstrated by SXML remains intelligible, inspectable, and portable with more than one host language. A detailed proposal for an actor-oriented SCXML successor would be substantial enough to stand as work in its own right.

11.4 The first specification targets

If the federation route is pursued, the targets should be layered so that each stage yields an independently testable boundary:

1. Publish the profile inventories, language-choice ledger, common term and error vocabulary, extension rules for answer metadata, and conformance-test format.
2. Specify RELATION over HTTP as the smallest cross-system query surface, including paging, consistency claims, and JSON representation rules.
3. Specify the resumable toplevel and PTCP lifecycle: start, call, continue, stop, abort, success, failure, error, output, and prompt.
4. Specify the ACTOR substrate and its WebSocket binding: identity, send and selective receive, monitors, lifetime dependence, termination, local names, public service names, authorisation, and containment.
5. Incubate SXML separately against its stated SCXML relationship and a dedicated conformance suite.

This order does not imply that implementation must occur sequentially. It says only that later specifications should reuse earlier contracts rather than define competing vocabularies.

11.4.1 The actor substrate as the distinctive target

`spawn/1-3` is the point at which a Prolog goal becomes an addressable, communicating entity. The ACTOR specification must define the observable contracts of `self/1`, `!/2` and `send/2-3`, `receive/1-2`, monitoring and demonitoring, parent–child lifetime dependence, termination, local registration, and public service publication. It must preserve the deliberate differences from Erlang established in the actor chapters rather than silently inherit the semantics of one host implementation.

11.4.2 Actor identity, messaging, and lifecycle

Identity must remain globally meaningful when a pid crosses a node boundary. The constructible `Id@Node` form is part of the observable contract: construction and decomposition are supported, while creating the term alone does not create an actor, establish liveness, or confer authorisation. Message delivery must define per-sender ordering, no-process behaviour, and renamed-apart copying with aliasing preserved inside each message but never between processes or separate sends. Receive must define tentative matching, binding rollback for rejected candidates, once-only guards, the treatment of guard exceptions and effects, deferral, timeout, and committed mailbox effects. These rules require conformance tests that exercise both local and remote placement. Monitoring must standardise the `down/3` vocabulary. Local registration and the owner-controlled `register_service/2` and `unregister_service/1` namespace must remain distinct. Authorisation and lifetime containment are part of the observable Web contract, not merely deployment advice.

11.4.3 The semantics of `spawn/3`

The `spawn/3` options expose several of the design's trust decisions. Monitoring is atomic when requested at creation. Parent–child dependence is the default, while an unlinked remote actor requires owner permission. Source loading, module or knowledge-base selection, time limits, and sandbox policy determine the execution context. The child must execute a renamed-apart copy of the supplied goal, so its bindings cannot instantiate variables in the parent, regardless of placement. A specification should classify each option as portable, profile-specific, owner-only, or implementation-specific and say which failures occur before a pid is returned.

11.4.4 The resumable toplevel actor

The toplevel protocol is the bridge between nondeterministic Prolog execution and deterministic network messages. A call establishes one computation; success messages carry slices; `toplevel_next/1-2` resumes that same computation; stop discards the continuation; abort interrupts a running goal. Recomputation with a larger offset is an implementation strategy only when its stability and side-effect restrictions satisfy the contract. It is not the semantic definition of continuation. As Section 6.10.3 observed, this suspended-computation substrate is what keeps the toplevel a built-in behaviour rather than a statechart encoding.

This modest protocol is rich enough to support remote shells, notebooks, browser clients, RPC, teaching environments, and distributed services. It is also valuable in a

single-implementation platform: resumability is an interaction property, not merely an interoperability concession.

11.4.5 Solution cursors and the ISO liaison

The protocol points toward a possible language-level abstraction: a solution cursor that creates a resumable all-solutions computation, returns the next slice, and can be closed. A PIP developed with more than one implementation community would be a natural first step. ISO WG 17 is a plausible later forum for deciding whether the abstraction belongs in a future Prolog language standard. Web-facing work would specify how a cursor-backed toplevel is represented, controlled, secured, and tested. The protocol creates a use case for the language abstraction; it does not predetermine predicate names or implementation.

Such work should start from existing engine interfaces. Logtalk's `threaded_engine_*` predicates already provide engine creation, successive answer retrieval, and explicit destruction (Moura, n.d.). SWI-Prolog likewise provides `engine_create/3-4`, `engine_next/2`, and `engine_destroy/1`.[6] The standardisation question is a shared contract for resumable search, including batched slices, errors, and lifetime, informed by those interfaces and experience across implementations.

11.4.6 Layering the first standards

The central rule is that shared contracts fix observable behaviour while leaving implementations free behind the boundary. Actors may be implemented using threads, lightweight processes, event loops, or browser workers. Resumable queries may use engines, continuations, virtual-machine support, or a carefully bounded recomputation strategy. Those choices become standards matters only when they change what another participant can observe.

11.5 Implementation and interoperability

> We are not analyzing a world, we are building it. We are not experimental philosophers, we are philosophical engineers.
>
> *Tim Berners-Lee, 2003*[7]

[6] `https://www.swi-prolog.org/pldoc/man?section=engine-predicates`

[7] `https://lists.w3.org/Archives/Public/www-tag/2003Jul/0158.html`

11.5.1 Why implementation comes early

Implementation is a design instrument, not a ceremony performed after the specification. Running systems expose ambiguous errors, lifecycle races, unstated ordering assumptions, performance cliffs, and security boundaries that prose can hide. The demonstrator has served that role throughout this book. It should not be frozen as the standard merely because it arrived first, nor dismissed as "only a prototype" when its experiments reveal real design constraints.

On the production route, the next evidence comes from sustained use: workloads, failure injection, upgrades, monitoring, and applications maintained over time. On the federation route, the decisive evidence comes from another implementation meeting the same boundary. Both kinds of evidence can overturn the design, but they answer different questions.

11.5.2 Independent implementations and test suites

The GNU Prolog and Trealla prototypes provide different implementation strategies for selected query and actor contracts. Their reported interoperability tests are an initial result, not evidence that every requirement of ISOBASE or ACTOR has been met. The `trinity-conformance` runner adds implementation-neutral tests through the nodes' public interfaces; its current draft contracts cover only the listed cases (Section E.6.2).

For federation, the next task is to extend that coverage and obtain independently engineered implementations and validation by other groups. A further implementation can begin with RELATION and its term/error encoding without attempting the full ACTOR profile. The conformance suite should link every normative requirement to tests and publish an implementation matrix. Disagreement then becomes evidence: the prose is ambiguous, a requirement is overconstrained, or an implementation is wrong.

For the SWI-Prolog platform, redundancy means something different: separate clients, deployments, browser engines, security reviews, and applications exercise the same implementation under varied conditions. A platform can become robust without becoming portable. It should simply avoid presenting that robustness as proof of cross-system interoperability.

11.6 AI-assisted implementation and standardisation

The demonstrator and this manuscript were developed with extensive assistance from language models, including ChatGPT, Claude, and Gemini. That experience supports a bounded claim: AI assistance can reduce the cost of producing and reviewing code, tests, documentation, and alternative designs. It does not transfer responsibility for

architecture, protocol semantics, security, or standards decisions. Several model-generated implementations also do not count as independent evidence when they share the same specification assumptions and human direction.

AI assistance was most effective once the intended boundary was explicit: constructing interfaces, generating routine code, proposing variants, debugging transport interactions, and keeping the demonstrator aligned with revised contracts. It was less reliable when asked to infer the contract from an incomplete implementation. This reinforces the need for small stated surfaces and executable tests.

Language models can generate protocol variations, boundary cases, and hostile inputs cheaply, but generated tests require an independently stated oracle. An AI editorial agent can also check terminology, cross-references, normative keywords, requirement identifiers, test coverage, and implementation reports, and help keep code, manuals, and manuscript examples aligned as the design changes. For sandboxing in particular, these uses supplement rather than replace threat modelling, code review, and deployment hardening. Human participants must resolve issues and own every normative decision. It is tempting to let Trinity agents help maintain Trinity specifications, but humans must still own the decisions.

11.7 Shared ownership

Chapter 10 considered the conditions for adoption: useful services, visible examples, documentation, and places where people can meet the technology. The narrower question here is who acquires the right to shape it. On the production route, the first constituency can be SWI-Prolog users, application builders, and operators who subject one implementation to sustained use. On the federation route, independent Prolog implementers must participate from the beginning, because interoperability cannot be established by one implementation speaking to itself.

The neighbouring communities considered in Chapter 10 are therefore partners rather than audiences. Formal-semantics researchers, Semantic Web practitioners, agent researchers, Erlang implementers, browser engineers, security reviewers, and web developers possess expertise that may change the proposal. Shared ownership means more than help with implementation: it includes the ability to challenge terminology, reject assumptions, narrow requirements, and propose different boundaries.

Protocols do not manufacture a community, but shared objects can give one something concrete to negotiate: a useful platform, a public service, a failing conformance test, a disputed requirement, or an application that could not easily have been built before. The work becomes communal when those objects can answer criticism from people who did not design them.

11.8 Closure

The routes described in this chapter allow several kinds of success. A production SWI-Prolog platform can be worthwhile even if federation never follows. A portable profile can be valuable without determining the internals of every Prolog system. Statechart work may develop alongside either route or independently of both. Keeping these paths distinct makes cooperation possible without making the largest vision a precondition for every smaller success.

Measured against the five goals stated in Section 1.4, the present result is uneven. The actor-oriented concurrency model has been specified and exercised in the SWI-Prolog demonstrator and ported in part to Trealla. The GNU Prolog and Trealla prototypes provide initial cross-system evidence for querying and, in Trealla's case, actor interaction. Complete profile conformance and independent validation remain to be established. Web Prolog defines a candidate lingua franca whose shared boundaries now have preliminary tests across runtimes; broader implementation and adoption are still needed. The examples provide a substrate for web agents and hybrid symbolic–neural systems, without constituting a complete high-level agent language. Finally, the demonstrator provides an interactive node environment, but still requires engineering to become the secure, production-quality, turn-key node envisaged in the introduction. The five goals separate what has been demonstrated from what still requires engineering, independent implementation, or adoption.

Execution continues throughout the development of a design. Users, independent implementations, hostile environments, and operational limits expose assumptions that running examples alone cannot test. Designs must then be revised, constrained, or sometimes abandoned. Some parts of the Trinity may mature quickly, others may remain speculative, and still others may prove mistaken. Shared infrastructure earns trust by making these outcomes visible.

The book's contribution is to make a design explicit, embody much of it in a running demonstrator, and identify boundaries at which others can disagree. The next step is to publish the contracts, tests, implementation reports, and unresolved questions in forms that invite independent implementation and critical review. The provisional banners with which this chapter opened mark possible institutional homes for that work; any authority would have to come from the public processes of the communities concerned.

An ecosystem cannot simply be declared into existence. If the Trinity becomes infrastructure, it will be because a community made it answerable to reality.

Appendix A
Glossary

This appendix gives working definitions for the principal terms used across the book. The definitions are specific to the Prolog Trinity where that usage differs from a broader one. They describe the architecture first and the current implementation where this helps to make the concept concrete. Individual predicates, options, commands, and endpoint fields remain documented in Appendix B; they are included here only when they also name an important architectural abstraction.

Actor. An actor is an independently scheduled concurrent process with a pid, a mailbox, and an explicit message interface. It executes a goal until that goal completes or the actor is terminated. Actors may be short-lived workers or durable services. Actors such as toplevels and statechart actors are specialised Erlang-style forms.

Agent. An agent is an addressable participant in interaction, defined by its interface and behaviour rather than by its implementation language. In the book's deliberately modest architectural sense, actors (toplevels, servers, supervisors, statechart actors, . . .) as well as nodes are all Prolog agents because they receive goals or messages expressed as Prolog terms and respond with answers, messages, or effects in the same term-based form. Qualified terms such as *BDI agent* and *LLM agent* retain their richer meanings from AI.

Agentic Web. The Agentic Web is the emerging conception of a Web in which autonomous, communicating processes operate alongside documents, services, and human users as first-class participants. The term does not yet denote one settled architecture or standard. The Prolog Web is proposed as one possible execution substrate within this larger design space.

Answer term. An answer term is a PTCP message reporting the outcome of a toplevel computation. It has one of three forms: `success(Pid,Slice,More)`, `failure(Pid)`, or `error(Pid,Error)`. In a success term, `Slice` is a solution slice: a finite, ordered list of instances of the call's answer template, containing at most the configured number of consecutive solutions; `More` says whether a request to continue may produce another slice. A failure term reports that no solution, or no further solution, was found. An error term carries the exception raised by the

computation. Stateless calls use the corresponding forms `success(Slice,More)`, `failure`, and `error(Error)`, without a toplevel pid.

Behaviour. A behaviour is a reusable control pattern for actors. The generic part supplies such machinery as message handling, state threading, monitoring, lifecycle control, and cleanup; an application specialises it through a callback, child specification, or statechart model. Servers, supervisors, toplevels, and statechart actors are the principal behaviours in the book. Predicate-oriented generics such as `parallel/1` use actors internally but expose an ordinary Prolog predicate interface instead of a message protocol.

Browser node. A browser node is a Web Prolog node whose execution environment is a browser. In the demonstrator's default architecture, a JavaScript node controller runs on the main thread and each local actor runs an independent SWI-Prolog WASM instance in a web worker. The design preserves the same pid and message abstractions for actors in the current tab and actors on remote nodes. A separate main-thread mode exists for self-rendering Prolog pages that require direct DOM access.

Capability. In the security sense used in this book, a capability is an unforgeable reference whose possession confers authority to perform particular operations on a resource. A pid is not a capability; it is a constructible address: knowing it grants messaging authority only where node policy permits a holder to send to that actor without further authorisation. Where policy requires additional credentials or permissions, the address alone is insufficient. Neither addressability nor pid secrecy establishes unforgeability. A restricted capability endpoint combines a protected reference with enforcement of its permitted operations and scope. The internal terminal endpoint, for example, uses an unguessable token for one terminal lineage and requires authenticated node transport; it is not a general actor address. The node's authorisation layer separately assigns named policy permissions, such as `execute`, `admin`, and `internal_transport`, to a principal. Capability values and policy permissions are different mechanisms, although both delimit what a client is authorised to do.

Committed effect. A committed effect is an effect that crosses the boundary of Prolog backtracking and is not undone when a later goal fails. Sending a message, selecting and removing a message from a mailbox, spawning an actor, and performing external I/O are examples. The distinction is central to the separation between Prolog engine state and actor process state.

Concurrent, sequential, and programmable Prolog Web. These are three complementary views of the Prolog Web. The *concurrent* Prolog Web provides remote actors and message passing. The *sequential* Prolog Web presents remote computation as nondeterministic calls, which may or may not be realised with actors. The *programmable* Prolog Web lets code and data accompany calls or spawned actors. They are layers of one architecture rather than separate networks.

Configuration. A statechart configuration is the set of states currently active in the machine. In a hierarchical chart it includes the active leaf states and their ancestors;

parallel regions may contribute several active leaves simultaneously. Transition selection, event visibility, deferral, entry and exit, and history restoration are all defined relative to the current configuration.

Conformance. Conformance is satisfaction of the normative requirements of a declared profile or specification. A RELATION-conforming node is not deficient because it does not implement ACTOR; it has made a narrower promise. Credible conformance requires executable tests and feature-by-feature implementation reports, especially when interoperability between independent implementations is claimed.

Containment. Containment is the structural security discipline that bounds client-created computation in lifetime and effect. By default, a spawned actor is lifetime-dependent on its parent. Its linked subtree is reclaimed once parent termination or loss of the owning connection is detected. A lease bounds detection of lost contact while the runtime services the connection, but does not prove remote completion. Long-lived detached services are owner-managed exceptions rather than the default.

Distribution-first. Distribution-first is the design stance that placement, remote identity, latency, partial failure, and trust boundaries belong to the programming model from the outset. It does not mean pretending that local and remote execution have identical costs. Web Prolog keeps the same core interaction constructs across node boundaries while making the network's timing and failure boundaries observable.

Event deferral. Event deferral postpones an event that is not to be handled in the current control state. In ordinary selective receive, a non-matching mailbox message remains available to later receive operations. In SXML, an unmatched selected event is discarded by default, while a `<defer>` declaration explicitly protects matching events and offers them again after the active configuration changes.

Eventless transition. An eventless transition is considered without requiring an incoming or internally raised event. It may be unconditional or guarded by a Prolog goal and is used for automatic progress, completion-style steps, and data-driven decisions. Eventless transitions run inside the current macrostep; an unending enabled cycle creates livelock.

Federation. Federation standardises what independently developed systems share at an interaction boundary without requiring their internals to become identical. In the Trinity, profiles, protocols, representations, and observable behaviour provide the shared surface, while different Prolog systems retain their own extensions, optimisations, libraries, and runtime architectures.

Flat SXML. Flat SXML is the subset of SXML without compound states, parallel regions, or history pseudostates. It retains ordinary states, transitions, executable content, the data model, timed transitions, event deferral, and run-to-completion execution. Every Flat SXML document is also a full SXML document; the name identifies a restricted vocabulary, not a separate language or behaviour.

History state. A history state is a statechart pseudostate that records which subconfiguration of a compound state was active when that state was left. A later transition to the history state restores that configuration instead of re-entering the compound state's ordinary initial state. It is useful for interruption, resumption, and conversational repair.

Host isolation. Host isolation constrains the runtime process outside the Web Prolog language. Operating-system confinement, containers, Web workers, WebAssembly runtimes, and WASI capability boundaries are possible mechanisms. Host isolation is a backstop to language sandboxing, not a substitute for it: one limits what the process can reach, while the other limits what submitted code can express.

Hot code replacement. Hot code replacement changes the callback used by a running server while preserving the server actor and its current state. It separates application logic from the generic protocol loop, but it is safe only when the replacement accepts the existing state representation or performs an explicit migration.

Inexpressibility. Inexpressibility is the structural security property that dangerous operations cannot be expressed in client code. The exposed language fragment excludes direct file I/O, raw operating-system or socket access, durable node storage, and other ambient-authority mechanisms. Profile checks and sandbox policy approximate this boundary in the current node.

Instant portability. Instant portability is the limiting case of portability in which relocation requires no adaptation. A program can be downloaded for local execution or shipped to another node immediately because all its dependencies are bundled with it or guaranteed by the target profile.

Interaction portability. Interaction portability is preservation of a program's permitted observable behaviours when it is relocated across a declared class of execution environments under stated dependencies. The relevant observations may include queries, answers, effects, messages, and lifecycle events. Preservation concerns the permitted set of behaviours, not identical timing or one fixed trace.

Interaction surface. An interaction surface is the externally visible contract through which systems meet: routes, APIs, message and term formats, protocol rules, errors, timing behaviour, and profile promises. The term emphasises that interoperability depends on shared boundaries rather than shared internals.

Interoperability. Interoperability is the ability of independently implemented systems to work together correctly through shared protocols, representations, and behavioural contracts. It is stronger than syntactic compatibility: a client must be able to rely on the promised observable behaviour at the advertised profile boundary.

Invisibility. Invisibility is the name discipline that limits discovery through public client interfaces. In ordinary public client execution, `actors/1` intentionally lists actors belonging to the same client interaction; it does not reveal actors belonging to other clients or private owner resources. Node-owned runtime or administrative execution has broader visibility. Node-assigned pids and owner-curated service publication support this name discipline, but non-disclosure is not an access-control

guarantee. A secret pid is not, by itself, a replacement for authentication or authorisation.

Link. A link in Web Prolog is a unidirectional lifetime dependence from a child actor to its parent. With the default `link(true)`, parent termination forces the child to terminate, and dependence propagates down the actor tree. Child termination does not kill the parent. This deliberately differs from Erlang's bidirectional links; detached `link(false)` actors are reserved for node owners and rehomed under the node, with output sent to its log.

Logical update view. The logical update view is the Prolog database semantics under which a predicate call sees the set of clauses that existed when the call began, even if clauses are asserted or retracted while its remaining solutions are enumerated. A retained continuation can preserve this per-invocation view across solution slices. It does not by itself give an arbitrary multi-predicate goal a transaction-wide snapshot, since predicate calls first entered later may see later updates. Stateless offset-based recomputation may lose even the per-invocation continuity unless the underlying relations remain stable; a point-in-time guarantee requires an additional transaction or versioning mechanism.

Mailbox. A mailbox is an actor's queue of incoming messages. Sending is asynchronous: a message is placed in the destination mailbox and the receiving actor selects it later. Web Prolog guarantees per-sender FIFO within one stable pid or name-addressed routing epoch, but not across mixed addressing, name rebinding, reconnects, different senders, or the signal channel. Selective receive may leave older non-matching messages in the mailbox.

Monitor. A monitor is a one-way observation of another actor's lifecycle. When the observed actor terminates, each monitor produces a `down(Pid,Ref,Reason)` message, where the reference identifies the monitor instance. Monitoring reports termination but creates no lifetime dependence. Installing `monitor(true)` during spawning avoids the race between process creation and a later call to `monitor/2`. For a remote monitor, `connection_closed` reports that observation was lost, not that the target was observed to terminate; the monitor is then discharged.

Multi-agent system. A multi-agent system is a system whose behaviour arises from several interacting agents. The Prolog Web qualifies in a minimal architectural sense because nodes and actors have identities, private state, protocols, and concurrent lifecycles. This says nothing by itself about autonomy, rationality, or the BDI model traditionally associated with multi-agent systems research.

Mutual constraint. Mutual constraint is the design principle that the Trinity's language profile, agent model, and network substrate must be designed together. The language limits what can safely cross the boundary; the agent model determines the protocols and lifecycle guarantees required; and the open Web determines which representations, timing assumptions, and trust relations are realistic.

Neuro-symbolic AI. Neuro-symbolic AI combines learned neural components with explicit symbolic representation or reasoning. In the book's architectural patterns, a

model may supply language or perception while Prolog supplies rules, constraints, planning, verification, explanations, or control. The division is useful only where the interface between the two components is explicit and failures remain governed.

Node. A node is a network-addressable host of execution, knowledge, and policy. It exposes one or more Web Prolog APIs, advertises a profile, enforces security and resource policy, and may host shared predicates, sessions, and node-resident actors. A node is also a Prolog agent at a larger granularity: it has a persistent identity, receives requests, returns answers or effects, and participates in the network even if it is not implemented in Prolog.

Node controller. The node controller is the component that manages the node as a collection of actors and sessions. Its responsibilities include pid allocation, actor creation and termination, registries, routing, connection lifecycle, and enforcement of node-level policy. In the browser implementation it is JavaScript on the main thread; actor logic remains in web workers.

Node-resident actor. A node-resident actor is a durable actor installed and started under the node owner's authority rather than borrowed temporarily by a client. Such actors can provide shared state, coordination, or conversational services to many clients. They normally require stable publication, supervision, recovery, and resource policies.

Open-world assumption. Under the open-world assumption, the absence of a statement does not imply that the statement is false. OWL adopts this assumption because independently published knowledge may always be incomplete. Ordinary Prolog commonly uses closed-world idioms such as Negation As Failure within a delimited knowledge base. This semantic difference prevents OWL reasoning and plain Prolog execution from being treated as interchangeable.

OWL. OWL, the Web Ontology Language, is a family of W3C languages for expressing shared ontology semantics, including class hierarchies, property characteristics, and cardinality restrictions. OWL reasoners operate under open-world semantics and, in OWL 2 DL and its profiles, within decidable fragments. The Prolog Web treats OWL reasoning as an interoperable external service rather than attempting to replace it with ordinary Horn clauses.

Node owner. A node owner is the authority responsible for a node's public surface and long-lived resources. The node owner installs shared predicates, publishes or withdraws services, configures profiles and policy, and accepts operational responsibility for durable services, supervision, and recovery. Node-owner authority is a role in the security model, not merely the first client to connect.

Parallel region (orthogonality). Parallel, or orthogonal, regions are statechart regions that are active at the same time within one compound state. An event is visible across the active configuration and may enable transitions in several regions during the same macrostep. This is structured coordination within one actor, not independently scheduled actor-level parallelism.

Pid. A pid is a globally meaningful process identifier for an actor. Its canonical global form is the constructible Prolog compound `Id@Node`, where the node assigns `Id`; a shell may suppress the node component when displaying a local pid. Constructing the term neither creates an actor nor establishes liveness or authority. A valid pid is a routable handle; possession confers messaging authority only to the extent permitted by node policy (see Capability). It is distinct from a descriptive registered or published name.

PIP. A Prolog Improvement Proposal, or PIP, documents a language or library facility supported, or intended to be supported, by more than one Prolog system through the Prolog Improvements Forum. PIPs are not ISO standards. They provide an incremental, implementation-led route for enlarging the portable common ground and may supply evidence for later standardisation.

Portability. Portability is the extent to which source code or an interaction pattern can move among nodes or implementations without semantic breakage or manual rewriting. The Trinity pursues portability through explicit profiles and dependencies rather than demanding complete internal uniformity. See also Interaction portability and Instant portability.

Principal. A principal is the identity on whose behalf a request is made. It is not the same thing as a user, connection, session, or pid. A principal may be an anonymous caller, a human user, a service account, or a trusted internal transport identity. Authorisation maps a principal to the operations it may perform.

Private database. A private database is the actor-local collection of clauses and facts that forms part of an actor's workspace. It can be populated at spawn time through `src_*` options and updated without modifying the node's shared database. Private definitions may shadow shared definitions, allowing one actor to specialise its logical environment.

Profile. A profile is a named, cumulative conformance contract stating which language constructs, predicate families, transports, APIs, and interaction patterns a node provides. It constrains both the node's *language surface* and its *interaction surface*. Profiles make implementation latitude explicit and allow a portable operational subset without turning Web Prolog into a separate fork of Prolog. A profile promise must include observable semantics and conformance tests, not only a feature list.

Profile hierarchy (RELATION, ISOBASE, ISOTOPE, ACTOR). The profile hierarchy is a nested sequence in which each profile extends the preceding one. RELATION exposes relational querying. ISOBASE adds a restricted ISO-style goal language and permits client-supplied source in a call's temporary execution context. ISOTOPE adds persistent toplevel sessions: supplied definitions and dynamic database updates can survive across requests within the session, which also supports interactive I/O. ACTOR adds Erlang-style concurrency, distribution, and actor behaviours. A system need only implement the narrowest profile that matches the surface it can reliably provide.

Prolog engine. A Prolog engine is the execution mechanism that evaluates a goal, maintains variable bindings and choice points, and produces successive solutions. In

the Trinity architecture, engine state is distinct from actor process state: backtracking can restore substitutions and proof search, but cannot unsend messages, unspawn actors, or restore selected mailbox messages.

Prolog Trinity ecosystem. The Prolog Trinity ecosystem comprises Web Prolog as a language profile, Prolog agents as the execution entities, and the Prolog Web as the infrastructure in which those agents interact. The three components are distinct but mutually dependent: the Web is populated by agents, the agents are programmed in and communicate with Web Prolog, and the language acquires its web meaning through the nodes and protocols that host it.

Prolog Web. The Prolog Web is the network formed by network-addressable nodes that host Web Prolog knowledge, execution, actors, and services. It extends the conventional Web with executable relations, nondeterministic remote querying, long-lived communicating processes, and mobile source. It is intended as an independent but interoperable execution stack, not as a replacement for the conventional or Semantic Web.

PTCP. PTCP is the Prolog Toplevel Communication Protocol. It governs conversation with a toplevel actor. A call submits a goal; answer terms report success, failure, or error; and a success term carries a solution slice and indicates whether continuation may produce another. *Next* resumes a retained computation, *stop* discards its continuation, *abort* interrupts a running goal and returns a persistent session to its idle state, and *halt* terminates the toplevel actor. Output and prompts are communicated by PTCP messages.

PTCP over HTTP. PTCP over HTTP is the semi-stateful binding that maps PTCP onto the `/toplevel_*` endpoints. A session pid identifies the persistent toplevel, while each HTTP exchange remains an ordinary client-initiated request and response. Answer terms, output, prompts, and acknowledgements become ordered response events. After receiving output, the client polls for the next event; after receiving a prompt, it supplies input with a *respond* request. The additional prompt-waiting state and this retrieval choreography belong to the HTTP mapping, not to core PTCP; see Section E.2.2.

Public surface. A node's public surface is the set of predicates, services, routes, behaviours, and policies it intentionally exposes to clients. It is smaller than the node's internal runtime and should be an explicit, auditable design choice. Everything outside it should be inexpressible, invisible, contained, or protected by deployment policy.

Pure Web Prolog and the pure Prolog Web. *Pure Web Prolog* is the pure fragment obtained when pure Prolog is extended with `rpc/2-3` calls to pure, stable remote predicates. The *pure Prolog Web* is the aspirational distributed logical environment formed by such fragments. Network failure may truncate an answer stream, but does not change the logical status of answers already derived from the stated pure context.

RDF. The Resource Description Framework is the Semantic Web data model in which information is expressed as subject–predicate–object triples. Resources and

properties are identified globally by IRIs. In SWI-Prolog's Semantic Web libraries, the elementary triple pattern maps naturally to the relation `rdf/3`; those libraries are host-provided facilities rather than part of the standard Web Prolog profiles.

Registered name. A registered name is a symbolic name bound to an actor for lookup and messaging. Ordinary client-side registration is distinct from owner-controlled service publication. The former provides a convenient name within the applicable registry; the latter deliberately adds a durable service to the node's public namespace.

Reply channel. A reply channel is an endpoint created to receive one reply or a narrowly bounded class of replies. A temporary proxy actor can serve this role. Across a trust boundary, a reply channel avoids disclosing the client's general pid and grants only the authority needed for the current protocol step.

Resource governance. Resource governance controls how much work an otherwise permitted computation may consume. Time limits, memory ceilings, spawn limits, quotas, admission control, overload policy, and accounting are examples. Resource governance is distinct from sandboxing: a goal may use only allowed operations and still exhaust CPU, memory, process slots, or bandwidth.

RPC meta-predicate. `rpc/2-3` is the nondeterministic remote procedure-call interface of the sequential Prolog Web. A caller submits a goal to a node and consumes its solutions through ordinary Prolog backtracking. The current SWI-Prolog implementation uses stateless HTTP and sliced retrieval; an actor-based reconstruction shows how the same abstraction can be expressed through a toplevel actor and PTCP.

Run-to-completion (macrostep). Run-to-completion is the statechart discipline in which one external event is processed through a finite cascade of transition microsteps, eventless transitions, and internally raised or re-offered events before the next external message is selected. The complete reaction is a *macrostep*. When it terminates, the machine has reached a stable configuration; if it does not terminate, the statechart is livelocked.

Sandboxing. Sandboxing is the language-level enforcement of which goals submitted by an untrusted client may run. Whitelists, profile checks, meta-call validation, and restrictions on dynamically constructed goals are possible mechanisms. Sandboxing answers *what may this program do?*; host isolation and resource governance answer different questions and remain necessary.

Scoped inference. Scoped inference evaluates a goal with respect to a delimited logical context, such as the knowledge base at a named node. In `rpc(URI,Goal)`, the URI identifies the context whose predicates support or fail to support the goal. Delimiting scope is particularly important for Negation As Failure, since the open Web cannot be treated as one complete closed-world database.

SCXML. SCXML is the W3C State Chart XML Recommendation, an XML-based state-machine language with a specified transition-selection and run-to-completion processing model. SXML draws on that execution model but is not a conforming SCXML language binding: it adopts Web Prolog terms, unification, actor invocation, and a smaller surface syntax.

Selective receive. Selective receive scans an actor's mailbox for the oldest message that matches one of the current receive clauses, including any guard. Non-matching messages remain available for later receive operations. Once a message is selected, its removal is committed even if the selected clause body later fails or backtracks.

Semantic Web. The Semantic Web is the W3C-associated vision and technology stack for a Web of machine-processable data with shared identifiers, vocabularies, and formally stated meaning. RDF supplies a common data model, SPARQL a query language, and OWL ontology semantics. The Prolog Web complements this representation-centred stack with executable relations, actors, and services.

Sequential and concurrent partitions. The *sequential partition* is the world of goals, relations, substitutions, backtracking, and nondeterministic answers. The *concurrent partition* is the world of actors, mailboxes, messages, and lifecycle events. Implementations may realise a sequential abstraction using actors, and actors may reason sequentially, but the programming model keeps the two partitions distinct so that proof search is not confused with interaction state.

Server. A server is a stateful actor that maintains private state and serves clients through a request–response message protocol. The generic server behaviour provides routing, correlation, state threading, monitoring, upgrade, and shutdown machinery; an application callback defines how one request transforms the old state into a response and a new state. It is an Erlang-style protocol abstraction rather than a logical predicate interface.

Service. A service is a durable, shared facility intentionally offered by a node for repeated use by multiple clients. A service may expose a predicate or an actor protocol. An actor service differs from an arbitrary named actor because the node owner publishes it under a stable public name and accepts responsibility for its lifecycle and policy.

Service discovery. Service discovery is the process by which clients learn which published services a node offers. It resolves owner-curated public names rather than enumerating all running processes. The book's Discovery Hub illustrates a declarative directory that clients can query through `rpc/2-3`.

Service publication. Service publication is the owner-controlled act of placing a service name in a node's public service namespace. It makes the service part of the advertised public surface and is therefore distinct from ordinary client-side actor registration.

Session. A session is a bounded continuing interaction context in which state may persist across several protocol steps. A toplevel session can retain definitions, I/O state, and a current computation; an ACTOR WebSocket session can retain mailboxes and process relationships. Session lifetime, idle waiting, running computations, and transport connections have related but distinct failure boundaries.

Shadowing. Shadowing occurs when a predicate in an actor's private database takes precedence over a predicate with the same indicator in the node's shared database.

It supports local specialisation without modifying shared knowledge. The precise module and lookup rules are part of the actor's logical-environment contract.

Shared database. The shared database is the node-owned collection of predicates made available as shared knowledge and skills. It is installed by the owner and may be imported into actor contexts or used by node-level query execution. Clients can combine it with private source, but cannot treat it as their own mutable workspace.

Solution cursor. A solution cursor is a proposed language-level abstraction for a resumable all-solutions computation. It would create a computation, return bounded slices on demand, and support explicit closing. PTCP supplies the motivating web-facing use case, but the book does not claim that cursor predicate names or language semantics have already been standardised.

SPARQL. SPARQL is the W3C query language for RDF graphs. It defines graph-pattern matching together with projection, grouping, ordering, aggregation, optional matches, and other operations under specified multiset and error semantics. Similar-looking Prolog queries can be concise and composable, but are not automatically semantics-preserving translations of SPARQL.

Statechart actor. A statechart actor is an actor whose control logic is given by an SXML statechart document. Incoming messages become external events that drive an explicit hierarchy of states and transitions. The actor retains an ordinary pid and mailbox and can be monitored, supervised, and invoked like other actors.

Stateless, semi-stateful, and stateful APIs. The *stateless HTTP API* completes each request independently and retrieves solution slices by request parameters. It underlies both synchronous `rpc/2-3` and the split-phase `promise/3-4` and `yield/2-3` abstraction. The *semi-stateful HTTP API* maps PTCP onto a family of HTTP endpoints whose requests address the same persistent toplevel actor by pid. The *stateful WebSocket API* maintains a bidirectional connection through which actors, mailboxes, and PTCP conversations persist. These APIs, and the abstractions layered over them, expose different blocking and failure boundaries.

Supervisor. A supervisor is an actor whose primary responsibility is to start, monitor, terminate, and restart child actors according to a declared policy. It embodies the "let it crash" discipline: application workers may fail while the supervisor supplies recovery structure. Supervisors are especially important for durable node-resident services.

SXML. SXML, or Statechart XML, is the book's compact, SCXML-inspired statechart notation within Web Prolog. It provides statechart-specific control structure while using Web Prolog as its data model and executable content, Prolog terms and unification for events, explicit actor spawning for invocation, and SCXML-derived transition selection and run-to-completion semantics. Timed transitions and state-scoped event deferral are deliberate SXML additions.

Timed transition. A timed transition is an SXML transition with an `after` attribute. Its timer is armed when the source state is entered and cancelled automatically when

that state is exited. The timeout therefore belongs to one activation of a control state, avoiding the manual send-and-cancel bookkeeping otherwise needed to deliver a delayed event.

Timing and failure boundary. A timing and failure boundary is an operation at which waiting becomes observable and may end through timeout, disconnect, cancellation, or remote failure. A stateless HTTP request, a `yield/2-3` wait, a semi-stateful session request, and a mailbox receive are different boundaries even when they ultimately concern the same computation. Timeout values and recovery actions therefore belong to the interaction model, not merely to transport defaults.

Toplevel actor. A toplevel actor is an actor governed by PTCP for submitting goals, delivering solution slices, handling output and prompts, and controlling the current computation. A persistent toplevel supports a sequence of queries; a one-shot toplevel terminates when its query is exhausted or stopped. Toplevel actors underpin interactive shells and provide the actor model from which remote predicate calls can be reconstructed.

Unification. Unification is the operation that makes two Prolog terms equal by finding a consistent substitution for their variables. In the Trinity it is also a structural data contract: a goal states both the request and the expected shape of its answers. Receive patterns use one-sided matching: they may bind pattern variables but cannot specialise variables in a message. Statechart event patterns use unification. Unification guarantees structural compatibility, not shared real-world meaning.

WASM and WASI. WebAssembly (WASM) is a portable binary execution format supported by browsers and other hosts; SWI-WASM brings SWI-Prolog into that environment. The WebAssembly System Interface (WASI) defines capability-oriented interfaces for selected host resources outside a browser. WASM is an execution substrate, not by itself a sandbox policy or a complete resource-governance mechanism.

Web Prolog. Web Prolog is the language profile of the Prolog Trinity. It starts from a selected ISO Prolog base, adds standard library and Erlang-style actor facilities, and defines profile-scoped APIs for execution over the Web. It is a proposed interoperable language surface, not a new Prolog implementation: different systems may host it while retaining different internals.

Webizing Prolog and prologizing the Web. *Webizing Prolog* means adapting Prolog to URI-addressed nodes, open protocols, browsers, security boundaries, and partial failure. *Prologizing the Web* means bringing Prolog terms, unification, nondeterministic querying, executable rules, and actors to web interaction. The Trinity arises from pursuing both directions together rather than treating either as a thin compatibility layer.

Appendix B
Manual

This manual describes the Web Prolog language and predicate APIs implemented by the current demonstrator. The node's HTTP and WebSocket interfaces are specified separately in Appendix D. Both appendices should be read as snapshots of a design in progress, not as final specifications: terminology, interfaces, and behaviour remain tentative and may change as the implementation evolves.

The classification shown beside each entry identifies the minimum public node profile in which the predicate is available, or another relevant execution context. ISOBASE provides general stateless Prolog calls, ISOTOPE adds persistent toplevel sessions, and ACTOR adds the full stateful actor-messaging model. The profiles are cumulative, so an entry marked ISOTOPE is also available in ACTOR. The classification concerns the client language, not predicates used internally to implement a profile. Entries additionally marked OWNER ONLY are reserved for code running with node-owner authority and are not available to ordinary client code.

B.1 Predicates for programming with core actors

Predicate: `self/1` ACTOR

```
self(-Pid) is det.
```

Binds `Pid` to the process identifier of the calling process.

Note: A pid is an actor address whose canonical global form is `Id@Node`, where `Id` is assigned by `Node`. This ordinary Prolog compound is part of the language contract: programs may copy, store, construct, decompose, compare, and serialise it. A shell may omit `@Node` when displaying a pid on the shell's own node, but this is only a local abbreviation. Constructing a pid does not create an actor, confirm

that it is live, or grant access to it; an operation reaches an actor only when the pair designates that actor and the node authorises the operation. Arithmetic on a numeric `Id` has no defined actor-level meaning. Browser runtimes reserve `localhost` as the local node designator. Web worker actors are therefore presented as, for example, `21594388@localhost`, and the optional main-thread SWI-WASM engine is presented as `main@localhost`.

Predicate: `spawn/1-3` ACTOR

```
spawn(+Goal) is det.
spawn(+Goal, -Pid) is det.
spawn(+Goal, -Pid, +Options) is det.
```

Creates a new Web Prolog actor process running a copy of `Goal`. The new actor's pid is unified with `Pid` before `Goal` is copied. If the same variable occurs in `Goal` and the pid argument, those occurrences therefore carry the child's pid in the copy. Variables occurring more than once in `Goal` retain that sharing within the copy, but no variable identity is shared between the parent and child: bindings produced by the child do not bind variables in the parent, and subsequent parent bindings do not bind variables in the child. These rules are the same for local and remote spawning. Results that must cross the process boundary have to be returned explicitly, for example in a message or through a toplevel protocol.

Admission charges the principal's whole locally hosted subtree. At the actor or CPU quota, spawning raises `error(resource_error(actors),Context)` or `error(resource_error(cpu),Context)`. A non-owner's `link(false)` raises:

```
error(permission_error(spawn, detached_actor, link(false)),
      Context)
```

Loss of the transport during a remote spawn raises:

```
error(remote_spawn_failed(connection_closed), Context)
```

The remote outcome is unknown, so a retry of effectful work requires application duplicate suppression.

Valid options are:

- `node(+URI)`
  Creates the process locally or on a remote compatible node. Default is `localhost`.
- `monitor(+Boolean)`
  If `true`, monitoring is installed as part of process creation. Default is `false`.
- `link(+Boolean)`
  If `true`, installs directional parent-to-child link cleanup. Default is `true`. Only node owners may use `false`; detaching rehomes the actor under the node and sends its output to the node log.
- `src_text(+AtomOrString)`
  Loads the clauses specified by a Web Prolog source text into the actor's private Prolog database before calling `Goal`.

- `src_list(+ListOfClauses)`
  Converts the clauses to source text and loads them into the actor's private Prolog database before calling `Goal`.
- `src_uri(+URI)`
  Loads the clauses specified by a file path, file URI, or HTTP(S) URI into the actor's private Prolog database before calling `Goal`. In the demonstrator, the node where the actor is created resolves and fetches the source; remote creation sends the URI to that node. Relative URIs use the fetching node's context and source-access policy.
- `src_predicates(+ListOfPredicateIndicators)`
  Loads the local predicates denoted by `ListOfPredicateIndicators` into the actor's private Prolog database before calling `Goal`.

Note: In `spawn(+Goal, -Pid, +Options)`, the option `monitor(true)` installs monitoring as part of process creation. When the child terminates, the parent receives a message `down(Pid, Ref, Reason)` where `Pid` is the terminated actor, `Reason` is its exit reason, and `Ref` identifies the monitor instance. For monitoring created via `monitor(true)`, the portable contract is `Ref = Pid`; this lets the spawning process identify and remove that monitor even though `spawn/3` returns no separate monitor reference.

Down-reason contract: The third argument of `down(Pid, Ref, Reason)` uses the following portable vocabulary: `true` means that the actor goal succeeded; `false` means that it failed; `exception(E)` carries the public representation of the uncaught exception; an arbitrary nonvar term may be supplied explicitly through `exit/1-2`; and `no_process` means that the monitored target did not exist when the monitor was installed. The boundary reason `connection_closed` means that observation of a remote target was lost; it does not assert that the target terminated. For an uncaught SWI-Prolog `error(Formal,Context)`, the monitor notification preserves `Formal` but replaces `Context` with an unbound variable. An undefined-procedure error also omits a private generated actor-module qualification. Other exit reasons are unchanged.

Note: The semantics of `link(true)` are directional: if a parent spawns a child with linking enabled, termination of the parent causes linked children to be terminated. The link does not imply symmetric bidirectional exit propagation, and the child cannot trap the termination. For a remote child, its hosting controller enforces the relationship from explicit parent-termination information and, in the current demonstrator, from closure, failure, or lease expiry of the owning trusted connection and lineage. A finite ping/pong lease bounds contact-loss detection while the runtime services the connection, not completion of remote reclamation; see Section 4.1.7.

Note: It is possible to pass an arbitrary number of the `src_*` options to `spawn/3`, and possibly more than one instance of each variant. To ensure that clauses end up in a well-defined order, they are converted into Prolog source text before being loaded

into the database. The order of clauses and directives in the resulting source text is determined by the order of the `src_*` options in the option list.

The native demonstrator validates the complete `spawn/3` option list before local setup or remote dispatch. Options must be unary terms in a proper list. Unknown names, wrong arities, and options belonging to another operation raise `error(domain_error(spawn_option,Option),_)`. Malformed values raise type, domain, or instantiation errors with `spawn/3` in the context. `link/1` and `monitor/1` require booleans; node and source-URI values must be atoms or strings; source lists must be proper lists, and predicate indicators must have atom names and nonnegative integer arities. These checks validate representations; source-access, sandbox, and ownership policies still apply. All supplied options are checked, including repeated ones. Ordinary settings retain their first-occurrence behaviour, and source options retain their order and variable sharing. Plain `spawn/3` does not implement naming: `name/1` is rejected; naming wrappers consume that option themselves.

Native local spawning additionally accepts `target/1` and `io_target/1` as nonvariable routing terms for actor I/O, with `io_target/1` taking precedence. Bare WebSocket spawning retains the connection's control of these routes. The receiving WebSocket endpoint validates client options before preparing source or reserving an actor slot, then supplies its own monitor and I/O metadata. Internal options such as `source_module/1`, `inherit_goal_module/1`, `isolation_io/1`, `monitor_target/1`, and `monitor_ref/1` remain recognised for framework setup; supplying them as raw WebSocket spawn options raises `error(permission_error(option,internal_spawn_option`
Locally, effective options are checked again after runtime option rewriting and before pid allocation. Validation of specialised wrappers' own settings remains separate work.

Predicate: `actors/1` ACTOR

```
actors(-Pids) is det.
```

Binds `Pids` to the list of actor pids visible from the current execution context. In ordinary public client execution, this means the current actor or toplevel together with any actors that have been spawned on behalf of the same client interaction. Actors belonging to other clients are not included.

Note: Outside public client execution, for example in node-owned runtime code or administrative code, `actors/1` returns the pids of all active local actors on the node.

Predicate: `monitor/2` ACTOR

```
monitor(+PidOrName, -Ref) is det.
```

Installs a monitor and returns a fresh reference in `Ref`. The first argument may be a pid or a registered local name, subject to the same target policy as sending. Fresh references are single-use and provide correlation, not sender authentication. The demonstrator uses ten-digit references for compact output; production references should

use a much larger space to resist guessing. When the monitored process terminates, the monitoring process receives a message of the form `down(Pid, Ref, Reason)`. If observation of a remote target is lost, the runtime instead delivers the same form with `Reason = connection_closed`. This discharges the monitor without claiming that the target terminated; reconnection does not restore it automatically.

Note: Using `monitor(true)` in `spawn/3` installs monitoring before the child can terminate, so its actual exit reason is observed. If `monitor/2` is called after the target has already terminated, the runtime still discharges the monitor with `down(Pid,Ref,no_process)`.

Predicate: `demonitor/1-2` ACTOR

```
demonitor(+Ref) is det.
demonitor(+Ref, +Options) is det.
```

Stops the calling actor's monitor identified by `Ref`. An unknown reference or another actor's reference has no effect. This is idempotent. There is only one valid option:

- `flush`
  Non-blocking removal of all pending `down(_, Ref, _)` messages from the mailbox.

Predicate: `register/2` ACTOR

```
register(+Name, +Pid) is det.
```

Associates the atom `Name` with `Pid` in the caller's ordinary execution namespace. In the demonstrator, public namespaces are normally scoped to a WebSocket connection or HTTP session and inherited by spawned actors; a shared authenticated principal does not by itself imply a shared registry. Standalone actor-library use defaults to a global namespace. The ordinary registry is separate from the public service registry.

Within one namespace, a pid may have one ordinary name and a name may identify one pid. An already named pid raises `process_already_has_a_name(Pid)`; an occupied name raises `name_is_in_use(Name)`. An existing binding is never silently overwritten.

Native local named creation through the toplevel, server, supervisor, and statechart wrappers reserves the name before launching the actor, using the caller's existing namespace. A pending reservation is hidden from `whereis/2` and message routing, but counts as occupied for registration and competing named creation. `unregister/1` does not cancel a pending reservation. Once preparation has succeeded and the pid is resolvable, the name is published before the actor's application goal runs. Failed or cancelled preparation releases the reservation, and retirement removes a published binding under the same registry mutex. In particular, immediate termination cannot leave a name registered afterwards by the creation wrapper. This does not guarantee that the actor is still alive, or that its application has finished initialising, when creation returns.

A local alias may refer to a remote pid; it does not publish a service on the remote node. Native registration checks local actor existence under the same registry mutex as insertion and termination cleanup. After the existing collision checks, a missing local actor raises `error(existence_error(process, Pid), Context)`. The check also covers local URL-qualified pids and their accepted textual forms; `main` remains supported. A binding inserted before retirement is removed by cleanup; registration after retirement is rejected. Success does not guarantee that the actor remains alive afterwards. Remote aliases are accepted without a liveness check. Remote named creation retains its post-spawn alias registration; the local reservation guarantee does not extend across nodes. Remote registration installs no implicit monitor. When the native runtime observes a remote termination notification, it removes aliases for that exact pid across local namespaces, even if the notification is buffered during startup. Observed connection loss removes aliases for all pids on that node, including unmonitored aliases; this withdraws local names without establishing that the remote actors died. Cleanup shares the registry mutex with registration and precedes monitor notification. Termination cleanup preserves aliases for other pids; connection-loss cleanup preserves aliases on other nodes. A name reused for a different pid survives an old termination notification. A later remote registration may recreate an alias. If no lifecycle event is observed, cleanup still requires an explicit `unregister/1`; clients needing termination notifications must arrange their own monitoring. Stronger guarantees remain work for Section 11.1.

Predicate: `whereis/2` ACTOR

```
whereis(+Name, -Pid) is det.
```

Binds `Pid` to the value registered under `Name` in the caller's ordinary namespace, or to `undefined` if no such registration exists. Lookup does not establish that the actor is alive.

Predicate: `unregister/1` ACTOR

```
unregister(+Name) is det.
```

Removes the association in the caller's ordinary namespace. The actor itself is not terminated. The operation is idempotent and succeeds without effect if `Name` is not registered in that namespace.

Predicate: `register_service/2` ACTOR, OWNER ONLY

```
register_service(+Name, +Pid) is det.
```

Publishes the live local actor identified by `Pid` under `Name` in the node's public service namespace. `Name` must be an atom, `Pid` must identify an actor on the publishing node, and there may be at most one published service with a given name on a node. If the name is already published, `register_service/2` raises `permission_error(register, service, Name)`, even when the existing entry

denotes the same actor. Replacement is therefore explicit: the owner must first withdraw the old mapping.

The public service namespace is distinct from the ordinary registration namespace used by `register/2`; the same atom may occur in both without a collision. A qualified address `Name@Node` is resolved against the public service namespace of `Node`. `whereis/2` consults only the ordinary namespace; owner-side code may inspect the service namespace with `whereis_service/2`.

Publication neither changes the actor's links nor places it under supervision. In particular, `register_service/2` does not detach the actor from its parent. When the actor terminates, the node controller removes the service mapping atomically, so a published name never remains bound to a dead pid. A supervisor that starts a replacement actor must publish that actor again under the stable name.

Only code executing with node-owner authority may call this predicate. A call by any other principal raises `permission_error(register, service, Name)`. A non-atom `Name` raises `type_error(atom, Name)`; a `Pid` that does not identify a live actor raises `existence_error(process, Pid)`; and an attempt to publish an actor on another node raises `permission_error(register, service, Pid)`.

Predicate: `whereis_service/2` ACTOR, OWNER ONLY

```
whereis_service(+Name, -Pid) is det.
```

Binds `Pid` to the live local actor published as `Name`, or to `undefined` if no such service is published. This predicate inspects only the public service namespace. Only code executing with node-owner authority may call it; other callers receive:

```
permission_error(access, actor_service_registry,
                 whereis_service(Name, Pid))
```

A non-atom `Name` raises `type_error(atom, Name)`.

Predicate: `unregister_service/1` ACTOR, OWNER ONLY

```
unregister_service(+Name) is det.
```

Withdraws `Name` from the node's public service namespace. This operation removes only the name; it neither terminates nor otherwise changes the actor that the name denoted. The operation is idempotent: if no published service called `Name` exists, it succeeds without effect.

Only code executing with node-owner authority may call this predicate. A call by any other principal raises `permission_error(unregister, service, Name)`, and a non-atom `Name` raises `type_error(atom, Name)`.

Predicate: `exit/1` ACTOR

```
exit(+Reason) is det.
```

Terminates the calling process with `Reason`. If termination of the calling process has already been committed, the call has no effect and the original reason stands.

Predicate: `exit/2` ACTOR

```
exit(+Pid, +Reason) is det.
```

Sends an untrappable termination signal to the process identified by `Pid`, using `Reason` as its exit reason. The signal is not a mailbox message. The target must belong to the caller's ownership tree unless the caller is a node owner; otherwise the call raises `error(permission_error(exit,actor,Pid),Context)`. Host thread numbers and aliases are not actor pids. For remote actors, the runtime routes this request through the remote node. Termination commits when the target first accepts a termination signal; the first reason committed is final, and later termination or abort signals have no further observable effect. The signal takes effect at the target's next safe point, which some operations can delay; see Section 2.3.3.

Predicate: `!/2, send/2-3` ACTOR

```
+PidOrName ! +Message is det.
send(+PidOrName, +Message) is det.
send(+PidOrName, +Message, +Options) is det.
```

Sends `Message` to the mailbox of the process identified as `PidOrName`. `PidOrName` may be a local pid, a local registered name, a global pid of the form `Id@Node`, or a published service address of the form `Name@Node`. A global pid may be a value returned by the runtime or a term constructed directly from a known identifier and node; the two are equivalent. Qualified service addresses are resolved only through the destination node's live service registry. If such a service is absent or the sender is not authorised to use it, the send succeeds without delivery and does not reveal which condition applied. By contrast, an unknown local registered name raises `existence_error(actor_name,Name)`; a send to an unknown pid succeeds without delivery. Language-level sending is open subject to node policy; admission may raise `resource_error(mailbox)` or `resource_error(term_depth)`. Valid options for `send/3` are:

- `delay(+Number)`
  Delays the sending by a specified number of seconds. Default is `0`.
- `id(+ID)`
  `ID` is a user-supplied identifier that can be used by `cancel/1` to stop the sending from taking place.

Portable actor application messages are acyclic ordinary Prolog terms. Atoms, strings, numbers, variables, lists, and compound terms are portable. Each send constructs a fresh copy in the recipient: repeated occurrences of one variable remain shared within that copied message, but no variable identity is shared with the sender or with any separately sent message. There is consequently no distributed unification

channel hidden in message passing. Attributed variables, streams, and other non-text blob values must be encoded explicitly by the application. A double-quoted literal denotes a string. The demonstrator validates this subset whenever serialisation is required. Native local delivery may happen to carry additional host terms, but programs using them are outside the placement-independent contract.

Ordering: For one sender, messages sent repeatedly to the same pid are delivered in send order while one controller route remains in use. The same holds for one unchanged name binding during that routing epoch. There is no portable order across pid/name addressing, name rebinding, reconnects or rerouting, different senders, or messages and runtime signals. For delayed sends, ordering begins when the delayed operation actually emits the message. See Section 4.1.6.

Predicate: `cancel/1` ACTOR

```
cancel(+ID) is det.
```

Tries to cancel all delayed messages with the specified `ID` that were scheduled by the calling actor. Identifiers are actor-local: another actor may use the same `ID` without its delayed sends being affected. Cancellation is best-effort only, since a message may already have been sent by the time the call is made.

Predicate: `output/1-2` ACTOR

```
output(+Data) is det.
output(+Data, +Options) is det.
```

Sends a message `output(Pid, Data)` to the target process. `Pid` is the pid of the current process. Valid option:

- `target(+Pid)`
  Send the message to `Pid`. Default is the parent process.

This is a convenience predicate for ordinary actor messaging. Terminal writes such as `format/2` and `writeln/1` instead use the inherited terminal I/O target described in Section 4.1.3. Any actor may use `!/2` to send any term to any process to which it has a pid.

Predicate: `input/2-3` ACTOR

```
input(+Prompt, -Data) is det.
input(+Prompt, -Data, +Options) is det.
```

Sends a message `prompt(Pid, Prompt)` to the target process and waits for its input. `Prompt` may be any Prolog term. `Pid` is the pid of the current process. `Data` will be bound to the term that the target process sends using `respond/2`. Valid option:

- `target(+Pid)`
  Send the `prompt` message to `Pid`. Default is the current I/O target when one has been installed, or otherwise the parent process.

Predicate: `respond/2` ACTOR

```
respond(+Pid, +Input) is det.
```

Sends `Input` to `Pid` in response to a `prompt(Pid,Prompt)` message.

The ACTOR badges on `output/1-2`, `input/2-3`, and `respond/2` describe their availability to client programs. ISOTOPE sessions provide standard I/O predicates such as `read/1` and `writeln/1`; the runtime implements session I/O through internal actor communication without exposing these actor predicates to ISOTOPE clients.

Predicate: `receive/1-2` ACTOR

```
receive(+Clauses) is nondet.
receive(+Clauses, +Options) is nondet.
```

Waits for, selects, and consumes one message from the calling process's mailbox. A call is *single-message*, but not necessarily single-solution. During one invocation it selects and consumes at most one message. Messages are considered in arrival order, and each message is tested against the receive clauses in textual order. The first message accepted by a clause is removed from the mailbox. This selection is committed: backtracking through the invocation never selects another message or another clause.

The selected clause body is ordinary Prolog code. The call as a whole therefore has the determinism of the fired body: a body may fail, succeed once, throw an exception, or yield several solutions. Choice points left by the body remain available, but they enumerate solutions of that same body; they do not inspect the mailbox again. The selected message remains consumed if the body fails or throws, and is not restored on backtracking. Thus the `nondet` declaration concerns the possible number of logical solutions, not the number of messages consumed by one invocation. For example, after one `foo` message has arrived, `receive({foo -> member(X, [a,b])})` consumes that message once but can yield both `X = a` and `X = b`.

A cut executed in a selected clause body is scoped to that body, as if the body were invoked through `call/1`. It can prune choice points created since entry to the body, but it cannot prune choice points in the caller; nor does it alter the already committed message and clause selection.

`Clauses` has the following form:

```
{  Pattern1 [if Guard1] ->
        Body1 ;
   ...
```

```
   PatternN [if GuardN] ->
         BodyN
}
```

A clause accepts a message if its pattern matches the message and its optional guard succeeds. Matching is one-sided: the pattern must subsume the message, so matching may bind variables in the pattern but never binds variables that occur in the message. For a ground message this is the same as unification. A guard is an ordinary goal and is not restricted in this way; a guard that checks a correlation reference or a pid should test the value, for example with ==, rather than bind it. Candidate matching and guard evaluation use a tentative substitution. If matching or the guard fails, all bindings made while trying that candidate are discarded before the next clause or message is considered; the mailbox is not changed until a clause accepts. A guard is evaluated for its first solution only, as if by `once/1`. Bindings from the successful pattern and guard are in scope in the body.

For portable code, a guard must be observationally free of side effects; the runtime is not required to enforce this restriction, and effects performed by a non-conforming guard are not undone when that guard fails. If a guard raises an ordinary exception, the exception and its tentative bindings are discarded and the clause does not accept the message. Control exceptions (abort, termination, and the enforced running-time limit; see Section 2.3.3) are not discarded: they propagate out of `receive/1-2`, and the message being examined remains in the mailbox in its original position. This error-as-non-match rule follows the selection discipline of restricted actor guards, but it also means that a guard which calls a fallible external service should translate failures into explicit results when the actor needs to distinguish them from an ordinary non-match.

Messages accepted by no clause are *deferred*: they remain in the mailbox, without any change in their contents or relative order, and are available to later receive calls. `receive({})` accepts no message and waits, subject to `Options`. Valid options for `receive/2` are:

- `timeout(+Number)`
  Bounds the wait for an accepted message, in seconds. The value `0` inspects messages already in the mailbox without waiting for another. A positive timeout is converted once to an absolute deadline, so messages skipped during the scan do not restart the timeout. Default is no timeout.
- `deadline(+AbsoluteTime)`
  Sets an absolute deadline in the same time scale as `get_time/1`. This is useful when several operations must share one time budget. A deadline that has already passed expires immediately. If both `timeout/1` and `deadline/1` are present, `timeout/1` takes precedence.
- `on_timeout(+Goal)`
  Called if the relative timeout or absolute deadline expires with no message accepted. Its success, failure, exception, and any choice points determine the outcome of the receive on the timeout path. Default is `true`. This option is only meaningful when `timeout/1` or `deadline/1` is present.

Predicate: `flush/0` ACTOR

```
flush is det.
```

Removes all messages currently pending in the mailbox of the calling process. Each removed message is rendered through the actor terminal-output path as a line of the form `Shell got Message`, where `Message` is the full Prolog term.

Note: `flush/0` is primarily a shell-level inspection utility. It does not wait for future messages: once the mailbox is empty at the time of the call, it succeeds immediately.

Predicate: `listing/0-1` ISOTOPE

```
listing is det.
listing(+What) is det.
```

`listing/0` lists the content of the private database of the current actor process through the terminal-output path. Imported predicates and the injected I/O wrapper predicates are omitted.

`listing/1` lists the predicates or clauses in the current process's private database selected by `What`, following the conventional Prolog meaning of `listing/1`. `What` may be a predicate indicator (`Name/Arity` or `Name//Arity`), a callable head pattern, a clause reference, or a list of such specifications. It does not provide access to another actor's private database.

B.2 Predicates for programming with toplevel actors

Predicate: `toplevel_spawn/1-2` ACTOR

```
toplevel_spawn(-Pid) is det.
toplevel_spawn(-Pid, +Options) is det.
```

Spawns a toplevel actor and binds `Pid` to its pid. All options accepted by `spawn/3` are also accepted by `toplevel_spawn/2`. In the demonstrator, `src_uri/1` is resolved and fetched on the node where the toplevel actor is created, as for `spawn/3`. In addition, `toplevel_spawn/2` accepts the following options:

- `session(+Boolean)`
  If set to `false`, the toplevel actor terminates after having run a goal to completion. If `true`, further interaction is expected. Default is `false` in the actor API.
- `target(+Pid)`
  Send all answer terms to `Pid`. Default is the pid of the parent.
- `name(+Name)`
  Register the toplevel actor under `Name` in the local ordinary-name namespace.

- `time_limit(+SecondsOrInfinite)`
  Bounds each continuous period spent executing a goal in PTCP state **s2**. On expiry, the target receives `error(Pid,error(resource_error(time),_))`. A session toplevel remains alive and returns to **s1**, ready for another call. Time spent waiting for a continuation in **s3** does not count; the running time limit starts again when a continuation resumes. The actor API default is `infinite`; a public node may impose a finite ceiling.
- `idle_limit(+SecondsOrInfinite)`
  Bounds inactivity while a session toplevel waits for a call in PTCP state **s1** or for a continuation in **s3**. On expiry, the actor terminates normally, so a monitor observes `down(Pid,Ref,true)`. The actor API default is `infinite`; a public node may impose a finite ceiling.

Both finite lifecycle limits must be positive numbers of seconds. A public client may request tighter values, but cannot weaken the node owner's ceilings. These limits are distinct from the `timeout` on an ISOTOPE session HTTP request, which only bounds the wait for the next queued event and does not abort the goal or terminate the session.

Native `toplevel_spawn/2` validates its complete option list before local preparation or remote dispatch. `session/1` must contain a boolean, `name/1` an atom, and each lifecycle limit a positive number or `infinite`. Unbound values are rejected without binding them. The receiving HTTP and WebSocket endpoints apply the same checks before source preparation, with the same separation of client options and server-supplied metadata as bare actor spawning. Existing endpoint session defaults and node-owner ceilings still apply.

The native creation wrappers `toplevel_spawn/2`, `server_spawn/4`, `supervisor_spawn/3`, and `statechart_spawn/2` accept unary option terms in a proper list. Each validates its own options and the options it forwards to `spawn/3` before starting work. Unsupported options raise `error(domain_error(spawn_option,Option),Context)`; malformed values raise type, domain, or instantiation errors. The context identifies the creation wrapper. Wrapper-specific options do not become options of plain `spawn/3`. Every occurrence is checked, even when a later duplicate would otherwise be ignored. Ordinary settings retain first-occurrence behaviour, source options retain their order and variable sharing, and framework metadata remains supported. In addition to checking the shape of `name/1`, native local named creation reserves and publishes the name through the startup protocol described with `register/2` above. An occupied name is rejected before launching a local actor. Explicit unregistering and renaming after publication remain available. Remote alias registration retains its existing semantics. The native supervisor's child-specification checks are described in Section B.5.

Predicate: `toplevel_call/2-3` ACTOR

```
toplevel_call(+Pid, +Goal) is det.
toplevel_call(+Pid, +Goal, +Options) is det.
```

Asks the toplevel `Pid` for solutions to `Goal`. Valid options are:

- `template(+Template)`
  `Template` is a term sharing variables with `Goal`. By default, the template is identical to the goal.
- `offset(+Integer)`
  Collect only the solution slice starting from `Integer`. Default is `0`.
- `limit(+Integer)`
  Restrict the length of the returned list of solutions to `Integer`. The integer must be positive. If omitted, all solutions are requested, subject to execution-time and result-size bounds.
- `once(+Boolean)`
  When `true`, commits to the first returned slice and reports `More = false`, even if `Goal` has further solutions. Default is `false`. This option is useful primarily with `limit/1`.
- `target(+Pid)`
  Send the answer term to `Pid`. Default is the value of `target` when passed as an option to `toplevel_spawn/2`.

Variables in `Goal` are not bound directly in the caller. Instead, the outcome is reported by one of the following answer terms, delivered as a message to the target mailbox:

- `success(Pid, Data, More)`
  `Pid` is the pid of the toplevel process that succeeded in finding solutions to `Goal`. `Data` is a solution slice: a list holding instantiations of `Template`. `More` indicates whether continuing with `toplevel_next/1-2` may produce another answer slice.
- `failure(Pid)`
  `Pid` is the pid of the toplevel process that found no solution or no further solution.
- `error(Pid, Error)`
  `Pid` is the pid of the toplevel that raised the error. `Error` is the error term.

A toplevel serves one controlling client at a time, and each call starts afresh: when it accepts `Goal`, it discards any `toplevel_next/1-2`, `toplevel_stop/1` or input left over from an earlier query, that is, sent before the call; one sent after the call is kept. If `Goal` raises an exception after a finite `limit` has already collected some solutions, those solutions are delivered first as `success(Pid, Data, true)`, and the error answers the following `toplevel_next/1-2`; `toplevel_stop/1` discards it, and so does `once(true)`.

Answers are plain terms. A solution that still carries constraints, such as those left by `dif/2`, `freeze/2` or CLP(FD), cannot be represented in an answer, since printing it would drop the constraints. The toplevel then reports `error(Pid, error(representation_error(constraints), _))` instead of the page, and the query ends. This holds whether the client is local or remote. A goal that needs such answers can bind the constrained variables, for example by labelling, or return the constraints as ordinary terms.

Predicate: `toplevel_next/1-2` ACTOR

```
toplevel_next(+Pid) is det.
toplevel_next(+Pid, +Options) is det.
```

Asks toplevel `Pid` for more solutions. Valid options:

- `limit(+Integer)`
  An explicit limit must be a positive integer. If omitted, the actor API keeps using the most recent finite limit value from the previous toplevel call state.
- `target(+Pid)`
  Send the answer term to `Pid`. Default is the value of `target` from the previous call state.

Delivers the same answer messages as `toplevel_call/2-3` to the target mailbox.

Native synchronous wrappers: `toplevel_call_synch/3-4`, `toplevel_next_synch/2-3` ACTOR

```
toplevel_call_synch(+Pid, +Goal, -Answer) is semidet.
toplevel_call_synch(+Pid, +Goal, -Answer, +Options) is semidet.
toplevel_next_synch(+Pid, -Answer) is semidet.
toplevel_next_synch(+Pid, -Answer, +Options) is semidet.
```

These native demonstrator predicates send one call or next request and wait in the calling actor's mailbox for one `success(Pid,Data,More)`, `failure(Pid)`, or `error(Pid,Error)` answer. Use the toplevel's concrete pid. Goal variables remain independent in the caller, as with the asynchronous API. Error answers are returned as terms; they are not thrown by the wrapper. I/O and unrelated messages remain available in the mailbox.

Every synchronous request explicitly selects the calling actor as its answer target, overriding the target inherited from creation or an earlier request. An optional `target/1` must contain exactly the concrete pid returned by `self/1` in that caller. An unbound target raises an instantiation error; any other target, including an alias, raises `error(domain_error(synchronous_toplevel_target,Target),_)`. Every supplied target is checked before sending the command.

The call wrapper accepts query options `template/1`, `offset/1`, `limit/1`, and `once/1`, together with native `io_target/1`, which selects a separate actor I/O destination. The next wrapper accepts `limit/1`. Both accept the local receive options `timeout(Number)` and `on_timeout(Goal)`. Query options are sent only to the toplevel; receive options stay with the local wait. Options must be unary terms in a proper list, and values are checked before sending. Unsupported options raise `domain_error(toplevel_call_synch_option,Option)` or `domain_error(toplevel_next_synch_option,Option)`, inside the usual error wrapper. Repeated query and receive settings use their first occurrence; callbacks retain sharing with `Answer`.

There is no wait timeout by default. On timeout, the callback runs once in the caller's context; its bindings are retained, failure makes the wrapper fail, and exceptions propagate. Without a callback, a timeout succeeds with `Answer` still unbound, following `receive/2`. This differs from `yield/3`'s default `timeout` answer. Timing out does not stop the query or discard a later answer. A pre-instantiated `Answer` that does not match a collected answer makes the wrapper fail after consuming that message. These wrappers retain the one-client assumption; they do not add request identifiers or stale-answer correlation. The shorter forms use an empty option list.

Predicate: `toplevel_stop/1` ACTOR

```
toplevel_stop(+Pid) is det.
```

Asks toplevel `Pid` to stop searching for more solutions.

Predicate: `toplevel_abort/1` ACTOR

```
toplevel_abort(+Pid) is det.
```

Tells the local toplevel `Pid` to abort the execution of the currently running goal. Abort runs supported cleanup, bypasses recovery in client `catch/3`, and returns a persistent session to PTCP state **s1**; it does not terminate a persistent toplevel actor. A toplevel spawned with `session(false)` terminates instead. An abort that arrives while the toplevel is idle in **s1** leaves a persistent toplevel idle and ends a one-shot toplevel; one that arrives in **s3** abandons the suspended solution sequence, as `toplevel_stop/1` would. An abort that arrives after termination of the toplevel has been committed is ignored. Every other abort, including one in **s1**, is acknowledged: the toplevel sends `abort(Pid)` to its client (before any call, to its spawn target), after any answers of the aborted query. An unknown or non-local pid has no effect.

Predicate: `toplevel_halt/1-2` ACTOR

```
toplevel_halt(+Pid) is det.
toplevel_halt(+Pid, -Reply) is det.
```

Terminates toplevel actor `Pid` from any protocol state. The one-argument form requests termination through the actor runtime and returns without waiting for confirmation. The two-argument form installs a private monitor, requests termination, and waits for its notification. An observed termination yields `Reply = true`, regardless of its reason. The special reason `no_process` also yields `true`: the target is confirmed absent, without establishing when or why it terminated. For a remote target, `connection_closed` instead yields `Reply = error(connection_closed)`: observation was lost and termination is unconfirmed. Halt is an untrappable termination signal, not the interruption signal used by abort.

Note: The private monitor used by `toplevel_halt/2` is distinct from any existing monitor. If the toplevel was spawned with `monitor(true)`, the caller still receives its original `down(Pid, Pid, true)` notification.

B.3 Predicates for programming with server actors

Predicate: `server_spawn/3-4` ACTOR

```
server_spawn(+Pred, +State, -Pid) is det.
server_spawn(+Pred, +State, -Pid, +Options) is det.
```

Spawns a generic server actor using callback predicate `Pred/4` and initial state `State`. The callback receives `(Request, OldState, Response, NewState)`. All options are forwarded to `spawn/3` except:

- `name(+Name)`
  Register the server under `Name`.

Native `server_spawn/4` requires an atom name and validates all forwarded spawn options before callback preparation or actor creation. The common native wrapper-validation rules are described in Section B.2.

Predicate: `server_request/3-4` ACTOR

```
server_request(+To, +Request, -Response) is det.
server_request(+To, +Request, -Response, +Options) is det.
```

Makes a synchronous request-response call to the server `To`, where `To` may be a pid or a registered local name. `server_request/4` accepts the options of `receive/2`, for example `timeout(+Seconds)`. Monitoring is installed automatically. If the server terminates before replying, the call promptly throws `server_down(Reason)`.

Predicate: `server_promise/3-4` ACTOR

```
server_promise(+To, +Request, -Ref) is det.
server_promise(+To, +Request, -Ref, -MonRef) is det.
```

Sends `Request` to the server `To` and returns a correlation reference `Ref`. The reply must later be collected with `server_yield/2-4`. The four-argument variant additionally installs a monitor and returns its reference in `MonRef`.

Predicate: `server_yield/2-4` ACTOR

```
server_yield(+Ref, -Response) is det.
server_yield(+Ref, -Response, +Options) is det.
server_yield(+Ref, +MonRef, -Response, +Options) is det.
```

Waits for the response matching `Ref`. The three-argument variant accepts the options of `receive/2`. The four-argument variant uses `MonRef` to detect server termination, removes that monitor when the wait finishes normally or exceptionally, and throws `server_down(Reason)` if the server dies before replying.

Predicate: `server_upgrade/2-3` ACTOR

```
server_upgrade(+To, +Pred) is det.
server_upgrade(+To, +Pred, +Options) is det.
```

Replaces the callback predicate of the running server `To` without disturbing its current state. `Pred` must denote a callback predicate of arity four.

The two-argument form transfers no code and therefore requires the callback to be already available to the server process. The three-argument form accepts the following source-loading options:

- `src_text(+Text)`
  Load callback source from `Text`.
- `src_list(+ListOfClauses)`
  Convert the clauses to source text and load them into the server.
- `src_predicates(+ListOfPredicateIndicators)`
  Copy the caller's indicated predicates into the server's private database.
- `src_uri(+URI)`
  Fetch callback source from `URI` and load it into the server.

The operation replies only after the callback has been loaded and validated. It preserves the current state value and performs no state migration. If installation fails, `server_upgrade/2-3` rethrows the reported error and the state is unchanged. Source changes made in the server's private database before the error are not rolled back.

Predicate: `server_halt/2` ACTOR

```
server_halt(+To, -Reply) is det.
```

Asks the server `To` to stop gracefully and binds `Reply` to its acknowledgement. The request carries a fresh correlation reference, so an unrelated acknowledgement in the caller's mailbox cannot satisfy the operation. A private monitor is installed before the request; if the server terminates without acknowledging it, the operation throws `server_down(Reason)`.

B.4 Predicates for programming with statechart actors

Predicate: `statechart_spawn/2` ACTOR

```
statechart_spawn(-Pid, +Options) is det.
```

Spawns a statechart actor and binds `Pid` to its pid.

`statechart_spawn/2` requires exactly one source option:

- `src_uri(+URI)`
  Load and run a statechart from a URI or file path.
- `src_text(+Text)`
  Load and run a statechart from an XML text string.

In addition, `statechart_spawn/2` accepts:

- `name(+Name)`
  Register the statechart actor under `Name` in the local ordinary-name namespace.
- `trace(+Mode)`
  Set the per-actor trace mode to `true` or `false`. `true` emits trace events on the terminal-output path; `false` suppresses them. At most one `trace/1` option is accepted. Default is `true` for direct predicate calls; clients should pass the intended value explicitly.

All remaining options are passed through to `spawn/3`.

`src_list/1` and `src_predicates/1` are accepted as supplemental Prolog sources loaded into the statechart actor's private module. They do not count as the required XML source. The SXML document notation is specified in Appendix C.

Native `statechart_spawn/2` validates names as atoms and trace modes as booleans, together with all forwarded spawn options, before preparing the chart source or creating an actor. It retains the rejection of duplicate trace options and the requirement for exactly one XML source. The common native wrapper-validation rules are described in Section B.2.

Predicate: `statechart_halt/2-3` ACTOR

```
statechart_halt(+Pid, -Reply) is det.
statechart_halt(+Pid, -Reply, +Timeout) is det.
```

Asks the statechart actor `Pid` to halt gracefully, running the exit actions of its active states, and waits for an acknowledgement in `Reply`. The two-argument form waits indefinitely.

The three-argument form waits at most `Timeout` seconds. If the actor does not reply in time, it is killed and `Reply` becomes `killed`.

Predicate: `raise/1` STATECHART BUILTIN

```
raise(+Event) is det.
```

Enqueues `Event` on the *internal* event queue of the current statechart interpreter. It is handled after the current block of executable content and before a new external message.

Important: `raise/1` is only meaningful inside executable statechart content such as <onentry>, <onexit>, and <go>.

Predicate: `in/1` STATECHART BUILTIN

```
in(+Id) is semidet.
```

Succeeds if the state identified by `Id` belongs to the current active configuration of the statechart interpreter.

Important: `in/1` is only meaningful inside statechart guards or executable statechart content such as <onentry>, <onexit>, and <go>.

B.5 Predicates for programming with supervisor actors

Predicate: `supervisor_spawn/2-3` ACTOR

```
supervisor_spawn(+ChildSpecs, -Pid) is det.
supervisor_spawn(+ChildSpecs, -Pid, +Options) is det.
```

Spawns a supervisor actor and starts its children. Supported options are:

- `strategy(+Strategy)`
  One of `one_for_one` (default), `one_for_all`, or `rest_for_one`.
- `intensity(+Integer)`
  Maximum number of restarts permitted within one period. Must be a nonnegative integer; zero permits no restarts. Default is `1`.
- `period(+Number)`
  Restart-intensity window in positive seconds; fractional values are accepted. Default is `5`.
- `name(+Name)`
  Register the supervisor under `Name`.

Other options are passed through to `spawn/3`.

Native `supervisor_spawn/3` checks these settings, including an atom name, and all forwarded spawn options before starting the supervisor or its children. Nested supervisors use the same checks for their supervisor settings. Additional child-specification checks are described below; lifecycle behaviour is unchanged. The common native wrapper-validation rules are described in Section B.2.

Child specifications take the form `child(Id, ChildOptions)`. Child options are:

- `start(+Goal)`
  Required. Start goal for the child. The special form

  `server(Pred, ServerOptions)`

  starts a generic server child. The default initial state is `[]`. It may be replaced with `initial_state(+State)`. All remaining `ServerOptions` are passed to `server_spawn/4`.
- `restart(+Policy)`
  One of `permanent` (default), `transient`, or `temporary`.
- `shutdown(+Shutdown)`
  One of `brutal_kill`, `infinity`, or a nonnegative timeout in seconds, including zero. The default is `5` for workers and `infinity` for supervisor children. `brutal_kill` terminates the child at once with reason `kill`. With a timeout, a server child is first sent its ordinary stop request; if it has not exited when the timeout expires, it receives `exit(Pid, shutdown)`. With `infinity`, a server child is sent the stop request and the supervisor waits for it without limit. A child without a stop protocol receives `exit(Pid, shutdown)` at once. Since the first committed exit reason is final, a termination already in progress cannot be escalated to `kill`.
- `type(+Type)`
  `worker` (default) or `supervisor`.

The native demonstrator validates the complete initial child list before creating the supervisor or starting any child. The list and its option lists must be proper, and specifications must be acyclic `child(Id,Options)` terms. Ids must be ground; compound ids remain valid for ordinary children. Server-child ids must be atoms because they also become registered names. Duplicate ids within an initial child list raise `error(permission_error(duplicate,supervisor_child_id,Id),_)`.

Only `start/1`, `restart/1`, `shutdown/1`, and `type/1` are child options. Unsupported names or arities raise `error(domain_error(supervisor_child_option,Option),_)`; malformed structures and values raise type, domain, or instantiation errors. A missing start option retains `error(missing_start_option(Id),supervisor)`. Cyclic specifications raise `representation_error(cyclic_term)` inside the usual error wrapper. Every occurrence is checked, including duplicates, but the first occurrence still supplies the effective value. Defaults and variable sharing within start goals and states are preserved; validation does not instantiate input variables.

Start goals must be callable, with atom module qualifiers where supplied. The recognised `server(Pred,ServerOptions)` form additionally checks the callback's callable shape, its server/spawn options, and any `initial_state/1` option; the state itself may contain variables. The recognised `start_sub_supervisor/2` form, unqualified or explicitly qualified by `supervisor_actor`, recursively checks its child list and supervisor options before outer startup. Other callable start goals are left to runtime. Validation does not execute goals, fetch source, check predicate availability, or reserve names; successful startup is still a separate matter.

Predicate: `supervisor_spawn_child/3` ACTOR

```
supervisor_spawn_child(+Pid, +ChildSpec, -Reply) is det.
```

Dynamically adds and starts one child. On success, `Reply` is `ok`. If a child with the same `Id` is already present, it is `error(already_present)`. If startup fails, it is `error(start_failed)`. Native `supervisor_spawn_child/3` uses the same single-child validator before contacting the supervisor; malformed specifications therefore raise exceptions without changing its child table. A valid specification whose id already exists retains the `error(already_present)` reply.

Predicate: `supervisor_terminate_child/3` ACTOR

```
supervisor_terminate_child(+Pid, +Id, -Reply) is det.
```

Stops child `Id` but keeps its specification. `Reply` is `ok` or `error(not_found)`.

Predicate: `supervisor_delete_child/3` ACTOR

```
supervisor_delete_child(+Pid, +Id, -Reply) is det.
```

Removes a non-running child specification. `Reply` is `ok`, `error(running)`, or `error(not_found)`.

Predicate: `supervisor_respawn_child/3` ACTOR

```
supervisor_respawn_child(+Pid, +Id, -Reply) is det.
```

Restarts a previously terminated child from its stored specification. `Reply` is either `ok(NewPid)`, `error(running)`, `error(not_found)`, or `error(start_failed)`.

Predicate: `supervisor_which_children/2` ACTOR

```
supervisor_which_children(+Pid, -Children) is det.
```

Returns a list of:

```
info(Id, Pid, Type, Restart)
```

where `Pid` may be `undefined` for stopped children.

Predicate: `supervisor_count_children/2` ACTOR

```
supervisor_count_children(+Pid, -Counts) is det.
```

Returns:

```
[specs-N, active-N, supervisors-N, workers-N]
```

Predicate: `supervisor_halt/1` ACTOR

```
supervisor_halt(+Pid) is det.
```

Requests supervisor shutdown and waits for acknowledgement of acceptance. The supervisor then shuts down its children in reverse start order. Returning from this call does not confirm completed shutdown. To observe completed supervisor termination, install a monitor before calling `supervisor_halt/1` and wait for its termination notification.

Note on synchronous supervisor calls

The dynamic/query API above, including `supervisor_halt/1`, uses monitored synchronous calls and may throw:

- `supervisor_down(Reason)`
- `supervisor_call_timeout(Sup, Request)`

B.6 Prolog-style generics

Predicate: `parallel/1` ACTOR

```
parallel(+Goals) is semidet.
```

Runs the proper list of independent goals `Goals` concurrently, using one monitored worker actor per goal. The call succeeds when every goal has produced its first solution and returns the resulting variable bindings. `parallel([])` succeeds immediately. Further solutions are not enumerated on backtracking.

If any goal fails, the remaining workers are terminated and the call fails. If any goal throws an exception, the remaining workers are terminated and the exception is rethrown. If a worker terminates without a result for any other reason, including `true`, the remaining workers are terminated and `worker_exit(Pid,Reason)` is thrown. Once a worker's result has been accepted, its termination notification only confirms that it has stopped. If several workers have abnormal outcomes, the first failure or error notification observed by `parallel/1` determines the result; this selection may depend on scheduling and does not promise leftmost-goal precedence. Result and monitor messages belonging to the workers are removed before the predicate returns, fails, or throws.

The goals must not depend on bindings or side effects produced by sibling goals. Actor creation and message copying add overhead, and the one-worker-per-goal model gives large goal lists a correspondingly large resource footprint.

A non-list `Goals` raises `type_error(list, Goals)`. An element that is not callable raises the corresponding callable type error from its worker, which is propagated to the caller.

Predicate: `first_solution/2-3` ACTOR

```
first_solution(?Solution, +Goals) is semidet.
first_solution(?Solution, +Goals, +Options) is semidet.
```

Runs the proper list of alternative goals `Goals` concurrently in monitored worker actors and unifies `Solution` with the result produced by the first solver to succeed. Once a winner has been selected, all remaining workers are terminated and their result and monitor messages are removed. The predicate does not enumerate later solutions on backtracking, and an empty goal list fails.

The two-argument form continues after an individual solver fails and stops on the first error. An exception retains its original term. Any other exit without a result, apart from ordinary failure, is reported as `worker_exit(Pid,Reason)`, including an exit with reason `true`. These exits use the existing `on_error` policy. The three-argument form accepts:

- `on_fail(stop|continue)`
  `continue` removes the failed solver and keeps waiting while alternatives remain; `stop` terminates all workers and fails immediately. Default is `continue`.
- `on_error(stop|continue)`
  `continue` removes the solver that ended with an error and keeps waiting while alternatives remain; `stop` terminates all workers and throws the reported error. Default is `stop`.

If all alternatives are removed under a `continue` policy, the predicate fails. Goals should not depend on side effects produced by sibling solvers.

A non-list `Goals` or `Options` raises `type_error(list, Term)`. An action other than `stop` or `continue` raises `type_error(oneof([stop,continue]), Action)`. An unknown option raises `domain_error(first_solution_option, Option)`.

B.7 Predicates for remote Prolog-style calls

Predicate: `rpc/2-3` ISOBASE

```
rpc(+URI, +Goal) is nondet.
rpc(+URI, +Goal, +Options) is nondet.
```

Executes a copy of `Goal` through the stateless `/call` endpoint of the node identified by `URI`, then unifies the local goal with each returned solution on backtracking. This remains a stateless call even when the destination is an ACTOR node: actor-only predicates such as `self/1`, `spawn/1-3`, and `receive/1-2` are not available through `rpc/2-3`.

The first argument alone selects the execution node. `localhost` means the node on which `rpc/2-3` is running. Source-loading options supply source for the call's temporary private database. For example:

```
rpc('http://n2.org', Goal, [
   src_uri('http://n3.org')
])
```

executes `Goal` on `n2.org`. In the demonstrator, the calling runtime fetches the source from `n3.org` and sends its text to `n2.org`. All conforming native and browser runtimes accept the following options:

- `limit(+Integer)`
  Limits the solutions retrieved per roundtrip to a positive integer. If omitted, all solutions are requested. Exceeding an execution-time or result-size bound produces an error, not a truncated success response; the client may retry with a finite limit. Execution-time exhaustion is reported as `error(resource_error(time),_)`; memory or result-size exhaustion as `error(resource_error(space),_)`. The context argument is elided in normal display. RPC clients propagate the remote exception without an additional wrapper. A client transport timeout is a separate condition and does not establish that the remote computation has stopped.
- `once(+Boolean)`
  When `true`, retrieves only the first slice and does not request later slices. Default is `false`.
- `timeout(+Number)`
  Requests a timeout in seconds for goal execution on the destination node. The effective timeout is the minimum of this value and the node owner's configured timeout.
- `http_timeout(+Number)`
  Sets the client-side HTTP transport timeout in seconds for each request. It does not change the destination node's execution timeout.
- `src_text(+AtomOrString)`
  Loads source text into the temporary private database before calling `Goal`.
- `src_list(+ListOfClauses)`
  Converts the clauses to source text and loads them into the temporary private database.
- `src_uri(+URI)`
  Fetches source from `URI` and loads it into the temporary private database. This URI is a source location only. In the demonstrator, the caller resolves and fetches it, then sends source text to the execution node. Relative URIs use the caller's context and source-access policy.

- `src_predicates(+ListOfPredicateIndicators)`
  Copies the caller's indicated predicates into the temporary private database.

The native demonstrator additionally accepts the following HTTP transport options. They affect the client's requests to the execution node, including later slice requests; they do not configure source fetching or remote goal execution:

- `request_header(+Name=+Value)`
  Adds a request header. `Name` is an atom and `Value` an atom or string. Several headers may be supplied as separate options.
- `authorization(+Credentials)`
  Sends HTTP credentials: `basic(User,Password)` or `bearer(Token)`, with atom or string arguments.
- `user_agent(+Agent)`
  Sets the HTTP user-agent value to the given atom or string.

There is no arbitrary pass-through of SWI-Prolog HTTP options.

Native `rpc/3` validates the entire option list before resolving the execution URI, fetching source, or making a request. Options must be unary terms in a proper list. An unsupported name or arity raises `error(domain_error(rpc_option,Option),_)`; malformed values raise type, domain, or instantiation errors with the public predicate in the context. All supplied options are checked, including repeated ones. Repeated source options contribute source in list order; for query settings such as `limit/1`, `once/1`, and `timeout/1`, the first occurrence takes effect. Native execution timeouts retain their existing normalization: a negative number becomes zero, while `none` or an unbound value leaves the requested execution timeout unspecified. A negative `http_timeout/1` likewise becomes zero.

Predicate: `promise/3-4` ISOBASE

```
promise(+URI, +Goal, -Ref) is det.
promise(+URI, +Goal, -Ref, +Options) is det.
```

Starts one asynchronous stateless call to node `URI` with `Goal` and returns a local promise reference in `Ref`. As with `rpc/2-3`, the first argument selects the execution node and `localhost` denotes the current node. Source-loading options populate the promised call's temporary private database; they do not change its execution node. In the demonstrator, the caller completes source preparation, including any `src_uri/1` fetch, before returning `Ref`; errors at this stage are raised by `promise/3-4` itself. The remote query runs asynchronously. It accepts the same `limit/1`, `once/1`, `timeout/1`, `http_timeout/1`, and `src_*/1` options as `rpc/3`, together with:

- `template(+Template)`
  `Template` is a term sharing variables with the goal. By default, the template is identical to the goal.
- `offset(+Integer)`
  Selects the starting offset of the single solution slice collected by the promise. Default is `0`.

The reference returned in `Ref` can later be used by `yield/2-3` to collect the single answer term. The native `promise/4` accepts the same HTTP transport subset and validates all options before source fetching or worker creation. Unsupported options raise `error(domain_error(promise_option,Option),_)` directly from `promise/4`. The offset must be a nonnegative integer; `template/1` may contain variables and retains its sharing with the goal. Repeated query settings use their first occurrence.

Predicate: `yield/2-3` ISOBASE

```
yield(+Ref, -Answer) is det.
yield(+Ref, -Answer, +Options) is det.
```

Waits for the answer term from a previous call to `promise/3-4`. The reference is local to the runtime that created it. The intended contract is one collection of one answer term; backtracking does not enumerate further results. Timeout handling must return a defined answer term or throw an exception. The native SWI-Prolog demonstrator stores references in a runtime-wide table without enforcing issuing-process ownership. Transfer between processes or stateless requests is implementation-specific; a portable ownership and isolation contract remains unresolved. Remote answers are `success(Data,More)`, `failure`, or `error(Error)`. A local wait timeout returns `timeout` by default; a timeout callback may supply its own defined result. This wait outcome is distinct from remote logical failure or an error answer. `yield/2` is equivalent to `yield/3` with an empty option list. Valid options are:

- `timeout(+Number)`
  Waits at most `Number` seconds for the promised answer. Default is no timeout.
- `on_timeout(+Goal)`
  Calls `Goal` once in the caller's context if the wait expires. It is not called when an answer is collected. Bindings it makes to `Answer` are retained; if `Answer` remains unbound, it is set to `timeout`. Callback failure throws `error(timeout_callback_failed(Ref),_)`; an exception propagates. Callback alternatives do not become additional results of `yield/3`.

Native `yield/3` accepts only these two options and validates the whole list before waiting or collecting an answer. Unsupported options raise `error(domain_error(yield_option,Option),_)`. The timeout must be a number; a negative value gives an immediate wait timeout if no answer is available. The callback must be callable. The first occurrence of each option takes effect. Validation does not execute the callback or consume the promise. Successful collection returns one answer, including a `failure` or `error(Error)` answer, and removes the reference's lookup entry. A wait timeout does not cancel or consume the promise, even if its callback fails or throws; a later call may collect the answer if the reference has not expired.

For an unknown, consumed, or expired reference, both native yield variants throw `error(existence_error(promise,Ref),_)`. An unavailable reference does not invoke the timeout callback. Native promise workers deliver request-preparation,

transport, parsing, and malformed-response errors as `error(Error)` answers. A worker that completes without an answer reports `error(error(promise_worker_failed,_))`.

The native demonstrator schedules removal of unconsumed lookup entries 300 seconds after creation. Expiry does not establish coordinated reclamation of queues and background workers, unblock an already waiting collector, or cancel the remote query. Lifecycle hardening is part of the agenda in Section 11.1.

Predicate: `runtime_property/1` ISOBASE

```
runtime_property(?Property) is nondet.
```

Enumerates properties of the host runtime. This predicate reports host-runtime properties; it does not alter the Web Prolog profile or the semantics of the predicates above. Defined properties are:

- `implementation(swi_native|swi_wasm_worker|swi_wasm_main)`
- `persistent(+Boolean)`
- `inbound_addressable(+Boolean)`
- `dom(+Boolean)`
- `actor_isolation(native_thread|web_worker|cooperative_engine)`
- `hard_termination(+Boolean)`

Native nodes are persistent and inbound-addressable and use native threads with hard termination, but have no DOM. Web worker SWI-WASM actors have web worker isolation and hard termination but share the page's lifetime, are not inbound-addressable, and have no DOM. The optional main-thread SWI-WASM engine has DOM access but cooperative engine isolation and no hard termination; it too shares the page's lifetime and initiates rather than accepts remote connections.

B.8 Node introspection

Predicate: `node_setting/2` ALL PUBLIC PROFILES

```
node_setting(?Key, ?Value) is nondet.
```

Enumerates publicly visible settings of the node servicing the current request. With `Key` bound, succeeds for its visible current value; an unknown or unavailable key fails. Shared-database source, credentials, and principal policies are not exposed. The public keys are:

General settings: `url`, `profile`, `sandbox`, `auth`, `timeout`, and `rate_window_seconds`.
Stateless API: `max_inflight_calls`, `max_term_text_bytes`, and `max_call_requests_per_window`.
Private database: `max_load_text_bytes` and `load_uri_allowed_origins`.

Session API: `max_sessions_per_principal` and `max_session_spawns_per_window`.
Actor API: `max_ws_actors_per_principal`, `max_ws_frame_bytes`, and `max_ws_commands_per_window`.

API-specific keys are visible only when their corresponding built-in family is enabled for the node's current profile. The predicate reports settings; it does not change them.

B.9 Common error contract

Predicate entries above state specialised errors where they matter. Across the API, the common Prolog error vocabulary has the following roles:

Form	Meaning
`instantiation_error`	A required input was insufficiently instantiated.
`type_error/2`	The value has the wrong Prolog type; arguments identify the expected type and culprit.
`domain_error/2`	The type is acceptable but the value is outside the operation's domain.
`existence_error/2`	A required process, procedure, source, or other object does not exist.
`permission_error/3`	The operation is defined but not authorised in this execution context.
`resource_error/1`	The runtime cannot supply a required bounded resource.
`error(resource_error(time),_)`	A configured execution limit expired.

A direct predicate call throws the error term. PTCP reports a submitted-goal error as `error(Pid,E)`; stateless calls use `error(E)`; HTTP and WebSocket bindings place its public representation in a `type:"error"` envelope. An uncaught actor exception is observed by a monitor as `down(Pid,Ref,exception(E))`. Boundary sanitisation may remove host-private context but must preserve the portable formal error in Prolog-format responses and node-to-node term fields. Browser-facing JSON responses carry display diagnostics instead; these are not a stable error-classification interface. Appendix D specifies these representations and the available machine-readable error codes.

B.10 ISO predicates and related constructs

Web Prolog supports the following ISO predicates and related constructs, including the extensions listed here. Directives and DCG rules are source-level items, and `read/1` and the write predicates are provided through the session I/O layer rather than ambient ISO stream I/O.

Control and meta-call

`!/0, (,)/2, (;)/2, (->)/2, (\+)/1, call/1-8, catch/3, fail/0, false/0, once/1, repeat/0, throw/1, true/0`

Directives and DCGs

`-->/2, discontiguous/1, dynamic/1, multifile/1, op/3, phrase/2-3`
Operator declarations are subject to the node's admitted language; a restricted profile may omit `op/3`.

Term relations, comparison, and construction

`(=)/2, (=..)/2, (==)/2, (@<)/2, (@=<)/2, (@>)/2, (@>=)/2, (\=)/2, (\==)/2, arg/3, compare/3, copy_term/2, functor/3, subsumes_term/2, term_variables/2, unify_with_occurs_check/2`

Arithmetic predicates

`(<)/2, (=:=)/2, (=<)/2, (=\=)/2, (>)/2, (>=)/2, is/2`

Arithmetic evaluable functors

The arithmetic evaluable functors specified by ISO/IEC 13211-1 are supported in arithmetic expressions.

Type testing

`acyclic_term/1, atom/1, atomic/1, callable/1, compound/1, float/1, ground/1, integer/1, nonvar/1, number/1, var/1`

Database and all-solutions

`abolish/1, assert/1, asserta/1, assertz/1, bagof/3, clause/2, findall/3, keysort/2, retract/1, retractall/1, setof/3, sort/2`

Atomic term processing

`atom_chars/2, atom_codes/2, atom_concat/3, atom_length/2, char_code/2, number_chars/2, number_codes/2, sub_atom/5`

Session I/O

`display/1, format/1-2, nl/0, print/1, read/1, time/1, write/1, write_canonical/1, write_term/2, writeln/1, writeq/1`
The goal-executing `~@` format directive, including forms with numeric arguments or a colon modifier, is rejected with `permission_error(use, format_specifier, '~@')`. An escaped tilde in `~~@` produces literal text.

B.11 Standard library and utility predicates

Nodes that accept general Prolog goals supply these library predicates without explicit loading, as described in Chapter 2:

List relations and indexing: `member/2`, `memberchk/2`, `append/3`, `length/2`, `select/3`, `nth0/3-4`, `nth1/3-4`.
Integer relations: `between/3`, `succ/2`.
Iteration and solution processing: `maplist/2-8`, `foldl/4-6`, `call_nth/2`, `countall/2`, `forall/2`.

`memberchk/2` commits to the first matching member. `maplist/2-8` applies a goal to corresponding list elements; `foldl/4-6` additionally threads an accumulator. `call_nth/2` numbers a goal's solutions from one, and `countall/2` counts them. `forall/2` tests that its second goal succeeds for every solution of its first; it exports no bindings.

Predicate: `make_ref/1` ACTOR

```
make_ref(-Ref) is det.
```

Produces a fresh opaque correlation reference suitable for transport in messages. Programs compare references with == and must not depend on their representation. This is distinct from installing a monitor: no process is monitored and no notification is requested.

Predicate: `sleep/1` ISOBASE

```
sleep(+Seconds) is det.
```

Suspends execution for a non-negative number of seconds, which may be fractional. It does not consume mailbox messages and may be interrupted by control signals or an execution limit. Chapter 2 shows how an actor can express the delay using a timed `receive/2`.

Appendix C
SXML notation reference

This appendix is the reference manual for the XML notation accepted by the current Web Prolog statechart demonstrator. Chapter 6 introduces the ideas through examples; the present appendix instead proceeds element by element and states the operational rules on which documents may rely.

SXML is a design in progress. This appendix describes what the demonstrator implements at the time of writing.

C.1 A complete minimal document

The following document is a complete SXML statechart. It begins in `idle`, reports entry into that state, accepts a Prolog term as an event, and terminates by entering the top-level final state `finished`.

```
<statechart version="0.2" initial="idle">
    <datamodel>permitted(report).</datamodel>
    <state id="idle">
        <onentry>output(ready)</onentry>
        <go to="finished" on="start(Job)" if="permitted(Job)">
            output(starting(Job))
        </go>
    </state>
    <final id="finished"/>
</statechart>
```

The event is the Prolog term `start(Job)`. If the actor receives, for example, `start(report)`, unification binds `Job` to `report`; the same binding is visible in the guard specified by the attribute `if` and in the transition body. If `permitted(report)` succeeds, the action runs and the statechart enters `finished`. If matching or the guard fails, this transition is not enabled.

An application normally supplies the XML to `statechart_spawn/2`:

```
?- statechart_spawn(Pid, [
       src_uri('controller.xml'),
       monitor(true),
       trace(false)
   ]).
```

The document is supplied to `statechart_spawn/2`, whose options are specified in Appendix B.4.

C.2 Vocabulary at a glance

Element	Principal attributes	Purpose
<statechart>	version, initial, id	Document root and top-level compound state.
<state>	id, initial	Atomic or compound sequential state.
<parallel>	id	State whose child regions are simultaneously active.
<final>	id	Completion state for a compound state or for the whole statechart.
<history>	id, type	Shallow or deep remembered configuration.
<go>	to, on, if, after	Transition, optional guard, and optional Prolog action.
<onentry>	—	Prolog goal run when its parent state is entered.
<onexit>	—	Prolog goal run when its parent state is exited.
<datamodel>	—	Prolog clauses and directives loaded into the statechart's private module.
<defer>	on, if	State-scoped rule for postponing an otherwise unmatched event.
<spawn>	type, options, goal, callback, state, children	State-scoped invocation of an actor, toplevel, server, supervisor, or nested statechart.

Table C.1 SXML elements, their principal attributes, and their purposes.

Element and attribute names are case-sensitive. State identifiers are also case-sensitive: `Ready` and `ready` denote different states. Every <state>, <parallel>, <final>, and <history> element must have a document-wide unique value for the attribute `id`. Before constructing the model, the validator rejects missing or duplicate identifiers and elements placed outside the content model defined by the SXML 0.2 DTD. Section C.10 distinguishes DTD validation, cross-element semantic validation, and the further checks performed by the model parser.

C.3 XML and embedded Prolog

C.3.1 XML conventions

An SXML source is a well-formed XML document with a <statechart> element as its root. Ordinary XML escaping rules apply. In an attribute, a Prolog comparison such as X < 10 must therefore be written as X < 10; an ampersand must be written as &. A large body of source that contains XML punctuation is often clearer in a CDATA section:

```
<onentry><![CDATA[
    X < 10,
    output(value(X))
]]></onentry>
```

Validation rejects unknown elements and attributes; authors and tools should use only the vocabulary in Section C.2.

C.3.2 Where Prolog occurs

SXML embeds Web Prolog in four distinct places:

- the value of the attribute on is a Prolog event pattern;
- the value of the attribute if is a Prolog goal used as a guard;
- the text in the <onentry>, <onexit>, and <go> elements is a Prolog goal used as executable content; and
- the text in the <datamodel> element is Prolog program source: clauses, directives, DCG rules, and declarations.

A block of executable content denotes one Prolog term. Several operations should be joined as a conjunction:

```
<onentry>
    retractall(count(_)),
    assert(count(0)),
    output(ready)
</onentry>
```

The event pattern, guard, and action of one <go> element share variables by name. The guard is tried for its first solution. Failed matching or a failed guard leaves no logical bindings behind when the next transition candidate is tried. Guards should nevertheless be free of observable side effects: a database update, message, or output performed by a guard is not undone when that guard later fails.

Executable content is committed and is run once. Its alternatives are not exposed through backtracking. Failure raises the internal event error(failure(Goal)); an

exception `E` raises the internal event `error(E)`. A statechart may handle these terms with ordinary patterns in the attribute `on`. Database effects performed before failure are not rolled back.

C.4 Document root and states

C.4.1 The <statechart> element

```
<statechart version="0.2"
    id="machine"
    initial="start">
    ...
</statechart>
```

The <statechart> element is the document root. Its attributes are:

The `version` attribute (required)
: The SXML notation version. The demonstrator accepts exactly `0.2`; a missing or different value is an error.

The `initial` attribute (required)
: The identifier of the single top-level state entered at startup.

The `id` attribute (optional)
: The identifier of the root state. If omitted, the implementation uses the internal identifier `statechart_actor`. Give the root an explicit identifier if an action or trace consumer needs to refer to it.

All embedded patterns, guards, actions, and data-model source are Web Prolog. The useful direct children are the <datamodel>, <state>, <parallel>, and <final> elements. The root participates in the state hierarchy, but normal documents do not attach transitions or invocation directly to it.

C.4.2 The <state> element

```
<state id="connected" initial="idle">
    <onentry>output(connected)</onentry>
    <state id="idle">...</state>
    <state id="busy">...</state>
    <go to="disconnected" on="close"/>
    <onexit>output(disconnected)</onexit>
</state>
```

A <state> element is *atomic* when it contains no state children and *compound* when it contains a nested control structure. When a compound state is active, one of its

sequential children is active as well. Outgoing transitions and deferral declarations on the compound state are inherited by active descendants: selection searches from the innermost active state outwards.

The attribute `id` is required. A compound state must also have the attribute `initial`, whose value identifies its default child.

Useful children are the <onentry>, <onexit>, <go>, <defer>, <spawn>, <datamodel>, <history>, <state>, <parallel>, and <final> elements. Their XML order also establishes document order for entry, exit, and transition priority.

C.4.3 The <parallel> element

```
<parallel id="controller">
    <state id="connection" initial="offline">...</state>
    <state id="work" initial="idle">...</state>
</parallel>
```

A <parallel> element makes all its child regions active. The regions do not run on separate threads. They form one configuration inside one actor, and a selected event is visible to each active region under a single run-to-completion scheduler. Compatible transitions in different regions may therefore be selected in the same microstep. Section C.5.1 specifies transition priority and conflict resolution.

The attribute `id` is required. A parallel state does not have the attribute `initial`: entry activates each child and recursively enters that child's explicit initial state where necessary. It may have <onentry>, <onexit>, <go>, <defer>, <spawn>, <datamodel>, <history>, <state>, <parallel>, and <final> child elements.

C.4.4 The <final> element and completion

```
<state id="processing" initial="request">
    <state id="request" initial="working">
        <state id="working">
            <go to="complete" on="reply(_)"/>
        </state>
        <final id="complete"/>
    </state>
    <go to="all_done" on="done(request)"/>
</state>
<final id="all_done"/>
```

Place this fragment inside the document root and set its `initial` attribute to `processing`. The enclosing state owns the completion transition. The target `all_done` is a top-level final state.

The attribute `id` is required. A final state may contain an `<onentry>` element. Entering a final child of a compound state with identifier `P` raises the internal Prolog event `done(P)`. When every region of a parallel state `Q` has reached a final configuration, the runtime raises `done(Q)`.

Entering a top-level final state terminates the statechart normally. Active states are exited in order, the executable content in their `<onexit>` elements runs, and invoked children owned by those states are stopped. A monitor on the statechart then receives `down(Pid,Ref,true)`. Completion data must be sent explicitly as an actor message before the statechart terminates.

C.4.5 The <history> element

```
<state id="work" initial="first">
    <history id="resume" type="deep">
        <go to="first"/>
    </history>
    ...
</state>
```

A history pseudostate records the active configuration below its parent when that parent is exited. A transition whose attribute `to` names the history identifier restores the recorded configuration. When the attribute `type` has the value `shallow` (the default), the statechart remembers immediate active children; with the value `deep`, it remembers the complete active atomic descendants.

The attribute `id` is required. Supply one eventless `<go>` child element as the fallback target used before any history has been recorded. The fallback is structural in the current implementation; put effects in the destination's `<onentry>` element, not in the fallback `<go>` element's body.

C.5 Transitions: the <go> element

```
<go to="running" on="start(Job)" if="valid(Job)">
    output(started(Job))
</go>
```

A `<go>` element may have the following attributes:

The `on` attribute
: A single Prolog event pattern. It is matched by unification. Variables bound by matching are visible in the guard and action. Unlike `receive/1-2`, this matching is not one-sided; correlation guards must test values rather than bind them (Section 6.5.1).

The `if` attribute
: A Web Prolog guard. It defaults to `true`. The first solution is used. A guard without the attribute `on` makes an eventless transition.

The `to` attribute
: One or more state identifiers separated by XML whitespace. Omit it for a targetless transition. Multiple targets are intended for a legal orthogonal-state configuration; most transitions have one target.

The `after` attribute
: A finite, non-negative delay in seconds. Integer and fractional values are accepted. The attributes `after` and `on` are mutually exclusive. A guard supplied with the attribute `if` on a timed transition is tested when the timer fires.

At least one of the attributes `on`, `if`, `to`, or `after` should be present. The parser currently accepts an entirely empty <go> element, but it is an unconditionally enabled, eventless, targetless transition and normally creates an infinite internal cascade. Table C.2 summarises the common forms.

Transition form	**Typical use / notes**
`<go to="s" on="e">...</go>`	Triggered state change. The ordinary event-driven case: on event pattern `e`, transition to `s` and run actions.
`<go to="s" on="e" if="c">...</go>`	Triggered and guarded state change. The standard ECA pattern: a matching event plus a condition enables the transition; convenient when several transitions compete on the same event.
`<go to="s" after="n">...</go>`	State-scoped timed transition. Arms on entry to the source state and is cancelled automatically on exit; `n` is a non-negative number of seconds.
`<go on="e" if="c">...</go>`	Guarded, targetless event reaction. Consumes events matching `e`, checks `c`, runs actions, and stays in the same configuration. Useful for logging, emitting output, raising internal events, or updating the data model without a state change.
`<go to="s">...</go>`	Eventless, unconditional state change. Useful for automatic progress (for example, completion-style transitions) or compact control-flow steps.
`<go to="s" if="c">...</go>`	Eventless but guarded state change. Useful for decision points driven by the data model, or for encoding automatic progress only when a condition holds.
`<go if="c">...</go>`	Guarded, targetless reaction (no control-state change). Useful as a condition-action rule that updates the data model, emits output, raises an internal event, logs, etc., while staying in the same configuration.

Table C.2 Common forms of the <go> element and their typical uses.

Targetless and self-targeted transitions differ. A targetless transition runs its body without exiting or entering any state. A transition whose target is its own source performs an external self-transition: exit handlers run, timers and invoked children are cancelled, and the state is entered again.

C.5.1 Selection and execution order

For each active atomic state, candidates are considered from that state outward through its ancestors. The first matching and successful candidate in document order is selected for that region. Duplicate selections are removed. If two remaining transitions have overlapping exit sets, the transition whose source is the descendant pre-empts the transition from its ancestor. Compatible transitions selected in parallel regions may run together; document order resolves any remaining choice.

One microstep has three ordered phases:

1. exit affected active states from the inside out, running the <onexit> elements;
2. execute selected transition bodies in document order; and
3. enter target states from the outside in, running the <onentry> elements.

Eventless transitions and internal events are drained before another actor message is received. This complete reaction to one external event is a macrostep. Its atomicity means that no newer external event is selected in the middle; it does not make Prolog database effects transactional.

C.5.2 Timed transitions

A timer is armed when its source state is entered and cancelled before that state's exit actions run. Re-entry creates a fresh activation. A private activation reference prevents a late firing from an old activation from triggering the timer of a newer one. A timeout does not interrupt a Prolog action that is already running; it becomes eligible only when the current work yields to event processing.

C.6 Entry, exit, and the Prolog data model

C.6.1 The <onentry> and <onexit> elements

```
<state id="open">
    <onentry>output(opened)</onentry>
    ...
    <onexit>output(closed)</onexit>
```

```
</state>
```

The text of each element is one Web Prolog goal. There may be more than one entry or exit element; they run in document order. On entry, the state is already in the active configuration, so `in(Id)` can observe it. On exit, timers have first been invalidated, but the state remains in the active configuration while its exit action runs. Invoked children are stopped after the state's exit actions.

Actions should terminate promptly. A blocking `receive/1`, a long `sleep/1`, or a synchronous operation prevents the macrostep from stabilising and prevents the actor from selecting further external events. Use asynchronous actor interaction and handle the reply as a later event.

C.6.2 The <datamodel> element

```
<datamodel>
    :- dynamic count/1.
    count(0).

    permitted(Job) :- policy_allows(Job).
</datamodel>
```

The <datamodel> element contains ordinary Prolog source. It is loaded before the initial configuration is entered. The source may define facts, rules, DCGs, dynamic declarations, and other directives allowed by the active node policy. Code supplied through the statechart actor's `src_list/1` and `src_predicates/1` options is loaded into the same private execution module.

The statechart's own predicates shadow predicates with the same name in the node's shared database; otherwise an action may call shared predicates offered by the node. Dynamic updates belong to the statechart actor's private database and are sequentially isolated from other actors. A public node may sandbox embedded goals and reject predicates outside its advertised profile.

Although the parser accepts the <datamodel> element below compound states, all such source is loaded at statechart startup; it is not installed and removed on state entry and exit. Prefer one document-level element unless locality in the XML source materially improves readability.

C.6.3 Statechart built-ins

The statechart built-ins `raise/1` and `in/1` are specified in Appendix B.4. The latter is particularly useful in guards that depend on another parallel region.

Ordinary actor and I/O predicates remain the communication layer. For example, `send(Pid,Term)` or `Pid ! Term` sends an event to an actor, while `output(Term)`

sends actor output along the configured I/O path. Communication and logging are thus expressed with Web Prolog predicates.

C.7 Deferred events: the <defer> element

```
<state id="draining">
    <defer on="job(J)" if="deferrable(J)"/>
    <go to="idle" on="drained"/>
</state>
```

The attribute `on` is required and contains one Prolog pattern. The optional attribute `if` contains a guard and defaults to `true`; variables from the pattern are visible in the guard.

Deferral is considered only if no transition is enabled anywhere in the active configuration. An enabled transition always wins. The runtime then searches active states from the inside out, and one matching declaration in any active region is sufficient to append the event to the postponed queue. Timed-transition implementation events are never deferrable.

After internal processing settles, a net change in the active-state configuration triggers re-offering of a snapshot of the postponed queue, in original arrival order, before a newer external message is selected. The snapshot is removed from the postponed queue for this pass. An event can be deferred again if the current configuration still declares it; otherwise it is handled or discarded. Re-deferral alone does not trigger another pass: re-deferred events wait for a further net configuration change. In a parallel statechart, a change in one region can therefore trigger reconsideration of events still deferred by another region, without causing repeated re-offering while the configuration remains unchanged.

A targetless transition alone does not trigger re-offering. Nor does an external self-transition that exits and re-enters the same states, unless subsequent internal processing leaves a different active-state configuration. Entry and exit actions still run; the trigger depends on the net configuration change rather than on those actions having executed. The queue has no implicit application-level bound, so broad patterns in a long-lived state require explicit resource discipline.

C.8 Actor invocation: the <spawn> element

The <spawn> element belongs to the state that contains it. Creation is delayed until the macrostep in which that state was entered has stabilised. A state that is entered and exited within the same macrostep never creates its child. After successful creation the interpreter raises `spawned(Pid)` on its internal event queue. This is a notification from the interpreter, not a message sent by the child.

The child remains owned by the invoking state. Exiting that state stops the child after the state's <onexit> content. The lifetime rule applies to all five types of the <spawn> element and is independent of whether link(true) or monitor(true) was also requested.

C.8.1 Common syntax and value rules

```
<spawn type="server"
       callback="counter"
       state="0"
       options="[name(counter_server), monitor(true), link(false)]">
    counter(inc, N, N1, N1) :- N1 is N + 1.
</spawn>
```

The attribute type is required and selects one of the five invocation contracts below. The required invocation operands are given by direct attributes such as goal, callback, state, and children. Every optional setting is a member of the Prolog list in the attribute options; optional settings are not accepted as direct XML attributes.

Values of required operand attributes and members of the list in the attribute options are parsed as Prolog terms and must be ground. The attribute options must itself contain a ground Prolog list. Use Prolog quoting where a value would otherwise be parsed incorrectly, for example options="[src_uri('child.xml')]". Non-source option names may occur only once in the list.

Nonempty body text becomes one src_text/1 source fragment. It is preserved as source text rather than being parsed and printed again, so clause layout and operators survive. For ordinary Prolog children it is a convenient place for callback or worker clauses. Multiple source fragments are allowed for those children and are loaded in their declared order, with the inline body first.

C.8.2 Option matrix

Type	Invocation contract
actor	Required: goal(Callable) via the goal attribute. Type-specific option: target/1.
toplevel	No required operand. Type-specific options: session/1, time_limit/1, idle_limit/1, target/1, and name/1.
server	Required: callback(Callable) and state(Term) via attributes. Type-specific option: name/1.
supervisor	Required: children(List) via the children attribute. Type-specific options: strategy/1, intensity/1, period/1, and name/1.
statechart	Required: exactly one XML src_text/1 or src_uri/1 source. Type-specific options: name/1 and trace/1.

Table C.3 Invocation contracts for the five types of the <spawn> element.

All five types accept `monitor/1`, `link/1`, `node/1`, and `io_target/1`. The first two take `true` or `false`. `io_target` chooses where the child's terminal and `output/1` traffic is delivered. An actor or toplevel `target` chooses its protocol/result recipient; it is not a synonym for the I/O target. For a toplevel, an explicit `target` option replaces the containing statechart as the recipient of query results, while the containing statechart still receives its own `spawned(Pid)` event.

Actor, toplevel, server, and supervisor children accept `src_text/1`, `src_uri/1`, `src_list/1`, and `src_predicates/1` as Prolog sources.

For `time_limit` and `idle_limit`, a value is either `infinite` or a positive number of seconds. Supervisor strategy, intensity, and period have the same meanings as in the supervisor API; strategies are `one_for_one`, `one_for_all`, and `rest_for_one`. A `target` is supported only for actor and toplevel children; the option `trace/1` is supported only for a nested statechart.

C.8.3 Actor, toplevel, server, and supervisor examples

An actor <spawn> element supplies its initial goal and may define that goal in the body:

```
<spawn type="actor" goal="worker" options="[monitor(true)]">
    worker :-
        receive({stop -> true;
                 Job  -> perform(Job), worker}).
</spawn>
```

A toplevel body becomes its private program. Once `spawned(Pid)` is handled, an action may initiate a query with `toplevel_call/2-3`; successive `success/3` events then arrive through the actor protocol.

The server example at the start of this section uses a <spawn> element to create a server whose callback is `counter/4` and whose initial state is `0`. The registered name `counter_server` lets statechart actions address the child without retaining its pid. A supervisor <spawn> element instead places a ground list of ordinary supervisor child specifications in the attribute `children`:

```
<spawn type="supervisor"
       children="[child(worker,
                       [start(sleep(60)), restart(temporary)])]"
       options="[strategy(one_for_all), intensity(2), period(5)]"/>
```

The child specification and policy are passed to the supervisor behaviour. The statechart merely owns the supervisor actor and observes `spawned(Pid)`; supervision events and calls use the ordinary supervisor API.

C.8.4 Nested statecharts

A nested statechart needs exactly one XML source. A `src_uri` can name an external document. Inline XML must use CDATA because it is source for the child, not structural XML belonging to the parent document's <spawn> element:

```
<spawn type="statechart"
       options="[monitor(true),
                 name(inner_chart),
                 trace(false),
                 src_list([child_value(ok)])]"><![CDATA[
    <statechart version="0.2" initial="waiting">
        <state id="waiting">
            <onentry>
                child_value(Value),
                output(nested(Value))
            </onentry>
            <go to="finished" on="finish"/>
        </state>
        <final id="finished"/>
    </statechart>
]]></spawn>
```

The CDATA body supplies the nested statechart's XML through `src_text/1`. The option `src_list/1` supplies the fact `child_value(ok)` as supplemental Prolog source for the nested statechart's private module. Selected predicates may instead be copied with `src_predicates/1`. Neither is a second XML source. Supplying zero or more than one of `src_text/1` and `src_uri/1` is an error.

Unless `io_target` says otherwise, the nested statechart's `output(nested(ok))` follows its inherited I/O path and is observable by the outer statechart as the actor event `output(ChildPid,nested(ok))`. The outer statechart must include a matching <go> element if it wants to act on or print that value; output is not automatically converted into a state change. The option `trace(false)` suppresses only the nested statechart's trace. It does not change the trace setting of the containing statechart.

Normal completion of a monitored nested statechart is reported by the canonical monitor event `down(Pid,Ref,true)`. The outer statechart may match `down/3`, send its own protocol messages, or use the registered child name as appropriate.

C.9 Run-to-completion and event queues

The statechart actor consumes external actor messages in arrival order. Once one message has been selected, its pattern is tested against enabled transitions across the active configuration. An unmatched event is discarded unless a <defer> element

postpones it; unlike selective `receive/1-2`, it does not remain in the actor mailbox implicitly.

The interpreter gives work the following broad priority:

1. enabled eventless transitions;
2. events already in the internal queue, including events placed there by `raise/1`, completion and error events (`done/1` and `error/1`), `spawned/1` events, and valid timer firings;
3. postponed events re-offered after a configuration change;
4. invocation requests belonging to newly stable states; and
5. the next external actor message.

This priority makes one external reaction deterministic given the active configuration, private database, and event, but it also makes termination of internal cascades an author responsibility. An eventless rule that remains true after its action, or two `raise/1` handlers that regenerate one another, can livelock the statechart. While a macrostep is stuck, even an orderly `statechart_halt/2` request cannot be selected; use the bounded `statechart_halt/3` when forced escalation is required.

C.10 Validation, tracing, and diagnosis

Before constructing a model, the demonstrator validates every document against the versioned SXML 0.2 DTD. The implementation supplies the DTD to the XML parser, so an SXML document neither needs nor should repeat a `DOCTYPE` declaration. A running node also serves the distributable grammar at `/sxml-0.2.dtd` for editors and external validators. The DTD checks the permitted elements, their placement and textual content, and the permitted attributes. Unknown or misplaced elements and unknown attributes are errors rather than constructs that the model builder silently ignores.

DTD validation is followed by a side-effect-free semantic pass. It checks that identifiers are unique, that the attributes `initial` and `to` name existing identifiers, and that an initial state is a direct child of the element that selects it. The model parser then checks the parts of SXML that an XML grammar cannot express: Web Prolog syntax, the combination of the attributes `on` and `after`, delay values, and the type-dependent operands and options of the <spawn> element. Thus the DTD is the XML grammar of SXML, not the whole statechart validity contract.

Set `trace(true)` or `trace(false)` explicitly when spawning a statechart if observable tracing matters. With tracing enabled, the actor emits terms of the form `statechart_trace(TraceEvent)` on its terminal-output path. Current trace events include:

- `external_event(Event)` and `internal_event(Event)`;
- `unmatched(Event)`;
- `deferred(Event)` and `reoffered(Events)`;
- `transition(Source,Targets,ExitSet,EntrySet)`;
- `microstep(ExitSet,EntrySet)` and `configuration(States)`;
- `execution(Goal)`; and

- `invoked(Type,Pid,OwningState)`.

Trace terms expose implementation-level ordering and are chiefly diagnostic; application logic should react to its protocol events, not depend on trace delivery. For a nested `<spawn type="statechart">` element, its `trace/1` option, with value `true` or `false`, controls that statechart independently.

Spawn a statechart with `monitor(true)` while developing it. Malformed XML, missing initial states, rejected spawn options, and uncaught startup errors then appear in the monitor's `down/3` reason instead of looking like a statechart that simply failed to react. Validation detects vocabulary and cross-reference errors before the model is built; tracing the initial configuration remains useful for diagnosing the behaviour of a valid model.

C.11 Relationship to SCXML 1.0

SXML retains SCXML's central statechart ideas: atomic and compound states, parallel regions, initial and final states, shallow and deep history, configuration-wide transition selection, inside-out exit, outside-in entry, and run-to-completion processing. It then instantiates those ideas in the Web Prolog actor model. SCXML 1.0 is defined by the W3C Recommendation (Barnett et al., 2015), available at `https://www.w3.org/TR/scxml/`. SXML is not a W3C conformance profile. The following table is a translation guide; it does not define a mechanical conversion procedure for arbitrary SCXML.

The SCXML `<data>`, `<assign>`, `<donedata>`, `<content>`, `<param>`, `<script>`, `<raise>`, `<if>`, `<elseif>`, `<else>`, `<foreach>`, `<log>`, `<send>`, `<cancel>`, `<invoke>`, and `<finalize>` elements, and their associated expression attributes, are therefore not SXML elements. Their useful effects are normally expressible in embedded Web Prolog, but their SCXML syntax and service protocols are not accepted.

There are also three current structural restrictions to remember when adapting an SCXML document:

- SXML requires explicit root and compound-state initial targets; it does not choose the first child by default.
- The SCXML `<initial>` element is not an SXML element; use the attribute `initial` on the containing state instead.
- SXML has external transition behaviour and targetless transitions, but no transition mode corresponding to the attribute value `type="internal"`.

These differences are intentional where they follow from Prolog terms and actors, and provisional where they reflect the present demonstrator. Code that must be portable to a conforming SCXML processor should be authored as SCXML, not as SXML; code that targets the Web Prolog statechart behaviour should follow this appendix rather than assuming that visually similar SCXML markup has been implemented.

SCXML 1.0	SXML	Principal difference
`<scxml>`	`<statechart>`	No SCXML namespace or binding attributes; the SXML version is `0.2`.
`<transition>`	`<go>`	Prolog-oriented spelling; no current equivalent of the attribute value `type="internal"`.
The attribute `event`	The attribute `on`	One Prolog pattern matched by unification, not a list of dotted event descriptors.
The attribute `cond`	The attribute `if`	Web Prolog goal whose first solution enables the transition.
The attribute `target`	The attribute `to`	Space-separated state identifiers; targetless semantics are retained.
The `<datamodel>`, `<data>`, and `<assign>` elements	The `<datamodel>` element containing Prolog source	Private clauses and ordinary Prolog database operations replace named data objects and XML assignment.
XML executable elements	Prolog text	Sequencing, conditionals, iteration, logging, and communication use Prolog predicates and control constructs.
The `<raise>` element	`raise(Event)`	Internal events are Prolog terms.
The `<send>` and `<cancel>` elements	Actor predicates	Communication uses Web Prolog pids, names, messages, delayed sends, and cancellation.
The `<invoke>` element	The `<spawn>` element	Invokes actor behaviours with state-scoped lifetime.
The delayed `<send>` element	The attribute `after` of the `<go>` element	Direct state-scoped timer with automatic cancellation on exit.
No corresponding element	The `<defer>` element	Explicit state-scoped postponement of unmatched Prolog event patterns.
`done.state.id`	`done(Id)`	Completion is an internal Prolog term.
`done.invoke.id` and the `<donedata>` element	explicit messages and `down/3`	No automatic invocation-completion data protocol.
`_event` system record	pattern bindings	Event structure is the term itself; matching exposes its components directly.

Table C.4 Correspondence between SCXML 1.0 and SXML.

C.12 Authoring checklist

Before running a new statechart, check the following:

1. The document is well-formed XML that validates as SXML 0.2, with one `<statechart>` root element whose attribute `version` is `0.2`.
2. The root element has an explicit attribute `initial`; every compound `<state>` element also has that attribute.
3. The identifiers of all `<state>`, `<parallel>`, `<final>`, and `<history>` elements are present and unique.
4. Every target named by an attribute `to` exists, and a multiple-target transition names a legal orthogonal configuration.
5. Each value of an attribute `on` or `if`, each action, and each `<spawn>` element's Prolog-valued attributes are valid Web Prolog after XML entity decoding.
6. Guards are side-effect free and all eventless/internal cascades are guaranteed to terminate.
7. A timed transition does not have both the attributes `after` and `on`.
8. Deferred-event patterns are narrow enough that their queue remains bounded by application behaviour.
9. Each `<spawn>` element has the required attribute `type`, the required operands, ground options, and no duplicate non-source option names.
10. A nested statechart has exactly one XML source and uses `src_list` or `src_predicates` only for supplemental Prolog.
11. Development runs use `monitor(true)` and an intentional trace setting so startup and selection errors are visible.

The demonstrator examples provide progressively larger executable documents, including history, parallel regions, timed and deferred transitions, the private and shared Prolog data model, and all five types of the <spawn> element. They are useful as runnable companions to this reference, but the contracts in this appendix take precedence over incidental details of any one example.

Appendix D
Node protocol reference

This appendix describes the public node protocols implemented by the current demonstrator. It complements the predicate reference in Appendix B: that appendix specifies the Prolog interface seen by a program, whereas this one specifies the HTTP and WebSocket interfaces seen by a client of a node.

The protocols remain a design in progress. The descriptions below distinguish the canonical interface from compatibility forms that the implementation may continue to accept. Clients should generate only the canonical forms.

D.1 Profiles and protocol surfaces

The node profiles are cumulative interaction contracts, not alternative names for transports. RELATION and ISOBASE use the same stateless endpoint but admit different classes of query. ISOTOPE adds a persistent toplevel session, and ACTOR adds general actor creation and messaging.

Profile	Interaction contract	Public surface
RELATION	Queries over advertised relations	HTTP `/call`
ISOBASE	Stateless Prolog execution and RPC	HTTP `/call`
ISOTOPE	Persistent toplevel sessions	HTTP `/toplevel_*`, plus lower surfaces
ACTOR	Actors, messaging, and server push	WebSocket `/ws`, plus lower surfaces

RELATION accepts only query patterns allowed by the relations that the node advertises. ISOBASE admits general goals and temporary source loading, subject to the node's sandbox and resource policy. Both remain stateless from the client's perspective: temporary source does not establish a session. The stateless route rejects client database-update predicates such as `assert/1` and `retract/1`, even on an ACTOR node.

ISOTOPE associates requests with a long-lived toplevel actor. Its private database and session I/O therefore survive from one call to the next. ACTOR adds bare actors, mailboxes, monitors, and asynchronous events over a persistent full-duplex connection. Authentication and authorisation may control who may use a surface, but do not change the semantics of the profile once access has been granted.

D.2 Common conventions

D.2.1 Versioning

The current WebSocket and cross-node wire vocabulary has protocol version `1`. A node reports the version, together with implementation and build information, at `/version`. An incompatible wire change requires a new version. Additive response fields may be ignored by clients that do not recognise them.

D.2.2 Terms, pids, and formats

Goals, templates, options, messages, input, monitor references, and exit reasons are Prolog terms represented as text wherever a table below says *term text*. Implementations must parse a goal together with any template that shares its variables; parsing them independently would lose variable identity. In the current SWI-Prolog demonstrator's version-1 binding, term text is read with `double_quotes` set to `string`, so a double-quoted literal denotes a native SWI-Prolog string in goals, options, input, answers and loaded source alike.

This convention is not yet a shared portable text contract. The GNU Prolog and Trealla prototypes have text-sensitive differences; their reported interoperability tests do not establish agreement on all text values and operations. The opt-in `portable-text-0.1` draft proposes character-list semantics, but remains experimental (Section 11.1). This question concerns Prolog literals inside term text; JSON envelope fields specified as strings remain JSON strings.

Term text is written in one canonical way: quoted, without operator syntax, without the writer's portray hooks, without treating `'$VAR'(N)` terms as variable names, and with explicit variable names. A goal and its template written for the same request share those names, so the receiving node, which reads them together, restores the variables they share. A term such as `a :- b` is therefore sent as `:-(a,b)`, which any Prolog reader parses whatever operators it has. The JSON binding values described below are display text and follow the display conventions instead.

Conformance concerns the terms a client decodes and the defined error categories, not the printed text. A test suite compares decoded terms; the canonical writing is how an implementation meets that contract, and two implementations may differ in spacing or variable names without disagreeing.

An answer is a plain term. A solution whose variables still carry constraints, such as those left by `dif/2`, `freeze/2` or CLP(FD), cannot be represented without losing them. The toplevel then reports an `error` whose Prolog term is `error(representation_error(constraints),_)` instead of the page, and the query ends.

A pid has the canonical global form `Id@Node`, where `Id` is an actor identifier assigned by `Node`. This representation is part of the protocol contract: clients may copy, construct, decompose, compare, and serialise it. The demonstrator allocates ten-digit integers and never reuses them during the node's lifetime. These local pids and numeric references are JSON numbers, safely within JavaScript's exact integer range. The book's eight-digit examples abbreviate this presentation choice. Production identifiers and correlation references should use a substantially larger space, for example 128 random bits. For interoperability, the demonstrator also supports identities larger than $2^{53} - 1$ as decimal strings; clients must decode them without rounding through floating point. A pid qualified with another node uses its `Id@Node` text. Neither size nor secrecy grants authority. Constructing such a value does not create an actor, confirm that it is live, or grant access to it. Resolution and use remain subject to actor existence and node authorisation; arithmetic on a numeric identifier has no defined actor-level meaning.

HTTP execution endpoints accept `format=json` or `format=prolog`. JSON is the default. The Prolog form returns one answer term as plain text. In JSON, the mandatory `type` field selects the answer or event form. JSON in this specification means the data-interchange format defined by RFC 8259, whose registered media type is `application/json`.[1]

A JSON response or WebSocket frame is a protocol *envelope*. Control fields such as `type`, `command`, `more`, and `pid` use the JSON forms specified in this appendix. When a field is described as *term text*, however, its JSON value is a string containing Prolog source text; parsing the surrounding JSON does not decode that string as a Prolog term. Conversely, structured solution data is mapped to JSON as specified for the corresponding endpoint. The JSON envelope does not provide an automatic, lossless JSON representation for every Prolog term.

In this appendix, an *event* is a node-to-client protocol item selected by its `type` field. It may be carried in an HTTP response or a WebSocket server frame. Actor application payloads are called *messages*; *reply* is reserved for a correlated request–reply interaction.

D.2.3 Compression, binary formats, and transport protection

Representation, compression, and transport protection are separate layers. HTTP peers may negotiate a content coding such as `gzip` by means of `Accept-Encoding`

[1] T. Bray, "The JavaScript Object Notation (JSON) Data Interchange Format," RFC 8259, `https://www.rfc-editor.org/rfc/rfc8259.html`.

and `Content-Encoding`; this compresses the HTTP representation without changing whether its underlying media type is JSON or Prolog text.[2] A WebSocket connection may likewise negotiate a compression extension during its opening handshake; the standard `permessage-deflate` extension is defined by RFC 7692.[3] Neither form of compression changes the Web Prolog protocol vocabulary, and neither is required by protocol version 1 unless a deployment profile says otherwise.

Protocol version 1 defines JSON envelopes and Prolog term text; it does not define a binary term representation. Formats such as Erlang's External Term Format or UBF are relevant precedents, but adopting either would require an explicit mapping for Prolog values and semantics rather than simply reusing its existing encoding.[4] Any future binary representation must be identified by a negotiated media type or protocol capability and must specify terms, variables, sharing, cycles, errors, and versioning. Sending a binary WebSocket frame alone does not constitute such negotiation.

HTTPS and secure WebSocket connections use TLS to protect bytes in transit. TLS supplies confidentiality and integrity and normally authenticates the server; client authentication depends on the deployment. It does not select a term representation, compress the application data, or replace Web Prolog's application-level authentication and authorisation rules.[5]

---

[2] See RFC 9110, Sections 8.4 and 12.5.3, `https://www.rfc-editor.org/rfc/rfc9110.html`.

[3] `https://www.rfc-editor.org/rfc/rfc7692.html`.

[4] For the External Term Format, see `https://www.erlang.org/doc/apps/erts/erl_ext_dist.html`; for UBF, see `https://ubf.github.io/ubf/`.

[5] TLS 1.3 is specified by RFC 8446, `https://www.rfc-editor.org/rfc/rfc8446.html`.

D.2.4 Common answer and event forms

type	Additional fields	Meaning
success	data, more; optionally pid, term	A solution slice; more says whether another slice may exist.
failure	optionally pid	The goal has no solution, or no further solution.
error	data; optionally pid, details, code, term	Execution or protocol error.
spawned	pid	An actor or session was created.
output	pid, data	Session or actor output.
prompt	pid, data	The session is waiting for input.
timeout	pid	A blocking wait reached its timeout.
stop	pid	The current solution sequence was stopped.
abort	pid	An abort has taken effect.
responded	pid	Input or a trace setting was accepted.
halted	pid, reply	A toplevel actor was terminated.
down	pid, ref, reason	A monitored actor terminated, or remote observation was lost.
statechart_trace	pid, data	Statechart trace output.

A stateless response has no pid. Session and actor events normally do. In JSON success responses, data is a list. With the default JSON query format, each element is an object containing the named variables of one solution. Each binding value is a string containing a Prolog display term, including numbers and compound values; it is not a recursively converted JSON value. For example, X = 42 appears as {"X":"42"}. With Prolog formatting, the corresponding terms are success(Data,More) for stateless calls and success(Pid,Data,More) for session calls.

Output and prompts may precede the final success, failure, or error event. A timeout at a polling or receive boundary ends that wait; it does not by itself mean that the session has been halted.

A PTCP running-limit expiry is instead an error event. Its Prolog error term is error(resource_error(time),_); the session actor remains alive and returns to its idle state.

In a JSON error event, data is the concise diagnostic intended for ordinary display. An implementation may additionally provide details, containing a display form of the Prolog exception term with implementation-supplied context elided. Both fields

are presentation text, and clients must not parse them as a stable error-classification interface; in particular, `details` is not a claim about the exact runtime term. For a toplevel error, Prolog-facing PTCP messages and responses using `format=prolog` retain the actual caught exception term. Monitor notifications instead use the boundary form described below. Implementations may further redact paths, goal arguments, or other sensitive information from client-facing diagnostics.
A node that connects to another node's WebSocket endpoint announces itself with the `X-Web-Prolog-Protocol` header. Its `success` and `error` events then also carry a `term` field: the canonical text of the solution list or of the exception term. The receiving node restores the exact terms from it, so a toplevel on another node answers its Prolog client with the same terms as a local one. Browsers do not send the header and receive the display fields only.
An error that a client program is expected to act on carries a `code` field, a stable string. At present the only code is `session_lost`, described in Section D.5.

D.3 The toplevel communication protocol

PTCP is the logical protocol spoken by a toplevel actor. It is independent of whether commands arrive as actor messages, HTTP requests, or WebSocket frames. The HTTP operation `poll` is not a PTCP command: it merely retrieves an event already waiting in a transport queue.

Operation	Purpose	Principal outcomes
`call`	Submit a goal and request its first solution slice	`success`, `failure`, `error`
`next`	Continue a suspended solution sequence	`success`, `failure`, `error`
`stop`	Abandon a suspended solution sequence	`stop`
`abort`	Interrupt the current computation and return a persistent session to its idle state	`abort`
`halt`	Terminate the toplevel actor from any protocol state	`halted`, and a `down` event for each monitor
`respond`	Supply input requested by a prompt	`responded`

The distinction between `stop`, `abort`, and `halt` is semantic. `stop` applies while the actor is waiting between solution slices. `abort` interrupts a running goal but preserves a persistent session; it ends a toplevel spawned with `session(false)`, leaves a persistent toplevel idle in **s1** unchanged, abandons the suspended sequence in **s3**, and is ignored once termination of the toplevel has been committed. `halt` crosses the final lifecycle boundary and terminates the actor. Unless termination has been committed, the toplevel acknowledges every abort, including one that reaches it idle in **s1**, with `abort(Pid)`, sent to the client of its most recent call (before any call,

to its spawn target) after any answers of the aborted query. An abort that races the end of a query is therefore still answered. Abort and halt are runtime signals, not PTCP mailbox messages. Abort bypasses application recovery while running supported cleanup and is consumed by the privileged PTCP boundary; halt is untrappable and terminates the actor. Sections 2.3.3 and 3.3.3 give the normative signal semantics.

A toplevel serves one controlling client at a time, and each `call` starts afresh: `next`, `stop` and input that arrived before the call are discarded. PTCP messages carry no query identifier; per-sender ordering, this rule and the abort acknowledgement together tell a client which query an answer belongs to. When a goal raises an exception after a finite `limit` has collected some solutions, the slice is reported as `success` with more to come, and the error answers the next `next`.

For a remotely linked actor, the child-hosting controller maintains the lifetime relation. Known parent termination is propagated explicitly. The demonstrator scopes remotely hosted actors to their trusted WebSocket and lineage and tears down the owned subtree when that transport reports close, error, or lease expiry. Several logical links may share the connection, so this carrier does not replace explicit per-parent propagation. The default 30-second ping/pong lease bounds lost-contact detection while the runtime services the connection. It does not prove remote termination or bound safe-point delays. Contact loss yields `connection_closed`, not a confirmed actor outcome; an interrupted remote spawn raises `error(remote_spawn_failed(connection_closed),Context)`.

D.4 The stateless HTTP binding

RELATION and ISOBASE expose `GET /call`. The only required parameter is `goal`.

Parameter	Meaning	Default
`goal`	Goal as Prolog term text	required
`template`	Answer template sharing variables with the goal; used with `format=prolog`	the goal
`offset`	Zero-based offset in the solution sequence	`0`
`limit`	Maximum number of solutions in the returned slice	no solution-count limit
`once`	Commit to the returned slice even if further solutions exist	`false`
`src_text`	Source loaded into the temporary execution context	empty
`timeout`	Requested execution timeout, capped by node policy	node policy
`format`	`json` or `prolog`	`json`

On a RELATION node, `goal` must match an advertised relation pattern and `src_text` is forbidden. ISOBASE permits general stateless goals and source loading, subject to policy. The client-side `rpc/2-3` predicate converts its other `src_*` options to `src_text` before making this request.

With `format=json`, the server replaces the supplied template with an object template for the visible named query variables. To retrieve an explicit template, use `format=prolog`. An omitted `limit` requests all solutions. An explicit limit must be a positive integer; neither `infinity` nor `infinite` is accepted. Enumeration either completes or reports an error when an execution-time or result-size bound is exceeded. Execution-time exhaustion is reported as `error(resource_error(time), _)`; memory or result-size exhaustion as `error(resource_error(space),_)`. The context argument is elided in normal display. RPC clients propagate the remote exception without an additional wrapper. A client transport timeout is a separate condition and does not establish that the remote computation has stopped. An owner-side resource bound must not silently truncate a successful answer. A finite client-supplied limit still selects a slice normally; see Section 4.2.4.

The response is `success`, `failure`, or `error`. The server may retain an internal continuation to serve later offsets efficiently, but this cache is not part of the interaction contract: the client holds no session identity and an equivalent node may recompute the slice.

D.4.1 HTTP method semantics and caching

HTTP safety and idempotence concern the requested effects of an operation. GET is a safe method: the client requests retrieval rather than a change to application state. Idempotence means that repeating an identical request has the same intended effect as making it once; it does not require identical response content. A read-only query may therefore return different answers after the owner updates the database without becoming non-idempotent. Putting an effectful goal in a GET URI does not make the requested operation safe: the endpoint must uphold the method's semantics for the goals it admits.[6]

HTTP caches store response representations. Storage and reuse depend on the request method, response status, cache directives, authorisation, and freshness or validation. JSON and Prolog representations must also be distinguished when selecting a cached response, for example through `Vary: Accept` when that header selects the representation. A bookmarkable URI alone does not establish cacheability.[7] These rules apply independently of whether the node obtained the answer by re-executing a goal or resuming a cached computation.

An explicitly addressed session has different operation semantics. Its `next` operation advances the current computation, and repeating that operation can advance

[6] RFC 9110, Sections 9.2.1–9.2.2, `https://www.rfc-editor.org/rfc/rfc9110.html#section-9.2.1`.

[7] RFC 9111, Sections 3–4, `https://www.rfc-editor.org/rfc/rfc9111.html#section-3`.

it again. The semi-stateful binding below therefore prohibits caching its event-consuming responses. This is a rule for those operations, not a general property of every resource associated with a session.

D.5 The semi-stateful HTTP binding

ISOTOPE maps PTCP onto a family of HTTP endpoints. Spawning a session through this binding returns a pid that identifies the persistent session. Every later operation is authorised against that session and requires its `pid`.

Endpoint	Parameters	Immediate response
`/toplevel_spawn`	`options`, `statechart_trace`, `format`	`spawned` or `error`
`/toplevel_call`	`pid`, `goal`, `template`, `offset`, `limit`, `once`, `src_text`, `timeout`, `format`	next session event
`/toplevel_next`	`pid`; optional `limit`, `timeout`, `format`	next session event
`/toplevel_poll`	`pid`, `timeout`, `format`	next queued session event
`/toplevel_respond`	`pid`, `input`, `format`	`responded` or `error`
`/toplevel_stop`	`pid`, `format`	`stop` or `error`
`/toplevel_abort`	`pid`, `format`	`abort` or `error`
`/toplevel_halt`	`pid`, `format`	`halted` or `error`
`/toplevel_trace`	`pid`, `statechart_trace`, `format`	`responded` or `error`

All ISOTOPE endpoints use POST. Parameters are normally sent as URL-encoded form data in the request body; spawn also accepts a JSON body. The `options` value of `/toplevel_spawn` is a Prolog option list, and source options belong in that list. The endpoint forces `session(true)`, irrespective of a supplied `session/1` option. The list may contain `time_limit(SecondsOrInfinite)` and `idle_limit(SecondsOrInfinite)`. The former applies only while the session executes a goal in PTCP state **s2**; the latter applies while it waits in **s1** or **s3**. Public clients may request tighter values, but cannot weaken the node owner's lifecycle ceilings. These session operations are not safe retrieval requests and their responses must not be cached: even polling dequeues an event from a persistent session.

`/toplevel_call` uses the same query parameters and defaults as `/call`, with the addition of the required session pid. If `limit` is omitted from `/toplevel_next`, the limit of the active call is inherited.

The HTTP binding still accepts nonempty `src_text` on `/toplevel_call`: changed source is validated and reloaded before the goal runs. Empty source and source identical to the last successful load do not trigger a reload. This is a compatibility path; clients should normally supply source through spawn options. In particular, it must not be copied into a WebSocket call, which rejects call-time source.

The call and next endpoints wait for one event. If output or a prompt arrives before the solution slice, the client receives that event first and uses `/toplevel_poll` to collect the following event. Polling does not advance the solution sequence. A request timeout is capped by node policy and applies at this blocking surface. It returns a non-destructive `timeout` event: it neither aborts the PTCP goal nor terminates the session, and a later poll may retrieve the result.

`/toplevel_abort` waits for acknowledgement. An `abort` event marks the end of the aborted query's events; earlier queued events are discarded. The endpoint can instead return `timeout` if the wait expires, or `down` if the actor terminates before acknowledgement.

Because an HTTP response can be lost after the node has acted, each event-consuming session response carries a `Web-Prolog-Seq` header that numbers the event it returns. A client that receives no response may repeat the request with a `Web-Prolog-Ack` header holding the last number it did receive. If that number is lower than the session's latest, the node returns the latest event again instead of performing the operation a second time; a repeated `next` therefore cannot skip a page. The headers leave the JSON bodies unchanged, and a client that ignores them behaves as before. Spawn and halt responses do not carry this header. Retrying a spawn creates another session; the sequence mechanism does not make session creation idempotent.

A session's former owner that addresses it after it has ended, expired or been reclaimed receives an `error` event with `code session_lost`. Other principals receive the ordinary authorisation error, which does not reveal whether the session ever existed.

For a `down` event, the `reason` field preserves the contract in Appendix B: success, logical failure, uncaught exception, explicit exit term, `no_process`, or `connection_closed`. The last reports loss of a remote monitoring relationship rather than observed actor death; it discharges the monitor and is not silently reversed after reconnection. JSON encodes `reason` as a string containing the corresponding public Prolog term, without collapsing these cases into a single error string. When an uncaught SWI-Prolog `error(Formal,Context)` crosses the monitor boundary, `Formal` is preserved while `Context` is elided. A private generated actor-module qualification is also omitted from an undefined-procedure error. Other exit reasons are unchanged.

D.6 The stateful WebSocket binding

ACTOR exposes `/ws`. Each client frame is a JSON object with a mandatory `command` field; each server frame has a mandatory `type` field. Events are pushed as they occur, so no polling command is needed. Actors and sessions created through a connection are owned by it and are terminated when the connection closes.

D.6.1 Version negotiation

A client may begin with:

```
{"command":"transport_hello","version":1}
```

A compatible node answers with `type:"transport_welcome"`, the protocol name, and version 1. Version 1 clients may omit this exchange, but explicit negotiation is recommended for new clients.

The welcome frame currently has the following shape:

```
{"type":"transport_welcome",
 "protocol":"web_prolog_browser_actor","version":1,
 "browser_pids":true,"io_ack":true}
```

The two Boolean fields advertise supported extensions. Each extension is enabled for a connection only if the client requests it with the matching `true` field in `transport_hello`. An unsupported version produces a protocol error. The browser extensions are described below.

Node-to-node spawn commands also carry a fresh decimal-string `request_id`, echoed on the corresponding `spawned` or spawn-error response. The sender matches that request before accepting an acknowledgement; a late reply to an abandoned spawn cannot satisfy the next spawn. Older peers omitting the field retain legacy serial reply handling and do not provide this cancellation-race guarantee.

Trusted node-to-node commands carry optional `lineage` and `account` fields, each a 32-hex-digit token. The receiving node scopes ownership, visibility, and cleanup to the connection and lineage, and charges lineages sharing an account to one principal budget. Public clients may not supply these fields. An internal `lineage_close` command reclaims that connection's matching lineage when its source subtree ends, including spawns still awaiting acknowledgement. Peers omitting the fields retain per-connection scope for compatibility; that mode cannot isolate clients sharing a transport. These fields are a trusted-peer delegation mechanism, not authority granted by an ordinary actor pid.

D.6.2 Toplevel session commands

Command	Canonical fields and response
`toplevel_spawn`	`options`; optional `statechart_trace`. Events: `spawned`, `error`.
`toplevel_call`	`pid`, `goal`, `options`. Events: answer, output, or prompt.
`toplevel_next`	`pid`; optional JSON integer `limit`. Events: answer, output, or prompt.
`toplevel_stop`	`pid`. Event: `stop`.
`toplevel_abort`	`pid`. Event: `abort`.
`toplevel_halt`	`pid`. Events: `down`, then `halted`.
`toplevel_respond`	`pid`, `input`. Event: `responded`.
`set_statechart_trace`	`pid`, `statechart_trace`. Event: `responded`.

The `options` fields are Prolog option lists represented as text. For `toplevel_spawn`, the list may contain the source options and `session/1`. Unlike the ISOTOPE spawn endpoint, WebSocket spawn does not force a persistent session; the actor predicate's default is `session(false)`.

For `toplevel_call`, the canonical options are drawn from:

```
[template(T), offset(I), limit(K), once(B), format(F)]
```

The defaults are the goal as template, offset `0`, no solution-count limit, `once(false)`, and `format(json)`. JSON formatting replaces the template with named variable bindings, as in HTTP. With `format(prolog)`, solution elements are instead rendered as Prolog-term strings inside the JSON `data` array; WebSocket frames remain JSON. Goal and options are parsed together to preserve variable sharing. Unknown call options, including source-loading and `timeout` options, are rejected. The session's lifecycle limits govern running time.

A top-level `src_text` field on `toplevel_call` is rejected, even when empty. Source belongs in the `toplevel_spawn` options. For compatibility, spawn still accepts a separate `src_text` field, and calls without `options` may use flattened `template`, `offset`, `limit`, `once`, and `format` fields. When `options` is supplied, that list takes precedence. Omitting `limit` from `toplevel_next` preserves the active call's limit.

The session's `statechart_trace` flag defaults to `false` in both HTTP and WebSocket spawn bindings. The current demonstrator stores and acknowledges this flag, but its event delivery code does not consult it to suppress statechart traces. Control actual trace generation with the individual statechart's `trace/1` spawn option. The older spawn field `trace` is still accepted as a fallback.

D.6.3 Bare actor commands

Command	Fields	Response or event
`spawn`	`goal`; optional `options`	`spawned`
`send`	`pid`, `message`	none
`monitor`	`pid`, `ref`	a later `down` event
`demonitor`	`ref`	none
`exit`	`pid`; optional `reason`	a later `down` event for each monitor

The target of `send` may be an actor owned by the connection or an owner-published service for which access is authorised. Send is fire-and-forget. A send to a missing, unpublished, or unauthorised service is accepted without delivery and does not reveal which condition applied. A binding may reject a command when no usable connection exists, but transport loss after acceptance creates no send response; delivery assurance requires an application acknowledgement. The monitor reference is supplied by the client and is returned unchanged in the corresponding `down` event, allowing several monitors involving the same pid to be distinguished. References are supplied as term text and must be fresh and single-use. Production references should also resist guessing; the demonstrator's compact ten-digit references have only about 33 bits of entropy. Wide numeric references return as decimal strings, as described above; clients compare decoded values rather than JSON string types. A party that legitimately holds a reference can construct a matching event, so the reference does not authenticate the sender. Monitoring uses the send-target policy; exit requires ownership, and demonitor removes only monitors installed by the calling connection. The default exit reason is `kill`.

The connection installs its own lifecycle monitor when it creates an actor or toplevel. Its `down` event has a node-generated reference distinct from references explicitly supplied by the client. Removing an explicit monitor does not suppress this connection lifecycle event.

D.6.4 Browser actor addresses and output acknowledgements

A browser runtime requests both extensions with:

```
{"command":"transport_hello","version":1,
 "browser_pids":true,"io_ack":true}
```

With `browser_pids` enabled, qualified browser-local pids such as `21594388@localhost` and `main@localhost` embedded in spawn goals, toplevel goals/templates, and sent messages acquire a return route scoped to that connection. Bare numbers, ordinary atoms, and source strings remain data. A message addressed to such a browser actor is delivered as:

```
{"type":"actor_message","target":"21594388@localhost",
 "message":"reply(ok)"}
```

Both `target` and `message` are Prolog term text. This route does not make browser actors independently reachable from the network and does not survive connection closure.

With `io_ack` enabled, terminal output uses an acknowledgement exchange. An output event is wrapped in a request:

```
{"type":"io_request","request_id":"opaque-reference",
 "event":{"type":"output","pid":123,"data":"hello"}}
```

After accepting the output, the client responds on the same connection:

```
{"command":"browser_io_reply","request_id":"opaque-reference",
 "status":"ok"}
```

The alternative status is `error`. Only the owning connection can satisfy a pending request. The actor's I/O operation waits for acceptance; an error status, connection closure, or a 30-second acknowledgement timeout raises an I/O error. Late replies are discarded. This acknowledges output acceptance, not completion of browser painting. Prompts and query answers continue to use their ordinary event forms. Without this extension, output is delivered directly as an `output` event.

Node-to-node terminal forwarding additionally uses private transport commands protected by `internal_transport` authority. Those commands are not part of the ordinary browser-client command surface.

D.7 Lifecycle and ownership

HTTP sessions and WebSocket-owned actors are resources scoped to their creating principal. A client cannot operate on an arbitrary pid merely by knowing its printed form. The exception is sending to an owner-published service when policy grants access; publication does not grant control over the service actor's lifecycle.

For commands originating from one connection, repeated sends to the same pid use that connection as one ordered routing epoch. Reconnection begins a new epoch and carries no portable order relative to sends accepted before the disconnect. Sends through a service name are ordered only while the name's binding is unchanged. There is no ordering guarantee between messages and runtime signals.

Closing a WebSocket terminates the actors and sessions owned by that connection. An ISOTOPE session instead survives individual HTTP requests until it is halted, expires under node policy, or the node terminates. A transport timeout, network failure, or abandoned client choice point therefore does not by itself prove that a remote computation has ended. Clients that create persistent remote work remain responsible for bounded waiting and explicit cleanup.

D.8 Starting a native node

Starting a node is an owner operation, not part of the predicate API available to client programs. From a source checkout, the normal entry point is the environment-driven launcher in `Deployment/start_node.pl`. For local development, the following command starts an ACTOR node on port 3060 and grants the execution capability to clients connecting directly from the local machine:

```
WP_AUTH=dev WP_PORT=3060 ./Deployment/start_node.pl
```

Then open `http://localhost:3060/demonstrator`. The bare node URL, `http://localhost:3060/`, is not a landing page: by design it returns the node's shared-database source, which in this case is the default example database.

The process remains in the foreground until interrupted. It prints the effective configuration at startup and handles an interrupt or termination signal by putting the node into maintenance, allowing current work a bounded grace period, and then stopping the server.

Configuration is supplied through `WP_*` environment variables or a configuration file selected by `WP_CONFIG`. The most important variables are `WP_PORT`, `WP_PROFILE`, `WP_AUTH`, `WP_SANDBOX`, `WP_TIMEOUT`, and `WP_SHARED_DB_FILE`. A configuration can be validated without binding a port:

```
WP_CHECK=1 WP_PROFILE=actor WP_AUTH=private \
    ./Deployment/start_node.pl
```

The launcher defaults to private authentication. Starting a world-open node requires both `WP_AUTH=open` and the explicit acknowledgement `WP_ACK_PUBLIC=yes`, since such a node accepts code from untrusted network clients.

For interactive experimentation or for embedding a node in a larger native SWI-Prolog application, the underlying owner interface is `node/1-2`:

```
?- use_module(library(web_prolog)).
?- node(3060, [profile(actor), auth(dev)]).
```

The first form, `node(Port)`, is equivalent to `node(Port,[])`. The second accepts startup options for the node profile, authentication and sandbox policy, shared-database source, request timeout, PTCP `time_limit` and `idle_limit`, cache policy, resource limits, rate limits, and HTTP server configuration. It is an embedding interface for the node owner rather than a Web Prolog predicate offered to clients. The routes created by the server are therefore documented by interaction profile in the preceding sections instead of being repeated as a deployment-time list.

Appendix E
How to implement a Prolog node

The preceding chapters have described Web Prolog chiefly from the programmer's point of view. This appendix turns the system around and asks what a node implementer must provide. The answer depends on the node's profile, its host Prolog system, and the environment in which it is deployed.

At the time of writing, the demonstrator provides substantial implementations of all four profiles, including native and browser-resident actors, cross-node communication, source installation, statechart actors, authentication, policy enforcement, resource limits, and administrative facilities. It should nevertheless be regarded as a research implementation, not as a final interoperable standard or a production platform suitable for arbitrary deployment. SWI-Prolog remains the principal proof-of-concept implementation and the basis of the code explained here. Complementary GNU Prolog ISOBASE and Trealla ACTOR prototypes now allow selected observable contracts to be tested across runtimes; their scope and limitations are described in Section E.6.2. These experiments do not yet establish complete cross-system conformance.

The distinction matters throughout this appendix. The public interfaces are specified in Appendix B; the code below is explanatory. Some fragments closely follow the demonstrator, while others deliberately omit machinery that would obscure the central idea. Whenever a fragment is pedagogical rather than complete, we say so and identify what a deployable implementation must add.

E.1 From a profile to a running node

A Prolog node is not simply a Prolog system with an HTTP server attached. Even a node implementing the narrowest public profile must translate between a network request and a Prolog computation while preserving the contracts for terms, answers, errors, limits, and profiles. Broader profiles add persistent execution contexts, actors, distribution, and lifecycle management.

The profiles therefore imply a progression of implementation responsibilities:

RELATION
: Parse a call to an owner-defined relation, execute it within the advertised relation surface, and encode its answers. Such a node need not implement ISO Prolog generally and need not itself be written in Prolog.

ISOBASE
: Add the advertised ISOBASE language fragment, the stateless `/call` API, term and error conversion, and bounded nondeterministic query evaluation.

ISOTOPE
: Add private execution contexts, source installation, and the semi-stateful toplevel API. The node must retain query state between requests and reclaim it when a session is stopped, halted, abandoned, or expired.

ACTOR
: Add actors, mailboxes, selective receive, links, monitors, naming, remote placement, actor I/O, and the stateful WebSocket protocol. The actor profile also requires a clear account of isolation and of the lifetime relationship between a client, a connection, and the actors created through it.

These are language and protocol responsibilities. A public deployment adds another layer: authentication, authorisation, sandboxing, quotas, overload control, transport security, logging, observability, and administration. A small teaching implementation can omit that layer. An Internet-facing node cannot.

E.1.1 A useful decomposition

The native demonstrator separates the request path into several layers. The exact module boundaries are implementation choices, but the decomposition is portable:

1. A transport adapter accepts HTTP or WebSocket input and converts it into a typed internal request.
2. Authentication identifies a principal, and profile and permission checks decide whether that principal may perform the requested operation.
3. Input validation checks term size, syntax, source options, relation visibility, and the permitted language fragment.
4. An execution-context layer creates or selects the private database, actor, toplevel, or stateless query engine in which the goal will run.
5. The runtime applies time, inference, memory, actor, mailbox, and result limits as appropriate.
6. A response layer converts success, failure, errors, prompts, output, and lifecycle events into the representation required by the transport.

This order is significant. Validation must occur before untrusted source is loaded or an untrusted goal is called, and resource ownership must be established before work is started. Conversely, transport-specific JSON or WebSocket details should not leak into the actor and query semantics.

E.2 Notes on node implementation

Chapter 3 specifies node behaviour primarily in terms of externally observable contracts: profiles describe what a client may rely upon, and the web APIs describe the shapes of requests and responses. Implementation techniques are not part of the contract, and different Prolog systems – and different host platforms – will naturally realise the contracts in different ways.

E.2.1 Statelessness as a protocol property

The stateless HTTP API is *stateless for the client*: each request contains all information needed to interpret it, and the node is not required to remember a conversational session in order to respond. This does not prohibit the node from keeping internal caches. It merely means that caches are best-effort optimisations whose presence must not be observable as a behavioural dependency. A node may evict cached state at any time and still remain correct and conformant.

As noted in Section 3.5.1, statelessness allows self-contained requests with meaningful URIs; safe repetition and response caching require their own conditions. A corresponding limitation is that a stateless request cannot be *externally* interrupted by the client once evaluation has begun. There is no control channel associated with the computation, and no session actor to signal. In practice, therefore, the stateless API must rely on node-enforced resource policy, most importantly running timeouts, to avoid nonterminating or overly expensive computations.

Paged solution retrieval

The `offset` and `limit` parameters make it possible to retrieve solutions in slices without prescribing how a node retains or recomputes search state. That choice is outside the node profile: clients observe solution slices, ordering, and continuation metadata, not the caching or execution strategy behind them. Sections 4.2.2 and 4.2.4 described restart-and-skip, save-and-resume, cursor semantics, and the consistency limits of paged retrieval in their distributed setting.

E.2.2 Semi-stateful HTTP sessions and response retrieval

The semi-stateful HTTP API differs from the stateless API in one crucial respect: each request targets a long-lived toplevel actor, identified by a pid, and the observable behaviour of the conversation depends on the evolving state of that actor. Nevertheless, each individual HTTP exchange is still an ordinary request–response

transaction. In this sense, the protocol is *semi-stateful*: the transport remains stateless, but the addressed computation is not.

From an implementation perspective, this style aligns naturally with the architecture of SWI-Prolog's `library(pengines)` and systems such as SWISH: the server maintains per-session engines, while the client repeatedly issues short requests that either advance the computation, or retrieve the next available response produced by that computation.

Output and prompts fit this model once they are treated as ordinary protocol events emitted by the session actor, rather than as exceptional side effects. Concretely, the toplevel actor can be understood as producing a sequence of *responses*: success, failure, and error answer terms, as well as output fragments, prompts, and acknowledgements. The client retrieves this sequence over a series of short HTTP exchanges.

The statechart in Figure E.1 extends and specialises the core PTCP in Figure 3.3 with the states and transitions needed for HTTP delivery.

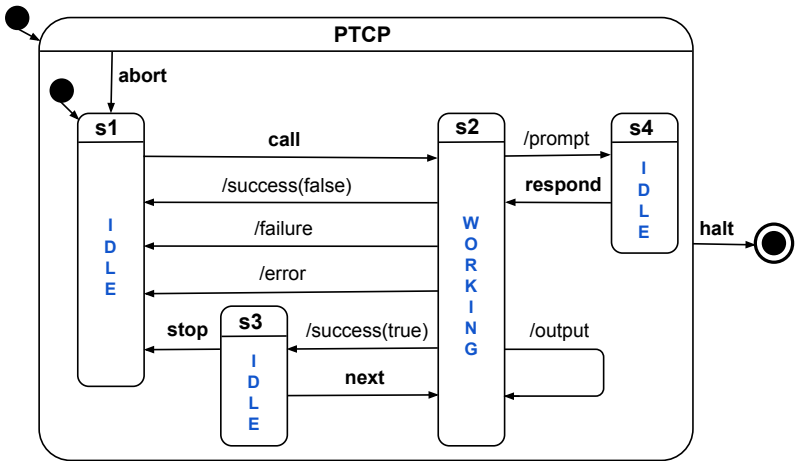


Fig. E.1 Statechart specifying the PTCP for a successful conversation with a toplevel over HTTP.

The additional state **s4** shown in the figure belongs to the HTTP transport mapping rather than to PTCP itself. It records that the client has received a prompt and must send a **respond** request before the toplevel can resume work in **s2**. Output does not require another protocol state: an `/output` event is dequeued while the computation remains notionally in **s2**, and the client immediately calls the `/toplevel_poll` endpoint (the **poll** transition in the figure) for the next pending event. This local pull cycle repeats until the toplevel produces a terminal response (`/success`, `/failure`, `/error`).

Prompts require one additional step: when the next pending response is a prompt, the client retrieves it, presents it to the user, and sends a follow-up request that

delivers the user's reply back to the session actor. From the protocol's perspective, this is still a request–response rhythm and no polling is needed here; the client is simply advancing the conversation by alternately retrieving pending events and, when prompted, supplying input. Ordering is preserved because responses are dequeued in the order they were produced. Transient transport failures may delay retrieval but do not change the logical meaning of the session.

E.2.3 Stateful WebSockets and push-driven interaction

The stateful WebSocket API exists for interactions that are naturally push-driven. Once a connection is established, the node can deliver answers, output fragments, prompts, and monitor events as soon as they are produced, without requiring a new client request for each response. This improves latency and avoids the request choreography of our long-polling trick. It also supports interleaved traffic from multiple actors over one connection, which is central to ACTOR-profile use cases.

Implementation complexity, however, shifts from request handling to connection lifecycle management. A robust node must maintain per-connection routing state (which pids belong to which client), preserve ordering within each actor stream, and enforce outbound back-pressure so that a slow client cannot cause unbounded buffering on the node. It must also define clear disconnect semantics: when a client vanishes, session actors and child actors are either terminated or detached according to policy, and monitored parties observe the corresponding `down/exit` events.

E.3 A native node on top of SWI-Prolog

The native demonstrator wraps a node around SWI-Prolog. Native threads supply the scheduling substrate for actors, temporary modules supply private clause databases, and the HTTP and WebSocket libraries supply the network transports. This is not the only possible design, but it shows how much of a node can be implemented as a library on top of a mature Prolog system.

E.3.1 The stateless HTTP endpoint

Chapter 5.3 developed answer terms and Chapter 4 showed how `rpc/2-3` reconstructs Prolog nondeterminism from stateless HTTP responses. We do not repeat those client-side mechanisms here. Instead, the following program implements a small but complete stateless endpoint. It accepts Prolog-formatted calls only; JSON, source installation, and policy enforcement are added later by broader-profile implementations.

```
:- use_module(library(http/http_server)).
:- use_module(library(http/http_parameters)).
:- use_module(library(solution_sequences)).

:- http_handler(root(call), call_handler, []).

node(Port) :-
    http_server(http_dispatch, [port(Port)]).

call_handler(Request) :-
    http_parameters(Request, [
        goal(GoalAtom, [atom]),
        template(TemplateAtom, [
            atom,
            default(GoalAtom)
        ]),
        offset(Offset, [integer, default(0)]),
        limit(Limit, [
            integer,
            optional(true)
        ]),
        format(prolog, [default(prolog)])
    ]),
    parse_goal_template(
        GoalAtom, TemplateAtom, Goal, Template
    ),
    compute_answer(Goal, Template, Offset, Limit, Answer),
    reply_prolog(Answer).

parse_goal_template(GoalAtom, TemplateAtom, Goal, Template) :-
    atomic_list_concat([
        '(', GoalAtom, ')+(', TemplateAtom, ')'
    ], Text),
    read_term_from_atom(Text, Goal+Template, [
        syntax_errors(error), double_quotes(string)
    ]).

compute_answer(Goal, Template, Offset, Limit, Answer) :-
    catch((once(answer(Goal, Template, Offset, Limit, Candidate)),
           term_attvars(Candidate, Vars),
           (Vars == [] -> true;
            throw(error(representation_error(constraints), _))),
           Answer = Candidate),
          E, Answer = error(E)).
```

```
answer(Goal, Template, Offset, Limit, Answer) :-
    Partial = collected([]),
    catch(
        call_cleanup(
            slice(Goal, Template, Offset, Limit, Partial, Slice),
            Deterministic = true
        ),
        Error,
        Caught = caught(Error)
    ),
    (   nonvar(Caught)
    ->  arg(1, Partial, Reversed), reverse(Reversed, Saved),
        (nonvar(Limit), Saved \== []
        -> Answer = success(Saved, true)
        ;  Caught = caught(E), Answer = error(E))
    ;   Slice == []
    ->  Answer = failure
    ;   var(Deterministic)
    ->  Answer = success(Slice, true)
    ;   Answer = success(Slice, false)
    ).

slice(Goal, Template, Offset, Limit, Partial, Slice) :-
    (   var(Limit)
    ->  findall(Template, offset(Offset, Goal), Slice)
    ;   must_be(positive_integer, Limit),
        findnsols(Limit, Template,
          (offset(Offset, Goal),
           arg(1, Partial, Prior),
           nb_setarg(1, Partial, [Template|Prior])), Slice)
    ).

reply_prolog(Answer) :-
    format(
        'Content-type: application/x-prolog; charset=UTF-8~n~n'
    ),
    write_term(Answer, [
        quoted(true),
        ignore_ops(true),
        numbervars(false),
        fullstop(true),
        nl(true)
    ]).
```

The predicates `answer/5` and `slice/6` correspond directly to the two terms used throughout the protocol description. `answer/5` constructs an answer term of the form

`success(Slice,More)`, `failure`, or `error(Error)`. For a successful answer, `slice/6` constructs the solution slice: the list of instances of `Template` placed in `Slice`.

The goal and template are parsed together so that occurrences of the same variable name denote the same variable. A call to `once/1` makes the HTTP handler deterministic even though `answer/5` is itself nondeterministic. An omitted `limit` leaves `Limit` unbound in this HTTP sketch and selects `findall/3`. An explicit positive integer selects `findnsols/4`; there is no default solution-count bound. A finite slice records collected solutions separately, so a later exception can return the partial page with `More = true`.[1] A subsequent request with the advanced offset repeats the pure search and reaches that exception. The wrapper rejects attributed answers, including partial pages, and the writer uses canonical term syntax. The complete node also retains a pending error in its bounded continuation cache, avoiding this recomputation.

This program must be bound only to a trusted local interface. It executes arbitrary goals with the authority of the host Prolog process. The complete node surrounds the call to the query engine with parsing limits, authentication, profile and relation checks, sandbox validation, a private execution context, time and inference limits, maintenance and overload checks, and response logging. Those are substantial subsystems, so we describe their placement rather than disguise them as undefined predicates in an otherwise executable example.

E.3.2 Avoiding spurious recomputation

The `compute_answer/5` implementation above is correct, but successive HTTP requests may repeat work. Consider a goal whose first solution is expensive and whose second solution is cheap:

```
?- time((sleep(1), X=foo ; X=bar)).
X = foo ;
X = bar.
?-
```

If the two solutions are requested in separate slices, a purely offset-based implementation recomputes the expensive first branch when serving the second request. More generally, computing the nth slice requires recomputing and discarding the preceding $n-1$ slices. Such recomputation also repeats any side effects performed while reaching those earlier solutions, so goals paged in this way should be pure whenever repeated effects would be observable or unsafe.

The demonstrator avoids this by retaining a bounded cache of suspended toplevel actors. After producing a non-final slice, a toplevel is waiting at the PTCP slice

[1] SWI-Prolog's `nb_setarg/3` copies the assigned non-atomic term, so the saved answers survive subsequent backtracking and exception unwinding.

boundary with its remaining choice points intact. The node may associate that continuation with the next offset. A later request for the same computation can resume the actor rather than repeat the search.

This does not turn the HTTP interface into a client session. The cached actor is an internal optimisation: the client neither receives its pid nor gains exclusive ownership of it. The node may expire or evict it, in which case a later request is recomputed from the information carried by that request. A request without a limit starts a fresh collection of all remaining solutions; it does not resume a cached finite slice.

For pure goals over stable code and dependencies, replay and resumption can produce the same answer sequence. A stateless request can then be routed to another backend that supplies an equivalent execution context, even if that backend must recompute. The absence of session affinity does not by itself make different backends' code, data, or policies agree. An explicit session, by contrast, must be routed to the identified actor; losing it must be reported rather than silently substituting a fresh execution.

Keys, ownership, and cleanup

A cache key must contain every input that can affect the computation, including the goal, template, installed source, and relevant evaluation options. Treating a bounded `term_hash/2` result as a unique identity would be unsound unless collisions are checked. A simple pedagogical implementation can instead retain the canonical term itself:

```
goal_key(Goal-Template-Source-Once, Key) :-
    copy_term(Goal-Template-Source-Once, Key),
    numbervars(Key, 0, _).
```

A production key may use a hash for indexing, but it must still establish equality of the full canonical input. It may also have to include the node instance or execution-policy context so that continuations are not mixed across incompatible authorities.

The following sketch exposes the control structure of the cache. It assumes a node-internal adapter, named `cache_toplevel_*` below, around the toplevel behaviour of Section 5.3. These names denote host integration points, not additional Web Prolog predicates. The adapter establishes a node-managed execution context and loads supplied source when called from an HTTP worker, delivers PTCP answers to the private host queue given by `target(Queue)`, and updates that destination on each call or resumption. A host queue is not an actor pid and cannot simply be passed to the public actor send operation. The cached toplevel outlives the creating request: `link(false)` leaves its lifetime under cache control, and the adapter's halt operation terminates it on disposal. The adapter's implementation is omitted so that the listing stays focused on caching:

```
:- use_module(library(aggregate)).
```

```
:- dynamic cached/3.

max_cache_size(100).

compute_answer_cached(Goal, Template, Offset, Limit,
                      Source, Answer) :-
    goal_key(Goal-Template-Source-false, Key),
    setup_call_cleanup(
        message_queue_create(Queue),
        compute_answer_cached(
            Queue, Key, Goal, Template, Offset, Limit, Source,
            Answer
        ),
        message_queue_destroy(Queue)
    ).

compute_answer_cached(Queue, Key, Goal, Template, Offset,
                      Limit, Source, Answer) :-
    (   nonvar(Limit) -> LimitOptions = [limit(Limit)]
    ;   LimitOptions = []
    ),
    (   nonvar(Limit), take_cached(Key, Offset, Pid)
    ->  cache_toplevel_next(Pid, [
            limit(Limit),
            target(Queue)
        ])
    ;   spawn_options(Source, Queue, SpawnOptions),
        cache_toplevel_spawn(Pid, SpawnOptions),
        cache_toplevel_call(Pid, Goal, [
            template(Template),
            offset(Offset),
            target(Queue)
          | LimitOptions
        ])
    ),
    await_slice(Queue, Key, Offset, Limit, Pid, Answer).

spawn_options('', Queue, [
    session(false),
    link(false),
    target(Queue)
]) :-
    !.
spawn_options(Source, Queue, [
    session(false),
```

```
    link(false),
    target(Queue),
    src_text(Source)
]).

await_slice(Queue, Key, Offset, Limit, Pid, Answer) :-
    (   thread_get_message(Queue, Message, [timeout(30)])
    ->  slice_message(
            Message, Key, Offset, Limit, Pid, Answer
        )
    ;   stop_toplevel(Pid),
        Answer = error(error(resource_error(time), _))
    ).

slice_message(success(Pid, Slice, true),
              Key, Offset, _Limit, Pid,
              success(Slice, true)) :-
    length(Slice, Count),
    Next is Offset + Count,
    save_cached(Key, Next, Pid).
slice_message(success(Pid, Slice, false),
              _, _, _, Pid, success(Slice, false)).
slice_message(failure(Pid),
              _, _, _, Pid, failure).
slice_message(error(Pid, Error),
              _, _, _, Pid, error(Error)).
slice_message(Message, _, _, _, Pid,
              error(unexpected_message(Message))) :-
    stop_toplevel(Pid).

take_cached(Key, Offset, Pid) :-
    with_mutex(
        continuation_cache,
        once(retract(cached(Key, Offset, Pid)))
    ).

save_cached(Key, Offset, Pid) :-
    with_mutex(
        continuation_cache,
        (
            assert(cached(Key, Offset, Pid)),
            evict_excess(Evicted)
        )
    ),
    dispose_cached(Evicted).
```

```
evict_excess(Evicted) :-
    max_cache_size(Max),
    aggregate_all(count, cached(_, _, _), Count),
    (   Count > Max
    ->  once(retract(cached(_, _, Evicted)))
    ;   Evicted = none
    ).

dispose_cached(none) :-
    !.
dispose_cached(Pid) :-
    stop_toplevel(Pid).

stop_toplevel(Pid) :-
    catch(cache_toplevel_halt(Pid), _, true).
```

The mutex makes acquisition and insertion race-safe. Eviction removes the oldest clause and terminates its toplevel rather than merely forgetting its pid. The next offset counts the solutions actually returned, including when an exception leaves a partial slice. With the host adapter in place, use this version in the HTTP controller by replacing its call to `compute_answer/5` with `compute_answer_cached/6` and passing the empty atom as the source.

The example has a size bound but no expiry clock. The demonstrator also associates an alarm with every cache entry, cancels the alarm when the entry is acquired, and terminates the toplevel when the entry expires. It further uses a node-specific cache key and preserves the timeout budget while discarding unrelated internal messages.

E.3.3 Implementing the actor primitives

The actor layer can be explained economically on a Prolog system that implements the ISO thread draft. The following host predicates provide the principal mechanisms:

Web Prolog	Host Prolog
spawn/1-3	thread_create/3
self/1	thread_self/1
send/2, !/2	thread_send_message/2
receive/1-2	thread_get_message/3
exit/2	thread_signal/2

This table is only a starting point. A thread identifier is not a Web Prolog pid, and a thread queue does not by itself implement links, monitors, names, remote addressing,

private databases, or the specified selective-receive semantics. The implementation must place an actor layer between the public predicates and the host thread API.

Pids and process creation

The small implementation below separates pids from host thread identifiers. The mappings also make it possible for a distribution layer to replace the local pid generator without exposing thread identities to programs.

```
:- use_module(library(option)).

:- op(800, xfx, !).

:- meta_predicate
    spawn(0),
    spawn(0, -),
    spawn(0, -, +).

:- dynamic pid_thread/2.
:- dynamic thread_pid/2.
:- dynamic link/2.
:- dynamic monitor/3.
:- dynamic exit_reason/2.

:- thread_local parent/1.

:- initialization(init_main_actor, after_load).

init_main_actor :-
    thread_self(Thread),
    (   thread_pid(Thread, _)
    ->  true
    ;   fresh_pid(Pid),
        assert(pid_thread(Pid, Thread)),
        assert(thread_pid(Thread, Pid))
    ).

self(Pid) :-
    thread_self(Thread),
    thread_pid(Thread, Pid).

fresh_pid(Pid) :-
    flag(web_prolog_pid, N, N + 1),
    Pid is 10 000 000 + N.
```

```
spawn(Goal) :-
    spawn(Goal, _).

spawn(Goal, Pid) :-
    spawn(Goal, Pid, []).

spawn(Goal, Pid, Options) :-
    self(Parent),
    fresh_pid(Pid),
    setup_call_cleanup(
        create_actor(Parent, Pid, Goal, Options),
        true,
        true
    ).

create_actor(Parent, Pid, Goal, Options) :-
    with_mutex(actor_lifecycle,
        create_actor_locked(Parent, Pid, Goal, Options)).

create_actor_locked(Parent, Pid, Goal, Options) :-
    install_lifetime_options(Parent, Pid, Options),
    thread_create(
        actor_boot(Parent, Goal),
        Thread,
        [at_exit(actor_stopped(Pid))]
    ),
    assert(pid_thread(Pid, Thread)),
    assert(thread_pid(Thread, Pid)),
    thread_send_message(Thread, '$start').

actor_boot(Parent, Goal) :-
    thread_get_message('$start'),
    assert(parent(Parent)),
    call(Goal).

install_lifetime_options(Parent, Pid, Options) :-
    option(link(Link), Options, true),
    (   Link == true
    ->  assert(link(Parent, Pid))
    ;   true
    ),
    option(monitor(Monitor), Options, false),
    (   Monitor == true
    ->  assert(monitor(Parent, Pid, Pid))
    ;   true
```

```
    ).
```

The call to `fresh_pid/1` binds the pid argument before `thread_create/3` copies the goal into the child. Thus a call such as `spawn(session(Pid, Parent), Pid, Options)` passes the child's pid as a value in the copied goal; it does not share a variable between threads. The same ordering is required for remote spawning, using the child's canonical pid before copying the goal for transport.

The counter in `fresh_pid/1` is sufficient for this single-runtime sketch. A node must prevent pid reuse and collision across concurrent allocation, and its distribution layer must add the node component of the canonical pid. The parent records the link and the spawn-time monitor *before* the child thread exists, and it does all of the creation as the setup goal of `setup_call_cleanup/3`, which SWI-Prolog protects from signals. A termination signal that reaches the parent while it is spawning therefore waits until the child, its mappings, its link and its monitor all exist. When the parent then terminates, its cleanup finds the link. The private `'$start'` message prevents the child's normal goal from running before the parent has installed the two mappings. If a host interruption nevertheless terminates the child during startup, its exit hook acquires the same `actor_lifecycle` mutex as creation, so cleanup waits for those mappings to exist before notifying the monitor and removing them. The early-termination guarantee in this sketch depends on that serialisation. The spawn-time monitor uses the pid itself as its reference, matching the public convention documented in Appendix B. A separately installed monitor would receive a fresh opaque reference.

When an actor terminates, the runtime applies the lifetime relations and reports the reason:

```
actor_stopped(Pid) :-
    with_mutex(actor_lifecycle, actor_stopped_locked(Pid)).

actor_stopped_locked(Pid) :-
    termination_reason(Pid, Reason),
    forall(
        retract(link(Pid, Child)),
        exit(Child, kill)
    ),
    retractall(link(_, Pid)),
    forall(
        retract(monitor(Watcher, Pid, Ref)),
        Watcher ! down(Pid, Ref, Reason)
    ),
    retract(pid_thread(Pid, Thread)),
    retractall(thread_pid(Thread, Pid)).

termination_reason(Pid, Reason) :-
    retract(exit_reason(Pid, Reason)),
    !.
```

```
termination_reason(Pid, Reason) :-
    pid_thread(Pid, Thread),
    thread_property(Thread, status(Reason)).
```

This makes links directional: if the parent terminates, its linked children are terminated, whereas termination of a child does not by itself terminate the parent. Monitoring is observational and produces `down(Pid,Ref,Reason)`.

Sending and termination

Local sending resolves the pid to a thread and delegates copying and queueing to the host:

```
Pid ! Message :-
    send(Pid, Message).

send(Pid, Message) :-
    (   pid_thread(Pid, Thread)
    ->  catch(
            thread_send_message(Thread, Message),
            error(existence_error(_, _), _),
            true
        )
    ;   true
    ).
```

Sending to a pid that no longer exists is a no-op. Sending to an unknown registered name is different: the public interface reports an existence error because the name is a deliberate promise made by the node.

Self-termination records the reason before unwinding the actor. Termination of another local actor is implemented as a host-thread interrupt:

```
exit(Reason) :-
    self(Self),
    (   exit_reason(Self, _)
    ->  true
    ;   assertz(exit_reason(Self, Reason)),
        abort
    ).

terminating :-
    self(Self),
    exit_reason(Self, _).

exit(Pid, Reason) :-
    (   pid_thread(Pid, Thread)
```

```
    ->  catch(
            thread_signal(Thread, exit(Reason)),
            error(existence_error(_, _), _),
            true
        )
    ;   true
    ).
```

Recording the exit reason is the commit point of termination (Section 2.3.3). The first reason recorded is final: a later `exit/1` or exit signal finds it and does nothing, not even raise a second abort. `terminating/0` succeeds from that moment, which is how the PTCP boundary in Chapter 5 recognises that an abort must not resume the actor. The real implementation must serialise termination with monitor installation and removal so that a monitor is either removed or notified exactly once. It must also translate host termination statuses into the documented Web Prolog reasons.

Local result delivery and termination

The cleanup algorithm in Section 5.5.3 requires local results to be enqueued before termination notifications. The demonstrator provides this ordering through synchronous local queue insertion: `thread_send_message/2` places the result in the caller's queue before the send returns.[2] If termination interrupts a send, the result is either already enqueued or will not be sent by that worker; there is no independent local delivery task that can insert it later. The worker's exit handler delivers monitor notifications after its goal has stopped. Monitor installation, notification delivery, and removal of the pid mapping share a lifecycle mutex. A new monitor therefore either joins the notifications delivered at termination, or finds the retired pid and produces `down(Pid,Ref,no_process)`. In the latter case, any result sent by that former worker was already enqueued before retirement. Here `no_process` confirms absence; it does not recover the worker's termination reason.

The same serialization makes local `demonitor(Ref,[flush])` sufficient to retire an old monitor: its notification is either already enqueued and flushed, or its delivery is prevented. It cannot arrive after the flush. These local guarantees do not order an incoming exit signal against an ordinary message, and do not strengthen the remote channel contract.

Exception terms in monitor notifications

An uncaught SWI-Prolog system exception is sanitised when it crosses the monitor boundary, whether the observer is local or remote. The notification retains its formal error classification but elides the implementation-supplied context. For an undefined

[2] `https://www.swi-prolog.org/pldoc/doc_for?object=thread_send_message/2`.

procedure in a private actor module, it also omits that module qualification. Thus observers receive, for example, `exception(error(existence_error(procedure,q/1),_))`, rather than implementation details such as a generated actor-module name or stack context. Code inside the actor can still catch the original exception before it reaches the boundary. Non-exception exit reasons are not rewritten.

Selective receive

Host message queues already support pattern-directed reception, but Web Prolog receive clauses also have ordering, optional guards, and committed selection. One clear implementation keeps unmatched messages in a thread-local deferred queue:

```
:- thread_local deferred/1.
:- meta_predicate discard_deferred(1).

discard_deferred(Test) :-
    forall((clause(deferred(Message), true, Ref),
            copy_term(Message, Copy), once(call(Test, Copy))),
           erase(Ref)).

receive(Clauses) :-
    receive(Clauses, []).

receive(Clauses, Options) :-
    thread_self(Mailbox),
    (   clause(deferred(Message), true, Ref),
        select_body(Clauses, Message, Body)
    ->  erase(Ref),
        call(Body)
    ;   receive_new(Mailbox, Clauses, Options)
    ).

receive_new(Mailbox, Clauses, Options) :-
    (   selectchk(timeout(T), Options, Rest), T > 0
    ->  get_time(Now), D is Now + T,
        scan(Mailbox, Clauses, [deadline(D)|Rest], Options)
    ;   scan(Mailbox, Clauses, Options, Options)
    ).

scan(Mailbox, Clauses, GetOptions, Options) :-
    (   thread_get_message(Mailbox, Message, GetOptions)
    ->  assertz(deferred(Message), Ref),
        (   select_body(Clauses, Message, Body)
        ->  erase(Ref),
            call(Body)
```

```
        ;   scan(Mailbox, Clauses, GetOptions, Options)
        )
    ;   option(on_timeout(Body), Options, true),
        call(Body)
    ).

select_body({Clauses}, Message, Body) :-
    select_clause(Clauses, Message, Body),
    !.

select_clause((Clause ; _), Message, Body) :-
    select_clause(Clause, Message, Body).
select_clause((_ ; Clauses), Message, Body) :-
    select_clause(Clauses, Message, Body).
select_clause((Head -> Body), Message, Body) :-
    (   nonvar(Head),
        Head = if(Pattern, Guard)
    ->  match_message(Pattern, Message),
        catch(once(Guard), E, guard_exception(E))
    ;   match_message(Head, Message)
    ).

match_message(Pattern, Message) :-
    subsumes_term(Pattern, Message),
    Pattern = Message.

guard_exception(E) :-
    control_exception(E),
    !,
    throw(E).
guard_exception(_) :-
    fail.

control_exception('$abort_goal').
control_exception('$ptcp_time_limit').
control_exception(unwind(_)).
```

The cut commits to the first matching message and clause. The selected body is then an ordinary Prolog goal and may itself have several solutions; backtracking through that body never returns the consumed message to the mailbox.

Matching first checks `subsumes_term(Pattern, Message)`, then unifies the two terms to establish bindings for the guard and body. The subsumption check ensures that pattern matching does not specialise variables in the message. Thus `offer(X)` matches `offer(coffee)`, binding `X` to `coffee`, but `offer(coffee)` does not match a received `offer(X)`. Nonground messages remain supported, and

the sender's variables remain unaffected. For ground messages, this policy agrees with ordinary unification.

The check constrains pattern matching, not subsequent execution of the guard or body. A guard that validates a correlation reference or actor identity must therefore reject a nonground value rather than instantiate it to an expected value. If matching or a guard fails, Prolog backtracking undoes the candidate's bindings before another clause or message is tried. The `nonvar/1` check distinguishes a guarded clause from a variable catch-all pattern without instantiating that pattern to `if/2`.

A guard that fails, or raises an ordinary exception, rejects the candidate. A control exception is different: `guard_exception/1` rethrows it, so abort, termination and enforced limits are never mistaken for a non-match (Section 2.3.3). The terms listed by `control_exception/1` are those of the demonstrator: its abort and running-limit exceptions, and the `unwind/1` exceptions with which recent versions of SWI-Prolog unwind an aborted or terminating thread. Because a control exception can now escape from a guard, `scan/4` records each new message as deferred *before* its guards run, and erases the record only when a clause is selected. A message that was being examined when the exception arrived therefore stays in the mailbox, in its original position.

A positive timeout is converted once to an absolute deadline, which is reused while scanning new messages. Unmatched arrivals therefore do not restart the waiting period. The original options remain available for the `on_timeout` goal. A zero timeout is passed through unchanged: `thread_get_message/3` with `timeout(0)` inspects the queue without waiting, whereas an expired `deadline/1` fails immediately even when messages are queued. The deadline bounds waiting; it does not interrupt a guard already being evaluated.

This fragment omits module propagation, detailed option checking, and the atomic interaction between deferred and newly arriving messages. Those are important in an implementation, but they do not alter the semantic idea: scan in arrival order, preserve messages that do not currently match, and commit when a receive clause is selected.

Why the real actor layer is larger

The preceding code is intentionally smaller than the demonstrator. The real native runtime additionally provides:

- collision-safe pids with the standard constructible `Id@Node` representation, and fresh monitor references;
- atomic monitor installation, dead-pid notification, and `demonitor/1-2` with optional flushing;
- registered actor names and node-resident service names;
- delayed sends and cancellation;
- spawn admission and per-actor stack limits;
- private execution modules and controlled source installation;

- hooks for remote spawn, send, exit, monitoring, and naming;
- Erlang's save-queue ordering, so that after a successful receive the deferred messages are rescanned from oldest before new arrivals: the thread-local store yields this ordering directly, but is checked in full on every receive rather than resumed mid-scan as on the BEAM — semantically equivalent, quadratic rather than amortised;
- checks that terms crossing a web worker or network boundary have a portable representation; and
- cleanup for failed actor initialisation, node shutdown, and abandoned sessions.

These are not embellishments. They are what turns a mapping from actors to threads into a node that preserves the same public semantics under failure, concurrency, and distribution. The pedagogical core remains useful because it shows where each of those additions attaches. In particular, the sketch implements the spawn-time `monitor(true)` option but deliberately does not present a naïve `monitor/2` implementation: correct monitoring of an independently supplied pid requires the atomic dead-pid and termination handling described above.

The node layer also attaches authority checks to the actor core. A language send remains open subject to node policy, and monitoring follows that target policy. Exit checks the caller's ownership tree independently of pid entropy; its resolver accepts only runtime actor mappings, never raw host thread numbers or aliases. Monitor removal matches the installer as well as the reference. Only an owner may detach a language-level child.

The counter allocator in the sketch above illustrates identity allocation, not the production security policy. The demonstrator uses CSPRNG rejection sampling over the ten-digit integer space and retains issued identities for the node's lifetime, including identities of terminated actors. Monitor and request references use the same compact space and non-reuse discipline. These values serialize as JSON numbers without loss. The book uses eight-digit examples for readability. Production identities and correlation references should be larger, for example with 128 random bits; the demonstrator's roughly 33-bit space is a presentation compromise, not strong guessing resistance. Wider JSON identities remain supported as decimal strings to avoid JavaScript's 53-bit integer limit. These values supply identity or correlation, not authorisation.

E.3.4 Actor I/O

Actor I/O is expressed as a message protocol rather than as access to the host process's terminal. The basic predicates can therefore be implemented on top of sending and receiving:

```
output(Term) :-
    output(Term, []).
```

```
output(Term, Options) :-
    self(Self),
    (   option(target(Target), Options)
    ->  true
    ;   parent(Target)
    ),
    Target ! output(Self, Term).

input(Prompt, Input) :-
    input(Prompt, Input, []).

input(Prompt, Input, Options) :-
    self(Self),
    (   option(target(Target), Options)
    ->  true
    ;   parent(Target)
    ),
    Target ! prompt(Self, Prompt),
    receive({
        '$input'(Target, Reply) -> Input = Reply
    }).

respond(Pid, Input) :-
    self(Self),
    Pid ! '$input'(Self, Input).
```

The sender identity in the private response term prevents an unrelated process from satisfying the prompt accidentally. Transport adapters convert the outward-facing `output/2` and `prompt/2` events into JSON, terminal output, or another presentation protocol as appropriate.

Acknowledged terminal writes.

The sketch above describes actor messages. Sending `output(Self, Term)` to an ordinary actor does not wait for that actor to process it. Terminal writes made through predicates such as `format/2` need an additional runtime contract: a remote write returns only after the terminal service has accepted it. This is the request/reply principle described in Section 4.1.4; it does not turn every actor message into a synchronous call.

For each remote write, the runtime allocates a fresh request reference and registers a private reply queue before sending the request. The receiver validates the inherited terminal endpoint, delivers the output to its terminal queue, and sends a reply carrying the same reference. The sender uses that reference to wake the waiting writer. A reply from another connection must not satisfy the request.

The wait must release any shared connection or send lock. Other actors must remain able to use the connection, and the connection's reader must remain able to dispatch the reply. Per-request queues keep acknowledgements out of the actor's ordinary mailbox, where selective receive could otherwise consume or defer them.

A browser-hosted SWI-WASM terminal requires a further acknowledgement at the common destination. The remote node forwards the output to the browser and waits for the browser terminal to accept it before replying to the writer. The bridge negotiates support for this exchange when the connection is established. An older peer that cannot provide it must not silently weaken the ordering guarantee. A remote descendant that inherits the terminal endpoint follows the same acknowledged route.

Every completion path must remove the pending request and release its reply queue. A rejected or revoked endpoint, a lost connection, or an expired acknowledgement wait raises an I/O error rather than reporting a successful write. Connection cleanup wakes pending writers so that they do not remain blocked indefinitely. A missing acknowledgement does not prove that no output was accepted, so blindly retrying the write could duplicate it. These measures preserve the ordering contract while keeping failure handling separate from ordinary actor-message delivery.

E.3.5 Private databases and source installation

An actor needs a private clause database, not merely a private stack and mailbox. A native implementation can create a temporary module for each actor, load the requested `src_*` material into it, and arrange for the actor goal to resolve against that module. Node-published predicates may be imported deliberately, while another client's session module must remain invisible.

This boundary is easy to weaken accidentally. If a spawned goal retains an unrestricted module reference to its creator, the child can observe later changes to the creator's dynamic database. That is sharing, not source installation. The implementation must instead copy explicitly requested predicates or source text into the child context and expose shared node state only through the node's public surface.

Browser actors obtain a stronger version of this separation as a consequence of their execution substrate: each web worker contains a distinct Prolog virtual machine. Native temporary modules must emulate the language-level part of that isolation inside one host process.

Client source is loaded under tighter control than ordinary `consult`. The node reads it as raw terms, so a client cannot hide behavior behind its own term or goal expansion: a clause or declaration that would define `term_expansion/2` or `goal_expansion/2` is refused, and the node performs DCG translation itself. Without this, a client-defined `goal_expansion/2` would fire while the loader reads the rest of the client's own source – before the per-goal sandbox check – and could rewrite a harmless-looking body into a forbidden call. The same check guards `assert/1`, so the hook cannot be installed at runtime either. Each source term, after the node's own translation, is validated against the active profile and sandbox before it is added.

Fresh private state does not require recompiling immutable runtime support. The current native implementation compiles the standard actor-I/O predicates once and imports them into each trusted actor module. Requested `src_*` material, dynamic predicates, and all other actor-specific state remain in the fresh temporary module. Public sandboxed actors retain actor-local I/O wrappers, since the sandbox uses their locality to distinguish actor I/O from unrestricted stream I/O. The effect of this prepared runtime module is measured in Appendix F.

E.3.6 Distribution and toplevel actors

Network transparency is most easily implemented by keeping local actor operations behind a dispatch layer. A local pid is resolved to a thread; a remote pid is resolved to a connection and encoded as a protocol message. The public `spawn/3`, `send/2`, `exit/2`, and monitor predicates need not know which branch was taken.

Remote communication adds responsibilities not visible in the pedagogical actor core: connection ownership, handshake and version negotiation, reconnection policy, pending-command correlation, remote monitor delivery, term conversion, and cleanup when a WebSocket closes. The owner of the remote node must also revalidate source, goals, and permissions rather than trust a decision made by the sending node.

Remote links are maintained by the controller at the child-hosting node. When it learns that a parent has terminated, it propagates a reason-bearing termination signal to the linked children. The demonstrator also associates remotely hosted actors with the trusted WebSocket and lineage that authorised their creation. When that connection reports close, error, or ping/pong lease expiry, the controller terminates its owned subtrees. A detached owner's service is rehomed under the node with log output and is excluded from the creator's connection sweep.

Several parent–child links may share one connection, and a parent may terminate while that connection remains healthy. The controller must therefore propagate each parent's termination independently of connection cleanup. Cleanup after detected transport loss provides a backstop; it does not replace per-parent link handling. The observable limits of remote links and monitors are described in Section 4.1.7.

The sender records a delegated route before waiting for a remote spawn acknowledgement. When the last locally hosted actor in that lineage ends, it sends `lineage_close` on the shared connection. This covers the race in which a parent dies before learning its remote child's pid without reclaiming another client's lineage. Lost contact remains an unknown outcome: cleanup requests and lease expiry do not certify remote completion.

Toplevel actors reuse the same substrate. Their PTCP implementation was developed in Section 5.3. In the current runtime, `toplevel_halt/1` terminates the actor through `exit/2`; `toplevel_halt/2` installs a private monitor and waits until termination has been observed. This works even when the toplevel is computing a nonterminating goal, provided that the substrate supports hard termination.

E.4 Sandboxing, isolation, and authority control

A production-quality node must enforce a boundary strong enough for running untrusted client code. Four concerns must be kept distinct:

Expressibility.
: Which language constructs and built-in predicates may client code invoke?

Authority.
: Which actors, services, node-owned predicates, and external resources may the computation affect?

Isolation.
: What prevents the Prolog runtime or a fault in its sandbox from escaping into the host or interfering with another computation?

Resource consumption.
: How much CPU time, memory, actor space, mailbox space, output, and network activity may the computation consume?

No single mechanism answers all four questions.

E.4.1 Whitelist and blacklist strategies

A language-level sandbox may be organised as a whitelist or a blacklist. Whitelisting permits only predicates explicitly declared safe. Blacklisting exposes a defined language fragment but excludes specified predicates, meta-call paths, and operation families.

For a full reflective Prolog system, whitelisting is the safer starting point. Authority may otherwise be reconstructed indirectly through meta-calling, predicate inspection, dynamic database operations, foreign predicates, parser changes, or unrestricted module qualification. A newly added host predicate is denied until it has been examined.

Blacklisting becomes more plausible when the exposed language is already narrow, the dangerous host facilities are absent or unreachable, and all meta-level paths are checked recursively. It remains one layer of a security architecture, not a sufficient foundation for trust, and it cannot be made exact: a predicate that calls a goal passed to it without declaring that argument as a goal slips past a name-based denylist. The demonstrator's blacklist guards explicit meta-calls, collection predicates, and declared callable arguments such as those of `maplist/N` and `foldl/N`. The higher-order guard checks the fully applied callback when invoked, preserving the calling context; DCG arguments receive their own translation, and unsupported meta-argument forms are rejected. This closes those declared call paths, but does not establish completeness against undeclared meta-calling or every reflective construction. Whitelisting should therefore be the default for a publicly reachable node, and the inexpressibility claim of Section 4.5.1 is made for whitelist mode. The

demonstrator supports both strategies because they make different trade-offs between compatibility and conservative denial; its public tutorial nodes run under a blacklist so that the chapter examples load unchanged. The inexpressibility claim does not apply to those current tutorial nodes.

E.4.2 Language sandboxing and host isolation

Language sandboxing decides what client code can express. Host isolation decides what the runtime process itself can do. A perfect predicate whitelist does not protect against a defect in the Prolog system or a foreign library. Conversely, a container with no useful filesystem or network authority does not prevent one client from reading another client's Prolog clauses if the language-level isolation is wrong.

A robust native deployment therefore places the Prolog process inside an operating-system, container, or virtual-machine boundary with minimal ambient authority. The Prolog sandbox, node authorisation policy, and host boundary then fail independently rather than standing or falling together.

E.4.3 WASM and WASI

WebAssembly provides another useful host boundary. In the browser, a WASM runtime inherits the browser's origin and web worker security model. In a standalone embedding, WASI-style capabilities can make filesystem handles, sockets, and host callbacks absent unless explicitly granted.

This substantially reduces ambient authority, but it does not solve Prolog-level reflection or resource exhaustion. Nor should a browser-resident demonstrator engine be confused with the server sandbox: code executed in the visitor's browser runs with the authority available to that page or web worker and is not governed by the remote node's Prolog sandbox.

E.4.4 Policy permissions inside the node

Even a host-isolated runtime needs an internal authority policy. Ordinary clients should receive session-scoped permissions: bounded calls, actor creation within their containment scope, messaging to permitted targets, and access to explicitly exported services.

Node owners may possess additional permissions for publishing services, registering public names, installing persistent code, modifying shared node state, inspecting operations, or detaching services from a client session. These operations must not

become available merely because a client can name the predicate that implements them.

The implementation should therefore establish a principal at the transport boundary, carry that principal into nested execution and actor creation, and check authority at every operation that crosses from client-owned state into node-owned state.

E.4.5 Resource governance

Timeouts, actor limits, mailbox bounds, RPC limits, source-size limits, cache bounds, and rate limits answer a different question from sandboxing: *how much may this program do?* A program can remain within its language authority and still deny service by spawning actors, constructing enormous terms, producing unbounded output, or repeatedly invoking expensive public relations.

Limits must be associated with an owner or principal rather than applied only to individual calls. Otherwise a client can remain below every per-call limit while exceeding the node's capacity through multiplicity. The runtime also needs a policy for cleanup: refusing new work is insufficient if abandoned actors, sessions, cached continuations, and queues remain resident.

The demonstrator records ownership and reserves actor capacity before a child can start. Its `node_lifetime` hooks charge all locally hosted children to their principal, including session descendants; trusted peers carry separate lineage and account tokens so several trees can share one account without sharing visibility or termination authority. Authenticated principals share that account across connections. Anonymous HTTP calls, sessions and WebSocket connections have separate lifetime accounts, so one anonymous visitor cannot exhaust another's actor or CPU budget. The default `max_principal_actors` is 32. Quota exhaustion raises `resource_error(actors)` rather than silently failing a spawn.

CPU accounting samples the principal's actor threads every 50 milliseconds. The defaults `principal_cpu_seconds(30)` and `principal_cpu_window(60)` bound aggregate sampled CPU per window; exhaustion terminates its actors and rejects further spawns with `resource_error(cpu)` until the window renews. Sampling and safe points permit overshoot. Budgets apply independently on each hosting node. Watcher threads finish when no local actors remain, ended lineage records are reclaimed, and unused account metadata expires after its accounting window.

Public node peers authenticate delegated lineage with bearer tokens granting `internal_transport`; the sending node associates each token with an exact destination URL through `peer_transport_tokens`. The existing header handshake is accepted only from trusted local or private peers. Public deployment therefore requires credentials in each direction that initiates a connection.

Mailbox admission counts queued and selectively deferred messages, releasing the charge only when a message is consumed or discarded. The default `max_mailbox_messages` is 1,024. Input and message terms are checked for acyclicity and nesting depth, with `max_term_depth(128)` by default. A flat list counts as

one level, independently of its length. Runtime monitor notifications bypass application admission limits and cannot prevent an actor's retirement; they still count until consumed or flushed. The associated errors are `resource_error(mailbox)` and `resource_error(term_depth)`. These checks complement inference, source-size, session, and node limits; there is no separate atom-creation budget.

E.4.6 A layered deployment strategy

A realistic public implementation combines all four layers:

1. expose only the intended Web Prolog language and public relation surface;
2. associate every request and spawned computation with a principal and authorisation policy;
3. bound calls, sessions, actors, queues, source, results, and network activity;
4. deploy the runtime with minimal host authority and secure transport; and
5. make denials, saturation, lifecycle events, and policy changes observable to the node owner.

Security is therefore an architectural property of the request path, not a predicate that can be called immediately before `call(Goal)`.

E.5 A browser-resident node

Prolog can be brought into a browser by interpreting or compiling it in JavaScript, by compiling an existing system to WebAssembly, or by installing a browser extension. The demonstrator uses SWI-Prolog's WebAssembly port. Its implementation problem is not merely how to run a Prolog query, but how to reproduce the actor and node abstractions on a single-threaded browser substrate.

E.5.1 Implementing a browser node

Cooperative engines are not isolated actors

The SWI-Prolog WASM interface lets JavaScript call into a Prolog virtual machine. Open queries and engines can retain search state and yield control to JavaScript. A heartbeat based on an inference budget allows asynchronous query drivers to return periodically to the browser event loop.

This is useful cooperative execution, but it is not actor isolation. Engines inside one WASM instance share a virtual machine and global Prolog state, and only one of them executes WASM instructions at a time. A synchronous call, a long-running

foreign operation, or JavaScript that does not yield can still seize that VM. Another engine in the same instance cannot forcibly terminate it.

The distinction determines the architecture. If actors shared one single-threaded instance, a non-cooperative actor could prevent every other actor in the node from receiving messages. It could also compromise the private-state model unless a second isolation mechanism were built inside the VM.

One web worker and one VM per actor

The demonstrator therefore runs each local browser actor in its own web worker, with a complete and private SWI-Prolog WASM instance. A JavaScript node controller remains on the main thread. It owns pid allocation, actor and name registries, monitors, delayed sends, web worker lifecycles, remote connections, stateless HTTP calls, the terminal, and the page.

An ordinary actor executes the goal passed to `spawn/1-3`. A toplevel actor runs the mailbox-driven PTCP loop. The shell's toplevel is also a PTCP actor in a web worker; it is special only because the controller creates it directly, attaches the terminal to it, and can replace its web worker during a hard shell reset.

Each web worker has its own linear memory, stacks, dynamic database, modules, atoms, and virtual filesystem. A non-yielding actor blocks only its web worker, and the controller can terminate that web worker. This is the browser equivalent of the containment that native actors obtain from the runtime's ability to interrupt a thread.

The cost is actor density. Every actor instantiates a VM and grows private memory, so web worker start-up and per-web-worker memory are much larger than the cost of an Erlang process or a native Prolog thread. Pools could reduce start-up latency, but multiplexing actors onto a shared web worker would give up some of the isolation that motivates the design.

Terms on the wire

Web workers cannot exchange Prolog heap objects. The sending web worker serialises a message as text, the controller routes the text, and the receiving web worker parses it into a fresh term. Variable bindings are therefore lost exactly as they are in a remote send. The text is the canonical term text of Appendix D, the same that the native node writes, so a message reads the same whether it came from a web worker or from another node.

Using the same representation locally and remotely has a useful consequence: local browser communication is not a privileged semantic fast path. Cyclic terms, attributed variables, streams, and non-text blobs are rejected at the boundary, while ordinary acyclic Prolog terms are copied. An implementation using native threads may copy terms more directly, but it must preserve the same observable contract if location transparency is to hold.

Abort and exit in a web worker

A native toplevel is aborted by a signal that raises an exception in its thread. The engine in a web worker offers no such signal: it can only abort the whole query it is running. The web worker therefore runs the PTCP loop as that query and starts it again after an abort. The toplevel returns to **s1** with its loaded program and its deferred messages intact. Before the restart the web worker sends the abort acknowledgement to the client of the aborted query, after the answers that query had already sent; a toplevel spawned with `session(false)` ends instead. An abort of an idle toplevel is also acknowledged; a persistent toplevel stays idle and a one-shot toplevel ends.

A toplevel awaiting a message, an RPC response, terminal input, or another asynchronous operation cannot consume an engine abort until that wait ends. The web worker therefore cancels the wait through a privileged control flag, which the bridge checks before returning a value to Prolog. No application message serves as an interrupt. The running-time limit uses the same route; if an abort coincides with it, the client receives both the time error and the abort acknowledgement.

`exit/1` records its reason with the web worker before it aborts the goal. The first reason recorded is the one reported, and a goal that is exiting is never restarted.

The DOM stays on the main thread

A web worker has no direct access to `window` or `document`. This is not merely an inconvenience. The DOM is a stateful, location-bound, single-threaded resource, precisely the kind of resource that actor systems normally place behind a message interface.

The main-thread controller already performs a limited version of this role: it renders terminal output and delivers user input to the shell actor. A general application can use the same pattern. A web-worker-resident application actor sends presentation messages to a main-thread view component, and the view returns browser events as actor messages. The application logic then has the same shape whether the actor is in the current tab or on a remote node.

This arrangement also fits component frameworks such as React or Vue. Framework-owned DOM is updated by changing component state, not by allowing an external Prolog engine to mutate reconciled nodes. An actor subscription can feed that state in the same way as a WebSocket or another asynchronous data source.

The optional main-thread mode

There is one useful exception. A self-contained Prolog page may want direct DOM access and may be willing to run a Prolog engine on the main thread. The demonstrator supports such a mode. It can use the browser integration facilities of SWI-WASM directly, including yielding while waiting for asynchronous input.

The trade-off is local and explicit. The main-thread engine has DOM access but lacks web worker isolation and hard termination. Actors it spawns still run in web workers, and its calls and messages use the same public contracts as the default model. Portable code can inspect such genuine host differences through `runtime_property/1` rather than infer them from a pid or node URI.

Summary of the browser architecture

The browser node can therefore be summarised as follows:

> A JavaScript controller on the main thread owns routing, networking, lifecycle, terminal, and page; each local actor owns an independent SWI-Prolog WASM instance in a web worker; messages cross web worker and node boundaries as portable Prolog terms; and an optional main-thread engine is available for the specialised case of a self-rendering page.

This is not necessarily the optimal browser architecture for all future Prolog systems. A threaded WASM port or a dedicated actor VM could support much lighter processes. The worker-per-actor design is instead a faithful realisation of actor isolation on the substrate available to the demonstrator. Its main achievement is that a local browser actor and a remote actor differ in placement and cost, but not in the programming model through which they are addressed.

E.6 Alternative implementation approaches

E.6.1 Other Prolog systems and host languages

A node need not be based on SWI-Prolog. Another Prolog system may offer threads, engines, modules, HTTP, or WebSocket facilities with different names and guarantees. The implementation task is to reproduce the observable Web Prolog contracts, not the internal structure of the demonstrator.

Nor must every node contain a general-purpose Prolog system. A RELATION node can be written in Python, Rust, Erlang, Java, or another host language if it accepts the specified calls, performs the advertised relation, and returns conforming answers. Broader profiles require increasingly complete implementations of unification, control, source installation, private databases, and actor semantics.

An Erlang-based implementation is particularly interesting. Erlang already provides lightweight isolated processes, mailboxes, distribution, monitors, and supervision, but it does not provide Prolog unification, backtracking, or the Web Prolog term and error contracts. A conforming ACTOR node on the BEAM would therefore be more than an adapter around an Erlang service: it would combine a Prolog execution engine with the specified Web Prolog process semantics. Erlog provides an existing Erlang-hosted Prolog interpreter (Virding, n.d.); Section 10.3.5 distinguishes that precedent from the additional requirements of an ACTOR implementation.

E.6.2 Complementary GNU Prolog and Trealla prototypes

The following account summarises the projects' reported status at the time of writing. The repositories provide build instructions, test runners, and more detailed status and compatibility notes. Passing the reported tests is evidence for the interactions exercised, not a certification of the complete profiles. SWI-Prolog remains the principal implementation described in this appendix.

A separate implementation-neutral conformance project is underway.[3] Its Python standard-library runner tests already running nodes through their public interface without importing their implementation code. The current `actor-core-0.1` and opt-in `portable-text-0.1` drafts have cases exercised against SWI-Prolog and Trealla; they are versioned test contracts, not additional levels in the cumulative node-profile hierarchy. Passing them establishes agreement only on the listed cases. Implementation-specific unit and regression tests remain in the implementation repositories unless recast as observable contracts. The intended report vocabulary distinguishes `pass`, `fail`, `unsupported`, and `not_applicable`, so that profile support and configuration are not confused with contract violations. Section 11.5.2 explains why this shared evidence is needed.

An ISOBASE node in GNU Prolog

The GNU Prolog project[4] targets ISOBASE; ISOTOPE is a possible later extension. The earlier actor roadmap has been dropped. GNU Prolog's lack of general heap garbage collection favours explicit query completion and recovery boundaries over arbitrary long-lived actor goals. Its public C query interface also supports nested queries rather than independently selectable engine contexts. The prototype consequently uses one process worker per active query lifetime. A worker can pause with live bindings and choice points and resume for subsequent pages; completion, cancellation, or expiry releases its resources.

A loopback C HTTP server exposes `/call`, with submitted source, JSON bindings, Prolog-text answers, live pagination, and query admission limits. C supervisors enforce execution and idle deadlines, stack and response bounds, and sampled memory budgets. Owner-supplied shared predicates are snapshotted and validated at startup, while request-local source remains separate. An optional native bundle compiles validated shared predicates; request-local source remains interpreted. HTTP(S) RPC, promises, and yield support source transfer and pagination, with bounded libcurl threads handling network I/O outside the Prolog runtime. Outbound requests require an explicit owner-supplied origin/IP policy.

The reported tests cover GNU-to-GNU, GNU-to-SWI, and SWI-to-GNU RPC, timeouts, cleanup, and policy enforcement. A 1,000-query recovery test reports a return to the same heap baseline. The documented build uses a pinned GNU

[3] `https://github.com/torbjornlager/trinity-conformance`

[4] `https://github.com/torbjornlager/gprolog-port`

Prolog 1.6.0 source checkout on Apple Silicon macOS, with a C toolchain, libcurl, and Python for tests. The ISOBASE suites are run with `make -C isobase test` and `make -C isobase rpc-test` after following the repository's runtime-build instructions.

The prototype is still incomplete: remaining profile facilities, hard security isolation, and documented compatibility gaps require further work. The memory budget is sampled and has documented overshoot limitations. The README also records a zero-page-limit mismatch between older tests and the current SWI reference; this is a contract/test discrepancy to resolve, not evidence of agreement. Validation on the reported platform does not establish portability to every operating system.

An ACTOR node in Trealla Prolog

The Trealla project[5] ports `library(actors)` and related facilities from the SWI-Prolog simple node, with the current reported tests using Trealla v3.12.6. The port includes selective receive with guards and timeouts, links and monitors, local names and published services, actor I/O, private source contexts, and the resumable toplevel control family. The client examples in `parallel.pl` run unchanged over the actor API.

Trealla threads provide the actor substrate. Thread-scoped blackboard storage replaces SWI-Prolog's `thread_local/1` state. A blocked receive preserves the thread's live Prolog stack and choice points, permitting a producer to resume between HTTP requests. PTCP paging uses `call_cleanup/2` around the original goal to detect deterministic exhaustion without evaluating the next solution's side effects. Private source modules are emptied on actor termination; Trealla currently lacks a public operation to destroy them, so their empty bootstrap modules remain until process exit.

The port supplies HTTP RPC, promises, and yield, together with a native Prolog WebSocket transport and the demonstrator's version-1 actor protocol. Persistent node connections support remote spawning, messaging, monitoring, and toplevel control, including source-bearing calls. The reported test matrix exercises Trealla-to-Trealla and bidirectional Trealla/SWI interactions. After the repository-local runtime build, `./tools/test.sh all` runs the isolated suites and interoperability matrix; `SWIPL` can select the SWI executable used for comparison.

Profile enforcement, source checks, authentication, resource governance, and an invitation-only Docker/Caddy deployment bundle are implemented, but they do not establish general public-deployment readiness. The convenience runtime build disables TLS by default, and the automated transport matrix uses plain loopback HTTP and WebSocket connections. An OpenSSL-enabled build provides TLS transport, but the reported Trealla socket implementation lacks complete hostname-verified certificate validation. The repository therefore recommends a verifying proxy boundary.

[5] `https://github.com/torbjornlager/trealla-port`

Trealla also lacks SWI-equivalent inference and per-thread stack ceilings; process or container containment remains necessary for those resources.

Together, these prototypes test different ways of preserving public contracts: process-owned query continuations in GNU Prolog, and thread-based actors in Trealla. They complement the integrated SWI-Prolog demonstrator without replacing its role. Chapter 11 discusses the resulting portability evidence and possible further targets.

E.6.3 Prospects for a dedicated Web Prolog VM

The BEAM demonstrates what can be achieved by a runtime designed around lightweight, isolated, pre-emptible processes. It is natural to ask whether a Web Prolog runtime could combine those properties with unification, logical variables, nondeterministic search, and dynamic symbolic code.

An earlier paper described such a runtime as a "holy grail" for Web Prolog (Lager, 2019). Richard O'Keefe, drawing on experience with both Prolog and Erlang, responded that the idea was feasible and pointed to the small processes of Logix and to the lineage connecting the WAM, JAM, BAM, and BEAM. The history does not supply an implementation, but it does show that the design space is less alien than the phrase "dedicated VM" may suggest.

Such a VM would need more than fast process switching. It would have to specify how backtracking interacts with committed message effects, how heaps and code are isolated or shared, how a process is interrupted safely, how terms are copied, and how local execution preserves the same observable semantics as remote execution. It would also need the network, policy, and administrative layers described earlier in this appendix.

A mature existing Prolog system consequently retains major advantages: optimising compilation, libraries, debugging, tabling, constraints, foreign interfaces, and decades of operational experience. A dedicated runtime would be justified only if its gains in actor density, scheduling, isolation, or portability outweighed the cost of rebuilding that environment.

E.6.4 Narrower profiles require less work

The profile structure allows implementation to proceed incrementally. A RELATION node can expose a single owner-defined predicate without implementing ISO control constructs. An ISOBASE node can add bounded stateless querying without persistent sessions. An ISOTOPE node can then add source installation and toplevel continuation before any general actor distribution is attempted.

This is not merely a convenience for implementers. Independent implementations are easier to compare first at a narrow boundary. Agreement on term syntax, answer

forms, errors, paging, and relation visibility provides a base on which the more demanding lifecycle and concurrency contracts can later be tested.

E.7 Summary

There is no single privileged way to implement a Prolog node. A native node may map actors onto threads and private modules; a browser node may map them onto web workers and independent WASM instances; a narrow relation node may not contain a Prolog runtime at all. What makes them nodes on the same Prolog Web is their agreement at the observable boundaries.

The implementation task is therefore layered. Query evaluation must preserve solutions, failure, errors, and paging. Actor execution must preserve mailboxes, selective receive, lifecycle relations, and location-transparent addressing. Source installation must preserve private execution contexts. Policy, sandboxing, host isolation, and resource governance must contain work performed for untrusted principals. Finally, transports must carry the same terms and events without quietly changing their semantics.

The demonstrator shows that these layers can be assembled on both native SWI-Prolog and SWI-WASM. The GNU Prolog and Trealla prototypes provide initial evidence at selected query and actor boundaries on different substrates. The next task is to make that evidence systematic: state the contracts precisely, record compatibility gaps, and test observable agreement across versions and implementations without requiring a shared internal design.

Appendix F
Benchmarking three actor substrates

Broad claims such as "Web Prolog is slower than Erlang" conceal what is actually being measured. The actor interface described in this book has three implementations of immediate interest: browser actors implemented as SWI-WASM web workers, native actors implemented as SWI-Prolog threads, and Erlang processes running on the BEAM virtual machine. They expose similar message-passing patterns, but they realise an actor at very different levels of granularity.

This appendix compares those three substrates. It is not a contest between Prolog and Erlang as languages, nor does it attempt to reduce the usefulness of a programming model to a single number. Its purpose is narrower: to measure some costs that a programmer can observe, relate them to the implementations described in Appendix E, and identify where optimisation work would have the greatest effect.

F.1 What is being compared

Table F.1 summarises the three implementations. The unit called an actor is intentionally the same at the interface and intentionally different underneath.

Table F.1 The three actor substrates.

Substrate	Actor representation	Communication path
Browser node	An independent SWI-WASM virtual machine in a web worker	Messages relayed between web workers by the browser-side actor bridge
Native node	An SWI-Prolog thread with a private actor context	In-process delivery through the native actor runtime
Erlang	A lightweight BEAM process	In-VM delivery through an Erlang process mailbox

These differences make the result partly predictable. A BEAM process is designed to be created cheaply and in great numbers. A native Web Prolog actor is heavier,

since it is built from a native Prolog thread and its execution context. A browser actor is heavier again: creating one means starting another web worker, instantiating SWI-WASM, installing its source, and connecting it to the page-level message router. The measurements tell us the size of these differences in the current implementations; they do not establish an intrinsic cost for Web Prolog.

F.2 Method

The browser measurements reported here were made on 29 July 2026 on a 2023 Apple iMac with an eight-core M3 processor and 16 GB of memory, running macOS 26.5.1. The native and Erlang measurements were repeated on the same machine on 29 August 2026. The native implementation used SWI-Prolog 10.1.3. The browser implementation used SWI-WASM 10.1.8 in the web-worker-based browser node. The Erlang implementation used Erlang/OTP 26 with ERTS 14.2.5.

All numerical results in this appendix belong to that historical series on the M3 iMac, with the runtime versions and configurations described here. They are not measurements of the latest demonstrator revision. Results from another computer would form a separate series: changes in hardware, software, and configuration must not be attributed to implementation changes alone. A refreshed comparison of the three substrates should measure them on the same machine under documented conditions.

The repeated native measurements were made after removing an unintended `:- debug.` directive from the node's loading path. That directive put SWI-Prolog into debug mode and disabled last-call optimisation. The results below use normal execution mode without `-O`. An exploratory check with `-O` did not suggest a material change in the medians; without a reported measure of spread, this is not evidence of statistical equivalence between the two modes.

Three microbenchmarks were used:

Actor lifecycle.
: Start one actor, send it an instruction to stop, wait for its monitor notification, and repeat. The timed interval therefore includes actor creation, initialisation, source installation where required, one message, termination, and monitor delivery. It is an end-to-end actor-lifecycle measurement, not a measurement of the underlying thread or process creation primitive alone.

Chained spawning.
: Have each actor create one unlinked child and send it a message, producing a chain of actor creations. This measures the layer-0 native actor core separately from the full isolated node.

Steady-state ping-pong.
: Create two actors before timing begins, warm them up with 100 round trips, and then time 1,000 round trips over the same pair. One round trip consists of a `ping` message and its `pong` reply. Actor creation and termination are outside the timed interval.

Elapsed wall time was measured in microseconds. A short warm-up preceded the recorded native and Erlang series. The tables report medians: three observations for the browser lifecycle test, seven for the native lifecycle test, and twenty-one for Erlang; the native ping-pong median is from eleven observations and the Erlang median from twenty-one. Browser ping-pong observations required the additional precaution described in Section F.4.1.

The chained-spawning medians use seven native observations at 1,000 actors, five at 10,000 and 100,000, twenty-one Erlang observations at 1,000 and 10,000, and eleven at 100,000. The final isolated-node check uses nine observations at 1,000 actors.

The tables report point estimates without a measure of dispersion. The medians and sample counts alone do not establish run-to-run stability, especially for the small browser samples. A future series should preserve the individual observations and report their spread alongside the medians.

The historical native figures have not been revalidated with the current lifetime and resource-accounting hooks. The recorded configuration details here do not establish the hook settings or allocator revision for every table, so these figures cannot quantify the cost of current public-node governance. Such a comparison requires new measurements with the implementation revision and policy settings recorded explicitly.

The Web Prolog and Erlang programs use the same lifecycle and message protocols. In particular, links are disabled in Web Prolog where Erlang's ordinary unlinked spawning is used, and both versions wait for monitor notifications before considering an actor terminated. The complete sources are also provided as `benchmarks/actor_benchmarks.pl` and `benchmarks/actor_benchmark.erl` in the demonstrator repository.

F.3 Actor lifecycle

Web Prolog actors are linked by default, whereas Erlang's `spawn_monitor/3` monitors the new process without linking it. The benchmark therefore disables Web Prolog's default link explicitly. It measures the following complete actor lifecycle:

```
benchmark_lifecycle(Actors, Microseconds) :-
    get_time(T0),
    forall(between(1, Actors, _),
           benchmark_one_lifecycle),
    get_time(T1),
    Microseconds is round((T1-T0)*1_000_000).

benchmark_one_lifecycle :-
    spawn(benchmark_idle, Pid, [
        link(false),
        monitor(true),
```

```
        src_predicates([benchmark_idle/0])
    ]),
    Pid ! stop,
    receive({down(Pid, _, true) -> true}).

benchmark_idle :-
    receive({stop -> true}).
```

Only one child is live at a time. The test consequently measures the cost of repeatedly creating and reclaiming actors, not the maximum number of simultaneously live actors. Capacity and memory consumption require a separate experiment.

The corresponding Erlang operation is:

```
one_lifecycle() ->
    {Pid, Ref} = spawn_monitor(?MODULE, idle, []),
    Pid ! stop,
    receive
        {'DOWN', Ref, process, Pid, normal} -> ok
    end.

idle() ->
    receive stop -> ok end.
```

Table F.2 gives the median elapsed time for a sequence of complete lifecycles.

Table F.2 Historical median time for complete actor lifecycles on the M3 iMac, in milliseconds.

Actors	Browser node	Native node	Erlang
10	1 271	3.297	0.012
30	3 971	9.363	0.037
100	13 107	31.230	0.121

The near-linear progression is more informative than the largest number alone. In this implementation, one web worker actor lifecycle costs about 130 ms, one native actor lifecycle about 0.312 ms, and one BEAM process lifecycle about 1.2 microseconds. At 100 actors, the browser node is about 420 times slower than the native node. For this complete lifecycle operation, the full native node is about 258 times slower than Erlang.

These ratios describe the chosen end-to-end operation. The browser and native measurements include preparation of private Prolog execution contexts and source installation through `src_predicates/1` on each creation; the Erlang module is already loaded. The table does not separate source-installation cost from the rest of the lifecycle. It should therefore be read as the price of the actor abstractions supplied by the three systems, not as a comparison of their lowest-level schedulers.

The native implementation does not pool actors or reuse their mutable modules. Every actor receives a fresh private context, but imports one immutable implementation of the standard I/O wrappers. Public sandboxed actors retain actor-local wrappers, since those predicates form part of the sandbox's distinction between actor I/O and general stream I/O.

F.3.1 Chained spawning

The lifecycle ratio must not be read as a raw spawning ratio. The chained spawning program used here is adapted from an Erlang program by Cesarini and Thompson (Cesarini and Thompson, 2009). In it, every actor creates the next actor in a chain, isolating a finer-grained creation pattern. With links made explicitly equivalent to Erlang's ordinary `spawn`, its Web Prolog form is:

```
start(Num) :-
    self(Self),
    start_proc(Num, Self).

start_proc(0, Pid) :- !,
    Pid ! ok.
start_proc(Num, Pid) :-
    Num1 is Num-1,
    spawn(start_proc(Num1, Pid), NPid, [
        link(false)
    ]),
    NPid ! ok,
    receive({ok -> true}).
```

Each child receives its `ok` message from its parent and the last actor sends an `ok` to the original caller. Unlike the lifecycle test, the program installs no monitor and does not serialise creation with a complete monitored shutdown. Each parent exits after consuming its own incoming `ok`, without waiting for its descendants to finish. The peak number of live actors therefore depends on scheduling; the chain measures propagation throughput, not the capacity to keep that many actors alive simultaneously.

The current stand-alone actor core runs the goal in the spawning module's context, without the private-module isolation added by the full node. Table F.3 compares this layer-0 core with Erlang.

The layer-0 spawning gap is therefore about 51 times. Running the same chain through the full native node still prepares a new private Prolog context at every generation, but imports the prepared actor-I/O implementation rather than compiling it again. It also omits the post-load repair pass when an actor has no source to load: without source, no imported predicate or hook-installed module state can have been shadowed.

Table F.3 Historical median time for the unlinked actor chain on the M3 iMac, in milliseconds.

Actors	Native actor core	Erlang	Ratio
1 000	33.419	0.670	49.9
10 000	316.579	6.237	50.8
100 000	3 137.031	61.163	51.3

The demonstrator's identifier allocation and non-reuse policy are described in Appendix E; the historical configuration limitations in Section F.2 also apply to the full-node chain result below.

The full native node completes the 1,000-actor chain in 49.929 ms, against 0.670 ms for Erlang – a ratio of about 74.5. Temporary module creation and destruction by themselves cost about 2.4 microseconds per actor on this machine; complete source-free module preparation raises that to about 8.8 microseconds. Fresh private contexts are therefore no longer the dominant cost in this test.

F.4 Steady-state message passing

Actor startup would dominate a short browser ping-pong test. The benchmark therefore starts its two actors first and reuses them:

```
benchmark_ping_once(Ping, N, Microseconds) :-
    self(Self),
    get_time(T0),
    Ping ! run(N, Self),
    receive({ping_done -> true}),
    get_time(T1),
    Microseconds is round((T1-T0)*1_000_000).

benchmark_ping_server(Pong) :-
    receive({
        run(N, From) ->
            benchmark_ping_rounds(N, Pong),
            From ! ping_done,
            benchmark_ping_server(Pong);
        stop ->
            true
    }).

benchmark_ping_rounds(0, _) :- !.
benchmark_ping_rounds(N, Pong) :-
    self(Self),
    Pong ! ping(Self),
```

```
    receive({pong -> true}),
    N1 is N-1,
    benchmark_ping_rounds(N1, Pong).

benchmark_pong_server :-
    receive({
        ping(Ping) ->
            Ping ! pong,
            benchmark_pong_server;
        stop ->
            true
    }).
```

The omitted setup routine starts and monitors both actors, with their default links disabled. It performs the warm-up and calls `benchmark_ping_once/3` for each observation. Finally, it stops both actors and receives both monitor notifications. The Erlang program uses the same sequence.

Table F.4 Historical steady-state time and throughput for 1,000 round trips on the M3 iMac.

Substrate	Median time	Round trips/s	Relative rate
Browser node	1 110 ms	901	1.0
Native node	11.300 ms	88 496	98.2
Erlang	0.572 ms	1 748 252	1 940.3

Moving actor creation outside the timed interval narrows the gap, but does not remove it. The native node completes about 98 times as many round trips per second as the browser node. Erlang in turn completes about 20 times as many as the native node. The browser path crosses two web worker boundaries and the page-level relay on every round trip; the native and BEAM paths remain inside one runtime.

F.4.1 A browser measurement caveat

Repeated browser batches in one page session did not behave as independent observations. Five consecutive batches of 100 round trips over one actor pair took 33, 49, 66, 84, and 103 ms. Recreating the pair did not remove all of the drift, whereas reloading the page did. The cause has not yet been isolated; possible locations include the web worker relay, retained mailbox or routing state, and demonstrator instrumentation.

The browser result in Table F.4 is therefore the median of three observations made in freshly loaded page sessions, followed by a fourth foreground run that reproduced the result. It should be treated as an indicative measurement of the current demonstrator, not as a stable limit of SWI-WASM or browser actors. The

drift is also an actionable performance finding: before larger browser benchmarks are meaningful, the retained state responsible for it should be identified.

F.5 RPC, paging, and query placement

RPC performance belongs to a separate transport and placement experiment. Consider, for example, a WordNet query distributed over two HTTP nodes:

```
rpc(URI1, type(S, v)),
rpc(URI2, word(S, W))
```

If `type(S,v)` produces many solutions, the second call is made once for each of them. The elapsed time then measures an N+1 remote-query plan: HTTP request construction, network round trips, connection reuse, and repeated remote setup. Options such as the solution-page `limit` and HTTP keep-alive policy can change the result substantially without changing the logical meaning of the query.

Moving a join to the node that holds the relevant data, batching requests, keeping connections alive, and choosing a suitable page size can therefore matter more than the raw speed of either Prolog engine. A reproducible RPC benchmark should distinguish local execution, one warm persistent connection, one fresh connection per call, and a genuinely remote network path; record the dataset and node policies; and report both latency to the first answer and throughput for all answers.

F.6 Interpretation and limits

In the reported historical medians, the browser node is slowest, the native node is substantially faster, and Erlang is fastest by a wide margin. The reasons are architectural. A browser actor is a strong isolation boundary containing a Prolog virtual machine. A native actor is a lighter thread and Prolog context. A BEAM process is the unit for which the Erlang virtual machine has been specialised over several decades.

That ordering should not be generalised beyond the evidence:

- The results come from one machine and one set of runtime versions.
- The browser measurements use the demonstrator in an embedded Chromium-based browser, whose exact browser-engine build was not exposed.
- The benchmark messages are small local terms. It does not measure large-term copying, cross-node communication, HTTP, WebSocket transport, or application computation.
- Lifecycle measurements include source and context preparation. A different browser architecture – for example, several actors in one web worker – would answer a different isolation and performance question.

- No claim is made here about maximum actor population or memory per actor. Those require a capacity benchmark with resource measurements and well-defined failure criteria.

The comparison nevertheless gives useful engineering direction. Browser actors should be used at a granularity that justifies a web worker and an independent Prolog VM, rather than as replacements for millions of tiny BEAM processes. Native actors can be substantially finer-grained, although their private Prolog contexts still make them much heavier than Erlang processes. The public actor semantics remain common across the substrates; what differs is the cost at which each implementation currently provides them.

References

Almeida D (2026) Introducing system one models & Jev. TypeSafe AI Blog, URL https://typesafe.ai/blog/introducing-system-one-models-and-jev, published 15 September 2026

Andersen C, Swift T (2023) The Janus system: A bridge to new Prolog applications. In: Prolog: The Next 50 Years, Springer, Lecture Notes in Computer Science, vol 13900, pp 93–104

Arias J, Carro M, Salazar E, Marple K, Gupta G (2018) Constraint answer set programming without grounding. Theory and Practice of Logic Programming 18(3–4):337–354, DOI 10.1017/S1471068418000285

Armstrong J (2003a) Concurrency oriented programming in Erlang. Talk and accompanying paper, GUUG-Frühjahrsfachgespräch (FFG) 2003, URL https://guug.de/veranstaltungen/ffg2003/papers/ffg2003-armstrong.html, Ruhr University Bochum, Germany, 26–28 March 2003

Armstrong J (2003b) Making reliable distributed systems in the presence of software errors. PhD thesis, Royal Institute of Technology, Stockholm

Armstrong J (2007a) A history of Erlang. In: Proceedings of the Third ACM SIGPLAN History of Programming Languages Conference (HOPL-III), San Diego, California, USA, 9-10 June 2007, pp 1–26, DOI 10.1145/1238844.1238850, URL https://doi.org/10.1145/1238844.1238850

Armstrong J (2007b) Programming Erlang: Software for a Concurrent World. Pragmatic Bookshelf

Barnett J, Akolkar R, Auburn R, Bodell M, Burnett D, Carter J, McGlashan S, Lager T, Helbing M, Hosn R, Raman TV, Reifenrath K, Rosenthal N, Roxendal J (2015) State Chart XML (SCXML) state machine notation for control abstraction. W3C recommendation

Beek W, Wielemaker J (2016) SWISH: An integrated Semantic Web notebook. CEUR Workshop Proceedings 1690, URL https://ceur-ws.org/Vol-1690/paper115.pdf

Berners-Lee T, Hendler J, Lassila O (2001) The Semantic Web. Scientific American 284(5):34–43, URL http://www.sciam.com/article.cfm?articleID=00048144-10D2-1C70-84A9809EC588EF21

Brusk J, Lager T (2008) Developing natural language enabled games in SCXML. Journal of Advanced Computational Intelligence and Intelligent Informatics (JACIII) 12(2):156–163, DOI 10.20965/jaciii.2008.p0156, URL https://doi.org/10.20965/jaciii.2008.p0156

Ceri S, Gottlob G, Tanca L (1989) What you always wanted to know about Datalog (and never dared to ask). IEEE Transactions on Knowledge and Data Engineering 1(1):146–166, DOI 10.1109/69.43410

Cesarini F, Thompson S (2009) ERLANG Programming, 1st edn. O'Reilly Media, Inc.

Cicirelli F, Furfaro A, Giordano A, Nigro L (2009) Statechart-based actors for modelling and distributed simulation of complex multi-agent systems. In: European Conference on Modelling and Simulation, ECMS 2009, Madrid, Spain, June 9-12, 2009, pp 233–239, DOI 10.7148/2009-0233-0239, URL https://doi.org/10.7148/2009-0233-0239

Clark HH, Brennan SE (1991) Grounding in communication. In: Resnick LB, Levine JM, Teasley SD (eds) Perspectives on Socially Shared Cognition, American Psychological Association, Washington, DC, pp 127–149, DOI 10.1037/10096-006

Clark KL (1978) Negation as failure. In: Gallaire H, Minker J (eds) Logic and Data Bases, Springer, pp 293–322

Clark KL, Gregory S (1986) PARLOG: Parallel programming in logic. ACM Transactions on Programming Languages and Systems 8(1):1–49, DOI 10.1145/5001.5390

Clark KL, Robinson PJ, Hagen R (2001) Multi-threading and message communication in Qu-Prolog. Theory and Practice of Logic Programming 1(3):283–301, DOI 10.1017/S147106840100120X

Costantini S, Tocchio A (2004) The DALI logic programming agent-oriented language. In: Lifschitz V, Niemelä I (eds) Logics in Artificial Intelligence, Lecture Notes in Artificial Intelligence, vol 3229, Springer, pp 685–688, DOI 10.1007/978-3-540-30227-8_57

Cropper A, Dumančić S, Evans R, Muggleton SH (2022) Inductive logic programming at 30. Machine Learning 111(1):147–172, DOI 10.1007/s10994-021-06089-1, URL https://doi.org/10.1007/s10994-021-06089-1

De Raedt L, Kimmig A, Toivonen H (2007) ProbLog: A probabilistic Prolog and its application in link discovery. In: Veloso MM (ed) Proceedings of the 20th International Joint Conference on Artificial Intelligence (IJCAI), pp 2462–2467

Frühwirth T (2009) Constraint Handling Rules. Cambridge University Press

d'Avila Garcez A, Lamb LC (2023) Neurosymbolic AI: The third wave. Artificial Intelligence Review 56(11):12387–12406, DOI 10.1007/s10462-023-10448-w, URL https://doi.org/10.1007/s10462-023-10448-w

Garvey C (2018) Broken promises and empty threats: The evolution of AI in the USA, 1956–1996. Technology's Stories 6(1), DOI 10.15763/jou.ts.2018.03.16.02, URL https://www.technologystories.org/ai-evolution/

Geifman Y, El-Yaniv R (2017) Selective classification for deep neural networks. In: Advances in Neural Information Processing Systems, Curran Associates, Inc., vol 30, URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/4a8423d5e91fda00bb7e46540e2b0cf1-Abstract.html

Gelernter D (1985) Generative communication in Linda. ACM Transactions on Programming Languages and Systems 7(1):80–112, DOI 10.1145/2363.2433

Gelfond M, Lifschitz V (1988) The stable model semantics for logic programming. In: Kowalski RA, Bowen KA (eds) Proceedings of the Fifth International Conference and Symposium on Logic Programming, MIT Press, pp 1070–1080

Green TJ, Huang SS, Loo BT, Zhou W (2013) Datalog and recursive query processing. Foundations and Trends in Databases 5(2):105–195, DOI 10.1561/1900000017

Guo C, Pleiss G, Sun Y, Weinberger KQ (2017) On calibration of modern neural networks. In: Proceedings of the 34th International Conference on Machine Learning, PMLR, Proceedings of Machine Learning Research, vol 70, pp 1321–1330, URL https://proceedings.mlr.press/v70/guo17a.html

Gupta G, Salazar E, Shakerin F, Arias J, Varanasi SC, Basu K, Wang H, Li F, Erbatur S, Padalkar P, Rajasekharan A, Zeng Y, Carro M (2023) Prolog: Past, present, and future. In: Warren DS, Dahl V, Eiter T, Hermenegildo MV, Kowalski R, Rossi F (eds) Prolog: The Next 50 Years, Lecture Notes in Computer Science, vol 13900, Springer, Cham, pp 48–61, DOI 10.1007/978-3-031-35254-6_4

Harel D (1987) Statecharts: A visual formalism for complex systems. Sci Comput Program 8(3):231–274, DOI 10.1016/0167-6423(87)90035-9, URL https://doi.org/10.1016/0167-6423(87)90035-9

Harel D, Naamad A (1996) The STATEMATE semantics of statecharts. ACM Trans Softw Eng Methodol 5:293–333, DOI 10.1145/235321.235322

Hébert F (2013) Learn You Some Erlang for Great Good!: A Beginner's Guide. No Starch Press, San Francisco, CA, USA

Hendler JA (2001) Agents and the Semantic Web. IEEE Intelligent Systems 16(2):30–37, DOI 10.1109/5254.920597

Hendler JA (2007) Where are all the intelligent agents? IEEE Intelligent Systems 22(3):2–3, DOI 10.1109/MIS.2007.62

Hewitt C, Bishop P, Steiger R (1973) A universal modular ACTOR formalism for artificial intelligence. In: Proceedings of the 3rd International Joint Conference on Artificial Intelligence. Stanford, CA, USA, August 20–23, 1973, pp 235–245, URL https://www.ijcai.org/Proceedings/73/Papers/027B.pdf

Horrocks I (1999) Constructing the user interface with statecharts. Addison-Wesley Longman Publishing Co., Inc.

Horrocks I, Parsia B, Patel-Schneider P, Hendler J (2005) Semantic Web architecture: Stack or two towers? In: Fages F, Soliman S (eds) Principles and Practice of Semantic Web Reasoning: Third International Workshop, PPSWR 2005, Dagstuhl

Castle, Germany, September 11–16, 2005, Proceedings, Springer Berlin Heidelberg, Berlin, Heidelberg, pp 37–41

Jaffar J, Lassez JL (1987) Constraint logic programming. Proceedings of the 14th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL) pp 111–119, DOI 10.1145/41625.41635

Junger D, Lager T, Roxendal J (2012) SCXML for building conversational agents in the Dialog Web Lab. In: Proceedings of the Swedish Language Technology Conference (SLTC) 2012

Kakas AC, Kowalski RA, Toni F (1992) Abductive logic programming. Journal of Logic and Computation 2(6):719–770, DOI 10.1093/logcom/2.6.719

Kifer M, de Bruijn J, Boley H, Fensel D (2005) A realistic architecture for the Semantic Web. In: Adi A, Stoutenburg S, Tabet S (eds) Rules and Rule Markup Languages for the Semantic Web: First International Conference, RuleML 2005, Galway, Ireland, November 10–12, 2005. Proceedings, Springer Berlin Heidelberg, Berlin, Heidelberg, pp 17–29

Körner P, Leuschel M, Barbosa J, Santos Costa V, Dahl V, Hermenegildo MV, Morales JF, Wielemaker J, Diaz D, Abreu S, Ciatto G (2022) Fifty years of Prolog and beyond. Theory and Practice of Logic Programming 22(6):776–858, DOI 10.1017/S1471068422000102

Kowalski R (1974) Predicate logic as programming language. Information Processing 74 pp 569–574

Kowalski R, Datoo A (2022) Logical English meets legal English for swaps and derivatives. Artificial Intelligence and Law 30(2):163–197, DOI 10.1007/s10506-021-09295-3, URL https://doi.org/10.1007/s10506-021-09295-3

Kowalski R, Dávila J, Sartor G, Calejo M (2023a) Logical English for law and education. In: Warren DS, Dahl V, Eiter T, Hermenegildo M, Kowalski R, Rossi F (eds) Prolog: The Next 50 Years, Lecture Notes in Computer Science, vol 13900, Springer, pp 287–299, DOI 10.1007/978-3-031-35254-6_24

Kowalski R, Sadri F, Calejo M, Dávila J (2023b) Combining logic programming and imperative programming in LPS. In: Warren DS, Dahl V, Eiter T, Hermenegildo M, Kowalski R, Rossi F (eds) Prolog: The Next 50 Years, Lecture Notes in Computer Science, vol 13900, Springer, pp 210–223, DOI 10.1007/978-3-031-35254-6_19

Kowalski RA (1979) Algorithm = logic + control. Commun ACM 22(7):424–436, DOI 10.1145/359131.359136, URL https://doi.org/10.1145/359131.359136

Kowalski RA, Sadri F (2015) Reactive computing as model generation. New Generation Computing 33(1):33–67, DOI 10.1007/s00354-015-0103-z

Lager T (2019) Intro to Web Prolog for Erlangers. In: Proceedings of the 18th ACM SIGPLAN International Workshop on Erlang, ACM, pp 18–29, DOI 10.1145/3331542.3342569

Lager T, Myrendal J (2012) Exploring the web of coined catchy phrases. In: Proceedings of WWW2012: Web Science Track

Lager T, Wielemaker J (2014) Pengines: Web logic programming made easy. Theory and Practice of Logic Programming 14(4-5):539–552,

DOI 10.1017/S1471068414000192, URL http://dx.doi.org/10.1017/S1471068414000192

Lindholm TG, O'Keefe RA (1987) Efficient implementation of a defensible semantics for dynamic Prolog code. In: Lassez JL (ed) Logic Programming: Proceedings of the Fourth International Conference, MIT Press, pp 21–39

Loke SW (2006) Declarative programming of integrated peer-to-peer and web based systems: the case of Prolog. Journal of Systems and Software 79(4):523–536, URL http://dx.doi.org/10.1016/j.jss.2005.04.005

Manhaeve R, Dumančić S, Kimmig A, Demeester T, De Raedt L (2021) Neural probabilistic logic programming in DeepProbLog. Artificial Intelligence 298:103504, DOI 10.1016/j.artint.2021.103504, URL https://www.sciencedirect.com/science/article/pii/S0004370221000552

Marcus G, Davis E (2019) Rebooting AI: Building Artificial Intelligence We Can Trust. Pantheon Books, New York

Merritt D (1989) Building Expert Systems in Prolog. Springer-Verlag

Moura P (n.d.) The Logtalk handbook: Multi-threading programming. Online documentation, section "Threaded engines". Accessed 16 October 2026. https://logtalk3.readthedocs.io/en/latest/userman/threads.html

Moura P, Crocker P, Nunes P (2008) High-Level Multi-threading Programming in Logtalk. In: Hudak P, Harren DS (eds) Proceedings of the Tenth International Symposium on Practical Aspects of Declarative Languages, Springer-Verlag, Berlin Heidelberg, Lecture Notes in Computer Science, vol 4902, p 265–281

Moura P, Rocha R, Madeira SC (2009) High Level Thread-Based Competitive Or-Parallelism in Logtalk. In: Gill A, Swift T (eds) Proceedings of the Eleventh International Symposium on Practical Aspects of Declarative Languages, Springer-Verlag, Berlin Heidelberg, Lecture Notes in Computer Science, vol 5418, p 107–121

Muggleton S, De Raedt L (1994) Inductive logic programming: Theory and methods. The Journal of Logic Programming 19–20(Supplement 1):629–679, DOI 10.1016/0743-1066(94)90035-3

O'Keefe RA (1990) The Craft of Prolog. The MIT Press, Cambridge, MA

Ousterhout JK (1998) Scripting: higher level programming for the 21st century. Computer 31(3):23–30

Radomski S, Schnelle-Walka D, Radeck-Arneth S (2013) A Prolog datamodel for State Chart XML. In: Proceedings of the SIGDIAL 2013 Conference, Association for Computational Linguistics, pp 127–131, URL http://aclweb.org/anthology/W13-4019

Ranta A (2011) Grammatical Framework: Programming with Multilingual Grammars. CSLI Publications, Stanford, iSBN-10: 1-57586-626-9 (Paper), 1-57586-627-7 (Cloth)

Riaza JA (2024) Tau Prolog: A Prolog interpreter for the web. Theory and Practice of Logic Programming 24(1):1–21, DOI 10.1017/S1471068423000352, URL https://doi.org/10.1017/S1471068423000352

Riguzzi F (2018) Foundations of Probabilistic Logic Programming: Languages, Semantics, Inference and Learning. River Publishers, Gistrup, Denmark, DOI 10.1201/9781003338192

Robinson JA (1965) A machine-oriented logic based on the resolution principle. Journal of the ACM 12(1):23–41, DOI 10.1145/321250.321253

Russell SJ, Norvig P (2016) Artificial intelligence: a modern approach, 3rd edn. Pearson Education Limited

Sacks H, Schegloff EA, Jefferson G (1974) A simplest systematics for the organization of turn-taking for conversation. Language 50(4):696–735, DOI 10.2307/412243

Samek M (2002) Practical statecharts in C/C++: Quantum programming for embedded systems. CRC Press

Shapiro EY (1987) Concurrent Prolog: Collected Papers. MIT Press

Skantze G, Moubayed SA (2012) IrisTK: a statechart-based toolkit for multi-party face-to-face interaction. In: International Conference on Multimodal Interaction, ICMI '12, Santa Monica, CA, USA, October 22-26, 2012, pp 69–76, DOI 10.1145/2388676.2388698, URL https://doi.org/10.1145/2388676.2388698

Smith RW, Biermann AW, Hipp DR (1995) An architecture for voice dialog systems based on Prolog-style theorem proving. Computational Linguistics 21(3):281–318, URL http://www.aclweb.org/anthology/J95-3001

Smolka G (1995) The Oz programming model. In: van Leeuwen J (ed) Computer Science Today: Recent Trends and Developments, Lecture Notes in Computer Science, vol 1000, Springer, Berlin, pp 324–343, DOI 10.1007/BFb0015252

van Steen M, Tanenbaum AS (2023) Distributed Systems, 4th edn. distributed-systems.net, free digital edition available from distributed-systems.net

Sterling L, Shapiro EY (1994) The Art of Prolog: Advanced Programming Techniques, 2nd edn. The MIT Press, Cambridge, MA

Svensson H, Fredlund LÅ, Earle CB (2010) A unified semantics for future Erlang. In: Fritchie SL, Sagonas KF (eds) Proceedings of the 9th ACM SIGPLAN Workshop on Erlang, Baltimore, Maryland, USA, September 30, 2010, ACM, pp 23–32, DOI 10.1145/1863509.1863514, URL https://doi.org/10.1145/1863509.1863514

Tarau P (2000) Fluents: A refactoring of Prolog for uniform reflection and interoperation with external objects. In: Lloyd JW, et al. (eds) Computational Logic - CL 2000, First International Conference, London, UK, 24-28 July, 2000, Proceedings, Springer, Lecture Notes in Computer Science, vol 1861, pp 1225–1239, DOI 10.1007/3-540-44957-4_82, URL https://doi.org/10.1007/3-540-44957-4_82

TypeSafe AI (2026a) OpenAPI specification. URL https://api.typesafe.ai/openapi.json, accessed 25 September 2026

TypeSafe AI (2026b) Primitives (questions). URL https://docs.typesafe.ai/primitives.md, accessed 23 September 2026

Ueda K (1986) Guarded Horn Clauses. In: Proceedings of the 1986 International Conference on Logic Programming, Springer, pp 168–179

Van Gelder A, Ross KA, Schlipf JS (1991) The well-founded semantics for general logic programs. Journal of the ACM 38(3):620–650, DOI 10.1145/116825.116838

Van Roy P (2009) Programming paradigms for dummies: What every programmer should know. In: Assayag G, Gerzso A (eds) New Computational Paradigms for Computer Music, IRCAM/Delatour France, pp 9–47, URL https://webperso.info.ucl.ac.be/~pvr/paradigms.html

Van Roy P, Haridi S (2004) Concepts, Techniques, and Models of Computer Programming. The MIT Press, Cambridge, Massachusetts

Van Roy P, Haridi S, Brand P, Smolka G, Mehl M, Scheidhauer R (1997) Mobile objects in Distributed Oz. ACM Transactions on Programming Languages and Systems 19(5):804–851, DOI 10.1145/265943.265972

Van Roy P, Brand P, Duchier D, Haridi S, Henz M, Schulte C (2003) Logic programming in the context of multiparadigm programming: The Oz experience. Theory and Practice of Logic Programming 3(6):715–763, DOI 10.1017/S1471068403001741

Verborgh R, Vander Sande M (2020) The Semantic Web identity crisis: in search of the trivialities that never were. Semantic Web Journal 11(1):19–27, DOI 10.3233/SW-190372, URL https://ruben.verborgh.org/articles/the-semantic-web-identity-crisis/

Virding R (n.d.) Erlog: Prolog for an Erlang application. Project repository and documentation. Accessed 16 October 2026. https://github.com/rvirding/erlog

Virding R, Wikström C, Williams M, Armstrong J (1996) Concurrent programming in ERLANG (2nd ed.). Prentice Hall International (UK) Ltd., GBR

Waldo J, Wyant G, Wollrath A, Kendall S (1994) A note on distributed computing. Tech. Rep. SMLI TR-94-29, Sun Microsystems Laboratories, URL https://waldo.scholars.harvard.edu/publications/note-distributed-computing

Wielemaker J (2005) An optimised Semantic Web query language implementation in Prolog. In: Gabbrielli M, Gupta G (eds) Logic Programming: 21st International Conference, ICLP 2005, Springer, Lecture Notes in Computer Science, vol 3668, pp 128–142, URL https://www.swi-prolog.org/download/publications/ICLP05-SeRQL.pdf

Wielemaker J (2023) Janus: A Bi-directional Interface Between Prolog and Python. URL https://www.swi-prolog.org/pldoc/package/janus.html, SWI-Prolog documentation

Wielemaker J, Schreiber G, Wielinga B (2003) Prolog-based infrastructure for RDF: Scalability and performance. In: Fensel D, Sycara K, Mylopoulos J (eds) The Semantic Web – ISWC 2003, Springer, Lecture Notes in Computer Science, vol 2870, pp 644–658, DOI 10.1007/978-3-540-39718-2_41, URL https://www.swi-prolog.org/download/publications/iswc-03.pdf

Wielemaker J, Schreiber G, Wielinga B (2005) Using triples for implementation: The Triple20 ontology-manipulation tool. In: The Semantic Web – ISWC 2005, Springer, Lecture Notes in Computer Science, vol 3729, pp

773–785, DOI 10.1007/11574620_55, URL https://www.swi-prolog.org/download/publications/ISWC05-Triple20.pdf

Wielemaker J, Hildebrand M, van Ossenbruggen J (2007) Using Prolog as the fundament for applications on the Semantic Web. In: Heymans S, Polleres A, Ruckhaus E, Pearce D, Gupta G (eds) Proceedings of the 2nd Workshop on Applications of Logic Programming to the Web, Semantic Web and Semantic Web Services, CEUR-WS.org, CEUR Workshop Proceedings, vol 287, pp 84–98, URL https://ceur-ws.org/Vol-287/paper_1.pdf

Wielemaker J, Huang Z, van der Meij L (2008) SWI-Prolog and the web. Theory and Practice of Logic Programming 8(3):363–392, DOI 10.1017/S1471068407003237, URL https://doi.org/10.1017/S1471068407003237

Wielemaker J, Schrijvers T, Triska M, Lager T (2012) SWI-Prolog. TPLP 12(1-2):67–96, DOI 10.1017/S1471068411000494, URL http://dx.doi.org/10.1017/S1471068411000494

Wielemaker J, Beek W, Hildebrand M, van Ossenbruggen J (2016) ClioPatria: A SWI-Prolog infrastructure for the Semantic Web. Semantic Web 7(5):529–541, DOI 10.3233/SW-150191, URL https://doi.org/10.3233/SW-150191

Wooldridge M, Jennings NR (1995) Intelligent agents: Theory and practice. The Knowledge Engineering Review 10(2):115–152, DOI 10.1017/S0269888900008122, URL https://doi.org/10.1017/S0269888900008122

Yang Z, Ishay A, Lee J (2020) NeurASP: Embracing neural networks into answer set programming. In: Bessiere C (ed) Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI-20), International Joint Conferences on Artificial Intelligence Organization, pp 1755–1762, DOI 10.24963/ijcai.2020/243, URL https://doi.org/10.24963/ijcai.2020/243

Index

A

Actor 7, 71, 274, 280, **327**
  API reference **339**
  as agent **73**
  bespoke 72
  comparison with statechart regions 207
  creation 37, 417
  creation performance 445
  distributed ping-pong 119
  distribution implementation 428
  examples 58
  hierarchy 46
  I/O implementation 425
  inherited terminal authority 119
  isolation 432
  lifecycle **74**, 321
  lifecycle performance 443
  message exchange 40
  performance 441
  ping-pong example 62
  primitive implementation **416**
  primitives **37**
  private state 73
  production implementation 424
  registration 42
  remote 118
  remote messaging 118
  server pattern 58
  standardisation 320
  statechart communication 211
  supervision 171
  termination 41, 43, 168, 420
Actor agent *see* Actor
Actor model 28
Actor process *see* Actor
ACTOR profile *see* Profile hierarchy
`actors/1` **342**
Admission control 431
Adoption 293
  community infrastructure **297**
  conditions 299
  risks 308
Agent 4, **71**, **327**
  AI 239
  AI motivation 240
  architectural definition 71
  beliefs 77, 284
  complementary views 283
  neuro-symbolic 252
  node 96
  overview **71**
  primary abstraction **280**
  service-request 260
  Web 305
Agent framework 250
Agentic Web 12, **239**, 304, **327**
  Prolog 240
  Prolog agents **245**
Alternative solvers 94, 194
Answer set programming 303
Answer term **83**, **327**
  implementation 409
API reference 339
Artificial intelligence
  community 304
  implementation assistance 323
Authentication 156
Authorisation 156, 430
  implementation 429
  node policy **430**

B

Backtracking 23
  actor answers 65
  as paging *see* Backtracking as paging
  interaction with effects 32
Backtracking as paging **15**, 84, **138**, **276**, 407, 448
BEAM 438, 441
Behaviour **161**, 287, **328**
  callback 168
  Erlang 290
  meta-interpreter 249
  overview 72
  predicate generics **361**
  server **162**
  statechart 200
  supervisor **169**
  toplevel 66, 80, 174
Benchmarking **441**
  actor lifecycle **443**
  actor substrates 441
  browser timing 447
  chained spawning 445
  interpretation 448
  limitations 448
  message passing **446**
  method **442**
  RPC 448
Browser
  ACTOR node 111
  ISOBASE node 107
  ISOTOPE node 108
  RELATION node 104
  two-hop call 136
Browser node 17, 36, 113, 286, **328**
  abort 434
  actor isolation 433
  architecture 432
  architecture summary 435
  benchmarking 447
  implementation **432**
  main thread 434
  main-thread mode 434
  message encoding 433
Browser-resident node *see* Browser node

C

Caching 141
  remote solutions 141
`cancel/1` **347**
Capability 152, **328**
  reply channel 154
  security **282**
  terminal endpoint 119
Closed-world assumption *see* Open-world assumption
Code shipping 147, 288
  data locality 149
  source injection 148
  source installation 427
Collaboration 297
Committed effect 32, 56, 269, 285, **328**
Committed-choice language 289
Compatibility 277
Concurrency
  message passing 62
Concurrent goals 93
Concurrent logic programming 289, 301
Concurrent Prolog Web *see* Prolog Web, concurrent
Concurrent, sequential, and programmable Prolog Web **328**
`concurrent/3` 194
Confidence threshold 260
Configuration **328**
Configuration (statechart) 203, 207, **213**
Conformance **104**, 278, 279, 323, **329**
  test suite 323
Containment 45, 46, 74, **125**, 155, 321, **329**
Context-dependent meaning 259
Control 243
  first-class dimension **287**
  interaction 287
Conversational agent 265, 306
Cursor *see* Solution cursor

D

Data locality 149, 448
`<datamodel>` **379**
`<defer>` **380**
Definite clause grammar **244**
  remote 249
Delayed sending 55, **62**
`demonitor/1-2` **343**
Deployment
  native node 403
Dialogue
  as proof 267
  data model 269
  governance 269
  parsing 269
  repair 266
  repair via history 268
  turn-taking 266
Dialogue manager 267

Discovery *see* Service discovery
Discovery Hub **132**
Distribution-first 123, **285**, **329**
Documentation 299
DOM 434
  direct access 434
Dynamic database 76

E

Education 298
Embodied agent 306
Engineering agenda 312
Erlang
  community 307
  comparison with Trinity 290
  `gen_statem` 218
  influence on Web Prolog **28**
  links *see* Link
  message passing 37
Error handling **41**
Event deferral 50, **210**, **329**
  SXML 380
Event-driven programming 64
Eventless transition **209**, **329**
Exception 41
  monitor notifications 421
Executable logic 15, **288**
Executable policy 254
Exit signal 124
`exit/1` **345**
`exit/2` 43, **346**
Expert system 243
  agent 245
Explainable AI 12, **247**
Explanation 243

F

Fail-fast call 166
Failure
  logical 41
Federation **294**, 295, 316, **329**
  diversity 296
`<final>` **375**
`findall/3` 92
`findnsols/4` 92
`first_solution/2-3` **94**, **194**
  API reference **362**
  example 197
  failure and errors 196
  racing solvers 195
Flat SXML **214**, **329**
`flush/0` 55, **350**
Forward chaining 209
Furhat 306

G

Generic programming 161
Generics *see* Behaviour
Global goal **143**
GNU Prolog
  ISOBASE prototype 436
`<go>` **376**
Governance of interaction 269, **287**

H

Harel, David 199
`<history>` **376**
History state **203**, 268, **330**
Host isolation **330**, 429, **430**
  WebAssembly 430
Host language 435
Hot code replacement **166**, **330**
HTTP 14
  caching 141, 396
  idempotence 396
  overlapping calls 144
  semi-stateful binding **397**
  stateless binding **395**
  stateless endpoint implementation **409**
  stateless RPC 185
  transport 101
HTTP API
  semi-stateful *see* Stateless, semi-stateful, and stateful APIs
  stateless *see* Stateless, semi-stateful, and stateful APIs

I

I/O protocol
  request/reply 122
Implementation
  independent 323
  interoperability **322**
  Prolog node 405
  role in standardisation 323
`in/1` **358**
Inexpressibility 36, **152**, **330**, 430
Inference 242
`input/2-3` **347**
Instant portability 149, **330**
Interaction
  as inference 281
Interaction portability 275, **277**, **330**

Interaction surface 274, 278, 296, 317, **330**, 406
  profile ceiling 104
Interoperability 11, 277, **293**, **330**
  Agentic Web 252
  agents 252
  Erlang 307
  federating force **295**
  independent implementations 322, 435
  neighbouring communities 300
  prototype evidence 436
  testing 323
Invisibility **153**, **330**
ISO 317
  liaison 318
  solution cursors 322
ISO Prolog
  supported predicates **367**
ISOBASE profile *see* Profile hierarchy
ISOTOPE profile *see* Profile hierarchy

J

Jev **257**
  actor wrapper 258

K

Knowledge representation 242

L

Large language model 12
Late reply 151
Let it crash principle 169
Linda 289
Link **45**, 125, **331**
  implementation 420
`listing/0-1` **350**
Livelock 206
Logic programming 23
  community 301
  languages 301
  layered architectures 234
  Prolog *see* Prolog
Logical update view **139**, 322, **331**, 407, 412

M

Mailbox 7, 49, **331**
  flushing 55
  implementation 422
`make_ref/1` **369**
Message loop 216
Message passing 28, **37**
  distributed 118
  implementation 416
  performance 446
  semantics 321
  send implementation 420
  standardisation 320
  two-step model 124
  uniform local and remote interface 123
Message queue *see* Mailbox
Meta-interpreter **243**
  as behaviour 249
Microbenchmark 442
Missing axiom 267
Model service 255
Monitor **41**, **331**
  implementation 420
  server calls 166
`monitor/2` **342**
Mozart 290
Multi-agent system 71, 266, 305, **331**
Multiparadigm programming 31, 290
Mutual constraint **281**, **331**

N

Negation As Failure 228, 284
Neuro-symbolic AI 12, **252**, **331**
  architectural patterns **254**
  failure containment 256
  integration boundary 254
  model calls Prolog 255
  motivation 253
  Prolog calls model 255
  research directions 264
  semantic observation 257
  statechart control 256
  tight integration 256
  Web Prolog 254
Node 7, 280, **332**
  alternative implementations 435
  as agent **96**
  controller 97
  cross-node messaging 119
  deployment 98
  deployment security 432
  distributed inference 247
  implementation **405**, 407
  implementation layers 406
  implementation responsibilities 405
  native implementation 409
  profiles 103
  settings 99

shared database 97
starting from the operating system **403**
Node controller **97**, **332**, 424
browser 432
routing 124, 428
Node owner 98, 127, **332**
Node-resident actor **127**, 297, **332**
count server 127
publish–subscribe 128
supervision 129
`node_setting/2` **366**
Non-player character 204
Nondeterminism
network interface 276
protocol mapping 183
remote *see* RPC meta-predicate
remote calls 134
Normative reasoning 254

O

Offset paging 139
`<onentry>` **378**
`<onexit>` **378**
Open-world assumption 227, **228**, **332**
Orthogonality *see* Parallel region
OTP 290
`output/1-2` **347**
OWL 222, **227**, **332**
external reasoning service 227
reasoning service 231
Ownership
shared 324
Oz 290

P

`<parallel>` **375**
Parallel execution 94, 189
remote calls 142
Parallel region **204**
comparison with actors 207
NPC example 204
Parallel region (orthogonality) **332**
`parallel/1` **94**
API reference **361**
cleanup 191
comparison with concurrent/3 194
failure and errors 190
goal lists 189
implementation **189**
performance 193
Parsing 244
remote 249
Partial failure 150, 285
Pid 7, 14, **333**
implementation 417
remote 118
standardisation 321
wire representation 390
Ping-pong benchmark 446
PIP **294**, **333**
Planning 242
Platform 316
Playground 17, **298**
Portability 294, **333**
Principal **333**, 406, 430
Priority queue 55
Private database 8, **73**, 77, **333**
dynamic predicates 76
implementation **427**
mutation 76
Probabilistic logic programming 303
Process identifier *see* Pid
Profile **333**
API ceiling 104
as interaction contract **278**
interoperability 279
Profile hierarchy **103**, 322
ACTOR 111, 114
conformance 104
extensions 115
implementation **405**
implementation cost 438
ISOBASE 107, 438
implementation 409
ISOTOPE 108
protocol surfaces **389**
RELATION 104, 438
Profile hierarchy (RELATION, ISOBASE, ISOTOPE, ACTOR) **333**
Programmable Prolog Web *see* Prolog Web, programmable
Prolog
AI facilities 241
browser implementations 15
current ecosystem 11
essence 23
fragmentation 11, 294
history 5
unification of systems 294
Web history **15**
Prolog actor *see* Actor
Prolog agent *see* Agent
Prolog agents v, 14
Prolog community 293, 297, 324
Prolog engine 274, **333**
cooperative 432

Prolog Improvements Forum 318
Prolog node *see* Node
Prolog systems
  node implementation 435
Prolog Trinity ecosystem v, 3, **4**, **334**
  as paradigm **273**
  comparisons 289
  demonstrator 6
  fit with Prolog **291**
  five views 283
  layered architecture 235
  motivation 11
  mutual constraint 281
Prolog Web v, 4, 14, **117**, **334**
  AI agents 245
  concurrent **118**
  effects 285
  programmable **147**
    design space 149
  relationship to Semantic Web 221, **228**
  Semantic Web cooperation 229
  sequential **134**
  standardisation 316
Promise and yield **144**
  overlapping waits 145
  server behaviour 164
  timeouts 146
`promise/3-4` **364**
Proof
  Semantic Web 232
Proof tree
  distributed 247
Protocol
  binary representation 391
  compression 391
  lifecycle and ownership 402
  statelessness 407
  versioning 390
Protocol abstraction 61, 164
Protocol reference 389
Provenance 232
PTCP **81**, 321, **334**
  abort 87
  calling a goal 83
  halt 88
  HTTP binding 407
  implementation 175
  offset and limit 84
  once option 86
  protocol reference **394**
  relationship to SXML 218
  template option 84
PTCP over HTTP **334**
Public surface **334**
Publish–subscribe 128
  supervision 129
Pure Web Prolog and the pure Prolog Web **283**, **334**
Purity 283
  distributed 283

Q

Qu-Prolog 289
Query-the-user 243
Quota 431

R

Race condition 191
`raise/1` **357**
RDF 222, **223**, 304, **334**
  bridge example 231
  Prolog rule layer 230
  querying 225
`rdf/3` 223
  rule layer 230
`receive/1-2` **49**, **348**
Recomputation 412
`register/2` **343**
`register_service/2` **344**
Registered name **42**, 131, 153, **335**
RELATION profile *see* Profile hierarchy
Remote module **142**
Remote procedure call *see* RPC meta-predicate
Reply channel 89, 151, **154**, **335**
Research agenda **312**
Resource governance 15, 156, **335**, 406, **431**
  cursor cleanup 413
`respond/2` **348**
Risk analysis **308**
RPC meta-predicate 90, 134, **134**, **335**
  actor-based implementation 136, 184
  API reference **362**
  global goals 143
  HTTP implementation 136, 185
  implementation **183**
  implementation comparison 188
  parallel calls 142
  performance 448
  programming patterns 141
  protocol mapping 183
  remote modules 142
  semantics **135**
  two-hop call 136
`rpc/2-3` **362**

rpc/3
limit option 134, 138
Run-to-completion (macrostep) **213**, **335**
SXML reference 383
runtime_property/1 **366**

S

Sandboxing 15, 36, 152, **335**
blacklist 429
implementation **429**
language level 430
whitelist 429
Scalability
Agentic Web 252
agents 252
Scoped inference 135, 281, **283**, **335**
epistemic reading 284
Scripting language 34
SCXML **218**, 319, **335**
actor-oriented successor 219
differences from SXML 385
Search 242
Security
capability-oriented 282
deployment 156
layered deployment 432
Prolog Web **152**
Selective receive **49**, **336**
deferred messages 50
guards 51
implementation **422**
matching 49
nondeterminism 56
priority 55
timeouts 54
self/1 37, **339**
Semantic observation 257
Semantic sensor 258
Semantic Web **221**, **336**
community 304
cooperation patterns 229
layer cake **222**
logic programming 234
relationship to Prolog Web 228
Trinity comparison 235
user interfaces 232
send operator 40
API reference **346**
send/2-3 62, **346**
Sequential and concurrent partitions 25, **274**, **336**
abstraction levels 274
Sequential Prolog Web *see* Prolog Web, sequential
Server **58**, 95, **162**, **336**
API reference **355**
client interface 61, 164
hot code replacement 166
monitoring 166
reusable behaviour 168
shutdown 168
state 59
Server push 409
server_halt/2 **356**
server_promise/3-4 **355**
server_request/3-4 **355**
server_spawn/3-4 **355**
server_upgrade/2-3 **356**
server_yield/2-4 **355**
Service **336**
specialised 303
Service discovery **131**, **336**
Discovery Hub 132
Service publication **131**, **336**
Service registry *see* Service publication
Session **336**
I/O 87, 425
semi-stateful HTTP 407
stateful WebSocket 409
Shadowing **77**, **336**, 427
Shared database 8, 77, **97**, **337**
mutation 76
Shared object 297
Shell 80
actor programming tools 39
toplevel interaction 83
Signal **47**, 124
Simulation 245
sleep/1 55, **369**
Social robot 114, **306**
Solution cursor **139**, **337**, **407**
cleanup 180
consistency 139
implementation 412
ownership and cleanup 413
standardisation 322
Solution slice 15, 84, 134
implementation 179, 409
large result sets 138
SPARQL 222, **225**, 304, **337**
Web Prolog client 229
<spawn> **380**
spawn/1-3 **340**
spawn/3 37
implementation 417
node option 123

semantics **321**
Specification
initial targets **320**
src_* options 148
Stable model semantics 303
Standard library **369**
Standardisation 311
AI assistance 323
incremental strategy 317
incubation **317**
initial targets 320
layering 322
PIPs 294
scope and timing 317
<state> **374**
State machine *see* Statechart actor
State-transition model 242
Statechart **199**
broadcast communication 204
broadcast semantics 207
data model **207**
deferred events 210
guards 208, 269
hierarchy **203**
internal event cascade 206
previous work 215
process invocation **211**
time 210
Statechart actor 64, **200**, 287, 306, **337**
API reference **357**
comparison 215
gen_statem 218
comparison with message loop 216
completion 375
dialogue management 267
example 200
execution 202
governing model interaction 256
semantic observations 259
service-request agent 213, 260
XML notation 371
<statechart> **374**
statechart_halt/2-3 **357**
statechart_spawn/2 202, **357**
Stateless, semi-stateful, and stateful APIs 82, **101**, **337**
hidden state 141
HTTP 101
semi-stateful 407
solution retrieval 139
stateful 409
statelessness 407
WebSocket 101
Supervision tree 173
Supervisor 95, **169**, **337**
API reference **358**
example 170
implementation 171
restart intensity 173
restart policy 170
restart strategy 173
service publication 129
supervisor_count_children/2 **360**
supervisor_delete_child/3 **360**
supervisor_halt/1 **361**
supervisor_respawn_child/3 **360**
supervisor_spawn/2-3 **358**
supervisor_spawn_child/3 **360**
supervisor_terminate_child/3 **360**
supervisor_which_children/2 **360**
SWI-Prolog
native node implementation **409**
native threads 441
production platform 316
SXML 199, **337**
actor invocation 380
compilation 215
data model 207
example 200
execution semantics **213**
flat subset 214
invoke 211
limits 218
notation reference **371**
Prolog data model 379
relationship to SCXML 218
standardisation 219, 319
tracing 384
Symbolic AI 12, **241**
Symbolic paradigm 273

T

Term representation 433
Terminal
browser node 113
distributed I/O 119
output ordering 122
Termination signal 47
Timed transition **210**, **337**
Timeout 54
remote calls 146
Timing and failure boundary **150**, 256, 285, **338**
blocking surfaces 150
design principle 152
semantic observation 258
TLS 391

Tool use
  Prolog 255
Tooling 299
Toplevel actor 65, **66**, **80**, **338**
  abort 87, 181
  answer redirection 89
  answer terms 179
  API reference **350**
  communication protocol 81
  goal calls 83
  halt 88, 181
  I/O 87
  implementation **174**, 428
  lifecycle 180
  programming interface 90
  protocol 80
  remote placement 123
  RPC 184
  solution collection 92
  standardisation 321
  state machine 175
  statefulness 82
  synchronous API 90
`toplevel_abort/1` **354**
`toplevel_call/2-3` **351**
`toplevel_call_synch/3-4` **353**
`toplevel_halt/1-2` **354**
`toplevel_next/1-2` **353**
`toplevel_next_synch/2-3` **353**
`toplevel_spawn/1-2` **350**
`toplevel_stop/1` **354**
Transition
  SXML 376
Transport 101
Trealla Prolog
  ACTOR prototype 436
Trinity demonstrator 6, **17**
  development routes 316
  production platform 316
Triple 223
Trust 232
Tuple space 289

U

Uncertainty
  semantic boundary 260
Unification **15**, 23, **275**, **338**
  RDF querying 225
  statechart guards 208
`unregister/1` **344**
`unregister_service/1` **345**
URI 14
User interface 232
  statecharts 306
User-interface agent **265**
  comparison with MAS agents 266

V

Virtual machine
  Web Prolog 438

W

W3C 317, 319
  incubation 318
WASM and WASI 36, **338**, **430**, 432, 433
Web
  executable logic 288
Web programming 305
Web Prolog v, 4, 14, **23**, **338**
  agent frameworks 250
  browser execution 36, 286
  manual **339**
  multiparadigm character 31
  overview **25**
  OWL orchestration 231
  programs and data 8
  protocol reference **389**
  scripting language **33**
  security 36
  SPARQL client 229
  suitability for the Web 35
  symbolic backend 254
  virtual machine 438
  web programmers 305
web worker **433**, 441
WebAssembly *see* WASM and WASI
Webizing Prolog and prologizing the Web **14**, **338**
  addressing 14
  dual process 291
  infrastructure 14
  prologizing the Web 15
  webizing Prolog 14
WebSocket 14
  stateful binding **398**
  transport 101
WebSocket API *see* Stateless, semi-stateful, and stateful APIs
`whereis/2` **344**
`whereis_service/2` **345**
World Wide Web
  history 5

X

XML
  SXML 215, 371

Y

`yield/2-3` **365**